Generated from the official open-source text

Mastering Bitcoin

Third Edition

Programming the Open Blockchain

Andreas M. Antonopoulos and David A. Harding

Source, attribution and permissions

Generated open-source edition for Bitsacco Jifunze. This is not the publisher's official PDF.

Source: https://github.com/bitcoinbook/bitcoinbook

Revision: 275c4eb8eab8800c6adc39f8def8e8f8fa356a57

Authors: Andreas M. Antonopoulos and David A. Harding

Book: Mastering Bitcoin: Programming the Open Blockchain, Third Edition

Original publication: O'Reilly Media, November/December 2023; print ISBN 978-1-098-15009-9.

Current licence: Creative Commons Attribution-ShareAlike 4.0 International

https://creativecommons.org/licenses/by-sa/4.0/

Grant:
https://github.com/bitcoinbook/bitcoinbook/commit/94c9ba64b454eb79ca77904a48251bd4efab73cf

Changes: typesetting, open fonts, generated links/index/bookmarks and this provenance notice. Source prose, technical content, code and illustrations are not rewritten. Publisher cover, logo and trade dress are not reproduced. The original publisher notices and colophon below describe the original edition.

Separate notices are retained: Bitcoin Design Guide images (CC BY 4.0), Satoshi Nakamoto's whitepaper (MIT), and the whitepaper errata (CC0). Figure source credits remain with their original captions/alternative text.

Fonts: DejaVu / Bitstream Vera; DejaVu Math TeX Gyre. See font notices included below.

Upstream licence notice

Copyright © 2014 by Andreas M Antonopoulos LLC
Copyright © 2017 by Andreas M Antonopoulos LLC
Copyright © 2018 by aantonop Books LLC
Copyright © 2023 David Harding

Different editions of this book are covered by different licenses. Contributions to this book are made under public domain (CC0) or attribution licenses (CC-BY) - see CONTRIBUTING.md

"Mastering Bitcoin: Unlocking Digital Currencies" (1st Edition) by Andreas M. Antonopoulos, which is tagged "Edition1Print2" is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit http://creativecommons.org/licenses/by-sa/4.0/.

"Mastering Bitcoin: Programming the Open Blockchain" (2nd Edition) by Andreas M. Antonopoulos, which is tagged "second_edition_print3" is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit http://creativecommons.org/licenses/by-sa/4.0/.

"Mastering Bitcoin: Programming the Open Blockchain" (3rd Edition) by Andreas M. Antonopoulos & David Harding, which is tagged "third_edition_print1" is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit http://creativecommons.org/licenses/by-sa/4.0/ .

These "Free Culture" licenses were approved by the publisher O'Reilly Media (http://oreilly.com). O'Reilly Media is not just the world's best publisher of technical books but also a strong supporter of an open culture and the sharing of knowledge.

Thank you O'Reilly!

Praise for *Mastering Bitcoin*, Third Edition

Mastering Bitcoin is very useful for anyone who wants or needs to understand the technology of Bitcoin and the concepts of the protocol.

René Pickhardt, Bitcoin Lightning Network developer

Delve into the fascinating world of Bitcoin with *Mastering Bitcoin*, the definitive guide that navigates through the intricacies of this digital currency. Whether you're a developer, an investor, or simply curious about the future of money, this comprehensive book serves as your roadmap, providing essential knowledge and empowering you to participate confidently in the Internet of money era.

Jorge Lesmes, Senior Director at NTT DATA

Nearly a decade after the initial publishing, the third edition of *Mastering Bitcoin* cements the book’s role as the go-to source of technical Bitcoin educational content. No other book is as comprehensive or up-to-date.

Olaoluwa Osuntokun, CTO at Lightning Labs

A comprehensive overview of what goes on under Bitcoin’s hood and how things fit together.

Mark "Murch" Erhardt, Bitcoin engineer, Chaincode Labs

Mastering Bitcoin

by Andreas M. Antonopoulos and David A. Harding

Copyright © 2024 David Harding. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (https://oreilly.com). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Acquisitions Editor: Michelle Smith
- Development Editor: Angela Rufino
- Production Editor: Clare Laylock
- Copyeditor: Kim Cofer
- Proofreader: Heather Walley
- Indexer: nSight, Inc.
- Interior Designer: David Futato
- Cover Designer: Randy Comer
- Illustrator: Kate Dullea

- December 2014: First Edition
- June 2017: Second Edition
- November 2023: Third Edition

Revision History for the Third Edition

- 2023-11-03: First Release

See https://oreilly.com/catalog/errata.csp?isbn=9781098150099 for release details.

The O’Reilly logo is a registered trademark of O’Reilly Media, Inc. *Mastering Bitcoin*, the cover image, and related trade dress are trademarks of O’Reilly Media, Inc.

While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation

responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-15009-9

[LSI]

Dedicated to my mum, Theresa (1946–2017)

She taught me to love books and question authority

Thank you, mum

—Andreas

For Amanda

It wasn't until I met you that I

actually began living in paradise

—Dave

Table of Contents

Preface
- Writing the Bitcoin Book
- Intended Audience
- Why Are There Bugs on the Cover?
- Conventions Used in This Book
- Code Examples
- Using Code Examples
- Changes Since the Previous Edition
- Bitcoin Addresses and Transactions in This Book
- O’Reilly Online Learning
- How to Contact Us
- Contacting the Authors
- Acknowledgments for the First and Second Editions
- Acknowledgments for the Third Edition
- Early Release Draft (GitHub Contributions)

1. Introduction
   - 1.1. History of Bitcoin
   - 1.2. Getting Started
2. How Bitcoin Works
   - 2.1. Bitcoin Overview
   - 2.2. Buying from an Online Store
   - 2.3. Bitcoin Transactions
   - 2.4. Constructing a Transaction
   - 2.5. Bitcoin Mining
   - 2.6. Spending the Transaction
3. Bitcoin Core: The Reference Implementation
   - 3.1. From Bitcoin to Bitcoin Core
   - 3.2. Bitcoin Development Environment
   - 3.3. Compiling Bitcoin Core from the Source Code
   - 3.4. Running a Bitcoin Core Node
   - 3.5. Configuring the Bitcoin Core Node
   - 3.6. Bitcoin Core API
   - 3.7. Alternative Clients, Libraries, and Toolkits

4. Keys and Addresses
    4.1. Public Key Cryptography
    4.2. Output and Input Scripts
    4.3. IP Addresses: The Original Address for Bitcoin (P2PK)
    4.4. Legacy Addresses for P2PKH
    4.5. Base58check Encoding
    4.6. Compressed Public Keys
    4.7. Legacy Pay to Script Hash (P2SH)
    4.8. Bech32 Addresses
    4.9. Advanced Keys and Addresses
5. Wallet Recovery
    5.1. Independent Key Generation
    5.2. A Wallet Technology Stack in Detail
6. Transactions
    6.1. A Serialized Bitcoin Transaction
    6.2. Version
    6.3. Extended Marker and Flag
    6.4. Inputs
    6.5. Outputs
    6.6. Witness Structure
    6.7. Lock Time
    6.8. Coinbase Transactions
    6.9. Weight and Vbytes
    6.10. Legacy Serialization
7. Authorization and Authentication
    7.1. Transaction Scripts and Script Language
    7.2. Scripted Multisignatures
    7.3. Pay to Script Hash
    7.4. Data Recording Output (OP_RETURN)
    7.5. Scripts with Flow Control (Conditional Clauses)
    7.6. Complex Script Example
    7.7. Merklized Alternative Script Trees (MAST)
    7.8. Pay to Contract (P2C)
    7.9. Scriptless Multisignatures and Threshold Signatures

7.10. Taproot
7.11. Tapscript
8. Digital Signatures
8.1. How Digital Signatures Work
8.2. Schnorr Signatures
8.3. ECDSA Signatures
8.4. The Importance of Randomness in Signatures
8.5. Segregated Witness’s New Signing Algorithm
9. Transaction Fees
9.1. Who Pays the Transaction Fee?
9.2. Fees and Fee Rates
9.3. Estimating Appropriate Fee Rates
9.4. Replace By Fee (RBF) Fee Bumping
9.5. Child Pays for Parent (CPFP) Fee Bumping
9.6. Package Relay
9.7. Transaction Pinning
9.8. CPFP Carve Out and Anchor Outputs
9.9. Adding Fees to Transactions
9.10. Timelock Defense Against Fee Sniping
10. The Bitcoin Network
10.1. Node Types and Roles
10.2. The Network
10.3. Compact Block Relay
10.4. Private Block Relay Networks
10.5. Network Discovery
10.6. Full Nodes
10.7. Exchanging "Inventory"
10.8. Lightweight Clients
10.9. Bloom Filters
10.10. Compact Block Filters
10.11. Lightweight Clients and Privacy
10.12. Encrypted and Authenticated Connections
10.13. Mempools and Orphan Pools
11. The Blockchain

11.1. Structure of a Block
11.2. Block Header
11.3. Block Identifiers: Block Header Hash and Block Height
11.4. The Genesis Block
11.5. Linking Blocks in the Blockchain
11.6. Merkle Trees
11.7. Merkle Trees and Lightweight Clients
11.8. Bitcoin’s Test Blockchains
11.9. Using Test Blockchains for Development
12. Mining and Consensus
12.1. Bitcoin Economics and Currency Creation
12.2. Decentralized Consensus
12.3. Independent Verification of Transactions
12.4. Mining Nodes
12.5. Constructing the Block Header
12.6. Mining the Block
12.7. Median Time Past (MTP)
12.8. Successfully Mining the Block
12.9. Validating a New Block
12.10. Assembling and Selecting Chains of Blocks
12.11. Mining and the Hash Lottery
12.12. Hashrate Attacks
12.13. Changing the Consensus Rules
13. Bitcoin Security
13.1. Security Principles
13.2. User Security Best Practices
14. Second-Layer Applications
14.1. Building Blocks (Primitives)
14.2. Applications from Building Blocks
14.3. Colored Coins
14.4. Payment Channels and State Channels
14.5. Routed Payment Channels (Lightning Network)
Appendix A: The Bitcoin Whitepaper by Satoshi Nakamoto
A.1. Bitcoin - A Peer-to-Peer Electronic Cash System

A.2. License
Appendix B: Errata to the Bitcoin Whitepaper
B.1. Abstract
B.2. Transactions
B.3. Proof of Work
B.4. Reclaiming Disk Space
B.5. Simplified Payment Verification
B.6. Privacy
B.7. Calculations
Appendix C: Bitcoin Improvement Proposals
Index

By Andreas M. Antonopoulos and David A. Harding.

Preface

Writing the Bitcoin Book

I (Andreas) first stumbled upon Bitcoin in mid-2011. My immediate reaction was more or less "Pfft! Nerd money!" and I ignored it for another six months, failing to grasp its importance. This is a reaction that I have seen repeated among many of the smartest people I know, which gives me some consolation. The second time I came across Bitcoin, in a mailing list discussion, I decided to read the whitepaper written by Satoshi Nakamoto and see what it was all about. I still remember the moment I finished reading those nine pages, when I realized that Bitcoin was not simply a digital currency, but a network of trust that could also provide the basis for so much more than just currencies. The realization that "this isn't money, it's a decentralized trust network," started me on a four-month journey to devour every scrap of information about Bitcoin I could find. I became obsessed and enthralled, spending 12 or more hours each day glued to a screen, reading, writing, coding, and learning as much as I could. I emerged from this state of fugue, more than 20 pounds lighter from lack of consistent meals, determined to dedicate myself to working on Bitcoin.

Two years later, after creating a number of small startups to explore various Bitcoin-related services and products, I decided that it was time to write my first book. Bitcoin was the topic that had driven me into a frenzy of creativity and consumed my thoughts; it was the most exciting technology I had encountered since the internet. It was now time to share my passion about this amazing technology with a broader audience.

Intended Audience

This book is mostly intended for coders. If you can use a programming language, this book will teach you how cryptographic currencies work, how to use them, and how to develop software that works with them. The first few chapters are also suitable as an in-depth introduction to Bitcoin for noncoders—those trying to understand the inner workings of Bitcoin and cryptocurrencies.

Why Are There Bugs on the Cover?

The leafcutter ant is a species that exhibits highly complex behavior in a colony super-organism, but each individual ant operates on a set of simple rules driven by social interaction and the exchange of chemical scents (pheromones). Per Wikipedia: "Next to humans, leafcutter ants form the largest and most complex animal societies on Earth." Leafcutter ants don't actually eat leaves, but rather use them to farm a fungus, which is the central food source for the colony. Get that? These ants are farming!

Although ants form a caste-based society and have a queen for producing offspring, there is no central authority or leader in an ant colony. The highly intelligent and sophisticated behavior exhibited by a multimillion-member colony is an emergent property from the interaction of the individuals in a social network.

Nature demonstrates that decentralized systems can be resilient and can produce emergent complexity and incredible sophistication without the need for a central authority, hierarchy, or complex parts.

Bitcoin is a highly sophisticated decentralized trust network that can support myriad financial processes. Yet, each node in the Bitcoin network follows a few simple rules. The interaction between many nodes is what leads to the emergence of the sophisticated behavior, not any inherent complexity or trust in any single node. Like an ant colony, the Bitcoin network is a resilient network of simple nodes following simple rules that together can do amazing things without any central coordination.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
: Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width
: Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

`Constant width bold`
: Shows commands or other text that should be typed literally by the user.

Constant width italic
: Shows text that should be replaced with user-supplied values or by values determined by context.

> **Tip** This element signifies a tip or suggestion.

> **Note** This element signifies a general note.

> **Warning** This element indicates a warning or caution.

Code Examples

All the code snippets can be replicated on most operating systems with a minimal installation of compilers and interpreters for the corresponding languages. Where necessary, we provide basic installation instructions and step-by-step examples of the output of those instructions.

Some of the code snippets and code output have been reformatted for print. In all such cases, the lines have been split by a backslash (\) character, followed by a newline character. When transcribing the examples, remove those two characters and join the lines again and you should see identical results as shown in the example.

All the code snippets use real values and calculations where possible, so that you can build from example to example and see the same results in any code you write to calculate the same values.

Using Code Examples

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Mastering Bitcoin*, 3rd ed., by Andreas M. Antonopoulos and David A. Harding (O'Reilly). Copyright 2024 David Harding, ISBN 978-1-098-15009-9."

Some editions of this book are offered under an open source license, such as CC-BY-NC, in which case the terms of that license apply.

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

Changes Since the Previous Edition

A particular focus in the third edition has been modernizing the 2017 second edition text and the remaining 2014 first edition text. In addition, many concepts that are relevant to contempory Bitcoin development in 2023 have been added:

Keys and Addresses
: We rearranged the address info so that we work through everything in historical order, adding a new section with P2PK (where "address" was "IP address"), refreshed the previous P2PKH and P2SH sections, and then added new sections for segwit/bech32 and taproot/bech32m.

Old Chapters 6 and 7
: Text from previous versions of Chapter 6, "Transactions," and Chapter 7, "Advanced Transactions," has been rearranged and expanded across four new chapters: #c_transactions (the structure of transactions), #c_authorization_authentication, #c_signatures, and #tx_fees.

Transactions
: We added almost entirely new text describing the structure of a transaction.

Authorization and Authentication
: We added new text about MAST, P2C, scriptless multisignatures, taproot, and tapscript.

Digital Signatures
: We revised the ECDSA text and added new text about schnorr signatures, multisignatures, and threshold signatures.

Transaction Fees
: We added almost entirely new text about fees, RBF and CPFP fee bumping, transaction pinning, package relay, and CPFP carve-out.

The Bitcoin Network
: We added text about compact block relay, added a significant update to bloom filters that better describes their privacy problems, and new text about compact block filters.

The Blockchain
: We added text about signet.

Mining and Consensus
: We added text about BIP8 and speedy trial.

Appendixes
: We removed library-specific appendixes. After the appendix containing the original whitepaper, we added a new appendix describing how the implementation and properties of Bitcoin differ from those proposed in the whitepaper.

Bitcoin Addresses and Transactions in This Book

The Bitcoin addresses, transactions, keys, QR codes, and blockchain data used in this book are, for the most part, real. That means you can browse the blockchain, look at the transactions offered as examples, retrieve them with your own scripts or programs, etc.

However, note that the private keys used to construct addresses are either printed in this book or have been "burned." That means if you send money to any of these addresses, the money will either be lost forever, or in some cases everyone who can read the book can take it using the private keys printed in here.

> **Warning** DO NOT SEND MONEY TO ANY OF THE ADDRESSES IN THIS BOOK. Your money will be taken by another reader or lost forever.

O’Reilly Online Learning

> **Note** For more than 40 years, *O’Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O’Reilly’s online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

- O’Reilly Media, Inc.
- 1005 Gravenstein Highway North
- Sebastopol, CA 95472
- 800-889-8969 (in the United States or Canada)
- 707-829-7019 (international or local)
- 707-829-0104 (fax)
- *support@oreilly.com*
- *https://www.oreilly.com/about/contact.html*

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at https://oreil.ly/MasteringBitcoin3e.

For news and information about our books and courses, visit https://oreilly.com.

Find us on LinkedIn: https://linkedin.com/company/oreilly-media.

Follow us on Twitter: https://twitter.com/oreillymedia.

Watch us on YouTube: https://youtube.com/oreillymedia.

Contacting the Authors

You can contact Andreas M. Antonopoulos on his personal site: *https://antonopoulos.com*.

Follow Andreas on Facebook: https://facebook.com/AndreasMAntonopoulos.

Follow Andreas on Twitter: https://twitter.com/aantonop.

Follow Andreas on LinkedIn: https://linkedin.com/company/aantonop.

Many thanks to all of Andreas's patrons who support his work through monthly donations. You can follow his Patreon page here: https://patreon.com/aantonop.

Information about *Mastering Bitcoin*, as well as Andreas's Open Edition and translations, is available on https://bitcoinbook.info.

You can contact David A. Harding on his personal site: https://dtrt.org.

Acknowledgments for the First and Second Editions

By Andreas M. Antonopoulos

This book represents the efforts and contributions of many people. I am grateful for all the help I received from friends, colleagues, and even complete strangers, who joined me in this effort to write the definitive technical book on cryptocurrencies and Bitcoin.

It is impossible to make a distinction between the Bitcoin technology and the Bitcoin community, and this book is as much a product of that community as it is a book on the technology. My work on this book was encouraged, cheered on, supported, and rewarded by the entire Bitcoin community from the very beginning until the very end. More than anything, this book has allowed me to be part of a wonderful community for two years and I can't thank you enough for accepting me into this community. There are far too many people to mention by name—people I've met at conferences, events, seminars, meetups, pizza gatherings, and small private gatherings, as well as many who communicated with me by Twitter, on reddit, on bitcointalk.org, and on GitHub who have had an impact on this book. Every idea, analogy, question, answer, and explanation you find in this book was at some point inspired, tested, or improved through my interactions with the community. Thank you all for your support; without you this book would not have happened. I am forever grateful.

The journey to becoming an author starts long before the first book, of course. My first language (and schooling) was Greek, so I had to take a remedial English writing course in my first year of university. I owe thanks to Diana Kordas, my English writing teacher, who helped me build confidence and skills that year. Later, as a professional, I developed my technical writing skills on the topic of data centers, writing for *Network World* magazine. I

owe thanks to John Dix and John Gallant, who gave me my first writing job as a columnist at *Network World* and to my editor Michael Cooney and my colleague Johna Till Johnson who edited my columns and made them fit for publication. Writing 500 words a week for four years gave me enough experience to eventually consider becoming an author.

Thanks also to those who supported me when I submitted my book proposal to O'Reilly by providing references and reviewing the proposal. Specifically, thanks to John Gallant, Gregory Ness, Richard Stiennon, Joel Snyder, Adam B. Levine, Sandra Gittlen, John Dix, Johna Till Johnson, Roger Ver, and Jon Matonis. Special thanks to Richard Kagan and Tymon Mattoszko, who reviewed early versions of the proposal and Matthew Taylor, who copyedited the proposal.

Thanks to Cricket Liu, author of the O'Reilly title *DNS and BIND*, who introduced me to O'Reilly. Thanks also to Michael Loukides and Allyson MacDonald at O'Reilly, who worked for months to help make this book happen. Allyson was especially patient when deadlines were missed and deliverables delayed as life intervened in our planned schedule. For the second edition, I thank Timothy McGovern for guiding the process, Kim Cofer for patiently editing, and Rebecca Panzer for illustrating many new diagrams.

The first few drafts of the first few chapters were the hardest, because Bitcoin is a difficult subject to unravel. Every time I pulled on one thread of the Bitcoin technology, I had to pull on the whole thing. I repeatedly got stuck and a bit despondent as I struggled to make the topic easy to understand and create a narrative around such a dense technical subject. Eventually, I decided to tell the story of Bitcoin through the stories of the people using Bitcoin and the whole book became a lot easier to write. I owe thanks to my friend and mentor, Richard Kagan, who helped me unravel the story and get past the moments of writer's block. I thank Pamela Morgan, who reviewed early drafts of each chapter in the first and second edition of the book and asked the hard questions to make them better. Also, thanks to the developers of the San Francisco Bitcoin Developers Meetup group as well as Taariq Lewis and Denise Terry for helping test the early material. Thanks also to Andrew Naugler for infographic design.

During the development of the book, I made early drafts available on GitHub and invited public comments. More than a hundred comments, suggestions, corrections, and contributions were submitted in response. Those contributions are explicitly acknowledged, with my thanks, in Early Release Draft (GitHub Contributions). Most of all, my sincere thanks to my volunteer GitHub editors Ming T. Nguyen (1st edition) and Will Binns (2nd edition), who worked tirelessly to curate, manage, and resolve pull requests, issue reports, and perform bug fixes on GitHub.

Once the book was drafted, it went through several rounds of technical review. Thanks to Cricket Liu and Lorne Lantz for their thorough review, comments, and support.

Several Bitcoin developers contributed code samples, reviews, comments, and encouragement. Thanks to Amir Taaki and Eric Voskuil for example code snippets and many great comments; Chris Kleeschulte for contributing information about Bitcore; Vitalik Buterin and Richard Kiss for help with elliptic curve math and code contributions;

Gavin Andresen for corrections, comments, and encouragement; Michalis Kargakis for comments, contributions, and btcd writeup; and Robin Inge for errata submissions improving the second print. In the second edition, I again received a lot of help from many Bitcoin Core developers, including Eric Lombrozo who demystified segregated witness, Luke Dashjr who helped improve the chapter on transactions, Johnson Lau who reviewed segregated witness and other chapters, and many others. I owe thanks to Joseph Poon, Tadge Dryja, and Olaoluwa Osuntokun who explained Lightning Network, reviewed my writing, and answered questions when I got stuck.

I owe my love of words and books to my mother, Theresa, who raised me in a house with books lining every wall. My mother also bought me my first computer in 1982, despite being a self-described technophobe. My father, Menelaos, a civil engineer who just published his first book at 80 years old, was the one who taught me logical and analytical thinking and a love of science and engineering.

Thank you all for supporting me throughout this journey.

Acknowledgments for the Third Edition

By David A. Harding

The introduction to the noninteractive schnorr signature protocol that starts with first describing the interactive schnorr identity protocol in Schnorr Signatures was heavily influenced by the introduction to the subject in "Borrommean Ring Signatures" (2015) by Gregory Maxwell and Andrew Poelstra. I am deeply indebted to each of them for all of their freely provided assistance over the past decade.

Invaluable technical reviews on drafts of this manuscript were provided by Jorge Lesmes, Olaoluwa Osuntokun, René Pickhardt, and Mark "Murch" Erhardt. In particular, Murch's incredibly in-depth and insightful review, and his willingness to evaluate multiple iterations of the same text, have elevated the quality of this book beyond my highest expectations.

I also owe a debt of gratitude to Jimmy Song for suggesting me for this project, to my coauthor Andreas for allowing me to update his bestselling text, to Angela Rufino for guiding me through the O'Reilly authorship process, and to all of the other staff at O'Reilly for making the writing of the third edition a pleasant and productive experience.

Finally, I don't know how I can thank all of the Bitcoin contributors who have helped me on my journey—from creating the software I use, to teaching me how it works, to helping me pass on what little knowledge I've gained. There are too many of you to list your names, but I think of you often and know that my contributions to this book would not have been possible without all that you've done for me.

Early Release Draft (GitHub Contributions)

Many contributors offered comments, corrections, and additions to the early-release draft on GitHub. Thank you all for your contributions to this book.

Following is a list of notable GitHub contributors, including their GitHub ID in parentheses:

- Abdussamad Abdurrazzaq (AbdussamadA)
- Adán SDPC (aesedepece)
- Akira Chiku (achiku)
- Alex Waters (alexwaters)
- Andrew Donald Kennedy (grkvlt)
- Andrey Esaulov (andremaha)
- andronoob
- AnejaBK
- Appaji (CITIZENDOT)
- ariesunny
- Arthur O'Dwyer (Quuxplusone)
- bargitta
- Basem Alasi (Bamskki)
- bisqfan
- bitcoinctf
- blip151
- Bryan Gmyrek (physicsdude)
- Carlos Sims (simsbluebox)
- Casey Flynn (cflynn07)
- cclauss
- Chapman Shoop (belovachap)
- chrisd95
- Christie D'Anna (avocadobreath)
- Cihat Imamoglu (cihati)
- Cody Scott (Siecje)
- coinradar
- Cragin Godley (cgodley)
- Craig Dodd (cdodd)
- dallyshalla
- Dan Nolan (Dan-Nolan)
- Dan Raviv (danra)
- Darius Kramer (dkrmr)
- Darko Janković (trulex)
- David Huie (DavidHuie)
- didongke
- Diego Viola (diegoviola)

- Dimitris Tsapakidis (dimitris-t)
- Dirk Jäckel (biafra23)
- Dmitry Marakasov (AMDmi3)
- drakos (Jolly-Pirate)
- drstrangeM
- Ed Eykholt (edeykholt)
- Ed Leafe (EdLeafe)
- Edward Posnak (edposnak)
- Elias Rodrigues (elias19r)
- Eric Voskuil (evoskuil)
- Eric Winchell (winchell)
- Erik Wahlström (erikwam)
- effectsToCause (vericoin)
- Esteban Ordano (eordano)
- ethers
- Evlix
- fabienhinault
- Fan (whiteath)
- Felix Filozov (ffilozov)
- Francis Ballares (fballares)
- François Wirion (wirion)
- Frank Höger (francyi)
- Gabriel Montes (gabmontes)
- Gaurav Rana (bitcoinsSG)
- genjix
- Geremia
- Gerry Smith (Hermetic)
- gmr81
- Greg (in3rsha)
- Gregory Trubetskoy (grisha)
- Gus (netpoe)
- halseth
- harelw
- Harry Moreno (morenoh149)
- Hennadii Stepanov (hebasto)
- Holger Schinzel (schinzelh)
- Ioannis Cherouvim (cherouvim)
- Ish Ot Jr. (ishotjr)
- ivangreene
- James Addison (jayaddison)
- Jameson Lopp (jlopp)
- Jason Bisterfeldt (jbisterfeldt)
- Javier Rojas (fjrojasgarcia)
- Jordan Baczuk (JBaczuk)

- Jeremy Bokobza (bokobza)
- JerJohn15
- jerzybrzoska
- Jimmy DeSilva (jimmydesilva)
- Jo Wo (jowo-io)
- Joe Bauers (joebauers)
- joflynn
- Johnson Lau (jl2012)
- Jonathan Cross (jonathancross)
- Jorgeminator
- jwbats
- Kai Bakker (kaibakker)
- kollokollo
- krupawan5618
- kynnjo
- Liangzx
- lightningnetworkstores
- lilianrambu
- Liu Yue (lyhistory)
- Lobbelt
- Lucas Betschart (lclc)
- Matt Wesley (MatthewWesley)
- Magomed Aliev (30mb1)
- Mai-Hsuan Chia (mhchia)
- Marco Falke (MarcoFalke)
- María Martín (mmartinbar)
- Marcus Kiisa (mkiisa)
- Mark Erhardt (Xekyo)
- Mark Pors (pors)
- Martin Harrigan (harrigan)
- Martin Vseticka (MartyIX)
- Marzig (marzig76)
- Matt McGivney (mattmcgiv)
- Matthijs Roelink (Matthiti)
- Maximilian Reichel (phramz)
- MG-ng (MG-ng)
- Michalis Kargakis (kargakis)
- Michael C. Ippolito (michaelcippolito)
- Michael Galero (mikong)
- Michael Newman (michaelbnewman)
- Mihail Russu (MihailRussu)
- mikew (mikew)
- milansismanovic

- Minh T. Nguyen (enderminh)
- montvid
- Morfies (morfies)
- Nagaraj Hubli (nagarajhubli)
- Nekomata (nekomata-3)
- nekonenene
- Nhan Vu (jobnomade)
- Nicholas Chen (nickycutesc)
- Ning Shang (syncom)
- Oge Nnadi (ogennadi)
- Oliver Maerz (OliverMaerz)
- Omar Boukli-Hacene (oboukli)
- Óscar Nájera (Titan-C)
- Parzival (Parz-val)
- Paul Desmond Parker (sunwukonga)
- Philipp Gille (philippgille)
- ratijas
- rating89us
- Raul Siles (raulsiles)
- Reproducibility Matters (TheCharlatan)
- Reuben Thomas (rrthomas)
- Robert Furse (Rfurse)
- Roberto Mannai (robermann)
- Richard Kiss (richardkiss)
- rszheng
- Ruben Alexander (hizzvizz)
- Sam Ritchie (sritchie)
- Samir Sadek (netsamir)
- Sandro Conforto (sandroconforto)
- Sanjay Sanathanan (sanjays95)
- Sebastian Falbesoner (theStack)
- Sergei Tikhomirov (s-tikhomirov)
- Sergej Kotliar (ziggamon)
- Seiichi Uchida (topecongiro)
- shaysw
- Simon de la Rouviere (simondlr)
- simone-cominato
- sindhoor7
- Stacie (staciewaleyko)
- Stephan Oeste (Emzy)
- Stéphane Roche (Janaka-Steph)
- takaya-imai
- Thiago Arrais (thiagoarrais)

- Thomas Kerin (afk11)
- Tochi Obudulu (tochicool)
- Tosin (tkuye)
- Vasil Dimov (vasild)
- venzen
- Vlad Stan (motorina0)
- Vijay Chavda (VijayChavda)
- Vincent Déniel (vincentdnl)
- weinim
- wenxiaolong (QingShiLuoGu)
- wenzhenxiang
- Will Binns (wbnns)
- wintercooled
- wjx
- wll2007
- Wojciech Langiewicz (wlk)
- Yancy Ribbens (yancyribbens)
- yjjnls
- Yoshimasa Tanabe (emag)
- yuntai
- yurigeorgiev4
- Zheng Jia (zhengjia)
- Zhou Liang (zhouguoguo)

1. Introduction

Bitcoin is a collection of concepts and technologies that form the basis of a digital money ecosystem. Units of currency called bitcoin are used to store and transmit value among participants in the Bitcoin network. Bitcoin users communicate with each other using the Bitcoin protocol primarily via the internet, although other transport networks can also be used. The Bitcoin protocol stack, available as open source software, can be run on a wide range of computing devices, including laptops and smartphones, making the technology easily accessible.

> **Tip** In this book, the unit of currency is called "bitcoin" with a small *b*, and the system is called "Bitcoin," with a capital *B*.

Users can transfer bitcoin over the network to do just about anything that can be done with conventional currencies, including buying and selling goods, sending money to people or organizations, or extending credit. Bitcoin can be purchased, sold, and exchanged for other currencies at specialized currency exchanges. Bitcoin is arguably the perfect form of money for the internet because it is fast, secure, and borderless.

Unlike traditional currencies, the bitcoin currency is entirely virtual. There are no physical coins or even individual digital coins. The coins are implied in transactions that transfer value from spender to receiver. Users of Bitcoin control keys that allow them to prove ownership of bitcoin in the Bitcoin network. With these keys, they can sign transactions to unlock the value and spend it by transferring it to a new owner. Keys are often stored in a digital wallet on each user’s computer or smartphone. Possession of the key that can sign a transaction is the only prerequisite to spending bitcoin, putting the control entirely in the hands of each user.

Bitcoin is a distributed, peer-to-peer system. As such, there is no central server or point of control. Units of bitcoin are created through a process called "mining," which involves repeatedly performing a computational task that references a list of recent Bitcoin transactions. Any participant in the Bitcoin network may operate as a miner, using their computing devices to help secure transactions. Every 10 minutes, on average, one Bitcoin miner can add security to past transactions and is rewarded with both brand new bitcoins and the fees paid by recent transactions. Essentially, Bitcoin mining decentralizes the currency-issuance and clearing functions of a central bank and replaces the need for any central bank.

The Bitcoin protocol includes built-in algorithms that regulate the mining function across the network. The difficulty of the computational task that miners must perform is adjusted dynamically so that, on average, someone succeeds every 10 minutes regardless of how many miners (and how much processing) are competing at any moment. The protocol also periodically decreases the number of new bitcoins that are created, limiting the total

number of bitcoins that will ever be created to a fixed total just below 21 million coins. The result is that the number of bitcoins in circulation closely follows an easily predictable curve where half of the remaining coins are added to circulation every four years. At approximately block 1,411,200, which is expected to be produced around the year 2035, 99% of all bitcoins that will ever exist will have been issued. Due to Bitcoin's diminishing rate of issuance, over the long term, the Bitcoin currency is deflationary. Furthermore, nobody can force you to accept any bitcoins that were created beyond the expected issuance rate.

Behind the scenes, Bitcoin is also the name of the protocol, a peer-to-peer network, and a distributed computing innovation. Bitcoin builds on decades of research in cryptography and distributed systems and includes at least four key innovations brought together in a unique and powerful combination. Bitcoin consists of:

- A decentralized peer-to-peer network (the Bitcoin protocol)
- A public transaction journal (the blockchain)
- A set of rules for independent transaction validation and currency issuance (consensus rules)
- A mechanism for reaching global decentralized consensus on the valid blockchain (proof-of-work algorithm)

As a developer, I see Bitcoin as akin to the internet of money, a network for propagating value and securing the ownership of digital assets via distributed computation. There's a lot more to Bitcoin than first meets the eye.

In this chapter we'll get started by explaining some of the main concepts and terms, getting the necessary software, and using Bitcoin for simple transactions. In the following chapters, we'll start unwrapping the layers of technology that make Bitcoin possible and examine the inner workings of the Bitcoin network and protocol.

Digital Currencies Before Bitcoin

The emergence of viable digital money is closely linked to developments in cryptography. This is not surprising when one considers the fundamental challenges involved with using bits to represent value that can be exchanged for goods and services. Three basic questions for anyone accepting digital money are:

- Can I trust that the money is authentic and not counterfeit?
- Can I trust that the digital money can only be spent once (known as the "double-spend" problem)?
- Can I be sure that no one else can claim this money belongs to them and not me?

Issuers of paper money are constantly battling the counterfeiting problem by using increasingly sophisticated papers and printing technology. Physical money addresses the double-spend issue easily because the same paper note cannot be in two places at once. Of

course, conventional money is also often stored and transmitted digitally. In these cases, the counterfeiting and double-spend issues are handled by clearing all electronic transactions through central authorities that have a global view of the currency in circulation. For digital money, which cannot take advantage of esoteric inks or holographic strips, cryptography provides the basis for trusting the legitimacy of a user's claim to value. Specifically, cryptographic digital signatures enable a user to sign a digital asset or transaction proving the ownership of that asset. With the appropriate architecture, digital signatures also can be used to address the double-spend issue.

When cryptography started becoming more broadly available and understood in the late 1980s, many researchers began trying to use cryptography to build digital currencies. These early digital currency projects issued digital money, usually backed by a national currency or precious metal such as gold.

Although these earlier digital currencies worked, they were centralized and, as a result, were easy to attack by governments and hackers. Early digital currencies used a central clearinghouse to settle all transactions at regular intervals, just like a traditional banking system. Unfortunately, in most cases these nascent digital currencies were targeted by worried governments and eventually litigated out of existence. Some failed in spectacular crashes when the parent company liquidated abruptly. To be robust against intervention by antagonists, whether legitimate governments or criminal elements, a *decentralized* digital currency was needed to avoid a single point of attack. Bitcoin is such a system, decentralized by design, and free of any central authority or point of control that can be attacked or corrupted.

1.1. History of Bitcoin

Bitcoin was first described in 2008 with the publication of a paper titled "Bitcoin: A Peer-to-Peer Electronic Cash System,"[1] written under the alias of Satoshi Nakamoto (see The Bitcoin Whitepaper by Satoshi Nakamoto). Nakamoto combined several prior inventions such as digital signatures and Hashcash to create a completely decentralized electronic cash system that does not rely on a central authority for currency issuance or settlement and validation of transactions. A key innovation was to use a distributed computation system (called a "proof-of-work" algorithm) to conduct a global lottery every 10 minutes on average, allowing the decentralized network to arrive at *consensus* about the state of transactions. This elegantly solves the issue of double-spend where a single currency unit can be spent twice. Previously, the double-spend problem was a weakness of digital currency and was addressed by clearing all transactions through a central clearinghouse.

The Bitcoin network started in 2009, based on a reference implementation published by Nakamoto and since revised by many other programmers. The number and power of machines running the proof-of-work algorithm (mining) that provides security and resilience for Bitcoin have increased exponentially, and their combined computational power now exceeds the combined number of computing operations of the world's top supercomputers.

Satoshi Nakamoto withdrew from the public in April 2011, leaving the responsibility of developing the code and network to a thriving group of volunteers. The identity of the person or people behind Bitcoin is still unknown. However, neither Satoshi Nakamoto nor anyone else exerts individual control over the Bitcoin system, which operates based on fully transparent mathematical principles, open source code, and consensus among participants. The invention itself is groundbreaking and has already spawned new science in the fields of distributed computing, economics, and econometrics.

A Solution to a Distributed Computing Problem
Satoshi Nakamoto's invention is also a practical and novel solution to a problem in distributed computing, known as the "Byzantine Generals' Problem." Briefly, the problem consists of trying to get multiple participants without a leader to agree on a course of action by exchanging information over an unreliable and potentially compromised network. Satoshi Nakamoto's solution, which uses the concept of proof of work to achieve consensus *without a central trusted authority*, represents a breakthrough in distributed computing.

1.2. Getting Started

Bitcoin is a protocol that can be accessed using an application that speaks the protocol. A "Bitcoin wallet" is the most common user interface to the Bitcoin system, just like a web browser is the most common user interface for the HTTP protocol. There are many implementations and brands of Bitcoin wallets, just like there are many brands of web browsers (e.g., Chrome, Safari, and Firefox). And just like we all have our favorite browsers, Bitcoin wallets vary in quality, performance, security, privacy, and reliability. There is also a reference implementation of the Bitcoin protocol that includes a wallet, known as "Bitcoin Core," which is derived from the original implementation written by Satoshi Nakamoto.

1.2.1. Choosing a Bitcoin Wallet

Bitcoin wallets are one of the most actively developed applications in the Bitcoin ecosystem. There is intense competition, and while a new wallet is probably being developed right now, several wallets from last year are no longer actively maintained. Many wallets focus on specific platforms or specific uses and some are more suitable for beginners while others are filled with features for advanced users. Choosing a wallet is highly subjective and depends on the use and user expertise. Therefore, it would be pointless to recommend a specific brand or wallet. However, we can categorize Bitcoin wallets according to their platform and function and provide some clarity about all the different types of wallets that exist. It is worth trying out several different wallets until you find one that fits your needs.

Types of Bitcoin wallets

Bitcoin wallets can be categorized as follows, according to the platform:

Desktop wallet
: A desktop wallet was the first type of Bitcoin wallet created as a reference implementation. Many users run desktop wallets for the features, autonomy, and control they offer. Running on general-use operating systems such as Windows and macOS has certain security disadvantages, however, as these platforms are often insecure and poorly configured.

Mobile wallet
: A mobile wallet is the most common type of Bitcoin wallet. Running on smart-phone operating systems such as Apple iOS and Android, these wallets are often a great choice for new users. Many are designed for simplicity and ease-of-use, but there are also fully featured mobile wallets for power users. To avoid downloading and storing large amounts of data, most mobile wallets retrieve information from remote servers, reducing your privacy by disclosing to third parties information about your Bitcoin addresses and balances.

Web wallet
: Web wallets are accessed through a web browser and store the user's wallet on a server owned by a third party. This is similar to webmail in that it relies entirely on a third-party server. Some of these services operate using client-side code running in the user's browser, which keeps control of the Bitcoin keys in the hands of the user, although the user's dependence on the server still compromises their privacy. Most, however, take control of the Bitcoin keys from users in exchange for ease-of-use. It is inadvisable to store large amounts of bitcoin on third-party systems.

Hardware signing devices
: Hardware signing devices are devices that can store keys and sign transactions using special-purpose hardware and firmware. They usually connect to a desktop, mobile, or web wallet via USB cable, near-field-communication (NFC), or a camera with QR codes. By handling all Bitcoin-related operations on the specialized hardware, these wallets are less vulnerable to many types of attacks. Hardware signing devices are sometimes called "hardware wallets", but they need to be paired with a full-featured wallet to send and receive transactions, and the security and privacy offered by that paired wallet plays a critical role in how much security and privacy the user obtains when using the hardware signing device.

Full node versus Lightweight

Another way to categorize Bitcoin wallets is by their degree of autonomy and how they interact with the Bitcoin network:

Full node
: A full node is a program that validates the entire history of Bitcoin transactions (every transaction by every user, ever). Optionally, full nodes can also store previously validated transactions and serve data to other Bitcoin programs, either on

the same computer or over the internet. A full node uses substantial computer resources—about the same as watching an hour-long streaming video for each day of Bitcoin transactions—but the full node offers complete autonomy to its users.

Lightweight client
: A lightweight client, also known as a simplified-payment-verification (SPV) client, connects to a full node or other remote server for receiving and sending Bitcoin transaction information, but stores the user wallet locally, partially validates the transactions it receives, and independently creates outgoing transactions.

Third-party API client
: A third-party API client is one that interacts with Bitcoin through a third-party system of APIs rather than by connecting to the Bitcoin network directly. The wallet may be stored by the user or by third-party servers, but the client trusts the remote server to provide it with accurate information and protect its privacy.

> **Tip** Bitcoin is a peer-to-peer (P2P) network. Full nodes are the *peers:* each peer individually validates every confirmed transaction and can provide data to its user with complete authority. Lightweight wallets and other software are *clients:* each client depends on one or more peers to provide it with valid data. Bitcoin clients can perform secondary validation on some of the data they receive and make connections to multiple peers to reduce their dependence on the integrity of a single peer, but the security of a client ultimately relies on the integrity of its peers.

Who controls the keys

A very important additional consideration is *who controls the keys*. As we will see in subsequent chapters, access to bitcoins is controlled by "private keys," which are like very long PINs. If you are the only one to have control over these private keys, you are in control of your bitcoins. Conversely, if you do not have control, then your bitcoins are managed by a third-party who ultimately controls your funds on your behalf. Key management software falls into two important categories based on control: *wallets*, where you control the keys, and the funds and accounts with custodians where some third-party controls the keys. To emphasize this point, I (Andreas) coined the phrase: *Your keys, your coins. Not your keys, not your coins*.

Combining these categorizations, many Bitcoin wallets fall into a few groups, with the three most common being desktop full node (you control the keys), mobile lightweight wallet (you control the keys), and web-based accounts with third parties (you don’t control the keys). The lines between different categories are sometimes blurry, as software runs on multiple platforms and can interact with the network in different ways.

1.2.2. Quick Start

Alice is not a technical user and only recently heard about Bitcoin from her friend Joe. While at a party, Joe is enthusiastically explaining Bitcoin to everyone around him and is offering a demonstration. Intrigued, Alice asks how she can get started with Bitcoin. Joe says that a mobile wallet is best for new users and he recommends a few of his favorite wallets. Alice downloads one of Joe's recommendations and installs it on her phone.

When Alice runs her wallet application for the first time, she chooses the option to create a new Bitcoin wallet. Because the wallet she has chosen is a noncustodial wallet, Alice (and only Alice) will be in control of her keys. Therefore, she bears responsibility for backing them up, since losing the keys means she loses access to her bitcoins. To facilitate this, her wallet produces a *recovery code* that can be used to restore her wallet.

1.2.3. Recovery Codes

Most modern noncustodial Bitcoin wallets will provide a recovery code for their user to back up. The recovery code usually consists of numbers, letters, or words selected randomly by the software, and is used as the basis for the keys that are generated by the wallet. See [recovery_code_sample] for examples.

Sample recovery codes

Wallet	Recovery code
BlueWallet	(1) media (2) suspect (3) effort (4) dish (5) album (6) shaft (7) price (8) junk (9) pizza (10) situate (11) oyster (12) rib
Electrum	nephew dog crane clever quantum crazy purse traffic repeat fruit old clutch
Muun	LAFV TZUN V27E NU4D WPF4 BRJ4 ELLP BNFL

> **Tip** A recovery code is sometimes called a "mnemonic" or "mnemonic phrase," which implies you should memorize the phrase, but writing the phrase down on paper takes less work and tends to be more reliable than most people's memories. Another alternative name is "seed phrase" because it provides the input ("seed") to the function that generates all of a wallet's keys.

If something happens to Alice's wallet, she can download a new copy of her wallet software and enter this recovery code to rebuild the wallet database of all the onchain transactions she's ever sent or received. However, recovering from the recovery code will not by itself restore any additional data Alice entered into her wallet, such as the labels she associated with particular addresses or transactions. Although losing access to that metadata isn't as important as losing access to money, it can still be important in its own way. Imagine you

need to review an old bank or credit card statement and the name of every entity you paid (or who paid you) has been blanked out. To prevent losing metadata, many wallets provide an additional backup feature beyond recovery codes.

For some wallets, that additional backup feature is even more important today than it used to be. Many Bitcoin payments are now made using *offchain* technology, where not every payment is stored in the public blockchain. This reduces user's costs and improves privacy, among other benefits, but it means that a mechanism like recovery codes that depends on onchain data can't guarantee recovery of all of a user's bitcoins. For applications with offchain support, it's important to make frequent backups of the wallet database.

Of note, when receiving funds to a new mobile wallet for the first time, many wallets will often re-verify that you have securely backed-up your recovery code. This can range from a simple prompt to requiring the user to manually re-enter the code.

> **Warning** Although many legitimate wallets will prompt you to re-enter your recovery code, there are also many malware applications that mimic the design of a wallet, insist you enter your recovery code, and then relay any entered code to the malware developer so they can steal your funds. This is the equivalent of phishing websites that try to trick you into giving them your bank passphrase. For most wallet applications, the only times they will ask for your recovery code are during the initial set up (before you have received any bitcoins) and during recovery (after you lost access to your original wallet). If the application asks for your recovery code any other time, consult with an expert to ensure you aren't being phished.

1.2.4. Bitcoin Addresses

Alice is now ready to start using her new Bitcoin wallet. Her wallet application randomly generated a private key (described in more detail in Private Keys) that will be used to derive Bitcoin addresses that direct to her wallet. At this point, her Bitcoin addresses are not known to the Bitcoin network or "registered" with any part of the Bitcoin system. Her Bitcoin addresses are simply numbers that correspond to her private key that she can use to control access to the funds. The addresses are generated independently by her wallet without reference or registration with any service.

> **Tip** There are a variety of Bitcoin addresses and invoice formats. Addresses and invoices can be shared with other Bitcoin users who can use them to send bitcoins directly to your wallet. You can share an address or invoice with other people without worrying about the security of your bitcoins. Unlike a bank account number, nobody who learns one of your Bitcoin addresses can withdraw money from your wallet—you must initiate all spends. However, if you give two people the same address, they will be able to see how many bitcoins the other person sent you. If you post your address publicly, everyone will be able to see how much bitcoin other people sent to that address. To protect your privacy, you should generate a new invoice with a new address each time you request a payment.

1.2.5. Receiving Bitcoin

Alice uses the *Receive* button, which displays a QR code, shown in Alice uses the Receive screen on her mobile Bitcoin wallet and displays her address in a QR code format..

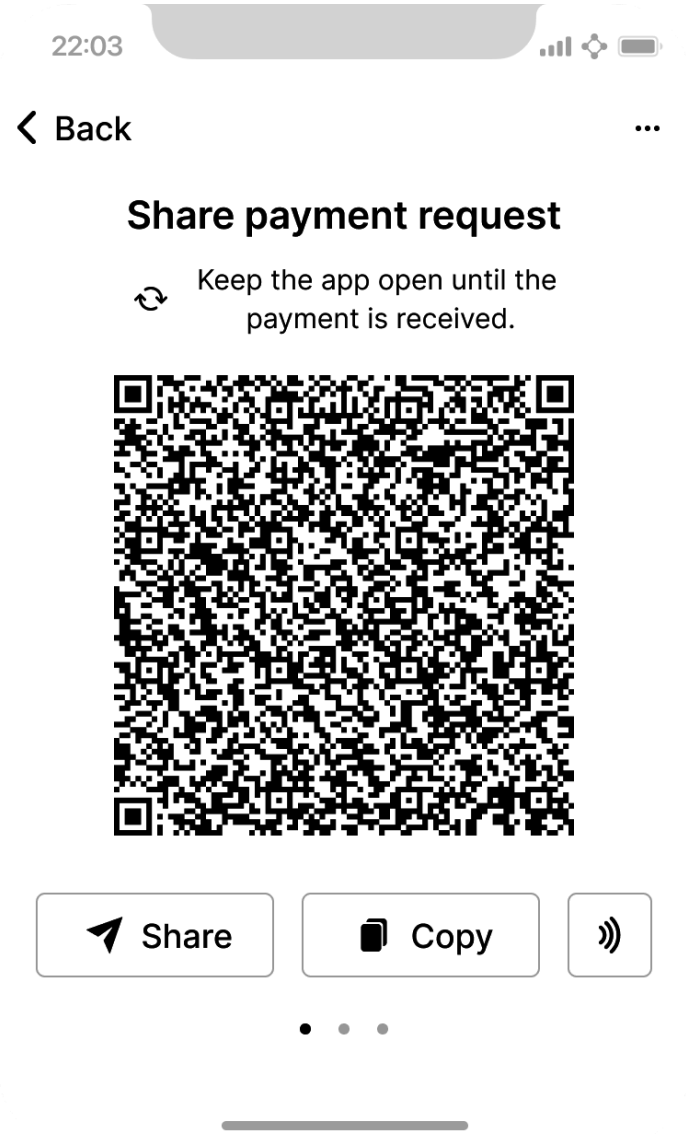


Figure 1. Alice uses the Receive screen on her mobile Bitcoin wallet and displays her address in a QR code format.

The QR code is the square with a pattern of black and white dots, serving as a form of barcode that contains the same information in a format that can be scanned by Joe’s smartphone camera.

> **Warning** Any funds sent to the addresses in this book will be lost. If you want to test sending bitcoins, please consider donating it to a bitcoin-accepting charity.

1.2.6. Getting Your First Bitcoin

The first task for new users is to acquire some bitcoin.

Bitcoin transactions are irreversible. Most electronic payment networks such as credit cards, debit cards, PayPal, and bank account transfers are reversible. For someone selling bitcoin, this difference introduces a very high risk that the buyer will reverse the electronic payment after they have received bitcoin, in effect defrauding the seller. To mitigate this risk, companies accepting traditional electronic payments in return for bitcoin usually require buyers to undergo identity verification and credit-worthiness checks, which may take several days or weeks. As a new user, this means you cannot buy bitcoin instantly with a credit card. With a bit of patience and creative thinking, however, you won't need to.

Here are some methods for acquiring bitcoin as a new user:

- Find a friend who has bitcoins and buy some from him or her directly. Many Bitcoin users start this way. This method is the least complicated. One way to meet people with bitcoins is to attend a local Bitcoin meetup listed at Meetup.com.
- Earn bitcoin by selling a product or service for bitcoin. If you are a programmer, sell your programming skills. If you're a hairdresser, cut hair for bitcoins.
- Use a Bitcoin ATM in your city. A Bitcoin ATM is a machine that accepts cash and sends bitcoins to your smartphone Bitcoin wallet.
- Use a Bitcoin currency exchange linked to your bank account. Many countries now have currency exchanges that offer a market for buyers and sellers to swap bitcoins with local currency. Exchange-rate listing services, such as BitcoinAverage, often show a list of Bitcoin exchanges for each currency.

> **Tip** One of the advantages of Bitcoin over other payment systems is that, when used correctly, it affords users much more privacy. Acquiring, holding, and spending bitcoin does not require you to divulge sensitive and personally identifiable information to third parties. However, where bitcoin touches traditional systems, such as currency exchanges, national and international regulations often apply. In order to exchange bitcoin for your national currency, you will often be required to provide proof of identity and banking information. Users should be aware that once a Bitcoin address is attached to an identity, other associated Bitcoin transactions may also become easy to identify and track—including transactions made earlier. This is one reason many users choose to maintain dedicated exchange accounts independent from their wallets.

Alice was introduced to Bitcoin by a friend, so she has an easy way to acquire her first bitcoins. Next, we will look at how she buys bitcoins from her friend Joe and how Joe sends the bitcoins to her wallet.

1.2.7. Finding the Current Price of Bitcoin

Before Alice can buy bitcoin from Joe, they have to agree on the *exchange rate* between bitcoin and US dollars. This brings up a common question for those new to Bitcoin: "Who sets the price of bitcoins?" The short answer is that the price is set by markets.

Bitcoin, like most other currencies, has a *floating exchange rate*. That means that the value of bitcoin fluctuates according to supply and demand in the various markets where it is traded. For example, the "price" of bitcoin in US dollars is calculated in each market based on the most recent trade of bitcoins and US dollars. As such, the price tends to fluctuate minutely several times per second. A pricing service will aggregate the prices from several markets and calculate a volume-weighted average representing the broad market exchange rate of a currency pair (e.g., BTC/USD).

There are hundreds of applications and websites that can provide the current market rate. Here are some of the most popular:

Bitcoin Average
: A site that provides a simple view of the volume-weighted average for each currency.

CoinCap
: A service listing the market capitalization and exchange rates of hundreds of cryptocurrencies, including bitcoins.

Chicago Mercantile Exchange Bitcoin Reference Rate
: A reference rate that can be used for institutional and contractual reference, provided as part of investment data feeds by the CME.

In addition to these various sites and applications, some Bitcoin wallets will automatically convert amounts between bitcoin and other currencies.

1.2.8. Sending and Receiving Bitcoin

Alice has decided to buy 0.001 bitcoins. After she and Joe check the exchange rate, she gives Joe an appropriate amount of cash, opens her mobile wallet application, and selects Receive. This displays a QR code with Alice’s first Bitcoin address.

Joe then selects Send on his smartphone wallet and opens the QR code scanner. This allows Joe to scan the barcode with his smartphone camera so that he doesn’t have to type in Alice’s Bitcoin address, which is quite long.

Joe now has Alice’s Bitcoin address set as the recipient. Joe enters the amount as 0.001 bitcoins (BTC); see Bitcoin wallet send screen.. Some wallets may show the amount in a different denomination: 0.001 BTC is 1 millibitcoin (mBTC) or 100,000 satoshis (sats).

Some wallets may also suggest Joe enter a label for this transaction; if so, Joe enters "Alice". Weeks or months from now, this will help Joe remember why he sent these 0.001 bitcoins. Some wallets may also prompt Joe about fees. Depending on the wallet and how

the transaction is being sent, the wallet may ask Joe to either enter a transaction fee rate or prompt him with a suggested fee (or fee rate). The higher the transaction fee, the faster the transaction will be confirmed (see Confirmations).

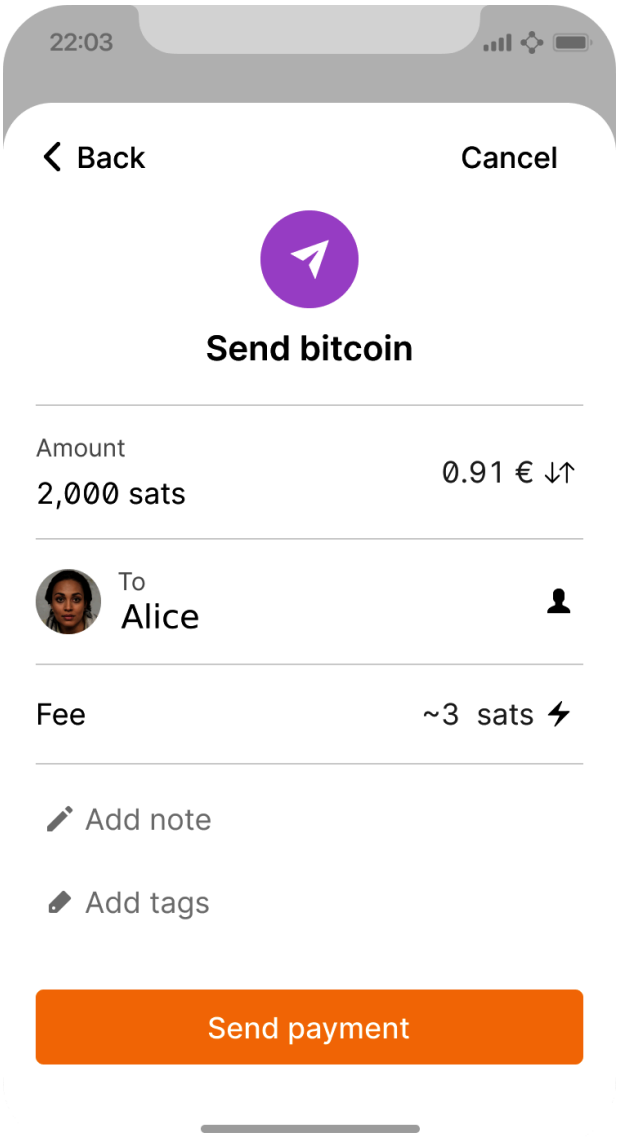


Figure 2. Bitcoin wallet send screen.

Joe then carefully checks to make sure he has entered the correct amount, because he is about to transmit money and mistakes will soon become irreversible. After double-checking the address and amount, he presses Send to transmit the transaction. Joe's mobile Bitcoin wallet constructs a transaction that assigns 0.001 BTC to the address provided by Alice, sourcing the funds from Joe's wallet, and signing the transaction with

Joe’s private keys. This tells the Bitcoin network that Joe has authorized a transfer of value to Alice’s new address. As the transaction is transmitted via the peer-to-peer protocol, it quickly propagates across the Bitcoin network. After just a few seconds, most of the well-connected nodes in the network receive the transaction and see Alice’s address for the first time.

Meanwhile, Alice’s wallet is constantly "listening" for new transactions on the Bitcoin network, looking for any that match the addresses it contains. A few seconds after Joe’s wallet transmits the transaction, Alice’s wallet will indicate that it is receiving 0.001 BTC.

Confirmations

At first, Alice’s address will show the transaction from Joe as "Unconfirmed." This means that the transaction has been propagated to the network but has not yet been recorded in the Bitcoin transaction journal, known as the blockchain. To be confirmed, a transaction must be included in a block and added to the blockchain, which happens every 10 minutes, on average. In traditional financial terms this is known as *clearing*. For more details on propagation, validation, and clearing (confirmation) of bitcoin transactions, see Mining and Consensus.

Alice is now the proud owner of 0.001 BTC that she can spend. Over the next few days, Alice buys more bitcoin using an ATM and an exchange. In the next chapter we will look at her first purchase with Bitcoin, and examine the underlying transaction and propagation technologies in more detail.

2. How Bitcoin Works

The Bitcoin system, unlike traditional banking and payment systems, does not require trust in third parties. Instead of a central trusted authority, in Bitcoin, each user can use software running on their own computer to verify the correct operation of every aspect of the Bitcoin system. In this chapter, we will examine Bitcoin from a high level by tracking a single transaction through the Bitcoin system and watch as it is recorded on the blockchain, the distributed journal of all transactions. Subsequent chapters will delve into the technology behind transactions, the network, and mining.

2.1. Bitcoin Overview

The Bitcoin system consists of users with wallets containing keys, transactions that are propagated across the network, and miners who produce (through competitive computation) the consensus blockchain, which is the authoritative journal of all transactions.

Each example in this chapter is based on an actual transaction made on the Bitcoin network, simulating the interactions between several users by sending funds from one wallet to another. While tracking a transaction through the Bitcoin network to the blockchain, we will use a *blockchain explorer* site to visualize each step. A blockchain explorer is a web application that operates as a Bitcoin search engine, in that it allows you to search for addresses, transactions, and blocks and see the relationships and flows between them.

Popular blockchain explorers include the following:

- Blockstream Explorer
- Mempool.Space
- BlockCypher Explorer

Each of these has a search function that can take a Bitcoin address, transaction hash, block number, or block hash and retrieve corresponding information from the Bitcoin network. With each transaction or block example, we will provide a URL so you can look it up yourself and study it in detail.

Warning Block Explorer Privacy Warning
Searching information on a block explorer may disclose to its operator that you're interested in that information, allowing them to associate it with your IP address, browser details, past searches, or other identifiable information. If you look up the transactions in this book, the operator of the block explorer might guess that you're learning about Bitcoin, which shouldn't be a problem. But if you look up your own transactions, the operator may be able to guess how many bitcoins you've received, spent, and currently own.

2.2. Buying from an Online Store

Alice, introduced in the previous chapter, is a new user who has just acquired her first bitcoins. In Getting Your First Bitcoin, Alice met with her friend Joe to exchange some cash for bitcoins. Since then, Alice has bought additional bitcoins. Now Alice will make her first spending transaction, buying access to a premium podcast episode from Bob's online store.

Bob's web store recently started accepting bitcoin payments by adding a Bitcoin option to its website. The prices at Bob's store are listed in the local currency (US dollars), but at checkout, customers have the option of paying in either dollars or bitcoin.

Alice finds the podcast episode she wants to buy and proceeds to the checkout page. At checkout, Alice is offered the option to pay with bitcoin in addition to the usual options. The checkout cart displays the price in US dollars and also in bitcoin (BTC), at Bitcoin's prevailing exchange rate.

Bob's ecommerce system will automatically create a QR code containing an *invoice* (Invoice QR code.).



Figure 3. Invoice QR code.

Unlike a QR code that simply contains a destination Bitcoin address, this invoice is a QR-encoded URI that contains a destination address, a payment amount, and a description. This allows a Bitcoin wallet application to prefill the information used to send the payment while showing a human-readable description to the user. You can scan the QR code with a bitcoin wallet application to see what Alice would see:

The invoice QR code encodes the following URI, defined in BIP21:

```
bitcoin:bc1qk2g6u8p4qm2s2lh3gts5cpt2mrv5skcuu7u3e4?amount=0.01577764&
label=Bob%27s%20Store&
message=Purchase%20at%20Bob%27s%20Store

Components of the URI

A Bitcoin address: "bc1qk2g6u8p4qm2s2lh3gts5cpt2mrv5skcuu7u3e4"
The payment amount: "0.01577764"
A label for the recipient address: "Bob's Store"
A description for the payment: "Purchase at Bob's Store"
```

> **Tip** Try to scan this with your wallet to see the address and amount but DO NOT SEND MONEY.

Alice uses her smartphone to scan the barcode on display. Her smartphone shows a payment for the correct amount to Bob's Store and she selects Send to authorize the payment. Within a few seconds (about the same amount of time as a credit card authorization), Bob sees the transaction on the register.

> **Note** The Bitcoin network can transact in fractional values, e.g., from millibitcoin (1/1000th of a bitcoin) down to 1/100,000,000th of a bitcoin, which is known as a satoshi. This book uses the same pluralization rules used for dollars and other traditional currencies when talking about amounts greater than one bitcoin and when using decimal notation, such as "10 bitcoins" or "0.001 bitcoins." The same rules also apply to other bitcoin bookkeeping units, such as millibitcoins and satoshis.

You can use a block explorer to examine blockchain data, such as the payment made to Bob in Alice’s transaction.

In the following sections, we will examine this transaction in more detail. We’ll see how Alice’s wallet constructed it, how it was propagated across the network, how it was verified, and finally, how Bob can spend that amount in subsequent transactions.

2.3. Bitcoin Transactions

In simple terms, a transaction tells the network that the owner of certain bitcoins has authorized the transfer of that value to another owner. The new owner can now spend the bitcoin by creating another transaction that authorizes the transfer to another owner, and so on, in a chain of ownership.

2.3.1. Transaction Inputs and Outputs

Transactions are like lines in a double-entry bookkeeping ledger. Each transaction contains one or more *inputs*, which spend funds. On the other side of the transaction, there are one or more *outputs*, which receive funds. The inputs and outputs do not necessarily add up to the same amount. Instead, outputs add up to slightly less than inputs and the difference represents an implied *transaction fee*, which is a small payment collected by the miner who includes the transaction in the blockchain. A Bitcoin transaction is shown as a bookkeeping ledger entry in Transaction as double-entry bookkeeping..

The transaction also contains proof of ownership for each amount of bitcoins (inputs) whose value is being spent, in the form of a digital signature from the owner, which can be independently validated by anyone. In Bitcoin terms, spending is signing a transaction that transfers value from a previous transaction over to a new owner identified by a Bitcoin address.

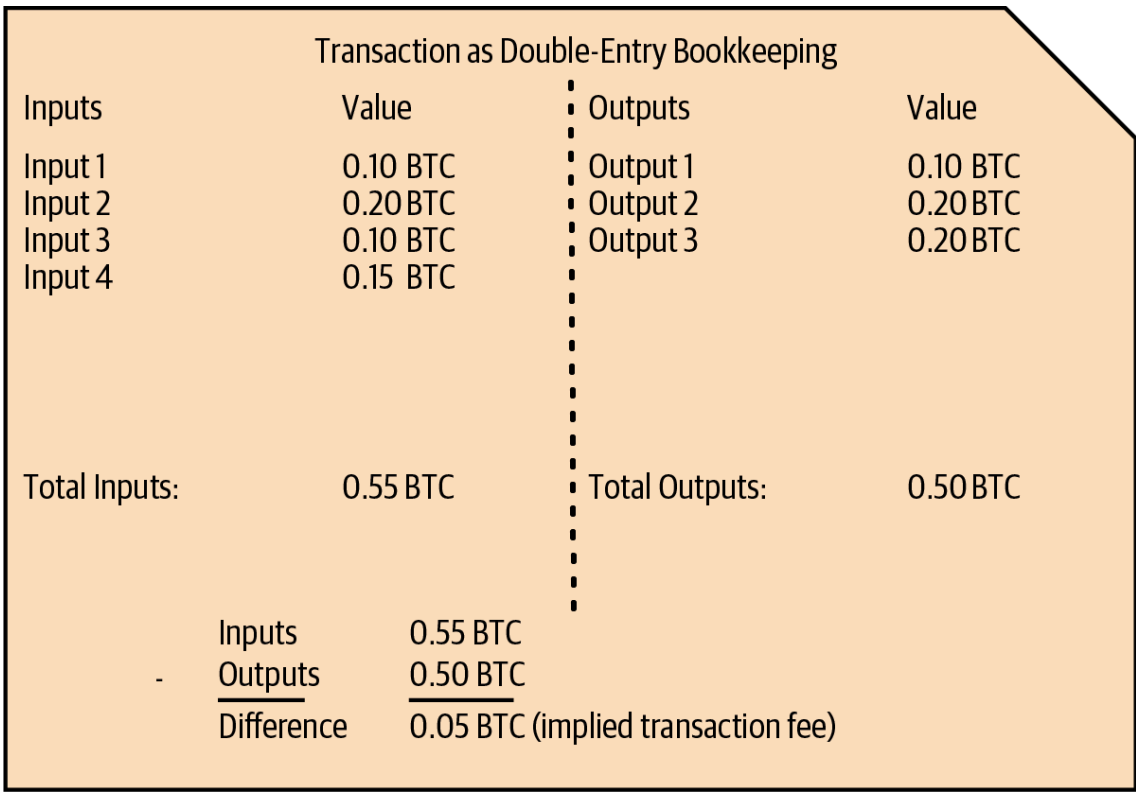


Figure 4. Transaction as double-entry bookkeeping.

2.3.2. Transaction Chains

Alice's payment to Bob's Store uses a previous transaction's output as its input. In the previous chapter, Alice received bitcoins from her friend Joe in return for cash. We've labeled that as *Transaction 1* (Tx1) in A chain of transactions, where the output of one transaction is the input of the next transaction..

Tx1 sent 0.001 bitcoins (100,000 satoshis) to an output locked by Alice's key. Her new transaction to Bob's Store (Tx2) references the previous output as an input. In the illustration, we show that reference using an arrow and by labeling the input as "Tx1:0". In an actual transaction, the reference is the 32-byte transaction identifier (txid) for the transaction where Alice received the money from Joe. The ":0" indicates the position of the output where Alice received the money; in this case, the first position (position 0).

As shown, actual Bitcoin transactions don't explicitly include the value of their input. To determine the value of an input, software needs to use the input's reference to find the previous transaction output being spent.

Alice's Tx2 contains two new outputs, one paying 75,000 satoshis for the podcast and another paying 20,000 satoshis back to Alice to receive change.

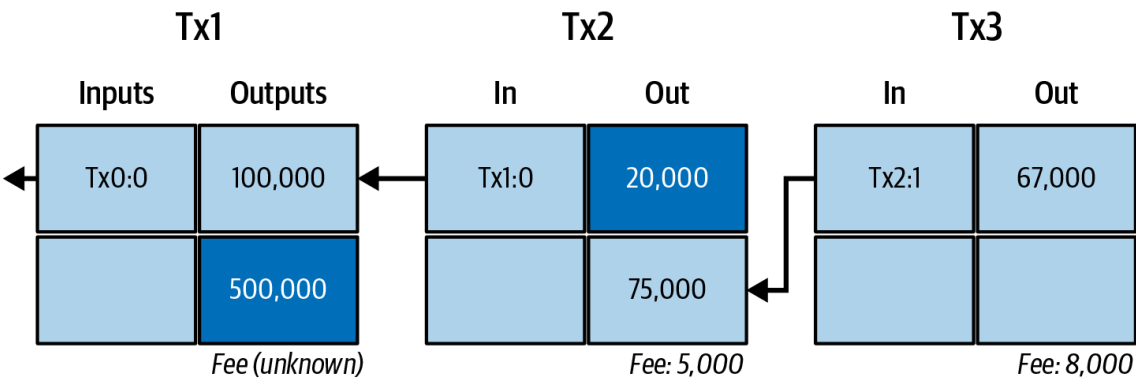


Figure 5. A chain of transactions, where the output of one transaction is the input of the next transaction.

Tip Serialized Bitcoin transactions—the data format that software uses for sending transactions—encodes the value to transfer using an integer of the smallest defined onchain unit of value. When Bitcoin was first created, this unit didn't have a name and some developers simply called it the *base unit.* Later many users began calling this unit a *satoshi* (sat) in honor of Bitcoin's creator. In A chain of transactions, where the output of one transaction is the input of the next transaction. and some other illustrations in this book, we use satoshi values because that's what the protocol itself uses.

2.3.3. Making Change

In addition to one or more outputs that pay the receiver of bitcoins, many transactions will also include an output that pays the spender of the bitcoins, called a *change* output. This is because transaction inputs, like currency notes, cannot be partly spent. If you purchase a $5 US item in a store but use a $20 bill to pay for the item, you expect to receive $15 in change. The same concept applies to Bitcoin transaction inputs. If you purchased an item that costs 5 bitcoins but only had an input worth 20 bitcoins to use, you would send one output of 5 bitcoins to the store owner and one output of 15 bitcoins back to yourself as change (not counting your transaction fee).

At the level of the Bitcoin protocol, there is no difference between a change output (and the address it pays, called a *change address*) and a payment output.

Importantly, the change address does not have to be the same address as that of the input and, for privacy reasons, is often a new address from the owner's wallet. In ideal circumstances, the two different uses of outputs both use never-before-seen addresses and otherwise look identical, preventing any third party from determining which outputs are change and which are payments. However, for illustration purposes, we've added shading to the change outputs in A chain of transactions, where the output of one transaction is the input of the next transaction..

Not every transaction has a change output. Those that don't are called *changeless transactions*, and they can have only a single output. Changeless transactions are only a practical option if the amount being spent is roughly the same as the amount available in the transaction inputs minus the anticipated transaction fee. In A chain of transactions, where the output of one transaction is the input of the next transaction., we see Bob creating Tx3 as a changeless transaction that spends the output he received in Tx2.

2.3.4. Coin Selection

Different wallets use different strategies when choosing which inputs to use in a payment, called *coin selection*.

They might aggregate many small inputs, or use one that is equal to or larger than the desired payment. Unless the wallet can aggregate inputs in such a way to exactly match the desired payment plus transaction fees, the wallet will need to generate some change. This is very similar to how people handle cash. If you always use the largest bill in your pocket, you will end up with a pocket full of loose change. If you only use the loose change, you'll often have only big bills. People subconsciously find a balance between these two extremes, and Bitcoin wallet developers strive to program this balance.

2.3.5. Common Transaction Forms

A very common form of transaction is a simple payment. This type of transaction has one input and two outputs and is shown in Most common transaction..

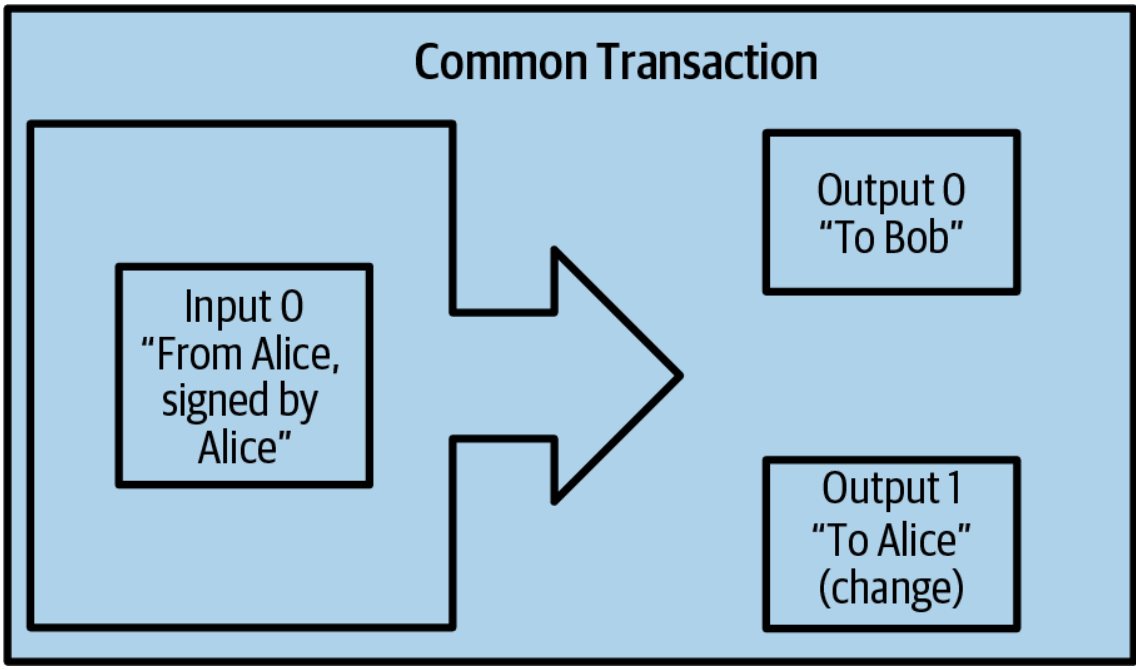


Figure 6. Most common transaction.

Another common form of transaction is a *consolidation transaction*, which spends several inputs into a single output (Consolidation transaction aggregating funds.). This represents the real-world equivalent of exchanging a pile of coins and currency notes for a single larger note. Transactions like these are sometimes generated by wallets and businesses to clean up lots of smaller amounts.

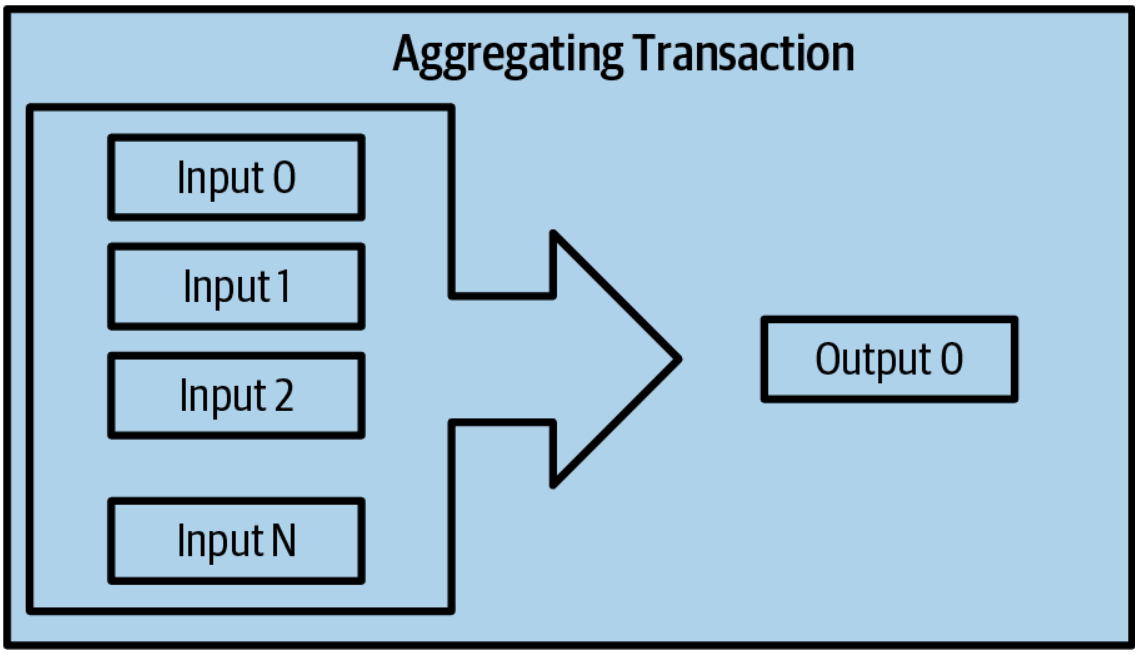


Figure 7. Consolidation transaction aggregating funds.

Finally, another transaction form that is seen often on the blockchain is *payment batching*, which pays to multiple outputs representing multiple recipients (Batch transaction distributing funds.). This type of transaction is sometimes used by commercial entities to distribute funds, such as when processing payroll payments to multiple employees.

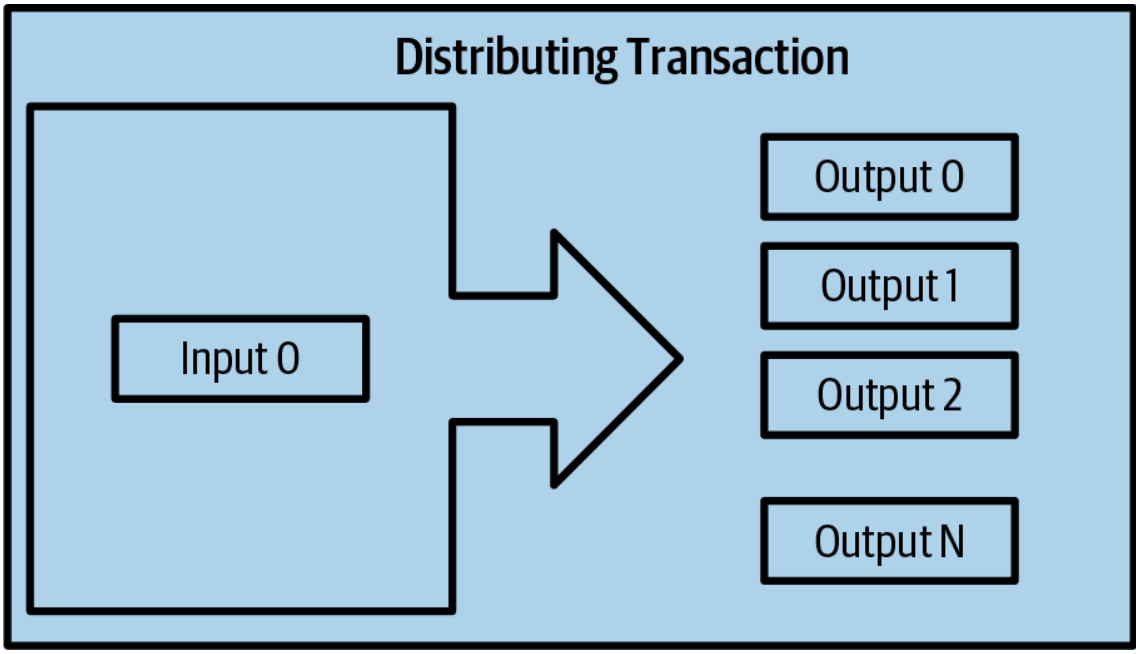


Figure 8. Batch transaction distributing funds.

2.4. Constructing a Transaction

Alice's wallet application contains all the logic for selecting inputs and generating outputs to build a transaction to Alice's specification. Alice only needs to choose a destination, amount, and transaction fee, and the rest happens in the wallet application without her seeing the details. Importantly, if a wallet already knows what inputs it controls, it can construct transactions even if it is completely offline. Like writing a check at home and later sending it to the bank in an envelope, the transaction does not need to be constructed and signed while connected to the Bitcoin network.

2.4.1. Getting the Right Inputs

Alice's wallet application will first have to find inputs that can pay the amount she wants to send to Bob. Most wallets keep track of all the available outputs belonging to addresses in the wallet. Therefore, Alice's wallet would contain a copy of the transaction output from Joe's transaction, which was created in exchange for cash (see Getting Your First Bitcoin). A Bitcoin wallet application that runs on a full node actually contains a copy of every confirmed transaction's unspent outputs, called *unspent transaction outputs* (UTXOs). However, because full nodes use more resources, many user wallets run lightweight clients that track only the user's own UTXOs.

In this case, this single UTXO is sufficient to pay for the podcast. Had this not been the case, Alice's wallet application might have to combine several smaller UTXOs, like picking coins from a purse, until it could find enough to pay for the podcast. In both cases, there might be a need to get some change back, which we will see in the next section, as the wallet application creates the transaction outputs (payments).

2.4.2. Creating the Outputs

A transaction output is created with a script that says something like, "This output is paid to whoever can present a signature from the key corresponding to Bob's public address." Because only Bob has the wallet with the keys corresponding to that address, only Bob's wallet can present such a signature to later spend this output. Alice will therefore *encumber* the output value with a demand for a signature from Bob.

This transaction will also include a second output because Alice's funds contain more money than the cost of the podcast. Alice's change output is created in the very same transaction as the payment to Bob. Essentially, Alice's wallet breaks her funds into two outputs: one to Bob and one back to herself. She can then spend the change output in a subsequent transaction.

Finally, for the transaction to be processed by the network in a timely fashion, Alice's wallet application will add a small fee. The fee is not explicitly stated in the transaction; it is implied by the difference in value between inputs and outputs. This transaction fee is collected by the miner as a fee for including the transaction in a block that gets recorded on the blockchain.

> **Tip** View the transaction from Alice to Bob's Store.

2.4.3. Adding the Transaction to the Blockchain

The transaction created by Alice's wallet application contains everything necessary to confirm ownership of the funds and assign new owners. Now, the transaction must be transmitted to the Bitcoin network where it will become part of the blockchain. In the next section we will see how a transaction becomes part of a new block and how the block is mined. Finally, we will see how the new block, once added to the blockchain, is increasingly trusted by the network as more blocks are added.

Transmitting the transaction

Because the transaction contains all the information necessary for it to be processed, it does not matter how or where it is transmitted to the Bitcoin network. The Bitcoin network is a peer-to-peer network, with each Bitcoin peer participating by connecting to several other Bitcoin peers. The purpose of the Bitcoin network is to propagate transactions and blocks to all participants.

How it propagates

Peers in the Bitcoin peer-to-peer network are programs that have both the software logic and the data necessary for them to fully verify the correctness of a new transaction. The connections between peers are often visualized as edges (lines) in a graph, with the peers themselves being the nodes (dots). For that reason, Bitcoin peers are commonly called "full verification nodes," or *full nodes* for short.

Alice's wallet application can send the new transaction to any Bitcoin node over any type of connection: wired, WiFi, mobile, etc. It can also send the transaction to another program (such as a block explorer) that will relay it to a node. Her Bitcoin wallet does not have to be connected to Bob's Bitcoin wallet directly and she does not have to use the internet connection offered by Bob, though both those options are possible too. Any Bitcoin node that receives a valid transaction it has not seen before will forward it to all other nodes to which it is connected, a propagation technique known as *gossiping*. Thus, the transaction rapidly propagates out across the peer-to-peer network, reaching a large percentage of the nodes within a few seconds.

Bob's view

If Bob's Bitcoin wallet application is directly connected to Alice's wallet application, Bob's wallet application might be the first to receive the transaction. However, even if Alice's wallet sends the transaction through other nodes, it will reach Bob's wallet within a few seconds. Bob's wallet will immediately identify Alice's transaction as an incoming payment because it contains an output redeemable by Bob's keys. Bob's wallet application can also independently verify that the transaction is well formed. If Bob is using his own full node, his wallet can further verify Alice's transaction only spends valid UTXOs.

2.5. Bitcoin Mining

Alice's transaction is now propagated on the Bitcoin network. It does not become part of the *blockchain* until it is included in a block by a process called *mining* and that block has been validated by full nodes. See Mining and Consensus for a detailed explanation.

Bitcoin's system of counterfeit protection is based on computation. Transactions are bundled into *blocks*. Blocks have a very small header that must be formed in a very specific way, requiring an enormous amount of computation to get right—but only a small amount of computation to verify as correct. The mining process serves two purposes in Bitcoin:

- Miners can only receive honest income from creating blocks that follow all of Bitcoin's *consensus rules*. Therefore, miners are normally incentivized to only include valid transactions in their blocks and the blocks they build upon. This allows users to optionally make a trust-based assumption that any transaction in a block is a valid transaction.

- Mining currently creates new bitcoins in each block, almost like a central bank printing new money. The amount of bitcoin created per block is limited and diminishes with time, following a fixed issuance schedule.

Mining achieves a fine balance between cost and reward. Mining uses electricity to solve a computational problem. A successful miner will collect a *reward* in the form of new bitcoins and transaction fees. However, the reward will only be collected if the miner has only included valid transactions, with the Bitcoin protocol's rules for *consensus* determining what is valid. This delicate balance provides security for Bitcoin without a central authority.

Mining is designed to be a decentralized lottery. Each miner can create their own lottery ticket by creating a *candidate block* that includes the new transactions they want to mine plus some additional data fields. The miner inputs their candidate into a specially designed algorithm that scrambles (or "hashes") the data, producing output that looks nothing like the input data. This *hash* function will always produce the same output for the same input —but nobody can predict what the output will look like for a new input, even if it is only slightly different from a previous input. If the output of the hash function matches a template determined by the Bitcoin protocol, the miner wins the lottery and Bitcoin users will accept the block with its transactions as a valid block. If the output doesn't match the template, the miner makes a small change to their candidate block and tries again. As of this writing, the number of candidate blocks miners need to try before finding a winning combination is about 168 billion trillion. That's also how many times the hash function needs to be run.

However, once a winning combination has been found, anyone can verify the block is valid by running the hash function just once. That makes a valid block something that requires an incredible amount of work to create but only a trivial amount of work to verify. The simple verification process is able to probabalistically prove the work was done, so the data necessary to generate that proof—in this case, the block—is called *proof of work (PoW)*.

Transactions are added to the new block, prioritized by the highest fee rate transactions first and a few other criteria. Each miner starts the process of mining a new candidate block of transactions as soon as they receive the previous block from the network, knowing that some other miner won that iteration of the lottery. They immediately create a new candidate block with a commitment to the previous block, fill it with transactions, and start calculating the PoW for the candidate block. Each miner includes a special transaction in their candidate blocks, one that pays their own Bitcoin address the block reward plus the sum of transaction fees from all the transactions included in the candidate block. If they find a solution that makes the candidate into a valid block, they receive this reward after their successful block is added to the global blockchain and the reward transaction they included becomes spendable. Miners who participate in a mining pool have set up their software to create candidate blocks that assign the reward to a pool address. From there, a share of the reward is distributed to members of the pool miners in proportion to the amount of work they contributed.

Alice’s transaction was picked up by the network and included in the pool of unverified transactions. Once validated by a full node, it was included in a candidate block. Approximately five minutes after the transaction was first transmitted by Alice’s wallet, a miner finds a solution for the block and announces it to the network. After each other miner validates the winning block, they start a new lottery to generate the next block.

The winning block containing Alice’s transaction became part of the blockchain. The block containing Alice’s transaction is counted as one *confirmation* of that transaction. After the block containing Alice’s transaction has propagated through the network, creating an alternative block with a different version of Alice’s transaction (such as a transaction that doesn’t pay Bob) would require performing the same amount of work as it will take all Bitcoin miners to create an entirely new block. When there are multiple alternative blocks to choose from, Bitcoin full nodes choose the chain of valid blocks with the most total PoW, called the *best blockchain*. For the entire network to accept an alternative block, an additional new block would need to be mined on top of the alternative.

That means miners have a choice. They can work with Alice on an alternative to the transaction where she pays Bob, perhaps with Alice paying miners a share of the money she previously paid Bob. This dishonest behavior will require they expend the effort required to create two new blocks. Instead, miners who behave honestly can create a single new block and receive all of the fees from the transactions they include in it, plus the block subsidy. Normally, the high cost of dishonestly creating two blocks for a small additional payment is much less profitable than honestly creating a new block, making it unlikely that a confirmed transaction will be deliberately changed. For Bob, this means that he can begin to believe that the payment from Alice can be relied upon.

> **Tip** You can see the block that includes Alice’s transaction.

Approximately 19 minutes after the block containing Alice’s transaction is broadcast, a new block is mined by another miner. Because this new block is built on top of the block that contained Alice’s transaction (giving Alice’s transaction two confirmations), Alice’s transaction can now only be changed if two alternative blocks are mined—plus a new block built on top of them—for a total of three blocks that would need to be mined for Alice to take back the money she sent Bob. Each block mined on top of the one containing Alice’s transaction counts as an additional confirmation. As the blocks pile on top of each other, it becomes harder to reverse the transaction, thereby giving Bob more and more confidence that Alice’s payment is secure.

In Alice’s transaction included in a block., we can see the block that contains Alice’s transaction. Below it are hundreds of thousands of blocks, linked to each other in a chain of blocks (blockchain) all the way back to block #0, known as the *genesis block*. Over time, as the "height" of new blocks increases, so does the computation difficulty for the chain as a whole. By convention, any block with more than six confirmations is considered very hard

to change, because it would require an immense amount of computation to recalculate six blocks (plus one new block). We will examine the process of mining and the way it builds confidence in more detail in Mining and Consensus.

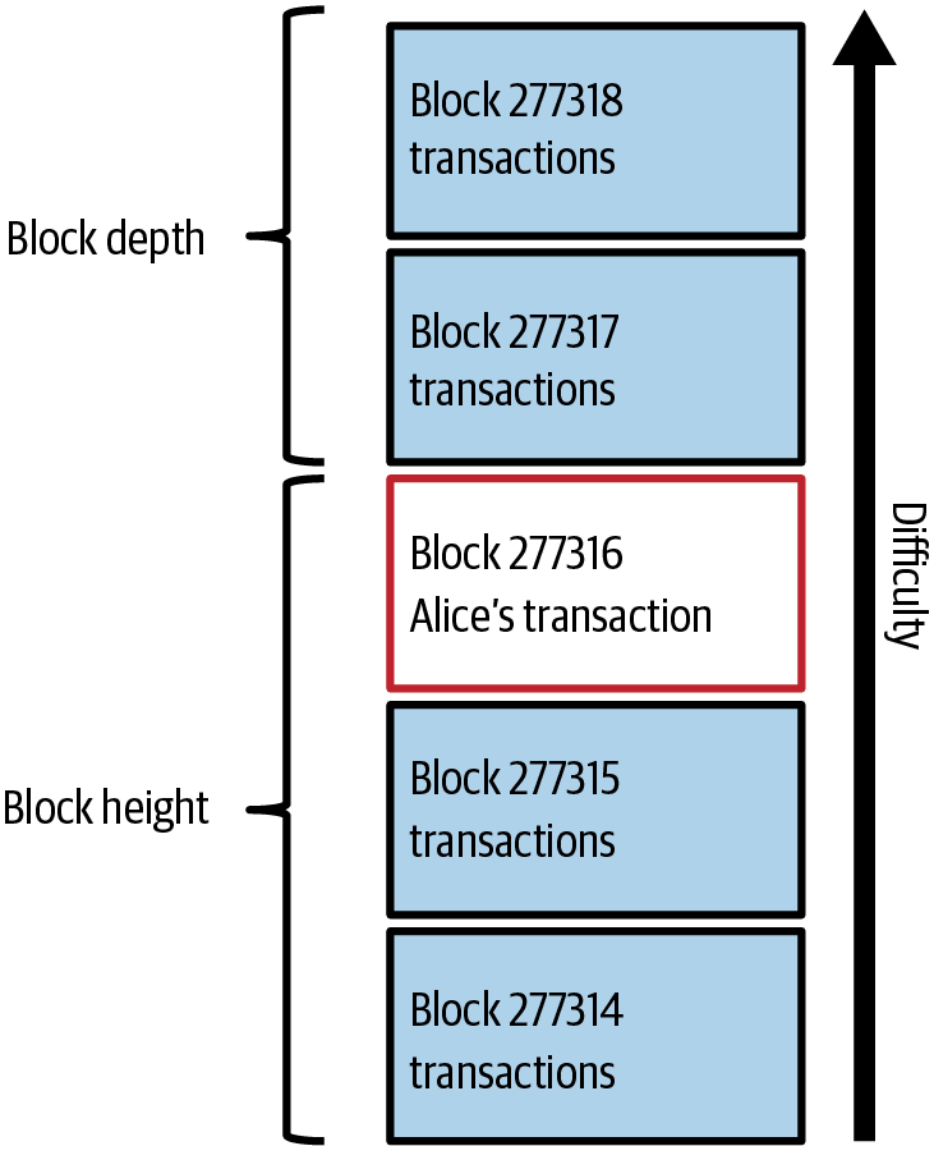


Figure 9. Alice's transaction included in a block.

2.6. Spending the Transaction

Now that Alice's transaction has been embedded in the blockchain as part of a block, it is visible to all Bitcoin applications. Each Bitcoin full node can independently verify the

transaction as valid and spendable. Full nodes validate every transfer of the funds from the moment the bitcoins were first generated in a block through each subsequent transaction until they reach Bob's address. Lightweight clients can partially verify payments by confirming that the transaction is in the blockchain and has several blocks mined after it, thus providing assurance that the miners expended significant effort committing to it (see Lightweight Clients).

Bob can now spend the output from this and other transactions. For example, Bob can pay a contractor or supplier by transferring value from Alice's podcast payment to these new owners. As Bob spends the payments received from Alice and other customers, he extends the chain of transactions. Let's assume that Bob pays his web designer Gopesh for a new website page. Now the chain of transactions will look like Alice's transaction as part of a transaction chain from Joe to Gopesh..

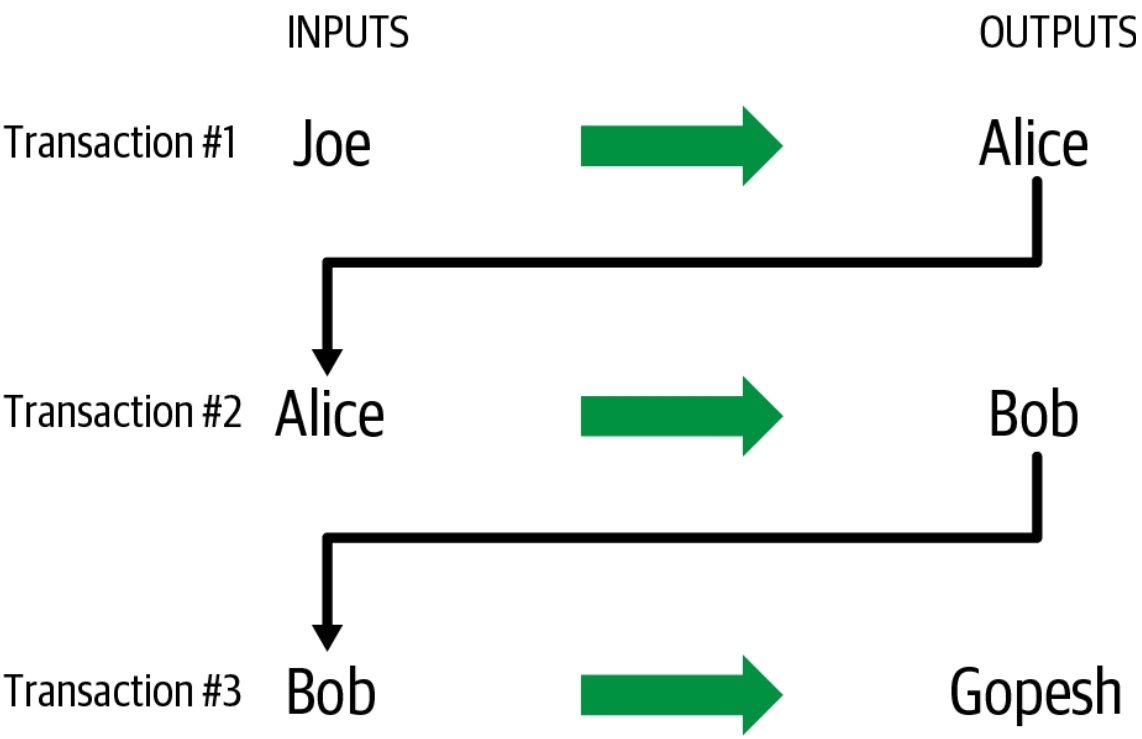


Figure 10. Alice's transaction as part of a transaction chain from Joe to Gopesh.

In this chapter, we saw how transactions build a chain that moves value from owner to owner. We also tracked Alice's transaction from the moment it was created in her wallet, through the Bitcoin network, and to the miners who recorded it on the blockchain. In the rest of this book, we will examine the specific technologies behind wallets, addresses, signatures, transactions, the network, and finally, mining.

3. Bitcoin Core: The Reference Implementation

People only accept money in exchange for their valuable goods and services if they believe that they'll be able to spend that money later. Money that is counterfeit or unexpectedly debased may not be spendable later, so every person accepting bitcoins has a strong incentive to verify the integrity of the bitcoins they receive. The Bitcoin system was designed so that it's possible for software running entirely on your local computer to perfectly prevent counterfeiting, debasement, and several other critical problems. Software which provides that function is called a *full verification node* because it verifies every confirmed Bitcoin transaction against every rule in the system. Full verification nodes, *full nodes* for short, may also provide tools and data for understanding how Bitcoin works and what is currently happening in the network.

In this chapter, we'll install Bitcoin Core, the implementation that most full node operators have used since the beginning of the Bitcoin network. We'll then inspect blocks, transactions, and other data from your node, data which is authoritative—not because some powerful entity designated it as such but because your node independently verified it. Throughout the rest of this book, we'll continue using Bitcoin Core to create and examine data related to the blockchain and network.

3.1. From Bitcoin to Bitcoin Core

Bitcoin is an *open source* project and the source code is available under an open (MIT) license, free to download and use for any purpose. More than just being open source, Bitcoin is developed by an open community of volunteers. At first, that community consisted of only Satoshi Nakamoto. By 2023, Bitcoin's source code had more than 1,000 contributors with about a dozen developers working on the code almost full time and several dozen more on a part-time basis. Anyone can contribute to the code—including you!

When Bitcoin was created by Satoshi Nakamoto, the software was mostly completed before publication of the whitepaper (reproduced in The Bitcoin Whitepaper by Satoshi Nakamoto). Satoshi wanted to make sure the implementation worked before publishing a paper about it. That first implementation, then simply known as "Bitcoin," has been heavily modified and improved. It has evolved into what is known as *Bitcoin Core*, to differentiate it from other implementations. Bitcoin Core is the *reference implementation* of the Bitcoin system, meaning that it provides a reference for how each part of the technology should be implemented. Bitcoin Core implements all aspects of Bitcoin, including wallets, a transaction and block validation engine, tools for block construction, and all modern parts of Bitcoin peer-to-peer communication.

Bitcoin Core architecture (Source: Eric Lombrozo). shows the architecture of Bitcoin Core.

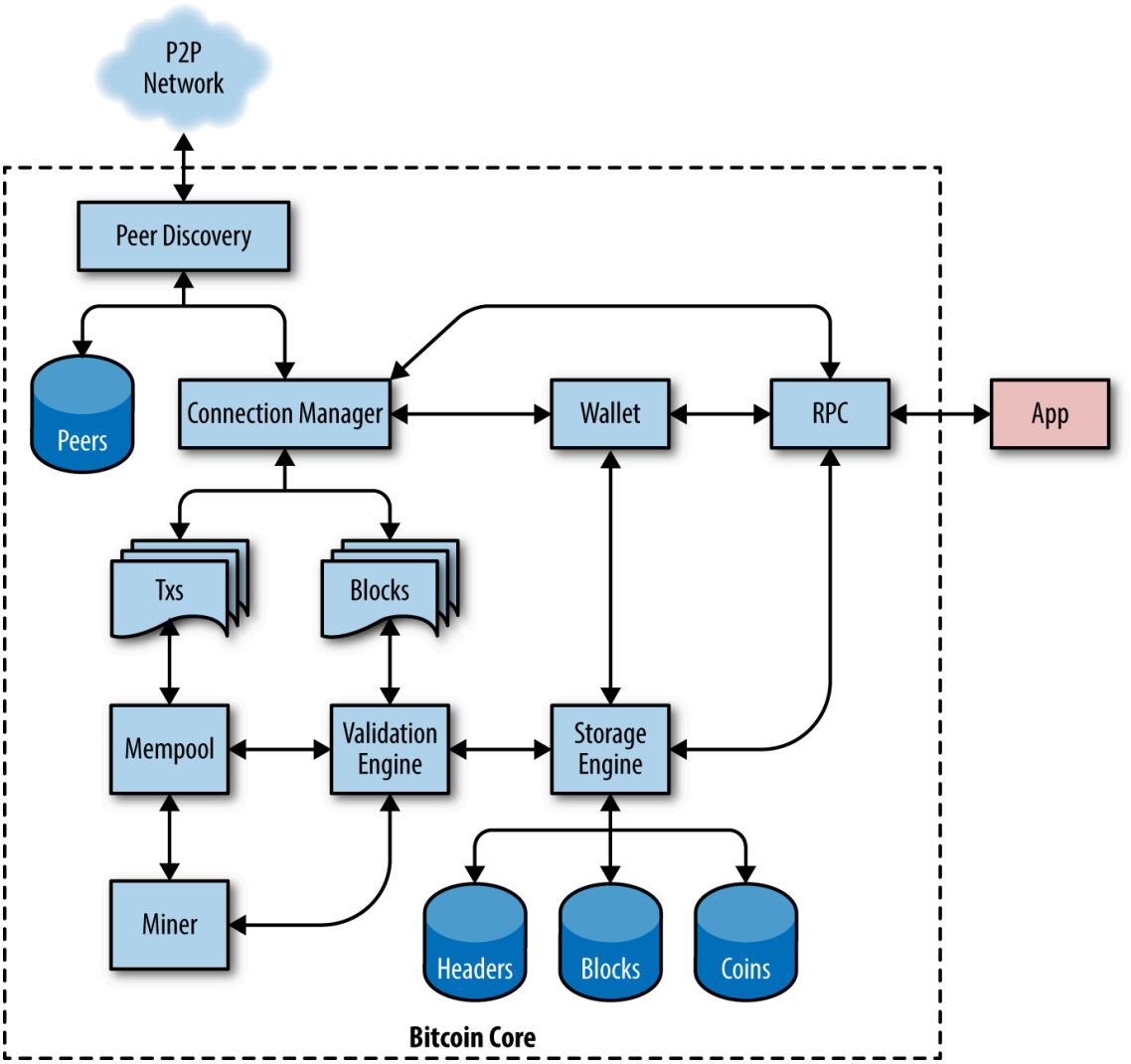


Figure 11. Bitcoin Core architecture (Source: Eric Lombrozo).

Although Bitcoin Core serves as a reference implementation for many major parts of the system, the Bitcoin whitepaper describes several early parts of the system. Most major parts of the system since 2011 have been documented in a set of Bitcoin Improvement Proposals (BIPs). Throughout this book, we refer to BIP specifications by their number; for example, BIP9 describes a mechanism used for several major upgrades to Bitcoin.

3.2. Bitcoin Development Environment

If you're a developer, you will want to set up a development environment with all the tools, libraries, and support software for writing Bitcoin applications. In this highly technical chapter, we'll walk through that process step by step. If the material becomes too dense (and you're not actually setting up a development environment) feel free to skip to the next chapter, which is less technical.

3.3. Compiling Bitcoin Core from the Source Code

Bitcoin Core's source code can be downloaded as an archive or by cloning the source repository from GitHub. On the Bitcoin Core download page, select the most recent version and download the compressed archive of the source code. Alternatively, use the Git command line to create a local copy of the source code from the GitHub Bitcoin page.

> **Tip** In many of the examples in this chapter, we will be using the operating system's command-line interface (also known as a "shell"), accessed via a "terminal" application. The shell will display a prompt, you type a command, and the shell responds with some text and a new prompt for your next command. The prompt may look different on your system, but in the following examples, it is denoted by a $ symbol. In the examples, when you see text after a $ symbol, don't type the $ symbol but type the command immediately following it, then press Enter to execute the command. In the examples, the lines below each command are the operating system's responses to that command. When you see the next $ prefix, you'll know it's a new command and you should repeat the process.

Here, we use the git command to create a local copy ("clone") of the source code:

```
$ git clone https://github.com/bitcoin/bitcoin.git
Cloning into 'bitcoin'...
remote: Enumerating objects: 245912, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 245912 (delta 1), reused 2 (delta 1), pack-reused 245909
Receiving objects: 100% (245912/245912), 217.74 MiB | 13.05 MiB/s, done.
Resolving deltas: 100% (175649/175649), done.
```

> **Tip** Git is the most widely used distributed version control system, an essential part of any software developer's toolkit. You may need to install the git command, or a graphical user interface for Git, on your operating system if you do not have it already.

When the Git cloning operation has completed, you will have a complete local copy of the source code repository in the directory *bitcoin*. Change to this directory using the cd command:

```
$ cd bitcoin
```

3.3.1. Selecting a Bitcoin Core Release

By default, the local copy will be synchronized with the most recent code, which might be an unstable or beta version of Bitcoin. Before compiling the code, select a specific version by checking out a release *tag*. This will synchronize the local copy with a specific snapshot of the code repository identified by a keyword tag. Tags are used by the developers to mark specific releases of the code by version number. First, to find the available tags, we use the git tag command:

```
$ git tag
v0.1.5
v0.1.6test1
v0.10.0
...
v0.11.2
v0.11.2rc1
v0.12.0rc1
v0.12.0rc2
...
```

The list of tags shows all the released versions of Bitcoin. By convention, *release candidates*, which are intended for testing, have the suffix "rc." Stable releases that can be run on production systems have no suffix. From the preceding list, select the highest version release, which at the time of writing was v24.0.1. To synchronize the local code with this version, use the git checkout command:

```
$ git checkout v24.0.1
Note: switching to 'v24.0.1'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

HEAD is now at b3f866a8d Merge bitcoin/bitcoin#26647: 24.0.1 final changes
```

You can confirm you have the desired version "checked out" by issuing the command git status:

```
HEAD detached at v24.0.1
nothing to commit, working tree clean
```

3.3.2. Configuring the Bitcoin Core Build

The source code includes documentation, which can be found in a number of files. Review the main documentation located in *README.md* in the *bitcoin* directory. In this chapter, we will build the Bitcoin Core daemon (server), also known as bitcoind on Linux (a Unix-like system). Review the instructions for compiling the bitcoind command-line client on your platform by reading *doc/build-unix.md*. Alternative instructions can be found in the *doc* directory; for example, *build-windows.md* for Windows instructions. As of this writing, instructions are available for Android, FreeBSD, NetBSD, OpenBSD, macOS (OSX), Unix, and Windows.

Carefully review the build prerequisites, which are in the first part of the build documentation. These are libraries that must be present on your system before you can begin to compile Bitcoin. If these prerequisites are missing, the build process will fail with an error. If this happens because you missed a prerequisite, you can install it and then resume the build process from where you left off. Assuming the prerequisites are installed, you start the build process by generating a set of build scripts using the *autogen.sh* script:

```
$ ./autogen.sh
libtoolize: putting auxiliary files in AC_CONFIG_AUX_DIR, 'build-aux'.
libtoolize: copying file 'build-aux/ltmain.sh'
libtoolize: putting macros in AC_CONFIG_MACRO_DIRS, 'build-aux/m4'.
 ...
configure.ac:58: installing 'build-aux/missing'
src/Makefile.am: installing 'build-aux/depcomp'
parallel-tests: installing 'build-aux/test-driver'
```

The *autogen.sh* script creates a set of automatic configuration scripts that will interrogate your system to discover the correct settings and ensure you have all the necessary libraries to compile the code. The most important of these is the configure script that offers a number of different options to customize the build process. Use the --help flag to see the various options:

```
$ ./configure --help
`configure' configures Bitcoin Core 24.0.1 to adapt to many kinds of systems.

Usage: ./configure [OPTION]... [VAR=VALUE]...

...
Optional Features:
  --disable-option-checking  ignore unrecognized --enable/--with options
  --disable-FEATURE       do not include FEATURE (same as --enable-FEATURE=no)
  --enable-FEATURE[=ARG]  include FEATURE [ARG=yes]
```

```
  --enable-silent-rules   less verbose build output (undo: "make V=1")
  --disable-silent-rules  verbose build output (undo: "make V=0")
...
```

The configure script allows you to enable or disable certain features of bitcoind through the use of the --enable-FEATURE and --disable-FEATURE flags, where `FEATURE` is replaced by the feature name, as listed in the help output. In this chapter, we will build the bitcoind client with all the default features. We won't be using the configuration flags, but you should review them to understand what optional features are part of the client. If you are in an academic setting, computer lab restrictions may require you to install applications in your home directory (e.g., using --prefix=$HOME).

Here are some useful options that override the default behavior of the configure script:

`--prefix=$HOME`
: This overrides the default installation location (which is */usr/local/*) for the resulting executable. Use `$HOME` to put everything in your home directory, or a different path.

`--disable-wallet`
: This is used to disable the reference wallet implementation.

`--with-incompatible-bdb`
: If you are building a wallet, allow the use of an incompatible version of the Berkeley DB library.

`--with-gui=no`
: Don't build the graphical user interface, which requires the Qt library. This builds server and command-line Bitcoin Core only.

Next, run the configure script to automatically discover all the necessary libraries and create a customized build script for your system:

```
$ ./configure
checking for pkg-config... /usr/bin/pkg-config
checking pkg-config is at least version 0.9.0... yes
checking build system type... x86_64-pc-linux-gnu
checking host system type... x86_64-pc-linux-gnu
checking for a BSD-compatible install... /usr/bin/install -c
...
[many pages of configuration tests follow]
...
```

If all went well, the configure command will end by creating the customized build scripts that will allow us to compile bitcoind. If there are any missing libraries or errors, the configure command will terminate with an error instead of creating the build scripts. If an error occurs, it is most likely because of a missing or incompatible library. Review the build documentation again and make sure you install the missing prerequisites. Then run configure again and see if that fixes the error.

3.3.3. Building the Bitcoin Core Executables

Next, you will compile the source code, a process that can take up to an hour to complete, depending on the speed of your CPU and available memory. If an error occurs, or the compilation process is interrupted, it can be resumed any time by typing make again. Type **make** to start compiling the executable application:

```
$ make
Making all in src
  CXX      bitcoind-bitcoind.o
  CXX      libbitcoin_node_a-addrdb.o
  CXX      libbitcoin_node_a-addrman.o
  CXX      libbitcoin_node_a-banman.o
  CXX      libbitcoin_node_a-blockencodings.o
  CXX      libbitcoin_node_a-blockfilter.o
[... many more compilation messages follow ...]
```

On a fast system with more than one CPU, you might want to set the number of parallel compile jobs. For instance, make -j 2 will use two cores if they are available. If all goes well, Bitcoin Core is now compiled. You should run the unit test suite with make check to ensure the linked libraries are not broken in obvious ways. The final step is to install the various executables on your system using the make install command. You may be prompted for your user password because this step requires administrative privileges:

```
$ make check && sudo make install
Password:
Making install in src
 ../build-aux/install-sh -c -d '/usr/local/lib'
libtool: install: /usr/bin/install -c bitcoind /usr/local/bin/bitcoind
libtool: install: /usr/bin/install -c bitcoin-cli /usr/local/bin/bitcoin-cli
libtool: install: /usr/bin/install -c bitcoin-tx /usr/local/bin/bitcoin-tx
...
```

The default installation of bitcoind puts it in */usr/local/bin*. You can confirm that Bitcoin Core is correctly installed by asking the system for the path of the executables, as follows:

```
$ which bitcoind
/usr/local/bin/bitcoind

$ which bitcoin-cli
/usr/local/bin/bitcoin-cli
```

3.4. Running a Bitcoin Core Node

Bitcoin’s peer-to-peer network is composed of network "nodes," run mostly by individuals and some of the businesses that provide Bitcoin services. Those running Bitcoin nodes have a direct and authoritative view of the Bitcoin blockchain, with a local copy of all the spendable bitcoins independently validated by their own system. By running a node, you

don’t have to rely on any third party to validate a transaction. Additionally, by using a Bitcoin node to fully validate the transactions you receive to your wallet, you contribute to the Bitcoin network and help make it more robust.

Running a node, however, requires downloading and processing over 500 GB of data initially and about 400 MB of Bitcoin transactions per day. These figures are for 2023 and will likely increase over time. If you shut down your node or get disconnected from the internet for several days, your node will need to download the data that it missed. For example, if you close Bitcoin Core for 10 days, you will need to download approximately 4 GB the next time you start it.

Depending on whether you choose to index all transactions and keep a full copy of the blockchain, you may also need a lot of disk space—at least 1 TB if you plan to run Bitcoin Core for several years. By default, Bitcoin nodes also transmit transactions and blocks to other nodes (called "peers"), consuming upload internet bandwidth. If your internet connection is limited, has a low data cap, or is metered (charged by the gigabit), you should probably not run a Bitcoin node on it, or run it in a way that constrains its bandwidth (see Configuring the Bitcoin Core Node). You may connect your node instead to an alternative network, such as a free satellite data provider like Blockstream Satellite.

> **Tip** Bitcoin Core keeps a full copy of the blockchain by default, with nearly every transaction that has ever been confirmed on the Bitcoin network since its inception in 2009. This dataset is hundreds of gigabytes in size and is downloaded incrementally over several hours or days, depending on the speed of your CPU and internet connection. Bitcoin Core will not be able to process transactions or update account balances until the full blockchain dataset is downloaded. Make sure you have enough disk space, bandwidth, and time to complete the initial synchronization. You can configure Bitcoin Core to reduce the size of the blockchain by discarding old blocks, but it will still download the entire dataset.

Despite these resource requirements, thousands of people run Bitcoin nodes. Some are running on systems as simple as a Raspberry Pi (a $35 USD computer the size of a pack of cards).

Why would you want to run a node? Here are some of the most common reasons:

- You do not want to rely on any third party to validate the transactions you receive.
- You do not want to disclose to third parties which transactions belong to your wallet.
- You are developing Bitcoin software and need to rely on a Bitcoin node for programmable (API) access to the network and blockchain.
- You are building applications that must validate transactions according to Bitcoin’s consensus rules. Typically, Bitcoin software companies run several nodes.

- You want to support Bitcoin. Running a node that you use to validate the transactions you receive to your wallet makes the network more robust.

If you're reading this book and interested in strong security, superior privacy, or developing Bitcoin software, you should be running your own node.

3.5. Configuring the Bitcoin Core Node

Bitcoin Core will look for a configuration file in its data directory on every start. In this section we will examine the various configuration options and set up a configuration file. To locate the configuration file, run bitcoind -printtoconsole in your terminal and look for the first couple of lines:

```
$ bitcoind -printtoconsole
2023-01-28T03:21:42Z Bitcoin Core version v24.0.1
2023-01-28T03:21:42Z Using the 'x86_shani(1way,2way)' SHA256 implementation
2023-01-28T03:21:42Z Using RdSeed as an additional entropy source
2023-01-28T03:21:42Z Using RdRand as an additional entropy source
2023-01-28T03:21:42Z Default data directory /home/harding/.bitcoin
2023-01-28T03:21:42Z Using data directory /home/harding/.bitcoin
2023-01-28T03:21:42Z Config file: /home/harding/.bitcoin/bitcoin.conf
...
[a lot more debug output]
...
```

You can hit Ctrl-C to shut down the node once you determine the location of the config file. Usually the configuration file is inside the *.bitcoin* data directory under your user's home directory. Open the configuration file in your preferred editor.

Bitcoin Core offers more than 100 configuration options that modify the behavior of the network node, the storage of the blockchain, and many other aspects of its operation. To see a listing of these options, run bitcoind --help:

```
$ bitcoind --help
Bitcoin Core version v24.0.1

Usage:  bitcoind [options]                     Start Bitcoin Core

Options:

  -?
       Print this help message and exit

  -alertnotify=<cmd>
       Execute command when an alert is raised (%s in cmd is replaced by
       message)
...
[many more options]
```

Here are some of the most important options that you can set in the configuration file, or as command-line parameters to bitcoind:

alertnotify
: Run a specified command or script to send emergency alerts to the owner of this node.

conf
: An alternative location for the configuration file. This only makes sense as a command-line parameter to bitcoind, as it can't be inside the configuration file it refers to.

datadir
: Select the directory and filesystem in which to put all the blockchain data. By default this is the *.bitcoin* subdirectory of your home directory. Depending on your configuration, this can use from about 10 GB to almost 1 TB as of this writing, with the maximum size expected to increase by several hundred gigabytes per year.

prune
: Reduce the blockchain disk space requirements to this many megabytes by deleting old blocks. Use this on a resource-constrained node that can't fit the full blockchain. Other parts of the system will use other disk space that can't currently be pruned, so you will still need at least the minimum amount of space mentioned in the datadir option.

txindex
: Maintain an index of all transactions. This allows you to programmatically retrieve any transaction by its ID provided that the block containing that transaction hasn't been pruned.

dbcache
: The size of the UTXO cache. The default is 450 mebibytes (MiB). Increase this size on high-end hardware to read and write from your disk less often, or reduce the size on low-end hardware to save memory at the expense of using your disk more frequently.

blocksonly
: Minimize your bandwidth usage by only accepting blocks of confirmed transactions from your peers instead of relaying unconfirmed transactions.

maxmempool
: Limit the transaction memory pool to this many megabytes. Use it to reduce memory use on memory-constrained nodes.

Transaction Database Index and txindex Option

By default, Bitcoin Core builds a database containing *only* the transactions related to the user's wallet. If you want to be able to access *any* transaction with commands like getrawtransaction (see Exploring and Decoding Transactions), you need to configure Bitcoin Core to build a complete transaction index, which can be achieved with the txindex option. Set txindex=1 in the Bitcoin Core configuration file. If you don't set this option at first and later set it to full indexing, you need to wait for it to rebuild the index.

Sample configuration of a full-index node shows how you might combine the preceding options with a fully indexed node, running as an API backend for a bitcoin application.

Example 1. Sample configuration of a full-index node

```
alertnotify=myemailscript.sh "Alert: %s"
datadir=/lotsofspace/bitcoin
txindex=1
```

Sample configuration of a resource-constrained system shows a resource-constrained node running on a smaller server.

Example 2. Sample configuration of a resource-constrained system

```
alertnotify=myemailscript.sh "Alert: %s"
blocksonly=1
prune=5000
dbcache=150
maxmempool=150
```

After you've edited the configuration file and set the options that best represent your needs, you can test bitcoind with this configuration. Run Bitcoin Core with the option printtoconsole to run in the foreground with output to the console:

```
$ bitcoind -printtoconsole
2023-01-28T03:43:39Z Bitcoin Core version v24.0.1
2023-01-28T03:43:39Z Using the 'x86_shani(1way,2way)' SHA256 implementation
2023-01-28T03:43:39Z Using RdSeed as an additional entropy source
2023-01-28T03:43:39Z Using RdRand as an additional entropy source
2023-01-28T03:43:39Z Default data directory /home/harding/.bitcoin
2023-01-28T03:43:39Z Using data directory /lotsofspace/bitcoin
2023-01-28T03:43:39Z Config file: /home/harding/.bitcoin/bitcoin.conf
2023-01-28T03:43:39Z Config file arg: [main] blockfilterindex="1"
2023-01-28T03:43:39Z Config file arg: [main] maxuploadtarget="1000"
2023-01-28T03:43:39Z Config file arg: [main] txindex="1"
2023-01-28T03:43:39Z Setting file arg: wallet = ["msig0"]
2023-01-28T03:43:39Z Command-line arg: printtoconsole=""
2023-01-28T03:43:39Z Using at most 125 automatic connections
2023-01-28T03:43:39Z Using 16 MiB out of 16 MiB requested for signature cache
2023-01-28T03:43:39Z Using 16 MiB out of 16 MiB requested for script execution
2023-01-28T03:43:39Z Script verification uses 3 additional threads
2023-01-28T03:43:39Z scheduler thread start
2023-01-28T03:43:39Z [http] creating work queue of depth 16
2023-01-28T03:43:39Z Using random cookie authentication.
2023-01-28T03:43:39Z Generated RPC cookie /lotsofspace/bitcoin/.cookie
2023-01-28T03:43:39Z [http] starting 4 worker threads
2023-01-28T03:43:39Z Using wallet directory /lotsofspace/bitcoin/wallets
2023-01-28T03:43:39Z init message: Verifying wallet(s)…
2023-01-28T03:43:39Z Using BerkeleyDB version Berkeley DB 4.8.30
2023-01-28T03:43:39Z Using /16 prefix for IP bucketing
2023-01-28T03:43:39Z init message: Loading P2P addresses…
2023-01-28T03:43:39Z Loaded 63866 addresses from peers.dat  114ms
[... more startup messages ...]
```

You can hit Ctrl-C to interrupt the process once you are satisfied that it is loading the correct settings and running as you expect.

To run Bitcoin Core in the background as a process, start it with the daemon option, as bitcoind -daemon.

To monitor the progress and runtime status of your Bitcoin node, start it in daemon mode and then use the command bitcoin-cli getblockchaininfo:

```
$ bitcoin-cli getblockchaininfo
```

```
{
  "chain": "main",
  "blocks": 0,
  "headers": 83999,
  "bestblockhash": "[...]19d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f",
  "difficulty": 1,
  "time": 1673379796,
  "mediantime": 1231006505,
  "verificationprogress": 3.783041623201835e-09,
  "initialblockdownload": true,
  "chainwork": "[...]0000000000000000000000000000000000000000000000100010001",
  "size_on_disk": 89087,
  "pruned": false,
  "warnings": ""
}
```

This shows a node with a blockchain height of 0 blocks and 83,999 headers. The node first fetches the block headers from its peers in order to find the blockchain with the most proof of work and afterward continues to download the full blocks, validating them as it goes.

Once you are happy with the configuration options you have selected, you should add Bitcoin Core to the startup scripts in your operating system, so that it runs continuously and restarts when the operating system restarts. You will find a number of example startup scripts for various operating systems in Bitcoin Core's source directory under *contrib/init* and a *README.md* file showing which system uses which script.

3.6. Bitcoin Core API

Bitcoin Core implements a JSON-RPC interface that can also be accessed using the command-line helper bitcoin-cli. The command line allows us to experiment interactively with the capabilities that are also available programmatically via the API. To start, invoke the help command to see a list of the available Bitcoin Core RPC commands:

```
$ bitcoin-cli help
+== Blockchain ==
getbestblockhash
getblock "blockhash" ( verbosity )
getblockchaininfo
...
walletpassphrase "passphrase" timeout
```

```
walletpassphrasechange "oldpassphrase" "newpassphrase"
walletprocesspsbt "psbt" ( sign "sighashtype" bip32derivs finalize )
```

Each of these commands may take a number of parameters. To get additional help, a detailed description, and information on the parameters, add the command name after help. For example, to see help on the getblockhash RPC command:

```
$ bitcoin-cli help getblockhash
getblockhash height

Returns hash of block in best-block-chain at height provided.

Arguments:
1. height    (numeric, required) The height index

Result:
"hex"    (string) The block hash

Examples:
> bitcoin-cli getblockhash 1000
> curl --user myusername --data-binary '{"jsonrpc": "1.0", "id": "curltest",
  "method": "getblockhash",
  "params": [1000]}' -H 'content-type: text/plain;' http://127.0.0.1:8332/
```

At the end of the help information you will see two examples of the RPC command, using the bitcoin-cli helper or the HTTP client curl. These examples demonstrate how you might call the command. Copy the first example and see the result:

```
$ bitcoin-cli getblockhash 1000
00000000c937983704a73af28acdec37b049d214adbda81d7e2a3dd146f6ed09
```

The result is a block hash, which is described in more detail in the following chapters. But for now, this command should return the same result on your system, demonstrating that your Bitcoin Core node is running, is accepting commands, and has information about block 1,000 to return to you.

In the next sections we will demonstrate some very useful RPC commands and their expected output.

3.6.1. Getting Information on Bitcoin Core's Status

Bitcoin Core provides status reports on different modules through the JSON-RPC interface. The most important commands include getblockchaininfo, getmempoolinfo, getnetworkinfo, and getwalletinfo.

Bitcoin's getblockchaininfo RPC command was introduced earlier. The getnetworkinfo command displays basic information about the status of the Bitcoin network node. Use bitcoin-cli to run it:

```
$ bitcoin-cli getnetworkinfo
```

```
{
  "version": 240001,
  "subversion": "/Satoshi:24.0.1/",
  "protocolversion": 70016,
  "localservices": "0000000000000409",
  "localservicesnames": [
    "NETWORK",
    "WITNESS",
    "NETWORK_LIMITED"
  ],
  "localrelay": true,
  "timeoffset": -1,
  "networkactive": true,
  "connections": 10,
  "connections_in": 0,
  "connections_out": 10,
  "networks": [
    "...detailed information about all networks..."
  ],
  "relayfee": 0.00001000,
  "incrementalfee": 0.00001000,
  "localaddresses": [
  ],
  "warnings": ""
}
```

The data is returned in JavaScript Object Notation (JSON), a format that can easily be "consumed" by all programming languages but is also quite human-readable. Among this data we see the version numbers for the Bitcoin Core software and Bitcoin protocol. We see the current number of connections and various information about the Bitcoin network and the settings related to this node.

> **Tip** It will take some time, perhaps more than a day, for bitcoind to catch up to the current blockchain height as it downloads blocks from other Bitcoin nodes and validates every transaction in those blocks—almost a billion transactions as of this writing. You can check its progress using getblockchaininfo to see the number of known blocks. The examples in the rest of this chapter assume you're at least at block 775,072. Because the security of Bitcoin transactions depends on blocks, some of the information in the following examples will change slightly depending on how many blocks your node has.

3.6.2. Exploring and Decoding Transactions

In Buying from an Online Store, Alice made a purchase from Bob's store. Her transaction was recorded on the blockchain. Let's use the API to retrieve and examine that transaction by passing the transaction ID (txid) as a parameter:

Alice’s serialized transaction

```
$ bitcoin-cli getrawtransaction 466200308696215bbc949d5141a49a41\
38ecdfdfaa2a8029c1f9bcecd1f96177

01000000000101eb3ae38f27191aa5f3850dc9cad00492b88b72404f9da13569
8679268041c54a0100000000ffffffff02204e0000000000002251203b41daba
4c9ace578369740f15e5ec880c28279ee7f51b07dca69c7061e07068f8240100
000000001600147752c165ea7be772b2c0acb7f4d6047ae6f4768e0141cf5efe
2d8ef13ed0af21d4f4cb82422d6252d70324f6f4576b727b7d918e521c00b51b
e739df2f899c49dc267c0ad280aca6dab0d2fa2b42a45182fc83e81713010000
0000
```

> **Tip** A transaction ID (txid) is not authoritative. Absence of a txid in the blockchain does not mean the transaction was not processed. This is known as "transaction malleability," because transactions can be modified prior to confirmation in a block, changing their txids. After a transaction is included in a block, its txid cannot change unless there is a blockchain reorganization where that block is removed from the best blockchain. Reorganizations are rare after a transaction has several confirmations.

The command getrawtransaction returns a serialized transaction in hexadecimal notation. To decode that, we use the decoderawtransaction command, passing the hex data as a parameter. You can copy the hex returned by getrawtransaction and paste it as a parameter to decoderawtransaction:

```
$ bitcoin-cli decoderawtransaction 01000000000101eb3ae38f27191aa5f3850dc9cad0\
0492b88b72404f9da135698679268041c54a0100000000ffffffff02204e00000000000022512\
03b41daba4c9ace578369740f15e5ec880c28279ee7f51b07dca69c7061e07068f82401000000\
00001600147752c165ea7be772b2c0acb7f4d6047ae6f4768e0141cf5efe2d8ef13ed0af21d4f\
4cb82422d6252d70324f6f4576b727b7d918e521c00b51be739df2f899c49dc267c0ad280aca6\
dab0d2fa2b42a45182fc83e817130100000000
```

```
{
  "txid": "466200308696215bbc949d5141a49a4138ecdfdfaa2a8029c1f9bcecd1f96177",
  "hash": "f7cdbc7cf8b910d35cc69962e791138624e4eae7901010a6da4c02e7d238cdac",
  "version": 1,
  "size": 194,
  "vsize": 143,
  "weight": 569,
  "locktime": 0,
  "vin": [
    {
      "txid": "4ac541802679866935a19d4f40728bb89204d0cac90d85f3a51a19...aeb",
      "vout": 1,
      "scriptSig": {
        "asm": "",
        "hex": ""
      },
      "txinwitness": [
        "cf5efe2d8ef13ed0af21d4f4cb82422d6252d70324f6f4576b727b7d918e5...301"
      ],
```

```
      "sequence": 4294967295
    }
  ],
  "vout": [
    {
      "value": 0.00020000,
      "n": 0,
      "scriptPubKey": {
        "asm": "1 3b41daba4c9ace578369740f15e5ec880c28279ee7f51b07dca...068",
        "desc": "rawtr(3b41daba4c9ace578369740f15e5ec880c28279ee7f51b...6ev",
        "hex": "51203b41daba4c9ace578369740f15e5ec880c28279ee7f51b07d...068",
        "address": "bc1p8dqa4wjvnt890qmfws83te0v3qxzsfu7ul63kp7u56w8q...5qn",
        "type": "witness_v1_taproot"
      }
    },
    {
      "value": 0.00075000,
      "n": 1,
      "scriptPubKey": {
        "asm": "0 7752c165ea7be772b2c0acb7f4d6047ae6f4768e",
        "desc": "addr(bc1qwafvze0200nh9vkq4jmlf4sy0tn0ga5w0zpkpg)#qq404gts",
        "hex": "00147752c165ea7be772b2c0acb7f4d6047ae6f4768e",
        "address": "bc1qwafvze0200nh9vkq4jmlf4sy0tn0ga5w0zpkpg",
        "type": "witness_v0_keyhash"
      }
    }
  ]
}
```

The transaction decode shows all the components of this transaction, including the transaction inputs and outputs. In this case we see that the transaction used one input and generated two outputs. The input to this transaction was the output from a previously confirmed transaction (shown as the input txid). The two outputs correspond to the payment to Bob and the change back to Alice.

We can further explore the blockchain by examining the previous transaction referenced by its txid in this transaction using the same commands (e.g., getrawtransaction). Jumping from transaction to transaction, we can follow a chain of transactions back as the coins are transmitted from one owner to the next.

3.6.3. Exploring Blocks

Exploring blocks is similar to exploring transactions. However, blocks can be referenced either by the block *height* or by the block *hash*. First, let's find a block by its height. We use the getblockhash command, which takes the block height as the parameter and returns the block *header hash* for that block:

```
$ bitcoin-cli getblockhash 123456
000000000002917ed80650c6174aac8dfc46f5fe36480aaef682ff6cd83c3ca
```

Now that we know the header hash for our chosen block, we can query that block. We use the getblock command with the block hash as the parameter:

```
$ bitcoin-cli getblock 000000000002917ed80650c6174aac8dfc46f5fe36480aaef682f\
f6cd83c3ca
```


```
{
  "hash": "000000000002917ed80650c6174aac8dfc46f5fe36480aaef682ff6cd83c3ca",
  "confirmations": 651742,
  "height": 123456,
  "version": 1,
  "versionHex": "00000001",
  "merkleroot": "0e60651a9934e8f0decd1c[...]48fca0cd1c84a21ddfde95033762d86c",
  "time": 1305200806,
  "mediantime": 1305197900,
  "nonce": 2436437219,
  "bits": "1a6a93b3",
  "difficulty": 157416.4018436489,
  "chainwork": "[...]0000000000000000000000000000000000000000541788211ac227bc",
  "nTx": 13,
  "previousblockhash": "[...]60bc96a44724fd72daf9b92cf8ad00510b5224c6253ac40095",
  "nextblockhash": "[...]00129f5f02be247070bf7334d3753e4ddee502780c2acaecec6d66",
  "strippedsize": 4179,
  "size": 4179,
  "weight": 16716,
  "tx": [
    "5b75086dafeede555fc8f9a810d8b10df57c46f9f176ccc3dd8d2fa20edd685b",
    "e3d0425ab346dd5b76f44c222a4bb5d16640a4247050ef82462ab17e229c83b4",
    "137d247eca8b99dee58e1e9232014183a5c5a9e338001a0109df32794cdcc92e",
    "5fd167f7b8c417e59106ef5acfe181b09d71b8353a61a55a2f01aa266af5412d",
    "60925f1948b71f429d514ead7ae7391e0edf965bf5a60331398dae24c6964774",
    "d4d5fc1529487527e9873256934dfb1e4cdcb39f4c0509577ca19bfad6c5d28f",
    "7b29d65e5018c56a33652085dbb13f2df39a1a9942bfe1f7e78e97919a6bdea2",
    "0b89e120efd0a4674c127a76ff5f7590ca304e6a064fbc51adffbd7ce3a3deef",
    "603f2044da9656084174cfb5812feaf510f862d3addcf70cacce3dc55dab446e",
    "9a4ed892b43a4df916a7a1213b78e83cd83f5695f635d535c94b2b65ffb144d3",
    "dda726e3dad9504dce5098dfab5064ecd4a7650bfe854bb2606da3152b60e427",
    "e46ea8b4d68719b65ead930f07f1f3804cb3701014f8e6d76c4bdbc390893b94",
    "864a102aeedf53dd9b2baab4eeb898c5083fde6141113e0606b664c41fe15e1f"
  ]
}
```


The confirmations entry tells us the *depth* of this block—how many blocks have been built on top of it, indicating the difficulty of changing any of the transactions in this block. The height tells us how many blocks preceeded this block. We see the block's version, the time it was created (according to its miner), the median time of the 11 blocks that precede this block (a time measurement that's harder for miners to manipulate), and the size of the block in three different measurements (its legacy stripped size, its full size, and its size in weight units). We also see some fields used for security and proof of work (merkle root, nonce, bits, difficulty, and chainwork); we'll examine those in detail in Mining and Consensus.

3.6.4. Using Bitcoin Core's Programmatic Interface

The bitcoin-cli helper is very useful for exploring the Bitcoin Core API and testing functions. But the whole point of an API is to access functions programmatically. In this section we will demonstrate accessing Bitcoin Core from another program.

Bitcoin Core's API is a JSON-RPC interface. JSON is a very convenient way to present data that both humans and programs can easily read. RPC stands for remote procedure call, which means that we are calling procedures (functions) that are remote (on the Bitcoin Core node) via a network protocol. In this case, the network protocol is HTTP.

When we used the bitcoin-cli command to get help on a command, it showed us an example of using curl, the versatile command-line HTTP client to construct one of these JSON-RPC calls:

```
$ curl --user myusername --data-binary '{"jsonrpc": "1.0", "id":"curltest",
  "method": "getblockchaininfo",
  "params": [] }' -H 'content-type: text/plain;' http://127.0.0.1:8332/
```

This command shows that curl submits an HTTP request to the local host (127.0.0.1), connecting to the default Bitcoin RPC port (8332), and submitting a jsonrpc request for the getblockchaininfo method using text/plain encoding.

You might notice that curl will ask for credentials to be sent along with the request. Bitcoin Core will create a random password on each start and place it in the data directory under the name .cookie. The bitcoin-cli helper can read this password file given the data directory. Similarly, you can copy the password and pass it to curl (or any higher-level Bitcoin Core RPC wrappers), as seen in Using cookie-based authentication with Bitcoin Core.

Example 3. Using cookie-based authentication with Bitcoin Core

```
$ cat .bitcoin/.cookie
  __cookie__:17c9b71cef21b893e1a019f4bc071950c7942f49796ed061b274031b17b19cd0

$ curl
  --user __cookie__:17c9b71cef21b893e1a019f4bc071950c7942f49796ed061b274031b17b19cd0
  --data-binary '{"jsonrpc": "1.0", "id":"curltest",
  "method": "getblockchaininfo",
  "params": [] }' -H 'content-type: text/plain;' http://127.0.0.1:8332/

{"result":{"chain":"main","blocks":799278,"headers":799278,
"bestblockhash":"0000000000000000000018387c50988ec705a95d6f765b206b6629971e6978879",
"difficulty":53911173001054.59,"time":1689703111,"mediantime":1689701260,
"verificationprogress":0.9999979206082515,"initialblockdownload":false,
"chainwork":"0000000000000000000000000000000000000000004f3e111bf32bcb47f9dfad5b",
"size_on_disk":563894577967,"pruned":false,"warnings":""},"error":null,
"id":"curltest"}
```

Alternatively, you can create a static password with the helper script provided in *./share/rpcauth/rpcauth.py* in Bitcoin Core's source directory.

If you're implementing a JSON-RPC call in your own program, you can use a generic HTTP library to construct the call, similar to what is shown in the preceding curl example.

However, there are libraries in most popular programming languages that "wrap" the Bitcoin Core API in a way that makes this a lot simpler. We will use the python-bitcoinlib library to simplify API access. This library is not part of the Bitcoin Core project and needs to be installed the usual way you install Python libraries. Remember, this requires you to have a running Bitcoin Core instance, which will be used to make JSON-RPC calls.

The Python script in Running getblockchaininfo via Bitcoin Core's JSON-RPC API makes a simple getblockchaininfo call and prints the block parameter from the data returned by Bitcoin Core.

Example 4. Running getblockchaininfo via Bitcoin Core's JSON-RPC API

```
from bitcoin.rpc import RawProxy

# Create a connection to local Bitcoin Core node
p = RawProxy()

# Run the getblockchaininfo command, store the resulting data in info
info = p.getblockchaininfo()

# Retrieve the 'blocks' element from the info
print(info['blocks'])
```

Running it gives us the following result:

```
$ python rpc_example.py
773973
```

It tells us how many blocks our local Bitcoin Core node has in its blockchain. Not a spectacular result, but it demonstrates the basic use of the library as a simplified interface to Bitcoin Core's JSON-RPC API.

Next, let's use the getrawtransaction and decodetransaction calls to retrieve the details of Alice's payment to Bob. In Retrieving a transaction and iterating its outputs, we retrieve Alice's transaction and list the transaction's outputs. For each output, we show the recipient address and value. As a reminder, Alice's transaction had one output paying Bob and one output for change back to Alice.

Example 5. Retrieving a transaction and iterating its outputs

```
from bitcoin.rpc import RawProxy

p = RawProxy()

# Alice's transaction ID
txid = "466200308696215bbc949d5141a49a4138ecdfdfaa2a8029c1f9bcecd1f96177"

# First, retrieve the raw transaction in hex
raw_tx = p.getrawtransaction(txid)
```

```
# Decode the transaction hex into a JSON object
decoded_tx = p.decoderawtransaction(raw_tx)

# Retrieve each of the outputs from the transaction
for output in decoded_tx['vout']:
    print(output['scriptPubKey']['address'], output['value'])
```

Running this code, we get:

```
$ python rpc_transaction.py
bc1p8dqa4wjvnt890qmfws83te0v3qxzsfu7ul63kp7u56w8qc0qwp5qv995qn 0.00020000
bc1qwafvze0200nh9vkq4jmlf4sy0tn0ga5w0zpkpg 0.00075000
```

Both of the preceding examples are rather simple. You don't really need a program to run them; you could just as easily use the bitcoin-cli helper. The next example, however, requires several hundred RPC calls and more clearly demonstrates the use of a programmatic interface.

In Retrieving a block and adding all the transaction outputs, we first retrieve a block, then retrieve each of the transactions within it by reference to each transaction ID. Next, we iterate through each of the transaction's outputs and add up the value.

Example 6. Retrieving a block and adding all the transaction outputs

```
from bitcoin.rpc import RawProxy

p = RawProxy()

# The block height where Alice's transaction was recorded
blockheight = 775072

# Get the block hash of the block at the given height
blockhash = p.getblockhash(blockheight)

# Retrieve the block by its hash
block = p.getblock(blockhash)

# Element tx contains the list of all transaction IDs in the block
transactions = block['tx']

block_value = 0

# Iterate through each transaction ID in the block
for txid in transactions:
    tx_value = 0
    # Retrieve the raw transaction by ID
    raw_tx = p.getrawtransaction(txid)
    # Decode the transaction
    decoded_tx = p.decoderawtransaction(raw_tx)
    # Iterate through each output in the transaction
    for output in decoded_tx['vout']:
        # Add up the value of each output
        tx_value = tx_value + output['value']

    # Add the value of this transaction to the total
```

```
    block_value = block_value + tx_value

print("Total value in block: ", block_value)
```

Running this code, we get:

```
$ python rpc_block.py

Total value in block:  10322.07722534
```

Our example code calculates that the total value transacted in this block is 10,322.07722534 BTC (including 25 BTC reward and 0.0909 BTC in fees). Compare that to the amount reported by a block explorer site by searching for the block hash or height. Some block explorers report the total value excluding the reward and excluding the fees. See if you can spot the difference.

3.7. Alternative Clients, Libraries, and Toolkits

There are many alternative clients, libraries, toolkits, and even full-node implementations in the Bitcoin ecosystem. These are implemented in a variety of programming languages, offering programmers native interfaces in their preferred language.

The following sections list some of the best libraries, clients, and toolkits, organized by programming languages.

3.7.1. C/C++

Bitcoin Core
: The reference implementation of Bitcoin

3.7.2. JavaScript

bcoin
: A modular and scalable full-node implementation with API

Bitcore
: Full node, API, and library by Bitpay

BitcoinJS
: A pure JavaScript Bitcoin library for node.js and browsers

3.7.3. Java

bitcoinj
: A Java full-node client library

3.7.4. Python

python-bitcoinlib
: A Python bitcoin library, consensus library, and node by Peter Todd

pycoin
: A Python bitcoin library by Richard Kiss

3.7.5. Go

btcd
: A Go language, full-node Bitcoin client

3.7.6. Rust

rust-bitcoin
: Rust bitcoin library for serialization, parsing, and API calls

3.7.7. Scala

bitcoin-s
: A Bitcoin implementation in Scala

3.7.8. C#

NBitcoin
: Comprehensive bitcoin library for the .NET framework

Many more libraries exist in a variety of other programming languages, and more are created all the time.

If you followed the instructions in this chapter, you now have Bitcoin Core running and have begun exploring the network and blockchain using your own full node. From now on you can independently use software you control—on a computer you control—to verify that any bitcoins you receive follow every rule in the Bitcoin system without having to trust any outside authority. In the coming chapters, we'll learn more about the rules of the system and how your node and your wallet use them to secure your money, protect your privacy, and make spending and receiving convenient.

4. Keys and Addresses

Alice wants to pay Bob, but the thousands of Bitcoin full nodes who will verify her transaction don't know who Alice or Bob are—and we want to keep it that way to protect their privacy. Alice needs to communicate that Bob should receive some of her bitcoins without tying any aspect of that transaction to Bob's real-world identity or to other Bitcoin payments that Bob receives. The method Alice uses must ensure that only Bob can further spend the bitcoins he receives.

The original Bitcoin paper describes a very simple scheme for achieving those goals, shown in Transaction chain from original Bitcoin paper..

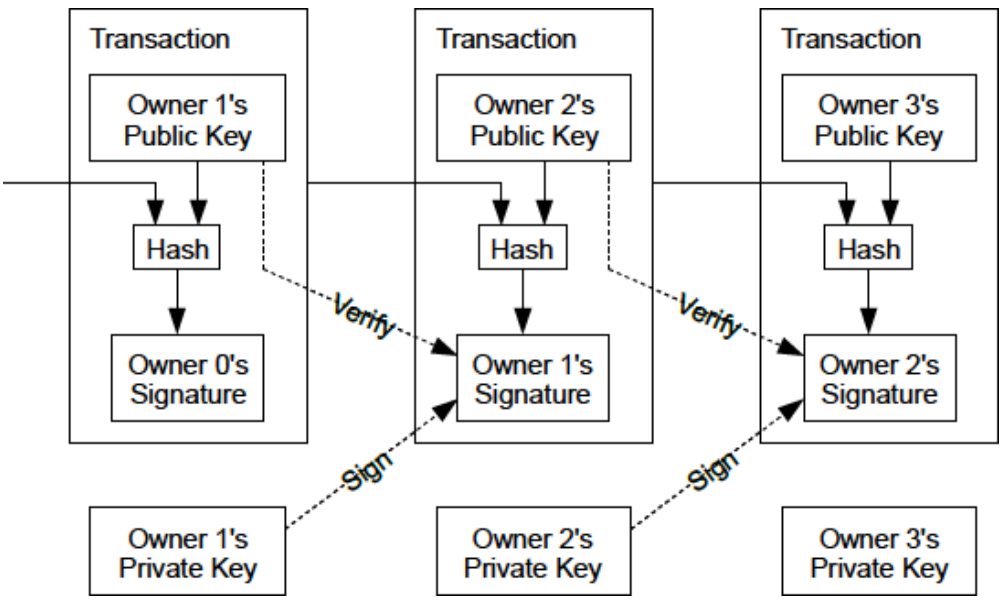


Figure 12. Transaction chain from original Bitcoin paper.

A receiver like Bob accepts bitcoins to a public key in a transaction that is signed by the spender (like Alice). The bitcoins that Alice is spending had been previously received to one of her public keys, and she uses the corresponding private key to generate her signature. Full nodes can verify that Alice's signature commits to the output of a hash function that itself commits to Bob's public key and other transaction details.

We'll examine public keys, private keys, signatures, and hash functions in this chapter, and then use all of them together to describe the addresses used by modern Bitcoin software.

4.1. Public Key Cryptography

Public key cryptography was invented in the 1970s and is a mathematical foundation for modern computer and information security.

Since the invention of public key cryptography, several suitable mathematical functions, such as prime number exponentiation and elliptic curve multiplication, have been discovered. These mathematical functions are easy to calculate in one direction and infeasible to calculate in the opposite direction using the computers and algorithms available today. Based on these mathematical functions, cryptography enables the creation of unforgeable digital signatures. Bitcoin uses elliptic curve addition and multiplication as the basis for its cryptography.

In Bitcoin, we can use public key cryptography to create a key pair that controls access to bitcoins. The key pair consists of a private key and a public key derived from the private key. The public key is used to receive funds, and the private key is used to sign transactions to spend the funds.

There is a mathematical relationship between the public and the private key that allows the private key to be used to generate signatures on messages. These signatures can be validated against the public key without revealing the private key.

> **Tip** In some wallet implementations, the private and public keys are stored together as a *key pair* for convenience. However, the public key can be calculated from the private key, so storing only the private key is also possible.

A Bitcoin wallet contains a collection of key pairs, each consisting of a private key and a public key. The private key (k) is a number, usually derived from a number picked at random. From the private key, we use elliptic curve multiplication, a one-way cryptographic function, to generate a public key (K).

Why Use Asymmetric Cryptography (Public/Private Keys)?
Why is asymmetric cryptography used in Bitcoin? It’s not used to "encrypt" (make secret) the transactions. Rather, a useful property of asymmetric cryptography is the ability to generate *digital signatures*. A private key can be applied to a transaction to produce a numerical signature. This signature can only be produced by someone with knowledge of the private key. However, anyone with access to the public key and the transaction can use them to *verify* the signature. This useful property of asymmetric cryptography makes it possible for anyone to verify every signature on every transaction, while ensuring that only the owners of private keys can produce valid signatures.

4.1.1. Private Keys

A private key is simply a number, picked at random. Control over the private key is the root of user control over all funds associated with the corresponding Bitcoin public key. The private key is used to create signatures that are used to spend bitcoins by proving control of funds used in a transaction. The private key must remain secret at all times because revealing it to third parties is equivalent to giving them control over the bitcoins secured by that key. The private key must also be backed up and protected from accidental loss because if it's lost, it cannot be recovered and the funds secured by it are forever lost too.

> **Tip** A Bitcoin private key is just a number. You can pick your private keys randomly using just a coin, pencil, and paper: toss a coin 256 times and you have the binary digits of a random private key you can use in a Bitcoin wallet. The public key can then be generated from the private key. Be careful, though, as any process that's less than completely random can significantly reduce the security of your private key and the bitcoins it controls.

The first and most important step in generating keys is to find a secure source of randomness (which computer scientists call *entropy*). Creating a Bitcoin key is almost the same as "Pick a number between 1 and 2^{256}." The exact method you use to pick that number does not matter as long as it is not predictable or repeatable. Bitcoin software uses cryptographically secure random number generators to produce 256 bits of entropy.

More precisely, the private key can be any number between 0 and n - 1 inclusive, where n is a constant ($n = 1.1578 \times 10^{77}$, slightly less than 2^{256}) defined as the order of the elliptic curve used in Bitcoin (see Elliptic Curve Cryptography Explained). To create such a key, we randomly pick a 256-bit number and check that it is less than n. In programming terms, this is usually achieved by feeding a larger string of random bits, collected from a cryptographically secure source of randomness, into the SHA256 hash algorithm, which will conveniently produce a 256-bit value that can be interpreted as a number. If the result is less than n, we have a suitable private key. Otherwise, we simply try again with another random number.

> **Warning** Do not write your own code to create a random number or use a "simple" random number generator offered by your programming language. Use a cryptographically secure pseudorandom number generator (CSPRNG) with a seed from a source of sufficient entropy. Study the documentation of the random number generator library you choose to make sure it is cryptographically secure. Correct implementation of the CSPRNG is critical to the security of the keys.

The following is a randomly generated private key (k) shown in hexadecimal format (256 bits shown as 64 hexadecimal digits, each 4 bits):

```
1E99423A4ED27608A15A2616A2B0E9E52CED330AC530EDCC32C8FFC6A526AEDD
```

> **Tip** The size of Bitcoin's private key space (2^{256}) is an unfathomably large number. It is approximately 10^{77} in decimal. For comparison, the visible universe is estimated to contain 10^{80} atoms.

4.1.2. Elliptic Curve Cryptography Explained

Elliptic curve cryptography (ECC) is a type of asymmetric or public key cryptography based on the discrete logarithm problem as expressed by addition and multiplication on the points of an elliptic curve.

An elliptic curve. is an example of an elliptic curve, similar to that used by Bitcoin.

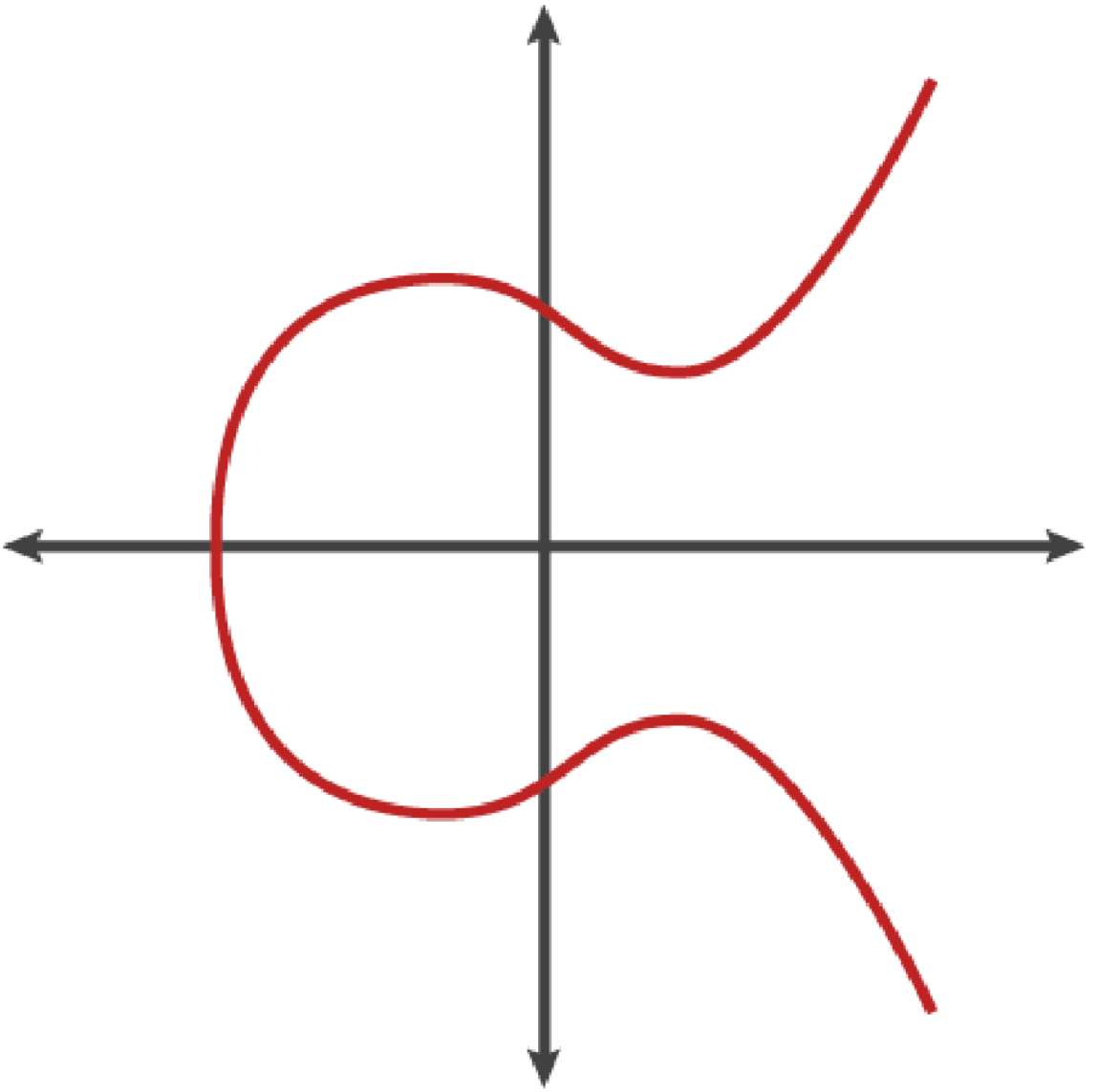


Figure 13. An elliptic curve.

Bitcoin uses a specific elliptic curve and set of mathematical constants, as defined in a standard called secp256k1, established by the National Institute of Standards and Technology (NIST). The secp256k1 curve is defined by the following function, which produces an elliptic curve:

$$y^2 = (x^3 + 7) \text{ over } (\mathbb{F}_p)$$

or

$$y^2 \mod p = (x^3 + 7) \mod p$$

The *mod p* (modulo prime number *p*) indicates that this curve is over a finite field of prime order *p*, also written as $\mathbb{F}_p$, where $p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$, a very large prime number.

Because this curve is defined over a finite field of prime order instead of over the real numbers, it looks like a pattern of dots scattered in two dimensions, which makes it difficult to visualize. However, the math is identical to that of an elliptic curve over real numbers. As an example, Elliptic curve cryptography: visualizing an elliptic curve over F(p), with p=17. shows the same elliptic curve over a much smaller finite field of prime order 17, showing a pattern of dots on a grid. The secp256k1 Bitcoin elliptic curve can be thought of as a much more complex pattern of dots on a unfathomably large grid.

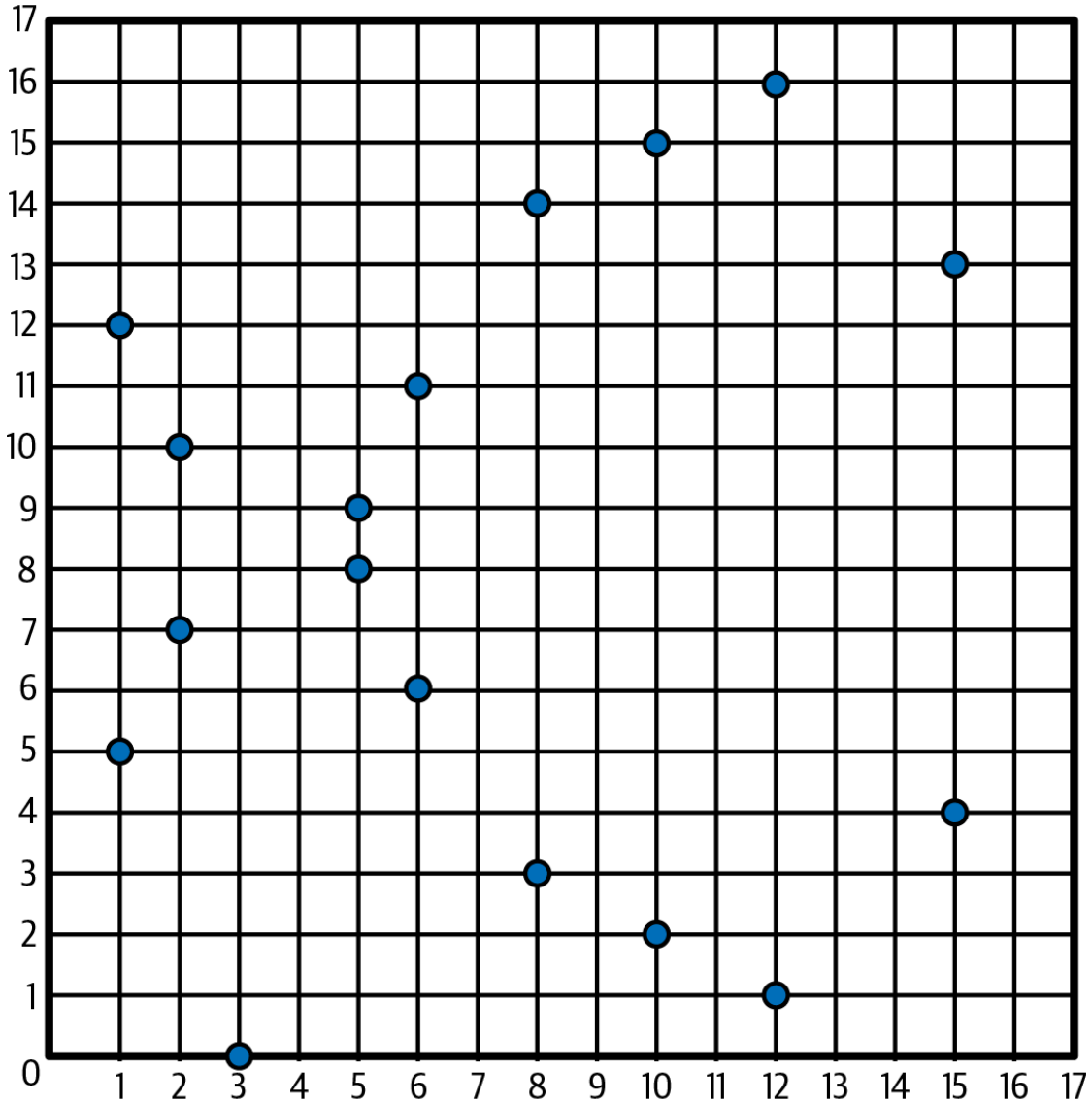


Figure 14. Elliptic curve cryptography: visualizing an elliptic curve over F(p), with p=17.

So, for example, the following is a point P with coordinates (x, y) that is a point on the secp256k1 curve:

```
P =
(55066263022277343669578718895168534326250603453777594175500187360389116729240,
32670510020758816978083085130507043184471273380659243275938904335757337482424)
```

Using Python to confirm that this point is on the elliptic curve shows how you can check this yourself using Python.

Example 7. Using Python to confirm that this point is on the elliptic curve

```
Python 3.10.6 (main, Nov 14 2022, 16:10:14) [GCC 11.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
> p = 115792089237316195423570985008687907853269984665640564039457584007908834671663
> x = 55066263022277343669578718895168534326250603453777594175500187360389116729240
> y = 32670510020758816978083085130507043184471273380659243275938904335757337482424
> (x ** 3 + 7 - y**2) % p
0
```

In elliptic curve math, there is a point called the "point at infinity," which roughly corresponds to the role of zero in addition. On computers, it's sometimes represented by x = y = 0 (which doesn't satisfy the elliptic curve equation, but it's an easy separate case that can be checked).

There is also a + operator, called "addition," which has some properties similar to the traditional addition of real numbers that gradeschool children learn. Given two points P_1 and P_2 on the elliptic curve, there is a third point $P_3 = P_1 + P_2$, also on the elliptic curve.

Geometrically, this third point P_3 is calculated by drawing a line between P_1 and P_2. This line will intersect the elliptic curve in exactly one additional place. Call this point P_3' = (x, y). Then reflect in the x-axis to get P_3 = (x, –y).

There are a couple of special cases that explain the need for the "point at infinity."

If P_1 and P_2 are the same point, the line "between" P_1 and P_2 should extend to be the tangent on the curve at this point P_1. This tangent will intersect the curve in exactly one new point. You can use techniques from calculus to determine the slope of the tangent line. These techniques curiously work, even though we are restricting our interest to points on the curve with two integer coordinates!

In some cases (i.e., if P_1 and P_2 have the same x values but different y values), the tangent line will be exactly vertical, in which case P_3 = "point at infinity."

If P_1 is the "point at infinity," then $P_1 + P_2 = P_2$. Similarly, if P_2 is the point at infinity, then $P_1 + P_2 = P_1$. This shows how the point at infinity plays the role of zero.

It turns out that + is associative, which means that (A + B) + C = A + (B + C). That means we can write A + B + C without parentheses and without ambiguity.

Now that we have defined addition, we can define multiplication in the standard way that extends addition. For a point P on the elliptic curve, if k is a whole number, then kP = P + P + P + … + P (k times). Note that k is sometimes confusingly called an "exponent" in this case.

4.1.3. Public Keys

The public key is calculated from the private key using elliptic curve multiplication, which is irreversible: $K = k \times G$, where k is the private key, G is a constant point called the *generator point*, and K is the resulting public key. The reverse operation, known as "finding the discrete logarithm"—calculating k if you know K—is as difficult as trying all possible values of k (i.e., a brute-force search). Before we demonstrate how to generate a public key from a private key, let's look at elliptic curve cryptography in a bit more detail.

> **Tip** Elliptic curve multiplication is a type of function that cryptographers call a "trap door" function: it is easy to do in one direction (multiplication) and impossible to do in the reverse direction (division). Someone with a private key can easily create the public key and then share it with the world knowing that no one can reverse the function and calculate the private key from the public key. This mathematical trick becomes the basis for unforgeable and secure digital signatures that prove control over bitcoin funds.

Starting with a private key in the form of a randomly generated number k, we multiply it by a predetermined point on the curve called the *generator point G* to produce another point somewhere else on the curve, which is the corresponding public key K. The generator point is specified as part of the secp256k1 standard and is always the same for all keys in bitcoin:

$$K = k \times G$$

where k is the private key, G is the generator point, and K is the resulting public key, a point on the curve. Because the generator point is always the same for all Bitcoin users, a private key k multiplied with G will always result in the same public key K. The relationship between k and K is fixed but can only be calculated in one direction, from k to K. That's why a Bitcoin public key (K) can be shared with anyone and does not reveal the user's private key (k).

> **Tip** A private key can be converted into a public key, but a public key cannot be converted back into a private key because the math only works one way.

Implementing the elliptic curve multiplication, we take the private key k generated previously and multiply it with the generator point G to find the public key K:

```
K = 1E99423A4ED27608A15A2616A2B0E9E52CED330AC530EDCC32C8FFC6A526AEDD × G
```

Public key K is defined as a point $K = (x, y)$:

$$K = (x, y)$$

where,

```
x = F028892BAD7ED57D2FB57BF33081D5CFCF6F9ED3D3D7F159C2E2FFF579DC341A
y = 07CF33DA18BD734C600B96A72BBC4749D5141C90EC8AC328AE52DDFE2E505BDB
```

To visualize multiplication of a point with an integer, we will use the simpler elliptic curve over real numbers—remember, the math is the same. Our goal is to find the multiple kG of the generator point G, which is the same as adding G to itself, k times in a row. In elliptic curves, adding a point to itself is the equivalent of drawing a tangent line on the point and finding where it intersects the curve again, then reflecting that point on the x-axis.

Elliptic curve cryptography: visualizing the multiplication of a point G by an integer k on an elliptic curve. shows the process for deriving G, $2G$, $4G$, as a geometric operation on the curve.

Tip Many Bitcoin implementations use the libsecp256k1 cryptographic library to do the elliptic curve math.

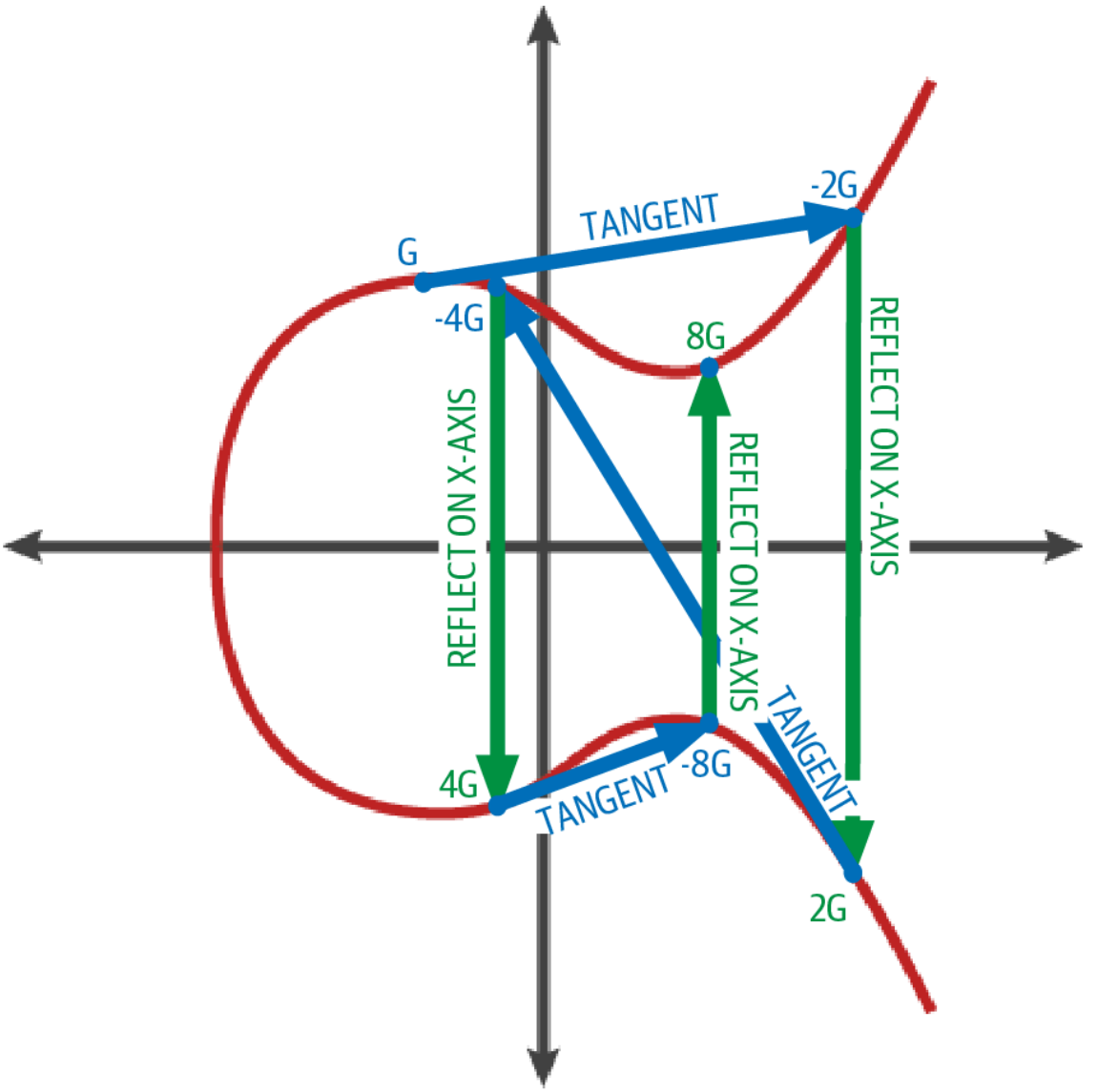


Figure 15. Elliptic curve cryptography: visualizing the multiplication of a point G by an integer k on an elliptic curve.

4.2. Output and Input Scripts

Although the illustration from the original Bitcoin paper, Transaction chain from original

Bitcoin paper., shows public keys (pubkeys) and signatures (sigs) being used directly, the first version of Bitcoin instead had payments sent to a field called *output script* and had spends of those bitcoins authorized by a field called *input script*. These fields allow additional operations to be performed in addition to (or instead of) verifying that a signature corresponds to a public key. For example, an output script can contain two public keys and require two corresponding signatures be placed in the spending input script.

Later, in Transaction Scripts and Script Language, we'll learn about scripts in detail. For now, all we need to understand is that bitcoins are received to an output script that acts like a public key, and bitcoin spending is authorized by an input script that acts like a signature.

4.3. IP Addresses: The Original Address for Bitcoin (P2PK)

We've established that Alice can pay Bob by assigning some of her bitcoins to one of Bob's public keys. But how does Alice get one of Bob's public keys? Bob could just give her a copy, but let's look again at the public key we worked with in Public Keys. Notice that it's quite long. Imagine Bob trying to read that to Alice over the phone:

```
x = F028892BAD7ED57D2FB57BF33081D5CFCF6F9ED3D3D7F159C2E2FFF579DC341A
y = 07CF33DA18BD734C600B96A72BBC4749D5141C90EC8AC328AE52DDFE2E505BDB
```

Instead of direct public key entry, the earliest version of Bitcoin software allowed a spender to enter the receiver's IP address, as shown in Early send screen for Bitcoin via The Internet Archive.. This feature was later removed—there are many problems with using IP addresses—but a quick description of it will help us better understand why certain features may have been added to the Bitcoin protocol.

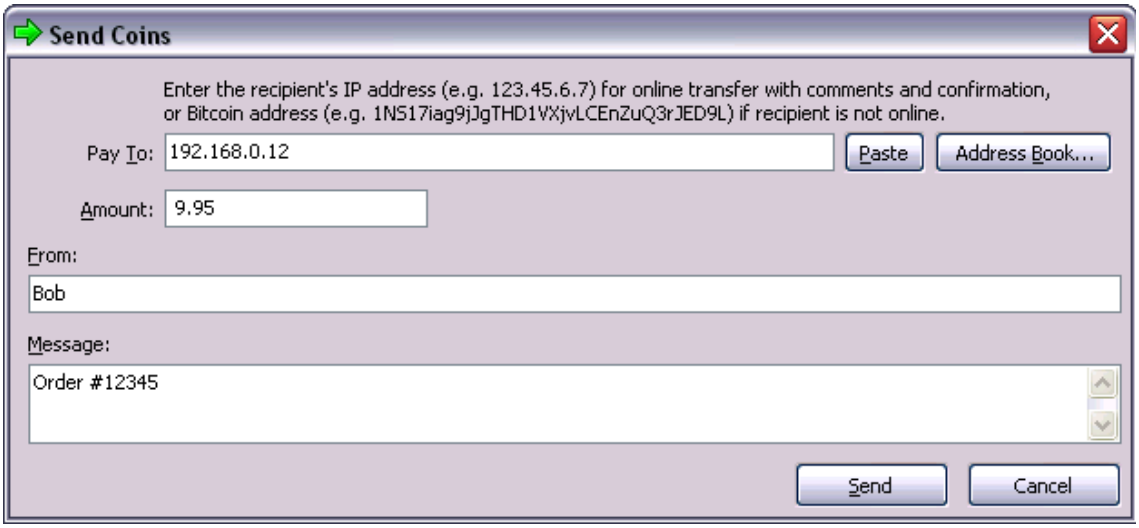


Figure 16. Early send screen for Bitcoin via The Internet Archive.

If Alice entered Bob's IP address in Bitcoin 0.1, her full node would establish a connection with his full node and receive a new public key from Bob's wallet that his node had never previously given anyone. This being a new public key was important to ensure that different transactions paying Bob couldn't be connected together by someone looking at the blockchain and noticing that all of the transactions paid the same public key.

Using the public key her node received from Bob's node, Alice's wallet would construct a transaction output paying a very simple output script:

```
<Bob's public key> OP_CHECKSIG
```

Bob would later be able to spend that output with an input script consisting entirely of his signature:

```
<Bob's signature>
```

To figure out what an output and input script are doing, you can combine them together (input script first) and then note that each piece of data (shown in angle brackets) is placed at the top of a list of items, called a stack. When an operation code (opcode) is encountered, it uses items from the stack, starting with the topmost items. Let's look at how that works by beginning with the combined script:

```
<Bob's signature> <Bob's public key> OP_CHECKSIG
```

For this script, Bob's signature is put on the stack, then Bob's public key is placed on top of it. The OP_CHECKSIG operation consumes two elements, starting with the public key and followed by the signature, removing them from the stack. It verifies the signature corresponds to the public key and also commits to (signs) the various fields in the

transaction. If the signature is correct, OP_CHECKSIG replaces itself on the stack with the value 1; if the signature was not correct, it replaces itself with a 0. If there's a nonzero item on top of the stack at the end of evaluation, the script passes. If all scripts in a transaction pass, and all of the other details about the transaction are valid, then full nodes will consider the transaction to be valid.

In short, the preceding script uses the same public key and signature described in the original paper but adds in the complexity of two script fields and an opcode. That seems like extra work here, but we'll begin to see the benefits when we look at the following section.

This type of output is known today as *pay to public key*, or *P2PK* for short. It was never widely used for payments, and no widely used program has supported IP address payments for almost a decade.

4.4. Legacy Addresses for P2PKH

Entering the IP address of the person you want to pay has a number of advantages, but it also has a number of downsides. One particular downside is that the receiver needs their wallet to be online at their IP address, and it needs to be accessible from the outside world. For a lot of people, that isn't an option. They turn their computers off at night, their laptops go to sleep, they're behind firewalls, or they're using Network Address Translation (NAT).

This brings us back to the problem of receivers like Bob having to give spenders like Alice a long public key. The shortest version of Bitcoin public keys known to the developers of early Bitcoin were 65 bytes, the equivalent of 130 characters when written in hexadecimal. However, Bitcoin already contains several data structures much larger than 65 bytes that need to be securely referenced in other parts of Bitcoin using the smallest amount of data that was secure.

Bitcoin accomplishes that with a *hash function*, a function that takes a potentially large amount of data, scrambles it (hashes it), and outputs a fixed amount of data. A cryptographic hash function will always produce the same output when given the same input, and a secure function will also make it impractical for somebody to choose a different input that produces a previously-seen output. That makes the output a *commitment* to the input. It's a promise that, in practice, only input x will produce output X.

For example, imagine I want to ask you a question and also give you my answer in a form that you can't read immediately. Let's say the question is, "in what year did Satoshi Nakamoto start working on Bitcoin?" I'll give you a commitment to my answer in the form of output from the SHA256 hash function, the function most commonly used in Bitcoin:

```
94d7a772612c8f2f2ec609d41f5bd3d04a5aa1dfe3582f04af517d396a302e4e
```

Later, after you tell me your guess to the answer of the question, I can reveal my answer and prove to you that my answer, as input to the hash function, produces exactly the same output I gave you earlier:

```
$ echo "2007.  He said about a year and a half before Oct 2008" | sha256sum
94d7a772612c8f2f2ec609d41f5bd3d04a5aa1dfe3582f04af517d396a302e4e
```

Now imagine that we ask Bob the question, "what is your public key?" Bob can use a hash function to give us a cryptographically secure commitment to his public key. If he later reveals his key, and we verify it produces the same commitment he previously gave us, we can be sure it was the exact same key that was used to create that earlier commitment.

The SHA256 hash function is considered to be very secure and produces 256 bits (32 bytes) of output, less than half the size of original Bitcoin public keys. However, there are other slightly less secure hash functions that produce smaller output, such as the RIPEMD-160 hash function whose output is 160 bits (20 bytes). For reasons Satoshi Nakamoto never stated, the original version of Bitcoin made commitments to public keys by first hashing the key with SHA256 and then hashing that output with RIPEMD-160; this produced a 20-byte commitment to the public key.

We can look at that algorithmically. Starting with the public key K, we compute the SHA256 hash and then compute the RIPEMD-160 hash of the result, producing a 160-bit (20-byte) number:

$$A = RIPEMD160(SHA256(K))$$

where K is the public key and A is the resulting commitment.

Now that we understand how to make a commitment to a public key, we need to figure out how to use it in a transaction. Consider the following output script:

```
OP_DUP OP_HASH160 <Bob's commitment> OP_EQUAL OP_CHECKSIG
```

And also the following input script:

```
<Bob's signature> <Bob's public key>
```

Together, they form the following script:

```
<sig> <pubkey> OP_DUP OP_HASH160 <commitment> OP_EQUALVERIFY OP_CHECKSIG
```

As we did in IP Addresses: The Original Address for Bitcoin (P2PK), we start putting items on the stack. Bob’s signature goes on first; his public key is then placed on top of the stack. The OP_DUP operation duplicates the top item, so the top and second-to-top item on the stack are now both Bob’s public key. The OP_HASH160 operation consumes (removes) the top public key and replaces it with the result of hashing it with RIPEMD160(SHA256(K)), so now the top of the stack is a hash of Bob’s public key. Next, the commitment to Bob’s

public key is added to the top of the stack. The OP_EQUALVERIFY operation consumes the top two items and verifies that they are equal; that should be the case if the public key Bob provided in the input script is the same public key used to create the commitment in the output script that Alice paid. If OP_EQUALVERIFY fails, the whole script fails. Finally, we're left with a stack containing just Bob's signature and his public key; the OP_CHECKSIG opcode verifies they correspond with each other and that the signature commits to the transaction.

Although this process of paying to a public key hash (*P2PKH*) may seem convoluted, it allows Alice's payment to Bob to contain only a 20 byte commitment to his public key instead of the key itself, which would've been 65 bytes in the original version of Bitcoin. That's a lot less data for Bob to have to communicate to Alice.

However, we haven't yet discussed how Bob gets those 20 bytes from his Bitcoin wallet to Alice's wallet. There are commonly used encodings for byte values, such as hexadecimal, but any mistake made in copying a commitment would result in the bitcoins being sent to an unspendable output, causing them to be lost forever. In the next section, we'll look at compact encoding and reliable checksums.

4.5. Base58check Encoding

In order to represent long numbers in a compact way, using fewer symbols, many computer systems use mixed-alphanumeric representations with a base (or radix) higher than 10. For example, whereas the traditional decimal system uses 10 numerals, 0 through 9, the hexadecimal system uses 16, with the letters A through F as the six additional symbols. A number represented in hexadecimal format is shorter than the equivalent decimal representation. Even more compact, base64 representation uses 26 lowercase letters, 26 capital letters, 10 numerals, and 2 more characters such as "+" and "/" to transmit binary data over text-based media such as email.

Base58 is a similar encoding to base64, using upper- and lowercase letters and numbers, but omitting some characters that are frequently mistaken for one another and can appear identical when displayed in certain fonts. Specifically, base58 is base64 without the 0 (number zero), O (capital o), l (lower L), I (capital i), and the symbols "+" and "/." Or, more simply, it is a set of lowercase and capital letters and numbers without the four (0, O, l, I) just mentioned. Bitcoin's base58 alphabet shows the full base58 alphabet.

Example 8. Bitcoin's base58 alphabet

```
123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz
```

To add extra security against typos or transcription errors, base58check includes a *checksum* encoded in the base58 alphabet. The checksum is an additional four bytes added to the end of the data that is being encoded. The checksum is derived from the hash of the encoded data and can therefore be used to detect transcription and typing errors. When presented with base58check code, the decoding software will calculate the checksum of

the data and compare it to the checksum included in the code. If the two do not match, an error has been introduced and the base58check data is invalid. This prevents a mistyped Bitcoin address from being accepted by the wallet software as a valid destination, an error that would otherwise result in loss of funds.

To convert data (a number) into a base58check format, we first add a prefix to the data, called the "version byte," which serves to easily identify the type of data that is encoded. For example, the prefix zero (0x00 in hex) indicates that the data should be used as the commitment (hash) in a legacy P2PKH output script. A list of common version prefixes is shown in [base58check_versions].

Next, we compute the "double-SHA" checksum, meaning we apply the SHA256 hash-algorithm twice on the previous result (the prefix concatenated with the data):

```
checksum = SHA256(SHA256(prefix||data))
```

From the resulting 32-byte hash (hash-of-a-hash), we take only the first four bytes. These four bytes serve as the error-checking code, or checksum. The checksum is appended to the end.

The result is composed of three items: a prefix, the data, and a checksum. This result is encoded using the base58 alphabet described previously. Base58check encoding: a base58, versioned, and checksummed format for unambiguously encoding bitcoin data. illustrates the base58check encoding process.

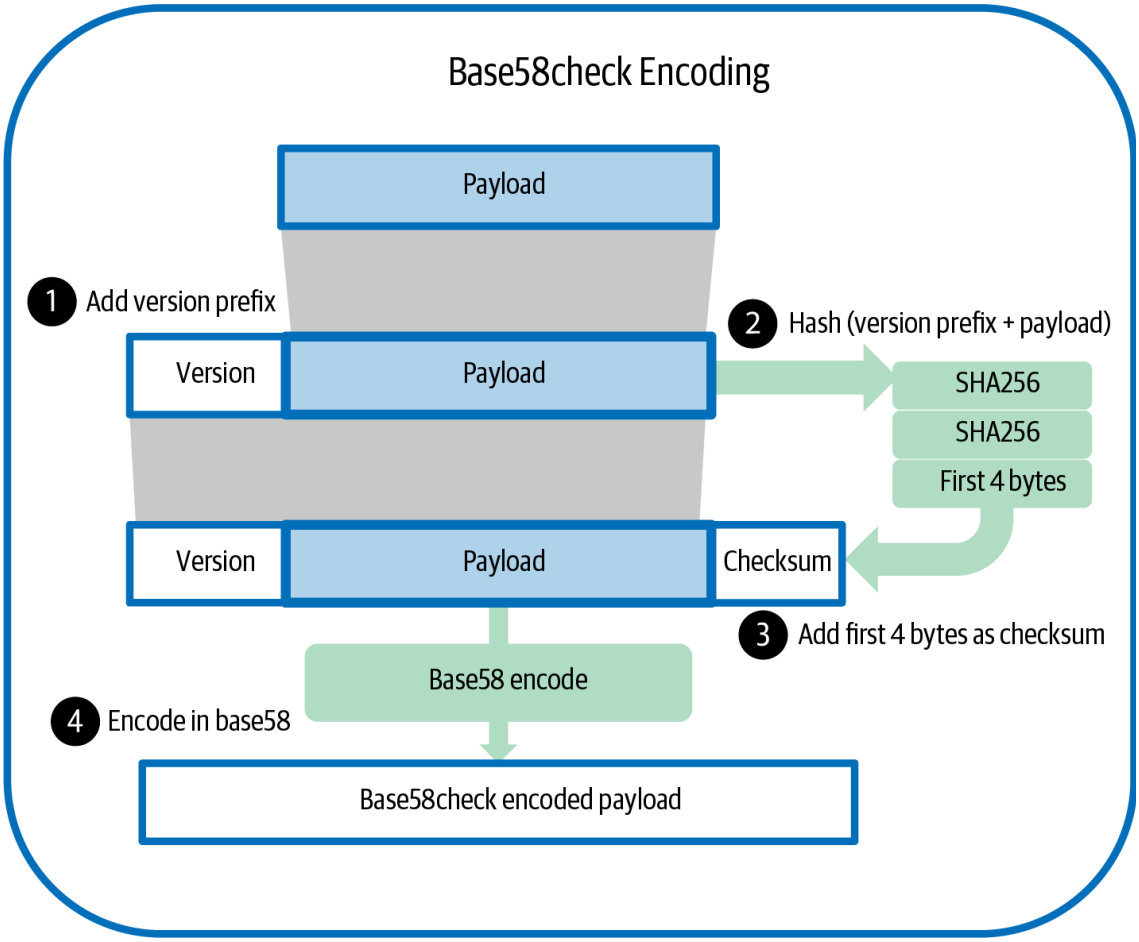


Figure 17. Base58check encoding: a base58, versioned, and checksummed format for unambiguously encoding bitcoin data.

In Bitcoin, other data besides public key commitments are presented to the user in base58check encoding to make that data compact, easy to read, and easy to detect errors. The version prefix in base58check encoding is used to create easily distinguishable formats, which when encoded in base58 contain specific characters at the beginning of the base58check-encoded payload. These characters make it easy for humans to identify the type of data that is encoded and how to use it. This is what differentiates, for example, a base58check-encoded Bitcoin address that starts with a 1 from a base58check-encoded private key wallet import format (WIF) that starts with a 5. Some example version prefixes and the resulting base58 characters are shown in #base58check_versions.

Base58check version prefix and encoded result examples

Type	Version prefix (hex)	Base58 result prefix

Type	Version prefix (hex)	Base58 result prefix
Address for pay to public key hash (P2PKH)	0x00	1
Address for pay to script hash (P2SH)	0x05	3
Testnet Address for P2PKH	0x6F	m or n
Testnet Address for P2SH	0xC4	2
Private Key WIF	0x80	5, K, or L
BIP32 Extended Public Key	0x0488B21E	xpub

Combining public keys, hash-based commitments, and base58check encoding, Public key to Bitcoin address: conversion of a public key to a Bitcoin address. illustrates the conversion of a public key into a Bitcoin address.

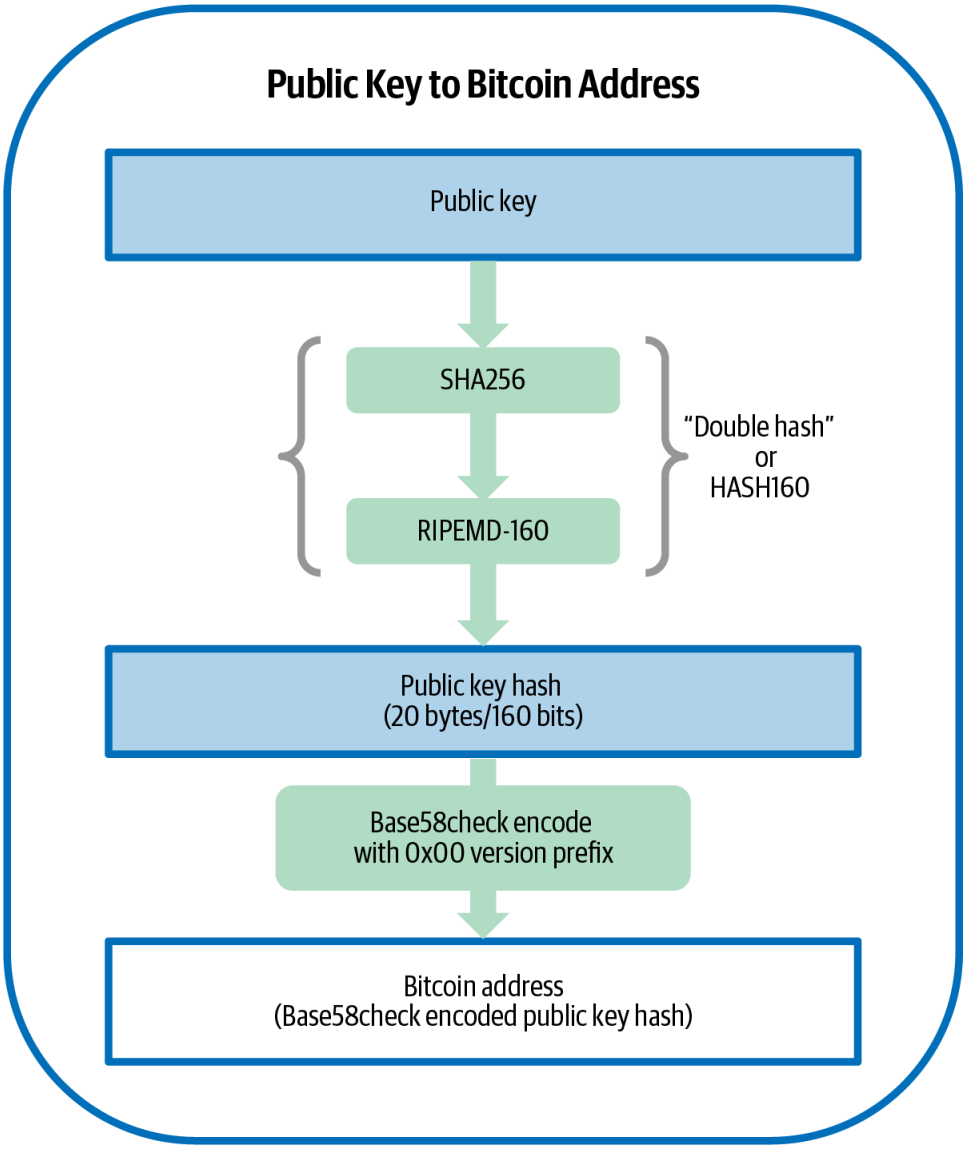


Figure 18. Public key to Bitcoin address: conversion of a public key to a Bitcoin address.

4.6. Compressed Public Keys

When Bitcoin was first authored, its developers only knew how to create 65-byte public keys. However, a later developer became aware of an alternative encoding for public keys that used only 33 bytes and which was backward compatible with all Bitcoin full nodes at the time, so there was no need to change the Bitcoin protocol. Those 33-byte public keys

are known as *compressed public keys*, and the original 65-byte keys are known as *uncompressed public keys*. Using smaller public keys results in smaller transactions, allowing more payments to be made in the same block.

As we saw in the section Public Keys, a public key is a point (x, y) on an elliptic curve. Because the curve expresses a mathematical function, a point on the curve represents a solution to the equation and, therefore, if we know the *x* coordinate, we can calculate the *y* coordinate by solving the equation $y^2 \bmod p = (x^3 + 7) \bmod p$. That allows us to store only the *x* coordinate of the public key point, omitting the *y* coordinate and reducing the size of the key and the space required to store it by 256 bits. An almost 50% reduction in size in every transaction adds up to a lot of data saved over time!

Here is the public key generated by the private key we created in Public Keys:

```
x = F028892BAD7ED57D2FB57BF33081D5CFCF6F9ED3D3D7F159C2E2FFF579DC341A
y = 07CF33DA18BD734C600B96A72BBC4749D5141C90EC8AC328AE52DDFE2E505BDB
```

Here's the same public key shown as a 520-bit number (130 hex digits) with the prefix 04 followed by x and then y coordinates, as 04 x y:

```
K = 04F028892BAD7ED57D2FB57BF33081D5CFCF6F9ED3D3D7F159C2E2FFF579DC341A\
    07CF33DA18BD734C600B96A72BBC4749D5141C90EC8AC328AE52DDFE2E505BDB
```

Whereas uncompressed public keys have a prefix of 04, compressed public keys start with either a 02 or a 03 prefix. Let's look at why there are two possible prefixes: because the left side of the equation is y^2, the solution for *y* is a square root, which can have a positive or negative value. Visually, this means that the resulting *y* coordinate can be above or below the x-axis. As you can see from the graph of the elliptic curve in An elliptic curve., the curve is symmetric, meaning it is reflected like a mirror by the x-axis. So, while we can omit the *y* coordinate, we have to store the *sign* of *y* (positive or negative); in other words, we have to remember if it was above or below the x-axis because each of those options represents a different point and a different public key. When calculating the elliptic curve in binary arithmetic on the finite field of prime order p, the *y* coordinate is either even or odd, which corresponds to the positive/negative sign as explained earlier. Therefore, to distinguish between the two possible values of *y*, we store a compressed public key with the prefix 02 if the *y* is even, and 03 if it is odd, allowing the software to correctly deduce the *y* coordinate from the *x* coordinate and uncompress the public key to the full coordinates of the point. Public key compression is illustrated in Public key compression..

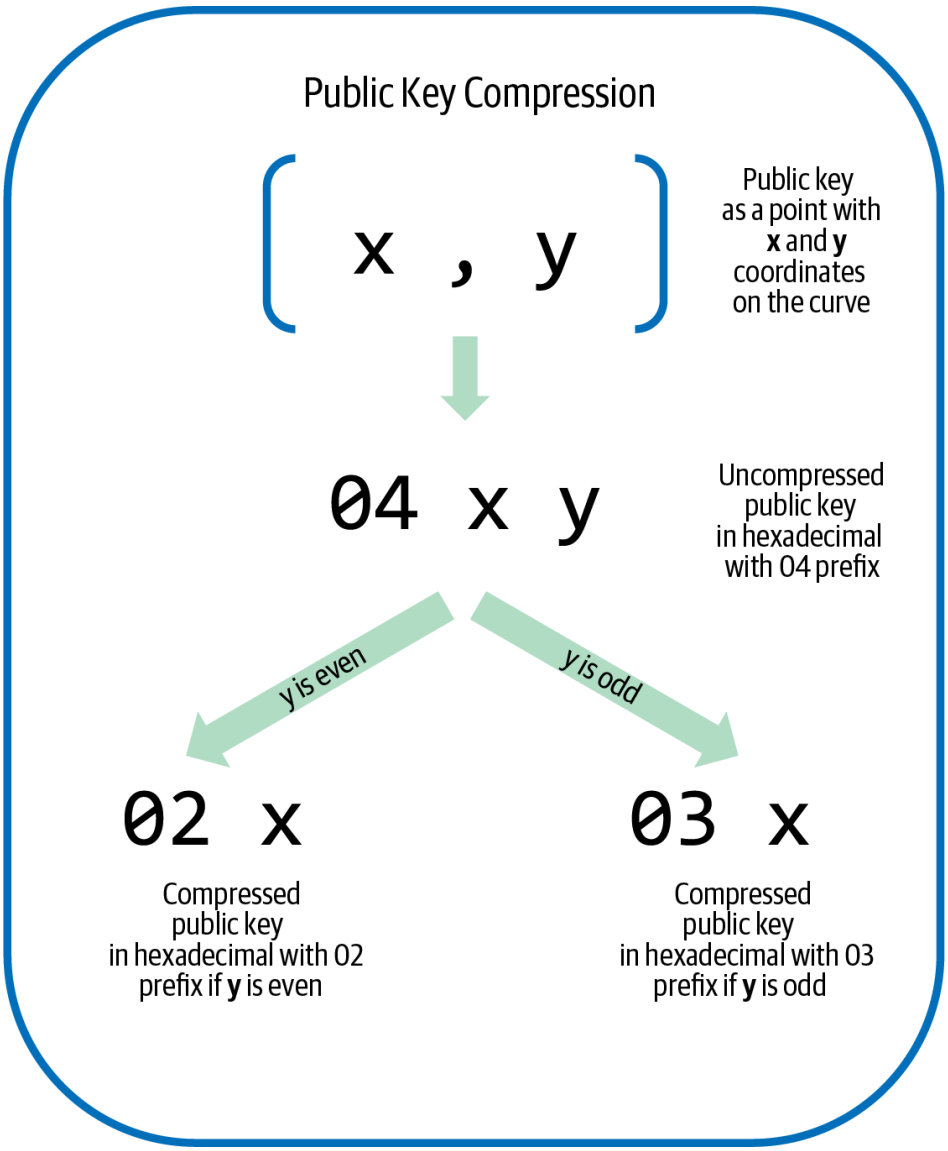


Figure 19. Public key compression.

Here's the same public key generated in Public Keys, shown as a compressed public key stored in 264 bits (66 hex digits) with the prefix 03 indicating the *y* coordinate is odd:

```
K = 03F028892BAD7ED57D2FB57BF33081D5CFCF6F9ED3D3D7F159C2E2FFF579DC341A
```

This compressed public key corresponds to the same private key, meaning it is generated from the same private key. However, it looks different from the uncompressed public key. More importantly, if we convert this compressed public key to a commitment using the HASH160 function (RIPEMD160(SHA256(K))), it will produce a *different* commitment than

the uncompressed public key, leading to a different address. This can be confusing because it means that a single private key can produce a public key expressed in two different formats (compressed and uncompressed) that produce two different Bitcoin addresses. However, the private key is identical for both Bitcoin addresses.

Compressed public keys are now the default in almost all Bitcoin software and were required when using certain new features added in later protocol upgrades.

However, some software still needs to support uncompressed public keys, such as a wallet application importing private keys from an older wallet. When the new wallet scans the blockchain for old P2PKH outputs and inputs, it needs to know whether to scan the 65-byte keys (and commitments to those keys) or 33-byte keys (and their commitments). Failure to scan for the correct type can lead to the user not being able to spend their full balance. To resolve this issue, when private keys are exported from a wallet, the WIF that is used to represent them is implemented slightly differently in newer Bitcoin wallets to indicate that these private keys have been used to produce compressed public keys.

4.7. Legacy Pay to Script Hash (P2SH)

As we've seen in preceding sections, someone receiving bitcoins (like Bob) can require that payments to him contain certain constraints in their output script. Bob will need to fulfill those constraints using an input script when he spends those bitcoins. In IP Addresses: The Original Address for Bitcoin (P2PK), the constraint was simply that the input script needed to provide an appropriate signature. In Legacy Addresses for P2PKH, an appropriate public key also needed to be provided.

For a spender (like Alice) to place the constraints Bob wants in the output script she uses to pay him, Bob needs to communicate those constraints to her. This is similar to the problem of Bob needing to communicate his public key to her. Like that problem, where public keys can be fairly large, the constraints Bob uses can also be quite large—potentially thousands of bytes. That's not only thousands of bytes that need to be communicated to Alice, but thousands of bytes for which she needs to pay transaction fees every time she wants to spend money to Bob. However, the solution of using hash functions to create small commitments to large amounts of data also applies here.

The BIP16 upgrade to the Bitcoin protocol in 2012 allows an output script to commit to a *redemption script* (*redeem script*). When Bob spends his bitcoins, his input script needs to provide a redeem script that matches the commitment and also any data necessary to satisfy the redeem script (such as signatures). Let's start by imagining Bob wants to require two signatures to spend his bitcoins, one signature from his desktop wallet and one from a hardware signing device. He puts those conditions into a redeem script:

```
<public key 1> OP_CHECKSIGVERIFY <public key 2> OP_CHECKSIG
```

He then creates a commitment to the redeem script using the same HASH160 mechanism used for P2PKH commitments, RIPEMD160(SHA256(script)). That commitment is placed into the output script using a special template:

```
OP_HASH160 <commitment> OP_EQUAL
```

> **Warning** When using pay to script hash (P2SH), you must use the specific P2SH template with no extra data or conditions in the output script. If the output script is not exactly OP_HASH160 <20 bytes> OP_EQUAL, the redeem script will not be used and any bitcoins may either be unspendable or spendable by anyone (meaning anyone can take them).

When Bob goes to spend the payment he received to the commitment for his script, he uses an input script that includes the redeem script, with it serialized as a single data element. He also provides the signatures he needs to satisfy the redeem script, putting them in the order that they will be consumed by the opcodes:

```
<signature2> <signature1> <redeem script>
```

When Bitcoin full nodes receive Bob's spend, they'll verify that the serialized redeem script will hash to the same value as the commitment. Then they'll replace it on the stack with its deserialized value:

```
<signature2> <signature1> <pubkey1> OP_CHECKSIGVERIFY <pubkey2> OP_CHECKSIG
```

The script is executed and, if it passes and all of the other transaction details are correct, the transaction is valid.

Addresses for P2SH are also created with base58check. The version prefix is set to 5, which results in an encoded address starting with a 3. An example of a P2SH address is 3F6i6kwkevjR7AsAd4te2YB2zZyASEm1HM.

> **Tip** P2SH is not necessarily the same as a multisignature transaction. A P2SH address *most often* represents a multisignature script, but it might also represent a script encoding other types of transactions.

P2PKH and P2SH are the only two script templates used with base58check encoding. They are now known as legacy addresses and have become less common over time. Legacy addresses were supplanted by the bech32 family of addresses.

P2SH Collision Attacks
All addresses based on hash functions are theoretically vulnerable to an attacker independently finding the same input that produced the hash function output (commitment). In the case of Bitcoin, if they find the input the same way the original user

did, they'll know the user's private key and be able to spend that user's bitcoins. The chance of an attacker independently generating the input for an existing commitment is proportional to the strength of the hash algorithm. For a secure 160-bit algorithm like HASH160, the probability is 1-in-2^{160}. This is a *preimage attack*.

An attacker can also try to generate two different inputs (e.g., redeem scripts) that produce the same commitment. For addresses created entirely by a single party, the chance of an attacker generating a different input for an existing commitment is also about 1-in-2^{160} for the HASH160 algorithm. This is a *second preimage attack*.

However, this changes when an attacker is able to influence the original input value. For example, an attacker participates in the creation of a multisignature script where they don't need to submit their public key until after they learn all of the other partys' public keys. In that case, the strength of hash algorithm is reduced to its square root. For HASH160, the probability becomes 1-in-2^{80}. This is a *collision attack*.

To put those numbers in context, as of early 2023, all Bitcoin miners combined execute about 2^{80} hash functions every hour. They run a different hash function than HASH160, so their existing hardware can't create collision attacks for it, but the existence of the Bitcoin network proves that collision attacks against 160-bit functions like HASH160 are practical. Bitcoin miners have spent the equivalent of billions of US dollars on special hardware, so creating a collision attack wouldn't be cheap, but there are organizations that expect to receive billions of dollars in bitcoins to addresses generated by processes involving multiple parties, which could make the attack profitable.

There are well-established cryptographic protocols for preventing collision attacks, but a simple solution that doesn't require any special knowledge on the part of wallet developers is to simply use a stronger hash function. Later upgrades to Bitcoin made that possible, and newer Bitcoin addresses provide at least 128 bits of collision resistance. To perform 2^{128} hash operations would take all current Bitcoin miners about 32 billion years.

Although we do not believe there is any immediate threat to anyone creating new P2SH addresses, we recommend all new wallets use newer types of addresses to eliminate address collision attacks as a concern.

4.8. Bech32 Addresses

In 2017, the Bitcoin protocol was upgraded. When the upgrade is used, it prevents transaction identifiers (txids) from being changed without the consent of a spending user (or a quorum of signers when multiple signatures are required). The upgrade, called *segregated witness* (or *segwit* for short), also provided additional capacity for transaction data in blocks and several other benefits. However, users wanting direct access to segwit's benefits had to accept payments to new output scripts.

As mentioned in Pay to Script Hash, one of the advantages of the P2SH output type was that a spender (such as Alice) didn't need to know the details of the script the receiver (such as Bob) used. The segwit upgrade was designed to use this mechanism, allowing users to immediately begin accessing many of the new benefits by using a P2SH address. But for Bob to gain access to all of the benefits, he would need Alice's wallet to pay him using a different type of script. That would require Alice's wallet to upgrade to support the new scripts.

At first, Bitcoin developers proposed BIP142, which would continue using base58check with a new version byte, similar to the P2SH upgrade. But getting all wallets to upgrade to new scripts with a new base58check version was expected to require almost as much work as getting them to upgrade to an entirely new address format, so several Bitcoin contributors set out to design the best possible address format. They identified several problems with base58check:

- Its mixed-case presentation made it inconvenient to read aloud or transcribe. Try reading one of the legacy addresses in this chapter to a friend who you have transcribe it. Notice how you have to prefix every letter with the words "uppercase" and "lowercase." Also, note when you review their writing that the uppercase and lowercase versions of some letters can look similar in many people's handwriting.
- It can detect errors, but it can't help users correct those errors. For example, if you accidentally transpose two characters when manually entering an address, your wallet will almost certainly warn that a mistake exists, but it won't help you figure out where the error is located. It might take you several frustrating minutes to eventually discover the mistake.
- A mixed-case alphabet also requires extra space to encode in QR codes, which are commonly used to share addresses and invoices between wallets. That extra space means QR codes need to be larger at the same resolution or they become harder to scan quickly.
- It requires every spender wallet upgrade to support new protocol features like P2SH and segwit. Although the upgrades themselves might not require much code, experience shows that many wallet authors are busy with other work and can sometimes delay upgrading for years. This adversely affects everyone who wants to use the new features.

The developers working on an address format for segwit found solutions for each of these problems in a new address format called bech32 (pronounced with a soft "ch", as in "besh thirty-two"). The "bech" stands for BCH, the initials of the three individuals who discovered the cyclic code in 1959 and 1960 upon which bech32 is based. The "32" stands for the number of characters in the bech32 alphabet (similar to the 58 in base58check):

- Bech32 uses only numbers and a single case of letters (preferably rendered in lowercase). Despite its alphabet being almost half the size of the base58check alphabet, a bech32 address for a pay to witness public key hash (P2WPKH) script is

only slightly longer than a legacy address for an equivalent P2PKH script.

- Bech32 can both detect and help correct errors. In an address of an expected length, it is mathematically guaranteed to detect any error affecting four characters or less; that's more reliable than base58check. For longer errors, it will fail to detect them less than one time in a billion, which is roughly the same reliability as base58check. Even better, for an address typed with just a few errors, it can tell the user where those errors occurred, allowing them to quickly correct minor transcription mistakes. See Bech32 typo detection for an example of an address entered with errors.

  Example 9. Bech32 typo detection
  Address: bc1p9nh05ha8wrljf7ru236aw**n**4t2x0d5ctkkywm**v**9sclnm4t0av2vgs4k3au7

  Detected errors shown in bold and underlined. Generated using the bech32 address decoder demo.

- Bech32 is preferably written with only lowercase characters, but those lowercase characters can be replaced with uppercase characters before encoding an address in a QR code. This allows the use of a special QR encoding mode that uses less space. Notice the difference in size and complexity of the two QR codes for the same address in The same bech32 address QR encoded in lowercase and uppercase..



Figure 20. The same bech32 address QR encoded in lowercase and uppercase.

- Bech32 takes advantage of an upgrade mechanism designed as part of segwit to make it possible for spender wallets to be able to pay output types that aren't in use yet. The goal was to allow developers to build a wallet today that allows spending to a bech32 address and have that wallet remain able to spend to bech32 addresses for users of new features added in future protocol upgrades. It was hoped that we might never again need to go through the system-wide upgrade cycles necessary to allow people to fully use P2SH and segwit.

4.8.1. Problems with Bech32 Addresses

Bech32 addresses would have been a success in every area except for one problem. The mathematical guarantees about their ability to detect errors only apply if the length of the address you enter into a wallet is the same length of the original address. If you add or remove any characters during transcription, the guarantee doesn't apply and your wallet may spend funds to a wrong address. However, even without the guarantee, it was thought that it would be very unlikely that a user adding or removing characters would produce a string with a valid checksum, ensuring users' funds were safe.

Unfortunately, the choice for one of the constants in the bech32 algorithm just happened to make it very easy to add or remove the letter "q" in the penultimate position of an address that ends with the letter "p." In those cases, you can also add or remove the letter "q" multiple times. This will be caught by the checksum some of the time, but it will be missed far more often than the one-in-a-billion expectations for bech32's substitution errors. For an example, see Extending the length of bech32 address without invalidating its checksum.

Example 10. Extending the length of bech32 address without invalidating its checksum

```
Intended bech32 address:
bc1pqqqsq9txsqp

Incorrect addresses with a valid checksum:
bc1pqqqsq9txsqqqqp
bc1pqqqsq9txsqqqqqqp
bc1pqqqsq9txsqqqqqqqqp
bc1pqqqsq9txsqqqqqqqqqp
bc1pqqqsq9txsqqqqqqqqqqp
```

For the initial version of segwit (version 0), this wasn't a practical concern. Only two valid lengths were defined for v0 segwit outputs: 22 bytes and 34 bytes. Those correspond to bech32 addresses that are 42 characters or 62 characters long, so someone would need to add or remove the letter "q" from the penultimate position of a bech32 address 20 times in order to send money to an invalid address without a wallet being able to detect it. However, it would become a problem for users in the future if a segwit-based upgrade were ever to be implemented.

4.8.2. Bech32m

Although bech32 worked well for segwit v0, developers didn't want to unnecessarily constrain output sizes in later versions of segwit. Without constraints, adding or removing a single "q" in a bech32 address could result in a user accidentally sending their money to an output that was either unspendable or spendable by anyone (allowing those bitcoins to be taken by anyone). Developers exhaustively analyzed the bech32 problem and found that changing a single constant in their algorithm would eliminate the problem, ensuring that any insertion or deletion of up to five characters will only fail to be detected less often than one time in a billion.

The version of bech32 with a single different constant is known as bech32 modified (bech32m). All of the characters in bech32 and bech32m addresses for the same underlying data will be identical except for the last six (the checksum). That means a wallet will need to know which version is in use in order to validate the checksum, but both address types contain an internal version byte that makes determining that easy.

To work with both bech32 and bech32m, we'll look at the encoding and parsing rules for bech32m Bitcoin addresses since they encompass the ability to parse bech32 addresses and are the current recommended address format for Bitcoin wallets.

Bech32m addresses start with a human readable part (HRP). There are rules in BIP173 for creating your own HRPs, but for Bitcoin you only need to know about the HRPs already chosen, shown in [bech32_hrps_for_bitcoin].

Bech32 HRPs for Bitcoin

HRPs	Network
bc	Bitcoin mainnet
tb	Bitcoin testnet

The HRP is followed by a separator, the number "1." Earlier proposals for a protocol separator used a colon but some operating systems and applications that allow a user to double-click a word to highlight it for copy and pasting won't extend the highlighting to and past a colon. A number ensured double-click highlighting would work with any program that supports bech32m strings in general (which include other numbers). The number "1" was chosen because bech32 strings don't otherwise use it in order to prevent accidental transliteration between the number "1" and the lowercase letter "l."

The other part of a bech32m address is called the "data part." There are three elements to this part:

Witness version
: A single byte that encodes as a single character in a bech32m Bitcoin address immediately following the separator. This letter represents the segwit version. The letter "q" is the encoding of "0" for segwit v0, the initial version of segwit where bech32 addresses were introduced. The letter "p" is the encoding of "1" for segwit v1 (also called taproot) where bech32m began to be used. There are seventeen possible versions of segwit and it's required for Bitcoin that the first byte of a bech32m data part decode to the number 0 through 16 (inclusive).

Witness program
: From 2 to 40 bytes. For segwit v0, this witness program must be either 20 or 32 bytes; no other length is valid. For segwit v1, the only defined length as of this writing is 32 bytes but other lengths may be defined later.

Checksum

Exactly 6 characters. This is created using a BCH code, a type of error correction code (although for Bitcoin addresses, we'll see later that it's essential to use the checksum only for error detection—not correction).

Let's illustrate these rules by walking through an example of creating bech32 and bech32m addresses. For all of the following examples, we'll use the bech32m reference code for Python.

We'll start by generating four output scripts, one for each of the different segwit outputs in use at the time of publication, plus one for a future segwit version that doesn't yet have a defined meaning. The scripts are listed in [scripts_for_diff_segwit_outputs].

Scripts for different types of segwit outputs

Output type	Example script
P2WPKH	`OP_0 2b626ed108ad00a944bb2922a309844611d25468`
P2WSH	`OP_0 648a32e50b6fb7c5233b228f60a6a2ca4158400268844c4bc295ed5e8c3d626f`
P2TR	`OP_1 2ceefa5fa770ff24f87c5475d76eab519eda6176b11dbe1618fcf755bfac5311`
Future Example	`OP_16 0000`

For the P2WPKH output, the witness program contains a commitment constructed in exactly the same way as the commitment for a P2PKH output seen in Legacy Addresses for P2PKH. A public key is passed into a SHA256 hash function. The resultant 32-byte digest is then passed into a RIPEMD-160 hash function. The digest of that function (the commitment) is placed in the witness program.

For the pay to witness script hash (P2WSH) output, we don't use the P2SH algorithm. Instead we take the script, pass it into a SHA256 hash function, and use the 32-byte digest of that function in the witness program. For P2SH, the SHA256 digest was hashed again with RIPEMD-160, but that may not be secure in some cases; for details, see P2SH Collision Attacks. A result of using SHA256 without RIPEMD-160 is that P2WSH commitments are 32 bytes (256 bits) instead of 20 bytes (160 bits).

For the pay-to-taproot (P2TR) output, the witness program is a point on the secp256k1 curve. It may be a simple public key, but in most cases it should be a public key that commits to some additional data. We'll learn more about that commitment in Taproot.

For the example of a future segwit version, we simply use the highest possible segwit version number (16) and the smallest allowed witness program (2 bytes) with a null value.

Now that we know the version number and the witness program, we can convert each of them into a bech32 address. Let's use the bech32m reference library for Python to quickly generate those addresses, and then take a deeper look at what's happening:

```
$ github="https://raw.githubusercontent.com"
$ wget $github/sipa/bech32/master/ref/python/segwit_addr.py

$ python
>>> from segwit_addr import *
>>> from binascii import unhexlify

>>> help(encode)
encode(hrp, witver, witprog)
    Encode a segwit address.

>>> encode('bc', 0, unhexlify('2b626ed108ad00a944bb2922a309844611d25468'))
'bc1q9d3xa5gg45q2j39m9y32xzvygcgay4rgc6aaee'
>>> encode('bc', 0,
unhexlify('648a32e50b6fb7c5233b228f60a6a2ca4158400268844c4bc295ed5e8c3d626f'))
'bc1qvj9r9egtd7mu2gemy28kpf4zefq4ssqzdzzycj7zjhk4arpavfhsct5a3p'
>>> encode('bc', 1,
unhexlify('2ceefa5fa770ff24f87c5475d76eab519eda6176b11dbe1618fcf755bfac5311'))
'bc1p9nh05ha8wrljf7ru236awm4t2x0d5ctkkywmu9sclnm4t0av2vgs4k3au7'
>>> encode('bc', 16, unhexlify('0000'))
'bc1sqqqqkfw08p'
```

If we open the file *segwit_addr.py* and look at what the code is doing, the first thing we will notice is the sole difference between bech32 (used for segwit v0) and bech32m (used for later segwit versions) is the constant:

```
BECH32_CONSTANT = 1
BECH32M_CONSTANT = 0x2bc830a3
```

Next we notice the code that produces the checksum. In the final step of the checksum, the appropriate constant is merged into the value using an xor operation. That single value is the only difference between bech32 and bech32m.

With the checksum created, each 5-bit character in the data part (including the witness version, witness program, and checksum) is converted to alphanumeric characters.

For decoding back into an output script, we work in reverse. First let's use the reference library to decode two of our addresses:

```
>>> help(decode)
decode(hrp, addr)
    Decode a segwit address.

>>> _ = decode("bc", "bc1q9d3xa5gg45q2j39m9y32xzvygcgay4rgc6aaee")
>>>  _[0], bytes(_[1]).hex()
(0, '2b626ed108ad00a944bb2922a309844611d25468')
>>> _ = decode("bc",
        "bc1p9nh05ha8wrljf7ru236awm4t2x0d5ctkkywmu9sclnm4t0av2vgs4k3au7")
>>> _[0], bytes(_[1]).hex()
(1, '2ceefa5fa770ff24f87c5475d76eab519eda6176b11dbe1618fcf755bfac5311')
```

We get back both the witness version and the witness program. Those can be inserted into the template for our output script:

```
<version> <program>
```

For example:

```
OP_0 2b626ed108ad00a944bb2922a309844611d25468
OP_1 2ceefa5fa770ff24f87c5475d76eab519eda6176b11dbe1618fcf755bfac5311
```

> **Warning** One possible mistake here to be aware of is that a witness version of `0` is for `OP_0`, which uses the byte 0x00—but a witness version of `1` uses `OP_1`, which is byte 0x51. Witness versions `2` through `16` use 0x52 through 0x60, respectively.

When implementing bech32m encoding or decoding, we very strongly recommend that you use the test vectors provided in BIP350. We also ask that you ensure your code passes the test vectors related to paying future segwit versions that haven't been defined yet. This will help make your software usable for many years to come even if you aren't able to add support for new Bitcoin features as soon as they become available.

4.8.3. Private Key Formats

The private key can be represented in a number of different formats, all of which correspond to the same 256-bit number. [table_4-2] shows several common formats used to represent private keys. Different formats are used in different circumstances. Hexadecimal and raw binary formats are used internally in software and rarely shown to users. The WIF is used for import/export of keys between wallets and often used in QR code (barcode) representations of private keys.

Modern Relevancy of Private Key Formats
Early Bitcoin wallet software generated one or more independent private keys when a new user wallet was initialized. When the initial set of keys had all been used, the wallet might generate additional private keys. Individual private keys could be exported or imported. Any time new private keys were generated or imported, a new backup of the wallet needed to be created.

Later Bitcoin wallets began using deterministic wallets where all private keys are generated from a single seed value. These wallets only ever need to be backed up once for typical onchain use. However, if a user exports a single private key from one of these wallets and an attacker acquires that key plus some nonprivate data about the wallet, they can potentially derive any private key in the wallet—allowing the attacker to steal all of the wallet funds. Additionally, keys cannot be imported into deterministic wallets. This means almost no modern wallets support the ability to export or import an individual key. The information in this section is mainly of interest to anyone needing compatibility with early Bitcoin wallets.

See Hierarchical Deterministic (HD) Key Generation (BIP32) for more information.

Private key representations (encoding formats)

Type	Prefix	Description
Hex	None	64 hexadecimal digits
WIF	5	Base58check encoding: base58 with version prefix of 128 and 32-bit checksum
WIF-compressed	K or L	As above, with added suffix 0x01 before encoding

[table_4-3] shows the private key generated in several different formats.

Example: Same key, different formats

Format	Private key
Hex	1e99423a4ed27608a15a2616a2b0e9e52ced330ac530edcc32c8ffc6a526aedd
WIF	5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn
WIF-compressed	KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ

All of these representations are different ways of showing the same number, the same private key. They look different, but any one format can easily be converted to any other format.

4.8.4. Compressed Private Keys

The commonly used term "compressed private key" is a misnomer, because when a private key is exported as WIF-compressed, it is actually one byte *longer* than an "uncompressed" private key. That is because the private key has an added one-byte suffix (shown as 01 in hex in [table_4-4]), which signifies that the private key is from a newer wallet and should only be used to produce compressed public keys. Private keys are not themselves compressed and cannot be compressed. The term *compressed private key* really means "private key from which only compressed public keys should be derived," whereas *uncompressed private key* really means "private key from which only uncompressed public keys should be derived." You should only refer to the export format as "WIF-compressed" or "WIF" and not refer to the private key itself as "compressed" to avoid further confusion

[table_4-4] shows the same key, encoded in WIF and WIF-compressed formats.

Example: Same key, different formats

Format	Private key
Hex	1E99423A4ED27608A15A2616A2B0E9E52CED330AC530EDCC32C8FFC6A526AEDD

Format	Private key
WIF	5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn
Hex-compressed	1E99423A4ED27608A15A2616A2B0E9E52CED330AC530EDCC32C8FFC6A526AEDD01
WIF-compressed	KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ

Notice that the hex-compressed private key format has one extra byte at the end (01 in hex). While the base58 encoding version prefix is the same (0x80) for both WIF and WIF-compressed formats, the addition of one byte on the end of the number causes the first character of the base58 encoding to change from a 5 to either a *K* or *L*. Think of this as the base58 equivalent of the decimal encoding difference between the number 100 and the number 99. While 100 is one digit longer than 99, it also has a prefix of 1 instead of a prefix of 9. As the length changes, it affects the prefix. In base58, the prefix 5 changes to a *K* or *L* as the length of the number increases by one byte.

Remember, these formats are *not* used interchangeably. In a newer wallet that implements compressed public keys, the private keys will only ever be exported as WIF-compressed (with a *K* or *L* prefix). If the wallet is an older implementation and does not use compressed public keys, the private keys will only ever be exported as WIF (with a 5 prefix). The goal here is to signal to the wallet importing these private keys whether it must search the blockchain for compressed or uncompressed public keys and addresses.

If a Bitcoin wallet is able to implement compressed public keys, it will use those in all transactions. The private keys in the wallet will be used to derive the public key points on the curve, which will be compressed. The compressed public keys will be used to produce Bitcoin addresses and those will be used in transactions. When exporting private keys from a new wallet that implements compressed public keys, the WIF is modified, with the addition of a one-byte suffix 01 to the private key. The resulting base58check-encoded private key is called a "compressed WIF" and starts with the letter *K* or *L* instead of starting with "5," as is the case with WIF-encoded (uncompressed) keys from older wallets.

4.9. Advanced Keys and Addresses

In the following sections we will look at advanced forms of keys and addresses, such as vanity addresses and paper wallets.

4.9.1. Vanity Addresses

Vanity addresses are valid Bitcoin addresses that contain human-readable messages. For example, 1LoveBPzzD72PUXLzCkYAtGFYmK5vYNR33 is a valid address that contains the letters forming the word "Love" as the first four base58 letters. Vanity addresses require generating and testing billions of candidate private keys until a Bitcoin address with the desired pattern is found. Although there are some optimizations in the vanity generation

algorithm, the process essentially involves picking a private key at random, deriving the public key, deriving the Bitcoin address, and checking to see if it matches the desired vanity pattern, repeating billions of times until a match is found.

Once a vanity address matching the desired pattern is found, the private key from which it was derived can be used by the owner to spend bitcoins in exactly the same way as any other address. Vanity addresses are no less or more secure than any other address. They depend on the same elliptic curve cryptography (ECC) and secure hash algorithm (SHA) as any other address. You can no more easily find the private key of an address starting with a vanity pattern than you can any other address.

Eugenia is a children's charity director operating in the Philippines. Let's say that Eugenia is organizing a fundraising drive and wants to use a vanity Bitcoin address to publicize the fundraising. Eugenia will create a vanity address that starts with "1Kids" to promote the children's charity fundraiser. Let's see how this vanity address will be created and what it means for the security of Eugenia's charity.

Generating vanity addresses

It's important to realize that a Bitcoin address is simply a number represented by symbols in the base58 alphabet. The search for a pattern like "1Kids" can be seen as searching for an address in the range from 1Kids111111111111111111111111111111 to 1Kidszzzzzzzzzzzzzzzzzzzzzzzzzzzzzz. There are approximately 58^{29} (approximately 1.4×10^{51}) addresses in that range, all starting with "1Kids." [table_4-11] shows the range of addresses that have the prefix 1Kids.

The range of vanity addresses starting with "1Kids"

From	`1Kids111111111111111111111111111`
	`1Kids111111111111111111111111112`
	`1Kids111111111111111111111111113`
	…
To	`1Kidszzzzzzzzzzzzzzzzzzzzzzzzzz`

Let's look at the pattern "1Kids" as a number and see how frequently we might find this pattern in a Bitcoin address (see [table_4-12]). An average desktop computer PC, without any specialized hardware, can search approximately 100,000 keys per second.

The frequency of a vanity pattern (1KidsCharity) and average search time on a desktop PC

Length	Pattern	Frequency	Average search time

Length	Pattern	Frequency	Average search time
1	1K	1 in 58 keys	< 1 milliseconds
2	1Ki	1 in 3,364	50 milliseconds
3	1Kid	1 in 195,000	< 2 seconds
4	1Kids	1 in 11 million	1 minute
5	1KidsC	1 in 656 million	1 hour
6	1KidsCh	1 in 38 billion	2 days
7	1KidsCha	1 in 2.2 trillion	3–4 months
8	1KidsChar	1 in 128 trillion	13–18 years
9	1KidsChari	1 in 7 quadrillion	800 years
10	1KidsCharit	1 in 400 quadrillion	46,000 years
11	1KidsCharity	1 in 23 quintillion	2.5 million years

As you can see, Eugenia won't be creating the vanity address "1KidsCharity" anytime soon, even if she had access to several thousand computers. Each additional character increases the difficulty by a factor of 58. Patterns with more than seven characters are usually found by specialized hardware, such as custom-built desktops with multiple graphics processing units (GPUs). Vanity searches on GPU systems are many orders of magnitude faster than on a general-purpose CPU.

Another way to find a vanity address is to outsource the work to a pool of vanity miners. A vanity pool is a service that allows those with fast hardware to earn bitcoin searching for vanity addresses for others. For a fee, Eugenia can outsource the search for a seven-character pattern vanity address and get results in a few hours instead of having to run a CPU search for months.

Generating a vanity address is a brute-force exercise: try a random key, check the resulting address to see if it matches the desired pattern, repeat until successful.

Vanity address security and privacy

Vanity addresses were popular in the early years of Bitcoin but have almost entirely disappeared from use as of 2023. There are two likely causes for this trend:

Deterministic wallets

As we saw in Recovery Codes, it’s possible to back up every key in most modern wallets by simply writing down a few words or characters. This is achieved by deriving every key in the wallet from those words or characters using a deterministic algorithm. It’s not possible to use vanity addresses with a deterministic wallet unless the user backs up additional data for every vanity address they create. More practically, most wallets using deterministic key generation simply don’t allow importing a private key or key tweak from a vanity generator.

Avoiding address reuse
: Using a vanity address to receive multiple payments to the same address creates a link between all of those payments. This might be acceptable to Eugenia if her nonprofit needs to report its income and expenditures to a tax authority anyway. However, it also reduces the privacy of people who either pay Eugenia or receive payments from her. For example, Alice may want to donate anonymously and Bob may not want his other customers to know that he gives discount pricing to Eugenia.

We don’t expect to see many vanity addresses in the future unless the preceding problems are solved.

4.9.2. Paper Wallets

Paper wallets are private keys printed on paper. Often the paper wallet also includes the corresponding Bitcoin address for convenience, but this is not necessary because it can be derived from the private key.

> **Warning** Paper wallets are an OBSOLETE technology and are dangerous for most users. There are many subtle pitfalls involved in generating them, not least of which is the possibility that the generating code is compromised with a "back door." Many bitcoins have been stolen this way. Paper wallets are shown here for informational purposes only and should not be used for storing bitcoin. Use a recovery code to back up your keys, possibly with a hardware signing device to store keys and sign transactions. DO NOT USE PAPER WALLETS.

Paper wallets come in many designs and sizes, with many different features. An example of a simple paper wallet. shows a sample paper wallet.



Figure 21. An example of a simple paper wallet.

Some are intended to be given as gifts and have seasonal themes, such as Christmas and New Year's. Others are designed for storage in a bank vault or safe with the private key hidden in some way, either with opaque scratch-off stickers or folded and sealed with tamper-proof adhesive foil. Other designs feature additional copies of the key and address, in the form of detachable stubs similar to ticket stubs, allowing you to store multiple copies to protect against fire, flood, or other natural disasters.

From the original public-key focused design of Bitcoin to modern addresses and scripts like bech32m and pay to taproot—and even addresses for future Bitcoin upgrades—you've learned how the Bitcoin protocol allows spenders to identify the wallets that should receive their payments. But when it's actually your wallet receiving the payments, you're going to want the assurance that you'll still have access to that money even if something happens to your wallet data. In the next chapter, we'll look at how Bitcoin wallets are designed to protect their funds from a variety of threats.

5. Wallet Recovery

Creating pairs of private and public keys is a crucial part of allowing Bitcoin wallets to receive and spend bitcoins. But losing access to a private key can make it impossible for anyone to ever spend the bitcoins received to the corresponding public key. Wallet and protocol developers over the years have worked to design systems that allow users to recover access to their bitcoins after a problem without compromising security the rest of the time.

In this chapter, we'll examine some of the different methods employed by wallets to prevent the loss of data from becoming a loss of money. Some solutions have almost no downsides and are universally adopted by modern wallets. We'll simply recommend those solutions as best practices. Other solutions have both advantages and disadvantages, leading different wallet authors to make different trade-offs. In those cases, we'll describe the various options available.

5.1. Independent Key Generation

Wallets for physical cash hold that cash, so it's unsurprising that many people mistakenly believe that Bitcoin wallets contain bitcoins. In fact, what many people call a Bitcoin wallet—which we call a *wallet database* to distinguish it from wallet applications—contains only keys. Those keys are associated with bitcoins recorded on the blockchain. By proving to Bitcoin full nodes that you control the keys, you can spend the associated bitcoins.

Simple wallet databases contain both the public keys to which bitcoins are received and the private keys that allow creating the signatures necessary to authorize spending those bitcoins. Other wallets' databases may contain only public keys, or only some of the private keys necessary to authorize a spending transaction. Their wallet applications produce the necessary signatures by working with external tools, such as hardware signing devices or other wallets in a multisignature scheme.

It's possible for a wallet application to independently generate each of the wallet keys it later plans to use, as illustrated in Nondeterministic key generation: a collection of independently generated keys stored in a wallet database.. All early Bitcoin wallet applications did this, but it required users to back up the wallet database each time they generated and distributed new keys, which could be as often as each time they generated a new address to receive a new payment. Failure to back up the wallet database on time would lead to the user losing access to any funds received to keys that had not been backed up.

For each independently generated key, the user would need to back up about 32 bytes, plus overhead. Some users and wallet applications tried to minimize the amount of data that needed to be backed up by only using a single key. Although that can be secure, it severely reduces the privacy of that user and all of the people with whom they transact.

People who valued their privacy and those of their peers created new key pairs for each transaction, producing wallet databases that could only reasonably be backed up using digital media.

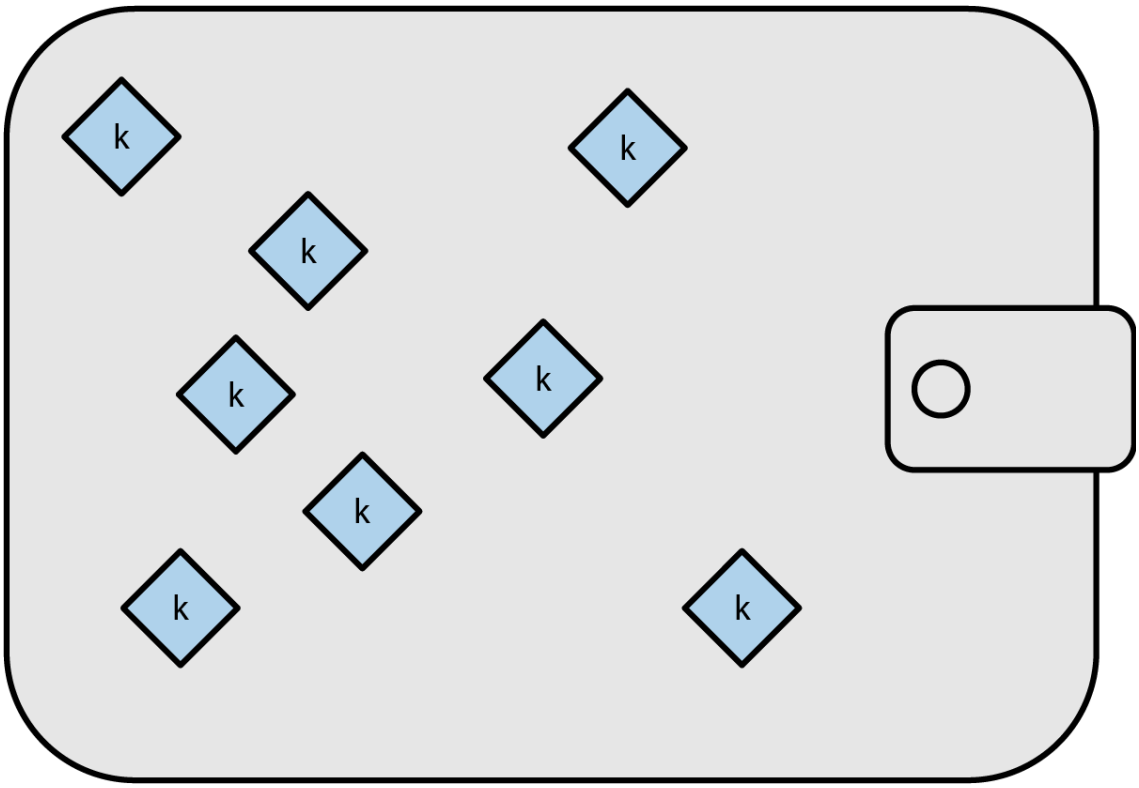


Figure 22. Nondeterministic key generation: a collection of independently generated keys stored in a wallet database.

Modern wallet applications don't independently generate keys but instead derive them from a single random seed using a repeatable (deterministic) algorithm.

5.1.1. Deterministic Key Generation

A hash function will always produce the same output when given the same input, but if the input is changed even slightly, the output will be different. If the function is cryptographically secure, nobody should be able to predict the new output—not even if they know the new input.

This allows us to take one random value and transform it into a practically unlimited number of seemingly random values. Even more useful, later using the same hash function with the same input (called a *seed*) will produce the same seemingly random values:

```
# Collect some entropy (randomness)
$ dd if=/dev/random count=1 status=none | sha256sum
f1cc3bc03ef51cb43ee7844460fa5049e779e7425a6349c8e89dfbb0fd97bb73  -

# Set our seed to the random value
$ seed=f1cc3bc03ef51cb43ee7844460fa5049e779e7425a6349c8e89dfbb0fd97bb73

# Deterministically generate derived values
$ for i in {0..2} ; do echo "$seed + $i" | sha256sum ; done
50b18e0bd9508310b8f699bad425efdf67d668cb2462b909fdb6b9bd2437beb3  -
a965dbcd901a9e3d66af11759e64a58d0ed5c6863e901dfda43adcd5f8c744f3  -
19580c97eb9048599f069472744e51ab2213f687d4720b0efc5bb344d624c3aa  -
```

If we use the derived values as our private keys, we can later generate exactly those same private keys by using our seed value with the algorithm we used before. A user of deterministic key generation can back up every key in their wallet by simply recording their seed and a reference to the deterministic algorithm they used. For example, even if Alice has a million bitcoins received to a million different addresses, all she needs to back up in order to later recover access to those bitcoins is:

```
f1cc 3bc0 3ef5 1cb4 3ee7 8444 60fa 5049
e779 e742 5a63 49c8 e89d fbb0 fd97 bb73
```

A logical diagram of basic sequential deterministic key generation is shown in Deterministic key generation: a deterministic sequence of keys derived from a seed for a wallet database.. However, modern wallet applications have a more clever way of accomplishing this that allows public keys to be derived separately from their corresponding private keys, making it possible to store private keys more securely than public keys.

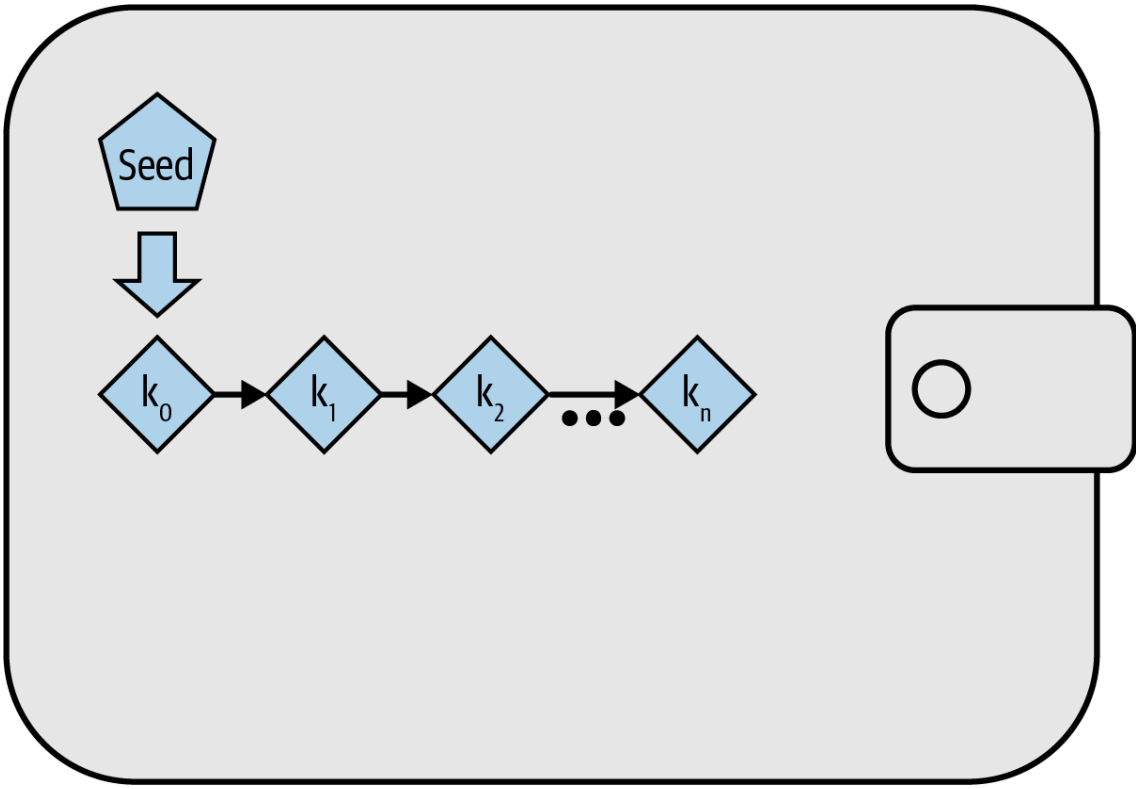


Figure 23. Deterministic key generation: a deterministic sequence of keys derived from a seed for a wallet database.

5.1.2. Public Child Key Derivation

In Public Keys, we learned how to create a public key from a private key using elliptic curve cryptography (ECC). Although operations on an elliptic curve are not intuitive, they are analogous to the addition, subtraction, and multiplication operations used in regular arithmetic. In other words, it's possible to add or subtract from a public key, or to multiply it. Consider the operation we used in Public Keys for generating a public key (K) from a private key (k) using the generator point (G):

$$K = k \times G$$

It's possible to create a derived key pair, called a child key pair, by simply adding the same value to both sides of the equation:

$$K + (123 \times G) == (k + 123) \times G$$

> **Tip** In equations throughout this book, we use a single equals sign for operations such as $K = k \times G$ where the value of a variable is calculated. We use a double equals sign to show both sides of an equation are equivalent, or that an operation should return false (not true) if the two sides aren't equivalent.

An interesting consequence of this is that adding 123 to the public key can be done using entirely public information. For example, Alice generates public key K and gives it to Bob. Bob doesn't know the private key, but he does know the global constant G, so he can add any value to the public key to produce a derived public child key. If he then tells Alice the value he added to the public key, she can add the same value to the private key, producing a derived private child key that corresponds to the public child key Bob created.

In other words, it's possible to create child public keys even if you don't know anything about the parent private key. The value added to a public key is known as a *key tweak*. If a deterministic algorithm is used for generating the key tweaks, then it's possible for someone who doesn't know the private key to create an essentially unlimited sequence of public child keys from a single public parent key. The person who controls the private parent key can then use the same key tweaks to create all the corresponding private child keys.

This technique is commonly used to separate wallet application frontends (which don't require private keys) from signing operations (which do require private keys). For example, Alice's frontend distributes her public keys to people wanting to pay her. Later, when she wants to spend the received money, she can provide the key tweaks she used to a *hardware signing device* (sometimes confusingly called a *hardware wallet*) that securely stores her original private key. The hardware signer uses the tweaks to derive the necessary child private keys and uses them to sign the transactions, returning the signed transactions to the less-secure frontend for broadcast to the Bitcoin network.

Public child key derivation can produce a linear sequence of keys similar to the previously seen Deterministic key generation: a deterministic sequence of keys derived from a seed for a wallet database., but modern wallet applications use one more trick to provide a tree of keys instead a single sequence, as described in the following section.

5.1.3. Hierarchical Deterministic (HD) Key Generation (BIP32)

Every modern Bitcoin wallet of which we're aware uses hierarchical deterministic (HD) key generation by default. This standard, defined in BIP32, uses deterministic key generation and optional public child key derivation with an algorithm that produces a tree of keys. In this tree, any key can be the parent of a sequence of child keys, and any of those child keys can be a parent for another sequence of child keys (grandchildren of the original key). There's no arbitrary limit on the depth of the tree. This tree structure is illustrated in HD wallet: a tree of keys generated from a single seed..

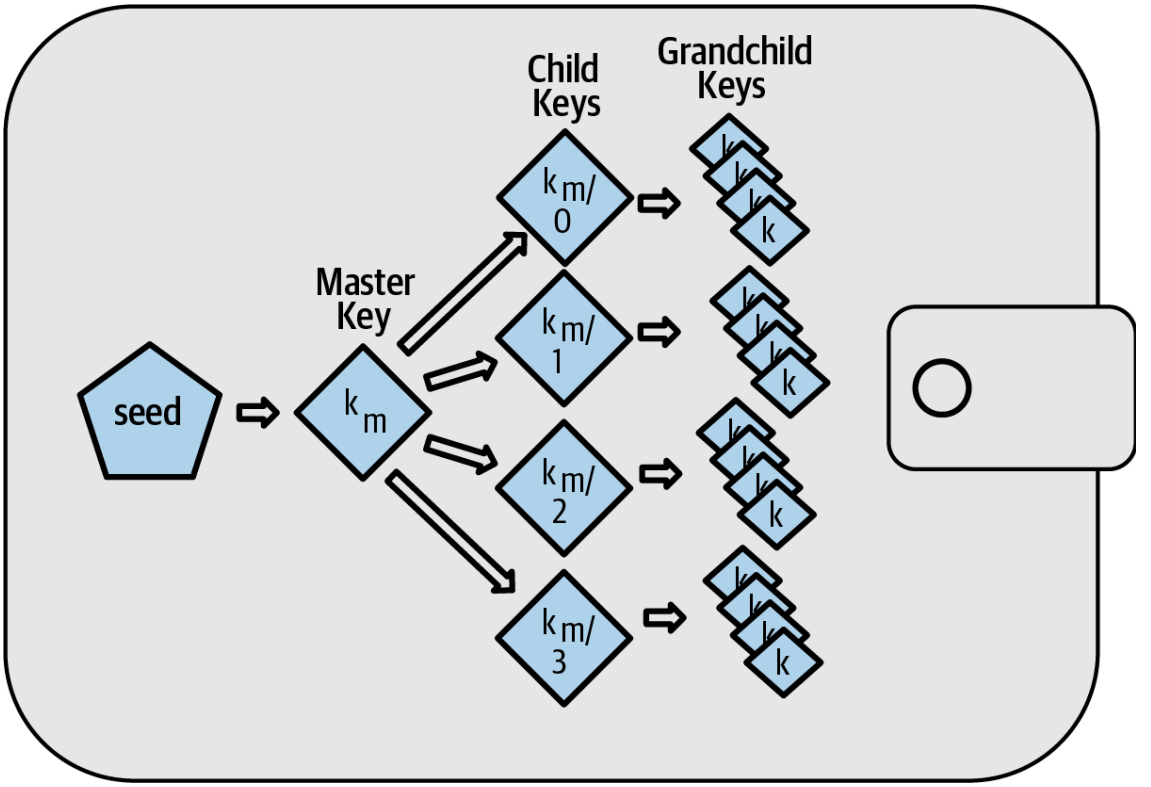


Figure 24. HD wallet: a tree of keys generated from a single seed.

The tree structure can be used to express additional organizational meaning, such as when a specific branch of subkeys is used to receive incoming payments and a different branch is used to receive change from outgoing payments. Branches of keys can also be used in corporate settings, allocating different branches to departments, subsidiaries, specific functions, or accounting categories.

We'll provide a detailed exploration of HD wallets in Creating an HD Wallet from the Seed.

5.1.4. Seeds and Recovery Codes

HD wallets are a very powerful mechanism for managing many keys all derived from a single seed. If your wallet database is ever corrupted or lost, you can regenerate all of the private keys for your wallet using your original seed. But, if someone else gets your seed, they can also generate all of the private keys, allowing them to steal all of the bitcoins from a single-sig wallet and reduce the security of bitcoins in multisignature wallets. In this section, we'll look at several *recovery codes*, which are intended to make backups easier and safer.

Although seeds are large random numbers, usually 128 to 256 bits, most recovery codes use human-language words. A large part of the motivation for using words was to make a recovery code easy to remember. For example, consider the recovery code encoded using both hexadecimal and words in A seed encoded in hex and in English words.

Example 11. A seed encoded in hex and in English words

```
Hex-encoded:
0C1E 24E5 9177 79D2 97E1 4D45 F14E 1A1A

Word-encoded:
army van defense carry jealous true
garbage claim echo media make crunch
```

There may be cases where remembering a recovery code is a powerful feature, such as when you are unable to transport physical belongings (like a recovery code written on paper) without them being seized or inspected by an outside party that might steal your bitcoins. However, most of the time, relying on memory alone is dangerous:

- If you forget your recovery code and lose access to your original wallet database, your bitcoins are lost to you forever.
- If you die or suffer a severe injury, and your heirs don't have access to your original wallet database, they won't be able to inherit your bitcoins.
- If someone thinks you have a recovery code memorized that will give them access to bitcoins, they may attempt to coerce you into disclosing that code. As of this writing, Bitcoin contributor Jameson Lopp has documented over 100 physical attacks against suspected owners of bitcoin and other digital assets, including at least three deaths and numerous occasions where someone was tortured, held hostage, or had their family threatened.

> **Tip** Even if you use a type of recovery code that was designed for easy memorization, we very strongly encourage you to consider writing it down.

Several different types of recovery codes are in wide use as of this writing:

BIP39
: The most popular method for generating recovery codes for the past decade, BIP39 involves generating a random sequence of bytes, adding a checksum to it, and encoding the data into a series of 12 to 24 words (which may be localized to a user's native language). The words (plus an optional passphrase) are run through a *key-stretching function*, and the output is used as a seed. BIP39 recovery codes have several shortcomings, which later schemes attempt to address.

Electrum v2

Used in the Electrum wallet (version 2.0 and above), this word-based recovery code has several advantages over BIP39. It doesn't rely on a global word list that must be implemented by every version of every compatible program, plus its recovery codes include a version number that improves reliability and efficiency. Like BIP39, it supports an optional passphrase (which Electrum calls a *seed extension*) and uses the same key-stretching function.

Aezeed
: Used in the LND wallet, this is another word-based recovery code that offers improvements over BIP39. It includes two version numbers: one is internal and eliminates several issues with upgrading wallet applications (like Electrum v2's version number); the other version number is external, which can be incremented to change the underlying cryptographic properties of the recovery code. It also includes a *wallet birthday* in the recovery code, a reference to the date when the user created the wallet database. This allows a restoration process to find all of the funds associated with a wallet without scanning the entire blockchain, which is especially useful for privacy-focused lightweight clients. It includes support for changing the passphrase or changing other aspects of the recovery code without needing to move funds to a new seed—the user need only back up a new recovery code. One disadvantage compared to Electrum v2 is that, like BIP39, it depends on both the backup and the recovery software supporting the same word list.

Muun
: Used in the Muun wallet, which defaults to requiring spending transactions be signed by multiple keys, this is a nonword code that must be accompanied by additional information (which Muun currently provides in a PDF). This recovery code is unrelated to the seed and is instead used to decrypt the private keys contained in the PDF. Although this is unwieldy compared to the BIP39, Electrum v2, and Aezeed recovery codes, it provides support for new technologies and standards that are becoming more common in new wallets, such as Lightning Network (LN) support, output script descriptors, and miniscript.

SLIP39
: A successor to BIP39 with some of the same authors, SLIP39 allows a single seed to be distributed using multiple recovery codes that can be stored in different places (or by different people). When you create the recovery codes, you can specify how many will be required to recover the seed. For example, you create five recovery codes but only require three of them to recover the seed. SLIP39 provides support for an optional passphrase, depends on a global word list, and doesn't directly provide versioning.

> **Note** A new system for distributing recovery codes with similarities to SLIP39 was proposed during the writing of this book. Codex32 allows creating and validating recovery codes with nothing except printed instructions, scissors, a precision knife, brass fasteners, and a pen—plus privacy and a few hours of spare time. Alternatively, those who trust computers can create recovery codes instantly using software on a digital device. You can create up to 31 recovery codes to be stored in different places, specifying how many of them will be required in order to recover the seed. As a new proposal, details about Codex32 may change significantly before this book is published, so we encourage any readers interested in distributed recovery codes to investigate its current status.

Recovery Code Passphrases

The BIP39, Electrum v2, Aezeed, and SLIP39 schemes may all be used with an optional passphrase. If the only place you keep this passphrase is in your memory, it has the same advantages and disadvantages as memorizing your recovery code. However, there's a further set of trade-offs specific to the way the passphrase is used by the recovery code.

Three of the schemes (BIP39, Electrum v2, and SLIP39) do not include the optional passphrase in the checksum they use to protect against data entry mistakes. Every passphrase (including not using a passphrase) will result in producing a seed for a BIP32 tree of keys, but they won't be the same trees. Different passphrases will result in different keys. That can be a positive or a negative, depending on your perspective:

- On the positive, if someone obtains your recovery code (but not your passphrase), they will see a valid BIP32 tree of keys. If you prepared for that contingency and sent some bitcoins to the nonpassphrase tree, they will steal that money. Although having some of your bitcoins stolen is normally a bad thing, it can also provide you with a warning that your recovery code has been compromised, allowing you to investigate and take corrective measures. The ability to create multiple passphrases for the same recovery code that all look valid is a type of *plausible deniability*.
- On the negative, if you're coerced to give an attacker a recovery code (with or without a passphrase) and it doesn't yield the amount of bitcoins they expected, they may continue trying to coerce you until you give them a different passphrase with access to more bitcoins. Designing for plausible deniability means there's no way to prove to an attacker that you've revealed all of your information, so they may continue trying to coerce you even after you've given them all of your bitcoins.
- An additional negative is the reduced amount of error detection. If you enter a slightly wrong passphrase when restoring from a backup, your wallet can't warn you about the mistake. If you were expecting a balance, you will know something is wrong when your wallet application shows you a zero balance for the regenerated key tree. However, novice users may think their money was permanently lost and do something foolish, such as give up and throw away their recovery code. Or, if you

were actually expecting a zero balance, you might use the wallet application for years after your mistake until the next time you restore with the correct passphrase and see a zero balance. Unless you can figure out what typo you previously made, your funds are gone.

Unlike the other schemes, the Aezeed seed encryption scheme authenticates its optional passphrase and will return an error if you provide an incorrect value. This eliminates plausible deniability, adds error detection, and makes it possible to prove that the passphrase has been revealed.

Many users and developers disagree on which approach is better, with some strongly in favor of plausible deniability and others preferring the increased safety that error detection gives novice users and those under duress. We suspect the debate will continue for as long as recovery codes continue to be widely used.

5.1.5. Backing Up Nonkey Data

The most important data in a wallet database is its private keys. If you lose access to the private keys, you lose the ability to spend your bitcoins. Deterministic key derivation and recovery codes provide a reasonably robust solution for backing up and recovering your keys and the bitcoins they control. However, it's important to consider that many wallet databases store more than just keys—they also store user-provided information about every transaction they sent or received.

For example, when Bob creates a new address as part of sending an invoice to Alice, he adds a *label* to the address he generates so that he can distinguish her payment from other payments he receives. When Alice pays Bob's address, she labels the transaction as paying Bob for the same reason. Some wallets also add other useful information to transactions, such as the current exchange rate, which can be useful for calculating taxes in some jurisdictions. These labels are stored entirely within their own wallets—not shared with the network—protecting their privacy and keeping unnecessary personal data out of the blockchain. For an example, see [alice_tx_labels].

Alice's transaction history with each transaction labeled

Date	Label	BTC
2023-01-01	Bought bitcoins from Joe	+0.00100
2023-01-02	Paid Bob for podcast	−0.00075

However, because address and transaction labels are stored only in each user's wallet database and because they aren't deterministic, they can't be restored by using just a recovery code. If the only recovery is seed-based, then all the user will see is a list of approximate transaction times and bitcoin amounts. This can make it quite difficult to

figure out how you used your money in the past. Imagine reviewing a bank or credit card statement from a year ago that had the date and amount of every transaction listed but a blank entry for the "description" field.

Wallets should provide their users with a convenient way to back up label data. That seems obvious, but there are a number of widely used wallet applications that make it easy to create and use recovery codes but that provide no way to back up or restore label data.

Additionally, it may be useful for wallet applications to provide a standardized format to export labels so that they can be used in other applications (e.g., accounting software). A standard for that format is proposed in BIP329.

Wallet applications implementing additional protocols beyond basic Bitcoin support may also need or want to store other data. For example, as of 2023, an increasing number of applications have added support for sending and receiving transactions over the Lightning Network (LN). Although the LN protocol provides a method to recover funds in the event of a data loss, called *static channel backups*, it can't guarantee results. If the node your wallet connects to realizes you've lost data, it may be able to steal bitcoins from you. If it loses its wallet database at the same time you lose your database, and neither of you has an adequate backup, you'll both lose funds.

Again, this means users and wallet applications need to do more than just back up a recovery code.

One solution implemented by a few wallet applications is to frequently and automatically create complete backups of their wallet database encrypted by one of the keys derived from their seed. Bitcoin keys must be unguessable and modern encryption algorithms are considered very secure, so nobody should be able to open the encrypted backup except someone who can generate the seed. This makes it safe to store the backup on untrusted computers such as cloud hosting services or even random network peers.

Later, if the original wallet database is lost, the user can enter their recovery code into the wallet application to restore their seed. The application can then retrieve the latest backup file, regenerate the encryption key, decrypt the backup, and restore all of the user's labels and additional protocol data.

5.1.6. Backing Up Key Derivation Paths

In a BIP32 tree of keys, there are approximately four billion first-level keys; each of those keys can have its own four billion children, with those children each potentially having four billion children of their own, and so on. It's not possible for a wallet application to generate even a small fraction of every possible key in a BIP32 tree, which means that recovering from data loss requires knowing more than just the recovery code, the algorithm for obtaining your seed (e.g., BIP39), and the deterministic key derivation algorithm (e.g., BIP32)—it also requires knowing what paths in the tree of keys your wallet application used for generating the specific keys it distributed.

Two solutions to this problem have been adopted. The first is using standard paths. Every time there's a change related to the addresses that wallet applications might want to generate, someone creates a BIP defining what key derivation path to use. For example, BIP44 defines `m/44'/0'/0'` as the path to use for keys in P2PKH scripts (a legacy address). A wallet application implementing this standard uses the keys in that path both when it is first started and after a restoration from a recovery code. We call this solution *implicit paths*. Several popular implicit paths defined by BIPs are shown in [bip_implicit_paths]

Implicit script paths defined by various BIPs

Standard	Script	BIP32 path
BIP44	P2PKH	`m/44'/0'/0'`
BIP49	Nested P2WPKH	`m/49'/1'/0'`
BIP84	P2WPKH	`m/84'/0'/0'`
BIP86	P2TR Single-key	`m/86'/0'/0'`

The second solution is to back up the path information with the recovery code, making it clear which path is used with which scripts. We call this *explicit paths*.

The advantage of implicit paths is that users don't need to keep a record of what paths they use. If the user enters their recovery code into the same wallet application they previously used, of the same version or higher, it will automatically regenerate keys for the same paths it previously used.

The disadvantage of implicit scripts is their inflexibility. When a recovery code is entered, a wallet application must generate the keys for every path it supports and it must scan the blockchain for transactions involving those keys, otherwise it might not find all of a user's transactions. This is wasteful in wallets that support many features each with their own path if the user only tried a few of those features.

For implicit path recovery codes that don't include a version number, such as BIP39 and SLIP39, a new version of a wallet application that drops support for an older path can't warn users during the restore process that some of their funds may not be found. The same problem happens in reverse if a user enters their recovery code into older software: it won't find newer paths to which the user may have received funds. Recovery codes that include version information, such as Electrum v2 and Aezeed, can detect that a user is entering an older or newer recovery code and direct them to appropriate resources.

The final consequence of implicit paths is that they can only include information that is either universal (such as a standardized path) or derived from the seed (such as keys). Important nondeterministic information that's specific to a certain user can't be restored using a recovery code. For example, Alice, Bob, and Carol receive funds that can only be spent with signatures from two out of three of them. Although Alice only needs either Bob's or Carol's signature to spend, she needs both of their public keys in order to find

their joint funds on the blockchain. That means each of them must back up the public keys for all three of them. As multisignature and other advanced scripts become more common on Bitcoin, the inflexibility of implicit paths becomes more significant.

The advantage of explicit paths is that they can describe exactly what keys should be used with what scripts. There's no need to support outdated scripts, no problems with backward or forward compatibility, and any extra information (like the public keys of other users) can be included directly. Their disadvantage is that they require users to back up additional information along with their recovery code. The additional information usually can't compromise a user's security, so it doesn't require as much protection as the recovery code, although it can reduce their privacy and does require some protection.

Almost all wallet applications that use explicit paths as of this writing use the *output script descriptors* standard (called *descriptors* for short) as specified in BIPs 380, 381, 382, 383, 384, 385, 386, and 389. Descriptors describe a script and the keys (or key paths) to be used with it. A few example descriptors are shown in [sample_descriptors].

Sample descriptors from Bitcoin Core documentation (with elision)

Descriptor	Explanation
`pkh(02c6…9ee5)`	P2PKH script for the provided public key
`sh(multi(2,022f…2a01,03ac…ccbe))`	P2SH multisignature requiring two signatures corresponding to these two keys
`pkh([d34db33f/44'/0'/0']xpub6ERA…RcEL/1/*)`	P2PKH scripts for the BIP32 `d34db33f` with the extended public key (xpub) at the path `M/44'/0'/0'`, which is `xpub6ERA…RcEL`, using the keys at `M/1/*` of that xpub

It has long been the trend for wallet applications designed only for single signature scripts to use implicit paths. Wallet applications designed for multiple signatures or other advanced scripts are increasingly adopting support for explicit paths using descriptors. Applications that do both will usually conform to the standards for implicit paths and also provide descriptors.

5.2. A Wallet Technology Stack in Detail

Developers of modern wallets can choose from a variety of different technologies to help users create and use backups—and new solutions appear every year. Instead of going into detail about each of the options we described earlier in this chapter, we'll focus the rest of this chapter on the stack of technologies we think is most widely used in wallets as of early 2023:

- BIP39 recovery codes
- BIP32 HD key derivation
- BIP44-style implicit paths

All of these standards have been around since 2014 or earlier, and you'll have no problem finding additional resources for using them. However, if you're feeling bold, we do encourage you to investigate more modern standards that may provide additional features or safety.

5.2.1. BIP39 Recovery Codes

BIP39 recovery codes are word sequences that represent (encode) a random number used as a seed to derive a deterministic wallet. The sequence of words is sufficient to re-create the seed and from there, re-create all the derived keys. A wallet application that implements deterministic wallets with a BIP39 recovery code will show the user a sequence of 12 to 24 words when first creating a wallet. That sequence of words is the wallet backup and can be used to recover and re-create all the keys in the same or any compatible wallet application. Recovery codes make it easier for users to back up because they are easy to read and correctly transcribe.

> **Tip** Recovery codes are often confused with "brainwallets." They are not the same. The primary difference is that a brainwallet consists of words chosen by the user, whereas recovery codes are created randomly by the wallet and presented to the user. This important difference makes recovery codes much more secure because humans are very poor sources of randomness.

Note that BIP39 is one implementation of a recovery code standard. BIP39 was proposed by the company behind the Trezor hardware wallet and is compatible with many other wallets applications, although certainly not all.

BIP39 defines the creation of a recovery code and seed, which we describe here in nine steps. For clarity, the process is split into two parts: steps 1 through 6 are shown in Generating a recovery code and steps 7 through 9 are shown in From recovery code to seed.

Generating a recovery code

Recovery codes are generated automatically by the wallet application using the standardized process defined in BIP39. The wallet starts from a source of entropy, adds a checksum, and then maps the entropy to a word list:

1. Create a random sequence (entropy) of 128 to 256 bits.
2. Create a checksum of the random sequence by taking the first (entropy-length/32) bits of its SHA256 hash.
3. Add the checksum to the end of the random sequence.
4. Split the result into 11-bit length segments.
5. Map each 11-bit value to a word from the predefined dictionary of 2,048 words.

6. The recovery code is the sequence of words.

Generating entropy and encoding as a recovery code. shows how entropy is used to generate a BIP39 recovery code.

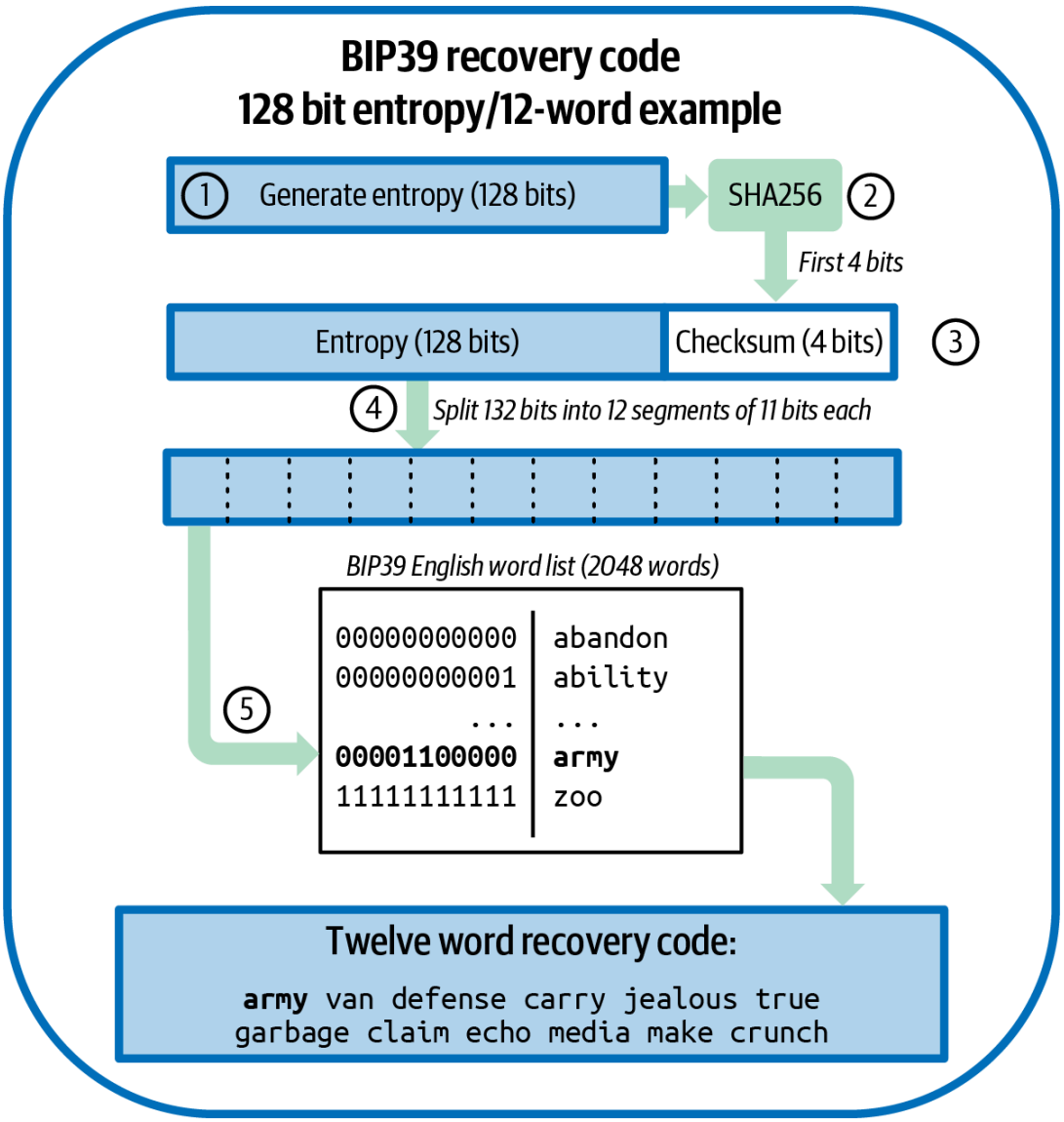


Figure 25. Generating entropy and encoding as a recovery code.

[table_4-5] shows the relationship between the size of the entropy data and the length of recovery code in words.

BIP39: entropy and word length

Entropy (bits)	Checksum (bits)	Entropy + checksum (bits)	Recovery code words
128	4	132	12
160	5	165	15
192	6	198	18
224	7	231	21
256	8	264	24

From recovery code to seed

The recovery code represents entropy with a length of 128 to 256 bits. The entropy is then used to derive a longer (512-bit) seed through the use of the key-stretching function PBKDF2. The seed produced is then used to build a deterministic wallet and derive its keys.

The key-stretching function takes two parameters: the entropy and a *salt*. The purpose of a salt in a key-stretching function is to make it difficult to build a lookup table enabling a brute-force attack. In the BIP39 standard, the salt has another purpose—it allows the introduction of a passphrase that serves as an additional security factor protecting the seed, as we will describe in more detail in Optional passphrase in BIP39.

> **Tip** The key-stretching function, with its 2,048 rounds of hashing, makes it slightly harder to brute-force attack the recovery code using software. Special-purpose hardware is not significantly affected. For an attacker who needs to guess a user's entire recovery code, the length of the code (128 bits at a minimum) provides more than sufficient security. But for cases where an attacker might learn a small part of the user's code, key-stretching adds some security by slowing down how fast an attacker can check different recovery code combinations. BIP39's parameters were considered weak by modern standards even when it was first published almost a decade ago, although that's likely a consequence of being designed for compatibility with hardware signing devices with low-powered CPUs. Some alternatives to BIP39 use stronger key-stretching parameters, such as Aezeed's 32,768 rounds of hashing using the more complex Scrypt algorithm, although they may not be as convenient to run on hardware signing devices.

The process described in steps 7 through 9 continues from the process described previously in Generating a recovery code:

7. The first parameter to the PBKDF2 key-stretching function is the *entropy* produced from step 6.
8. The second parameter to the PBKDF2 key-stretching function is a *salt*. The salt is composed of the string constant "mnemonic" concatenated with an optional user-supplied passphrase string.
9. PBKDF2 stretches the recovery code and salt parameters using 2,048 rounds of hashing with the HMAC-SHA512 algorithm, producing a 512-bit value as its final output. That 512-bit value is the seed.

From recovery code to seed. shows how a recovery code is used to generate a seed.

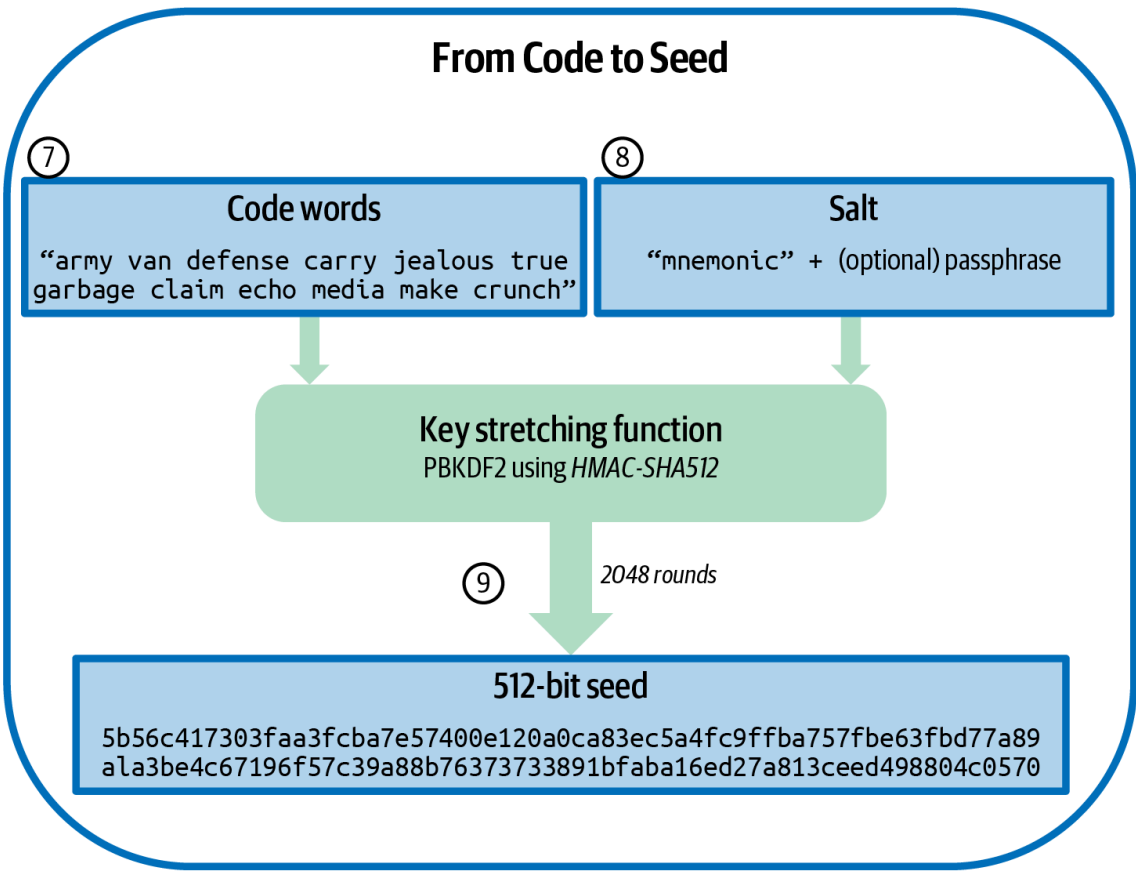


Figure 26. From recovery code to seed.

Tables #bip39_128_no_pass, #bip39_128_w_pass, and #bip39_256_no_pass show some examples of recovery codes and the seeds they produce.

128-bit entropy BIP39 recovery code, no passphrase, resulting seed

Entropy input (128 bits)	`0c1e24e5917779d297e14d45f14e1a1a`
Recovery Code (12 words)	`army van defense carry jealous true garbage claim echo media make crunch`
Passphrase	(none)
Seed (512 bits)	`5b56c417303faa3fcba7e57400e120a0ca83ec5a4fc9ffba757fbe63fbd77a89a1a3be4 c67196f57c39a88b76373733891bfaba16ed27a813ceed498804c0570`

128-bit entropy BIP39 recovery code, with passphrase, resulting seed

Entropy input (128 bits)	`0c1e24e5917779d297e14d45f14e1a1a`
Recovery Code (12 words)	`army van defense carry jealous true garbage claim echo media make crunch`
Passphrase	SuperDuperSecret
Seed (512 bits)	`3b5df16df2157104cfdd22830162a5e170c0161653e3afe6c88defeefb0818c793dbb28 ab3ab091897d0715861dc8a18358f80b79d49acf64142ae57037d1d54`

256-bit entropy BIP39 recovery code, no passphrase, resulting seed

Entropy input (256 bits)	`2041546864449caff939d32d574753fe684d3c947c3346713dd8423e74abcf8c`
Recovery Code (24 words)	`cake apple borrow silk endorse fitness top denial coil riot stay wolf luggage oxygen faint major edit measure invite love trap field dilemma oblige`
Passphrase	(none)
Seed (512 bits)	`3269bce2674acbd188d4f120072b13b088a0ecf87c6e4cae41657a0bb78f5315b33b3 a04356e53d062e55f1e0deaa082df8d487381379df848a6ad7e98798404`

How Much Entropy Do You Need?

BIP32 allows seeds to be from 128 to 512 bits. BIP39 accepts from 128 to 256 bits of entropy; Electrum v2 accepts 132 bits of entropy; Aezeed accepts 128 bits of entropy; SLIP39 accepts either 128 or 256 bits. The variation in these numbers makes it unclear how much entropy is needed for safety. We'll try to demystify that.

BIP32 extended private keys consist of a 256-bit key and a 256-bit chain code, for a total of 512 bits. That means there's a maximum of 2^{512} different possible extended private keys. If you start with more than 512 bits of entropy, you'll still get an extended private key

containing 512 bits of entropy—so there's no point in using more than 512 bits even if any of the standards we mentioned allowed that.

However, even though there are 2^{512} different extended private keys, there are only (slightly less than) 2^{256} regular private keys—and its those private keys that actually secure your bitcoins. That means, if you use more than 256 bits of entropy for your seed, you still get private keys containing only 256 bits of entropy. There may be future Bitcoin-related protocols where extra entropy in the extended keys provides extra security, but that's not currently the case.

The security strength of a Bitcoin public key is 128 bits. An attacker with a classical computer (the only kind which can be used for a practical attack as of this writing) would need to perform about 2^{128} operations on Bitcoin's elliptic curve in order to find a private key for another user's public key. The implication of a security strength of 128 bits is that there's no apparent benefit to using more than 128 bits of entropy (although you need to ensure your generated private keys are selected uniformly from within the entire 2^{256} range of private keys).

There is one extra benefit of greater entropy: if a fixed percentage of your recovery code (but not the whole code) is seen by an attacker, the greater the entropy, the harder it will be for them to figure out part of the code they didn't see. For example, if an attacker sees half of a 128-bit code (64 bits), it's plausible that they'll be able to brute force the remaining 64 bits. If they see half of a 256-bit code (128 bits), it's not plausible that they can brute force the other half. We don't recommend relying on this defense—either keep your recovery codes very safe or use a method like SLIP39 that lets you distribute your recovery code across multiple locations without relying on the safety of any individual code.

As of 2023, most modern wallets generate 128 bits of entropy for their recovery codes (or a value near 128, such as Electrum v2's 132 bits).

Optional passphrase in BIP39

The BIP39 standard allows the use of an optional passphrase in the derivation of the seed. If no passphrase is used, the recovery code is stretched with a salt consisting of the constant string "mnemonic", producing a specific 512-bit seed from any given recovery code. If a passphrase is used, the stretching function produces a *different* seed from that same recovery code. In fact, given a single recovery code, every possible passphrase leads to a different seed. Essentially, there is no "wrong" passphrase. All passphrases are valid and they all lead to different seeds, forming a vast set of possible uninitialized wallets. The set of possible wallets is so large (2^{512}) that there is no practical possibility of brute-forcing or accidentally guessing one that is in use.

> **Tip** There are no "wrong" passphrases in BIP39. Every passphrase leads to some wallet, which unless previously used will be empty.

The optional passphrase creates two important features:

- A second factor (something memorized) that makes a recovery code useless on its own, protecting recovery codes from compromise by a casual thief. For protection from a tech-savvy thief, you will need to use a very strong passphrase.
- A form of plausible deniability or "duress wallet," where a chosen passphrase leads to a wallet with a small amount of funds used to distract an attacker from the "real" wallet that contains the majority of funds.

It’s important to note that the use of a passphrase also introduces the risk of loss:

- If the wallet owner is incapacitated or dead and no one else knows the passphrase, the seed is useless and all the funds stored in the wallet are lost forever.
- Conversely, if the owner backs up the passphrase in the same place as the seed, it defeats the purpose of a second factor.

While passphrases are very useful, they should only be used in combination with a carefully planned process for backup and recovery, considering the possibility of surviving the owner and allowing his or her family to recover the cryptocurrency estate.

5.2.2. Creating an HD Wallet from the Seed

HD wallets are created from a single *root seed*, which is a 128-, 256-, or 512-bit random number. Most commonly, this seed is generated by or decrypted from a recovery code as detailed in the previous section.

Every key in the HD wallet is deterministically derived from this root seed, which makes it possible to re-create the entire HD wallet from that seed in any compatible HD wallet. This makes it easy to back up, restore, export, and import HD wallets containing thousands or even millions of keys by simply transferring only the recovery code that the root seed is derived from. The process of creating the master keys and master chain code for an HD wallet is shown in Creating master keys and chain code from a root seed..

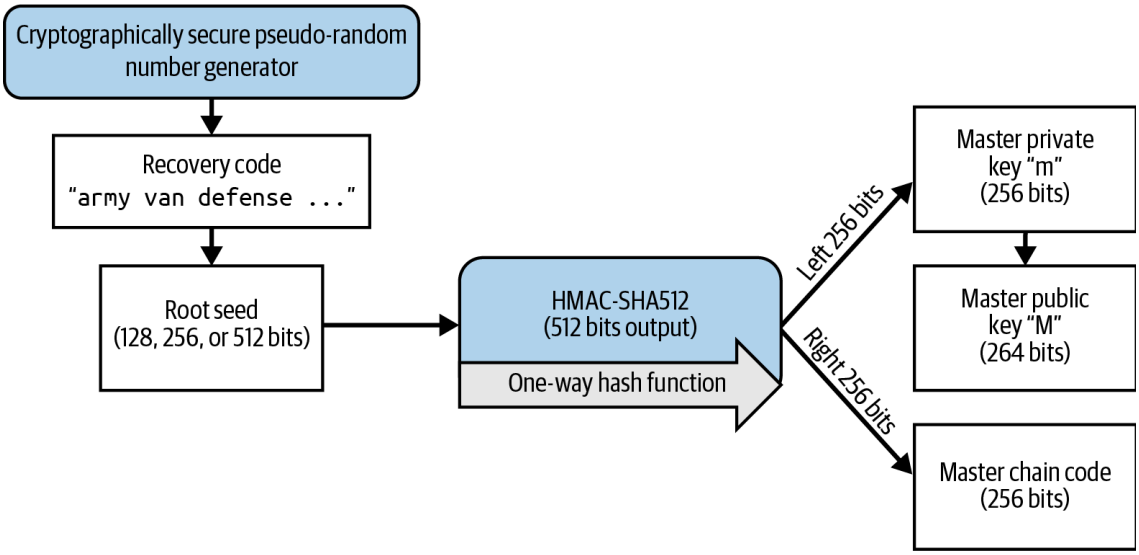


Figure 27. Creating master keys and chain code from a root seed.

The root seed is input into the HMAC-SHA512 algorithm and the resulting hash is used to create a *master private key* (*m*) and a *master chain code* (*c*).

The master private key (*m*) then generates a corresponding master public key (*M*) using the normal elliptic curve multiplication process $m \times G$ that we saw in Public Keys.

The master chain code (*c*) is used to introduce entropy in the function that creates child keys from parent keys, as we will see in the next section.

Private child key derivation

HD wallets use a *child key derivation* (CKD) function to derive child keys from parent keys.

The child key derivation functions are based on a one-way hash function that combines:

- A parent private or public key (uncompressed key)
- A seed called a chain code (256 bits)
- An index number (32 bits)

The chain code is used to introduce deterministic random data to the process, so that knowing the index and a child key is not sufficient to derive other child keys. Knowing a child key does not make it possible to find its siblings unless you also have the chain code. The initial chain code seed (at the root of the tree) is made from the seed, while subsequent child chain codes are derived from each parent chain code.

These three items (parent key, chain code, and index) are combined and hashed to generate children keys, as follows.

The parent public key, chain code, and the index number are combined and hashed with the HMAC-SHA512 algorithm to produce a 512-bit hash. This 512-bit hash is split into two 256-bit halves. The right-half 256 bits of the hash output become the chain code for the child. The left-half 256 bits of the hash are added to the parent private key to produce the child private key. In Extending a parent private key to create a child private key., we see this illustrated with the index set to 0 to produce the "zero" (first by index) child of the parent.

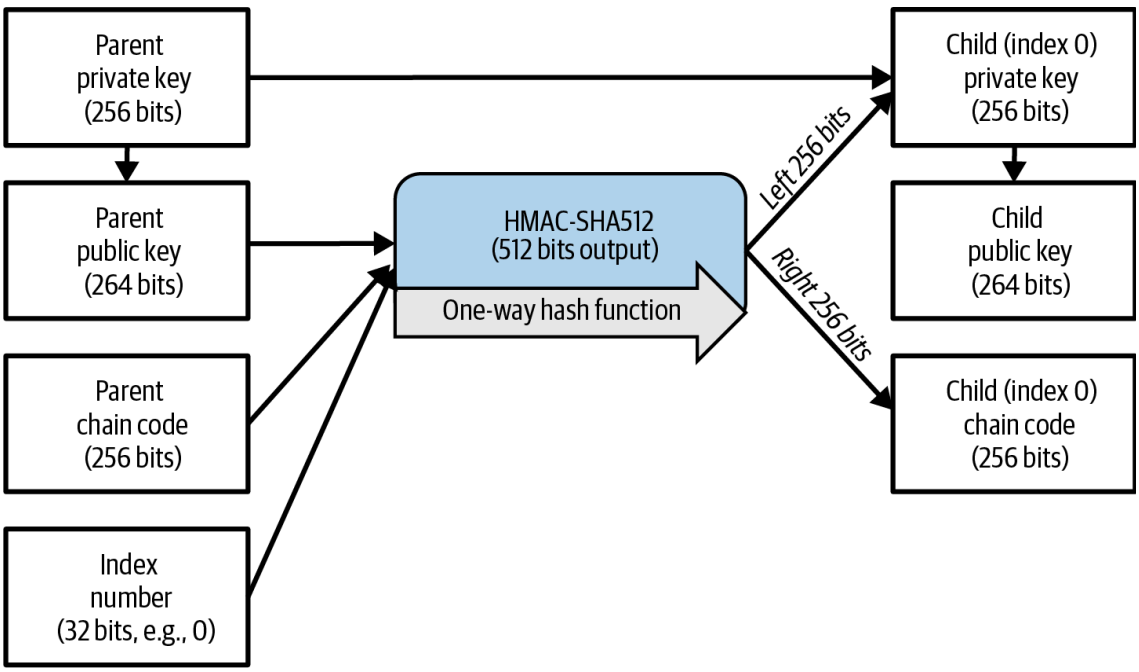


Figure 28. Extending a parent private key to create a child private key.

Changing the index allows us to extend the parent and create the other children in the sequence (e.g., Child 0, Child 1, Child 2, etc.). Each parent key can have 2,147,483,647 (2^{31}) children (2^{31} is half of the entire 2^{32} range available because the other half is reserved for a special type of derivation we will talk about later in this chapter).

Repeating the process one level down the tree, each child can in turn become a parent and create its own children, in an infinite number of generations.

Using derived child keys

Child private keys are indistinguishable from nondeterministic (random) keys. Because the derivation function is a one-way function, the child key cannot be used to find the parent key. The child key also cannot be used to find any siblings. If you have the nth child, you cannot find its siblings, such as the n–1 child or the n+1 child, or any other children that are part of the sequence. Only the parent key and chain code can derive all the children.

Without the child chain code, the child key cannot be used to derive any grandchildren either. You need both the child private key and the child chain code to start a new branch and derive grandchildren.

So what can the child private key be used for on its own? It can be used to make a public key and a Bitcoin address. Then, it can be used to sign transactions to spend anything paid to that address.

> **Tip** A child private key, the corresponding public key, and the Bitcoin address are all indistinguishable from keys and addresses created randomly. The fact that they are part of a sequence is not visible outside of the HD wallet function that created them. Once created, they operate exactly as "normal" keys.

Extended keys

As we saw earlier, the key derivation function can be used to create children at any level of the tree, based on the three inputs: a key, a chain code, and the index of the desired child. The two essential ingredients are the key and chain code, and combined these are called an *extended key*. The term "extended key" could also be thought of as "extensible key" because such a key can be used to derive children.

Extended keys are stored and represented simply as the concatenation of the key and chain code. There are two types of extended keys. An extended private key is the combination of a private key and chain code and can be used to derive child private keys (and from them, child public keys). An extended public key is a public key and chain code, which can be used to create child public keys (*public only*), as described in Public Keys.

Think of an extended key as the root of a branch in the tree structure of the HD wallet. With the root of the branch, you can derive the rest of the branch. The extended private key can create a complete branch, whereas the extended public key can *only* create a branch of public keys.

Extended keys are encoded using base58check, to easily export and import between different BIP32-compatible wallets. The base58check coding for extended keys uses a special version number that results in the prefix "xprv" and "xpub" when encoded in base58 characters to make them easily recognizable. Because the extended key contains many more bytes than regular addresses, it is also much longer than other base58check-encoded strings we have seen previously.

Here’s an example of an extended *private* key, encoded in base58check:

```
xprv9tyUQV64JT5qs3RSTJkXCWKMyUgoQp7F3hA1xzG6ZGu6u6Q9VMNjGr67Lctvy5P8oyaYAL9CA
WrUE9i6GoNMKUga5biW6Hx4tws2six3b9c
```

Here’s the corresponding extended *public* key, encoded in base58check:

```
xpub67xpozcx8pe95XVuZLHXZeG6XWXHpGq6Qv5cmNfi7cS5mtjJ2tgypeQbBs2UAR6KECeeMVKZBP
LrtJunSDMstweyLXhRgPxdp14sk9tJPW9
```

Public child key derivation

As mentioned previously, a very useful characteristic of HD wallets is the ability to derive public child keys from public parent keys *without* having the private keys. This gives us two ways to derive a child public key: either from the child private key or directly from the parent public key.

An extended public key can be used, therefore, to derive all of the *public* keys (and only the public keys) in that branch of the HD wallet structure.

This shortcut can be used to create public key–only deployments where a server or application has a copy of an extended public key and no private keys whatsoever. That kind of deployment can produce an infinite number of public keys and Bitcoin addresses but cannot spend any of the money sent to those addresses. Meanwhile, on another, more secure server, the extended private key can derive all the corresponding private keys to sign transactions and spend the money.

One common application of this solution is to install an extended public key on a web server that serves an ecommerce application. The web server can use the public key derivation function to create a new Bitcoin address for every transaction (e.g., for a customer shopping cart). The web server will not have any private keys that would be vulnerable to theft. Without HD wallets, the only way to do this is to generate thousands of Bitcoin addresses on a separate secure server and then preload them on the ecommerce server. That approach is cumbersome and requires constant maintenance to ensure that the ecommerce server doesn’t "run out" of keys.

Mind the Gap
An extended public key can generate approximately 4 billion direct child keys, far more than any store or application should ever need. However, it would also take a wallet application an unreasonable amount of time to generate all 4 billion keys and scan the blockchain for transactions involving those keys. For that reason, most wallets only generate a few keys at a time, scan for payments involving those keys, and generate additional keys in the sequence as the previous keys are used. For example, Alice’s wallet generates 100 keys. When it sees a payment to the first key, it generates the 101st key.

Sometimes a wallet application will distribute a key to someone who later decides not to pay, creating a gap in the key chain. That’s fine as long as the wallet has already generated keys after the gap so that it finds later payments and continues generating more keys. The maximum number of unused keys in a row that can fail to receive a payment without causing problems is called the *gap limit*.

When a wallet application has distributed all of the keys up to its gap limit and none of those keys have received a payment, it has three options about how to handle future requests for new keys:

1. It can refuse the requests, preventing it from receiving any further payments. This is obviously an unpalatable option, although it's the simplest to implement.
2. It can generate new keys beyond its gap limit. This ensures that every person requesting to pay gets a unique key, preventing address reuse and improving privacy. However, if the wallet needs to be restored from a recovery code, or if the wallet owner is using other software loaded with the same extended public key, those other wallets won't see any payments received after the extended gap.
3. It can distribute keys it previously distributed, ensuring a smooth recovery but potentially reducing the privacy of the wallet owner and the people with whom they transact.

Open source production systems for online merchants, such as BTCPay Server, attempt to dodge this problem by using very large gap limits and limiting the rate at which they generate invoices. Other solutions have been proposed, such as asking the spender's wallet to construct (but not broadcast) a transaction paying a possibly reused address before they receive a fresh address for the actual transaction. However, these other solutions have not been used in production as of this writing.

Another common application of this solution is for cold-storage or hardware signing devices. In that scenario, the extended private key can be stored on a paper wallet or hardware device, while the extended public key can be kept online. The user can create "receive" addresses at will, while the private keys are safely stored offline. To spend the funds, the user can use the extended private key on an offline software wallet application or the hardware signing device. Extending a parent public key to create a child public key. illustrates the mechanism for extending a parent public key to derive child public keys.

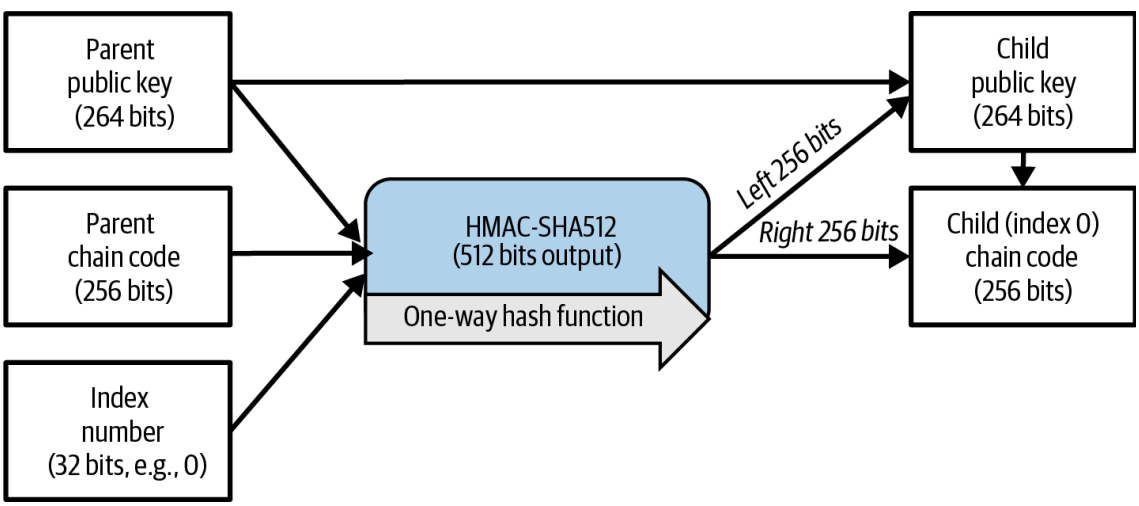


Figure 29. Extending a parent public key to create a child public key.

5.2.3. Using an Extended Public Key on a Web Store

Let’s see how HD wallets are used by looking at Gabriel’s web store.

Gabriel first set up his web store as a hobby, based on a simple hosted WordPress page. His store was quite basic with only a few pages and an order form with a single Bitcoin address.

Gabriel used the first Bitcoin address generated by his regular wallet as the main Bitcoin address for his store. Customers would submit an order using the form and send payment to Gabriel’s published Bitcoin address, triggering an email with the order details for Gabriel to process. With just a few orders each week, this system worked well enough, even though it weakened the privacy of Gabriel, his clients, and the people he paid.

However, the little web store became quite successful and attracted many orders from the local community. Soon, Gabriel was overwhelmed. With all the orders paying the same address, it became difficult to correctly match orders and transactions, especially when multiple orders for the same amount came in close together.

The only metadata that is chosen by the receiver of a typical Bitcoin transaction are the amount and payment address. There’s no subject or message field that can be used to hold a unique identifier invoice number.

Gabriel’s HD wallet offers a much better solution through the ability to derive public child keys without knowing the private keys. Gabriel can load an extended public key (xpub) on his website, which can be used to derive a unique address for every customer order. The unique address immediately improves privacy and also gives each order a unique identifier that can be used for tracking which invoices have been paid.

Using the HD wallet allows Gabriel to spend the funds from his personal wallet application, but the xpub loaded on the website can only generate addresses and receive funds. This feature of HD wallets is a great security feature. Gabriel’s website does not contain any private keys and therefore any hack of it can only steal the funds Gabriel would have received in the future, not any funds he received in the past.

To export the xpub from his Trezor hardware signing device, Gabriel uses the web-based Trezor wallet application. The Trezor device must be plugged in for the public keys to be exported. Note that most hardware signing devices will never export private keys—those always remain on the device.

Gabriel copies the xpub to his web store’s Bitcoin payment processing software, such as the widely used open source BTCPay Server.

Hardened child key derivation

The ability to derive a branch of public keys from an xpub is very useful, but it comes with a potential risk. Access to an xpub does not give access to child private keys. However, because the xpub contains the chain code, if a child private key is known, or somehow leaked, it can be used with the chain code to derive all the other child private keys. A

single leaked child private key, together with a parent chain code, reveals all the private keys of all the children. Worse, the child private key together with a parent chain code can be used to deduce the parent private key.

To counter this risk, HD wallets provide an alternative derivation function called *hardened derivation*, which breaks the relationship between parent public key and child chain code. The hardened derivation function uses the parent private key to derive the child chain code, instead of the parent public key. This creates a "firewall" in the parent/child sequence, with a chain code that cannot be used to compromise a parent or sibling private key. The hardened derivation function looks almost identical to the normal child private key derivation, except that the parent private key is used as input to the hash function, instead of the parent public key, as shown in the diagram in Hardened derivation of a child key; omits the parent public key..

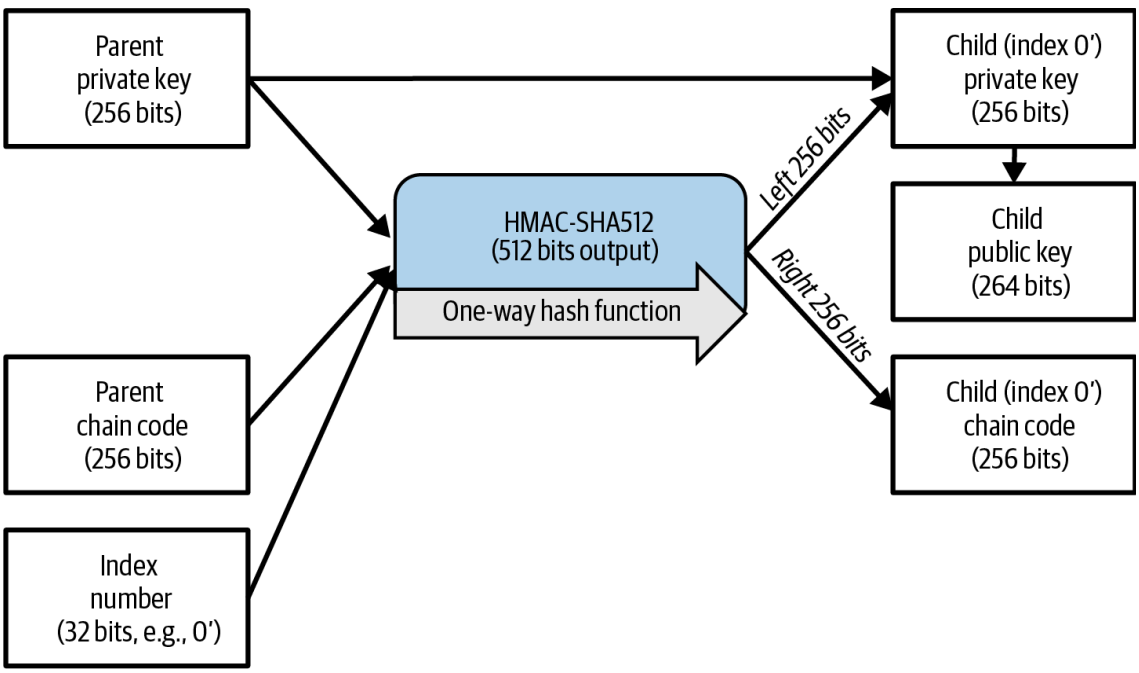


Figure 30. Hardened derivation of a child key; omits the parent public key.

When the hardened private derivation function is used, the resulting child private key and chain code are completely different from what would result from the normal derivation function. The resulting "branch" of keys can be used to produce extended public keys that are not vulnerable because the chain code they contain cannot be exploited to reveal any private keys for their siblings or parents. Hardened derivation is therefore used to create a "gap" in the tree above the level where extended public keys are used.

In simple terms, if you want to use the convenience of an xpub to derive branches of public keys, without exposing yourself to the risk of a leaked chain code, you should derive it from a hardened parent rather than a normal parent. As a best practice, the level-1 children of

the master keys are always derived through the hardened derivation to prevent compromise of the master keys.

Index numbers for normal and hardened derivation

The index number used in the derivation function is a 32-bit integer. To easily distinguish between keys created through the normal derivation function versus keys derived through hardened derivation, this index number is split into two ranges. Index numbers between 0 and 2^{31} – 1 (0x0 to 0x7FFFFFFF) are used *only* for normal derivation. Index numbers between 2^{31} and 2^{32} – 1 (0x80000000 to 0xFFFFFFFF) are used *only* for hardened derivation. Therefore, if the index number is less than 2^{31}, the child is normal, whereas if the index number is equal or above 2^{31}, the child is hardened.

To make the index number easier to read and display, the index number for hardened children is displayed starting from zero, but with a prime symbol. The first normal child key is therefore displayed as 0, whereas the first hardened child (index 0x80000000) is displayed as 0'. In a sequence then, the second hardened key would have index 0x80000001 and would be displayed as 1', and so on. When you see an HD wallet index i', that means 2^{31}+i. In regular ASCII text, the prime symbol is substituted with either a single apostrophe or the letter *h*. For situations, such as in output script descriptors, where text may be used in a shell or other context where a single apostrophe has special meaning, using the letter *h* is recommended.

HD wallet key identifier (path)

Keys in an HD wallet are identified using a "path" naming convention, with each level of the tree separated by a slash (/) character (see [table_4-8]). Private keys derived from the master private key start with "m." Public keys derived from the master public key start with "M." Therefore, the first child private key of the master private key is m/0. The first child public key is M/0. The second grandchild of the first child is m/0/1, and so on.

The "ancestry" of a key is read from right to left, until you reach the master key from which it was derived. For example, identifier m/x/y/z describes the key that is the z-th child of key m/x/y, which is the y-th child of key m/x, which is the x-th child of m.

HD wallet path examples

HD path	Key described
m/0	The first (0) child private key from the master private key (m)
m/0/0	The first grandchild private key from the first child (m/0)
m/0'/0	The first normal grandchild private key from the first *hardened* child (m/0')
m/1/0	The first grandchild private key from the second child (m/1)

HD path	Key described
M/23/17/0/0	The first great-great-grandchild public key from the first great-grandchild from the 18th grandchild from the 24th child

Navigating the HD wallet tree structure

The HD wallet tree structure offers tremendous flexibility. Each parent extended key can have 4 billion children: 2 billion normal children and 2 billion hardened children. Each of those children can have another 4 billion children, and so on. The tree can be as deep as you want, with an infinite number of generations. With all that flexibility, however, it becomes quite difficult to navigate this infinite tree. It is especially difficult to transfer HD wallets between implementations because the possibilities for internal organization into branches and subbranches are endless.

Two BIPs offer a solution to this complexity by creating some proposed standards for the structure of HD wallet trees. BIP43 proposes the use of the first hardened child index as a special identifier that signifies the "purpose" of the tree structure. Based on BIP43, an HD wallet should use only one level-1 branch of the tree, with the index number identifying the structure and namespace of the rest of the tree by defining its purpose. For example, an HD wallet using only branch m/i'/ is intended to signify a specific purpose, and that purpose is identified by index number "i."

Extending that specification, BIP44 proposes a multiaccount structure as "purpose" number 44' under BIP43. All HD wallets following the BIP44 structure are identified by the fact that they only used one branch of the tree: m/44'/.

BIP44 specifies the structure as consisting of five predefined tree levels:

```
m / purpose' / coin_type' / account' / change / address_index
```

The first-level "purpose" is always set to 44'. The second-level "coin_type" specifies the type of cryptocurrency coin, allowing for multicurrency HD wallets where each currency has its own subtree under the second level. Bitcoin is m/44'/0' and Bitcoin Testnet is m/44'/1'.

The third level of the tree is "account," which allows users to subdivide their wallets into separate logical subaccounts for accounting or organizational purposes. For example, an HD wallet might contain two Bitcoin "accounts": m/44'/0'/0' and m/44'/0'/1'. Each account is the root of its own subtree.

On the fourth level, "change," an HD wallet has two subtrees, one for creating receiving addresses and one for creating change addresses. Note that whereas the previous levels used hardened derivation, this level uses normal derivation. This is to allow this level of the tree to export extended public keys for use in a nonsecured environment. Usable addresses are derived by the HD wallet as children of the fourth level, making the fifth

level of the tree the "address_index." For example, the third receiving address for payments in the primary account would be M/44'/0'/0'/0/2. [table_4-9] shows a few more examples.

BIP44 HD wallet structure examples

HD path	Key described
M/44'/0'/0'/0/2	The third receiving public key for the primary Bitcoin account
M/44'/0'/3'/1/14	The fifteenth change-address public key for the fourth Bitcoin account
m/44'/2'/0'/0/1	The second private key in the Litecoin main account, for signing transactions

Many people focus on securing their bitcoins against theft and other attacks, but one of the leading causes of lost bitcoins—perhaps *the* leading cause—is data loss. If the keys and other essential data required to spend your bitcoins is lost, those bitcoins will forever be unspendable. Nobody can get them back for you. In this chapter, we looked at the systems that modern wallet applications use to help you prevent losing that data. Remember, however, that it’s up to you to actually use the systems available to make good backups and regularly test them.

6. Transactions

The way we typically transfer physical cash has little resemblance to the way we transfer bitcoins. Physical cash is a bearer token. Alice pays Bob by handing him some number of tokens, such as dollar bills. By comparison, bitcoins don't exist either physically or as digital data—Alice can't hand Bob some bitcoins or send them by email.

Instead, consider how Alice might transfer control over a parcel of land to Bob. She can't physically pick up the land and hand it to Bob. Rather there exists some sort of record (usually maintained by a local government) that describes the land Alice owns. Alice transfers that land to Bob by convincing the government to update the record to say that Bob now owns the land.

Bitcoin works in a similar way. There exists a database on every Bitcoin full node that says that Alice controls some number of bitcoins. Alice pays Bob by convincing full nodes to update their database to say that some of Alice's bitcoins are now controlled by Bob. The data that Alice uses to convince full nodes to update their databases is called a *transaction*. This is done without directly using either Alice's or Bob's identities, as we'll see in Authorization and Authentication.

In this chapter we'll deconstruct a Bitcoin transaction and examine each of its parts to see how they facilitate the transfer of value in a way that's highly expressive and amazingly reliable.

6.1. A Serialized Bitcoin Transaction

In Exploring and Decoding Transactions, we used Bitcoin Core with the txindex option enabled to retrieve a copy of Alice's payment to Bob. Let's retrieve the transaction containing that payment again, as shown in Alice's serialized transaction.

Example 12. Alice's serialized transaction

```
$ bitcoin-cli getrawtransaction 466200308696215bbc949d5141a49a41\
38ecdfdfaa2a8029c1f9bcecd1f96177

01000000000101eb3ae38f27191aa5f3850dc9cad00492b88b72404f9da13569
8679268041c54a0100000000ffffffff02204e0000000000002251203b41daba
4c9ace578369740f15e5ec880c28279ee7f51b07dca69c7061e07068f8240100
0000000001600147752c165ea7be772b2c0acb7f4d6047ae6f4768e0141cf5efe
2d8ef13ed0af21d4f4cb82422d6252d70324f6f4576b727b7d918e521c00b51b
e739df2f899c49dc267c0ad280aca6dab0d2fa2b42a45182fc83e81713010000
0000
```

Bitcoin Core's serialization format is special because it's the format used to make commitments to transactions and to relay them across Bitcoin's P2P network, but otherwise programs can use a different format as long as they transmit all of the same

data. However, Bitcoin Core's format is reasonably compact for the data it transmits and simple to parse, so many other Bitcoin programs use this format.

> **Tip** The only other widely used transaction serialization format that we're aware of is the partially signed bitcoin transaction (PSBT) format documented in BIPs 174 and 370 (with extensions documented in other BIPs). PSBT allows an untrusted program to produce a transaction template that can be verified and updated by trusted programs (such as hardware signing devices) that have the necessary private keys or other sensitive data to fill in the template. To accomplish this, PSBT allows storing a significant amount of metadata about a transaction, making it much less compact than the standard serialization format. This book does not go into detail about PSBT, but we strongly recommend it to developers of wallets that plan to support signing with multiple keys.

The transaction displayed in hexadecimal in Alice's serialized transaction is replicated as a byte map in A byte map of Alice's transaction.. Note that it takes 64 hexadecimal characters to display 32 bytes. This map shows only the top-level fields. We'll examine each of them in the order they appear in the transaction and describe any additional fields that they contain.

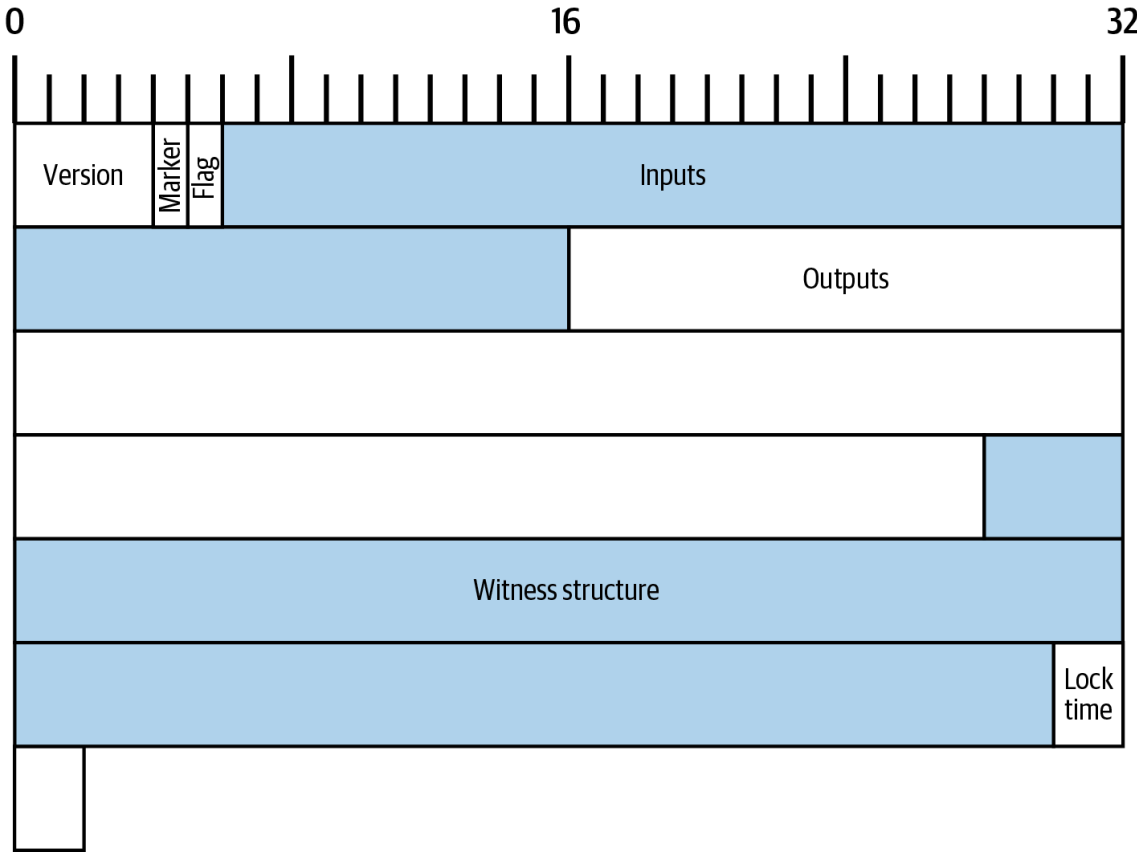


Figure 31. A byte map of Alice's transaction.

6.2. Version

The first four bytes of a serialized Bitcoin transaction are its version. The original version of Bitcoin transactions was version 1 (0x01000000). All transactions in Bitcoin must follow the rules of version 1 transactions, with many of those rules being described throughout this book.

Version 2 Bitcoin transactions were introduced in the BIP68 soft fork change to Bitcoin's consensus rules. BIP68 places additional constraints on the sequence field, but those constraints only apply to transactions with version 2 or higher. Version 1 transactions are unaffected. BIP112, which was part of the same soft fork as BIP68, upgraded an opcode (OP_CHECKSEQUENCEVERIFY), which will now fail if it is evaluated as part of a transaction with a version less than 2. Beyond those two changes, version 2 transactions are identical to version 1 transactions.

Protecting Presigned Transactions

The last step before broadcasting a transaction to the network for inclusion in the blockchain is to sign it. However, it's possible to sign a transaction without broadcasting it immediately. You can save that presigned transaction for months or years in the belief that it can be added to the blockchain later when you do broadcast it. In the interim, you may even lose access to the private key (or keys) necessary to sign an alternative transaction spending the funds. This isn't hypothetical: several protocols built on Bitcoin, including Lightning Network, depend on presigned transactions.

This creates a challenge for protocol developers when they assist users in upgrading the Bitcoin consensus protocol. Adding new constraints—such as BIP68 did to the sequence field—may invalidate some presigned transactions. If there's no way to create a new signature for an equivalent transaction, then the money being spent in the presigned transaction is permanently lost.

This problem is solved by reserving some transaction features for upgrades, such as version numbers. Anyone creating presigned transactions prior to BIP68 should have been using version 1 transactions, so only applying BIP68's additional constraints on sequence to transactions v2 or higher should not invalidate any presigned transactions.

If you implement a protocol that uses presigned transactions, ensure that it doesn't use any features that are reserved for future upgrades. Bitcoin Core's default transaction relay policy does not allow the use of reserved features. You can test whether a transaction complies with that policy by using Bitcoin Core's testmempoolaccept RPC on Bitcoin mainnet.

As of this writing, a proposal to begin using version 3 transactions is being widely considered. That proposal does not seek to change the consensus rules but only the policy that Bitcoin full nodes use to relay transactions. Under the proposal, version 3 transactions would be subject to additional constraints in order to prevent certain denial of service (DoS) attacks that we'll discuss further in Transaction Pinning.

6.3. Extended Marker and Flag

The next two fields of the example serialized transaction were added as part of the segregated witness (segwit) soft fork change to Bitcoin's consensus rules. The rules were changed according to BIPs 141 and 143, but the *extended serialization format* is defined in BIP144.

If the transaction includes a witness structure (which we'll describe in Witness Structure), the marker must be zero (0x00) and the flag must be nonzero. In the current P2P protocol, the flag should always be one (0x01); alternative flags are reserved for later protocol upgrades.

If the transaction doesn't need a witness stack, the marker and flag must not be present. This is compatible with the original version of Bitcoin's transaction serialization format, now called *legacy serialization*. For details, see Legacy Serialization.

In legacy serialization, the marker byte would have been interpreted as the number of inputs (zero). A transaction can't have zero inputs, so the marker signals to modern programs that extended serialization is being used. The flag field provides a similar signal and also simplifies the process of updating the serialization format in the future.

6.4. Inputs

The inputs field contains several other fields, so let's start by showing a map of those bytes in Map of bytes in the inputs field of Alice's transaction..

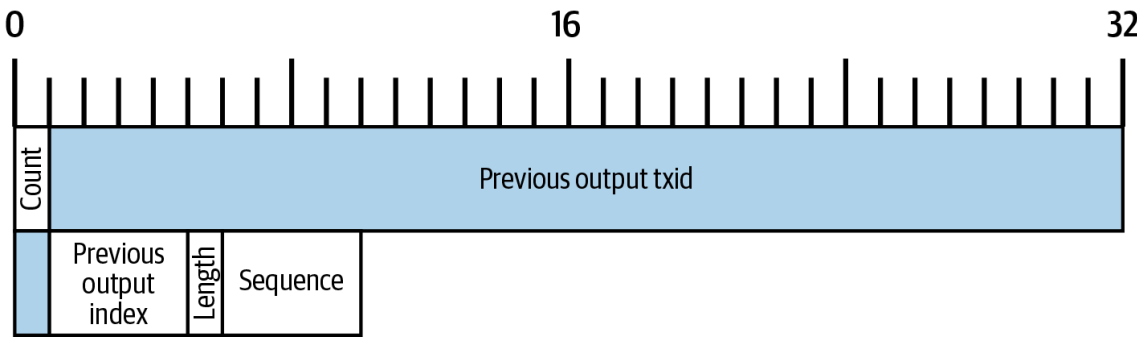


Figure 32. Map of bytes in the inputs field of Alice's transaction.

6.4.1. Length of Transaction Input List

The transaction input list starts with an integer indicating the number of inputs in the transaction. The minimum value is one. There's no explicit maximum value, but restrictions on the maximum size of a transaction effectively limit transactions to a few thousand inputs. The number is encoded as a compactSize unsigned integer.

CompactSize Unsigned Integers
Unsigned integers in Bitcoin that often have low values, but which may sometimes have high values, are usually encoded using the compactSize data type. CompactSize is a version of a variable-length integer, so it's sometimes called var_int or varint (see, for example, documentation for BIPs 37 and 144).

> **Warning** Several varieties of variable length integers are used in different programs, including in different Bitcoin programs. For example, Bitcoin Core serializes its UTXO database using a data type it calls VarInts, which is different from compactSize. Additionally, the nBits field in a Bitcoin block header is encoded using a custom data type known as Compact, which is unrelated to compactSize. When talking about the variable length integers used in Bitcoin transaction serialization and other parts of the Bitcoin P2P protocol, we will always use the full name compactSize.

For numbers from 0 to 252, compactSize unsigned integers are identical to the C-language data type uint8_t, which is probably the native encoding familiar to any programmer. For other numbers up to 0xffffffffffffffff, a byte is prefixed to the number to indicate its length—but otherwise the numbers look like regular C-language encoded unsigned integers:

Value	Bytes used	Format
≥ 0 && ≤ 252 (0xfc)	1	uint8_t
≥ 253 && ≤ 0xffff	3	0xfd followed by the number as uint16_t
≥ 0x10000 && ≤ 0xffffffff	5	0xfe followed by the number as uint32_t
≥ 0x100000000 && ≤ 0xffffffffffffffff	9	0xff followed by the number as uint64_t

Each input in a transaction must contain three fields: an *outpoint* field, a length-prefixed *input script* field, and a *sequence*

We'll look at each of those fields in the following sections. Some inputs also include a witness stack, but this is serialized at the end of a transaction so we'll examine it later.

6.4.2. Outpoint

A Bitcoin transaction is a request for full nodes to update their database of coin ownership information. For Alice to transfer control of some of her bitcoins to Bob, she first needs to tell full nodes how to find the previous transfer where she received those bitcoins. Since control over bitcoins is assigned in transaction outputs, Alice *points* to the previous *output* using an *outpoint* field. Each input must contain a single outpoint.

The outpoint contains a 32-byte txid for the transaction where Alice received the bitcoins she now wants to spend. This txid is in Bitcoin's internal byte order for hashes; see Internal and Display Byte Orders.

Because transactions may contain multiple outputs, Alice also needs to identify which particular output from that transaction to use, called its *output index*. Output indexes are 4-byte unsigned integers starting from zero.

When a full node encounters an outpoint, it uses that information to try to find the referenced output. Full nodes are only required to look at earlier transactions in the blockchain. For example, Alice's transaction is included in block 774,958. A full node verifying her transaction only looks for the previous output referenced by her outpoint in that block and previous blocks, not any later blocks. Within block 774,958, they will only look at transactions placed in the block prior to Alice's transaction, as determined by the order of leaves in the block's merkle tree (see Merkle Trees).

Upon finding the previous output, the full node obtains several critical pieces of information from it:

- The amount of bitcoins assigned to that previous output. All of those bitcoins will be transferred in this transaction. In the example transaction, the value of the previous output was 100,000 satoshis.
- The authorization conditions for that previous output. These are the conditions that must be fulfilled in order to spend the bitcoins assigned to that previous output.
- For confirmed transactions, the height of the block that confirmed it and the median time past (MTP) for that block. This is required for relative timelocks (described in Sequence as a consensus-enforced relative timelock) and outputs of coinbase transactions (described in Coinbase Transactions).
- Proof that the previous output exists in the blockchain (or as a known unconfirmed transaction) and that no other transaction has spent it. One of Bitcoin's consensus rules forbids any output from being spent more than once within a valid blockchain. This is the rule against *double spending*: Alice can't use the same previous output to pay both Bob and Carol in separate transactions. Two transactions that each try to spend the same previous output are called *conflicting transactions* because only one of them can be included in a valid blockchain.

Different approaches to tracking previous outputs have been tried by different full node implementations at various times. Bitcoin Core currently uses the solution believed to be most effective at retaining all necessary information while minimizing disk space: it keeps a database that stores every UTXO and essential metadata about it (like its confirmation block height). Each time a new block of transactions arrives, all of the outputs they spend are removed from the UTXO database and all of the outputs they create are added to the database.

Internal and Display Byte Orders

Bitcoin uses the output of hash functions, called *digests*, in various ways. Digests provide unique identifiers for blocks and transactions; they're used in commitments for addresses, blocks, transactions, signatures, and more; and digests are iterated upon in Bitcoin's proof-of-work function. In some cases, hash digests are displayed to users in one byte order but are used internally in a different byte order, creating confusion. For example, consider the previous output txid from the outpoint in our example transaction:

```
eb3ae38f27191aa5f3850dc9cad00492b88b72404f9da135698679268041c54a
```

If we try using that txid to retrieve that transaction using Bitcoin Core, we get an error and must reverse its byte order:

```
$ bitcoin-cli getrawtransaction \
  eb3ae38f27191aa5f3850dc9cad00492b88b72404f9da135698679268041c54a
error code: -5
error message:
No such mempool or blockchain transaction.
Use gettransaction for wallet transactions.

$ echo eb3ae38f27191aa5f3850dc9cad00492b88b72404f9da135698679268041c54a \
  | fold -w2 | tac | tr -d "\n"
4ac541802679866935a19d4f40728bb89204d0cac90d85f3a51a19278fe33aeb

$ bitcoin-cli getrawtransaction \
  4ac541802679866935a19d4f40728bb89204d0cac90d85f3a51a19278fe33aeb
02000000000101c25ae90c9f3d40cc1fc509ecfd54b06e35450702...
```

This odd behavior is probably an unintentional consequence of a design decision in early Bitcoin software. As a practical matter, it means developers of Bitcoin software need to remember to reverse the order of bytes in transaction and block identifiers they show to users.

In this book, we use the term *internal byte order* for the data that appears within transactions and blocks. We use *display byte order* for the form displayed to users. Another set of common terms is *little-endian byte order* for the internal version and *big-endian byte order* for the display version.

6.4.3. Input Script

The input script field is a remnant of the legacy transaction format. Our example transaction input spends a native segwit output that doesn't require any data in the input script, so the length prefix for the input script is set to zero (0x00).

For an example of a length-prefixed input script that spends a legacy output, we use one from an arbitrary transaction in the most recent block as of this writing:

```
6b483045022100a6cc4e8cd0847951a71fad3bc9b14f24d44ba59d19094e0a8c
fa2580bb664b020220366060ea8203d766722ed0a02d1599b99d3c95b97dab8e
41d3e4d3fe33a5706201210369e03e2c91f0badec46c9c903d9e9edae67c167b
9ef9b550356ee791c9a40896
```

The length prefix is a compactSize unsigned integer indicating the length of the serialized input script field. In this case, it's a single byte (0x6b) indicating the input script is 107 bytes. We'll cover parsing and using scripts in detail in Authorization and Authentication.

6.4.4. Sequence

The final four bytes of an input are its *sequence* number. The use and meaning of this field has changed over time.

Original sequence-based transaction replacement

The sequence field was originally intended to allow creation of multiple versions of the same transaction, with later versions replacing earlier versions as candidates for confirmation. The sequence number tracked the version of the transaction.

For example, imagine Alice and Bob want to bet on a game of cards. They start by each signing a transaction that deposits some money into an output with a script that requires signatures from both of them to spend, a *multisignature* script (*multisig* for short). This is called the *setup transaction*. They then create a transaction that spends that output:

- The first version of the transaction, with nSequence 0 (0x00000000), pays Alice and Bob back the money they initially deposited. This is called a *refund transaction*. Neither of them broadcasts the refund transaction at this time. They only need it if there's a problem.
- Alice wins the first round of the card game, so the second version of the transaction, with sequence 1, increases the amount of money paid to Alice and decreases Bob's share. They both sign the updated transaction. Again, they don't need to broadcast this version of the transaction unless there's a problem.
- Bob wins the second round, so the sequence is incremented to 2, Alice's share is decreased, and Bob's share is increased. They again sign but don't broadcast.
- After many more rounds where the sequence is incremented, the funds redistributed, and the resulting transaction is signed but not broadcast, they decide to finalize the transaction. Creating a transaction with the final balance of funds, they set sequence to its maximum value (0xffffffff), finalizing the transaction. They broadcast this version of the transaction, it's relayed across the network, and eventually confirmed by miners.

We can see the replacement rules for sequence at work if we consider alternative scenarios:

- Imagine that Alice broadcasts the final transaction, with a sequence of 0xffffffff, and then Bob broadcasts one of the earlier transactions where his balance was higher. Because Bob's version of the transaction has a lower sequence number, full nodes using the original Bitcoin code won't relay it to miners, and miners who also used the original code won't mine it.
- In another scenario, imagine that Bob broadcasts an earlier version of the transaction a few seconds before Alice broadcasts the final version. Nodes will relay Bob's version and miners will attempt to mine it, but when Alice's version with its

higher sequence number arrives, nodes will also relay it and miners using the original Bitcoin code will try to mine it instead of Bob's version. Unless Bob got lucky and a block was discovered before Alice's version arrived, it's Alice's version of the transaction that will get confirmed.

This type of protocol is what we now call a *payment channel*. Bitcoin's creator, in an email attributed to him, called these *high-frequency transactions* and described a number of features added to the protocol to support them. We'll learn about several of those other features later and also discover how modern versions of payment channels are increasingly being used in Bitcoin today.

There were a few problems with purely sequence-based payment channels. The first was that the rules for replacing a lower-sequence transaction with a higher-sequence transaction were only a matter of software policy. There was no direct incentive for miners to prefer one version of the transaction over any other. The second problem was that the first person to send their transaction might get lucky and have it confirmed even if it wasn't the highest-sequence transaction. A security protocol that fails a few percent of the time due to bad luck isn't a very effective protocol.

The third problem was that it was possible to replace one version of a transaction with a different version an unlimited number of times. Each replacement would consume the bandwidth of all the relaying full nodes on the network. For example, as of this writing, there are about 50,000 relaying full nodes; an attacker creating 1,000 replacement transactions per minute at 200 bytes each would use about 20 KB of their personal bandwidth but about 10 GB of full node network bandwidth every minute. Except for the cost of their 20 KB/minute bandwidth and the occasional fee when a transaction got confirmed, the attacker wouldn't need to pay any costs for the enormous burden they placed on full node operators.

To eliminate the risk of this attack, the original type of sequence-based transaction replacement was disabled in an early version of the Bitcoin software. For several years, Bitcoin full nodes would not allow an unconfirmed transaction containing a particular input (as indicated by its outpoint) to be replaced by a different transaction containing the same input. However, that situation didn't last forever.

Opt-in transaction replacement signaling

After the original sequence-based transaction replacement was disabled due to the potential for abuse, a solution was proposed: programming Bitcoin Core and other relaying full node software to allow a transaction that paid a higher transaction fee rate to replace a conflicting transaction that paid a lower fee rate. This is called *replace by fee*, or *RBF* for short. Some users and businesses objected to adding support for transaction replacement back into Bitcoin Core, so a compromise was reached that once again used the sequence field in support of replacement.

As documented in BIP125, an unconfirmed transaction with any input that has a sequence set to a value below 0xfffffffe (i.e., at least 2 below the maximum value) signals to the network that its signer wants it to be replaceable by a conflicting transaction paying a higher fee rate. Bitcoin Core allowed those unconfirmed transactions to be replaced and continued to disallow other transactions from being replaced. This allowed users and businesses that objected to replacement to simply ignore unconfirmed transactions containing the BIP125 signal until they became confirmed.

There's more to modern transaction replacement policies than fee rates and sequence signals, which we'll see in Replace By Fee (RBF) Fee Bumping.

Sequence as a consensus-enforced relative timelock

In Version, we learned that the BIP68 soft fork added a new constraint to transactions with version numbers 2 or higher. That constraint applies to the sequence field.

Transaction inputs with sequence values less than 2^{31} are interpreted as having a relative timelock. Such a transaction may only be included in the blockchain once the previous output (referenced by the outpoint) has aged by the relative timelock amount. For example, a transaction with one input with a relative timelock of 30 blocks can only be confirmed in a block with at least 29 blocks between it and the block containing the output being spent on the same blockchain. Since sequence is a per-input field, a transaction may contain any number of timelocked inputs, all of which must have sufficiently aged for the transaction to be valid. A disable flag allows a transaction to include both inputs with a relative timelock (sequence $< 2^{31}$) and inputs without a relative timelock (sequence $\geq 2^{31}$).

The sequence value is specified in either blocks or seconds. A type-flag is used to differentiate between values counting blocks and values counting time in seconds. The type-flag is set in the 23rd least-significant bit (i.e., value 1<<22). If the type-flag is set, then the sequence value is interpreted as a multiple of 512 seconds. If the type-flag is not set, the sequence value is interpreted as a number of blocks.

When interpreting sequence as a relative timelock, only the 16 least significant bits are considered. Once the flags (bits 32 and 23) are evaluated, the sequence value is usually "masked" with a 16-bit mask (e.g., sequence & 0x0000FFFF). The multiple of 512 seconds is roughly equal to the average amount of time between blocks, so the maximum relative timelock in both blocks and seconds from 16 bits (2^{16}) is a bit more than one year.

BIP68 definition of sequence encoding (Source: BIP68). shows the binary layout of the sequence value, as defined by BIP68.

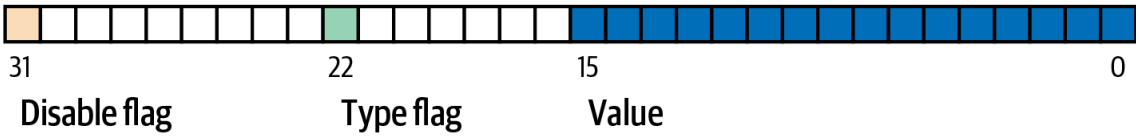


Figure 33. BIP68 definition of sequence encoding (Source: BIP68).

Note that any transaction that sets a relative timelock using sequence also sends the signal for opt-in replace by fee as described in Opt-in transaction replacement signaling.

6.5. Outputs

The outputs field of a transaction contains several fields related to specific outputs. Just as we did with the inputs field, we'll start by looking at the specific bytes of the outputs field from the example transaction where Alice pays Bob, displayed as a map of those bytes in A byte map of the outputs field from Alice's transaction..

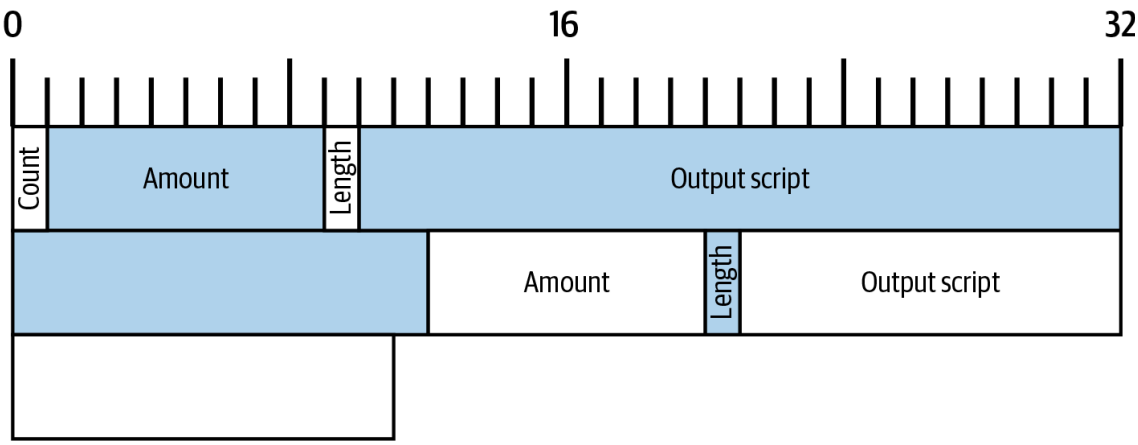


Figure 34. A byte map of the outputs field from Alice's transaction.

6.5.1. Outputs Count

Identical to the start of the inputs section of a transaction, the outputs field begins with a count indicating the number of outputs in this transaction. It's a compactSize integer and must be greater than zero.

The example transaction has two outputs.

6.5.2. Amount

The first field of a specific output is its *amount*, also called "value" in Bitcoin Core. This is an 8-byte signed integer indicating the number of satoshis to transfer. A satoshi is the smallest unit of bitcoin that can be represented in an onchain Bitcoin transaction. There

are 100 million satoshis in a bitcoin.

Bitcoin's consensus rules allow an output to have a value as small as zero and as large as 21 million bitcoins (2.1 quadrillion satoshis).

Uneconomical outputs and disallowed dust

Despite not having any value, a zero-value output can be spent under the same rules as any other output. However, spending an output (using it as the input in a transaction) increases the size of a transaction, which increases the amount of fee that needs to be paid. If the value of the output is less than the cost of the additional fee, then it doesn't make economic sense to spend the output. Such outputs are known as *uneconomical outputs*.

A zero-value output is always an uneconomical output; it wouldn't contribute any value to a transaction spending it even if the transaction's fee rate was zero. However, many other outputs with low values can be uneconomical as well, even unintentionally. For example, at a typical fee rate on the network today, an output might add more value to a transaction than it costs to spend—but tomorrow, fee rates might rise and make the output uneconomical.

The need for full nodes to keep track of all UTXOs, as described in Outpoint, means that every UTXO makes it slightly harder to run a full node. For UTXOs containing significant value, there's an incentive to eventually spend them, so they aren't a problem. But there's no incentive for the person controlling an uneconomical UTXO to ever spend it, potentially making it a perpetual burden on operators of full nodes. Because Bitcoin's decentralization depends on many people being willing to run full nodes, several full node implementations such as Bitcoin Core discourage the creation of uneconomical outputs using policies that affect the relay and mining of unconfirmed transactions.

The policies against relaying or mining transactions creating new uneconomical outputs are called *dust* policies, based on a metaphorical comparison between outputs with very small values and particles with very small size. Bitcoin Core's dust policy is complicated and contains several arbitrary numbers, so many programs we're aware of simply assume outputs with less than 546 satoshis are dust and will not be relayed or mined by default. There are occasionally proposals to lower dust limits, and counterproposals to raise them, so we encourage developers using presigned transactions or multiparty protocols to check whether the policy has changed since publication of this book.

> **Tip** Since Bitcoin's inception, every full node has needed to keep a copy of every UTXO, but that might not always be the case. Several developers have been working on Utreexo, a project that allows full nodes to store a commitment to the set of UTXOs rather than the data itself. A minimal commitment might be only a kilobyte or two in size—compare that to the over five gigabytes Bitcoin Core stores as of this writing.
>
> However, Utreexo will still require some nodes to store all UTXO data, especially nodes serving miners and other operations that need to quickly validate new blocks. That means uneconomical outputs can still be a problem for full nodes even in a possible future where most nodes use Utreexo.

Bitcoin Core's policy rules about dust do have one exception: output scripts starting with OP_RETURN, called *data carrier outputs*, can have a value of zero. The OP_RETURN opcode causes the script to immediately fail no matter what comes after it, so these outputs can never be spent. That means full nodes don't need to keep track of them, a feature Bitcoin Core takes advantage of to allow users to store small amounts of arbitrary data in the blockchain without increasing the size of its UTXO database. Since the outputs are unspendable, they aren't uneconomical—any satoshis assigned to them become permanently unspendable—so allowing the amount to be zero ensures satoshis aren't being destroyed.

6.5.3. Output Scripts

The output amount is followed by a compactSize integer indicating the length of the *output script*, the script that contains the conditions that will need to be fulfilled in order to spend the bitcoins. According to Bitcoin's consensus rules, the minimum size of an output script is zero.

The consensus maximum allowed size of an output script varies depending on when it's being checked. There's no explicit limit on the size of an output script in the output of a transaction, but a later transaction can only spend a previous output with a script of 10,000 bytes or smaller. Implicitly, an output script can be almost as large as the transaction containing it, and a transaction can be almost as large as the block containing it.

> **Tip** An output script with zero length can be spent by an input script containing OP_TRUE. Anyone can create that input script, which means anyone can spend an empty output script. There are an essentially unlimited number of scripts that anyone can spend, and they are known to Bitcoin protocol developers as *anyone can spends*. Upgrades to Bitcoin's script language often take an existing anyone-can-spend script and add new constraints to it, making it only spendable under the new conditions. Application developers should never need to use an anyone-can-spend script, but if you do, we highly recommend that you loudly announce your plans to Bitcoin users and developers so that future upgrades don't accidentally interfere with your system.

Bitcoin Core's policy for relaying and mining transactions effectively limits output scripts to just a few templates, called *standard transaction outputs*. This was originally implemented after the discovery of several early bugs in Bitcoin related to the Script language and is retained in modern Bitcoin Core to support anyone-can-spend upgrades and to encourage the best practice of placing script conditions in P2SH redeem scripts, segwit v0 witness scripts, and segwit v1 (taproot) leaf scripts.

We'll look at each of the current standard transaction templates and learn how to parse scripts in Authorization and Authentication.

6.6. Witness Structure

In court, a witness is someone who testifies that they saw something important happen. Human witnesses aren't always reliable, so courts have various processes for interrogating witnesses to (ideally) only accept evidence from those who are reliable.

Imagine what a witness would look like for a math problem. For example, if the important problem was *x + 2 == 4* and someone claimed they witnessed the solution, what would we ask them? We'd want a mathematical proof that showed a value that could be summed with two to equal four. We could even omit the need for a person and just use the proposed value for *x* as our witness. If we were told that the witness was *two*, then we could fill in the equation, check that it was correct, and decide that the important problem had been solved.

When spending bitcoins, the important problem we want to solve is determining whether the spend was authorized by the person or people who control those bitcoins. The thousands of full nodes that enforce Bitcoin's consensus rules can't interrogate human witnesses, but they can accept *witnesses* that consist entirely of data for solving math problems. For example, a witness of *2* will allow spending bitcoins protected by the following script:

```
2 OP_ADD 4 OP_EQUAL
```

Obviously, allowing your bitcoins to be spent by anyone who can solve a simple equation wouldn't be secure. As we'll see in Digital Signatures, an unforgeable digital signature scheme uses an equation that can only be solved by someone in possession of certain data they're able to keep secret. They're able to reference that secret data using a public identifier. That public identifier is called a *public key* and a solution to the equation is called a *signature*.

The following script contains a public key and an opcode that requires a corresponding signature commit to the data in the spending transaction. Like the number *2* in our simple example, the signature is our witness:

```
<public key> OP_CHECKSIG
```

Witnesses, the values used to solve the math problems that protect bitcoins, need to be included in the transactions where they're used in order for full nodes to verify them. In the legacy transaction format used for all early Bitcoin transactions, signatures and other data are placed in the input script field. However, when developers started to implement contract protocols on Bitcoin, such as we saw in Original sequence-based transaction replacement, they discovered several significant problems with placing witnesses in the input script field.

6.6.1. Circular Dependencies

Many contract protocols for Bitcoin involve a series of transactions that are signed out of order. For example, Alice and Bob want to deposit funds into a script that can only be spent with signatures from both of them, but they each also want to get their money back if the other person becomes unresponsive. A simple solution is to sign transactions out of order:

- Tx_0 pays money from Alice and money from Bob into an output with a script that requires signatures from both Alice and Bob to spend.
- Tx_1 spends the previous output to two outputs, one refunding Alice her money and one refunding Bob his money (minus a small amount for transaction fees).
- If Alice and Bob sign Tx_1 before they sign Tx_0, then they're both guaranteed to be able to get a refund at any time. The protocol doesn't require either of them to trust the other, making it a *trustless protocol*.

A problem with this construction in the legacy transaction format is that every field, including the input script field that contains signatures, is used to derive a transaction's identifier (txid). The txid for Tx_0 is part of the input's outpoint in Tx_1. That means there's no way for Alice and Bob to construct Tx_1 until both signatures for Tx_0 are known—but if they know the signatures for Tx_0, one of them can broadcast that transaction before signing the refund transaction, eliminating the guarantee of a refund. This is a *circular dependency*.

6.6.2. Third-Party Transaction Malleability

A more complex series of transactions can sometimes eliminate a circular dependency, but many protocols will then encounter a new concern: it's often possible to solve the same script in different ways. For example, consider our simple script from Witness Structure:

```
2 OP_ADD 4 OP_EQUAL
```

We can make this script pass by providing the value *2* in an input script, but there are several ways to put that value on the stack in Bitcoin. Here are just a few:

```
OP_2
OP_PUSH1 0x02
OP_PUSH2 0x0002
OP_PUSH3 0x000002
...
OP_PUSHDATA1 0x0102
OP_PUSHDATA1 0x020002
...
OP_PUSHDATA2 0x000102
OP_PUSHDATA2 0x00020002
...
OP_PUSHDATA4 0x0000000102
OP_PUSHDATA4 0x000000020002
...
```

Each alternative encoding of the number *2* in an input script will produce a slightly different transaction with a completely different txid. Each different version of the transaction spends the same inputs (outpoints) as every other version of the transaction, making them all *conflict* with each other. Only one version of a set of conflicting transactions can be contained within a valid blockchain.

Imagine Alice creates one version of the transaction with OP_2 in the input script and an output that pays Bob. Bob then immediately spends that output to Carol. Anyone on the network can replace OP_2 with OP_PUSH1 0x02, creating a conflict with Alice's original version. If that conflicting transaction is confirmed, then there's no way to include Alice's original version in the same blockchain, which means there's no way for Bob's transaction to spend its output. Bob's payment to Carol has been made invalid even though neither Alice, Bob, nor Carol did anything wrong. Someone not involved in the transaction (a third party) was able to change (mutate) Alice's transaction, a problem called *unwanted third-party transaction malleability*.

> **Tip** There are cases when people want their transactions to be malleable and Bitcoin provides several features to support that, most notably the signature hashes (sighash) we'll learn about in Signature Hash Types (SIGHASH). For example, Alice can use a sighash to allow Bob to help her pay some transaction fees. This mutates Alice's transaction but only in a way that Alice wants. For that reason, we will occasionally prefix the word *unwanted* to the term *transaction malleability*. Even when we and other Bitcoin technical writers use the shorter term, we're almost certainly talking about the unwanted variant of malleability.

6.6.3. Second-Party Transaction Malleability

When the legacy transaction format was the only transaction format, developers worked on proposals to minimize third-party malleability, such as BIP62. However, even if they were able to entirely eliminate third-party malleability, users of contract protocols faced another problem: if they required a signature from someone else involved in the protocol, that person could generate alternative signatures and change the txid.

For example, Alice and Bob have deposited their money into a script requiring a signature from both of them to spend. They've also created a refund transaction that allows each of them to get their money back at any time. Alice decides she wants to spend just some of the money, so she cooperates with Bob to create a chain of transactions:

- Tx_0 includes signatures from both Alice and Bob, spending its bitcoins to two outputs. The first output spends some of Alice's money; the second output returns the remainder of the bitcoins back to the script requiring Alice and Bob's signatures. Before signing this transaction, they create a new refund transaction, Tx_1.
- Tx_1 spends the second output of Tx_0 to two new outputs, one to Alice for her share of the joint funds, and one to Bob for his share. Alice and Bob both sign this transaction before they sign Tx_0.

There's no circular dependency here and, if we ignore third-party transaction malleability, this looks like it should provide us with a trustless protocol. However, it's a property of Bitcoin signatures that the signer has to choose a large random number when creating their signature. Choosing a different random number will produce a different signature even if everything being signed stays the same. It's sort of like how, if you provide a handwritten signature for two copies of the same contract, each of those physical signatures will look slightly different.

This mutability of signatures means that, if Alice tries to broadcast Tx_0 (which contains Bob's signature), Bob can generate an alternative signature to create a conflicting transaction with a different txid. If Bob's alternative version of Tx_0 gets confirmed, then

Alice can't use the presigned version of Tx_1 to claim her refund. This type of mutation is called *unwanted second-party transaction malleability*.

6.6.4. Segregated Witness

As early as 2011, protocol developers knew how to solve the problems of circular dependence, third-party malleability, and second-party malleability. The idea was to avoid including the input script in the calculation that produces a transaction's txid. Recall that an abstract name for the data held by an input script is a *witness*. The idea of separating the rest of the data in a transaction from its witness for the purpose of generating a txid is called *segregated witness* (segwit).

The obvious method for implementing segwit requires a change to Bitcoin's consensus rules that would not be compatible with older full nodes, also called a *hard fork*. Hard forks come with a lot of challenges, as we'll discuss further in Hard Forks.

An alternative approach to segwit was described in late 2015. This would use a backward-compatible change to the consensus rules, called a *soft fork*. Backward compatible means that full nodes implementing the change must not accept any blocks that full nodes without the change would consider invalid. As long as they obey that rule, newer full nodes can reject blocks that older full nodes would accept, giving them the ability to enforce new consensus rules (but only if the newer full nodes represent the economic consensus among Bitcoin users—we'll explore the details of upgrading Bitcoin's consensus rules in Mining and Consensus).

The soft fork segwit approach is based on anyone-can-spend output scripts. A script that starts with any of the numbers 0 to 16 and followed by 2 to 40 bytes of data is defined as a segwit output script template. The number indicates its version (e.g., 0 is segwit version 0, or *segwit v0*). The data is called a *witness program*. It's also possible to wrap the segwit template in a P2SH commitment, but we won't deal with that in this chapter.

From the perspective of old nodes, these output script templates can be spent with an empty input script. From the perspective of a new node that is aware of the new segwit rules, any payment to a segwit output script template must only be spent with an empty input script. Notice the difference here: old nodes *allow* an empty input script; new nodes *require* an empty input script.

An empty input script keeps witnesses from affecting the txid, eliminating circular dependencies, third-party transaction malleability, and second-party transaction malleability. But, with no ability to put data in an input script, users of segwit output script templates need a new field. That field is called the *witness structure*.

The introduction of witness programs and the witness structure complicates Bitcoin, but it follows an existing trend of increasing abstraction. Recall from Keys and Addresses that the original Bitcoin whitepaper describes a system where bitcoins were received to public keys (pubkeys) and spent with signatures (sigs). The public key defined who was *authorized* to spend the bitcoins (whoever controlled the corresponding private key) and

the signature provided *authentication* that the spending transaction came from someone who controlled the private key. To make that system more flexible, the initial release of Bitcoin introduced scripts that allow bitcoins to be received to output scripts and spent with input scripts. Later experience with contract protocols inspired allowing bitcoins to be received to witness programs and spent with the witness structure. The terms and fields used in different versions of Bitcoin are shown in [terms_used_authorization_authentication].

Terms used for authorization and authentication data in different parts of Bitcoin

	Authorization	Authentication
Whitepaper	Public key	Signature
Original (Legacy)	Output script	Input script
Segwit	Witness program	Witness structure

6.6.5. Witness Structure Serialization

Similar to the inputs and outputs fields, the witness structure contains other fields, so we'll start with a map of those bytes from Alice's transaction in A byte map of the witness structure from Alice's transaction..

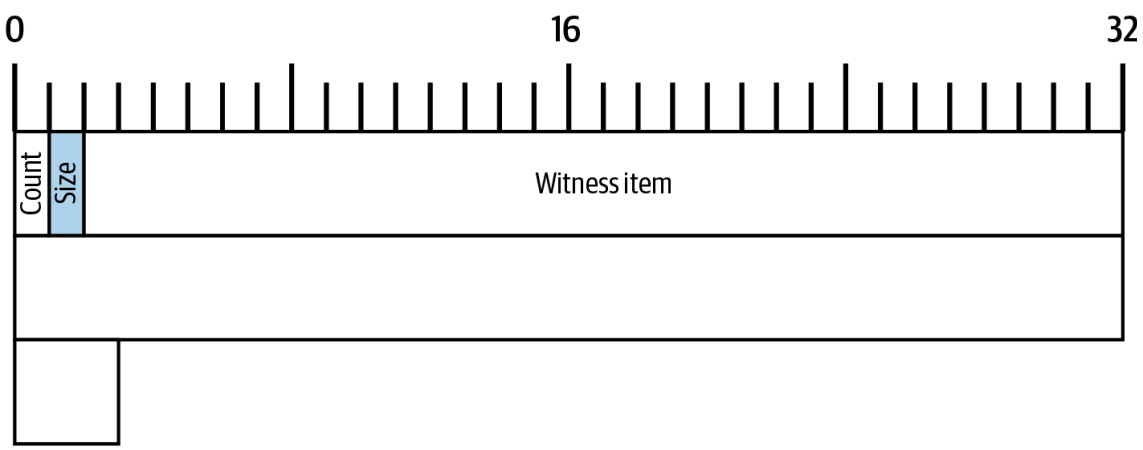


Figure 35. A byte map of the witness structure from Alice's transaction.

Unlike the inputs and outputs fields, the overall witness structure doesn't start with any indication of the total number of witness stacks it contains. Instead, this is implied by the inputs field—there's one witness stack for every input in a transaction.

The witness structure for a particular input does start with a count of the number of elements they contain. Those elements are called *witness items*. We'll explore them in detail in Authorization and Authentication, but for now we need to know that each witness item is prefixed by a compactSize integer indicating its size.

Legacy inputs don't contain any witness items, so their witness stack consists entirely of a count of zero (0x00).

Alice's transaction contains one input and one witness item.

6.7. Lock Time

The final field in a serialized transaction is its lock time. This field was part of Bitcoin's original serialization format, but it was initially only enforced by Bitcoin's policy for choosing which transactions to mine. Bitcoin's earliest known soft fork added a rule that, starting at block height 31,000, forbid the inclusion of a transaction in a block unless it satisfies one of the following rules:

- The transaction indicates that it should be eligible for inclusion in any block by setting its lock time to 0.
- The transaction indicates that it wants to restrict which blocks it can be included in by setting its lock time to a value less than 500,000,000. In this case, the transaction can only be included in a block that has a height equal to the lock time or higher. For example, a transaction with a lock time of 123,456 can be included in block 123,456 or any later block.
- The transaction indicates that it wants to restrict when it can be included in the blockchain by setting its lock time to a value of 500,000,000 or greater. In this case, the field is parsed as epoch time (the number of seconds since 1970-01-01T00:00 UTC) and the transaction can only be included in a block with a *median time past* (MTP) greater than the lock time. MTP is normally about an hour or two behind the current time. The rules for MTP are described in Median Time Past (MTP).

6.8. Coinbase Transactions

The first transaction in each block is a special case. Most older documentation calls this a *generation transaction*, but most newer documentation calls it a *coinbase transaction* (not to be confused with transactions created by the company named "Coinbase").

Coinbase transactions are created by the miner of the block that includes them and gives the miner the option to claim any fees paid by transactions in that block. Additionally, up until block 6,720,000, miners are allowed to claim a subsidy consisting of bitcoins that have never previously been circulated, called the *block subsidy*. The total amount a miner can claim for a block—the combination of fees and subsidy—is called the *block reward*.

Some of the special rules for coinbase transactions include:

- They may only have one input.
- The single input must have an outpoint with a null txid (consisting entirely of zeros) and a maximal output index (0xffffffff). This prevents the coinbase transaction from referencing a previous transaction output, which would (at the very least) be confusing given that the coinbase transaction pays out fees and subsidy.
- The field that would contain an input script in a normal transaction is called a *coinbase*. It's this field that gives the coinbase transaction its name. The coinbase field must be at least two bytes and not longer than 100 bytes. This script is not executed but legacy transaction limits on the number of signature-checking operations (sigops) do apply to it, so any arbitrary data placed in it should be prefixed by a data-pushing opcode. Since a 2013 soft fork defined in BIP34, the first few bytes of this field must follow additional rules we'll describe in Coinbase Data.
- The sum of the outputs must not exceed the value of the fees collected from all the transactions in that block plus the subsidy. The subsidy started at 50 BTC per block and halves every 210,000 blocks (approximately every four years). Subsidy values are rounded down to the nearest satoshi.
- Since the 2017 segwit soft fork documented in BIP141, any block that contains a transaction spending a segwit output must contain an output to the coinbase transaction that commits to all of the transactions in the block (including their witnesses). We'll explore this commitment in Mining and Consensus.

A coinbase transaction can have any other outputs that would be valid in a normal transaction. However, a transaction spending one of those outputs cannot be included in any block until after the coinbase transaction has received 100 confirmations. This is called the *maturity rule*, and coinbase transaction outputs that don't yet have 100 confirmations are called *immature*.

Most Bitcoin software doesn't need to deal with coinbase transactions, but their special nature does mean they can occasionally be the cause of unusual problems in software that's not designed to expect them.

6.9. Weight and Vbytes

Each Bitcoin block is limited in the amount of transaction data it can contain, so most Bitcoin software needs to be able to measure the transactions it creates or processes. The modern unit of measurement for Bitcoin is called *weight*. An alternative version of weight is *vbytes*, where four units of weight equal one vbyte, providing an easy comparison to the original *byte* measurement unit used in legacy Bitcoin blocks.

Blocks are limited to 4 million weight. The block header takes up 240 weight. An additional field, the transaction count, uses either 4 or 12 weight. All of the remaining weight may be used for transaction data.

To calculate the weight of a particular field in a transaction, the size of that serialized field in bytes is multiplied by a factor. To calculate the weight of a transaction, sum together the weights of all of its fields. The factors for each of the fields in a transaction are shown in [weight_factors]. To provide an example, we also calculate the weight of each field in this chapter's example transaction from Alice to Bob.

The factors, and the fields to which they are applied, were chosen to reduce the weight used when spending a UTXO. This helps discourage the creation of uneconomical outputs as described in Uneconomical outputs and disallowed dust.

Weight factors for all fields in a Bitcoin transaction

Field	Factor	Weight in Alice's Tx
Version	4	16
Marker & Flag	1	2
Inputs Count	4	4
Outpoint	4	144
Input script	4	4
Sequence	4	16
Outputs Count	4	4
Amount	4	64 (2 outputs)
Output script	4	232 (2 outputs with different scripts)
Witness Count	1	1
Witness items	1	66
Lock time	4	16
Total	*N/A*	**569**

We can verify our weight calculation by getting the total for Alice's transaction from Bitcoin Core:

```
$ bitcoin-cli getrawtransaction 466200308696215bbc949d5141a49a41\
38ecdfdfaa2a8029c1f9bcecd1f96177 2 | jq .weight
569
```

Alice's transaction from Alice's serialized transaction at the beginning of this chapter is shown represented in weight units in A byte map of Alice's transaction.. You can see the factor at work by comparing the difference in size between the various fields in the two images.

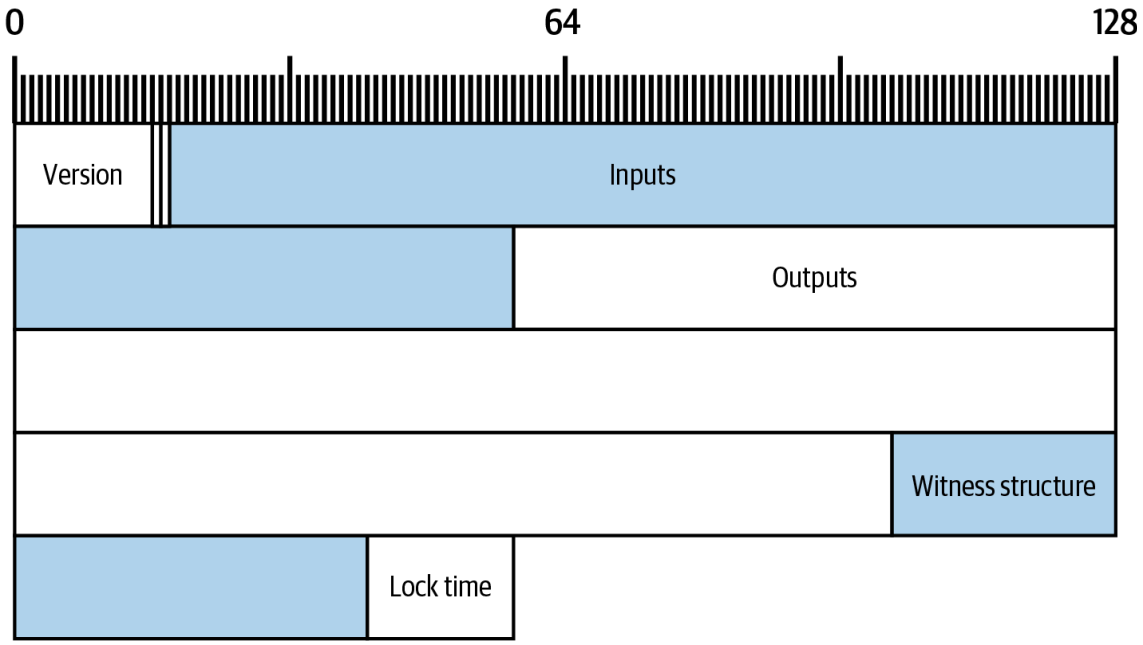


Figure 36. A byte map of Alice's transaction.

6.10. Legacy Serialization

The serialization format described in this chapter is used for the majority of new Bitcoin transactions as of the writing of this book, but an older serialization format is still used for many transactions. That older format, called *legacy serialization*, must be used on the Bitcoin P2P network for any transaction with an empty witness structure (which is only valid if the transaction doesn't spend any witness programs).

Legacy serialization does not include the marker, flag, and witness structure fields.

In this chapter, we looked at each of the fields in a transaction and discovered how they communicate to full nodes the details about the bitcoins to be transferred between users. We only briefly looked at the output script, input script, and witness structure that allow specifying and satisfying conditions that restrict who can spend what bitcoins. Understanding how to construct and use these conditions is essential to ensuring that only Alice can spend her bitcoins, so they will be the subject of the next chapter.

7. Authorization and Authentication

When you receive bitcoins, you have to decide who will have permission to spend them, called *authorization*. You also have to decide how full nodes will distinguish the authorized spenders from everyone else, called *authentication*. Your authorization instructions and the spender proof of authentication will be checked by thousands of independent full nodes, which all need to come to the same conclusion that a spend was authorized and authenticated in order for the transaction containing it to be valid.

The original description of Bitcoin used a public key for authorization. Alice paid Bob by putting his public key in the output of a transaction. Authentication came from Bob in the form of a signature that committed to a spending transaction, such as from Bob to Carol.

The actual version of Bitcoin that was originally released provided a more flexible mechanism for both authorization and authentication. Improvements since then have only increased that flexibility. In this chapter, we'll explore those features and see how they're most commonly used.

7.1. Transaction Scripts and Script Language

The original version of Bitcoin introduced a new programming language called *Script*, a Forth-like stack-based language. Both the script placed in an output and the legacy input script used in a spending transaction are written in this scripting language.

Script is a very simple language. It requires minimal processing and cannot easily do many of the fancy things modern programming languages can do.

When legacy transactions were the most commonly used type of transaction, the majority of transactions processed through the Bitcoin network had the form "Payment to Bob's Bitcoin address" and used a script called a pay to public key hash (P2PKH) script. However, Bitcoin transactions are not limited to the "Payment to Bob's Bitcoin address" script. In fact, scripts can be written to express a vast variety of complex conditions. In order to understand these more complex scripts, we must first understand the basics of transaction scripts and Script language.

In this section, we will demonstrate the basic components of the Bitcoin transaction scripting language and show how it can be used to express conditions for spending and how those conditions can be satisfied.

> **Tip** Bitcoin transaction validation is not based on a static pattern but instead is achieved through the execution of a scripting language. This language allows for a nearly infinite variety of conditions to be expressed.

7.1.1. Turing Incompleteness

The Bitcoin transaction script language contains many operators, but is deliberately limited in one important way—there are no loops or complex flow control capabilities other than conditional flow control. This ensures that the language is not *Turing Complete*, meaning that scripts have limited complexity and predictable execution times. Script is not a general-purpose language. These limitations ensure that the language cannot be used to create an infinite loop or other form of "logic bomb" that could be embedded in a transaction in a way that causes a denial-of-service attack against the Bitcoin network. Remember, every transaction is validated by every full node on the Bitcoin network. A limited language prevents the transaction validation mechanism from being used as a vulnerability.

7.1.2. Stateless Verification

The Bitcoin transaction script language is stateless, in that there is no state prior to execution of the script or state saved after execution of the script. All the information needed to execute a script is contained within the script and the transaction executing the script. A script will predictably execute the same way on any system. If your system verified a script, you can be sure that every other system in the Bitcoin network will also verify the script, meaning that a valid transaction is valid for everyone and everyone knows this. This predictability of outcomes is an essential benefit of the Bitcoin system.

7.1.3. Script Construction

Bitcoin’s legacy transaction validation engine relies on two parts of scripts to validate transactions: an output script and an input script.

An output script specifies the conditions that must be met to spend the output in the future, such as who is authorized to spend the output and how they will be authenticated.

An input script is a script that satisfies the conditions placed in an output script and allows the output to be spent. Input scripts are part of every transaction input. Most of the time in legacy transactions they contain a digital signature produced by the user’s wallet from his or her private key, but not all input scripts must contain signatures.

Every Bitcoin validating node will validate transactions by executing the output and input scripts. As we saw in Transactions, each input contains an outpoint that refers to a previous transaction output. The input also contains an input script. The validation software will copy the input script, retrieve the UTXO referenced by the input, and copy

the output script from that UTXO. The input and output scripts are then executed together. The input is valid if the input script satisfies the output script's conditions (see Separate execution of output and input scripts). All the inputs are validated independently as part of the overall validation of the transaction.

Note that the preceding steps involve making copies of all data. The original data in the previous output and current input is never changed. In particular, the previous output is invariable and unaffected by failed attempts to spend it. Only a valid transaction that correctly satisfies the conditions of the output script results in the output being considered as "spent."

Combining input and output scripts to evaluate a transaction script. is an example of the output and input scripts for the most common type of legacy Bitcoin transaction (a payment to a public key hash), showing the combined script resulting from the concatenation of the scripts prior to validation.

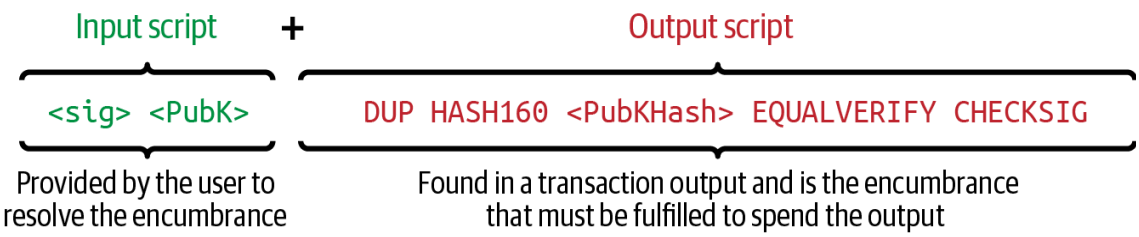


Figure 37. Combining input and output scripts to evaluate a transaction script.

The script execution stack

Bitcoin's scripting language is called a stack-based language because it uses a data structure called a *stack*. A stack is a very simple data structure that can be visualized as a stack of cards. A stack has two base operations: push and pop. Push adds an item on top of the stack. Pop removes the top item from the stack.

The scripting language executes the script by processing each item from left to right. Numbers (data constants) are pushed onto the stack. Operators push or pop one or more parameters from the stack, act on them, and might push a result onto the stack. For example, OP_ADD will pop two items from the stack, add them, and push the resulting sum onto the stack.

Conditional operators evaluate a condition, producing a boolean result of TRUE or FALSE. For example, OP_EQUAL pops two items from the stack and pushes TRUE (TRUE is represented by the number 1) if they are equal or FALSE (represented by 0) if they are not equal. Bitcoin transaction scripts usually contain a conditional operator so that they can produce the TRUE result that signifies a valid transaction.

A simple script

Now let's apply what we've learned about scripts and stacks to some simple examples.

As we will see in Bitcoin's script validation doing simple math., the script 2 3 OP_ADD 5 OP_EQUAL demonstrates the arithmetic addition operator OP_ADD, adding two numbers and putting the result on the stack, followed by the conditional operator OP_EQUAL, which checks that the resulting sum is equal to 5. For brevity, the OP_ prefix may sometimes be omitted in examples in this book. For more details on the available script operators and functions, see Bitcoin Wiki's script page.

Although most legacy output scripts refer to a public key hash (essentially, a legacy Bitcoin address), thereby requiring proof of ownership to spend the funds, the script does not have to be that complex. Any combination of output and input scripts that results in a TRUE value is valid. The simple arithmetic we used as an example of the scripting language is also a valid script.

Use part of the arithmetic example script as the output script:

```
3 OP_ADD 5 OP_EQUAL
```

which can be satisfied by a transaction containing an input with the input script:

```
2
```

The validation software combines the scripts:

```
2 3 OP_ADD 5 OP_EQUAL
```

As we see in Bitcoin's script validation doing simple math., when this script is executed, the result is OP_TRUE, making the transaction valid. Although this is a valid transaction output script, note that the resulting UTXO can be spent by anyone with the arithmetic skills to know that the number 2 satisfies the script.

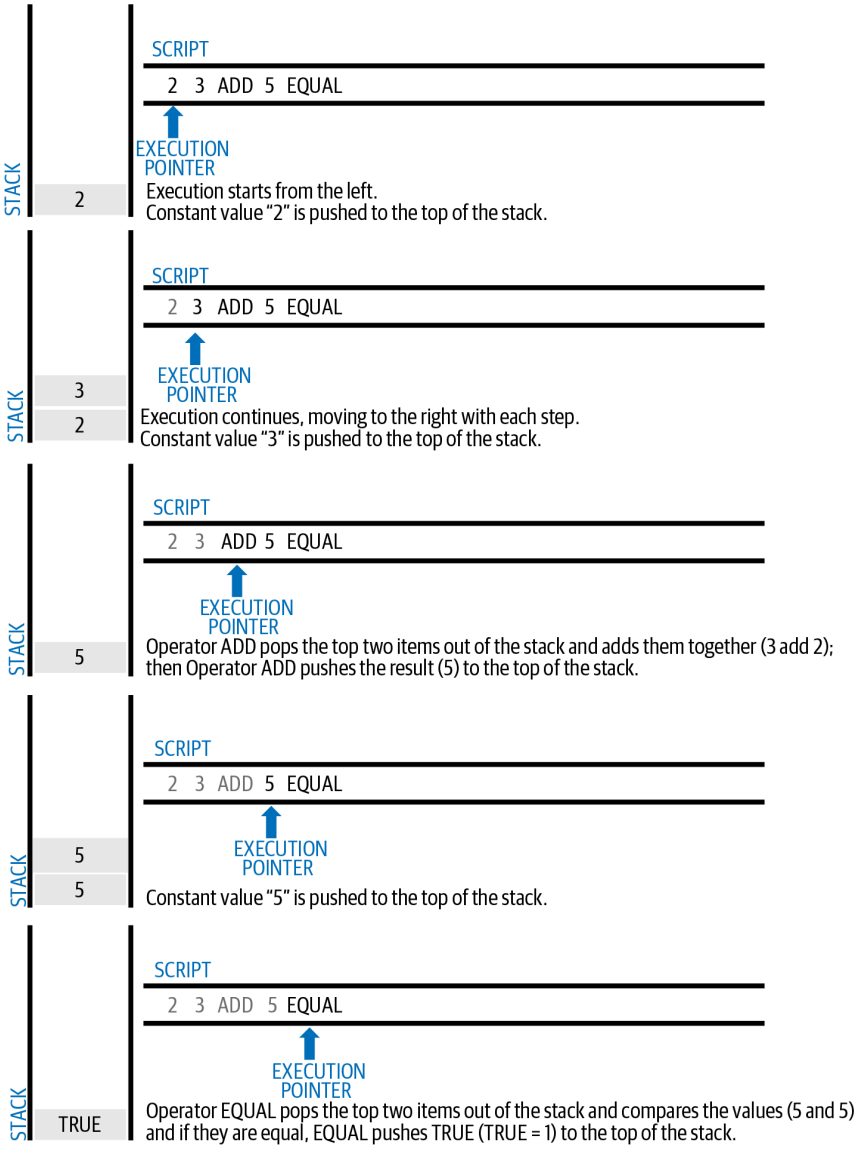


Figure 38. Bitcoin's script validation doing simple math.

> **Tip** Transactions are valid if the top result on the stack is TRUE, which is any nonzero value. Transactions are invalid if the top value on the stack is FALSE (the value zero or an empty stack), the script execution is halted explicitly by an operator (such as VERIFY, OP_RETURN), or the script was not semantically valid (such as containing an OP_IF statement that was not terminated by an OP_ENDIF opcode). For details, see Bitcoin Wiki's script page.

The following is a slightly more complex script, which calculates 2 + 7 – 3 + 1. Notice that

when the script contains several operators in a row, the stack allows the results of one operator to be acted upon by the next operator:

```
2 7 OP_ADD 3 OP_SUB 1 OP_ADD 7 OP_EQUAL
```

Try validating the preceding script yourself using pencil and paper. When the script execution ends, you should be left with a TRUE value on the stack.

Separate execution of output and input scripts

In the original Bitcoin client, output and input scripts were concatenated and executed in sequence. For security reasons, this was changed in 2010 because of a vulnerability known as the 1 OP_RETURN bug. In the current implementation, the scripts are executed separately with the stack transferred between the two executions.

First, the input script is executed using the stack execution engine. If the input script is executed without errors and has no operations left over, the stack is copied and the output script is executed. If the result of executing the output script with the stack data copied from the input script is TRUE, the input script has succeeded in resolving the conditions imposed by the output script and, therefore, the input is a valid authorization to spend the UTXO. If any result other than TRUE remains after execution of the combined script, the input is invalid because it has failed to satisfy the spending conditions placed on the output.

7.1.4. Pay to Public Key Hash

A pay to public key hash (P2PKH) script uses an output script that contains a hash that commits to a public key. P2PKH is best known as the basis for a legacy Bitcoin address. A P2PKH output can be spent by presenting a public key that matches the hash commitment and a digital signature created by the corresponding private key (see Digital Signatures). Let’s look at an example of a P2PKH output script:

```
OP_DUP OP_HASH160 <Key Hash> OP_EQUALVERIFY OP_CHECKSIG
```

The Key Hash is the data that would be encoded into a legacy base58check address. Most applications would show the *public key hash* in a script using hexadecimal encoding and not the familiar Bitcoin address base58check format that begins with a "1."

The preceding output script can be satisfied with an input script of the form:

```
<Signature> <Public Key>
```

The two scripts together would form the following combined validation script:

```
<Sig> <Pubkey> OP_DUP OP_HASH160 <Hash> OP_EQUALVERIFY OP_CHECKSIG
```

The result will be TRUE if the input script has a valid signature from Bob's private key that corresponds to the public key hash set as an encumbrance.

Figures #P2PubKHash1 and #P2PubKHash2 show (in two parts) a step-by-step execution of the combined script, which will prove this is a valid transaction.

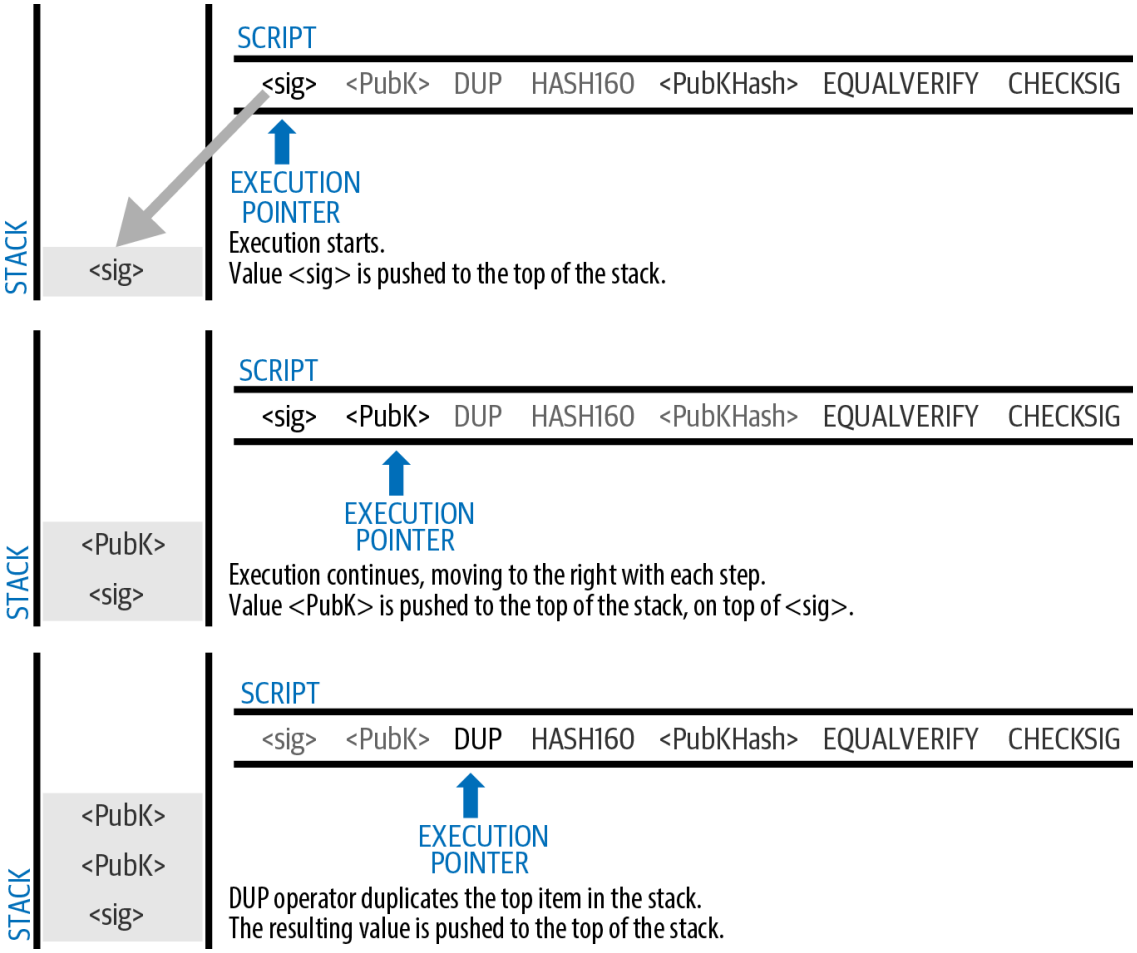


Figure 39. Evaluating a script for a P2PKH transaction (part 1 of 2).

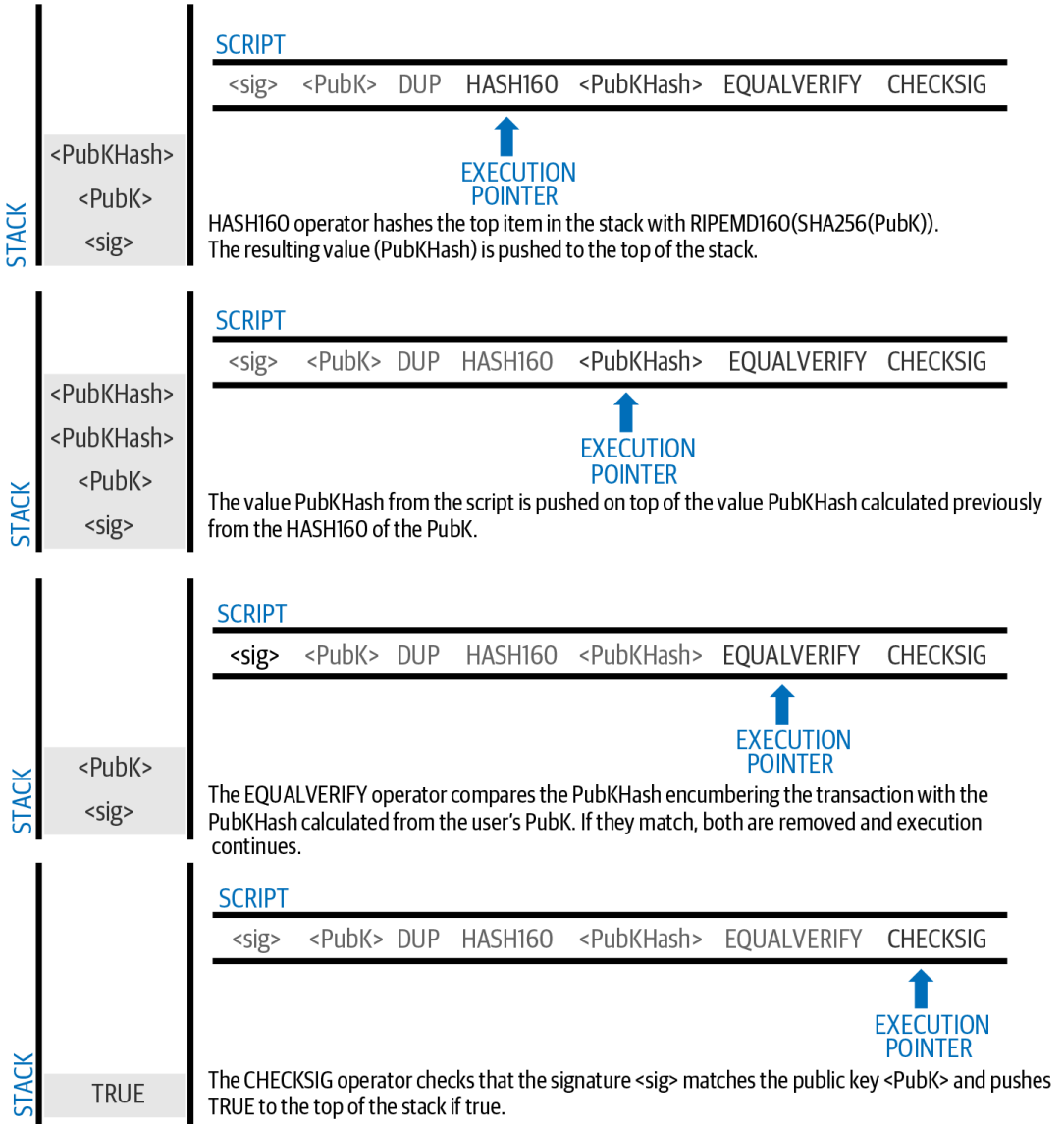


Figure 40. Evaluating a script for a P2PKH transaction (part 2 of 2).

7.2. Scripted Multisignatures

Multisignature scripts set a condition where *k* public keys are recorded in the script and at least *t* of those must provide signatures to spend the funds, called *t*-of-*k*. For example, a 2-of-3 multisignature is one where three public keys are listed as potential signers and at least two of those must be used to create signatures for a valid transaction to spend the funds.

> **Tip** Some Bitcoin documentation, including earlier editions of this book, uses the term "m-of-n" for a traditional multisignature. However, it's hard to tell "m" and "n" apart when they're spoken, so we use the alternative *t*-of-*k*. Both phrases refer to the same type of signature scheme.

The general form of an output script setting a *t*-of-*k* multisignature condition is:

```
t <Public Key 1> <Public Key 2> ... <Public Key k> k OP_CHECKMULTISIG
```

where *k* is the total number of listed public keys and *t* is the threshold of required signatures to spend the output.

An output script setting a 2-of-3 multisignature condition looks like this:

```
2 <Public Key A> <Public Key B> <Public Key C> 3 OP_CHECKMULTISIG
```

The preceding output script can be satisfied with an input script containing signatures:

```
<Signature B> <Signature C>
```

or any combination of two signatures from the private keys corresponding to the three listed public keys.

The two scripts together would form the combined validation script:

```
<Sig B> <Sig C> 2 <Pubkey A> <Pubkey B> <Pubkey C> 3 OP_CHECKMULTISIG
```

When executed, this combined script will evaluate to TRUE if the input script has two valid signatures from private keys that correspond to two of the three public keys set as an encumbrance.

At this time, Bitcoin Core's transaction relay policy limits multisignature output scripts to, at most, three listed public keys, meaning you can do anything from a 1-of-1 to a 3-of-3 multisignature or any combination within that range. You may want to check the IsStandard() function to see what is currently accepted by the network. Note that the limit of three keys applies only to standard (also known as "bare") multisignature scripts, not to scripts wrapped in another structure like P2SH, P2WSH, or P2TR. P2SH multisignature scripts are limited by both policy and consensus to 15 keys, allowing for up to a 15-of-15 multisignature. We will learn about P2SH in Pay to Script Hash. All other scripts are consensus limited to 20 keys per OP_CHECKMULTISIG or OP_CHECKMULTISIGVERIFY opcode, although one script may include multiple of those opcodes.

7.2.1. An Oddity in CHECKMULTISIG Execution

There is an oddity in OP_CHECKMULTISIG's execution that requires a slight workaround. When OP_CHECKMULTISIG executes, it should consume $t + k + 2$ items on the stack as parameters. However, due to the oddity, OP_CHECKMULTISIG will pop an extra value or one value more than expected.

Let's look at this in greater detail using the previous validation example:

```
<Sig B> <Sig C> 2 <Pubkey A> <Pubkey B> <Pubkey C> 3 OP_CHECKMULTISIG
```

First, OP_CHECKMULTISIG pops the top item, which is k (in this example "3"). Then it pops k items, which are the public keys that can sign; in this example, public keys A, B, and C. Then, it pops one item, which is t, the quorum (how many signatures are needed). Here $t = 2$. At this point, OP_CHECKMULTISIG should pop the final t items, which are the signatures, and see if they are valid. However, unfortunately, an oddity in the implementation causes OP_CHECKMULTISIG to pop one more item ($t + 1$ total) than it should. The extra item is called the *dummy stack element*, and it is disregarded when checking the signatures so it has no direct effect on OP_CHECKMULTISIG itself. However, the dummy element must be present because, if it isn't present when OP_CHECKMULTISIG attempts to pop on an empty stack, it will cause a stack error and script failure (marking the transaction as invalid). Because the dummy element is disregarded, it can be anything. It became the custom early on to use OP_0, which later became a relay policy rule and eventually a consensus rule (with the enforcement of BIP147).

Because popping the dummy element is part of the consensus rules, it must now be replicated forever. Therefore a script should look like this:

```
OP_0 <Sig B> <Sig C> 2 <Pubkey A> <Pubkey B> <Pubkey C> 3 OP_CHECKMULTISIG
```

Thus the input script actually used in multisig is not:

```
<Signature B> <Signature C>
```

but instead it is:

```
OP_0 <Sig B> <Sig C>
```

Some people believe this oddity was a bug in the original code for Bitcoin, but a plausible alternative explanation exists. Verifying t-of-k signatures can require many more than t or k signature checking operations. Let's consider a simple example of 1-in-5, with the following combined script:

```
<dummy> <Sig4> 1 <key0> <key1> <key2> <key3> <key4> 5 OP_CHECKMULTISIG
```

The signature is checked first against key0, then key1, and then the other keys before it is finally compared to its corresponding key4. That means five signature checking operations need to be performed even though there's only one signature. One way to eliminate this redundancy would have been to provide OP_CHECKMULTISIG a map indicating which provided signature corresponds to which public key, allowing the OP_CHECKMULTISIG operation to only perform exactly *t* signature-checking operations. It's possible that Bitcoin's original developer added the extra element (which we now call the dummy stack element) in the original version of Bitcoin so they could add the feature for allowing a map to be passed in a later soft fork. However, that feature was never implemented, and the BIP147 update to the consensus rules in 2017 makes it impossible to add that feature in the future.

Only Bitcoin's original developer could tell us whether the dummy stack element was the result of a bug or a plan for a future upgrade. In this book, we simply call it an oddity.

From now on, if you see a multisig script, you should expect to see an extra OP_0 in the beginning, whose only purpose is as a workaround to an oddity in the consensus rules.

7.3. Pay to Script Hash

Pay to script hash (P2SH) was introduced in 2012 as a powerful new type of operation that greatly simplifies the use of complex scripts. To explain the need for P2SH, let's look at a practical example.

Mohammed is an electronics importer based in Dubai. Mohammed's company uses Bitcoin's multisignature feature extensively for its corporate accounts. Multisignature scripts are one of the most common uses of Bitcoin's advanced scripting capabilities and are a very powerful feature. Mohammed's company uses a multisignature script for all customer payments. Any payments made by customers are locked in such a way that they require at least two signatures to release. Mohammed, his three partners, and their attorney can each provide one signature. A multisignature scheme like that offers corporate governance controls and protects against theft, embezzlement, or loss.

The resulting script is quite long and looks like this:

```
2 <Mohammed's Public Key> <Partner1 Public Key> <Partner2 Public Key>
<Partner3 Public Key> <Attorney Public Key> 5 OP_CHECKMULTISIG
```

Although multisignature scripts are a powerful feature, they are cumbersome to use. Given the preceding script, Mohammed would have to communicate this script to every customer prior to payment. Each customer would have to use special Bitcoin wallet software with the ability to create custom transaction scripts. Furthermore, the resulting transaction would be about five times larger than a simple payment transaction, because this script contains very long public keys. The burden of that extra data would be borne by the

customer in the form of extra transaction fees. Finally, a large transaction script like this would be carried in the UTXO set in every full node, until it was spent. All of these issues make using complex output scripts difficult in practice.

P2SH was developed to resolve these practical difficulties and to make the use of complex scripts as easy as a payment to a single-key Bitcoin address. With P2SH payments, the complex script is replaced with a commitment, the digest of a cryptographic hash. When a transaction attempting to spend the UTXO is presented later, it must contain the script that matches the commitment in addition to the data that satisfies the script. In simple terms, P2SH means "pay to a script matching this hash, a script that will be presented later when this output is spent."

In P2SH transactions, the script that is replaced by a hash is referred to as the *redeem script* because it is presented to the system at redemption time rather than as an output script. [without_p2sh] shows the script without P2SH and [with_p2sh] shows the same script encoded with P2SH.

Complex script without P2SH

Output script	2 PubKey1 PubKey2 PubKey3 PubKey4 PubKey5 5 OP_CHECKMULTISIG
Input script	Sig1 Sig2

Complex script as P2SH

Redeem script	2 PubKey1 PubKey2 PubKey3 PubKey4 PubKey5 5 OP_CHECKMULTISIG
Output script	OP_HASH160 <20-byte hash of redeem script> OP_EQUAL
Input script	Sig1 Sig2 <redeem script>

As you can see from the tables, with P2SH, the complex script that details the conditions for spending the output (redeem script) is not presented in the output script. Instead, only a hash of it is in the output script, and the redeem script itself is presented later as part of the input script when the output is spent. This shifts the burden in fees and complexity from the spender to the receiver of the transaction.

Let’s look at Mohammed’s company, the complex multisignature script, and the resulting P2SH scripts.

First, the multisignature script that Mohammed’s company uses for all incoming payments from customers:

```
2 <Mohammed's Public Key> <Partner1 Public Key> <Partner2 Public Key>
<Partner3 Public Key> <Attorney Public Key> 5 OP_CHECKMULTISIG
```

This entire script can instead be represented by a 20-byte cryptographic hash by first applying the SHA256 hashing algorithm and then applying the RIPEMD-160 algorithm on the result. For example, starting with the hash of Mohammed's redeem script:

```
54c557e07dde5bb6cb791c7a540e0a4796f5e97e
```

A P2SH transaction locks the output to this hash instead of the longer redeem script, using a special output script template:

```
OP_HASH160 54c557e07dde5bb6cb791c7a540e0a4796f5e97e OP_EQUAL
```

which, as you can see, is much shorter. Instead of "pay to this 5-key multisignature script," the P2SH equivalent transaction is "pay to a script with this hash." A customer making a payment to Mohammed's company need only include this much shorter output script in his payment. When Mohammed and his partners want to spend this UTXO, they must present the original redeem script (the one whose hash locked the UTXO) and the signatures necessary to unlock it, like this:

```
<Sig1> <Sig2> <2 PK1 PK2 PK3 PK4 PK5 5 OP_CHECKMULTISIG>
```

The two scripts are combined in two stages. First, the redeem script is checked against the output script to make sure the hash matches:

```
<2 PK1 PK2 PK3 PK4 PK5 5 OP_CHECKMULTISIG> OP_HASH160 <script hash> OP_EQUAL
```

If the redeem script hash matches, the redeem script is executed:

```
<Sig1> <Sig2> 2 <PK1> <PK2> <PK3> <PK4> <PK5> 5 OP_CHECKMULTISIG
```

7.3.1. P2SH Addresses

Another important part of the P2SH feature is the ability to encode a script hash as an address, as defined in BIP13. P2SH addresses are base58check encodings of the 20-byte hash of a script, just like Bitcoin addresses are base58check encodings of the 20-byte hash of a public key. P2SH addresses use the version prefix "5," which results in base58check-encoded addresses that start with a "3."

For example, Mohammed's complex script, hashed and base58check-encoded as a P2SH address, becomes 39RF6JqABiHdYHkfChV6USGMe6Nsr66Gzw.

Now, Mohammed can give this "address" to his customers, and they can use almost any Bitcoin wallet to make a simple payment, like any other Bitcoin address. The 3 prefix gives them a hint that this is a special type of address, one corresponding to a script instead of a public key, but otherwise it works in exactly the same way as a payment to any other Bitcoin address.

P2SH addresses hide all of the complexity so the person making a payment does not see the script.

7.3.2. Benefits of P2SH

The P2SH feature offers the following benefits compared to the direct use of complex scripts in outputs:

- The similarity to original legacy addresses means the sender and the sender's wallet don't need complex engineering to implement P2SH.
- P2SH shifts the burden in data storage for the long script from the output (which additionally to being stored on the blockchain is in the UTXO set) to the input (only stored on the blockchain).
- P2SH shifts the burden in data storage for the long script from the present time (payment) to a future time (when it is spent).
- P2SH shifts the transaction fee cost of a long script from the sender to the recipient, who has to include the long redeem script to spend it.

7.3.3. Redeem Script and Validation

You are not able to put a P2SH inside a P2SH redeem script because the P2SH specification is not recursive. Also, while it is technically possible to include OP_RETURN (see Data Recording Output (OP_RETURN)) in a redeem script, as nothing in the rules prevents you from doing so, it is of no practical use because executing OP_RETURN during validation will cause the transaction to be marked invalid.

Note that because the redeem script is not presented to the network until you attempt to spend a P2SH output, if you create an output with the hash of an invalid redeem script, you will not be able to spend it. The spending transaction, which includes the redeem script, will not be accepted because it is an invalid script. This creates a risk because you can send bitcoin to a P2SH address that cannot be spent later.

> **Warning** P2SH output scripts contain the hash of a redeem script, which gives no clues as to the content of the redeem script. The P2SH output will be considered valid and accepted even if the redeem script is invalid. You might accidentally receive bitcoin in such a way that it cannot later be spent.

7.4. Data Recording Output (OP_RETURN)

Bitcoin's distributed and timestamped blockchain has potential uses beyond payments. Many developers have tried to use the transaction scripting language to take advantage of the security and resilience of the system for applications such as digital notary services.

Early attempts to use Bitcoin's script language for these purposes involved creating transaction outputs that recorded data on the blockchain; for example, to record a commitment to a file in such a way that anyone could establish proof-of-existence of that file on a specific date by reference to that transaction.

The use of Bitcoin's blockchain to store data unrelated to Bitcoin payments is a controversial subject. Many people consider such use abusive and want to discourage it. Others view it as a demonstration of the powerful capabilities of blockchain technology and want to encourage such experimentation. Those who object to the inclusion of nonpayment data argue that it burdens those running full Bitcoin nodes with carrying the cost of disk storage for data that the blockchain was not intended to carry. Moreover, such transactions may create UTXOs that cannot be spent, using a legacy Bitcoin address as a freeform 20-byte field. Because the address is used for data, it doesn't correspond to a private key and the resulting UTXO can *never* be spent; it's a fake payment. These transactions that can never be spent are therefore never removed from the UTXO set and cause the size of the UTXO database to forever increase, or "bloat."

A compromise was reached that allows an output script starting with OP_RETURN to add nonpayment data to a transaction output. However, unlike the use of "fake" UTXOs, the OP_RETURN operator creates an explicitly *provably unspendable* output, which does not need to be stored in the UTXO set. OP_RETURN outputs are recorded on the blockchain, so they consume disk space and contribute to the increase in the blockchain's size, but they are not stored in the UTXO set and therefore do not bloat full nodes with the cost of more expensive database operations.

OP_RETURN scripts look like this:

```
OP_RETURN <data>
```

The data portion often represents a hash, such as the output from the SHA256 algorithm (32 bytes). Some applications put a prefix in front of the data to help identify the application. For example, the Proof of Existence digital notarization service uses the 8-byte prefix DOCPROOF, which is ASCII encoded as 44 4f 43 50 52 4f 4f 46 in hexadecimal.

Keep in mind that there is no input script that corresponds to OP_RETURN that could possibly be used to "spend" an OP_RETURN output. The whole point of an OP_RETURN output is that you can't spend the money locked in that output, and therefore it does not need to be held in the UTXO set as potentially spendable: OP_RETURN outputs are *provably unspendable*. OP_RETURN outputs usually have a zero amount because any bitcoins assigned to such an output are effectively lost forever. If an OP_RETURN output is referenced as an input in a transaction, the script validation engine will halt the execution of the validation script and mark the transaction as invalid. The execution of OP_RETURN essentially causes the script to "RETURN" with a FALSE and halt. Thus, if you accidentally reference an OP_RETURN output as an input in a transaction, that transaction is invalid.

7.4.1. Transaction Lock Time Limitations

Use of the lock time allows a spender to restrict a transaction from being included in a block until a specific block height, but it does not prevent spending the funds in another transaction earlier than that. Let's explain that with the following example.

Alice signs a transaction spending one of her outputs to Bob's address and sets the transaction lock time to 3 months in the future. Alice sends that transaction to Bob to hold. With this transaction Alice and Bob know that:

- Bob cannot transmit the transaction to redeem the funds until 3 months have elapsed.
- Bob may transmit the transaction after 3 months.

However:

- Alice can create a conflicting transaction, spending the same inputs without a lock time. Thus, Alice can spend the same UTXO before the 3 months have elapsed.
- Bob has no guarantee that Alice won't do that.

It is important to understand the limitations of transaction lock time. The only guarantee is that Bob will not be able to redeem the presigned transaction before 3 months have elapsed. There is no guarantee that Bob will get the funds. One way to guarantee that Bob will receive the funds but cannot spend them until 3 months have elapsed is to place the timelock restriction on the UTXO itself as part of the script, rather than on the transaction. This is achieved by the next form of timelock, called Check Lock Time Verify.

7.4.2. Check Lock Time Verify (OP_CLTV)

In December 2015, a new form of timelock was introduced to Bitcoin as a soft fork upgrade. Based on a specification in BIP65, a new script operator called OP_CHECKLOCKTIMEVERIFY (OP_CLTV) was added to the scripting language. OP_CLTV is a per-output timelock rather than a per-transaction timelock, as is the case with lock time. This allows for additional flexibility in the way timelocks are applied.

In simple terms, by committing to the OP_CLTV opcode in an output, that output is restricted so that it can only be spent after the specified time has elapsed.

OP_CLTV doesn't replace lock time, but rather restricts specific UTXOs such that they can only be spent in a future transaction with lock time set to a greater or equal value.

The OP_CLTV opcode takes one parameter as input, expressed as a number in the same format as lock time (either a block height or Unix epoch time). As indicated by the VERIFY suffix, OP_CLTV is the type of opcode that halts execution of the script if the outcome is FALSE. If it results in TRUE, execution continues.

In order to use OP_CLTV, you insert it into the redeem script of the output in the transaction that creates the output. For example, if Alice is paying Bob, he might usually accept payment to the following P2SH script:

```
<Bob's public key> OP_CHECKSIG
```

To lock it to a time, say 3 months from now, his P2SH script would instead be:

```
<Bob's pubkey> OP_CHECKSIGVERIFY <now + 3 months> OP_CHECKLOCKTIMEVERIFY
```

where <now {plus} 3 months> is a block height or time value estimated 3 months from the time the transaction is mined: current block height + 12,960 (blocks) or current Unix epoch time + 7,760,000 (seconds).

When Bob tries to spend this UTXO, he constructs a transaction that references the UTXO as an input. He uses his signature and public key in the input script of that input and sets the transaction lock time to be equal or greater to the timelock in the OP_CHECKLOCKTIMEVERIFY Alice set. Bob then broadcasts the transaction on the Bitcoin network.

Bob's transaction is evaluated as follows. If the OP_CHECKLOCKTIMEVERIFY parameter Alice set is less than or equal to the spending transaction's lock time, script execution continues (acts as if a *no operation* or OP_NOP opcode was executed). Otherwise, script execution halts and the transaction is deemed invalid.

More precisely, BIP65 explains that OP_CHECKLOCKTIMEVERIFY fails and halts execution if one of the following occurs:

- The stack is empty.
- The top item on the stack is less than 0.
- The lock-time type (height versus timestamp) of the top stack item and the lock time field are not the same.
- The top stack item is greater than the transaction's lock time field.
- The sequence field of the input is 0xffffffff.

Timelock Conflicts
OP_CLTV and lock time use the same format to describe timelocks, either a block height or the time elapsed in seconds since the Unix epoch. Critically, when used together, the format of lock time must match that of OP_CLTV in the outputs—they must both reference either block height or time in seconds.

This implies that a script can never be valid if it must execute two different calls to OP_CLTV, one that uses a height and one that uses a time. It can be easy to make this mistake when writing advanced scripts, so be sure to thoroughly test your scripts on a test network or use a tool designed to prevent this issue, like a Miniscript compiler.

An additional implication is that only one variety of OP_CLTV can be used in any of the scripts of a transaction. If the script for one input uses the height variety and a different script for a different input uses the time variety, there is no way to construct a valid transaction that spends both inputs.

After execution, if OP_CLTV is satisfied, the parameter that preceded it remains as the top item on the stack and may need to be dropped, with OP_DROP, for correct execution of subsequent script opcodes. You will often see OP_CHECKLOCKTIMEVERIFY followed by OP_DROP in scripts for this reason. OP_CLTV, like OP_CSV (see Relative Timelocks) are unlike other CHECKVERIFY opcodes in leaving items on the stack because the soft forks that added them redefined existing opcodes that did not drop stack items, and the behavior of those previous no-operation (NOP) opcodes must be preserved.

By using lock time in conjunction with OP_CLTV, the scenario described in Transaction Lock Time Limitations changes. Alice sends her transaction immediately, assigning the funds to Bob's key. Alice can no longer spend the money, but Bob cannot spend it before the 3-month lock time has expired.

By introducing timelock functionality directly into the scripting language, OP_CLTV allows us to develop some very interesting complex scripts.

The standard is defined in BIP65 (OP_CHECKLOCKTIMEVERIFY).

7.4.3. Relative Timelocks

Lock time and OP_CLTV are both *absolute timelocks* in that they specify an absolute point in time. The next two timelock features we will examine are *relative timelocks* in that they specify, as a condition of spending an output, an elapsed time from the confirmation of the output in the blockchain.

Relative timelocks are useful because they allow imposing a time constraint on one transaction that is dependent on the elapsed time from the confirmation of a previous transaction. In other words, the clock doesn't start counting until the UTXO is recorded on the blockchain. This functionality is especially useful in bidirectional state channels and Lightning Networks (LNs), as we will see in Payment Channels and State Channels.

Relative timelocks, like absolute timelocks, are implemented with both a transaction-level feature and a script-level opcode. The transaction-level relative timelock is implemented as a consensus rule on the value of sequence, a transaction field that is set in every transaction input. Script-level relative timelocks are implemented with the OP_CHECKSEQUENCEVERIFY (OP_CSV) opcode.

Relative timelocks are implemented according to the specifications in BIP68, Relative Lock-Time Using Consensus-Enforced Sequence Numbers and BIP112, OP_CHECKSEQUENCEVERIFY.

BIP68 and BIP112 were activated in May 2016 as a soft fork upgrade to the consensus rules.

7.4.4. Relative Timelocks with OP_CSV

Just like OP_CLTV and lock time, there is a script opcode for relative timelocks that leverages the sequence value in scripts. That opcode is OP_CHECKSEQUENCEVERIFY, commonly referred to as OP_CSV for short.

The OP_CSV opcode when evaluated in a UTXO's script allows spending only in a transaction whose input sequence value is greater than or equal to the OP_CSV parameter. Essentially, this restricts spending the UTXO until a certain number of blocks or seconds have elapsed relative to the time the UTXO was mined.

As with CLTV, the value in OP_CSV must match the format in the corresponding sequence value. If OP_CSV is specified in terms of blocks, then so must sequence. If OP_CSV is specified in terms of seconds, then so must sequence.

> **Warning** A script executing multiple OP_CSV opcodes must only use the same variety, either time-based or height-based. Mixing varieties will produce an invalid script that can never be spent, the same problem we saw with OP_CLTV in Timelock Conflicts. However, OP_CSV allows any two valid inputs to be included in the same transaction, so the problem of interaction across inputs that occurs with OP_CLTV doesn't affect OP_CSV.

Relative timelocks with OP_CSV are especially useful when several (chained) transactions are created and signed but not propagated—that is, they're kept off the blockchain (*offchain*). A child transaction cannot be used until the parent transaction has been propagated, mined, and aged by the time specified in the relative timelock. One application of this use case is shown in Payment Channels and State Channels and Routed Payment Channels (Lightning Network).

OP_CSV is defined in detail in BIP112, CHECKSEQUENCEVERIFY.

7.5. Scripts with Flow Control (Conditional Clauses)

One of the more powerful features of Bitcoin Script is flow control, also known as conditional clauses. You are probably familiar with flow control in various programming languages that use the construct IF...THEN...ELSE. Bitcoin conditional clauses look a bit different but are essentially the same construct.

At a basic level, Bitcoin conditional opcodes allow us to construct a script that has two ways of being unlocked, depending on a TRUE/FALSE outcome of evaluating a logical condition. For example, if x is TRUE, the executed code path is A and the ELSE code path is B.

Additionally, Bitcoin conditional expressions can be "nested" indefinitely, meaning that a conditional clause can contain another within it, which contains another, etc. Bitcoin Script flow control can be used to construct very complex scripts with hundreds of possible

execution paths. There is no limit to nesting, but consensus rules impose a limit on the maximum size of a script in bytes.

Bitcoin implements flow control using the OP_IF, OP_ELSE, OP_ENDIF, and OP_NOTIF opcodes. Additionally, conditional expressions can contain boolean operators such as OP_BOOLAND, OP_BOOLOR, and OP_NOT.

At first glance, you may find the Bitcoin's flow control scripts confusing. That is because Bitcoin Script is a stack language. The same way that 1 {plus} 1 looks "backward" when expressed as 1 1 OP_ADD, flow control clauses in Bitcoin also look "backward."

In most traditional (procedural) programming languages, flow control looks like this:

Pseudocode of flow control in most programming languages

```
if (condition):
  code to run when condition is true
else:
  code to run when condition is false
endif
code to run in either case
```

In a stack-based language like Bitcoin Script, the logical condition comes before the IF, which makes it look "backward":

Bitcoin Script flow control

```
condition
IF
  code to run when condition is true
OP_ELSE
  code to run when condition is false
OP_ENDIF
code to run in either case
```

When reading Bitcoin Script, remember that the condition being evaluated comes *before* the IF opcode.

7.5.1. Conditional Clauses with VERIFY Opcodes

Another form of conditional in Bitcoin Script is any opcode that ends in VERIFY. The VERIFY suffix means that if the condition evaluated is not TRUE, execution of the script terminates immediately and the transaction is deemed invalid.

Unlike an IF clause, which offers alternative execution paths, the VERIFY suffix acts as a *guard clause*, continuing only if a precondition is met.

For example, the following script requires Bob's signature and a preimage (secret) that produces a specific hash. Both conditions must be satisfied to unlock:

A script with an OP_EQUALVERIFY guard clause.

```
OP_HASH160 <expected hash> OP_EQUALVERIFY <Bob's Pubkey> OP_CHECKSIG
```

To spend this, Bob must present a valid preimage and a signature:

Satisfying the above script

```
<Bob's Sig> <hash pre-image>
```

Without presenting the preimage, Bob can't get to the part of the script that checks for his signature.

This script can be written with an OP_IF instead:

A script with an IF guard clause

```
OP_HASH160 <expected hash> OP_EQUAL
OP_IF
   <Bob's Pubkey> OP_CHECKSIG
OP_ENDIF
```

Bob's authentication data is identical:

Satisfying the above script

```
<Bob's Sig> <hash pre-image>
```

The script with OP_IF does the same thing as using an opcode with a VERIFY suffix; they both operate as guard clauses. However, the VERIFY construction is more efficient, using two fewer opcodes.

So, when do we use VERIFY and when do we use OP_IF? If all we are trying to do is to attach a precondition (guard clause), then VERIFY is better. If, however, we want to have more than one execution path (flow control), then we need an OP_IF...OP_ELSE flow control clause.

7.5.2. Using Flow Control in Scripts

A very common use for flow control in Bitcoin Script is to construct a script that offers multiple execution paths, each a different way of redeeming the UTXO.

Let's look at a simple example where we have two signers, Alice and Bob, and either one is able to redeem. With multisig, this would be expressed as a 1-of-2 multisig script. For the sake of demonstration, we will do the same thing with an OP_IF clause:

```
OP_IF
 <Alice's Pubkey>
OP_ELSE
 <Bob's Pubkey>
```

```
OP_ENDIF
OP_CHECKSIG
```

Looking at this redeem script, you may be wondering: "Where is the condition? There is nothing preceding the IF clause!"

The condition is not part of the script. Instead, the condition will be offered at spending time, allowing Alice and Bob to "choose" which execution path they want:

Alice satisfies the above script:

```
<Alice's Sig> OP_TRUE
```

The OP_TRUE at the end serves as the condition (TRUE) that will make the OP_IF clause execute the fist redemption path. This conditions puts the public key on the stack for which Alice has a signature. The OP_TRUE opcode, also known as OP_1, will put the number 1 on the stack.

For Bob to redeem this, he would have to choose the second execution path in OP_IF by giving a FALSE value. The OP_FALSE opcode, also known as OP_0, pushes an empty byte array to the stack:

```
<Bob's Sig> OP_FALSE
```

Bob’s input script causes the OP_IF clause to execute the second (OP_ELSE) script, which requires Bob’s signature.

Since OP_IF clauses can be nested, we can create a "maze" of execution paths. The input script can provide a "map" selecting which execution path is actually executed:

```
OP_IF
  subscript A
OP_ELSE
  OP_IF
    subscript B
  OP_ELSE
    subscript C
  OP_ENDIF
OP_ENDIF
```

In this scenario, there are three execution paths (subscript A, subscript B, and subscript C). The input script provides a path in the form of a sequence of TRUE or FALSE values. To select path subscript B, for example, the input script must end in OP_1 OP_0 (TRUE, FALSE). These values will be pushed onto the stack so that the second value (FALSE) ends up at the top of the stack. The outer OP_IF clause pops the FALSE value and executes the first OP_ELSE clause. Then the TRUE value moves to the top of the stack and is evaluated by the inner (nested) OP_IF, selecting the B execution path.

Using this construct, we can build redeem scripts with tens or hundreds of execution paths, each offering a different way to redeem the UTXO. To spend, we construct an input script that navigates the execution path by putting the appropriate TRUE and FALSE values on the stack at each flow control point.

7.6. Complex Script Example

In this section we combine many of the concepts from this chapter into a single example.

Mohammed, a company owner in Dubai, operates an import/export business; he wishes to construct a company capital account with flexible rules. The scheme he creates requires different levels of authorization depending on timelocks. The participants in the multisig scheme are Mohammed, his two partners Saeed and Zaira, and their company lawyer. The three partners make decisions based on a majority rule, so two of the three must agree. However, in the case of a problem with their keys, they want their lawyer to be able to recover the funds with one of the three partner signatures. Finally, if all partners are unavailable or incapacitated for a while, they want the lawyer to be able to manage the account directly after he gains access to the capital account's transaction records.

Variable multi-signature with timelock is the redeem script that Mohammed designs to achieve this (line numbers have been prefixed).

Example 13. Variable multi-signature with timelock

```
01  OP_IF
02    OP_IF
03      2
04    OP_ELSE
05      <30 days> OP_CHECKSEQUENCEVERIFY OP_DROP
06      <Lawyer's Pubkey> OP_CHECKSIGVERIFY
07      1
08    OP_ENDIF
09    <Mohammed's Pubkey> <Saeed's Pubkey> <Zaira's Pubkey> 3 OP_CHECKMULTISIG
10  OP_ELSE
11    <90 days> OP_CHECKSEQUENCEVERIFY OP_DROP
12    <Lawyer's Pubkey> OP_CHECKSIG
13  OP_ENDIF
```

Mohammed's script implements three execution paths using nested OP_IF...OP_ELSE flow control clauses.

In the first execution path, this script operates as a simple 2-of-3 multisig with the three partners. This execution path consists of lines 3 and 9. Line 3 sets the quorum of the multisig to 2 (2-of-3). This execution path can be selected by putting OP_TRUE OP_TRUE at the end of the input script:

Spending data for the first execution path (2-of-3 multisig)

```
OP_0 <Mohammed's Sig> <Zaira's Sig> OP_TRUE OP_TRUE
```

> **Tip** The OP_0 at the beginning of this input script is because of an oddity in OP_CHECKMULTISIG that pops an extra value from the stack. The extra value is disregarded by the OP_CHECKMULTISIG, but it must be present or the script fails. Pushing an empty byte array with OP_0 is a workaround to the oddity, as described in An Oddity in CHECKMULTISIG Execution.

The second execution path can only be used after 30 days have elapsed from the creation of the UTXO. At that time, it requires the signature of the lawyer and one of the three partners (a 1-of-3 multisig). This is achieved by line 7, which sets the quorum for the multisig to 1. To select this execution path, the input script would end in OP_FALSE OP_TRUE:

Spending data for the second execution path (Lawyer + 1-of-3)

```
OP_0 <Saeed's Sig> <Lawer's Sig> OP_FALSE OP_TRUE
```

> **Tip** Why OP_FALSE OP_TRUE? Isn't that backward? FALSE is pushed onto the stack, and TRUE is pushed on top of it. TRUE is therefore popped *first* by the first OP_IF opcode.

Finally, the third execution path allows the lawyer to spend the funds alone, but only after 90 days. To select this execution path, the input script has to end in OP_FALSE:

Input script for the third execution path (Lawyer only)

```
<Lawyer's Sig> OP_FALSE
```

Try running the script on paper to see how it behaves on the stack.

7.6.1. Segregated Witness Output and Transaction Examples

Let's look at some of our example transactions and see how they would change with segregated witness. We'll first look at how a P2PKH payment can be accomplished as the segregated witness program. Then, we'll look at the segregated witness equivalent for P2SH scripts. Finally, we'll look at how both of the preceding segregated witness programs can be embedded inside a P2SH script.

Pay to witness public key hash (P2WPKH)

Let's start by looking at the example of a P2PKH output script:

Example P2PKH output script

```
OP_DUP OP_HASH160 ab68025513c3dbd2f7b92a94e0581f5d50f654e7
```

```
OP_EQUALVERIFY OP_CHECKSIG
```

With segregated witness, Alice would create a P2WPKH script. If that script commits to the same public key, it would look like this:

Example P2WPKH output script

```
0 ab68025513c3dbd2f7b92a94e0581f5d50f654e7
```

As you can see, a P2WPKH output script is much simpler than the P2PKH equivalent. It consists of two values that are pushed onto the script evaluation stack. To an old (nonsegwit-aware) Bitcoin client, the two pushes would look like an output that anyone can spend. To a newer, segwit-aware client, the first number (0) is interpreted as a version number (the *witness version*) and the second part (20 bytes) is a *witness program*. The 20-byte witness program is simply the hash of the public key, as in a P2PKH script.

Now, let's look at the corresponding transaction that Bob uses to spend this output. For the original script, the spending transaction would have to include a signature within the transaction input:

Decoded transaction showing a P2PKH output being spent with a signature

```
[...]
"vin" : [
  "txid": "abcdef12345...",
  "vout": 0,
  "scriptSig": “<Bob’s scriptSig>”,
]
[...]
```

However, to spend the P2WPKH output, the transaction has no signature on that input. Instead, Bob's transaction has an empty input script and includes a witness structure:

Decoded transaction showing a P2WPKH output being spent with a witness structure

```
[...]
"vin" : [
  "txid": "abcdef12345...",
  "vout": 0,
  "scriptSig": “”,
]
[...]
“witness”: “<Bob’s witness structure>”
[...]
```

Wallet construction of P2WPKH

It is extremely important to note that P2WPKH witness programs should only be created by the receiver and not converted by the spender from a known public key, P2PKH script, or address. The spender has no way of knowing if the receiver's wallet has the ability to

construct segwit transactions and spend P2WPKH outputs.

Additionally, P2WPKH outputs must be constructed from the hash of a *compressed* public key. Uncompressed public keys are nonstandard in segwit and may be explicitly disabled by a future soft fork. If the hash used in the P2WPKH came from an uncompressed public key, it may be unspendable and you may lose funds. P2WPKH outputs should be created by the payee's wallet by deriving a compressed public key from their private key.

> **Warning** P2WPKH should be constructed by the receiver by converting a compressed public key to a P2WPKH hash. Neither the spender nor anyone else should ever transform a P2PKH script, Bitcoin address, or uncompressed public key to a P2WPKH witness script. In general, a spender should only send to the receiver in the manner that the receiver indicated.

Pay to witness script hash (P2WSH)

The second type of segwit v0 witness program corresponds to a P2SH script. We saw this type of script in Pay to Script Hash. In that example, P2SH was used by Mohammed's company to express a multisignature script. Payments to Mohammed's company were encoded with a script like this:

Example P2SH output script

```
OP_HASH160 54c557e07dde5bb6cb791c7a540e0a4796f5e97e OP_EQUAL
```

This P2SH script references the hash of a *redeem script* that defines a 2-of-3 multisignature requirement to spend funds. To spend this output, Mohammed's company would present the redeem script (whose hash matches the script hash in the P2SH output) and the signatures necessary to satisfy that redeem script, all inside the transaction input:

Decoded transaction showing a P2SH output being spent

```
[...]
"vin" : [
  "txid": "abcdef12345...",
  "vout": 0,
  "scriptSig": "<SigA> <SigB> <2 PubA PubB PubC PubD PubE 5 OP_CHECKMULTISIG>",
]
```

Now, let's look at how this entire example would be upgraded to segwit v0. If Mohammed's customers were using a segwit-compatible wallet, they would make a payment, creating a P2WSH output that would look like this:

Example P2WSH output script

```
0 a9b7b38d972cabc7961dbfbcb841ad4508d133c47ba87457b4a0e8aae86dbb89
```

Again, as with the example of P2WPKH, you can see that the segregated witness equivalent script is a lot simpler and reduces the template overhead that you see in P2SH scripts. Instead, the segregated witness output script consists of two values pushed to the stack: a witness version (0) and the 32-byte SHA256 hash of the witness script (the witness program).

> **Tip** While P2SH uses the 20-byte RIPEMD160(SHA256(script)) hash, the P2WSH witness program uses a 32-byte SHA256(script) hash. This difference in the selection of the hashing algorithm is deliberate to provide stronger security to P2WSH in certain use cases (128 bits of security in P2WSH versus 80 bits of security in P2SH). For details, see P2SH Collision Attacks.

Mohammed's company can spend the P2WSH output by presenting the correct witness script and sufficient signatures to satisfy it. The witness script and the signatures would be included as part of the witness structure. No data would be placed in the input script because this is a native witness program, which does not use the legacy input script field:

Decoded transaction showing a P2WSH output being spent with witness structure

```
[...]
"vin" : [
  "txid": "abcdef12345...",
  "vout": 0,
  "scriptSig": "",
]
[...]
"witness": "<SigA> <SigB> <2 PubA PubB PubC PubD PubE 5 OP_CHECKMULTISIG>"
[...]
```

Differentiating between P2WPKH and P2WSH

In the previous two sections, we demonstrated two types of witness programs: Pay to witness public key hash (P2WPKH) and Pay to witness script hash (P2WSH). Both types of witness programs consist of the same version number followed by a data push. They look very similar, but are interpreted very differently: one is interpreted as a public key hash, which is satisfied by a signature and the other as a script hash, which is satisfied by a witness script. The critical difference between them is the length of the witness program:

- The witness program in P2WPKH is 20 bytes.
- The witness program in P2WSH is 32 bytes.

This is the one difference that allows a full node to differentiate between the two types of witness programs. By looking at the length of the hash, a node can determine what type of witness program it is, P2WPKH or P2WSH.

7.6.2. Upgrading to Segregated Witness

As we can see from the previous examples, upgrading to segregated witness is a two-step process. First, wallets must create segwit type outputs. Then, these outputs can be spent by wallets that know how to construct segregated witness transactions. In the examples, Alice's wallet is able to create outputs paying segregated witness output scripts. Bob's wallet is also segwit-aware and able to spend those outputs.

Segregated witness was implemented as a backward-compatible upgrade, where *old and new clients can coexist*. Wallet developers independently upgraded wallet software to add segwit capabilities. Legacy P2PKH and P2SH continue to work for nonupgraded wallets. That leaves two important scenarios, which are addressed in the next section:

- Ability of a spender's wallet that is not segwit-aware to make a payment to a recipient's wallet that can process segwit transactions.
- Ability of a spender's wallet that is segwit-aware to recognize and distinguish between recipients that are segwit-aware and ones that are not, by their *addresses*.

Embedding segregated witness inside P2SH

Let's assume, for example, that Alice's wallet is not upgraded to segwit, but Bob's wallet is upgraded and can handle segwit transactions. Alice and Bob can use legacy non-segwit outputs. But Bob would likely want to use segwit to reduce transaction fees, taking advantage of the reduced cost of witness structure.

In this case, Bob's wallet can construct a P2SH address that contains a segwit script inside it. Alice's wallet can make payments to it without any knowledge of segwit. Bob's wallet can then spend this payment with a segwit transaction, taking advantage of segwit and reducing transaction fees.

Both forms of witness scripts, P2WPKH and P2WSH, can be embedded in a P2SH address. The first is noted as nested P2WPKH, and the second is noted as nested P2WSH.

Nested pay to witness public key hash

The first form of output script we will examine is nested P2WPKH. This is a pay to witness public key hash witness program, embedded inside a pay to script hash script, so that a wallet that is not aware of segwit can pay the output script.

Bob's wallet constructs a P2WPKH witness program with Bob's public key. This witness program is then hashed and the resulting hash is encoded as a P2SH script. The P2SH script is converted to a Bitcoin address, one that starts with a "3," as we saw in Pay to Script Hash.

Bob's wallet starts with the P2WPKH witness version and witness program we saw earlier:

Bob’s P2WPKH witness version and witness program

```
0 ab68025513c3dbd2f7b92a94e0581f5d50f654e7
```

The data consists of the witness version and Bob’s 20-byte public key hash.

Bob’s wallet then hashes the data, first with SHA256, then with RIPEMD-160, producing another 20-byte hash. Next, the redeem script hash is converted to a Bitcoin address. Finally, Alice’s wallet can make a payment to 37Lx99uaGn5avKBxiW26HjedQE3LrDCZru, just as it would to any other Bitcoin address.

To pay Bob, Alice’s wallet would lock the output with a P2SH script:

```
OP_HASH160 3e0547268b3b19288b3adef9719ec8659f4b2b0b OP_EQUAL
```

Even though Alice’s wallet has no support for segwit, the payment it creates can be spent by Bob with a segwit transaction.

Nested pay to witness script hash

Similarly, a P2WSH witness program for a multisig script or other complicated script can be embedded inside a P2SH script and address, making it possible for any wallet to make payments that are segwit compatible.

As we saw in Pay to witness script hash (P2WSH), Mohammed’s company is using segregated witness payments to multisignature scripts. To make it possible for any client to pay his company, regardless of whether their wallets are upgraded for segwit, Mohammed’s wallet can embed the P2WSH witness program inside a P2SH script.

First, Mohammed’s wallet hashes the witness script with SHA256 (just once), producing the hash:

```
9592d601848d04b172905e0ddb0adde59f1590f1e553ffc81ddc4b0ed927dd73
```

Next, the hashed witness script is turned into a version-prefixed P2WSH witness program:

```
0 9592d601848d04b172905e0ddb0adde59f1590f1e553ffc81ddc4b0ed927dd73
```

Then, the witness program itself is hashed with SHA256 and RIPEMD-160, producing a new 20-byte hash:

```
86762607e8fe87c0c37740cddee880988b9455b2
```

Next, the wallet constructs a P2SH Bitcoin address from this hash:

P2SH Bitcoin address

```
3Dwz1MXhM6EfFoJChHCxh1jWHb8GQqRenG
```

Now, Mohammed's clients can make payments to this address even if they don't support segwit. To send a payment to Mohammed, a wallet would lock the output with the following P2SH script:

P2SH script used to lock payments to Mohammed's multisig

```
OP_HASH160 86762607e8fe87c0c37740cddee880988b9455b2 OP_EQUAL
```

Mohammed's company can then construct segwit transactions to spend these payments, taking advantage of segwit features including lower transaction fees.

7.7. Merklized Alternative Script Trees (MAST)

Using OP_IF, you can authorize multiple different spending conditions, but this approach has several undesirable aspects:

Weight (cost)
: Every condition you add increases the size of the script, increasing the weight of the transaction and the amount of fee that will need to be paid in order to spend bitcoins protected by that script.

Limited size
: Even if you're willing to pay for extra conditions, there's a limit to the maximum number you can put in a script. For example, legacy script is limited to 10,000 bytes, practically limiting you to a few hundred conditional branches at most. Even if you could create a script as large as an entire block, it could still only contain about 20,000 useful branches. That's a lot for simple payments but tiny compared to some imagined uses of Bitcoin.

Lack of privacy
: Every condition you add to your script becomes public knowledge when you spend bitcoins protected by that script. For example, Mohammed's lawyer and business partners will be able to see the entire script in Variable multi-signature with timelock whenever anyone spends from it. That means their lawyer, even if he's not needed for signing, will be able to track all of their transactions.

However, Bitcoin already uses a data structure known as a merkle tree that allows verifying an element is a member of a set without needing to identify every other member of the set.

We'll learn more about merkle trees in Merkle Trees, but the essential information is that members of the set of data we want (e.g., authorization conditions of any length) can be passed into a hash function to create a short commitment (called a *leaf* of the merkle tree). Each of those leaves is then paired with another leaf and hashed again, creating a commitment to the leaves, called a *branch* commitment. A commitment to a pair of branches can be created the same way. This step is repeated for the branches until only

one identifier remains, called the *merkle root*. Using our example script from Variable multi-signature with timelock, we construct a merkle tree for each of the three authorization conditions in A MAST with three subscripts..

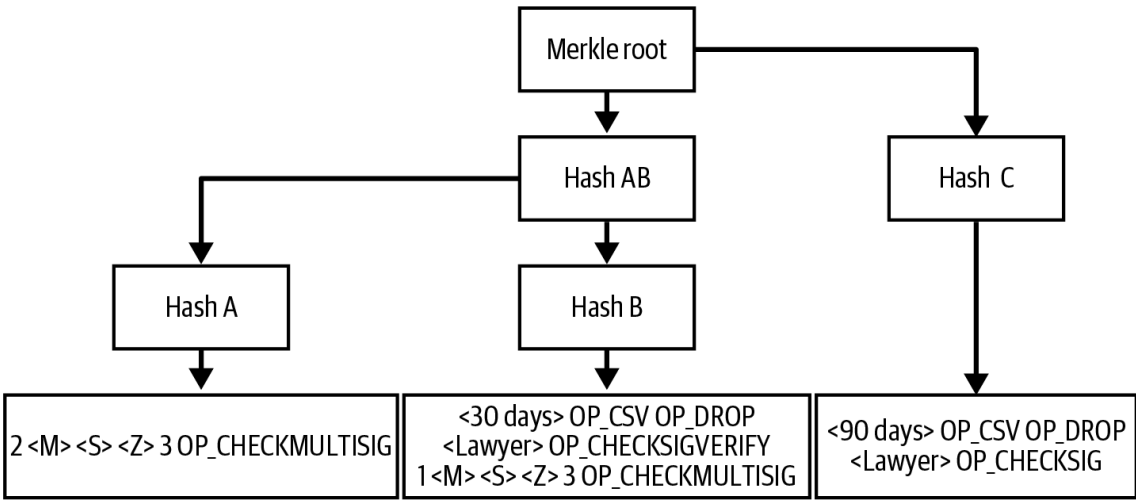


Figure 41. A MAST with three subscripts.

We can now create a compact membership proof that proves a particular authorization condition is a member of the merkle tree without disclosing any details about the other members of the merkle tree. See A MAST membership proof for one of the subscripts., and note that the shaded nodes can be computed from other data provided by the user, so they don’t need to be specified at spend time.

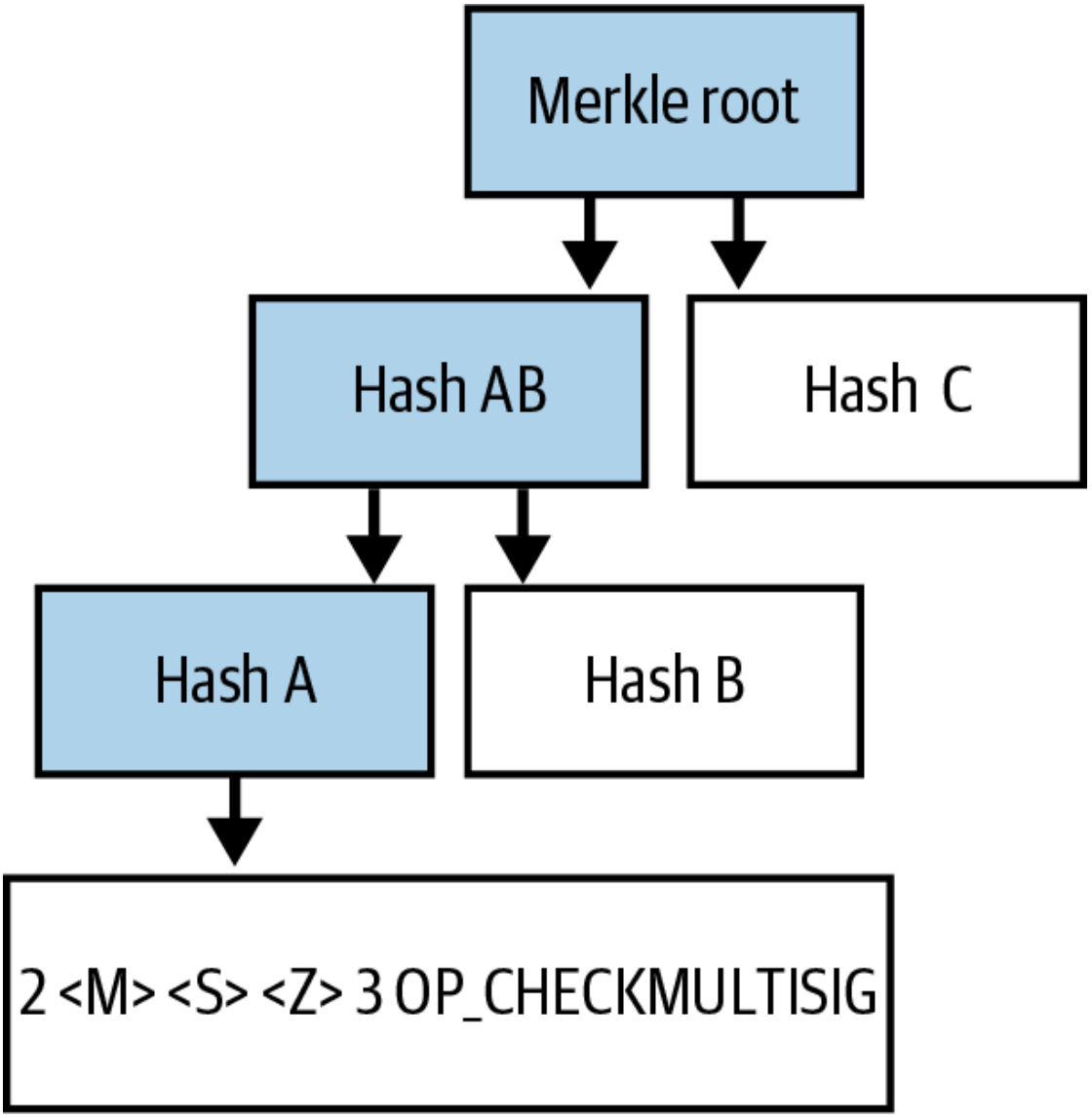


Figure 42. A MAST membership proof for one of the subscripts.

The hash digests used to create the commitments are each 32 bytes, so proving that a spend of A MAST membership proof for one of the subscripts. is authorized (using a merkle tree and the particular conditions) and authenticated (using signatures) uses 383 bytes. By comparison, the same spend without a merkle tree (i.e., providing all possible authorization conditions) uses 412 bytes.

Saving 29 bytes (7%) in this example doesn't fully capture the potential savings. The binary-tree nature of a merkle tree means that you only need an additional 32-byte commitment every time you double the number of members in the set (in this case,

authorization conditions). In this instance, with three conditions, we need to use three commitments (one of them being the merkle root, which will need to be included in the authorization data); we could also have four commitments for the same cost. An extra commitment would give us up to eight conditions. With just 16 commitments—512 bytes of commitments—we could have over 32,000 authorization conditions, far more than could be effectively used in an entire block of transactions filled with OP_IF statements. With 128 commitments (4,096 bytes), the number of conditions we could create in theory far exceeds the number of conditions that all the computers in the world could create.

It's commonly the case that not every authorization condition is equally as likely to be used. In the our example case, we expect Mohammed and his partners to spend their money frequently; the time delayed conditions only exist in case something goes wrong. We can restructure our tree with this knowledge as shown in A MAST with the most-expected script in the best position..

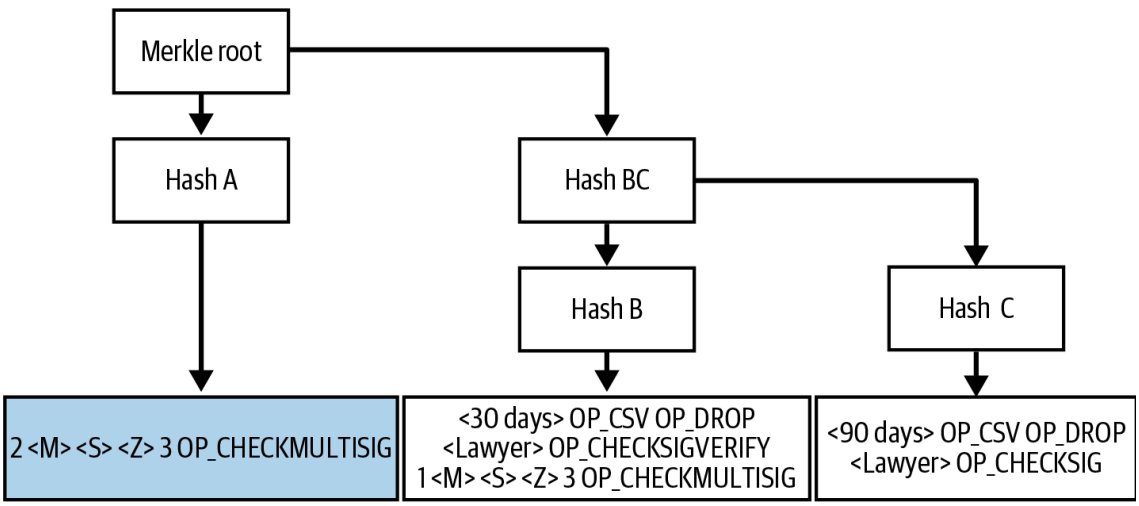


Figure 43. A MAST with the most-expected script in the best position.

Now we only need to provide two commitments for the common case (saving 32 bytes), although we still need three commitments for the less common cases. If you know (or can guess) the probabilities of using the different authorization conditions, you can use the Huffman algorithm to place them into a maximally efficient tree; see BIP341 for details.

Regardless of how the tree is constructed, we can see in the previous examples that we're only revealing the actual authorization conditions that get used. The other conditions remain private. Also remaining private are the number of conditions: a tree could have a single condition or a trillion conditions—there's no way for someone looking only at the onchain data for a single transaction to tell.

Except for increasing the complexity of Bitcoin slightly, there are no significant downsides of MAST for Bitcoin and there were two solid proposals for it, BIP114 and BIP116, before an improved approach was discovered, which we'll see in Taproot.

MAST Versus MAST

The earliest idea for what we now know as *MAST* in Bitcoin was *merklized abstract syntax trees*. In an abstract syntax tree (AST), every condition in a script creates a new branch, as show in An abstract syntax tree (AST) for a script..

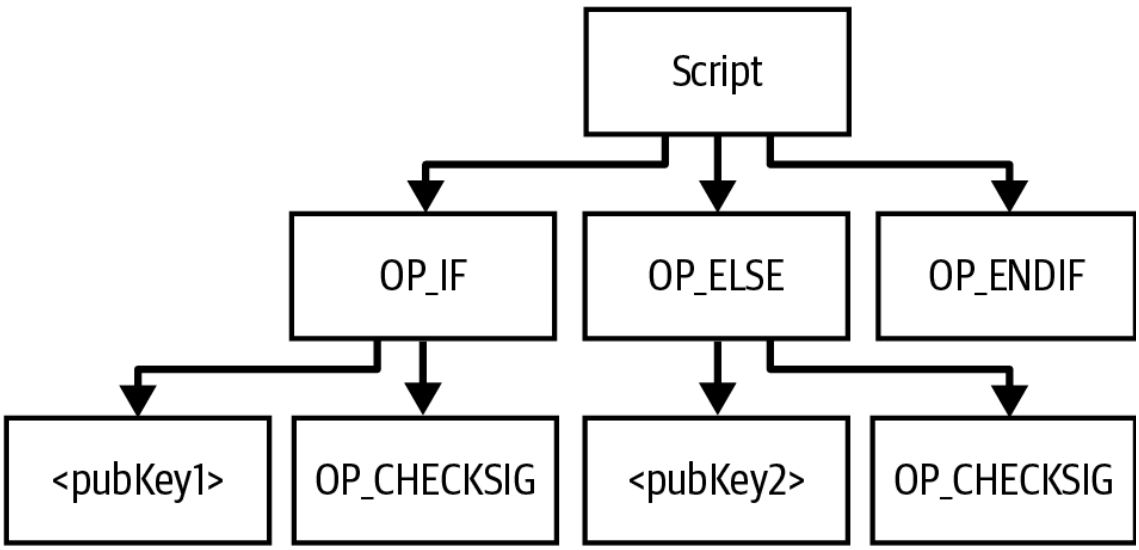


Figure 44. An abstract syntax tree (AST) for a script.

ASTs are widely used by programs that parse and optimize code for other programs, such as compilers. A merklized AST would commit to every part of a program and enable the features described in Merklized Alternative Script Trees (MAST), but it would require revealing at least one 32-byte digest for every separate part of the program, which would not be very space efficient on the blockchain for most programs.

What people in most cases call *MAST* in Bitcoin today is *merklized alternative script trees*, a backronym coined by developer Anthony Towns. An alternative script tree is a set of scripts, each one of them complete by itself, where only one can be selected—making them alternatives for each other, as shown in An alternative script tree..

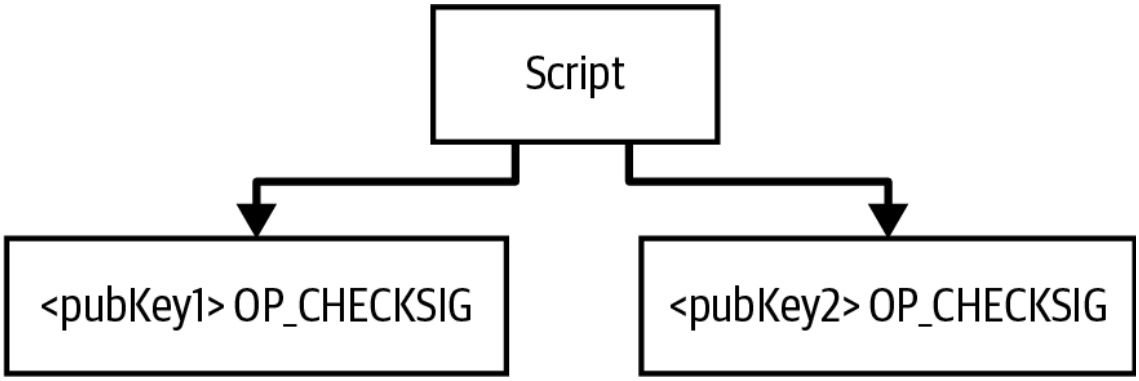


Figure 45. An alternative script tree.

Alternative script trees only require revealing one 32-byte digest for each level of depth between the spender's chosen script and the root of the tree. For most scripts, this is a much more efficient use of space in the blockchain.

7.8. Pay to Contract (P2C)

As we saw in Public Child Key Derivation, the math of elliptic curve cryptography (ECC) allows Alice to use a private key to derive a public key that she gives to Bob. He can add an arbitrary value to that public key to create a derived public key. If he gives that arbitrary value to Alice, she can add it to her private key to derive the private key for the derived public key. In short, Bob can create child public keys for which only Alice can create the corresponding private keys. This is useful for BIP32-style Hierarchical Deterministic (HD) wallet recovery, but it can also serve another use.

Let's imagine Bob wants to buy something from Alice but he also wants to be able prove later what he paid for in case there's any dispute. Alice and Bob agree on the name of the item or service being sold (e.g., "Alice's podcast episode #123"), and transform that description into a number by hashing it and interpreting the hash digest as a number. Bob adds that number to Alice's public key and pays it. The process is called *key tweaking*, and the number is known as a *tweak*.

Alice can spend the funds by tweaking her private key using the same number (tweak).

Later, Bob can prove to anyone what he paid Alice by revealing her underlying key and the description they used. Anyone can verify that the public key, which was paid, equals the underlying key plus the hash commitment to the description. If Alice admits that key is hers, then she received the payment. If Alice spent the funds, this further proves she knew the description at the time she signed the spending transaction since she could only create a valid signature for the tweaked public key if she knew the tweak (the description).

If neither Alice nor Bob decided to publicly reveal the description they use, the payment between them looks like any other payment. There's no privacy loss.

Because P2C is private by default, we can't know how often it is used for its original purpose—in theory every payment could be using it, although we consider that unlikely. However, P2C is widely used today in a slightly different form, which we'll see in Taproot.

7.9. Scriptless Multisignatures and Threshold Signatures

In Scripted Multisignatures, we looked at scripts that require signatures from multiple keys. However, there's another way to require cooperation from multiple keys, which is also confusingly called *multisignature*. To distinguish between the two types in this section, we'll call the version involving `OP_CHECKSIG`-style opcodes *script multisignatures* and the other version *scriptless multisignatures*.

Scriptless multisignatures involve each participant creating their own secret the same way they create a private key. We'll call this secret a *partial private key*, although we should note that it's the same length as a regular full private key. From the partial private key, each participant derives a partial public key using the same algorithm used for regular public keys we described in Public Keys. Each participant shares their partial public keys with all the other participants and then combines all of the keys together to create the scriptless multisignature public key.

This combined public key looks the same as any other Bitcoin public key. A third party can't distinguish between a multiparty public key and an ordinary key generated by a single user.

To spend bitcoins protected by the scriptless multisignature public key, each participant generates a partial signature. The partial signatures are then combined to create a regular full signature. There are many known methods for creating and combining the partial signatures; we'll look at this topic more in Digital Signatures. Similar to the public keys for scriptless multisignatures, the signatures generated by this process look the same as any other Bitcoin signature. Third parties can't determine whether a signature was created by a single person or a million people cooperating with each other.

Scriptless multisignatures are smaller and more private than scripted multisignatures. For scripted multisignatures, the number of bytes placed in a transaction increases for every key and signature involved. For scriptless multisignatures, the size is constant—a million participants each providing their own partial key and partial signature puts exactly the same amount of data in a transaction as an individual using a single key and signature. The story is the same for privacy: because each new key or signature adds data to a transaction, scripted multisignatures disclose data about how many keys and signatures are being used—which may make it easy to figure out which transactions were created by which group of participants. However, because every scriptless multisignatures looks like every other scriptless multisignature and every single-signature, no privacy-reducing data is leaked.

There are two downsides of scriptless multisignatures. The first is that all known secure algorithms for creating them for Bitcoin require more rounds of interaction or more careful management of state than scripted multisignatures. This can be challenging in cases where signatures are being generated by nearly stateless hardware signing devices and the keys are physically distributed. For example, if you keep a hardware signing device in a bank safe deposit box, you would need to visit that box once to create a scripted multisignature but possibly two or three times for a scriptless multisignature.

The other downside is that threshold signing doesn't reveal who signed. In scripted threshold signing, Alice, Bob, and Carol agree (for example) that any two of them signing will be sufficient to spend the funds. If Alice and Bob sign, this requires putting signatures from each of them on chain, proving to anyone who knows their keys that they signed and Carol didn't. In scriptless threshold signing, a signature from Alice and Bob is indistinguishable from a signature between Alice and Carol or Bob and Carol. This is beneficial for privacy, but it means that, even if Carol claims she didn't sign, she can't prove that she didn't, which may be bad for accountability and auditability.

For many users and use cases, the always reduced size and increased privacy of multisignatures outweighs its occasional challenges for creating and auditing signatures.

7.10. Taproot

One reason people choose to use Bitcoin is that it's possible to create contracts with highly predictable outcomes. Legal contracts enforced by a court of law depend in part on decisions by the judges and jurors involved in the case. By contrast, Bitcoin contracts often require actions by their participants but are otherwise enforced by thousands of full nodes all running functionally identical code. When given the same contract and the same input, every full node will always produce the same result. Any deviation would mean that Bitcoin was broken. Human judges and juries can be much more flexible than software, but when that flexibility isn't wanted or needed, the predictability of Bitcoin contracts is a major asset.

If all of the participants in a contract recognize that its outcome has become completely predictable, there's not actually any need for them to continue using the contract. They could just do whatever the contract compels them to do and then terminate the contract. In society, this is how most contracts terminate: if the interested parties are satisfied, they never take the contract before a judge or jury. In Bitcoin, it means that any contract that will use a significant amount of block space to settle should also provide a clause that allows it to instead be settled by mutual satisfaction.

In MAST and with scriptless multisignatures, a mutual satisfaction clause is easy to design. We simply make one of the top leaves of the script tree a scriptless multisignature between all interested parties. We already saw a complex contract between several parties with a simple mutual satisfaction clause in A MAST with the most-expected script in the best position.. We could make that more optimized by switching from scripted multisignature to scriptless multisignature.

That’s reasonably efficient and private. If the mutual satisfaction clause is used, we only need to provide a single merkle branch and all we reveal is that a signature was involved (it could be from one person or it could be from thousands of different participants). But developers in 2018 realized that we could do better if we also used pay to contract.

In our previous description of pay to contract in Pay to Contract (P2C), we tweaked a public key to commit to the text of an agreement between Alice and Bob. We can instead commit to the program code of a contract by committing to the root of a MAST. The public key we tweak is a regular Bitcoin public key, meaning it could require a signature from a single person or it could require a signature from multiple people (or it could be created in a special way to make it impossible to generate a signature for it). That means we can satisfy the contract either with a single signature from all interested parties or by revealing the MAST branch we want to use. That commitment tree involving both a public key and a MAST is shown in A taproot with the public key committing to a merkle root..

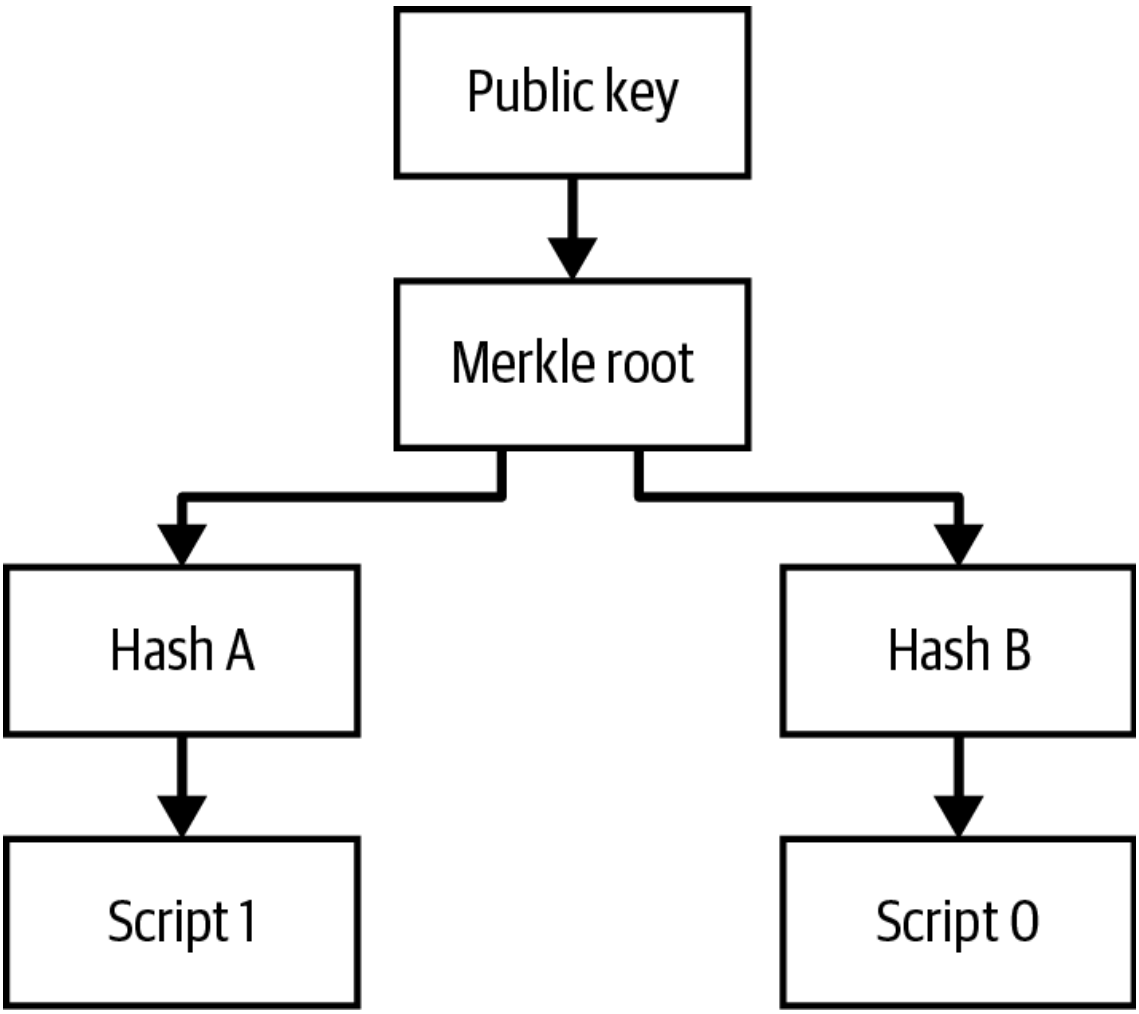


Figure 46. A taproot with the public key committing to a merkle root.

This makes the mutual satisfaction clause using a multisignature extremely efficient and very private. It's even more private than it may appear because any transaction created by a single user who wants it to be satisfied by a single signature (or a multisignature generated by multiple different wallets they control) looks identical onchain to a mutual-satisfaction spend. There's no onchain difference in this case between a spend by a million users involved in an extraordinarily complex contract or a single user just spending their saved bitcoins.

When spending is possible using just the key, such as for a single signature or scriptless multisignature, that is called *keypath spending*. When the tree of scripts is used, that is called *scriptpath spending*. For keypath spending, the data that gets put onchain is the public key (in a witness program) and the signature (on the witness stack).

For scriptpath spending, the onchain data also includes the public key, which is placed in a witness program and called the *taproot output key* in this context. The witness structure includes the following information:

- A version number.
- The underlying key—the key that existed before being tweaked by the merkle root to produce the taproot output key. This underlying key is called the *taproot internal key*.
- The script to execute, called the *leaf script*.
- One 32-byte hash for each junction in merkle tree along the path that connects the leaf to the merkle root.
- Any data necessary to satisfy the script (such as signatures or hash preimages).

We're only aware of one significant described downside of taproot: contracts whose participants want to use MAST but who don't want a mutual satisfaction clause have to include a taproot internal key on the blockchain, adding about 33 bytes of overhead. Given that almost all contracts are expected to benefit from a mutual satisfaction clause, or other multisignature clause that uses the top-level public key, and all users benefit from the increased anonymity set of outputs looking similar to each other, that rare overhead was not considered important by most users who participated in taproot's activation.

Support for taproot was added to Bitcoin in a soft fork that activated in November 2021.

7.11. Tapscript

Taproot enables MAST but only with a slightly different version of the Bitcoin Script language than previously used, the new version being called *tapscript*. The major differences include:

Scripted multisignature changes
: The old OP_CHECKMULTISIG and OP_CHECKMULTISIGVERIFY opcodes are removed. Those opcodes don't combine well with one of the other changes in the taproot soft fork, the ability to use schnorr signatures with batch validation (see Schnorr Signatures). A new OP_CHECKSIGADD opcode is provided instead. When it successfully verifies a signature, this new opcode increments a counter by one, making it possible to conveniently count how many signatures passed, which can be compared against the desired number of successful signatures to reimplement the same behavior as OP_CHECKMULTISIG.

Changes to all signatures
: All signature operations in tapscript use the schnorr signature algorithm as defined in BIP340. We'll explore schnorr signatures more in Digital Signatures.

Additionally, any signature-checking operation that is not expected to succeed must be fed the value OP_FALSE (also called OP_0) instead of an actual signature. Providing anything else to a failed signature-checking operation will cause the entire script to fail. This also helps support batch validation of schnorr signatures.

OP_SUCCESSx opcodes
: Opcodes in previous versions of Script that were unusable are now redefined to cause an entire script to succeed if they are used. This allows future soft forks to redefine them as not succeeding under certain circumstances, which is a restriction and so is possible to do in a soft fork. (The opposite, to define a not-succeeding operation as a success can only be done in a hard fork, which is a much more challenging upgrade path.)

Although we've looked at authorization and authentication in depth in this chapter, we've skipped over one very important part of how Bitcoin authenticates spenders: its signatures. We'll look at that next in Digital Signatures.

8. Digital Signatures

Two signature algorithms are currently used in Bitcoin, the *schnorr signature algorithm* and the *Elliptic Curve Digital Signature Algorithm* (*ECDSA*). These algorithms are used for digital signatures based on elliptic curve private/public key pairs, as described in Elliptic Curve Cryptography Explained. They are used for spending segwit v0 P2WPKH outputs, segwit v1 P2TR keypath spending, and by the script functions OP_CHECKSIG, OP_CHECKSIGVERIFY, OP_CHECKMULTISIG, OP_CHECKMULTISIGVERIFY, and OP_CHECKSIGADD. Any time one of those is executed, a signature must be provided.

A digital signature serves three purposes in Bitcoin. First, the signature proves that the controller of a private key, who is by implication the owner of the funds, has *authorized* the spending of those funds. Secondly, the proof of authorization is *undeniable* (nonrepudiation). Thirdly, that the authorized transaction cannot be changed by unauthenticated third parties—that its *integrity* is intact.

> **Note** Each transaction input and any signatures it may contain is *completely* independent of any other input or signature. Multiple parties can collaborate to construct transactions and sign only one input each. Several protocols use this fact to create multiparty transactions for privacy.

In this chapter we look at how digital signatures work and how they can present proof of control of a private key without revealing that private key.

8.1. How Digital Signatures Work

A digital signature consists of two parts. The first part is an algorithm for creating a signature for a message (the transaction) using a private key (the signing key). The second part is an algorithm that allows anyone to verify the signature, given also the message and the corresponding public key.

8.1.1. Creating a Digital Signature

In Bitcoin's use of digital signature algorithms, the "message" being signed is the transaction, or more accurately a hash of a specific subset of the data in the transaction, called the *commitment hash* (see Signature Hash Types (SIGHASH)). The signing key is the user's private key. The result is the signature:

$$Sig = F_{sig}(F_{hash}(m), x)$$

where:

- x is the signing private key
- m is the message to sign, the commitment hash (such as parts of a transaction)
- F_{hash} is the hashing function
- F_{sig} is the signing algorithm
- Sig is the resulting signature

You can find more details on the mathematics of schnorr and ECDSA signatures in Schnorr Signatures and ECDSA Signatures.

In both schnorr and ECDSA signatures, the function F_{sig} produces a signature Sig that is composed of two values. There are differences between the two values in the different algorithms, which we'll explore later. After the two values are calculated, they are serialized into a byte stream. For ECDSA signatures, the encoding uses an international standard encoding scheme called the *Distinguished Encoding Rules*, or *DER*. For schnorr signatures, a simpler serialization format is used.

8.1.2. Verifying the Signature

The signature verification algorithm takes the message (a hash of parts of the transaction and related data), the signer's public key and the signature, and returns TRUE if the signature is valid for this message and public key.

To verify the signature, one must have the signature, the serialized transaction, some data about the output being spent, and the public key that corresponds to the private key used to create the signature. Essentially, verification of a signature means "Only the controller of the private key that generated this public key could have produced this signature on this transaction."

8.1.3. Signature Hash Types (SIGHASH)

Digital signatures apply to messages, which in the case of Bitcoin, are the transactions themselves. The signature proves a *commitment* by the signer to specific transaction data. In the simplest form, the signature applies to almost the entire transaction, thereby committing to all the inputs, outputs, and other transaction fields. However, a signature can commit to only a subset of the data in a transaction, which is useful for a number of scenarios as we will see in this section.

Bitcoin signatures have a way of indicating which part of a transaction's data is included in the hash signed by the private key using a SIGHASH flag. The SIGHASH flag is a single byte that is appended to the signature. Every signature has either an explicit or implicit SIGHASH flag, and the flag can be different from input to input. A transaction with three signed inputs may have three signatures with different SIGHASH flags, each signature signing (committing) to different parts of the transaction.

Remember, each input may contain one or more signatures. As a result, an input may have signatures with different SIGHASH flags that commit to different parts of the transaction. Note also that Bitcoin transactions may contain inputs from different "owners," who may sign only one input in a partially constructed transaction, collaborating with others to gather all the necessary signatures to make a valid transaction. Many of the SIGHASH flag types only make sense if you think of multiple participants collaborating outside the Bitcoin network and updating a partially signed transaction.

There are three SIGHASH flags: ALL, NONE, and SINGLE, as shown in [sighash_types_and_their].

`SIGHASH` types and their meanings

`SIGHASH` flag	Value	Description
`ALL`	`0x01`	Signature applies to all inputs and outputs
`NONE`	`0x02`	Signature applies to all inputs, none of the outputs
`SINGLE`	`0x03`	Signature applies to all inputs but only the one output with the same index number as the signed input

In addition, there is a modifier flag, SIGHASH_ANYONECANPAY, which can be combined with each of the preceding flags. When ANYONECANPAY is set, only one input is signed, leaving the rest (and their sequence numbers) open for modification. The ANYONECANPAY has the value 0x80 and is applied by bitwise OR, resulting in the combined flags as shown in [sighash_types_with_modifiers].

`SIGHASH` types with modifiers and their meanings

`SIGHASH` flag	Value	Description
`ALL|ANYONECANPAY`	`0x81`	Signature applies to one input and all outputs
`NONE|ANYONECANPAY`	`0x82`	Signature applies to one input, none of the outputs
`SINGLE|ANYONECANPAY`	`0x83`	Signature applies to one input and the output with the same index number

The way SIGHASH flags are applied during signing and verification is that a copy of the transaction is made and certain fields within are either omitted or truncated (set to zero length and emptied). The resulting transaction is serialized. The SIGHASH flag is included in the serialized transaction data and the result is hashed. The hash digest itself is the "message" that is signed. Depending on which SIGHASH flag is used, different parts of the transaction are included. By including the SIGHASH flag itself, the signature commits the SIGHASH type as well, so it can’t be changed (e.g., by a miner).

In Serialization of ECDSA Signatures (DER), we will see that the last part of the DER-encoded signature was 01, which is the SIGHASH_ALL flag for ECDSA signatures. This locks the transaction data, so Alice's signature is committing to the state of all inputs and outputs. This is the most common signature form.

Let's look at some of the other SIGHASH types and how they can be used in practice:

ALL|ANYONECANPAY
: This construction can be used to make a "crowdfunding"-style transaction. Someone attempting to raise funds can construct a transaction with a single output. The single output pays the "goal" amount to the fundraiser. Such a transaction is obviously not valid, as it has no inputs. However, others can now amend it by adding an input of their own as a donation. They sign their own input with ALL|ANYONECANPAY. Unless enough inputs are gathered to reach the value of the output, the transaction is invalid. Each donation is a "pledge," which cannot be collected by the fundraiser until the entire goal amount is raised. Unfortunately, this protocol can be circumvented by the fundraiser adding an input of their own (or from someone who lends them funds), allowing them to collect the donations even if they haven't reached the specified value.

NONE
: This construction can be used to create a "bearer check" or "blank check" of a specific amount. It commits to all inputs but allows the outputs to be changed. Anyone can write their own Bitcoin address into the output script. By itself, this allows any miner to change the output destination and claim the funds for themselves, but if other required signatures in the transaction use SIGHASH_ALL or another type that commits to the output, it allows those spenders to change the destination without allowing any third parties (like miners) to modify the outputs.

NONE|ANYONECANPAY
: This construction can be used to build a "dust collector." Users who have tiny UTXOs in their wallets can't spend these without the cost in fees exceeding the value of the UTXO; see Uneconomical outputs and disallowed dust. With this type of signature, the uneconomical UTXOs can be donated for anyone to aggregate and spend whenever they want.

There are some proposals to modify or expand the SIGHASH system. The most widely discussed proposal as of this writing is BIP118, which proposes to add two new sighash flags. A signature using SIGHASH_ANYPREVOUT would not commit to an input's outpoint field, allowing it to be used to spend any previous output for a particular witness program. For example, if Alice receives two outputs for the same amount to the same witness program (e.g., requiring a single signature from her wallet), a SIGHASH_ANYPREVOUT signature for spending either one of those outputs could be copied and used to spend the other output to the same destination.

A signature using SIGHASH_ANYPREVOUTANYSCRIPT would not commit to the outpoint, the amount, the witness program, or the specific leaf in the taproot merkle tree (script tree), allowing it to spend any previous output that the signature could satisfy. For example, if Alice received two outputs for different amounts and different witness programs (e.g., one requiring a single signature and another requiring her signature plus some other data), a SIGHASH_ANYPREVOUTANYSCRIPT signature for spending either one of those outputs could be copied and used to spend the other output to the same destination (assuming the extra data for the second output was known).

The main expected use for the two SIGHASH_ANYPREVOUT opcodes is improved payment channels, such as those used in the Lightning Network (LN), although several other uses have been described.

> **Note** You will not often see SIGHASH flags presented as an option in a user's wallet application. Simple wallet applications sign with SIGHASH_ALL flags. More sophisticated applications, such as LN nodes, may use alternative SIGHASH flags, but they use protocols that have been extensively reviewed to understand the influence of the alternative flags.

8.2. Schnorr Signatures

In 1989, Claus Schnorr published a paper describing the signature algorithm that's become eponymous with him. The algorithm isn't specific to the elliptic curve cryptography (ECC) that Bitcoin and many other applications use, although it is perhaps most strongly associated with ECC today. Schnorr signatures have a number of nice properties:

Provable security
: A mathematical proof of the security of schnorr signatures depends on only the difficulty of solving the Discrete Logarithm Problem (DLP), particularly for elliptic curves (EC) for Bitcoin, and the ability of a hash function (like the SHA256 function used in Bitcoin) to produce unpredictable values, called the random oracle model (ROM). Other signature algorithms have additional dependencies or require much larger public keys or signatures for equivalent security to ECC-Schnorr (when the threat is defined as classical computers; other algorithms may provide more efficient security against quantum computers).

Linearity
: Schnorr signatures have a property that mathematicians call *linearity*, which applies to functions with two particular properties. The first property is that summing together two or more variables and then running a function on that sum will produce the same value as running the function on each of the variables independently and then summing together the results, e.g., *f(x + y + z) == f(x) + f(y) + f(z)*; this property is called *additivity*. The second property is that multiplying a variable and

then running a function on that product will produce the same value as running the function on the variable and then multiplying it by the same amount, e.g., *f(a × x)* == *a × f(x)*; this property is called *homogeneity of degree 1*.

In cryptographic operations, some functions may be private (such as functions involving private keys or secret nonces), so being able to get the same result whether performing an operation inside or outside of a function makes it easy for multiple parties to coordinate and cooperate without sharing their secrets. We'll see some of the specific benefits of linearity in schnorr signatures in Schnorr-based Scriptless Multisignatures and Schnorr-based Scriptless Threshold Signatures.

Batch verification
: When used in a certain way (which Bitcoin does), one consequence of schnorr's linearity is that it's relatively straightforward to verify more than one schnorr signature at the same time in less time than it would take to verify each signature independently. The more signatures that are verified in a batch, the greater the speed up. For the typical number of signatures in a block, it's possible to batch verify them in about half the amount of time it would take to verify each signature independently.

Later in this chapter, we'll describe the schnorr signature algorithm exactly as it's used in Bitcoin, but we're going to start with a simplified version of it and work our way toward the actual protocol in stages.

Alice starts by choosing a large random number (x), which we call her *private key*. She also knows a public point on Bitcoin's elliptic curve called the Generator (G) (see Public Keys). Alice uses EC multiplication to multiply G by her private key x, in which case x is called a *scalar* because it scales up G. The result is xG, which we call Alice's *public key*. Alice gives her public key to Bob. Even though Bob also knows G, the DLP prevents Bob from being able to divide xG by G to derive Alice's private key.

At some later time, Bob wants Alice to identify herself by proving that she knows the scalar x for the public key (xG) that Bob received earlier. Alice can't give Bob x directly because that would allow him to identify as her to other people, so she needs to prove her knowledge of x without revealing x to Bob, called a *zero-knowledge proof*. For that, we begin the schnorr identity process:

1. Alice chooses another large random number (k), which we call the *private nonce*. Again she uses it as a scalar, multiplying it by G to produce kG, which we call the *public nonce*. She gives the public nonce to Bob.
2. Bob chooses a large random number of his own, e, which we call the *challenge scalar*. We say "challenge" because it's used to challenge Alice to prove that she knows the private key (x) for the public key (xG) she previously gave Bob; we say "scalar" because it will later be used to multiply an EC point.
3. Alice now has the numbers (scalars) x, k, and e. She combines them together to produce a final scalar s using the formula $s = k + ex$. She gives s to Bob.

4. Bob now knows the scalars s and e, but not x or k. However, Bob does know xG and kG, and he can compute for himself sG and exG. That means he can check the equality of a scaled-up version of the operation Alice performed: $sG == kG + exG$. If that is equal, then Bob can be sure that Alice knew x when she generated s.

Schnorr Identity Protocol with Integers Instead of Points
It might be easier to understand the interactive schnorr identity protocol if we create an insecure oversimplification by substituting each of the preceding values (including G) with simple integers instead of points on an elliptic curve. For example, we'll use the prime numbers starting with 3:

Setup: Alice chooses $x = 3$ as her private key. She multiplies it by the generator $G = 5$ to get her public key $xG = 15$. She gives Bob 15.

1. Alice chooses the private nonce $k = 7$ and generates the public nonce $kG = 35$. She gives Bob 35.
2. Bob chooses $e = 11$ and gives it to Alice.
3. Alice generates $s = 40 = 7 + 11 \times 3$. She gives Bob 40.
4. Bob derives $sG = 200 = 40 \times 5$ and $exG = 165 = 11 \times 15$. He then verifies that 200 == 35 + 165. Note that this is the same operation that Alice performed, but all of the values have been scaled up by 5 (the value of G).

Of course, this is an oversimplified example. When working with simple integers, we can divide products by the generator G to get the underlying scalar, which isn't secure. This is why a critical property of the elliptic curve cryptography used in Bitcoin is that multiplication is easy but division by a point on the curve is impractical. Also, with numbers this small, finding underlying values (or valid substitutes) through brute force is easy; the numbers used in Bitcoin are much larger.

Let's discuss some of the features of the interactive schnorr identity protocol that make it secure:

The nonce (k)
: In step 1, Alice chooses a number that Bob doesn't know and can't guess and gives him the scaled form of that number, kG. At that point, Bob also already has her public key (xG), which is the scaled form of x, her private key. That means when Bob is working on the final equation ($sG = kG + exG$), there are two independent variables that Bob doesn't know (x and k). It's possible to use simple algebra to solve an equation with one unknown variable but not two independent unknown variables, so the presence of Alice's nonce prevents Bob from being able to derive her private key. It's critical to note that this protection depends on nonces being unguessable in any way. If there's anything predictable about Alice's nonce, Bob may be able to leverage that into figuring out Alice's private key. See The Importance of Randomness in Signatures for more details.

The challenge scalar (e)

Bob waits to receive Alice's public nonce and then proceeds in step 2 to give her a number (the challenge scalar) that Alice didn't previously know and couldn't have guessed. It's critical that Bob only give her the challenge scalar after she commits to her public nonce. Consider what could happen if someone who didn't know x wanted to impersonate Alice, and Bob accidentally gave them the challenge scalar e before they told him the public nonce kG. This allows the impersonator to change parameters on both sides of the equation that Bob will use for verification, $sG == kG + exG$; specifically, they can change both sG and kG. Think about a simplified form of that expression: $x = y + a$. If you can change both x and y, you can cancel out a using $x' = (x - a) + a$. Any value you choose for x will now satisfy the equation. For the actual equation the impersonator simply chooses a random number for s, generates sG, and then uses EC subtraction to select a kG that equals $kG = sG - exG$. They give Bob their calculated kG and later their random sG, and Bob thinks that's valid because $sG == (sG - exG) + exG$. This explains why the order of operations in the protocol is essential: Bob must only give Alice the challenge scalar after Alice has committed to her public nonce.

The interactive identity protocol described here matches part of Claus Schnorr's original description, but it lacks two essential features we need for the decentralized Bitcoin network. The first of these is that it relies on Bob waiting for Alice to commit to her public nonce and then Bob giving her a random challenge scalar. In Bitcoin, the spender of every transaction needs to be authenticated by thousands of Bitcoin full nodes—including future nodes that haven't been started yet but whose operators will one day want to ensure the bitcoins they receive came from a chain of transfers where every transaction was valid. Any Bitcoin node that is unable to communicate with Alice, today or in the future, will be unable to authenticate her transaction and will be in disagreement with every other node that did authenticate it. That's not acceptable for a consensus system like Bitcoin. For Bitcoin to work, we need a protocol that doesn't require interaction between Alice and each node that wants to authenticate her.

A simple technique, known as the Fiat-Shamir transform after its discoverers, can turn the schnorr interactive identity protocol into a noninteractive digital signature scheme. Recall the importance of steps 1 and 2—including that they be performed in order. Alice must commit to an unpredictable nonce; Bob must give Alice an unpredictable challenge scalar only after he has received her commitment. Recall also the properties of secure cryptographic hash functions we've used elsewhere in this book: it will always produce the same output when given the same input but it will produce a value indistinguishable from random data when given a different input.

This allows Alice to choose her private nonce, derive her public nonce, and then hash the public nonce to get the challenge scalar. Because Alice can't predict the output of the hash function (the challenge), and because it's always the same for the same input (the nonce), this ensures that Alice gets a random challenge even though she chooses the nonce and hashes it herself. We no longer need interaction from Bob. She can simply publish her

public nonce kG and the scalar s, and each of the thousands of full nodes (past and future) can hash kG to produce e, use that to produce exG, and then verify $sG == kG + exG$. Written explicitly, the verification equation becomes $sG == kG + hash(kG) \times xG$.

We need one other thing to finish converting the interactive schnorr identity protocol into a digital signature protocol useful for Bitcoin. We don't just want Alice to prove that she knows her private key; we also want to give her the ability to commit to a message. Specifically, we want her to commit to the data related to the Bitcoin transaction she wants to send. With the Fiat-Shamir transform in place, we already have a commitment, so we can simply have it additionally commit to the message. Instead of $hash(kG)$, we now also commit to the message m using $hash(kG \,||\, m)$, where || stands for concatenation.

We've now defined a version of the schnorr signature protocol, but there's one more thing we need to do to address a Bitcoin-specific concern. In BIP32 key derivation, as described in Public Child Key Derivation, the algorithm for unhardened derivation takes a public key and adds to it a nonsecret value to produce a derived public key. That means it's also possible to add that nonsecret value to a valid signature for one key to produce a signature for a related key. That related signature is valid but it wasn't authorized by the person possessing the private key, which is a major security failure. To protect BIP32 unhardened derivation and also support several protocols people wanted to build on top of schnorr signatures, Bitcoin's version of schnorr signatures, called *BIP340 schnorr signatures for secp256k1*, also commits to the public key being used in addition to the public nonce and the message. That makes the full commitment $hash(kG \,||\, xG \,||\, m)$.

Now that we've described each part of the BIP340 schnorr signature algorithm and explained what it does for us, we can define the protocol. Multiplication of integers are performed *modulus p*, indicating that the result of the operation is divided by the number p (as defined in the secp256k1 standard) and the remainder is used. The number p is very large, but if it was 3 and the result of an operation was 5, the actual number we would use is 2 (i.e., 5 divided by 3 has a remainder of 2).

Setup: Alice chooses a large random number (x) as her private key (either directly or by using a protocol like BIP32 to deterministically generate a private key from a large random seed value). She uses the parameters defined in secp256k1 (see Elliptic Curve Cryptography Explained) to multiply the generator G by her scalar x, producing xG (her public key). She gives her public key to everyone who will later authenticate her Bitcoin transactions (e.g., by having xG included in a transaction output). When she's ready to spend, she begins generating her signature:

1. Alice chooses a large random private nonce k and derives the public nonce kG.
2. She chooses her message m (e.g., transaction data) and generates the challenge scalar $e = hash(kG \,||\, xG \,||\, m)$.
3. She produces the scalar $s = k + ex$. The two values kG and s are her signature. She gives this signature to everyone who wants to verify that signature; she also needs to ensure everyone receives her message m. In Bitcoin, this is done by including her

signature in the witness structure of her spending transaction and then relaying that transaction to full nodes.

4. The verifiers (e.g., full nodes) use s to derive sG and then verify that $sG == kG + hash(kG || xG || m) \times xG$. If the equation is valid, Alice proved that she knows her private key x (without revealing it) and committed to the message m (containing the transaction data).

8.2.1. Serialization of Schnorr Signatures

A schnorr signature consists of two values, kG and s. The value kG is a point on Bitcoin's elliptic curve (called secp256k1) and would normally be represented by two 32-byte coordinates, e.g., (x, y). However, only the x coordinate is needed, so only that value is included. When you see kG in schnorr signatures for Bitcoin, note that it's only that point's x coordinate.

The value s is a scalar (a number meant to multiply other numbers). For Bitcoin's secp256k1 curve, it can never be more than 32 bytes long.

Although both kG and s can sometimes be values that can be represented with fewer than 32 bytes, it's improbable that they'd be much smaller than 32 bytes, so they're serialized as two 32-byte values (i.e., values smaller than 32 bytes have leading zeros). They're serialized in the order of kG and then s, producing exactly 64 bytes.

The taproot soft fork, also called v1 segwit, introduced schnorr signatures to Bitcoin and is the only way they are used in Bitcoin as of this writing. When used with either taproot keypath or scriptpath spending, a 64-byte schnorr signature is considered to use a default signature hash (sighash) that is SIGHASH_ALL. If an alternative sighash is used, or if the spender wants to waste space to explicitly specify SIGHASH_ALL, a single additional byte is appended to the signature that specifies the signature hash, making the signature 65 bytes.

As we'll see, either 64 or 65 bytes is considerably more efficient that the serialization used for ECDSA signatures described in Serialization of ECDSA Signatures (DER).

8.2.2. Schnorr-based Scriptless Multisignatures

In the single-signature schnorr protocol described in Schnorr Signatures, Alice uses a signature (kG, s) to publicly prove her knowledge of her private key, which in this case we'll call y. Imagine if Bob also has a private key (z) and he's willing to work with Alice to prove that together they know $x = y + z$ without either of them revealing their private key to each other or anyone else. Let's go through the BIP340 schnorr signature protocol again.

> **Warning** The simple protocol we are about to describe is not secure for the reasons we will explain shortly. We use it only to demonstrate the mechanics of schnorr multisignatures before describing related protocols that are believed to be secure.

Alice and Bob need to derive the public key for x, which is xG. Since it's possible to use elliptic curve operations to add two EC points together, they start by Alice deriving yG and Bob deriving zG. They then add them together to create $xG = yG + zG$. The point xG is their *aggregated public key*. To create a signature, they begin the simple multisignature protocol:

1. They each individually choose a large random private nonce, a for Alice and b for Bob. They also individually derive the corresponding public nonce aG and bG. Together, they produce an aggregated public nonce $kG = aG + bG$.
2. They agree on the message to sign, m (e.g., a transaction), and each generates a copy of the challenge scalar: $e = hash(kG \,||\, xG \,||\, m)$.
3. Alice produces the scalar $q = a + ey$. Bob produces the scalar $r = b + ez$. They add the scalars together to produce $s = q + r$. Their signature is the two values kG and s.
4. The verifiers check their public key and signature using the normal equation: $sG == kG + hash(kG \,||\, xG \,||\, m) \times xG$.

Alice and Bob have proven that they know the sum of their private keys without either one of them revealing their private key to the other or anyone else. The protocol can be extended to any number of participants (e.g., a million people could prove they knew the sum of their million different keys).

The preceding protocol has several security problems. Most notable is that one party might learn the public keys of the other parties before committing to their own public key. For example, Alice generates her public key yG honestly and shares it with Bob. Bob generates his public key using $zG – yG$. When their two keys are combined ($yG + zG – yG$), the positive and negative yG terms cancel out so the public key only represents the private key for z (i.e., Bob's private key). Now Bob can create a valid signature without any assistance from Alice. This is called a *key cancellation attack*.

There are various ways to solve the key cancellation attack. The simplest scheme would be to require each participant commit to their part of the public key before sharing anything about that key with all of the other participants. For example, Alice and Bob each individually hash their public keys and share their digests with each other. When they both have the other's digest, they can share their keys. They individually check that the other's key hashes to the previously provided digest and then proceed with the protocol normally. This prevents either one of them from choosing a public key that cancels out the keys of the other participants. However, it's easy to fail to implement this scheme correctly, such as using it in a naive way with unhardened BIP32 public key derivation. Additionally, it

adds an extra step for communication between the participants, which may be undesirable in many cases. More complex schemes have been proposed that address these shortcomings.

In addition to the key cancellation attack, there are a number of attacks possible against nonces. Recall that the purpose of the nonce is to prevent anyone from being able to use their knowledge of other values in the signature verification equation to solve for your private key, determining its value. To effectively accomplish that, you must use a different nonce every time you sign a different message or change other signature parameters. The different nonces must not be related in any way. For a multisignature, every participant must follow these rules or it could compromise the security of other participants. In addition, cancellation and other attacks need to be prevented. Different protocols that accomplish these aims make different trade-offs, so there's no single multisignature protocol to recommend in all cases. Instead, we'll note three from the MuSig family of protocols:

MuSig
: Also called *MuSig1*, this protocol requires three rounds of communication during the signing process, making it similar to the process we just described. MuSig1's greatest advantage is its simplicity.

MuSig2
: This only requires two rounds of communication and can sometimes allow one of the rounds to be combined with key exchange. This can significantly speed up signing for certain protocols, such as how scriptless multisignatures are planned to be used in the LN. MuSig2 is specified in BIP327 (the only scriptless multisignature protocol that has a BIP as of this writing).

MuSig-DN
: DN stands for Deterministic Nonce, which eliminates as a concern a problem known as the *repeated session attack*. It can't be combined with key exchange and it's significantly more complex to implement than MuSig or MuSig2.

For most applications, MuSig2 is the best multisignature protocol available at the time of writing.

8.2.3. Schnorr-based Scriptless Threshold Signatures

Scriptless multisignature protocols only work for k-of-k signing. Everyone with a partial public key that becomes part of the aggregated public key must contribute a partial signature and partial nonce to the final signature. Sometimes, though, the participants want to allow a subset of them to sign, such as t-of-k where a threshold (t) number of participants can sign for a key constructed by k participants. That type of signature is called a *threshold signature*.

We saw script-based threshold signatures in Scripted Multisignatures. But just as scriptless multisignatures save space and increase privacy compared to scripted multisignatures, *scriptless threshold signatures* save space and increase privacy compared to *scripted threshold signatures*. To anyone not involved in the signing, a *scriptless threshold signature* looks like any other signature that could've been created by a single-sig user or through a scriptless multisignature protocol.

Various methods are known for generating scriptless threshold signatures, with the simplest being a slight modification of how we created scriptless multisignatures previously. This protocol also depends on verifiable secret sharing (which itself depends on secure secret sharing).

Basic secret sharing can work through simple splitting. Alice has a secret number that she splits into three equal-length parts and shares with Bob, Carol, and Dan. Those three can combine the partial numbers they received (called *shares*) in the correct order to reconstruct Alice's secret. A more sophisticated scheme would involve Alice adding on some additional information to each share, called a correction code, that allows any two of them to recover the number. This scheme is not secure because each share gives its holder partial knowledge of Alice's secret, making it easier for the participant to guess Alice's secret than a nonparticipant who didn't have a share.

A secure secret sharing scheme prevents participants from learning anything about the secret unless they combine the minimum threshold number of shares. For example, Alice can choose a threshold of 2 if she wants any two of Bob, Carol, and Dan to be able to reconstruct her secret. The best known secure secret sharing algorithm is *Shamir's Secret Sharing Scheme*, commonly abbreviated SSSS and named after its discoverer, one of the same discoverers of the Fiat-Shamir transform we saw in Schnorr Signatures.

In some cryptographic protocols, such as the scriptless threshold signature schemes we're working toward, it's critical for Bob, Carol, and Dan to know that Alice followed her side of the protocol correctly. They need to know that the shares she creates all derive from the same secret, that she used the threshold value she claims, and that she gave each one of them a different share. A protocol that can accomplish all of that, and still be a secure secret sharing scheme, is a *verifiable secret sharing scheme*.

To see how multisignatures and verifiable secret sharing work for Alice, Bob, and Carol, imagine they each wish to receive funds that can be spent by any two of them. They collaborate as described in Schnorr-based Scriptless Multisignatures to produce a regular multisignature public key to accept the funds (k-of-k). Then each participant derives two secret shares from their private key—one for each of two the other participants. The shares allow any two of them to reconstruct the originating partial private key for the multisignature. Each participant distributes one of their secret shares to the other two participants, resulting in each participant storing their own partial private key and one share for every other participant. Subsequently, each participant verifies the authenticity and uniqueness of the shares they received compared to the shares given to the other participants.

Later on, when (for example) Alice and Bob want to generate a scriptless threshold signature without Carol's involvement, they exchange the two shares they possess for Carol. This enables them to reconstruct Carol's partial private key. Alice and Bob also have their private keys, allowing them to create a scriptless multisignature with all three necessary keys.

In other words, the scriptless threshold signature scheme just described is the same as a scriptless multisignature scheme except that a threshold number of participants have the ability to reconstruct the partial private keys of any other participants who are unable or unwilling to sign.

This does point to a few things to be aware about when considering a scriptless threshold signature protocol:

No accountability
: Because Alice and Bob reconstruct Carol's partial private key, there can be no fundamental difference between a scriptless multisignature produced by a process that involved Carol and one that didn't. Even if Alice, Bob, or Carol claim that they didn't sign, there's no guaranteed way for them to prove that they didn't help produce the signature. If it's important to know which members of the group signed, you will need to use a script.

Manipulation attacks
: Imagine that Bob tells Alice that Carol is unavailable, so they work together to reconstruct Carol's partial private key. Then Bob tells Carol that Alice is unavailable, so they work together to reconstruct Alice's partial private key. Now Bob has his own partial private key plus the keys of Alice and Carol, allowing him to spend the funds himself without their involvement. This attack can be addressed if all of the participants agree to only communicate using a scheme that allows any one of them to see all of the other's messages (e.g., if Bob tells Alice that Carol is unavailable, Carol is able to see that message before she begins working with Bob). Other solutions, possibly more robust solutions, to this problem were being researched at the time of writing.

No scriptless threshold signature protocol has been proposed as a BIP yet, although significant research into the subject has been performed by multiple Bitcoin contributors and we expect peer-reviewed solutions will become available after the publication of this book.

8.3. ECDSA Signatures

Unfortunately for the future development of Bitcoin and many other applications, Claus Schnorr patented the algorithm he discovered and prevented its use in open standards and open source software for almost two decades. Cryptographers in the early 1990s who were

blocked from using the schnorr signature scheme developed an alternative construction called the *Digital Signature Algorithm* (DSA), with a version adapted to elliptic curves called ECDSA.

The ECDSA scheme and standardized parameters for suggested curves it could be used with were widely implemented in cryptographic libraries by the time development on Bitcoin began in 2007. This was almost certainly the reason why ECDSA was the only digital signature protocol that Bitcoin supported from its first release version until the activation of the taproot soft fork in 2021. ECDSA remains supported today for all non-taproot transactions. Some of the differences compared to schnorr signatures include:

More complex
: As we'll see, ECDSA requires more operations to create or verify a signature than the schnorr signature protocol. It's not significantly more complex from an implementation standpoint, but that extra complexity makes ECDSA less flexible, less performant, and harder to prove secure.

Less provable security
: The interactive schnorr signature identification protocol depends only on the strength of the elliptic curve Discrete Logarithm Problem (ECDLP). The non-interactive authentication protocol used in Bitcoin also relies on the random oracle model (ROM). However, ECDSA's extra complexity has prevented a complete proof of its security being published (to the best of our knowledge). We are not experts in proving cryptographic algorithms, but it seems unlikely after 30 years that ECDSA will be proven to only require the same two assumptions as schnorr.

Nonlinear
: ECDSA signatures cannot be easily combined to create scriptless multisignatures or used in related advanced applications, such as multiparty signature adaptors. There are workarounds for this problem, but they involve additional extra complexity that significantly slows down operations and which, in some cases, has resulted in software accidentally leaking private keys.

8.3.1. ECDSA Algorithm

Let's look at the math of ECDSA. Signatures are created by a mathematical function F_{sig} that produces a signature composed of two values. In ECDSA, those two values are R and s.

The signature algorithm first generates a private nonce (k) and derives from it a public nonce (K). The R value of the digital signature is then the x coordinate of the nonce K.

From there, the algorithm calculates the s value of the signature. Like we did with schnorr signatures, operations involving integers are modulus p:

$$s = k^{-1}(Hash(m) + x \times R)$$

where:

- k is the private nonce
- R is the x coordinate of the public nonce
- x is the Alice's private key
- m is the message (transaction data)

Verification is the inverse of the signature generation function, using the R, s values and the public key to calculate a value K, which is a point on the elliptic curve (the public nonce used in signature creation):

$$K = s^{-1} \times Hash(m) \times G + s^{-1} \times R \times X$$

where:

- R and s are the signature values
- X is Alice's public key
- m is the message (the transaction data that was signed)
- G is the elliptic curve generator point

If the x coordinate of the calculated point K is equal to R, then the verifier can conclude that the signature is valid.

> **Tip** ECDSA is necessarily a fairly complicated piece of math; a full explanation is beyond the scope of this book. A number of great guides online take you through it step by step: search for "ECDSA explained."

8.3.2. Serialization of ECDSA Signatures (DER)

Let's look at the following DER-encoded signature:

```
3045022100884d142d86652a3f47ba4746ec719bbfbd040a570b1deccbb6498c75c4ae24cb02204
b9f039ff08df09cbe9f6addac960298cad530a863ea8f53982c09db8f6e381301
```

That signature is a serialized byte stream of the R and s values produced by the signer to prove control of the private key authorized to spend an output. The serialization format consists of nine elements as follows:

- 0x30, indicating the start of a DER sequence
- 0x45, the length of the sequence (69 bytes)
- 0x02, an integer value follows

- 0x21, the length of the integer (33 bytes)
- R, 00884d142d86652a3f47ba4746ec719bbfbd040a570b1deccbb6498c75c4ae24cb
- 0x02, another integer follows
- 0x20, the length of the integer (32 bytes)
- S, 4b9f039ff08df09cbe9f6addac960298cad530a863ea8f53982c09db8f6e3813
- A suffix (0x01) indicating the type of hash used (SIGHASH_ALL)

8.4. The Importance of Randomness in Signatures

As we saw in Schnorr Signatures and ECDSA Signatures, the signature generation algorithm uses a random number *k* as the basis for a private/public nonce pair. The value of *k* is not important, *as long as it is random*. If signatures from the same private key use the private nonce *k* with different messages (transactions), then the signing *private key* can be calculated by anyone. Reuse of the same value for *k* in a signature algorithm leads to exposure of the private key!

> **Warning** If the same value *k* is used in the signing algorithm on two different transactions, the private key can be calculated and exposed to the world!

This is not just a theoretical possibility. We have seen this issue lead to exposure of private keys in a few different implementations of transaction-signing algorithms in Bitcoin. People have had funds stolen because of inadvertent reuse of a *k* value. The most common reason for reuse of a *k* value is an improperly initialized random-number generator.

To avoid this vulnerability, the industry best practice is to not generate *k* with a random-number generator seeded only with entropy, but instead to use a process seeded in part with the transaction data itself plus the private key being used to sign. This ensures that each transaction produces a different *k*. The industry-standard algorithm for deterministic initialization of *k* for ECDSA is defined in RFC6979, published by the Internet Engineering Task Force. For schnorr signatures, BIP340 recommends a default signing algorithm.

BIP340 and RFC6979 can generate *k* entirely deterministically, meaning the same transaction data will always produce the same *k*. Many wallets do this because it makes it easy to write tests to verify their safety-critical signing code is producing *k* values correctly. BIP340 and RFC6979 both also allow including additional data in the calculation. If that data is entropy, then a different *k* will be produced even if the exact same transaction data is signed. This can increase protection against sidechannel and fault-injection attacks.

If you are implementing an algorithm to sign transactions in Bitcoin, you *must* use BIP340, RFC6979, or a similar algorithm to ensure you generate a different *k* for each transaction.

8.5. Segregated Witness’s New Signing Algorithm

Signatures in Bitcoin transactions are applied on a *commitment hash*, which is calculated from the transaction data, locking specific parts of the data indicating the signer’s commitment to those values. For example, in a simple SIGHASH_ALL type signature, the commitment hash includes all inputs and outputs.

Unfortunately, the way the legacy commitment hashes were calculated introduced the possibility that a node verifying a signature can be forced to perform a significant number of hash computations. Specifically, the hash operations increase roughly quadratically with respect to the number of inputs in the transaction. An attacker could therefore create a transaction with a very large number of signature operations, causing the entire Bitcoin network to have to perform hundreds or thousands of hash operations to verify the transaction.

Segwit represented an opportunity to address this problem by changing the way the commitment hash is calculated. For segwit version 0 witness programs, signature verification occurs using an improved commitment hash algorithm as specified in BIP143.

The new algorithm allows the number of hash operations to increase by a much more gradual O(n) to the number of signature operations, reducing the opportunity to create denial-of-service attacks with overly complex transactions.

In this chapter, we learned about schnorr and ECDSA signatures for Bitcoin. This explains how full nodes authenticate transactions to ensure that only someone controlling the key to which bitcoins were received can spend those bitcoins. We also examined several advanced applications of signatures, such as scriptless multisignatures and scriptless threshold signatures that can be used to improve the efficiency and privacy of Bitcoin. In the past few chapters, we’ve learned how to create transactions, how to secure them with authorization and authentication, and how to sign them. We will next learn how to encourage miners to confirm them by adding fees to the transactions we create.

9. Transaction Fees

The digital signature we saw Alice create in #c_signatures only proves that she knows her private key and that she committed to a transaction that pays Bob. She can create another signature that instead commits to a transaction paying Carol—a transaction that spends the same output (bitcoins) that she used to pay Bob. Those two transactions are now *conflicting transactions* because only one transaction spending a particular output can be included in the valid blockchain with the most proof of work—the blockchain that full nodes use to determine which keys control which bitcoins.

To protect himself against conflicting transactions, it would be wise for Bob to wait until the transaction from Alice is included in the blockchain to a sufficient depth before he considers the money he received as his to spend (see Confirmations). For Alice's transaction to be included in the blockchain, it must be included in a *block* of transactions. There are a limited number of blocks produced in a given amount of time, and each block only has a limited amount of space. Only the miner who creates that block gets to choose which transactions to include. Miners may select transactions by any criteria they want, including refusing to include any transactions at all.

When we say "transactions" in this chapter, we refer to every transaction in a block except for the first transaction. The first transaction in a block is a *coinbase transaction*, described in #coinbase_transactions, which allows the miner of the block to collect their reward for producing the block. Unlike other transactions, a coinbase transaction doesn't spend the output of a previous transaction and is also an exception to several other rules that apply to other transactions. Coinbase transactions don't pay transaction fees, don't need to be fee bumped, aren't subject to transaction pinning, and are largely uninteresting to the following discussion about fees—so we're going to ignore them in this chapter.

The criterion that almost all miners use to select which transactions to include in their blocks is to maximize their revenue. Bitcoin was specifically designed to accommodate this by providing a mechanism that allows a transaction to give money to the miner who includes that transaction in a block. We call that mechanism *transaction fees*, although it's not a fee in the usual sense of that word. It's not an amount set by the protocol or by any particular miner—it's much more like a bid in an auction. The good being purchased is the portion of limited space in a block that a transaction will consume. Miners choose the set of transactions whose bids will allow them to earn the greatest revenue.

In this chapter, we'll explore various aspects of those bids—transaction fees—and how they influence the creation and management of Bitcoin transactions.

9.1. Who Pays the Transaction Fee?

Most payment systems involve some sort of fee for transacting, but often this fee is hidden from typical buyers. For example, a merchant may advertise the same item for the same price whether you pay with cash or a credit card even though their payment processor may charge them a higher fee for credit transactions than their bank charges them for cash deposits.

In Bitcoin, every spend of bitcoins must be authenticated (typically with a signature), so it's not possible for a transaction to pay a fee without the permission of the spender. It is possible for the receiver of a transaction to pay a fee in a different transaction—and we'll see that in use later—but if we want a single transaction to pay its own fee, that fee needs to be something agreed upon by the spender. It can't be hidden.

Bitcoin transactions are designed so that it doesn't take any extra space in a transaction for a spender to commit to the fee it pays. That means that, even though it's possible to pay the fee in a different transaction, it's most efficient (and thus cheapest) to pay the fee in a single transaction.

In Bitcoin, the fee is a bid and the amount paid contributes to determining how long it will take the transaction to confirm. Both spenders and receivers of a payment typically have an interest in having it confirming quickly, so normally allowing only spenders to choose fees can sometimes be a problem; we'll look at a solution to that problem in Child Pays for Parent (CPFP) Fee Bumping. However, in many common payment flows, the parties with the highest desire to see a transaction confirm quickly—that is, the parties who would be the most willing to pay higher fees—are the spenders.

For those reasons, both technical and practical, it is customary in Bitcoin for spenders to pay transaction fees. There are exceptions, such as for merchants that accept unconfirmed transactions and in protocols that don't immediately broadcast transactions after they are signed (preventing the spender from being able to choose an appropriate fee for the current market). We'll explore those exceptions later.

9.2. Fees and Fee Rates

Each transaction only pays a single fee—it doesn't matter how large the transaction is. However, the larger transactions become, the fewer of them a miner will be able to fit in a block. For that reason, miners evaluate transactions the same way you might comparison shop between several equivalent items at the market: they divide the price by the quantity.

Whereas you might divide the cost of several different bags of rice by each bag's weight to find the lowest price per weight (best deal), miners divide the fee of a transaction by its size (also called its weight) to find the highest fee per weight (most revenue). In Bitcoin, we use the term *fee rate* for a transaction's size divided by weight. Due to changes in Bitcoin over the years, fee rate can be expressed in different units:

- BTC/Bytes (a legacy unit rarely used anymore)
- BTC/Kilobytes (a legacy unit rarely used anymore)
- BTC/Vbytes (rarely used)
- BTC/Kilo-vbyte (used mainly in Bitcoin Core)
- Satoshi/Vbyte (most commonly used today)
- Satoshi/Weight (also commonly used today)

We recommend either the sat/vbyte or sat/weight units for displaying fee rates.

> **Warning** Be careful accepting input for fee rates. If a user copies and pastes a fee rate printed in one denominator into a field using a different denominator, they could overpay fees by 1,000 times. If they instead switch the numerator, they could theoretically overpay by 100,000,000 times. Wallets should make it hard for the user to pay an excessive fee rate and may want to prompt the user to confirm any fee rate that was not generated by the wallet itself using a trusted data source.
>
> An excessive fee, also called an *absurd fee*, is any fee rate that's significantly higher than the amount that fee rate estimators currently expect is necessary to get a transaction confirmed in the next block. Note that wallets should not entirely prevent users from choosing an excessive fee rate—they should only make using such a fee rate hard to do by accident. There are legitimate reasons for users to overpay fees on rare occasions.

9.3. Estimating Appropriate Fee Rates

We've established that you can pay a lower fee rate if you're willing to wait longer for your transaction to be confirmed, with the exception that paying too low of a fee rate could result in your transaction never confirming. Because fee rates are bids in an open auction for block space, it's not possible to perfectly predict what fee rate you need to pay to get your transaction confirmed by a certain time. However, we can generate a rough estimate based on what fee rates other transactions have paid in the recent past.

A full node can record three pieces of information about each transactions it sees: the time (block height) when it first received that transaction, the block height when that transaction was confirmed, and the fee rate paid by that transaction. By grouping together transactions that arrived at similar heights, were confirmed at similar heights, and which paid similar fees, we can calculate how many blocks it took to confirm transactions paying a certain fee rate. We can then assume that a transaction paying a similar fee rate now will take a similar number of blocks to confirm. Bitcoin Core includes a fee rate estimator that

uses these principles, which can be called using the `estimatesmartfee` RPC with a parameter specifying how many blocks you're willing to wait before the transaction is highly likely to confirm (for example, 144 blocks is about 1 day):

```
$ bitcoin-cli -named estimatesmartfee conf_target=144
{
  "feerate": 0.00006570,
  "blocks": 144
}
```

Many web-based services also provide fee estimation as an API. For a current list, see https://oreil.ly/TB6IN.

As mentioned, fee rate estimation can never be perfect. One common problem is that the fundamental demand might change, adjusting the equilibrium and either increasing prices (fees) to new heights or decreasing them toward the minimum. If fee rates go down, then a transaction that previously paid a normal fee rate might now be paying a high fee rate and it will be confirmed earlier than expected. There's no way to lower the fee rate on a transaction you've already sent, so you're stuck paying a higher fee rate. But, when fee rates go up, there's a need for methods to be able to increase the fee rates on those transactions, which is called *fee bumping*. There are two commonly used types of fee bumping in Bitcoin, replace by fee (RBF) and child pays for parent (CPFP).

9.4. Replace By Fee (RBF) Fee Bumping

To increase the fee of a transaction using RBF fee bumping, you create a conflicting version of the transaction that pays a higher fee. Two or more transactions are considered to be *conflicting transactions* if only one of them can be included in a valid blockchain, forcing a miner to choose only one of them. Conflicts occur when two or more transactions each try to spend one of the same UTXOs, i.e., they each include an input that has the same outpoint (reference to the output of a previous transaction).

To prevent someone from consuming large amounts of bandwidth by creating an unlimited number of conflicting transactions and sending them through the network of relaying full nodes, Bitcoin Core and other full nodes that support transaction replacement require each replacement transaction to pay a higher fee rate than the transaction being replaced. Bitcoin Core also currently requires the replacement transaction to pay a higher total fee than the original transaction, but this requirement has undesired side effects and developers have been looking for ways to remove it at the time of writing.

Bitcoin Core currently supports two variations of RBF:

Opt-in RBF
: An unconfirmed transaction can signal to miners and full nodes that the creator of the transaction wants to allow it to be replaced by a higher fee rate version. This signal and the rules for using it are specified in BIP125. As of this writing, this has been enabled by default in Bitcoin Core for several years.

Full RBF

: Any unconfirmed transaction can be replaced by a higher fee rate version. As of this writing, this can be optionally enabled in Bitcoin Core (but it is disabled by default).

Why Are There Two Variants of RBF?

The reason for the two different versions of RBF is that full RBF has been controversial. Early versions of Bitcoin allowed transaction replacement, but this behavior was disabled for several releases. During that time, a miner or full node using the software now called Bitcoin Core would not replace the first version of an unconfirmed transaction they received with any different version. Some merchants came to expect this behavior: they assumed that any valid unconfirmed transaction that paid an appropriate fee rate would eventually become a confirmed transaction, so they provided their goods or services shortly after receiving such an unconfirmed transaction.

However, there's no way for the Bitcoin protocol to guarantee that any unconfirmed transaction will eventually be confirmed. As mentioned earlier in this chapter, every miner gets to choose for themselves which transactions they will try to confirm—including which versions of those transactions. Bitcoin Core is open source software, so anyone with a copy of its source code can add (or remove) transaction replacement. Even if Bitcoin Core wasn't open source, Bitcoin is an open protocol that can be reimplemented from scratch by a sufficiently competent programmer, allowing the reimplementor to include or not include transaction replacement.

Transaction replacement breaks the assumption of some merchants that every reasonable unconfirmed transaction will eventually be confirmed. An alternative version of a transaction can pay the same outputs as the original, but it isn't required to pay any of those outputs. If the first version of an unconfirmed transaction pays a merchant, the second version might not pay them. If the merchant provided goods or services based on the first version, but the second version gets confirmed, then the merchant will not receive payment for its costs.

Some merchants, and people supporting them, requested that transaction replacement not be reenabled in Bitcoin Core. Other people pointed out that transaction replacement provides benefits, including the ability to fee bump transactions that initially paid too low of a fee rate.

Eventually, developers working on Bitcoin Core implemented a compromise: instead of allowing every unconfirmed transaction to be replaced (full RBF), they only programmed Bitcoin Core to allow replacement of transactions that signaled they wanted to allow replacement (opt-in RBF). Merchants can check the transactions they receive for the opt-in signal and treat those transactions differently than those without the signal.

This doesn't change the fundamental concern: anyone can still alter their copy of Bitcoin Core, or create a reimplementation, to allow full RBF—and some developers even did this, but seemingly few people used their software.

After several years, developers working on Bitcoin Core changed the compromise slightly. In addition to keeping opt-in RBF by default, they added an option that allows users to enable full RBF. If enough mining hash rate and relaying full nodes enable this option, it will be possible for any unconfirmed transaction to eventually be replaced by a version paying a higher fee rate. As of this writing, it’s not clear whether or not that has happened yet.

As a user, if you plan to use RBF fee bumping, you will first need to choose a wallet that supports it, such as one of the wallets listed as having "Sending support" on https://oreil.ly/IhMzx.

As a developer, if you plan to implement RBF fee bumping, you will first need to decide whether to perform opt-in RBF or full RBF. At the time of writing, opt-in RBF is the only method that’s sure to work. Even if full RBF becomes reliable, there will likely be several years where replacements of opt-in transactions get confirmed slightly faster than full-RBF replacements. If you choose opt-in RBF, your wallet will need to implement the signaling specified in BIP125, which is a simple modification to any one of the sequence fields in a transaction (see Sequence). If you choose full RBF, you don’t need to include any signaling in your transactions. Everything else related to RBF is the same for both approaches.

When you need to fee bump a transaction, you will simply create a new transaction that spends at least one of the same UTXOs as the original transaction you want to replace. You will likely want to keep the same outputs in the transaction that pay the receiver (or receivers). You may pay the increased fee by reducing the value of your change output or by adding additional inputs to the transaction. Developers should provide users with a fee-bumping interface that does all of this work for them and simply asks them (or suggests to them) how much the fee rate should be increased.

Warning Be very careful when creating more than one replacement of the same transaction. You must ensure than all versions of the transactions conflict with each other. If they aren't all conflicts, it may be possible for multiple separate transactions to confirm, leading you to overpay the receivers. For example:

- Transaction version 0 includes input *A*.
- Transaction version 1 includes inputs *A* and *B* (e.g., you had to add input *B* to pay the extra fees)
- Transaction version 2 includes inputs *B* and *C* (e.g., you had to add input *C* to pay the extra fees but *C* was large enough that you no longer need input *A*).

In this scenario, any miner who saved version 0 of the transaction will be able to confirm both it and version 2 of the transaction. If both versions pay the same receivers, they'll be paid twice (and the miner will receive transaction fees from two separate transactions).

A simple method to avoid this problem is to ensure the replacement transaction always includes all of the same inputs as the previous version of the transaction.

The advantage of RBF fee bumping over other types of fee bumping is that it can be very efficient at using block space. Often, a replacement transaction is the same size as the transaction it replaces. Even when it's larger, it's often the same size as the transaction the user would have created if they had paid the increased fee rate in the first place.

The fundamental disadvantage of RBF fee bumping is that it can normally only be performed by the creator of the transaction—the person or people who were required to provide signatures or other authentication data for the transaction. An exception to this is transactions that were designed to allow additional inputs to be added by using sighash flags (see Signature Hash Types (SIGHASH)), but that presents its own challenges. In general, if you're the receiver of an unconfirmed transaction and you want to make it confirm faster (or at all), you can't use an RBF fee bump; you need some other method.

There are additional problems with RBF that we'll explore in Transaction Pinning.

9.5. Child Pays for Parent (CPFP) Fee Bumping

Anyone who receives the output of an unconfirmed transaction can incentivize miners to confirm that transaction by spending that output. The transaction you want to get confirmed is called the *parent transaction*. A transaction that spends an output of the parent transaction is called a *child transaction*.

As we learned in Outpoint, every input in a confirmed transaction must reference the unspent output of a transaction that appears earlier in the blockchain (whether earlier in the same block or in a previous block). That means a miner who wants to confirm a child transaction must also ensure that its parent transaction is confirmed. If the parent transaction hasn't been confirmed yet but the child transaction pays a high enough fee, the miner can consider whether it would be profitable to confirm both of them in the same block.

To evaluate the profitability of mining both a parent and child transaction, the miner looks at them as a *package of transactions* with an aggregate size and aggregate fees, from which the fees can be divided by the size to calculate a *package fee rate*. The miner can then sort all of the individual transactions and transaction packages they know about by fee rate and include the highest-revenue ones in the block they're attempting to mine, up to the maximum size (weight) allowed to be included in a block. To find even more packages that might be profitable to mine, the miner can evaluate packages across multiple generations (e.g., an unconfirmed parent transaction being combined with both its child and grandchild). This is called *ancestor fee rate mining*.

Bitcoin Core has implemented ancestor fee rate mining for many years, and it's believed that almost all miners use it at the time of writing. That means it's practical for wallets to use this feature to fee bump an incoming transaction by using a child transaction to pay for its parent (CPFP).

CPFP has several advantages over RBF. Anyone who receives an output from a transaction can use CPFP—that includes both the receivers of payments and the spender (if the spender included a change output). It also doesn't require replacing the original transaction, which makes it less disruptive to some merchants than RBF.

The primary disadvantage of CPFP compared to RBF is that CPFP typically uses more block space. In RBF, a fee bump transaction is often the same size as the transaction it replaces. In CPFP, a fee bump adds a whole separate transaction. Using extra block space requires paying extra fees beyond the cost of the fee bump.

There are several challenges with CPFP, some of which we'll explore in Transaction Pinning. One other problem that we specifically need to mention is the minimum relay fee rate problem, which is addressed by package relay.

9.6. Package Relay

Early versions of Bitcoin Core didn't place any limits on the number of unconfirmed transactions they stored for later relay and mining in their mempools (see Mempools and Orphan Pools). Of course, computers have physical limits, whether it's the memory (RAM) or disk space—it's not possible for a full node to store an unlimited number of unconfirmed transactions. Later versions of Bitcoin Core limited the size of the mempool to hold about one day's worth of transactions, storing only the transactions or packages with the highest fee rate.

That works extremely well for most things, but it creates a dependency problem. In order to calculate the fee rate for a transaction package, we need both the parent and descendant transactions—but if the parent transaction doesn't pay a high enough fee rate, it won't be kept in a node's mempool. If a node receives a child transaction without having access to its parent, it can't do anything with that transaction.

The solution to this problem is the ability to relay transactions as a package, called *package relay*, allowing the receiving node to evaluate the fee rate of the entire package before operating on any individual transaction. As of this writing, developers working on Bitcoin Core have made significant progress on implementing package relay, and a limited early version of it may be available by the time this book is published.

Package relay is especially important for protocols based on time-sensitive presigned transactions, such as Lightning Network (LN). In non-cooperative cases, some presigned transactions can't be fee bumped using RBF, forcing them to depend on CPFP. In those protocols, some transactions may also be created long before they need to be broadcast, making it effectively impossible to estimate an appropriate fee rate. If a presigned transaction pays a fee rate below the amount necessary to get into a node's mempool, there's no way to fee bump it with a child. If that prevents the transaction from confirming in time, an honest user might lose money. Package relay is the solution for this critical problem.

9.7. Transaction Pinning

Although both RBF and CPFP fee bumping work in the basic cases we described, there are rules related to both methods that are designed to prevent denial-of-service attacks on miners and relaying full nodes. An unfortunate side effect of those rules is that they can sometimes prevent someone from being able to use fee bumping. Making it impossible or difficult to fee bump a transaction is called *transaction pinning*.

One of the major denial of service concerns revolves around the effect of transaction relationships. Whenever the output of a transaction is spent, that transaction's identifier (txid) is referenced by the child transaction. However, when a transaction is replaced, the replacement has a different txid. If that replacement transaction gets confirmed, none of its descendants can be included in the same blockchain. It's possible to re-create and re-sign the descendant transactions, but that's not guaranteed to happen. This has related but divergent implications for RBF and CPFP:

- In the context of RBF, when Bitcoin Core accepts a replacement transaction, it keeps things simple by forgetting about the original transaction and all descendant transactions that depended on that original. To ensure that it's more profitable for miners to accept replacements, Bitcoin Core only accepts a replacement transaction if it pays more fees than all the transactions that will be forgotten.

The downside of this approach is that Alice can create a small transaction that pays Bob. Bob can then use his output to create a large child transaction. If Alice then wants to replace her original transaction, she needs to pay a fee that's larger than what both she and Bob originally paid. For example, if Alice's original transaction was about 100 vbytes and Bob's transaction was about 100,000 vbytes, and they both used the same fee rate, Alice now needs to pay more than 1,000 times as much as she originally paid in order to RBF fee bump her transaction.

- In the context of CPFP, any time the node considers including a package in a block, it must remove the transactions in that package from any other package it wants to consider for the same block. For example, if a child transaction pays for 25 ancestors, and each of those ancestors has 25 other children, then including the package in the block requires updating approximately 625 packages (25^2). Similarly, if a transaction with 25 descendants is removed from a node's mempool (such as for being included in a block), and each of those descendants has 25 other ancestors, another 625 packages need to be updated. Each time we double our parameter (e.g., from 25 to 50), we quadruple the amount of work our node needs to perform.

  Additionally, a transaction and all of its descendants is not useful to keep in a mempool long term if an alternative version of that transaction is mined—none of those transactions can now be confirmed unless there's a rare blockchain reorganization. Bitcoin Core will remove from its mempool every transaction that can no longer be confirmed on the current blockchain. At it's worst, that can waste an enormous amount of your node's bandwidth and possibly be used to prevent transactions from propagating correctly.

  To prevent these problems, and other related problems, Bitcoin Core limits a parent transaction to having a maximum of 25 ancestors or descendants in its mempool and limits the total size of all those transactions to 100,000 vbytes. The downside of this approach is that users are prevented from creating CPFP fee bumps if a transaction already has too many descendants (or if it and its descendants are too large).

Transaction pinning can happen by accident, but it also represents a serious vulnerability for multiparty time-sensitive protocols such as LN. If your counterparty can prevent one of your transactions from confirming by a deadline, they may be able to steal money from you.

Protocol developers have been working on mitigating problems with transaction pinning for several years. One partial solution is described in CPFP Carve Out and Anchor Outputs. Several other solutions have been proposed, and at least one solution is being actively developed as of this writing—ephemeral anchors.

9.8. CPFP Carve Out and Anchor Outputs

In 2018, developers working on LN had a problem. Their protocol uses transactions that require signatures from two different parties. Neither party wants to trust the other, so they sign transactions at a point in the protocol when trust isn't needed, allowing either of them to broadcast one of those transactions at a later time when the other party may not want to (or be able to) fulfill its obligations. The problem with this approach is that the transactions might need to be broadcast at an unknown time, far in the future, beyond any reasonable ability to estimate an appropriate fee rate for the transactions.

In theory, the developers could have designed their transactions to allow fee bumping with either RBF (using special sighash flags) or CPFP, but both of those protocols are vulnerable to transaction pinning. Given that the involved transactions are time sensitive, allowing a counterparty to use transaction pinning to delay confirmation of a transaction can easily lead to a repeatable exploit that malicious parties could use to steal money from honest parties.

LN developer Matt Corallo proposed a solution: give the rules for CPFP fee bumping a special exception, called *CPFP carve out*. The normal rules for CPFP forbid the inclusion of an additional descendant if it would cause a parent transaction to have 26 or more descendants or if it would cause a parent and all of its descendants to exceed 100,000 vbytes in size. Under the rules of CPFP carve out, a single additional transaction up to 1,000 vbytes in size can be added to a package even if it would exceed the other limits as long as it is a direct child of an unconfirmed transaction with no unconfirmed ancestors.

For example, Bob and Mallory both co-sign a transaction with two outputs, one to each of them. Mallory broadcasts that transaction and uses her output to attach either 25 child transactions or any smaller number of child transactions equaling 100,000 vbytes in size. Without carve-out, Bob would be unable to attach another child transaction to his output for CPFP fee bumping. With carve-out, he can spend one of the two outputs in the transaction, the one that belongs to him, as long as his child transaction is less than 1,000 vbytes in size (which should be more than enough space).

It’s not allowed to use CPFP carve-out more than once, so it only works for two-party protocols. There have been proposals to extend it to protocols involving more participants, but there hasn’t been much demand for that and developers are focused on building more generic solutions to transaction pinning attacks.

As of this writing, most popular LN implementations use a transaction template called *anchor outputs*, which is designed to be used with CPFP carve out.

9.9. Adding Fees to Transactions

The data structure of transactions does not have a field for fees. Instead, fees are implied as the difference between the sum of inputs and the sum of outputs. Any excess amount that remains after all outputs have been deducted from all inputs is the fee that is

collected by the miners:

$$Fees = Sum(Inputs) - Sum(Outputs)$$

This is a somewhat confusing element of transactions and an important point to understand because if you are constructing your own transactions, you must ensure you do not inadvertently include a very large fee by underspending the inputs. That means that you must account for all inputs, if necessary, by creating change, or you will end up giving the miners a very big tip!

For example, if you spend a 20-bitcoin UTXO to make a 1-bitcoin payment, you must include a 19-bitcoin change output back to your wallet. Otherwise, the 19-bitcoin "leftover" will be counted as a transaction fee and will be collected by the miner who mines your transaction in a block. Although you will receive priority processing and make a miner very happy, this is probably not what you intended.

> **Warning** If you forget to add a change output in a manually constructed transaction, you will be paying the change as a transaction fee. "Keep the change!" might not be what you intended.

9.10. Timelock Defense Against Fee Sniping

Fee sniping is a theoretical attack scenario where miners attempting to rewrite past blocks "snipe" higher-fee transactions from future blocks to maximize their profitability.

For example, let’s say the highest block in existence is block #100,000. If instead of attempting to mine block #100,001 to extend the chain, some miners attempt to remine #100,000. These miners can choose to include any valid transaction (that hasn’t been mined yet) in their candidate block #100,000. They don’t have to remine the block with the same transactions. In fact, they have the incentive to select the most profitable (highest fee per kB) transactions to include in their block. They can include any transactions that were in the "old" block #100,000, as well as any transactions from the current mempool. Essentially they have the option to pull transactions from the "present" into the rewritten "past" when they re-create block #100,000.

Today, this attack is not very lucrative because the block subsidy is much higher than total fees per block. But at some point in the future, transaction fees will be the majority of the reward (or even the entirety of the reward). At that time, this scenario becomes inevitable.

Several wallets discourage fee sniping by creating transactions with a lock time that limits those transactions to being included in the next block or any later block. In our scenario, our wallet would set lock time to 100,001 on any transaction it created. Under normal circumstances, this lock time has no effect—the transactions could only be included in block #100,001 anyway; it’s the next block.

But under a reorganization attack, the miners would not be able to pull high-fee transactions from the mempool because all those transactions would be timelocked to block #100,001. They can only remine #100,000 with whatever transactions were valid at that time, essentially gaining no new fees.

This does not entirely prevent fee sniping, but it does make it less profitable in some cases and can help preserve the stability of the Bitcoin network as the block subsidy declines. We recommend all wallets implement anti-fee sniping when it doesn’t interfere with the wallet’s other uses of the lock time field.

As Bitcoin continues to mature, and as the subsidy continues to decline, fees become more and more important to Bitcoin users, both in their day-to-day use for getting transactions confirmed quickly and in providing an incentive for miners to continue securing Bitcoin transactions with new proof of work.

10. The Bitcoin Network

Bitcoin is structured as a peer-to-peer network architecture on top of the internet. The term peer-to-peer, or P2P, means that the full nodes that participate in the network are peers to each other, that they can all perform the same functions, and that there are no "special" nodes. The network nodes interconnect in a mesh network with a "flat" topology. There is no server, no centralized service, and no hierarchy within the network. Nodes in a P2P network both provide and consume services at the same time. P2P networks are inherently resilient, decentralized, and open. A preeminent example of a P2P network architecture was the early internet itself, where nodes on the IP network were equal. Today's internet architecture is more hierarchical, but the Internet Protocol still retains its flat-topology essence. Beyond Bitcoin and the internet, the largest and most successful application of P2P technologies is file sharing, with Napster as the pioneer and BitTorrent as the most recent evolution of the architecture.

Bitcoin's P2P network architecture is much more than a topology choice. Bitcoin is a P2P digital cash system by design, and the network architecture is both a reflection and a foundation of that core characteristic. Decentralization of control is a core design principle that can only be achieved and maintained by a flat and decentralized P2P consensus network.

The term "Bitcoin network" refers to the collection of nodes running the Bitcoin P2P protocol. In addition to the Bitcoin P2P protocol, there are other protocols that are used for mining and lightweight wallets. These additional protocols are provided by gateway routing servers that access the Bitcoin network using the Bitcoin P2P protocol and then extend that network to nodes running other protocols. For example, Stratum servers connect Stratum mining nodes via the Stratum protocol to the main Bitcoin network and bridge the Stratum protocol to the Bitcoin P2P protocol. We will describe some of the most commonly used of those protocols in this chapter in addition to the base Bitcoin P2P protocol.

10.1. Node Types and Roles

Although full nodes (peers) in the Bitcoin P2P network are equal to each other, they may take on different roles depending on the functionality they are supporting. A Bitcoin full node validates blocks and may contain other functions, such as routing, mining, and wallet services.

Some nodes, called *archival full nodes*, also maintain a complete and up-to-date copy of the blockchain. Those nodes can serve data to clients that store only a subset of the blockchain and partly verify transactions using a method called *simplified payment verification*, or SPV. These clients are known as lightweight clients.

Miners compete to create new blocks by running specialized hardware to solve the proof-of-work algorithm. Some miners operate full nodes, validating every block on the blockchain, while others are clients participating in pool mining and depending on a pool server to provide them with work.

User wallets might connect to the user's own full node, as is sometimes the case with desktop Bitcoin clients, but many user wallets, especially those running on resource-constrained devices such as smartphones, are lightweight nodes.

In addition to the main node types on the Bitcoin P2P protocol, there are servers and nodes running other protocols, such as specialized mining pool protocols and lightweight client-access protocols.

10.2. The Network

As of this writing, the main Bitcoin network, running the Bitcoin P2P protocol, consists of about 10,000 listening nodes running various versions of Bitcoin Core and a few hundred nodes running various other implementations of the Bitcoin P2P protocol such as BitcoinJ, btcd, and bcoin. A small percentage of the nodes on the Bitcoin P2P network are also mining nodes. Various individuals and companies interface with the Bitcoin network by running archival full nodes, with full copies of the blockchain and a network node, but without mining or wallet functions. These nodes act as network edge routers, allowing various other services (exchanges, wallets, block explorers, merchant payment processing) to be built on top.

10.3. Compact Block Relay

When a miner finds a new block, they announce it to the Bitcoin network (which includes other miners). The miner who found that block can start building on top of it immediately; all other miners who haven't learned about the block yet will continue building on top of the previous block until they do learn about it.

If, before they learn about the new block, one of those other miners creates a block, their block will be in competition with the first miner's new block. Only one of the blocks will ever be included in the blockchain used by all full nodes, and miners only get paid for blocks that are widely accepted.

Whichever block has a second block built on top of it first wins (unless there's another near-tie), which is called a *block-finding race* and is illustrated in A blockchain fork requiring a mining race.. Block-finding races give the advantage to the largest miners, so they act in opposition to Bitcoin's essential decentralization. To prevent block-finding races and allow miners of any size to participate equally in the lottery that is Bitcoin mining, it's extremely useful to minimize the time between when one miner announces a new block and when other miners receive that block.

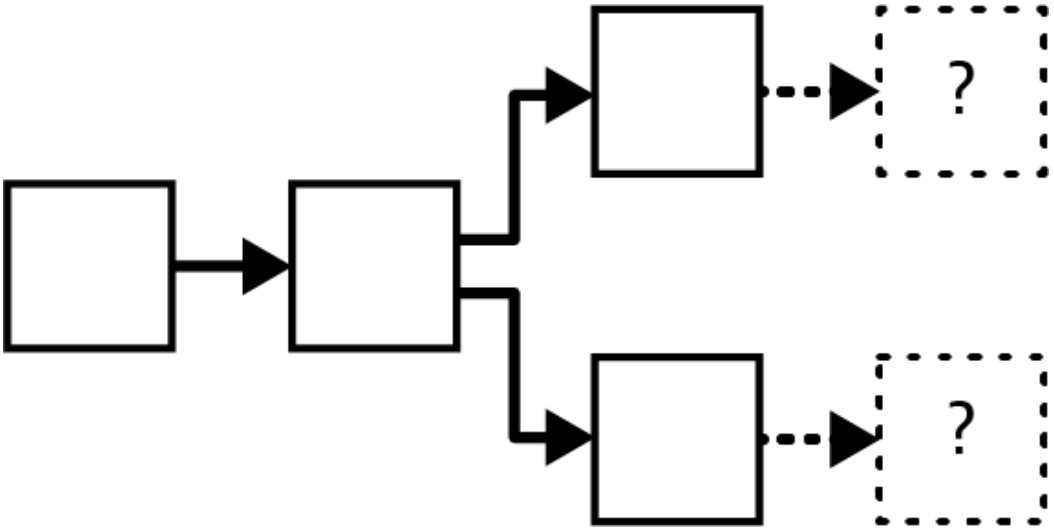


Figure 47. A blockchain fork requiring a mining race.

In 2015, a new version of Bitcoin Core added a feature called *compact block relay* (specified in BIP152) that allows transferring new blocks both faster and with less bandwidth.

As background, full nodes that relay unconfirmed transactions also store many of those transactions in their mempools (see Mempools and Orphan Pools). When some of those transactions are confirmed in a new block, the node doesn't need to receive a second copy of those transactions.

Instead of receiving redundant unconfirmed transactions, compact blocks allow a peer to instead send a short 6-byte identifier for each transaction. When your node receives a compact block with one or more identifiers, it checks its mempool for those transactions and uses them if they are found. For any transaction that isn't found in your local node's mempool, your node can send a request to the peer for a copy.

Conversely, if the remote peer believes your node's mempool doesn't have some of the transactions that appear in the block, it can include a copy of those transactions in the compact block. For example, Bitcoin Core always sends a block's coinbase transaction.

If the remote peer guesses correctly about what transactions your node has in its mempool, and which it does not, it will send a block nearly as efficiently as is theoretically possible (for a typical block, it'll be between 97% and 99% efficient).

> **Tip** Compact block relay does not decrease the size of blocks. It just prevents the redundant transfer of information that a node already has. When a node doesn't previously have information about a block, for example when a node is first started, it must receive complete copies of each block.

There are two modes that Bitcoin Core currently implements for sending compact blocks, illustrated in BIP152 modes compared (from BIP152). The shaded bar indicates the time it takes the node to validate the block.:

Low-bandwidth mode
: When your node requests that a peer use low-bandwidth mode (the default), that peer will tell your node the 32-byte identifier (header hash) of a new block but will not send your node any details about it. If your node acquires that block first from another source, this avoids wasting any more of your bandwidth acquiring a redundant copy of that block. If your node does need the block, it will request a compact block.

High-bandwidth mode
: When your node requests that a peer use high-bandwidth mode, that peer will send your node a compact block for a new block even before it has fully verified that the block is valid. The only validation the peer will perform is ensuring that the block's header contains the correct amount of proof of work. Since proof of work is expensive to generate (about $150,000 USD at the time of writing), it's unlikely that a miner would fake it just to waste the bandwidth of relay nodes. Skipping validation before relay allows new blocks to travel across the network with minimal delays at each hop.

  The downside of high-bandwidth mode is that your node is likely to receive redundant information from each high-bandwidth peer it chooses. As of this writing, Bitcoin Core currently only asks three peers to use high-bandwidth mode (and it tries to choose peers that have a history of quickly announcing blocks).

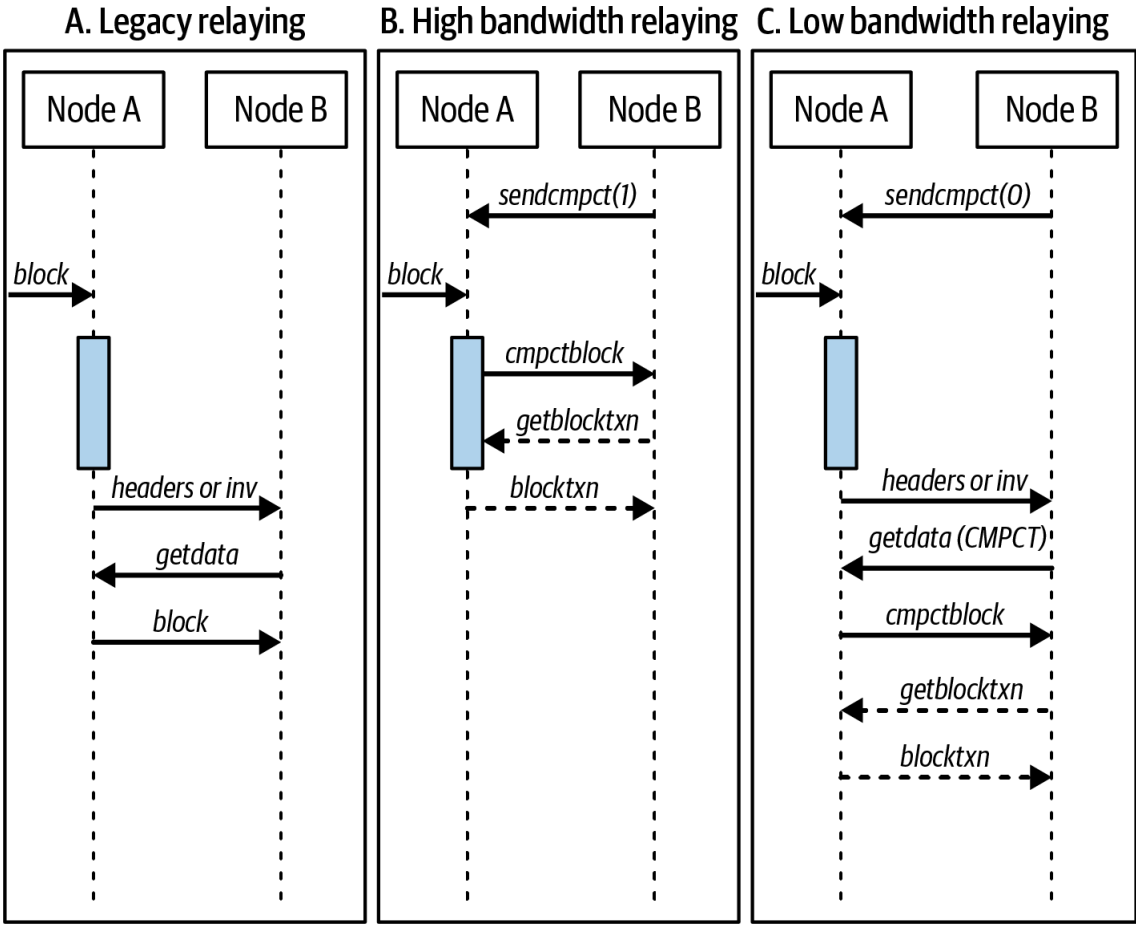


Figure 48. BIP152 modes compared (from BIP152). The shaded bar indicates the time it takes the node to validate the block.

The names of the two methods (which are taken from BIP152) can be a bit confusing. Low-bandwidth mode saves bandwidth by not sending blocks in most cases. High-bandwidth mode uses more bandwidth than low-bandwidth mode but, in most cases, much less bandwidth than was used for block relay before compact blocks were implemented.

10.4. Private Block Relay Networks

Although compact blocks go a long way toward minimizing the time it takes for blocks to propagate across the network, it’s possible to minimize latency further. Unlike compact blocks, though, the other solutions involve trade-offs that make them unavailable or unsuitable for the public P2P relay network. For that reason, there has been experimentation with private relay networks for blocks.

One simple technique is to preselect a route between endpoints. For example, a relay network with servers running in datacenters near major trans-oceanic fiber optic lines might be able to forward new blocks faster than waiting for the block to arrive at the node run by some home user many kilometers away from the fiber optic line.

Another, more complex technique, is Forward Error Correction (FEC). This allows a compact block message to be split into several parts, with each part having extra data appended. If any of the parts isn't received, that part can be reconstructed from the parts that are received. Depending on the settings, up to several parts may be reconstructed if they are lost.

FEC avoids the problem of a compact block (or some parts of it) not arriving due to problems with the underlying network connection. Those problems frequently occur but we don't often notice them because we mostly use protocols that automatically re-request the missing data. However, requesting missing data triples the time to receive it. For example:

1. Alice sends some data to Bob.
2. Bob doesn't receive the data (or it is damaged). Bob re-requests the data from Alice.
3. Alice sends the data again.

A third technique is to assume all nodes receiving the data have almost all of the same transactions in their mempool, so they can all accept the same compact block. That not only saves us time computing a compact block at each hop, but it means that each hop can simply relay the FEC packets to the next hop even before validating them.

The trade-off for each of the preceding methods is that they work well with centralization but not in a decentralized network where individual nodes can't trust other nodes. Servers in datacenters cost money and can often be accessed by operators of the datacenter, making them less trustworthy than a secure home computer. Relaying data before validating makes it easy to waste bandwidth, so it can only reasonably be used on a private network where there's some level of trust and accountability between parties.

The original Bitcoin Relay Network was created by developer Matt Corallo in 2015 to enable fast synchronization of blocks between miners with very low latency. The network consisted of several virtual private servers (VPSes) hosted on infrastructure around the world and served to connect the majority of miners and mining pools.

The original Bitcoin Relay Network was replaced in 2016 with the introduction of the *Fast Internet Bitcoin Relay Engine* or *FIBRE*, also created by developer Matt Corallo. FIBRE is software that allows operating a UDP-based relay network that relays blocks within a network of nodes. FIBRE implements FEC and the *compact block* optimization to further reduce the amount of data transmitted and the network latency.

10.5. Network Discovery

When a new node boots up, it must discover other Bitcoin nodes on the network in order to participate. To start this process, a new node must discover at least one existing node on the network and connect to it. The geographic location of other nodes is irrelevant; the Bitcoin network topology is not geographically defined. Therefore, any existing Bitcoin nodes can be selected at random.

To connect to a known peer, nodes establish a TCP connection, usually to port 8333 (the port generally known as the one used by Bitcoin), or an alternative port if one is provided. Upon establishing a connection, the node will start a "handshake" (see The initial handshake between peers.) by transmitting a version message, which contains basic identifying information, including:

Version
: The Bitcoin P2P protocol version the client "speaks" (e.g., 70002)

nLocalServices
: A list of local services supported by the node

nTime
: The current time

addrYou
: The IP address of the remote node, as seen from this node

addrMe
: The IP address of the local node, as discovered by the local node

subver
: A subversion showing the type of software running on this node (e.g., /Satoshi:0.9.2.1/)

BestHeight
: The block height of this node’s blockchain

fRelay
: A field added by BIP37 for requesting not to receive unconfirmed transactions

The version message is always the first message sent by any peer to another peer. The local peer receiving a version message will examine the remote peer’s reported Version and decide if the remote peer is compatible. If the remote peer is compatible, the local peer will acknowledge the version message and establish a connection by sending a verack.

How does a new node find peers? The first method is to query DNS using a number of *DNS seeds*, which are DNS servers that provide a list of IP addresses of Bitcoin nodes. Some of those DNS seeds provide a static list of IP addresses of stable Bitcoin listening nodes. Some of the DNS seeds are custom implementations of BIND (Berkeley Internet Name

Daemon) that return a random subset from a list of Bitcoin node addresses collected by a crawler or a long-running Bitcoin node. The Bitcoin Core client contains the names of several different DNS seeds. The diversity of ownership and diversity of implementation of the different DNS seeds offers a high level of reliability for the initial bootstrapping process. In the Bitcoin Core client, the option to use the DNS seeds is controlled by the option switch -dnsseed (set to 1 by default, to use the DNS seed).

Alternatively, a bootstrapping node that knows nothing of the network must be given the IP address of at least one Bitcoin node, after which it can establish connections through further introductions. The command-line argument -seednode can be used to connect to one node just for introductions using it as a seed. After the initial seed node is used to form introductions, the client will disconnect from it and use the newly discovered peers.

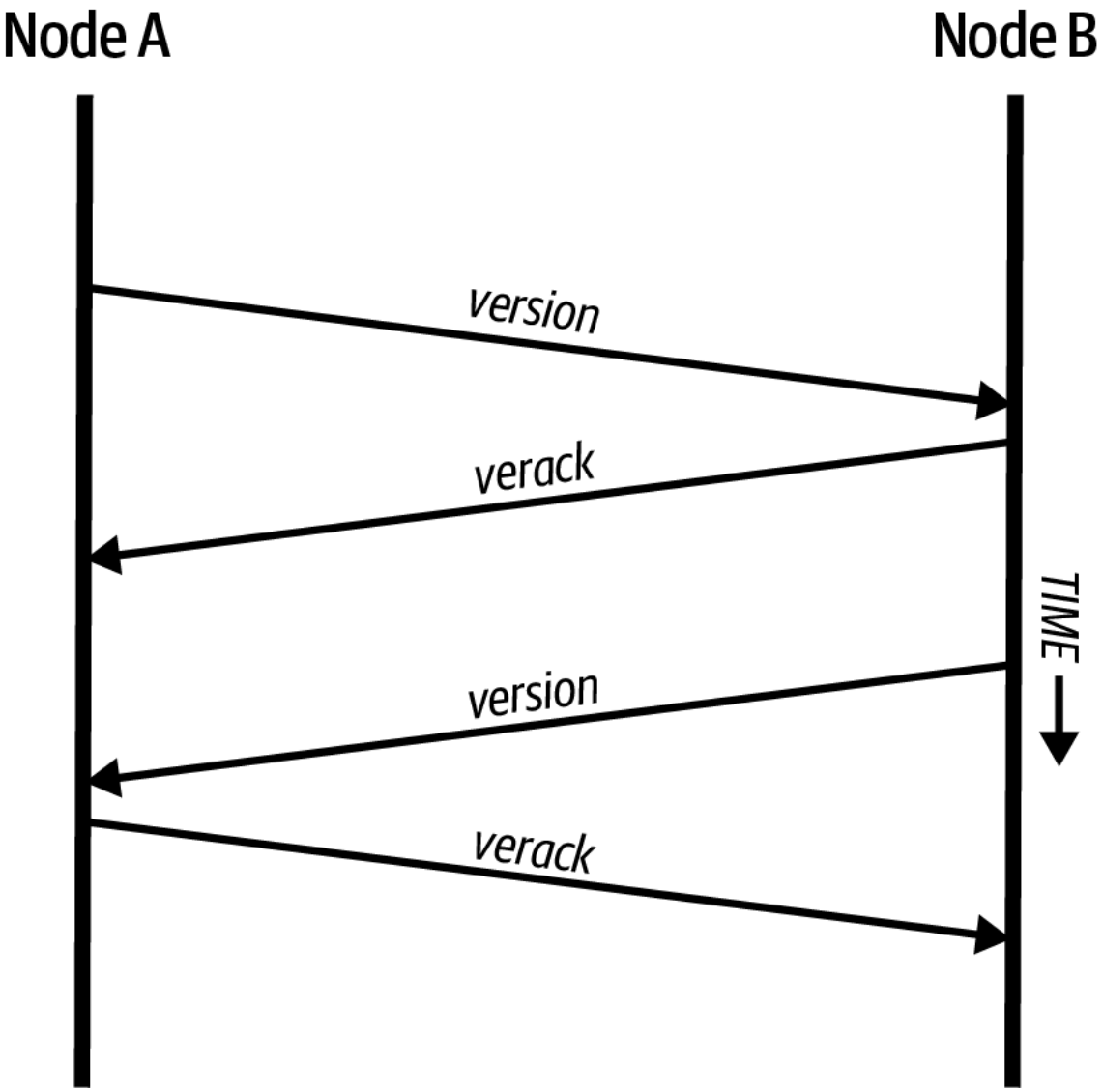


Figure 49. The initial handshake between peers.

Once one or more connections are established, the new node will send an addr message containing its own IP address to its neighbors. The neighbors will, in turn, forward the addr message to their neighbors, ensuring that the newly connected node becomes well known and better connected. Additionally, the newly connected node can send getaddr to its neighbors, asking them to return a list of IP addresses of other peers. That way, a node can find peers to connect to and advertise its existence on the network for other nodes to find it. Address propagation and discovery. shows the address discovery protocol.

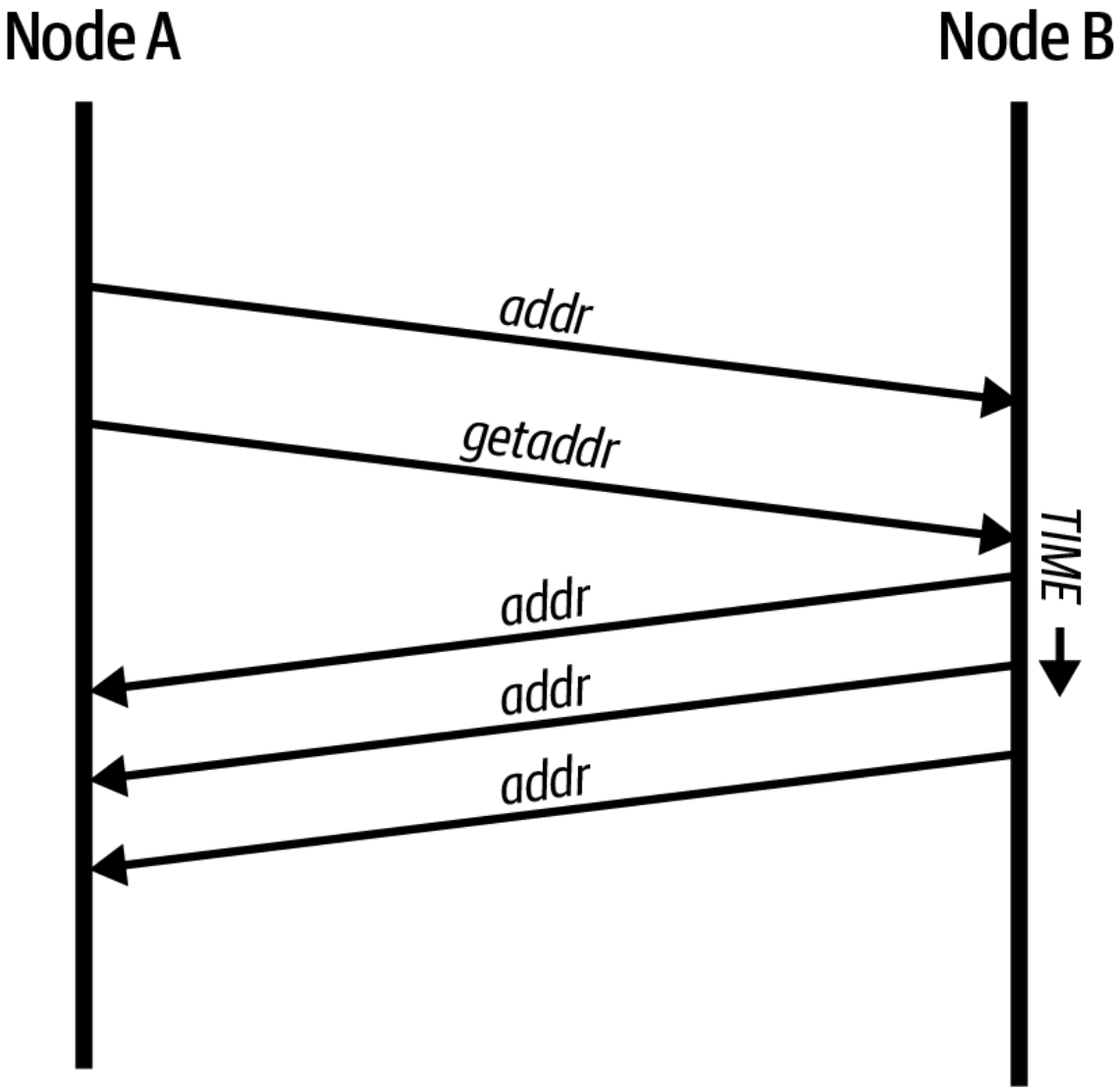


Figure 50. Address propagation and discovery.

A node must connect to a few different peers in order to establish diverse paths into the Bitcoin network. Paths are not reliable—nodes come and go—and so the node must continue to discover new nodes as it loses old connections as well as assist other nodes when they bootstrap. Only one connection is needed to bootstrap because the first node can offer introductions to its peer nodes and those peers can offer further introductions. It's also unnecessary and wasteful of network resources to connect to more than a handful of nodes. After bootstrapping, a node will remember its most recent successful peer connections so if it is rebooted, it can quickly reestablish connections with its former peer network. If none of the former peers respond to its connection request, the node can use the seed nodes to bootstrap again.

On a node running the Bitcoin Core client, you can list the peer connections with the command getpeerinfo:

```
$ bitcoin-cli getpeerinfo
```


```
[
  {
    "id": 0,
    "addr": "82.64.116.5:8333",
    "addrbind": "192.168.0.133:50564",
    "addrlocal": "72.253.6.11:50564",
    "network": "ipv4",
    "services": "0000000000000409",
    "servicesnames": [
      "NETWORK",
      "WITNESS",
      "NETWORK_LIMITED"
    ],
    "lastsend": 1683829947,
    "lastrecv": 1683829989,
    "last_transaction": 0,
    "last_block": 1683829989,
    "bytessent": 3558504,
    "bytesrecv": 6016081,
    "conntime": 1683647841,
    "timeoffset": 0,
    "pingtime": 0.204744,
    "minping": 0.20337,
    "version": 70016,
    "subver": "/Satoshi:24.0.1/",
    "inbound": false,
    "bip152_hb_to": true,
    "bip152_hb_from": false,
    "startingheight": 788954,
    "presynced_headers": -1,
    "synced_headers": 789281,
    "synced_blocks": 789281,
    "inflight": [
    ],
    "relaytxes": false,
    "minfeefilter": 0.00000000,
    "addr_relay_enabled": false,
    "addr_processed": 0,
    "addr_rate_limited": 0,
    "permissions": [
    ],
    "bytessent_per_msg": {
      ...
    },
    "bytesrecv_per_msg": {
      ...
    },
    "connection_type": "block-relay-only"
  },
]
```

To override the automatic management of peers and to specify a list of IP addresses, users can provide the option -connect=<IPAddress> and specify one or more IP addresses. If this option is used, the node will only connect to the selected IP addresses instead of discovering and maintaining the peer connections automatically.

If there is no traffic on a connection, nodes will periodically send a message to maintain the connection. If a node has not communicated on a connection for too long, it is assumed to be disconnected and a new peer will be sought. Thus, the network dynamically adjusts to transient nodes and network problems and can organically grow and shrink as needed without any central control.

10.6. Full Nodes

Full nodes are nodes that verify every transaction in every block on the valid blockchain with the most proof of work.

Full nodes independently process every block, starting after the very first block (genesis block) and building up to the latest known block in the network. A full node can independently and authoritatively verify any transaction. The full node relies on the network to receive updates about new blocks of transactions, which it then verifies and incorporates into its local view of which scripts control which bitcoins, called the set of *unspent transaction outputs* (UTXOs).

Running a full node gives you the pure Bitcoin experience: independent verification of all transactions without the need to rely on, or trust, any other systems.

There are a few alternative implementations of full nodes, built using different programming languages and software architectures, or which made different design decisions. However, the most common implementation is Bitcoin Core. More than 95% of full nodes on the Bitcoin network run various versions of Bitcoin Core. It is identified as "Satoshi" in the subversion string sent in the version message and shown by the command getpeerinfo as we saw earlier; for example, /Satoshi:24.0.1/.

10.7. Exchanging "Inventory"

The first thing a full node will do once it connects to peers is try to construct a complete chain of block headers. If it is a brand-new node and has no blockchain at all, it only knows one block, the genesis block, which is statically embedded in the client software. Starting after block #0 (the genesis block), the new node will have to download hundreds of thousands of blocks to synchronize with the network and reestablish the full blockchain.

The process of syncing the blockchain starts with the version message because that contains BestHeight, a node’s current blockchain height (number of blocks). A node will see the version messages from its peers, know how many blocks they each have, and be able to compare to how many blocks it has in its own blockchain. Peered nodes will exchange a getheaders message that contains the hash of the top block on their local

blockchain. One of the peers will be able to identify the received hash as belonging to a block that is not at the top, but rather belongs to an older block, thus deducing that its own local blockchain is longer than the remote node's blockchain.

The peer that has the longer blockchain has more blocks than the other node and can identify which headers the other node needs in order to "catch up." It will identify the first 2,000 headers to share using a headers message. The node will keep requesting additional headers until it has received one for every block the remote peer claims to have.

In parallel, the node will begin requesting the blocks for each header it previously received using a getdata message. The node will request different blocks from each of its selected peers, which allows it to drop connections to peers that are significantly slower than the average in order to find newer (and possibly faster) peers.

Let's assume, for example, that a node only has the genesis block. It will then receive a headers message from its peers containing the headers of the next 2,000 blocks in the chain. It will start requesting blocks from all of its connected peers, keeping a queue of up to 1,024 blocks. Blocks need to be validated in order, so if the oldest block in the queue—the block the node next needs to validate—hasn't been received yet, the node drops the connection to the peer that was supposed to provide that block. It then finds a new peer that may be able to provide one block before all of the node's other peers are able to provide 1,023 blocks.

As each block is received, it is added to the blockchain, as we will see in The Blockchain. As the local blockchain is gradually built up, more blocks are requested and received, and the process continues until the node catches up to the rest of the network.

This process of comparing the local blockchain with the peers and retrieving any missing blocks happens any time a node has been offline for an extended period of time.

10.8. Lightweight Clients

Many Bitcoin clients are designed to run on space- and power-constrained devices, such as smartphones, tablets, or embedded systems. For such devices, a *simplified payment verification* (SPV) method is used to allow them to operate without validating the full blockchain. These types of clients are called lightweight clients.

Lightweight clients download only the block headers and do not download the transactions included in each block. The resulting chain of headers, without transactions, is about 10,000 times smaller than the full blockchain. Lightweight clients cannot construct a full picture of all the UTXOs that are available for spending because they do not know about all the transactions on the network. Instead, they verify transactions using a slightly different method that relies on peers to provide partial views of relevant parts of the blockchain on demand.

As an analogy, a full node is like a tourist in a strange city, equipped with a detailed map of every street and every address. By comparison, a lightweight client is like a tourist in a strange city asking random strangers for turn-by-turn directions while knowing only one main avenue. Although both tourists can verify the existence of a street by visiting it, the tourist without a map doesn't know what lies down any of the side streets and doesn't know what other streets exist. Positioned in front of 23 Church Street, the tourist without a map cannot know if there are a dozen other "23 Church Street" addresses in the city and whether this is the right one. The mapless tourist's best chance is to ask enough people and hope some of them are not trying to mug him.

Lightweight clients verify transactions by reference to their *depth* in the blockchain. Whereas a full node will construct a fully verified chain of thousands of blocks and millions of transactions reaching down the blockchain (back in time) all the way to the genesis block, a lightweight client will verify the proof of work of all blocks (but not whether the blocks and all of their transactions are valid) and link that chain to the transaction of interest.

For example, when examining a transaction in block 800,000, a full node verifies all 800,000 blocks down to the genesis block and builds a full database of UTXOs, establishing the validity of the transaction by confirming that the transaction exists and its output remains unspent. A lightweight client can only verify that the transaction exists. The client establishes a link between the transaction and the block that contains it, using a *merkle path* (see Merkle Trees). Then, the lightweight client waits until it sees the six blocks 800,001 through 800,006 piled on top of the block containing the transaction and verifies it by establishing its depth under blocks 800,006 to 800,001. The fact that other nodes on the network accepted block 800,000 and that miners did the necessary work to produce six more blocks on top of it is proof, by proxy, that the transaction actually exists.

A lightweight client cannot normally be persuaded that a transaction exists in a block when the transaction does not in fact exist. The lightweight client establishes the existence of a transaction in a block by requesting a merkle path proof and by validating the proof of work in the chain of blocks. However, a transaction's existence can be "hidden" from a lightweight client. A lightweight client can definitely verify that a transaction exists but cannot verify that a transaction, such as a double-spend of the same UTXO, doesn't exist because it doesn't have a record of all transactions. This vulnerability can be used in a denial-of-service attack or for a double-spending attack against lightweight clients. To defend against this, a lightweight client needs to connect randomly to several clients to increase the probability that it is in contact with at least one honest node. This need to randomly connect means that lightweight clients also are vulnerable to network partitioning attacks or Sybil attacks, where they are connected to fake nodes or fake networks and do not have access to honest nodes or the real Bitcoin network.

For many practical purposes, well-connected lightweight clients are secure enough, striking a balance between resource needs, practicality, and security. For infallible security, however, nothing beats running a full node.

> **Tip** A full node verifies a transaction by checking the entire chain of thousands of blocks below it in order to guarantee that the UTXO exists and is not spent, whereas a lightweight client only proves that a transaction exists and checks that the block containing that transaction is buried by a handful of blocks above it.

To get the block headers it needs to verify a transaction is part of the chain, lightweight clients use a getheaders message. The responding peer will send up to 2,000 block headers using a single headers message. See the illustration in Lightweight client synchronizing the block headers..

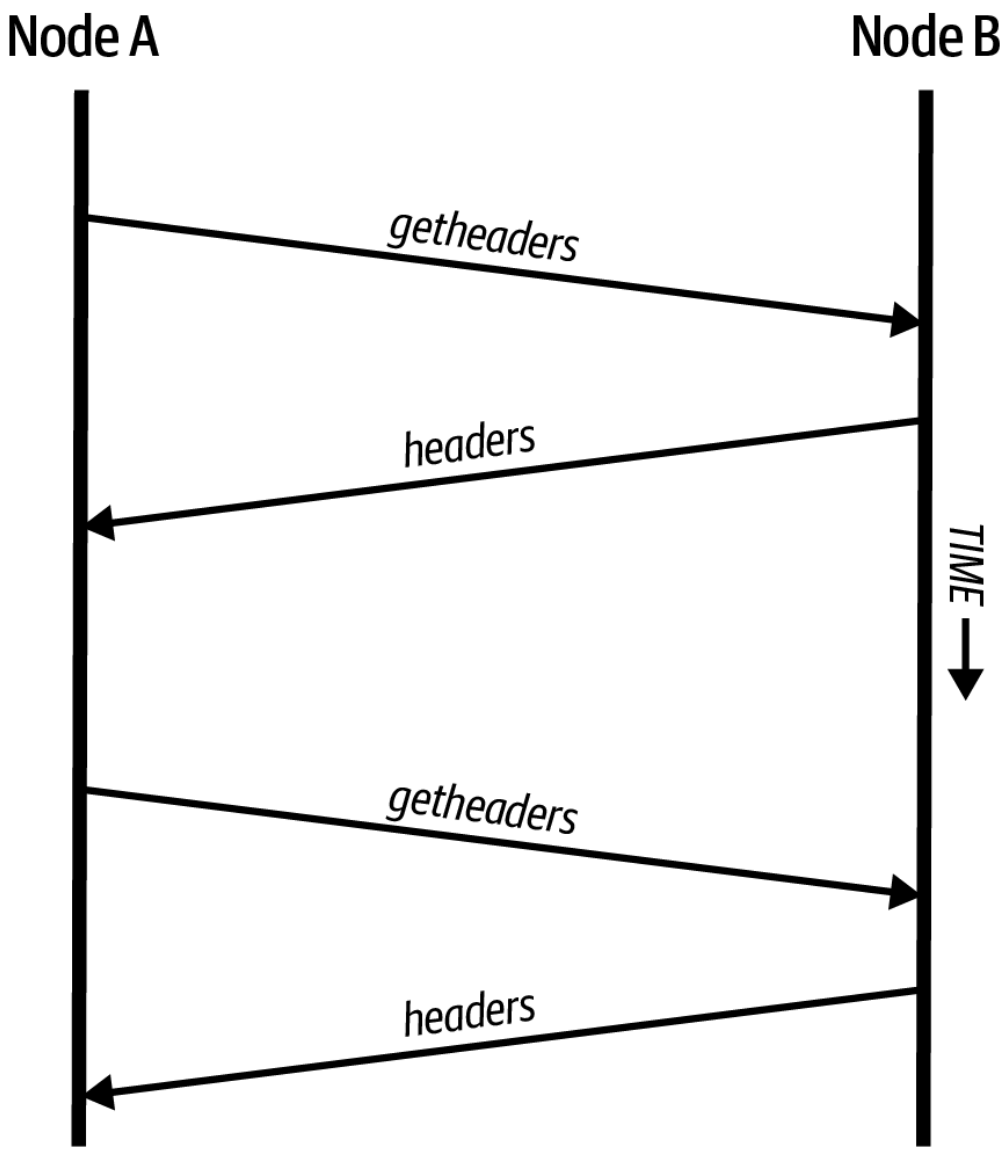


Figure 51. Lightweight client synchronizing the block headers.

Block headers allow a lightweight client to verify that any individual block belongs to the blockchain with the most proof of work, but they don’t tell the client which blocks contain transactions that are interesting to its wallet. The client could download every block and check, but that would use a large fraction of the resources it would take to run a full node, so developers have looked for other ways to solve the problem.

Shortly after the introduction of lightweight clients, Bitcoin developers added a feature called *bloom filters* in an attempt to reduce the bandwidth that lightweight clients needed to use to learn about their incoming and outgoing transactions. Bloom filters allow

lightweight clients to receive a subset of the transactions without directly revealing precisely which addresses they are interested in, through a filtering mechanism that uses probabilities rather than fixed patterns.

10.9. Bloom Filters

A bloom filter is a probabilistic search filter, a way to describe a desired pattern without specifying it exactly. Bloom filters offer an efficient way to express a search pattern while protecting privacy. They are used by lightweight clients to ask their peers for transactions matching a specific pattern without revealing exactly which addresses, keys, or transactions they are searching for.

In our previous analogy, a tourist without a map is asking for directions to a specific address, "23 Church St." If they ask a stranger for directions to this street, they inadvertently reveal their destination. A bloom filter is like asking, "Are there any streets in this neighborhood whose name ends in R-C-H?" A question like that reveals slightly less about the desired destination than asking for "23 Church St." Using this technique, a tourist could specify the desired address in more detail such as "ending in U-R-C-H" or less detail such as "ending in H." By varying the precision of the search, the tourist reveals more or less information at the expense of getting more or less specific results. If they ask for a less specific pattern, they get a lot more possible addresses and better privacy, but many of the results are irrelevant. If they ask for a very specific pattern, they get fewer results but lose privacy.

Bloom filters serve this function by allowing a lightweight client to specify a search pattern for transactions that can be tuned toward precision or privacy. A more specific bloom filter will produce accurate results, but at the expense of revealing what patterns the lightweight client is interested in, thus revealing the addresses owned by the user’s wallet. A less specific bloom filter will produce more data about more transactions, many irrelevant to the client, but will allow the client to maintain better privacy.

10.9.1. How Bloom Filters Work

Bloom filters are implemented as a variable-size array of N binary digits (a bit field) and a variable number of M hash functions. The hash functions are designed to always produce an output that is between 1 and N, corresponding to the array of binary digits. The hash functions are generated deterministically, so that any client implementing a bloom filter will always use the same hash functions and get the same results for a specific input. By choosing different length (N) bloom filters and a different number (M) of hash functions, the bloom filter can be tuned, varying the level of accuracy and therefore privacy.

In An example of a simplistic bloom filter, with a 16-bit field and three hash functions., we use a very small array of 16 bits and a set of three hash functions to demonstrate how bloom filters work.

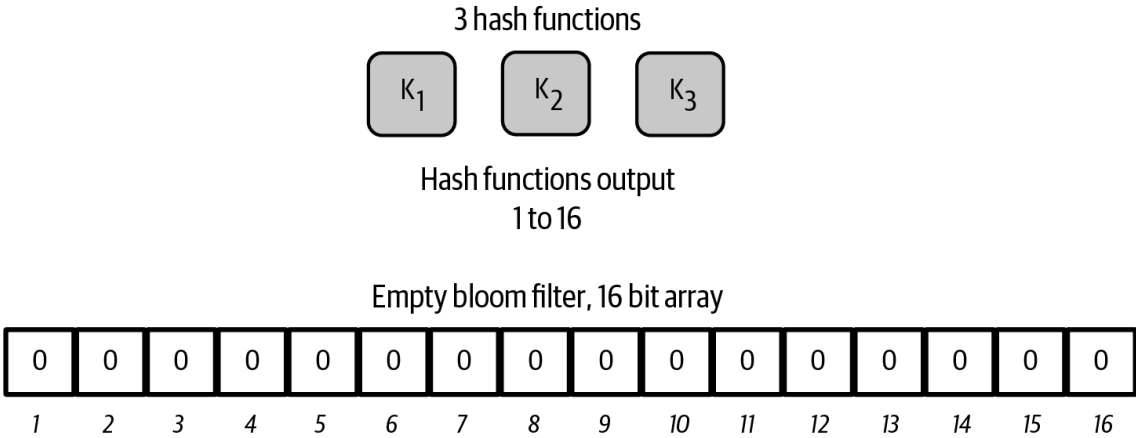


Figure 52. An example of a simplistic bloom filter, with a 16-bit field and three hash functions.

The bloom filter is initialized so that the array of bits is all zeros. To add a pattern to the bloom filter, the pattern is hashed by each hash function in turn. Applying the first hash function to the input results in a number between 1 and N. The corresponding bit in the array (indexed from 1 to N) is found and set to 1, thereby recording the output of the hash function. Then, the next hash function is used to set another bit and so on. Once all M hash functions have been applied, the search pattern will be "recorded" in the bloom filter as M bits that have been changed from 0 to 1.

Adding a pattern "A" to our simple bloom filter. is an example of adding a pattern "A" to the simple bloom filter shown in An example of a simplistic bloom filter, with a 16-bit field and three hash functions..

Adding a second pattern is as simple as repeating this process. The pattern is hashed by each hash function in turn, and the result is recorded by setting the bits to 1. Note that as a bloom filter is filled with more patterns, a hash function result might coincide with a bit that is already set to 1, in which case the bit is not changed. In essence, as more patterns record on overlapping bits, the bloom filter starts to become saturated with more bits set to 1 and the accuracy of the filter decreases. This is why the filter is a probabilistic data structure—it gets less accurate as more patterns are added. The accuracy depends on the number of patterns added versus the size of the bit array (N) and number of hash functions (M). A larger bit array and more hash functions can record more patterns with higher accuracy. A smaller bit array or fewer hash functions will record fewer patterns and produce less accuracy.

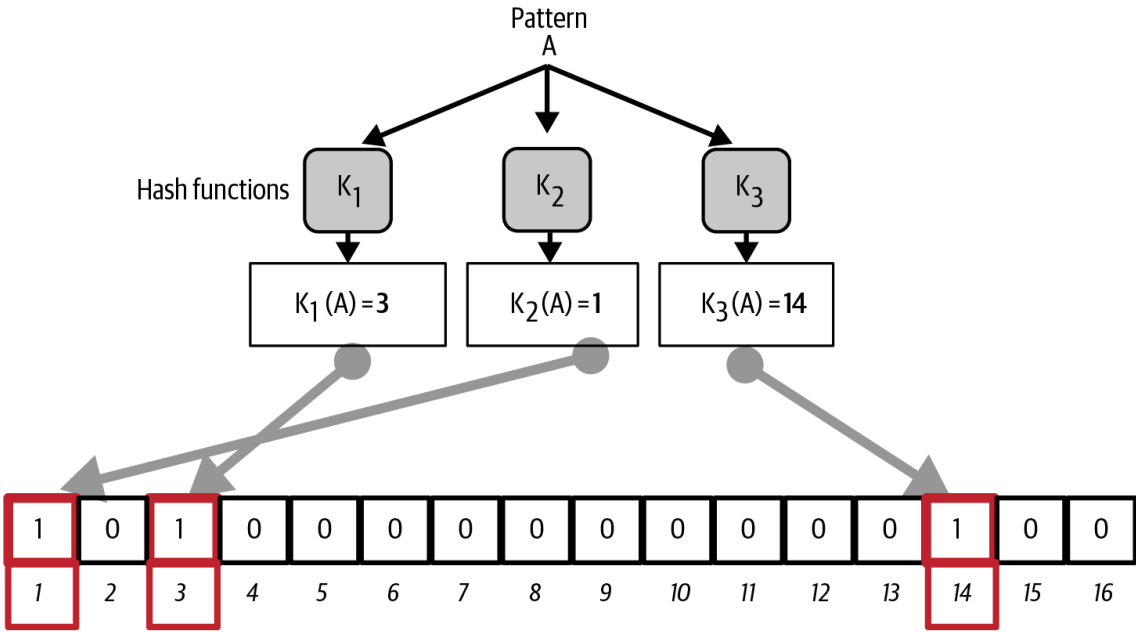


Figure 53. Adding a pattern "A" to our simple bloom filter.

Adding a second pattern "B" to our simple bloom filter. is an example of adding a second pattern "B" to the simple bloom filter.

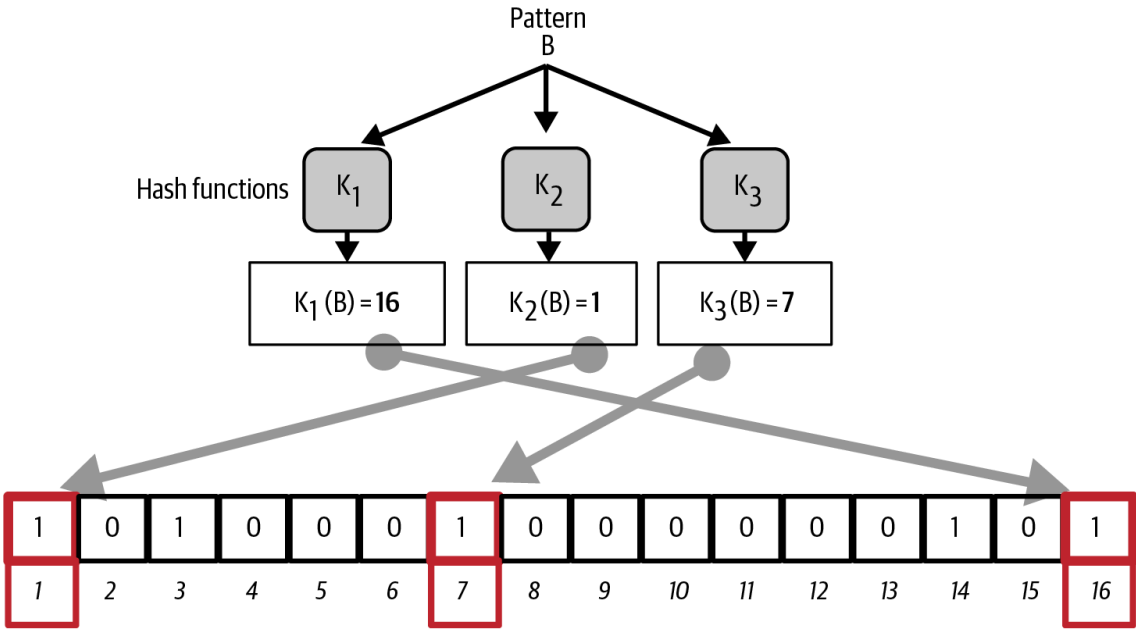


Figure 54. Adding a second pattern "B" to our simple bloom filter.

To test if a pattern is part of a bloom filter, the pattern is hashed by each hash function and the resulting bit pattern is tested against the bit array. If all the bits indexed by the hash functions are set to 1, then the pattern is *probably* recorded in the bloom filter. Because the bits may be set because of overlap from multiple patterns, the answer is not certain, but is rather probabilistic. In simple terms, a bloom filter positive match is a "Maybe, yes."

Testing the existence of pattern "X" in the bloom filter. The result is a probabilistic positive match, meaning "Maybe." is an example of testing the existence of pattern "X" in the simple bloom filter. The corresponding bits are set to 1, so the pattern is probably a match.

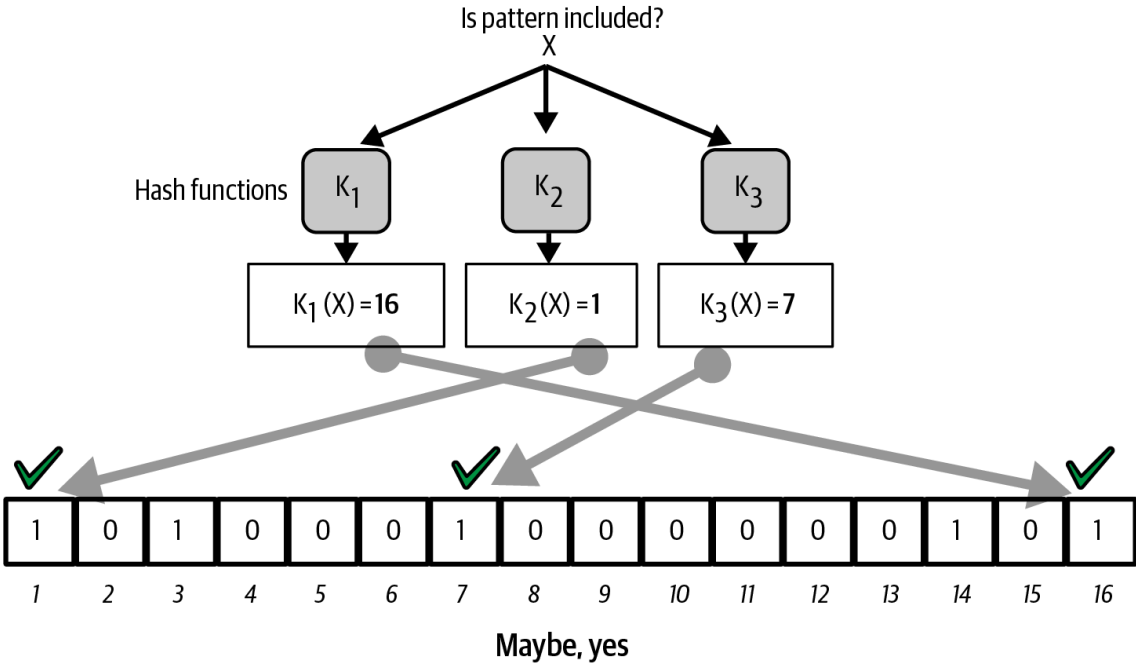


Figure 55. Testing the existence of pattern "X" in the bloom filter. The result is a probabilistic positive match, meaning "Maybe."

On the contrary, if a pattern is tested against the bloom filter and any one of the bits is set to 0, this proves that the pattern was not recorded in the bloom filter. A negative result is not a probability, it is a certainty. In simple terms, a negative match on a bloom filter is a "Definitely not!"

Testing the existence of pattern "Y" in the bloom filter. The result is a definitive negative match, meaning "Definitely Not!" is an example of testing the existence of pattern "Y" in the simple bloom filter. One of the corresponding bits is set to 0, so the pattern is definitely not a match.

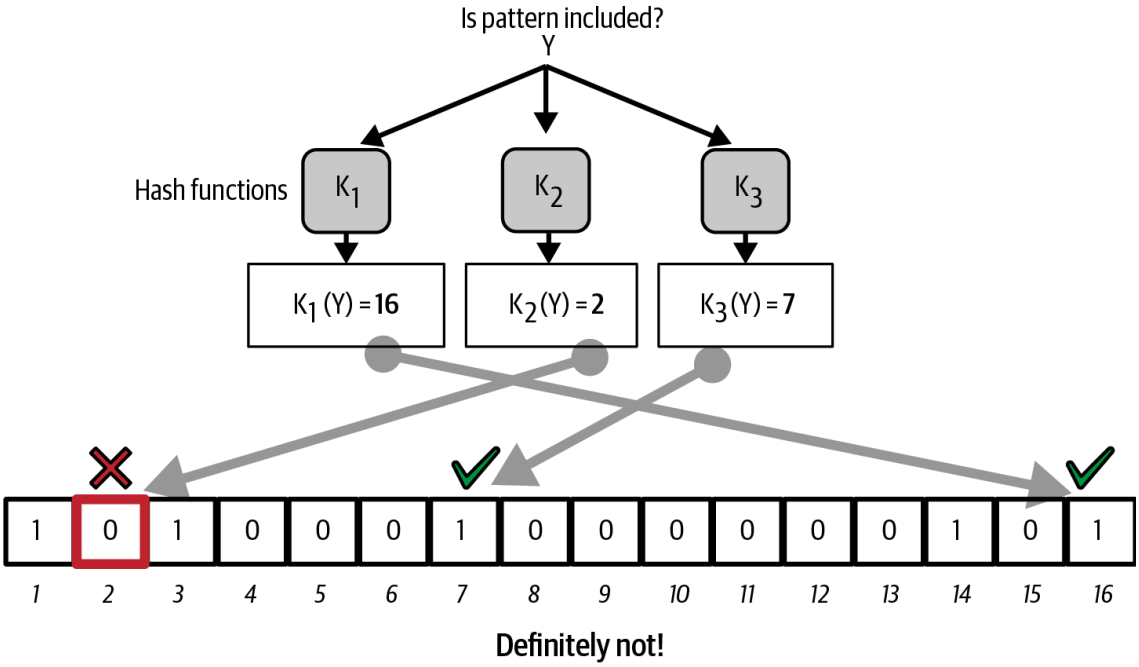


Figure 56. Testing the existence of pattern "Y" in the bloom filter. The result is a definitive negative match, meaning "Definitely Not!"

10.9.2. How Lightweight Clients Use Bloom Filters

Bloom filters are used to filter the transactions (and blocks containing them) that a lightweight client receives from its peers, selecting only transactions of interest to the lightweight client without revealing exactly which addresses or keys it is interested in.

A lightweight client will initialize a bloom filter as "empty"; in that state, the bloom filter will not match any patterns. The lightweight client will then make a list of all the addresses, keys, and hashes that it is interested in. It will do this by extracting the public key hash, script hash, and transaction IDs from any UTXO controlled by its wallet. The lightweight client then adds each of these to the bloom filter so that the bloom filter will "match" if these patterns are present in a transaction, without revealing the patterns themselves.

The lightweight client will then send a filterload message to the peer containing the bloom filter to use on the connection. On the peer, bloom filters are checked against each incoming transaction. The full node checks several parts of the transaction against the bloom filter, looking for a match including:

- The transaction ID
- The data components from the scripts of each of the transaction outputs (every key and hash in the script)
- Each of the transaction inputs

- Each of the input signature data components (or witness scripts)

By checking against all these components, bloom filters can be used to match public key hashes, scripts, OP_RETURN values, public keys in signatures, or any future component of a smart contract or complex script.

After a filter is established, the peer will then test each transaction's outputs against the bloom filter. Only transactions that match the filter are sent to the client.

In response to a getdata message from the client, peers will send a merkleblock message that contains only block headers for blocks matching the filter and a merkle path (see Merkle Trees) for each matching transaction. The peer will then also send tx messages containing the transactions matched by the filter.

As the full node sends transactions to the lightweight client, the lightweight client discards any false positives and uses the correctly matched transactions to update its UTXO set and wallet balance. As it updates its own view of the UTXO set, it also modifies the bloom filter to match any future transactions referencing the UTXO it just found. The full node then uses the new bloom filter to match new transactions and the whole process repeats.

The client setting the bloom filter can interactively add patterns to the filter by sending a filteradd message. To clear the bloom filter, the client can send a filterclear message. Because it is not possible to remove a pattern from a bloom filter, a client has to clear and resend a new bloom filter if a pattern is no longer desired.

The network protocol and bloom filter mechanism for lightweight clients is defined in BIP37.

Unfortunately, after the deployment of bloom filters, it became clear that they didn't offer very much privacy. A full node receiving a bloom filter from a peer could apply that filter to the entire blockchain to find all of the client's transactions (plus false positives). It could then look for patterns and relationships between the transactions. Randomly selected false positive transactions would be unlikely to have a parent-child relationship from output to input, but transactions from the user's wallet would be very likely to have that relationship. If all of the related transactions have certain characteristics, such as at least one P2PKH output, then transactions without that characteristic can be assumed not to belong to the wallet.

It was also discovered that specially constructed filters could force the full nodes that processed them to perform a large amount of work, which could lead to denial-of-service attacks.

For both of those reasons, Bitcoin Core eventually limited support for bloom filters to only clients on IP addresses that were explicitly allowed by the node operator. This meant that an alternative method for helping lightweight clients find their transactions was needed.

10.10. Compact Block Filters

An idea was posted to the Bitcoin-Dev mailing list by an anonymous developer in 2016 of reversing the bloom filter process. With a BIP37 bloom filter, each client hashes their addresses to create a bloom filter and nodes hash parts of each transaction to attempt to match that filter. In the new proposal, nodes hash parts of each transaction in a block to create a bloom filter and clients hash their addresses to attempt to match that filter. If a client finds a match, they download the entire block.

> **Note** Despite the similarities in names, BIP152 *compact blocks* and BIP157/158 *compact block filters* are unrelated.

This allows nodes to create a single filter for every block, which they can save to disk and serve over and over, eliminating the denial-of-service vulnerabilities with BIP37. Clients don't give full nodes any information about their past or future addresses. They only download blocks, which may contain thousands of transactions that weren't created by the client. They can even download each matching block from a different peer, making it harder for full nodes to connect transactions belonging to a single client across multiple blocks.

This idea for server-generated filters doesn't offer perfect privacy; it still places some costs on full nodes (and it does require lightweight clients to use more bandwidth for the block download), and the filters can only be used for confirmed transactions (not unconfirmed transactions). However, it is much more private and reliable than BIP37 client-requested bloom filters.

After the description of the original idea based on bloom filters, developers realized there was a better data structure for server-generated filters, called Golomb-Rice Coded Sets (GCS).

10.10.1. Golomb-Rice Coded Sets (GCS)

Imagine that Alice wants to send a list of numbers to Bob. The simple way to do that is to just send him the entire list of numbers:

```
849
653
476
900
379
```

But there's a more efficient way. First, Alice puts the list in numerical order:

```
379
476
653
```

```
849
900
```

Then, Alice sends the first number. For the remaining numbers, she sends the difference between that number and the preceding number. For example, for the second number, she sends 97 (476 – 379); for the third number, she sends 177 (653 – 476); and so on:

```
379
97
177
196
51
```

We can see that the differences between two numbers in an ordered list produces numbers that are shorter than the original numbers. Upon receiving this list, Bob can reconstruct the original list by simply adding each number with its predecessor. That means we save space without losing any information, which is called *lossless encoding*.

If we randomly select numbers within a fixed range of values, then the more numbers we select, the smaller the average (mean) size of the differences. That means the amount of data we need to transfer doesn't increase as fast as the length of our list increases (up to a point).

Even more usefully, the length of the randomly selected numbers in a list of differences is naturally biased toward smaller lengths. Consider selecting two random numbers from 1 to 6; this is the same as rolling two dice. There are 36 distinct combinations of two dice:

1 1	1 2	1 3	1 4	1 5	1 6
2 1	2 2	2 3	2 4	2 5	2 6
3 1	3 2	3 3	3 4	3 5	3 6
4 1	4 2	4 3	4 4	4 5	4 6
5 1	5 2	5 3	5 4	5 5	5 6
6 1	6 2	6 3	6 4	6 5	6 6

Let's find the difference between the larger of the numbers and the smaller of the numbers:

0	1	2	3	4	5
1	0	1	2	3	4
2	1	0	1	2	3

3	2	1	0	1	2
4	3	2	1	0	1
5	4	3	2	1	0

If we count the frequency of each difference occurring, we see that the small differences are much more likely to occur than the large differences:

Difference	Occurrences
0	6
1	10
2	8
3	6
4	4
5	2

If we know that we might need to store large numbers (because large differences can happen, even if they are rare), but we'll most often need to store small numbers, we can encode each number using a system that uses less space for small numbers and extra space for large numbers. On average, that system will perform better than using the same amount of space for every number.

Golomb coding provides that facility. Rice coding is a subset of Golomb coding that's more convenient to use in some situations, including the application of Bitcoin block filters.

10.10.2. What Data to Include in a Block Filter

Our primary goal is to allow wallets to learn whether a block contains a transaction affecting that wallet. For a wallet to be effective, it needs to learn two types of information:

When it has received money
: Specifically, when a transaction output contains a script that the wallet controls (such as by controlling the authorized private key)

When it has spent money
: Specifically, when a transaction input references a previous transaction output that the wallet controlled

A secondary goal during the design of compact block filters was to allow the wallet receiving the filter to verify that it received an accurate filter from a peer. For example, if the wallet downloaded the block from which the filter was created, the wallet could generate its own filter. It could then compare its filter to the peer's filter and verify that they were identical, proving the peer had generated an accurate filter.

For both the primary and secondary goals to be met, a filter would need to reference two types of information:

- The script for every output in every transaction in a block
- The outpoint for every input in every transaction in a block

An early design for compact block filters included both of those pieces of information, but it was realized there was a more efficient way to accomplish the primary goal if we sacrificed the secondary goal. In the new design, a block filter would still reference two types of information, but they'd be more closely related:

- As before, the script for every output in every transaction in a block.
- In a change, it would also reference the script of the output referenced by the outpoint for every input in every transaction in a block. In other words, the output script being spent.

This had several advantages. First, it meant that wallets didn't need to track outpoints; they could instead just scan for the output scripts to which they expected to receive money. Second, any time a later transaction in a block spends the output of an earlier transaction in the same block, they'll both reference the same output script. More than one reference to the same output script is redundant in a compact block filter, so the redundant copies can be removed, shrinking the size of the filters.

When full nodes validate a block, they need access to the output scripts for both the current transaction outputs in a block and the transaction outputs from previous blocks that are being referenced in inputs, so they're able to build compact block filters in this simplified model. But a block itself doesn't include the output scripts from transactions included in previous blocks, so there's no convenient way for a client to verify a block filter was built correctly. However, there is an alternative that can help a client detect if a peer is lying to it: obtaining the same filter from multiple peers.

10.10.3. Downloading Block Filters from Multiple Peers

A peer can provide a wallet with an inaccurate filter. There are two ways to create an inaccurate filter. The peer can create a filter that references transactions that don't actually appear in the associated block (a false positive). Alternatively, the peer can create a filter that doesn't reference transactions that do actually appear in the associated block (a false negative).

The first protection against an inaccurate filter is for a client to obtain a filter from multiple peers. The BIP157 protocol allows a client to download just a short 32-byte commitment to a filter to determine whether each peer is advertising the same filter as all of the client's other peers. That minimizes the amount of bandwidth the client must expend to query many different peers for their filters, if all of those peers agree.

If two or more different peers have different filters for the same block, the client can download all of them. It can then also download the associated block. If the block contains any transaction related to the wallet that is not part of one of the filters, then the wallet can be sure that whichever peer created that filter was inaccurate—Golomb-Rice Coded Sets will always include a potential match.

Alternatively, if the block doesn't contain a transaction that the filter said might match the wallet, that isn't proof that the filter was inaccurate. To minimize the size of a GCS, we allow a certain number of false positives. What the wallet can do is continue downloading additional filters from the peer, either randomly or when they indicate a match, and then track the client's false positive rate. If it differs significantly from the false positive rate that filters were designed to use, the wallet can stop using that peer. In most cases, the only consequence of the inaccurate filter is that the wallet uses more bandwidth than expected.

10.10.4. Reducing Bandwidth with Lossy Encoding

The data about the transactions in a block that we want to communicate is an output script. Output scripts vary in length and follow patterns, which means the differences between them won't be evenly distributed like we want. However, we've already seen in many places in this book that we can use a hash function to create a commitment to some data and also produce a value that looks like a randomly selected number.

In other places in this book, we've used a cryptographically secure hash function that provides assurances about the strength of its commitment and how indistinguishable from random its output is. However, there are faster and more configurable non-cryptographic hash functions, such as the SipHash function we'll use for compact block filters.

The details of the algorithm used are described in BIP158, but the gist is that each output script is reduced to a 64-bit commitment using SipHash and some arithmetic operations. You can think of this as taking a set of large numbers and truncating them to shorter numbers, a process that loses data (so it's called *lossy encoding*). By losing some information, we don't need to store as much information later, which saves space. In this case, we go from a typical output script that's 160 bits or longer down to just 64 bits.

10.10.5. Using Compact Block Filters

The 64-bit values for every commitment to an output script in a block are sorted, duplicate entries are removed, and the GCS is constructed by finding the differences (deltas) between each entry. That compact block filter is then distributed by peers to their clients

(such as wallets).

A client uses the deltas to reconstruct the original commitments. The client, such as a wallet, also takes all the output scripts it is monitoring for and generates commitments in the same way as BIP158. It checks whether any of its generated commitments match the commitments in the filter.

Recall our example of the lossiness of compact block filters being similar to truncating a number. Imagine a client is looking for a block that contains the number 123456 and that an accurate (but lossy) compact block filter contains the number 1234. When a client sees that 1234, it will download the associated block.

There's a 100% guarantee that an accurate filter containing 1234 will allow a client to learn about a block containing 123456, called a *true positive*. However, there's also a chance that the block might contain 123400, 123401, or almost a hundred other entries that are not what the client is looking for (in this example), called a *false positive*.

A 100% true positive match rate is great. It means that a wallet can depend on compact block filters to find every transaction affecting that wallet. A nonzero false positive rate means that the wallet will end up downloading some blocks that don't contain transactions interesting to the wallet. The main consequence of this is that the client will use extra bandwidth, which is not a huge problem. The actual false positive rate for BIP158 compact block filters is very low, so it's not a major problem. A false positive rate can also help improve a client's privacy, as it does with bloom filters, although anyone wanting the best possible privacy should still use their own full node.

In the long term, some developers advocate for having blocks commit to the filter for that block, with the most likely scheme having each coinbase transaction commit to the filter for that block. Full nodes would calculate the filter for each block themselves and only accept a block if it contained an accurate commitment. That would allow a lightweight client to download an 80-byte block header, a (usually) small coinbase transaction, and the filter for that block to receive strong evidence that the filter was accurate.

10.11. Lightweight Clients and Privacy

Lightweight clients have weaker privacy than a full node. A full node downloads all transactions and therefore reveals no information about whether it is using some address in its wallet. A lightweight client only downloads transactions that are related to its wallet in some way.

Bloom filters and compact block filters are ways to reduce the loss of privacy. Without them, a lightweight client would have to explicitly list the addresses it was interested in, creating a serious breach of privacy. However, even with filters, an adversary monitoring the traffic of a lightweight client or connected to it directly as a node in the P2P network may be able to collect enough information over time to learn the addresses in the wallet of the lightweight client.

10.12. Encrypted and Authenticated Connections

Most new users of Bitcoin assume that the network communications of a Bitcoin node are encrypted. In fact, the original implementation of Bitcoin communicates entirely in the clear, as does the modern implementation of Bitcoin Core at the time of writing.

As a way to increase the privacy and security of the Bitcoin P2P network, there is a solution that provides encryption of the communications: *Tor transport*.

Tor, which stands for *The Onion Routing network*, is a software project and network that offers encryption and encapsulation of data through randomized network paths that offer anonymity, untraceability, and privacy.

Bitcoin Core offers several configuration options that allow you to run a Bitcoin node with its traffic transported over the Tor network. In addition, Bitcoin Core can also offer a Tor hidden service allowing other Tor nodes to connect to your node directly over Tor.

As of Bitcoin Core version 0.12, a node will offer a hidden Tor service automatically if it is able to connect to a local Tor service. If you have Tor installed and the Bitcoin Core process runs as a user with adequate permissions to access the Tor authentication cookie, it should work automatically. Use the debug flag to turn on Bitcoin Core's debugging for the Tor service like this:

```
$ bitcoind --daemon --debug=tor
```

You should see tor: ADD_ONION successful in the logs, indicating that Bitcoin Core has added a hidden service to the Tor network.

You can find more instructions on running Bitcoin Core as a Tor hidden service in the Bitcoin Core documentation (*docs/tor.md*) and various online tutorials.

10.13. Mempools and Orphan Pools

Almost every node on the Bitcoin network maintains a temporary list of unconfirmed transactions called the *memory pool* (*mempool*). Nodes use this pool to keep track of transactions that are known to the network but are not yet included in the blockchain, called *unconfirmed transactions*.

As unconfirmed transactions are received and verified, they are added to the mempool and relayed to the neighboring nodes to propagate on the network.

Some node implementations also maintain a separate pool of orphaned transactions. If a transaction's inputs refer to a transaction that is not yet known, such as a missing parent, the orphan transaction will be stored temporarily in the orphan pool until the parent transaction arrives.

When a transaction is added to the mempool, the orphan pool is checked for any orphans that reference this transaction's outputs (its children). Any matching orphans are then validated. If valid, they are removed from the orphan pool and added to the mempool, completing the chain that started with the parent transaction. In light of the newly added transaction, which is no longer an orphan, the process is repeated recursively looking for any further descendants until no more descendants are found. Through this process, the arrival of a parent transaction triggers a cascade reconstruction of an entire chain of interdependent transactions by reuniting the orphans with their parents all the way down the chain.

Some implementations of Bitcoin also maintain a UTXO database, which is the set of all unspent outputs on the blockchain. This represents a different set of data from the mempool. Unlike the mempool and orphan pools, the UTXO database contains millions of entries of unspent transaction outputs, everything that is unspent from all the way back to the genesis block. The UTXO database is stored as a table on persistent storage.

Whereas the mempool and orphan pools represent a single node's local perspective and might vary significantly from node to node depending on when the node was started or restarted, the UTXO database represents the emergent consensus of the network and therefore will not usually vary between nodes.

Now that we have an understanding of many of the data types and structures used by nodes and clients to send data across the Bitcoin network, it's time to look at the software that's responsible for keeping the network secure and operational.

11. The Blockchain

The blockchain is the history of every confirmed Bitcoin transaction. It’s what allows every full node to independently determine what keys and scripts control which bitcoins. In this chapter, we’ll look at the structure of the blockchain and see how it uses cryptographic commitments and other clever tricks to make every part of it easy for full nodes (and sometimes lightweight clients) to validate.

The blockchain data structure is an ordered, back-linked list of blocks of transactions. The blockchain can be stored as a flat file or in a simple database. Blocks are linked "back," each referring to the previous block in the chain. The blockchain is often visualized as a vertical stack, with blocks layered on top of each other and the first block serving as the foundation of the stack. The visualization of blocks stacked on top of each other results in the use of terms such as "height" to refer to the distance from the first block, and "top" or "tip" to refer to the most recently added block.

Each block within the blockchain is identified by a hash, generated using the SHA256 cryptographic hash algorithm on the header of the block. Each block also commits to the previous block, known as the *parent* block, through the "previous block hash" field in the block header. The sequence of hashes linking each block to its parent creates a chain going back all the way to the first block ever created, known as the *genesis block*.

Although a block has just one parent, it can have multiple children. Each of the children commits to the same parent block. Multiple children arise during a blockchain "fork," a temporary situation that can occur when different blocks are discovered almost simultaneously by different miners (see Assembling and Selecting Chains of Blocks). Eventually only one child block becomes part of the blockchain accepted by all full nodes, and the "fork" is resolved.

The "previous block hash" field is inside the block header and thereby affects the *current* block’s hash. Any change to a parent block requires a child block’s hash to change, which requires a change in the pointer of the grandchild, which in turn changes the grandchild, and so on. This sequence ensures that, once a block has many generations following it, it cannot be changed without forcing a recalculation of all subsequent blocks. Because such a recalculation would require enormous computation (and therefore energy consumption), the existence of a long chain of blocks makes the blockchain’s deep history impractical to change, which is a key feature of Bitcoin’s security.

One way to think about the blockchain is like layers in a geological formation, or glacier core sample. The surface layers might change with the seasons, or even be blown away before they have time to settle. But once you go a few inches deep, geological layers become more and more stable. By the time you look a few hundred feet down, you are looking at a snapshot of the past that has remained undisturbed for millions of years. In the blockchain, the most recent few blocks might be revised if there is a chain reorganization due to a fork. The top six blocks are like a few inches of topsoil. But once

you go more deeply into the blockchain, beyond six blocks, blocks are less and less likely to change. After 100 blocks back there is so much stability that the coinbase transaction—the transaction containing the reward in bitcoin for creating a new block—can be spent. While the protocol always allows a chain to be undone by a longer chain and while the possibility of any block being reversed always exists, the probability of such an event decreases as time passes until it becomes infinitesimal.

11.1. Structure of a Block

A block is a container data structure that aggregates transactions for inclusion in the blockchain. The block is made of a header, containing metadata, followed by a long list of transactions that make up the bulk of its size. The block header is 80 bytes, whereas the total size of all transactions in a block can be up to about 4,000,000 bytes. A complete block, with all transactions, can therefore be almost 50,000 times larger than the block header. [block_structure1] describes how Bitcoin Core stores the structure of a block.

The structure of a block

Size	Field	Description
4 bytes	Block Size	The size of the block, in bytes, following this field
80 bytes	Block Header	Several fields form the block header
1–3 bytes (compactSize)	Transaction Counter	How many transactions follow
Variable	Transactions	The transactions recorded in this block

11.2. Block Header

The block header consists of block metadata as shown in [block_header_structure_ch09].

The structure of the block header

Size	Field	Description
4 bytes	Version	Originally a version field; its use has evolved over time
32 bytes	Previous Block Hash	A hash of the previous (parent) block in the chain
32 bytes	Merkle Root	The root hash of the merkle tree of this block’s transactions
4 bytes	Timestamp	The approximate creation time of this block (Unix epoch time)
4 bytes	Target	A compact encoding of the proof-of-work target for this block

Size	Field	Description
4 bytes	Nonce	Arbitrary data used for the proof-of-work algorithm

The nonce, target, and timestamp are used in the mining process and will be discussed in more detail in Mining and Consensus.

11.3. Block Identifiers: Block Header Hash and Block Height

The primary identifier of a block is its cryptographic hash, a commitment made by hashing the block header twice through the SHA256 algorithm. The resulting 32-byte hash is called the *block hash* but is more accurately the *block header hash*, because only the block header is used to compute it. For example, 000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f is the block hash of the first block on Bitcoin's blockchain. The block hash identifies a block uniquely and unambiguously and can be independently derived by any node by simply hashing the block header.

Note that the block hash is not actually included inside the block's data structure. Instead, the block's hash is computed by each node as the block is received from the network. The block hash might be stored in a separate database table as part of the block's metadata, to facilitate indexing and faster retrieval of blocks from disk.

A second way to identify a block is by its position in the blockchain, called the *block height*. The genesis block is at block height 0 (zero) and is the same block that was previously referenced by the following block hash 000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f. A block can thus be identified in two ways: by referencing the block hash or by referencing the block height. Each subsequent block added "on top" of that first block is one position "higher" in the blockchain, like boxes stacked one on top of the other. The block height 800,000 was reached during the writing of this book in mid-2023, meaning there were 800,000 blocks stacked on top of the first block created in January 2009.

Unlike the block hash, the block height is not a unique identifier. Although a single block will always have a specific and invariant block height, the reverse is not true—the block height does not always identify a single block. Two or more blocks might have the same block height, competing for the same position in the blockchain. This scenario is discussed in detail in the section Assembling and Selecting Chains of Blocks. In early blocks, the block height was also not a part of the block's data structure; it was not stored within the block. Each node dynamically identified a block's position (height) in the blockchain when it was received from the Bitcoin network. A later protocol change (BIP34) began including the block height in the coinbase transaction, although its purpose was to ensure each

block had a different coinbase transaction. Nodes still need to dynamically identify a block's height in order to validate the coinbase field. The block height might also be stored as metadata in an indexed database table for faster retrieval.

> **Tip** A block's *block hash* always identifies a single block uniquely. A block also always has a specific *block height*. However, it is not always the case that a specific block height identifies a single block. Rather, two or more blocks might compete for a single position in the blockchain.

11.4. The Genesis Block

The first block in the blockchain is called the *genesis block* and was created in 2009. It is the common ancestor of all the blocks in the blockchain, meaning that if you start at any block and follow the chain backward in time, you will eventually arrive at the genesis block.

Every node always starts with a blockchain of at least one block because the genesis block is statically encoded within Bitcoin Core, such that it cannot be altered. Every node always "knows" the genesis block's hash and structure, the fixed time it was created, and even the single transaction within. Thus, every node has the starting point for the blockchain, a secure "root" from which to build a trusted blockchain.

See the statically encoded genesis block inside the Bitcoin Core client in *chainparams.cpp*.

The following identifier hash belongs to the genesis block:

```
000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
```

You can search for that block hash in almost any block explorer website, such as *blockstream.info*, and you will find a page describing the contents of this block, with a URL containing that hash:

> https://blockstream.info/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

Alternatively, you can get the block using Bitcoin Core on the command line:

```
$ bitcoin-cli getblock \
  000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
```

```
{
  "hash": "000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f",
  "confirmations": 790496,
  "height": 0,
  "version": 1,
  "versionHex": "00000001",
  "merkleroot": "4a5e1e4baab89f3a32518a88c3[...]76673e2cc77ab2127b7afdeda33b",
```

```
  "time": 1231006505,
  "mediantime": 1231006505,
  "nonce": 2083236893,
  "bits": "1d00ffff",
  "difficulty": 1,
  "chainwork": "[...]0000000000000000000000000000000000000000000000100010001",
  "nTx": 1,
  "nextblockhash": "00000000839a8e6886ab5951d7[...]fc90947ee320161bbf18eb6048",
  "strippedsize": 285,
  "size": 285,
  "weight": 1140,
  "tx": [
    "4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b"
  ]
}
```

The genesis block contains a message within it. The coinbase transaction input contains the text "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks." This message was intended to offer proof of the earliest date this block could have been created, by referencing the headline of the British newspaper *The Times*. It also serves as a tongue-in-cheek reminder of the importance of an independent monetary system, with Bitcoin's launch occurring at the same time as an unprecedented worldwide monetary crisis. The message was embedded in the first block by Satoshi Nakamoto, Bitcoin's creator.

11.5. Linking Blocks in the Blockchain

Bitcoin full nodes validate every block in the blockchain after the genesis block. Their local view of the blockchain is constantly updated as new blocks are found and used to extend the chain. As a node receives incoming blocks from the network, it will validate these blocks and then link them to its view of the existing blockchain. To establish a link, a node will examine the incoming block header and look for the "previous block hash."

Let's assume, for example, that a node has 277,314 blocks in the local copy of the blockchain. The last block the node knows about is block 277,314, with a block header hash of:

```
0000000000000027e7ba6fe7bad39faf3b5a83daed765f05f7d1b71a1632249
```

The Bitcoin node then receives a new block from the network, which it parses as follows:

```
{
    "size" : 43560,
    "version" : 2,
    "previousblockhash" :
        "0000000000000027e7ba6fe7bad39faf3b5a83daed765f05f7d1b71a1632249",
    "merkleroot" :
        "5e049f4030e0ab2debb92378f53c0a6e09548aea083f3ab25e1d94ea1155e29d",
    "time" : 1388185038,
    "difficulty" : 1180923195.25802612,
    "nonce" : 4215469401,
```

```
    "tx" : [
        "257e7497fb8bc68421eb2c7b699dbab234831600e7352f0d9e6522c7cf3f6c77",
        "[... many more transactions omitted ...]",
        "05cfd38f6ae6aa83674cc99e4d75a1458c165b7ab84725eda41d018a09176634"
    ]
}
```

Looking at this new block, the node finds the previousblockhash field, which contains the hash of its parent block. It is a hash known to the node, that of the last block on the chain at height 277,314. Therefore, this new block is a child of the last block on the chain and extends the existing blockchain. The node adds this new block to the end of the chain, making the blockchain longer with a new height of 277,315. Blocks linked in a chain by each referencing the previous block header hash. shows the chain of three blocks, linked by references in the previousblockhash field.

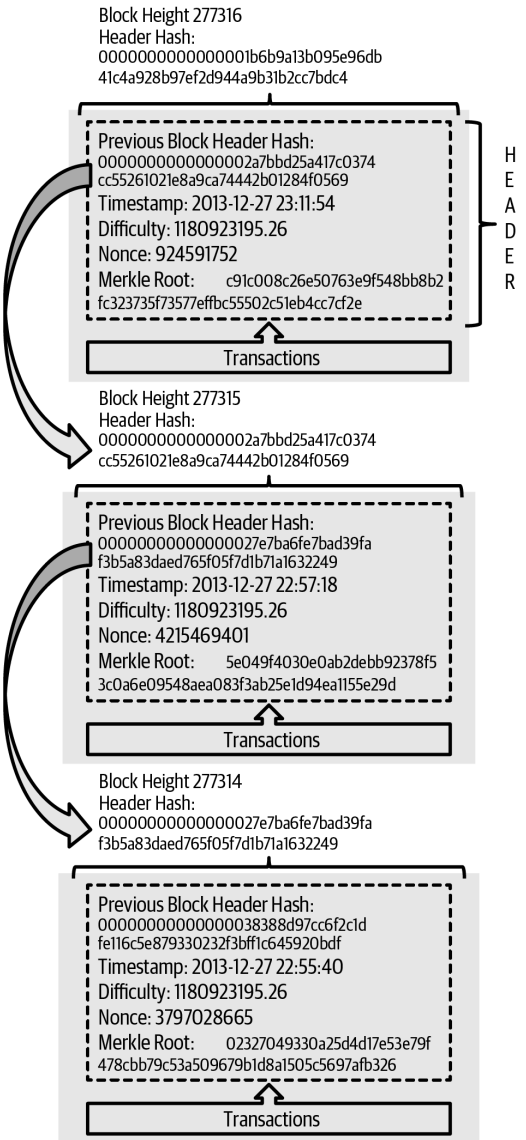


Figure 57. Blocks linked in a chain by each referencing the previous block header hash.

11.6. Merkle Trees

Each block in the Bitcoin blockchain contains a summary of all the transactions in the block using a *merkle tree*.

A *merkle tree,* also known as a *binary hash tree,* is a data structure used for efficiently summarizing and verifying the integrity of large sets of data. Merkle trees are binary trees containing cryptographic hashes. The term "tree" is used in computer science to describe a

branching data structure, but these trees are usually displayed upside down with the "root" at the top and the "leaves" at the bottom of a diagram, as you will see in the examples that follow.

Merkle trees are used in Bitcoin to summarize all the transactions in a block, producing an overall commitment to the entire set of transactions and permitting a very efficient process to verify whether a transaction is included in a block. A merkle tree is constructed by recursively hashing pairs of elements until there is only one hash, called the *root*, or *merkle root*. The cryptographic hash algorithm used in Bitcoin's merkle trees is SHA256 applied twice, also known as double-SHA256.

When N data elements are hashed and summarized in a merkle tree, you can check to see if any one data element is included in the tree with about log~2~(N) calculations, making this a very efficient data structure.

The merkle tree is constructed bottom-up. In the following example, we start with four transactions, A, B, C, and D, which form the *leaves* of the merkle tree, as shown in Calculating the nodes in a merkle tree.. The transactions are not stored in the merkle tree; rather, their data is hashed and the resulting hash is stored in each leaf node as H_A, H_B, H_C, and H_D:

```
H_A = SHA256(SHA256(Transaction A))
```

Consecutive pairs of leaf nodes are then summarized in a parent node by concatenating the two hashes and hashing them together. For example, to construct the parent node H_{AB}, the two 32-byte hashes of the children are concatenated to create a 64-byte string. That string is then double-hashed to produce the parent node's hash:

```
H_AB = SHA256(SHA256(H_A || H_B))
```

The process continues until there is only one node at the top, the node known as the merkle root. That 32-byte hash is stored in the block header and summarizes all the data in all four transactions. Calculating the nodes in a merkle tree. shows how the root is calculated by pair-wise hashes of the nodes.

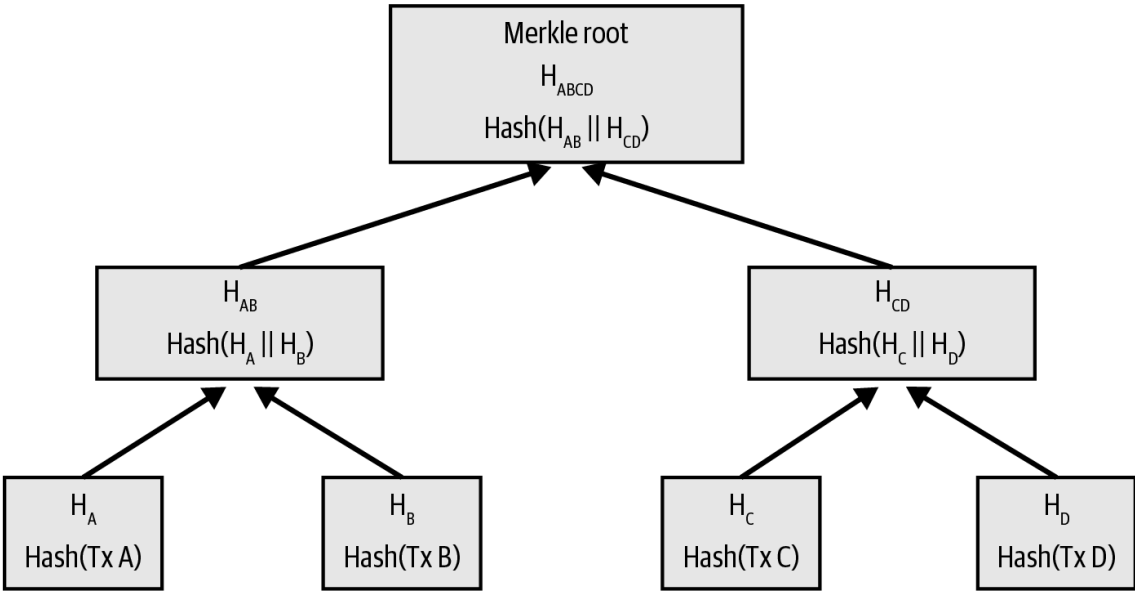


Figure 58. Calculating the nodes in a merkle tree.

Because the merkle tree is a binary tree, it needs an even number of leaf nodes. If there are an odd number of transactions to summarize, the last transaction hash will be duplicated to create an even number of leaf nodes, also known as a *balanced tree*. This is shown in Duplicating one data element achieves an even number of data elements., where transaction C is duplicated. Similarly, if there are an odd number of hashes to process at any level, the last hash is duplicated.

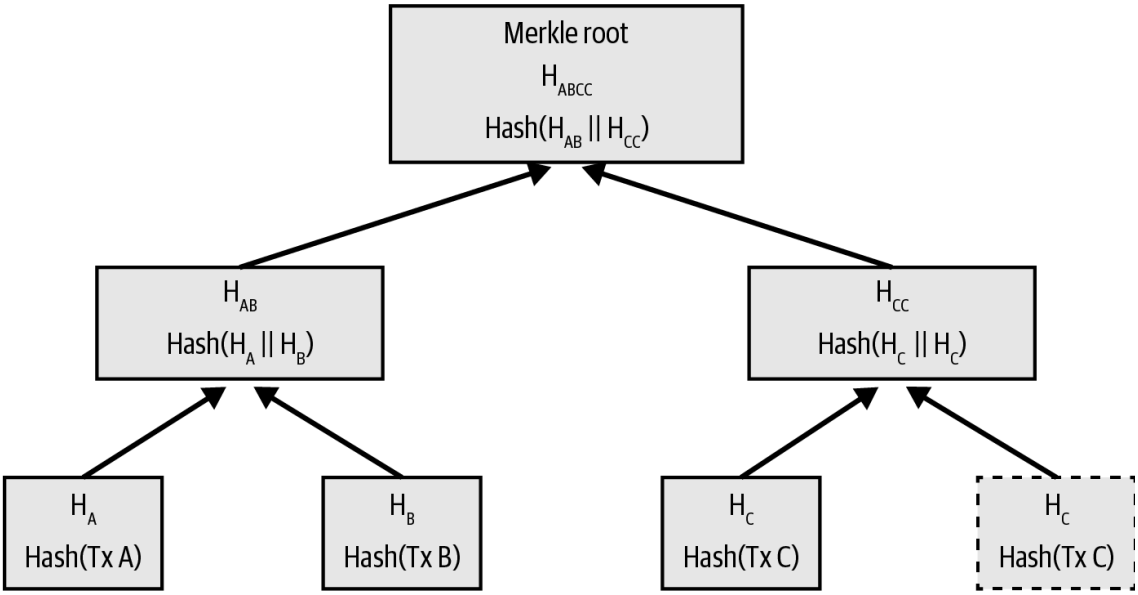


Figure 59. Duplicating one data element achieves an even number of data elements.

A Design Flaw in Bitcoin's Merkle Tree

An extended comment in Bitcoin Core's source code, reproduced here with slight revisions, describes a significant problem in the design of Bitcoin's duplication of odd elements in its merkle tree:

> WARNING! If you're reading this because you're learning about crypto and/or designing a new system that will use merkle trees, keep in mind that the following merkle tree algorithm has a serious flaw related to duplicate txids, resulting in a vulnerability (CVE-2012-2459).
>
> The reason is that if the number of hashes in the list at a given level is odd, the last one is duplicated before computing the next level (which is unusual in merkle trees). This results in certain sequences of transactions leading to the same merkle root. For example, the two trees in Two Bitcoin-style merkle trees with the same root but a different number of leaves.:

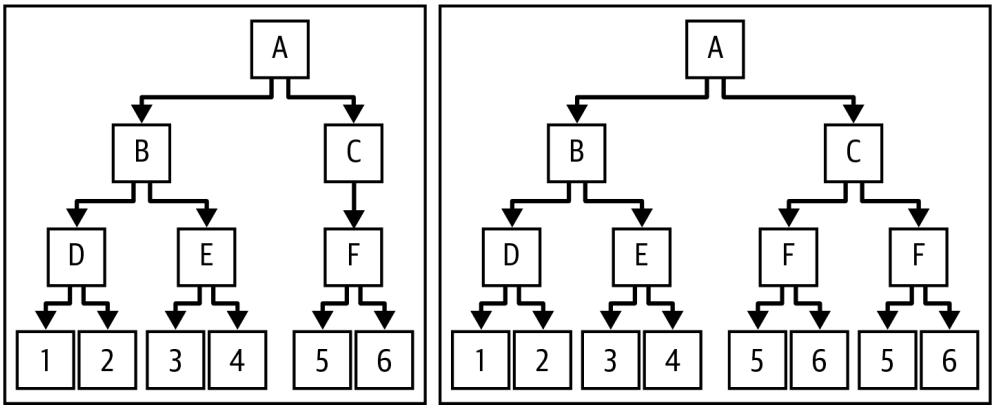


Figure 60. Two Bitcoin-style merkle trees with the same root but a different number of leaves.

The transaction lists [1,2,3,4,5,6] and [1,2,3,4,5,6,5,6] (where 5 and 6 are repeated) result in the same root hash A (because the hash of both of (F) and (F,F) is C).

The vulnerability results from being able to send a block with such a transaction list, with the same merkle root, and the same block hash as the original without duplication, resulting in failed validation. If the receiving node proceeds to mark that block as permanently invalid however, it will fail to accept further unmodified (and thus potentially valid) versions of the same block. We defend against this by detecting the case where we would hash two identical hashes at the end of the list together, and treating that identically to the block having an invalid merkle root. Assuming no double-SHA256 collisions, this will detect all known ways of changing the transactions without affecting the merkle root.

Bitcoin Core *src/consensus/merkle.cpp*

The same method for constructing a tree from four transactions can be generalized to construct trees of any size. In Bitcoin it is common to have several thousand transactions in a single block, which are summarized in exactly the same way, producing just 32 bytes of data as the single merkle root. In A merkle tree summarizing many data elements., you will see a tree built from 16 transactions. Note that although the root looks bigger than the leaf nodes in the diagram, it is the exact same size, just 32 bytes. Whether there is one transaction or ten thousand transactions in the block, the merkle root always summarizes them into 32 bytes.

To prove that a specific transaction is included in a block, a node only needs to produce approximately log~2~(N) 32-byte hashes, constituting an *authentication path* or *merkle path* connecting the specific transaction to the root of the tree. This is especially important as the number of transactions increases because the base-2 logarithm of the number of

transactions increases much more slowly. This allows Bitcoin nodes to efficiently produce paths of 10 or 12 hashes (320–384 bytes), which can provide proof of a single transaction out of more than a thousand transactions in a multimegabyte block.

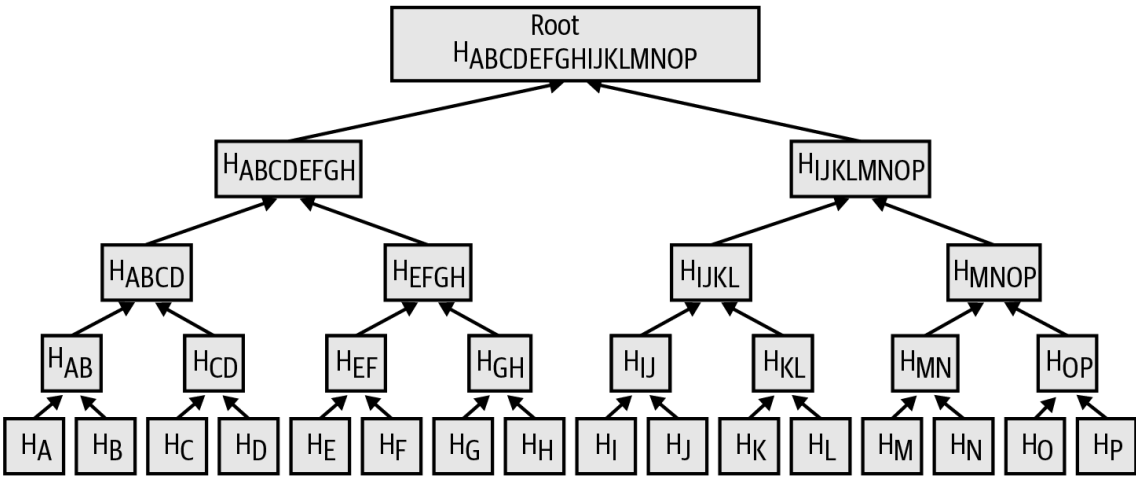


Figure 61. A merkle tree summarizing many data elements.

In A merkle path used to prove inclusion of a data element., a node can prove that a transaction K is included in the block by producing a merkle path that is only four 32-byte hashes long (128 bytes total). The path consists of the four hashes (shown with a shaded background) H_L, H_{IJ}, H_{MNOP}, and $H_{ABCDEFGH}$. With those four hashes provided as an authentication path, any node can prove that H_K (with a black background at the bottom of the diagram) is included in the merkle root by computing four additional pair-wise hashes H_{KL}, H_{IJKL}, $H_{IJKLMNOP}$, and the merkle tree root (outlined in a dashed line in the diagram).

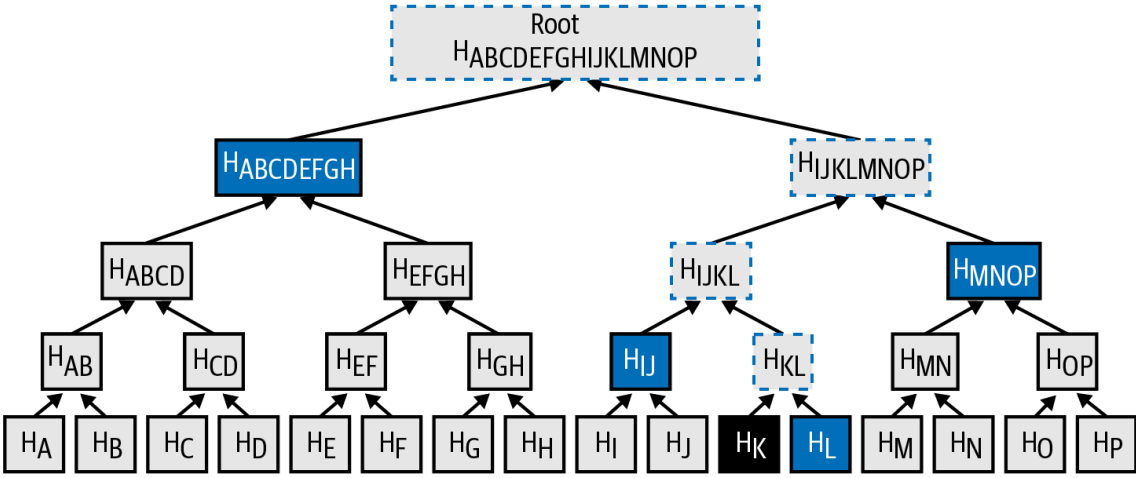


Figure 62. A merkle path used to prove inclusion of a data element.

The efficiency of merkle trees becomes obvious as the scale increases. The largest possible block can hold almost 16,000 transactions in 4,000,000 bytes, but proving any particular one of those 16,000 transactions is a part of that block only requires a copy of the transaction, a copy of the 80-byte block header, and 448 bytes for the merkle proof. That makes the largest possible proof almost 10,000 times smaller than the largest possible Bitcoin block.

11.7. Merkle Trees and Lightweight Clients

Merkle trees are used extensively by lightweight clients. Lightweight clients don't have all transactions and do not download full blocks, just block headers. In order to verify that a transaction is included in a block, without having to download all the transactions in the block, they use a merkle path.

Consider, for example, a lightweight client that is interested in incoming payments to an address contained in its wallet. The lightweight client will establish a bloom filter (see Bloom Filters) on its connections to peers to limit the transactions received to only those containing addresses of interest. When a peer sees a transaction that matches the bloom filter, it will send that block using a merkleblock message. The merkleblock message contains the block header as well as a merkle path that links the transaction of interest to the merkle root in the block. The lightweight client can use this merkle path to connect the transaction to the block header and verify that the transaction is included in the block. The lightweight client also uses the block header to link the block to the rest of the blockchain. The combination of these two links, between the transaction and block and between the block and blockchain, proves that the transaction is recorded in the blockchain. All in all, the lightweight client will have received less than a kilobyte of data for the block header and merkle path, an amount of data that is more than a thousand times less than a full block (about 2 MB currently).

11.8. Bitcoin’s Test Blockchains

You might be surprised to learn that there is more than one blockchain used with Bitcoin. The "main" Bitcoin blockchain, the one created by Satoshi Nakamoto on January 3rd, 2009, the one with the genesis block we studied in this chapter, is called *mainnet*. There are other Bitcoin blockchains that are used for testing purposes: at this time *testnet*, *signet*, and *regtest*. Let’s look at each in turn.

11.8.1. Testnet: Bitcoin’s Testing Playground

Testnet is the name of the test blockchain, network, and currency that is used for testing purposes. The testnet is a fully featured live P2P network, with wallets, test bitcoins (testnet coins), mining, and all the other features of mainnet. The most important difference is that testnet coins are meant to be worthless.

Any software development that is intended for production use on Bitcoin’s mainnet can first be tested on testnet with test coins. This protects both the developers from monetary losses due to bugs and the network from unintended behavior due to bugs.

The current testnet is called *testnet3*, the third iteration of testnet, restarted in February 2011 to reset the difficulty from the previous testnet. Testnet3 is a large blockchain, in excess of 30 GB in 2023. It will take a while to sync fully and use up resources on your computer. Not as much as mainnet, but not exactly "lightweight" either.

> **Tip** Testnet and the other test blockchains described in this book don’t use the same address prefixes as mainnet addresses to prevent someone from accidentally sending real bitcoins to a test address. Mainnet addresses begin with 1, 3, or bc1. Addresses for the test networks mentioned in this book begin with m, n, or tb1. Other test networks, or new protocols being developed on test networks, may use other address prefixes or alterations.

Using testnet

Bitcoin Core, like many other Bitcoin programs, has full support for operation on testnet as an alternative mainnet. All of Bitcoin Core’s functions work on testnet, including the wallet, mining testnet coins, and syncing a full testnet node.

To start Bitcoin Core on testnet instead of mainnet you use the testnet switch:

```
$ bitcoind -testnet
```

In the logs you should see that bitcoind is building a new blockchain in the testnet3 subdirectory of the default bitcoind directory:

```
bitcoind: Using data directory /home/username/.bitcoin/testnet3
```

To connect to bitcoind, you use the bitcoin-cli command-line tool, but you must also switch it to testnet mode:

```
$ bitcoin-cli -testnet getblockchaininfo
{
  "chain": "test",
  "blocks": 1088,
  "headers": 139999,
  "bestblockhash": "0000000063d29909d475a1c[...]368e56cce5d925097bf3a2084370128",
  "difficulty": 1,
  "mediantime": 1337966158,
  "verificationprogress": 0.001644065914099759,
  "chainwork": "[...]0000000000000000000000000000000000000000000044104410441",
  "pruned": false,
  "softforks": [

  [...]
```

You can also run on testnet3 with other full-node implementations, such as btcd (written in Go) and bcoin (written in JavaScript), to experiment and learn in other programming languages and frameworks.

Testnet3 supports all the features of mainnet, including segregated witness v0 and v1 (see Segregated Witness and Taproot). Therefore, testnet3 can also be used to test segregated witness features.

Problems with testnet

Testnet doesn't just use the same data structures as Bitcoin, it also uses almost exactly the same proof-of-work security mechanism as Bitcoin. The notable differences for testnet are that its minimum difficulty is half that of Bitcoin and that it's allowed to include a block at the minimum difficulty if that block's timestamp is more than 20 minutes after the previous block.

Unfortunately, Bitcoin's PoW security mechanism was designed to depend on economic incentives—incentives which don't exist in a test blockchain that is forbidden from having value. On mainnet, miners are incentivized to include user transactions in their blocks because those transactions pay fees. On testnet, transactions still contain something called fees, but those fees don't have any economic value. That means the only incentive for a testnet miner to include transactions is because they want to help users and developers to test their software.

Alas, people who like to disrupt systems often feel a stronger incentive, at least in the short term. Because PoW mining is designed to be permissionless, anyone can mine, whether their intention is good or not. That means disruptive miners can create many blocks in a row on testnet without including any user transactions. When those attacks happen, testnet becomes unusable for users and developers.

11.8.2. Signet: The Proof of Authority Testnet

There's no known way for a system dependent on permissionless PoW to provide a highly usable blockchain without introducing economic incentives, so Bitcoin protocol developers began considering alternatives. The primary goal was to preserve as much of the structure of Bitcoin as possible so that software could run on a testnet with minimal changes—but to also provide an environment that would remain useful. A secondary goal was to produce a reusable design that would allow developers of new software to easily create their own test networks.

The solution implemented in Bitcoin Core and other software is called *signet*, as defined by BIP325. A signet is a test network where each block must contain proof (such as a signature) that the creation of that block was sanctioned by a trusted authority.

Whereas mining in Bitcoin is permissionless—anyone can do it—mining on signet is fully permissioned. Only those with permission can do it. This would be a completely unacceptable change to Bitcoin's mainnet—no one would use that software—but it's reasonable on a testnet where coins have no value and the only purpose is testing software and systems.

BIP325 signets are designed to make it very easy to create your own. If you disagree with how someone else is running their signet, you can start your own signet and connect your software to it.

The default signet and custom signets

Bitcoin Core supports a default signet, which we believe to be the most widely used signet at the time of writing. It is currently operated by two contributors to that project. If you start Bitcoin Core with the signet parameter and no other signet-related parameters, this is the signet you will be using.

As of this writing, the default signet has about 150,000 blocks and is about a gigabyte in size. It supports all of the same features as Bitcoin's mainnet and is also used for testing proposed upgrades through the Bitcoin Inquisition project, which is a software fork of Bitcoin Core that's only designed to run on signet.

If you want to use a different signet, called a *custom signet*, you will need to know the script used to determine when a block is authorized, called the *challenge* script. This is a standard Bitcoin script, so it can use features such as multisig to allow multiple people to authorize blocks. You may also need to connect to a seed node that will provide you with the addresses of peers on the custom signet. For example:

```
bitcoind -signet -signetchallenge=0123...cdef -signetseednode=example.com:1234
```

As of this writing, we generally recommend that the public testing of mining software occur on testnet3 and that all other public testing of Bitcoin software occur on the default signet.

To interact with your chosen signet, you can use the -signet parameter with bitcoin-cli, similar to how you used testnet. For example:

```
$ bitcoin-cli -signet getblockchaininfo
{
  "chain": "signet",
  "blocks": 143619,
  "headers": 143619,
  "bestblockhash": "000000c46cb3505ddd296537[...]ad1c5768e2908439382447572a93",
  "difficulty": 0.003020638517858618,
  "time": 1684530244,
  "mediantime": 1684526116,
  "verificationprogress": 0.999997961940662,
  "initialblockdownload": false,
  "chainwork": "[...]0000000000000000000000000000000000000000000019ab37d2194",
  "size_on_disk": 769525915,
  "pruned": false,
  "warnings": ""
}
```

11.8.3. Regtest: The Local Blockchain

Regtest, which stands for "Regression Testing," is a Bitcoin Core feature that allows you to create a local blockchain for testing purposes. Unlike signet and testnet3, which are public and shared test blockchains, the regtest blockchains are intended to be run as closed systems for local testing. You launch a regtest blockchain from scratch. You may add other nodes to the network or run it with a single node only to test the Bitcoin Core software.

To start Bitcoin Core in regtest mode, you use the regtest flag:

```
$ bitcoind -regtest
```

Just like with testnet, Bitcoin Core will initialize a new blockchain under the *regtest* subdirectory of your bitcoind default directory:

```
bitcoind: Using data directory /home/username/.bitcoin/regtest
```

To use the command-line tool, you need to specify the regtest flag too. Let’s try the getblockchaininfo command to inspect the regtest blockchain:

```
$ bitcoin-cli -regtest getblockchaininfo
{
  "chain": "regtest",
  "blocks": 0,
  "headers": 0,
  "bestblockhash": "0f9188f13cb7b2c71f2a335e3[...]b436012afca590b1a11466e2206",
  "difficulty": 4.656542373906925e-10,
  "mediantime": 1296688602,
  "verificationprogress": 1,
  "chainwork": "[...]0000000000000000000000000000000000000000000000000000000000000002",
```

```
  "pruned": false,
  [...]
```

As you can see, there are no blocks yet. Let's create a default wallet, get an address, and then mine some (500 blocks) to earn the reward:

```
$ bitcoin-cli -regtest createwallet ""

$ bitcoin-cli -regtest getnewaddress
bcrt1qwvfhw8pf79kw6tvpmtxyxwcfnd2t4e8v6qfv4a

$ bitcoin-cli -regtest generatetoaddress 500 \
  bcrt1qwvfhw8pf79kw6tvpmtxyxwcfnd2t4e8v6qfv4a
[
  "3153518205e4630d2800a4cb65b9d2691ac68eea99afa7fd36289cb266b9c2c0",
  "621330dd5bdabcc03582b0e49993702a8d4c41df60f729cc81d94b6e3a5b1556",
  "32d3d83538ba128be3ba7f9dbb8d1ef03e1b536f65e8701893f70dcc1fe2dbf2",
  ...,
  "32d55180d010ffebabf1c3231e1666e9eeed02c905195f2568c987c2751623c7"
]
```

It will only take a few seconds to mine all these blocks, which certainly makes it easy for testing. If you check your wallet balance, you will see that you earned the rewards for the first 400 blocks (coinbase rewards must be 100 blocks deep before you can spend them):

```
$ bitcoin-cli -regtest getbalance
12462.50000000
```

11.9. Using Test Blockchains for Development

Bitcoin's various blockchains (regtest, signet, testnet3, mainnet) offer a range of testing environments for bitcoin development. Use the test blockchains whether you are developing for Bitcoin Core or another full-node consensus client; developing an application such as a wallet, exchange, ecommerce site; or even developing novel smart contracts and complex scripts).

You can use the test blockchains to establish a development pipeline. Test your code locally on a regtest as you develop it. Once you are ready to try it on a public network, switch to signet or testnet to expose your code to a more dynamic environment with more diversity of code and applications. Finally, once you are confident your code works as expected, switch to mainnet to deploy it in production. As you make changes, improvements, bug fixes, etc., start the pipeline again, deploying each change first on regtest, then on signet or testnet, and finally into production.

Now that we know what data the blockchain contains and how cryptographic commitments securely tie the various parts together, we will look at the special commitment that both provide computational security and ensure no block can be changed without invalidating all other blocks built on top of it: Bitcoin's mining function.

12. Mining and Consensus

The word "mining" is somewhat misleading. By evoking the extraction of precious metals, it focuses our attention on the reward for mining, the new bitcoins created in each block. Although mining is incentivized by this reward, the primary purpose of mining is not the reward or the generation of new bitcoins. If you view mining only as the process by which bitcoins are created, you are mistaking the means (incentives) as the goal of the process. Mining is the mechanism that underpins the decentralized clearinghouse, by which transactions are validated and cleared. Mining is one of the inventions that makes Bitcoin special, a decentralized consensus mechanism that is the basis for P2P digital cash.

Mining *secures the Bitcoin system* and enables the emergence of network-wide *consensus without a central authority*. The reward of newly minted bitcoins and transaction fees is an incentive scheme that aligns the actions of miners with the security of the network, while simultaneously implementing the monetary supply.

> **Tip** Mining is one of the mechanisms by which Bitcoin's *consensus security* is *decentralized*.

Miners record new transactions on the global blockchain. A new block, containing transactions that occurred since the last block, is *mined* every 10 minutes on average, thereby adding those transactions to the blockchain. Transactions that become part of a block and added to the blockchain are considered *confirmed*, which allows the new owners of the bitcoins to know that irrevocable effort was expended securing the bitcoins they received in those transactions.

Additionally, transactions in the blockchain have a *topological order* defined by their position in the blockchain. One transaction is earlier than another if it appears in an earlier block or if it appears earlier in the same block. In the Bitcoin protocol, a transaction is only valid if it spends the outputs of transactions that appeared earlier in the blockchain (whether they are earlier in the same block or in an earlier block), and only if no previous transaction spent any of those same outputs. Within a single chain of blocks, the enforcement of topological ordering ensures no two valid transactions can spend the same output, eliminating the problem of *double spending*.

In some protocols built on top of Bitcoin, the topological order of Bitcoin transactions is also used to establish a sequence of events; we'll discuss that idea further in Single-Use Seals.

Miners receive two types of rewards in return for the security provided by mining: new bitcoins created with each new block (called the *subsidy*), and transaction fees from all the transactions included in the block. To earn this reward, miners compete to satisfy a challenge based on a cryptographic hash algorithm. The solution to the problem, called the proof of work, is included in the new block and acts as proof that the miner expended

significant computing effort. The competition to solve the proof-of-work algorithm to earn the reward and the right to record transactions on the blockchain is the basis for Bitcoin's security model.

Bitcoin's money supply is created in a process that's similar to how a central bank issues new money by printing bank notes. The maximum amount of newly created bitcoin a miner can add to a block decreases approximately every four years (or precisely every 210,000 blocks). It started at 50 bitcoins per block in January 2009 and halved to 25 bitcoins per block in November 2012. It halved again to 12.5 bitcoins in July 2016, and again to 6.25 in May 2020. Based on this formula, mining rewards decrease exponentially until approximately the year 2140, when all bitcoins will have been issued. After 2140, no new bitcoin will be issued.

Bitcoin miners also earn fees from transactions. Every transaction may include a transaction fee in the form of a surplus of bitcoins between the transaction's inputs and outputs. The winning bitcoin miner gets to "keep the change" on the transactions included in the winning block. Today, the fees usually represent only a small percentage of a miner's income, with the vast majority coming from the newly minted bitcoins. However, as the reward decreases over time and the number of transactions per block increases, a greater proportion of mining earnings will come from fees. Gradually, the mining reward will be dominated by transaction fees, which will form the primary incentive for miners. After 2140, the amount of new bitcoins in each block drops to zero and mining will be incentivized only by transaction fees.

In this chapter, we will first examine mining as a monetary supply mechanism and then look at the most important function of mining: the decentralized consensus mechanism that underpins Bitcoin's security.

To understand mining and consensus, we will track Alice's transaction as it is received and added to a block by Jing's mining equipment. Then we will follow the block as it is mined, added to the blockchain, and accepted by the Bitcoin network through the process of emergent consensus.

12.1. Bitcoin Economics and Currency Creation

Bitcoin are minted during the creation of each block at a fixed and diminishing rate. Each block, generated on average every 10 minutes, contains entirely new bitcoins, created from nothing. Every 210,000 blocks, or approximately every four years, the currency issuance rate is decreased by 50%. For the first four years of operation of the network, each block contained 50 new bitcoins.

The first halving occurred at block 210,000. The next expected halving after publication of this book will occur at block 840,000, which will probably be produced in April or May of 2024. The rate of new bitcoins decreases exponentially over 32 of these *halvings* until block 6,720,000 (mined approximately in year 2137), when it reaches the minimum currency unit of 1 satoshi. Finally, after 6.93 million blocks, in approximately 2140, almost

2,099,999,997,690,000 satoshis, or almost 21 million bitcoin, will have been issued. Thereafter, blocks will contain no new bitcoins, and miners will be rewarded solely through the transaction fees. Supply of bitcoin currency over time based on a geometrically decreasing issuance rate. shows the total bitcoins in circulation over time, as the issuance of currency decreases.

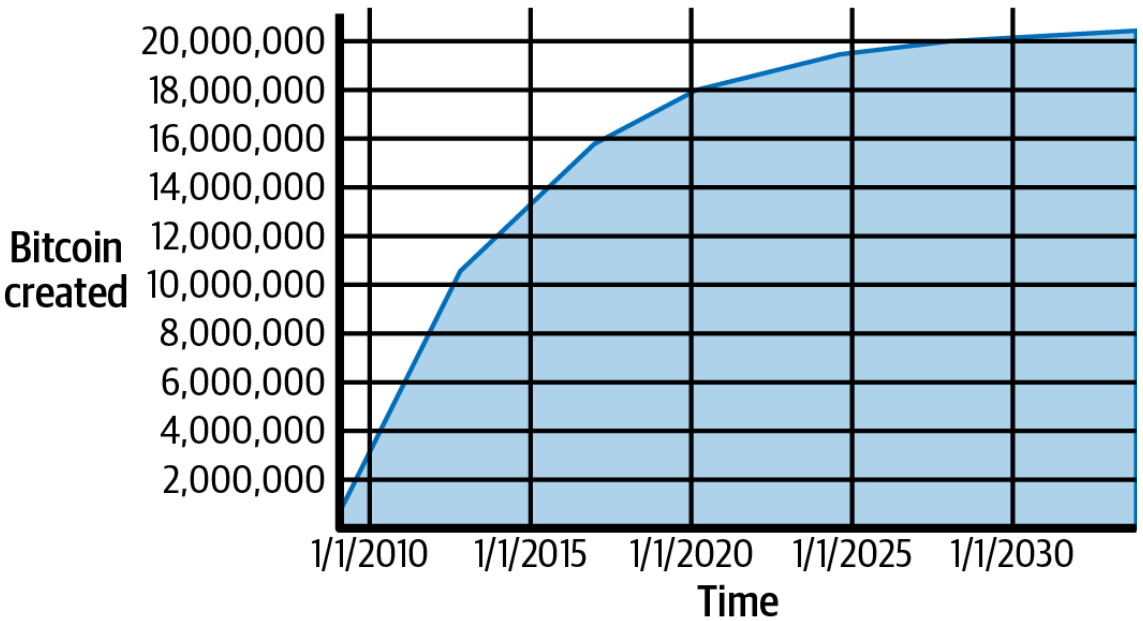


Figure 63. Supply of bitcoin currency over time based on a geometrically decreasing issuance rate.

> **Note** The maximum number of bitcoins mined is the *upper limit* of possible mining rewards for Bitcoin. In practice, a miner may intentionally mine a block taking less than the full reward. Such blocks have already been mined and more may be mined in the future, resulting in a lower total issuance of the currency.

In the code in A script for calculating how much total bitcoin will be issued, we calculate the total amount of bitcoin that will be issued.

Example 14. A script for calculating how much total bitcoin will be issued

```
# Original block reward for miners was 50 BTC
start_block_reward = 50
# 210000 is around every 4 years with a 10 minute block interval
reward_interval = 210000


def max_money():
    # 50 BTC = 50 0000 0000 Satoshis
    current_reward = 50 * 10**8
```

```
    total = 0
    while current_reward > 0:
        total += reward_interval * current_reward
        current_reward /= 2
    return total


print("Total BTC to ever be created:", max_money(), "Satoshis")
```

Running the max_money.py script shows the output produced by running this script.

Example 15. Running the max_money.py script

```
$ python max_money.py
Total BTC to ever be created: 2099999997690000 Satoshis
```

The finite and diminishing issuance creates a fixed monetary supply that resists inflation. Unlike a fiat currency, which can be printed in infinite numbers by a central bank, no individual party has the ability to inflate the supply of bitcoin.

Deflationary Money

The most important and debated consequence of fixed and diminishing monetary issuance is that the currency tends to be inherently *deflationary*. Deflation is the phenomenon of appreciation of value due to a mismatch in supply and demand that drives up the value (and exchange rate) of a currency. Price deflation is the opposite of inflation; it means that the money has more purchasing power over time.

Many economists argue that a deflationary economy is a disaster that should be avoided at all costs. That is because in a period of rapid deflation, people tend to hoard money instead of spending it, hoping that prices will fall. Such a phenomenon unfolded during Japan's "Lost Decade," when a complete collapse of demand pushed the currency into a deflationary spiral.

Bitcoin experts argue that deflation is not bad per se. Rather, deflation is associated with a collapse in demand because that is the most obvious example of deflation we have to study. In a fiat currency with the possibility of unlimited printing, it is very difficult to enter a deflationary spiral unless there is a complete collapse in demand and an unwillingness to print money. Deflation in Bitcoin is not caused by a collapse in demand, but by a predictably constrained supply.

The positive aspect of deflation, of course, is that it is the opposite of inflation. Inflation causes a slow but inevitable debasement of currency, resulting in a form of hidden taxation that punishes savers in order to bail out debtors (including the biggest debtors, governments themselves). Currencies under government control suffer from the moral hazard of easy debt issuance that can later be erased through debasement at the expense of savers.

It remains to be seen whether the deflationary aspect of the currency is a problem when it is not driven by rapid economic retraction, or an advantage because the protection from inflation and debasement outweighs the risks of deflation.

12.2. Decentralized Consensus

In the previous chapter we looked at the blockchain, the global list of all transactions, which everyone in the Bitcoin network accepts as the authoritative record of ownership transfers.

But how can everyone in the network agree on a single universal "truth" about who owns what, without having to trust anyone? All traditional payment systems depend on a trust model that has a central authority providing a clearinghouse service, basically verifying and clearing all transactions. Bitcoin has no central authority, yet somehow every full node has a complete copy of a public blockchain that it can trust as the authoritative record. The blockchain is not created by a central authority but is assembled independently by every node in the network. Somehow, every node in the network, acting on information transmitted across insecure network connections, can arrive at the same conclusion and assemble a copy of the same blockchain as everyone else. This chapter examines the process by which the Bitcoin network achieves global consensus without central authority.

One of Satoshi Nakamoto's inventions is the decentralized mechanism for *emergent consensus*. Emergent because consensus is not achieved explicitly—there is no election or fixed moment when consensus occurs. Instead, consensus is an emergent artifact of the asynchronous interaction of thousands of independent nodes, all following simple rules. All the properties of Bitcoin, including currency, transactions, payments, and the security model that does not depend on central authority or trust, derive from this invention.

Bitcoin's decentralized consensus emerges from the interplay of four processes that occur independently on nodes across the network:

- Independent verification of each transaction, by every full node, based on a comprehensive list of criteria
- Independent aggregation of those transactions into new blocks by mining nodes, coupled with demonstrated computation through a proof-of-work algorithm
- Independent verification of the new blocks by every node and assembly into a chain
- Independent selection, by every node, of the chain with the most cumulative computation demonstrated through proof of work

In the next few sections, we will examine these processes and how they interact to create the emergent property of network-wide consensus that allows any Bitcoin node to assemble its own copy of the authoritative, trusted, public, global blockchain.

12.3. Independent Verification of Transactions

In Transactions, we saw how wallet software creates transactions by collecting UTXOs, providing the appropriate authentication data, and then constructing new outputs assigned to a new owner. The resulting transaction is then sent to the neighboring nodes in the

Bitcoin network so that it can be propagated across the entire Bitcoin network.

However, before forwarding transactions to its neighbors, every Bitcoin node that receives a transaction will first verify the transaction. This ensures that only valid transactions are propagated across the network, while invalid transactions are discarded at the first node that encounters them.

Each node verifies every transaction against a long checklist of criteria:

- The transaction’s syntax and data structure must be correct.
- Neither lists of inputs nor outputs are empty.
- The transaction weight is low enough to allow it to fit in a block.
- Each output value, as well as the total, must be within the allowed range of values (zero or more, but not exceeding 21 million bitcoins).
- Lock time is equal to INT_MAX, or lock time and sequence values are satisfied according to the lock time and BIP68 rules.
- The number of signature operations (SIGOPS) contained in the transaction is less than the signature operation limit.
- The outputs being spent match outputs in the mempool or unspent outputs in a block in the main branch.
- For each input, if the referenced output transaction is a coinbase output, it must have at least COINBASE_MATURITY (100) confirmations. Any absolute or relative lock time must also be satisfied. Nodes may relay transactions a block before they mature since they will be mature if included in the next block.
- Reject if the sum of input values is less than sum of output values.
- The scripts for each input must validate against the corresponding output scripts.

Note that the conditions change over time, to add new features or address new types of denial-of-service attacks.

By independently verifying each transaction as it is received and before propagating it, every node builds a pool of valid (but unconfirmed) transactions known as the *memory pool* or *mempool*.

12.4. Mining Nodes

Some of the nodes on the Bitcoin network are specialized nodes called *miners*. Jing is a Bitcoin miner; he earns bitcoin by running a "mining rig," which is a specialized computer-hardware system designed to mine bitcoin. Jing’s specialized mining hardware is connected to a server running a full node. Like every other full node, Jing’s node receives and propagates unconfirmed transactions on the Bitcoin network. Jing’s node, however, also aggregates these transactions into new blocks.

Let’s follow the blocks that were created during the time Alice made a purchase from Bob (see Buying from an Online Store). For the purpose of demonstrating the concepts in this chapter, let’s assume the block containing Alice’s transaction was mined by Jing’s mining system and follow Alice’s transaction as it becomes part of this new block.

Jing’s mining node maintains a local copy of the blockchain. By the time Alice buys something, Jing’s node is caught up with the chain of blocks with the most proof of work. Jing’s node is listening for transactions, trying to mine a new block and also listening for blocks discovered by other nodes. As Jing’s node is mining, it receives a new block through the Bitcoin network. The arrival of this block signifies the end of the search for that block and the beginning of the search to create the next block.

During the previous several minutes, while Jing’s node was searching for a solution to the previous block, it was also collecting transactions in preparation for the next block. By now it has collected a few thousand transactions in its memory pool. Upon receiving the new block and validating it, Jing’s node will also compare it against all the transactions in the memory pool and remove any that were included in that block. Whatever transactions remain in the memory pool are unconfirmed and are waiting to be recorded in a new block.

Jing’s node immediately constructs a new partial block, a candidate for the next block. This block is called a *candidate block* because it is not yet a valid block, as it does not contain a valid proof-of-work. The block becomes valid only if the miner succeeds in finding a solution according to the proof-of-work algorithm.

When Jing’s node aggregates all the transactions from the memory pool, the new candidate block has several thousand transactions that each pay transaction fees he’ll attempt to claim.

12.4.1. The Coinbase Transaction

The first transaction in any block is a special transaction, called a *coinbase transaction*. This transaction is constructed by Jing’s node and pays out his *reward* for the mining effort.

Jing’s node creates the coinbase transaction as a payment to his own wallet. The total amount of reward that Jing collects for mining a block is the sum of the block subsidy (6.25 new bitcoins in 2023) and the transaction fees from all the transactions included in the block.

Unlike regular transactions, the coinbase transaction does not consume (spend) UTXOs as inputs. Instead, it has only one input, called the *coinbase input*, which implicitly contains the block reward. The coinbase transaction must have at least one output and may have as many outputs as will fit in the block. It’s common for coinbase transactions in 2023 to have two outputs: one of these is a zero-value output that uses OP_RETURN to commit to all of the witnesses for segregated witness (segwit) transactions in the block. The other output pays the miner their reward.

12.4.2. Coinbase Reward and Fees

To construct the coinbase transaction, Jing's node first calculates the total amount of transaction fees:

$$Total\ Fees = Sum(Inputs) - Sum(Outputs)$$

Next, Jing's node calculates the correct reward for the new block. The reward is calculated based on the block height, starting at 50 bitcoin per block and reduced by half every 210,000 blocks.

The calculation can be seen in function GetBlockSubsidy in the Bitcoin Core client, as shown in Calculating the block reward—Function GetBlockSubsidy, Bitcoin Core Client, main.cpp.

Example 16. Calculating the block reward—Function GetBlockSubsidy, Bitcoin Core Client, main.cpp

```
CAmount GetBlockSubsidy(int nHeight, const Consensus::Params& consensusParams)
{
    int halvings = nHeight / consensusParams.nSubsidyHalvingInterval;
    // Force block reward to zero when right shift is undefined.
    if (halvings >= 64)
        return 0;

    CAmount nSubsidy = 50 * COIN;
    // Subsidy is cut in half every 210,000 blocks.
    nSubsidy >>= halvings;
    return nSubsidy;
}
```

The initial subsidy is calculated in satoshis by multiplying 50 with the COIN constant (100,000,000 satoshis). This sets the initial reward (nSubsidy) at 5 billion satoshis.

Next, the function calculates the number of halvings that have occurred by dividing the current block height by the halving interval (SubsidyHalvingInterval).

Next, the function uses the binary-right-shift operator to divide the reward (nSubsidy) by two for each round of halving. In the case of block 277,316, this would binary-right-shift the reward of 5 billion satoshis once (one halving) and result in 2.5 billion satoshis, or 25 bitcoins. After the 33rd halving, the subsidy will be rounded down to zero. The binary-right-shift operator is used because it is more efficient than multiple repeated divisions. To avoid a potential bug, the shift operation is skipped after 63 halvings, and the subsidy is set to 0.

Finally, the coinbase reward (nSubsidy) is added to the transaction fees (nFees), and the sum is returned.

> **Tip** If Jing's mining node writes the coinbase transaction, what stops Jing from "rewarding" himself 100 or 1,000 bitcoin? The answer is that an inflated reward would result in the block being deemed invalid by everyone else, wasting Jing's electricity used for PoW. Jing only gets to spend the reward if the block is accepted by everyone.

12.4.3. Structure of the Coinbase Transaction

With these calculations, Jing's node then constructs the coinbase transaction to pay himself the block reward.

The coinbase transaction has a special format. Instead of a transaction input specifying a previous UTXO to spend, it has a "coinbase" input. We examined transaction inputs in Inputs. Let's compare a regular transaction input with a coinbase transaction input. [table_8-1] shows the structure of a regular transaction, while [table_8-2] shows the structure of the coinbase transaction's input.

The structure of a "normal" transaction input

Size	Field	Description
32 bytes	Transaction Hash	Pointer to the transaction containing the UTXO to be spent
4 bytes	Output Index	The index number of the UTXO to be spent, first one is 0
1–9 bytes (compactSize)	Script Size	Script length in bytes, to follow
Variable	Input Script	A script that fulfills the conditions of the UTXO output script
4 bytes	Sequence Number	Multipurpose field used for BIP68 timelocks and transaction replacement signaling

The structure of a coinbase transaction input

Size	Field	Description
32 bytes	Transaction Hash	All bits are zero: Not a transaction hash reference
4 bytes	Output Index	All bits are ones: 0xFFFFFFFF
1 byte	Coinbase Data Size	Length of the coinbase data, from 2 to 100 bytes

Size	Field	Description
Variable	Coinbase Data	Arbitrary data used for extra nonce and mining tags; in v2 blocks, must begin with block height
4 bytes	Sequence Number	Set to 0xFFFFFFFF

In a coinbase transaction, the first two fields are set to values that do not represent a UTXO reference. Instead of a "transaction hash," the first field is filled with 32 bytes all set to zero. The "output index" is filled with 4 bytes all set to 0xFF (255 decimal). The input script is replaced by coinbase data, a data field used by the miners, as we will see next.

12.4.4. Coinbase Data

Coinbase transactions do not have an input script field. Instead, this field is replaced by coinbase data, which must be between 2 and 100 bytes. Except for the first few bytes, the rest of the coinbase data can be used by miners in any way they want; it is arbitrary data.

In the genesis block, for example, Satoshi Nakamoto added the text "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks" in the coinbase data, using it as a proof of the earliest date this block could have been created and to convey a message. Currently, miners often use the coinbase data to include extra nonce values and strings identifying the mining pool.

The first few bytes of the coinbase used to be arbitrary, but that is no longer the case. As per BIP34, version-2 blocks (blocks with the version field set to 2 or higher) must contain the block height as a script "push" operation in the beginning of the coinbase field.

12.5. Constructing the Block Header

To construct the block header, the mining node needs to fill in six fields, as listed in [block_header_structure_ch10].

The structure of the block header

Size	Field	Description
4 bytes	Version	A multipurpose bitfield
32 bytes	Previous Block Hash	A reference to the hash of the previous (parent) block in the chain
32 bytes	Merkle Root	A hash that is the root of the merkle tree of this block’s transactions
4 bytes	Timestamp	The approximate creation time of this block (seconds from Unix Epoch)

Size	Field	Description
4 bytes	Target	The proof-of-work algorithm target for this block
4 bytes	Nonce	A counter used for the proof-of-work algorithm

The version field was originally an integer field and was used in three upgrades to the Bitcoin network, those defined in BIPs 34, 66, and 65. Each time, the version number was incremented. Later upgrades defined the version field as a bitfield, called *versionbits*, allowing up to 29 upgrades to be in progress simultaneously; see BIP9: Signaling and activation for details. Even later, miners began using some of the versionbits as an auxiliary nonce field.

> **Tip** The protocol upgrades defined in BIPs 34, 66, and 65 occurred in that order, with BIP66 (strict DER) occurring before BIP65 (OP_CHECKTIMELOCKVERIFY), so Bitcoin developers often list them in that order rather than sorted numerically.

Today, the versionbits field has no meaning unless there's an attempt to upgrade the consensus protocol underway, in which case you will need to read its documentation to determine how it is using versionbits.

Next, the mining node needs to add the "Previous Block Hash" (also known as prevhash). That is the hash of the block header of the previous block received from the network, which Jing's node has accepted and selected as the *parent* of his candidate block.

> **Tip** By selecting the specific *parent* block, indicated by the Previous Block Hash field in the candidate block header, Jing is committing his mining power to extending the chain that ends in that specific block.

The next step is to commit to all the transactions using merkle trees. Each transaction is listed using its witness transaction identifier (*wtxid*) in topographical order, with 32 0x00 bytes standing in for the wtxid of the first transaction (the coinbase). As we saw in the Merkle Trees the last wtxid is hashed with itself if there are an odd number of wtxids, creating nodes that each contain the hash of one transaction. The transaction hashes are then combined, in pairs, creating each level of the tree, until all the transactions are summarized into one node at the "root" of the tree. The root of the merkle tree summarizes all the transactions into a single 32-byte value, which is the *witness root hash*.

The witness root hash is added to an output of the coinbase transaction. This step may be skipped if none of the transactions in the block are required to contain a witness structure. Each transaction (including the coinbase transaction) is then listed using its transaction identifier (txid) and used to build a second merkle tree, the root of which becomes the merkle root, to which the block header commits.

Jing's mining node will then add a 4-byte timestamp, encoded as a Unix "epoch" timestamp, which is based on the number of seconds elapsed from January 1, 1970, midnight UTC/GMT.

Jing's node then fills in the nBits target, which must be set to a compact representation of the required PoW to make this a valid block. The target is stored in the block as a "target bits" metric, which is a mantissa-exponent encoding of the target. The encoding has a 1-byte exponent, followed by a 3-byte mantissa (coefficient). In block 277,316, for example, the target bits value is 0x1903a30c. The first part 0x19 is a hexadecimal exponent, while the next part, 0x03a30c, is the coefficient. The concept of a target is explained in Retargeting to Adjust Difficulty and the "target bits" representation is explained in Target Representation.

The final field is the nonce, which is initialized to zero.

With all the other fields filled, the header of the candidate block is now complete and the process of mining can begin. The goal is now to find a header that results in a hash that is less than the target. The mining node will need to test billions or trillions of variations of the header before a version is found that satisfies the requirement.

12.6. Mining the Block

Now that a candidate block has been constructed by Jing's node, it is time for Jing's hardware mining rig to "mine" the block, to find a solution to the proof-of-work algorithm that makes the block valid. Throughout this book we have studied cryptographic hash functions as used in various aspects of the Bitcoin system. The hash function SHA256 is the function used in Bitcoin's mining process.

In the simplest terms, mining is the process of hashing the candidate block header repeatedly, changing one parameter, until the resulting hash matches a specific target. The hash function's result cannot be determined in advance, nor can a pattern be created that will produce a specific hash value. This feature of hash functions means that the only way to produce a hash result matching a specific target is to try again and again, modifying the input until the desired hash result appears by chance.

12.6.1. Proof-of-Work Algorithm

A hash algorithm takes an arbitrary-length data input and produces a fixed-length deterministic result, called a *digest*. The digest is a digital commitment to the input. For any specific input, the resulting digest will always be the same and can be easily calculated and verified by anyone implementing the same hash algorithm. A key characteristic of a cryptographic hash algorithm is that it is computationally infeasible to find two different inputs that produce the same digest (known as a *collision*). As a corollary, it is also virtually impossible to select an input in such a way as to produce a desired digest, other than trying random inputs.

With SHA256, the output is always 256 bits long, regardless of the size of the input. For example, we will calculate the SHA256 hash of the phrase, "Hello, World!":

```
$ echo "Hello, world!" | sha256sum
d9014c4624844aa5bac314773d6b689ad467fa4e1d1a50a1b8a99d5a95f72ff5  -
```

This 256-bit output (represented in hex) is the *hash* or *digest* of the phrase and depends on every part of the phrase. Adding a single letter, punctuation mark, or any other character will produce a different hash.

A variable used in such a scenario is called a *nonce*. The nonce is used to vary the output of a cryptographic function, in this case to vary the output of the SHA256 commitment to the phrase.

To make a challenge out of this algorithm, let's set a target: find a phrase that produces a hexadecimal hash that starts with a zero. Fortunately, this isn't difficult, as shown in Simple proof-of-work implementation.

Example 17. Simple proof-of-work implementation

```
$ for nonce in $( seq 100 ) ; do echo "Hello, world! $nonce" | sha256sum ; done
3194835d60e85bf7f728f3e3f4e4e1f5c752398cbcc5c45e048e4dbcae6be782  -
bfa474bbe2d9626f578d7d8c3acc1b604ec4a7052b188453565a3c77df41b79e  -
[...]
f75a100821c34c84395403afd1a8135f685ca69ccf4168e61a90e50f47552f61  -
09cb91f8250df04a3db8bd98f47c7cecb712c99835f4123e8ea51460ccbec314  -
```

The phrase "Hello, World! 32" produces the following hash, which fits our criteria: 09cb91f8250df04a3db8bd98f47c7cecb712c99835f4123e8ea51460ccbec314. It took 32 attempts to find it. In terms of probabilities, if the output of the hash function is evenly distributed, we would expect to find a result with a 0 as the hexadecimal prefix once every 16 hashes (one out of 16 hexadecimal digits 0 through F). In numerical terms, that means finding a hash value that is less than 0x1000000000000000000000000000000000000000000000000000000000000000. We call this threshold the *target*, and the goal is to find a hash that is numerically less than the target. If we decrease the target, the task of finding a hash that is less than the target becomes more and more difficult.

To give a simple analogy, imagine a game where players throw a pair of dice repeatedly, trying to throw less than a specified target. In the first round, the target is 12. Unless you throw double-6, you win. In the next round the target is 11. Players must throw 10 or less to win, again an easy task. Let's say a few rounds later the target is down to 5. Now, more than half the dice throws will exceed the target and therefore be invalid. It takes more dice throws to win the lower the target gets. Eventually, when the target is 3 (the minimum possible), only one throw out of every 36, or about 3% of them, will produce a winning result.

From the perspective of an observer who knows that the target of the dice game is 3, if someone has succeeded in casting a winning throw it can be assumed that they attempted, on average, 36 throws. In other words, one can estimate the amount of work it takes to succeed from the difficulty imposed by the target. When the algorithm is based on a deterministic function such as SHA256, the input itself constitutes *proof* that a certain amount of *work* was done to produce a result below the target. Hence, *proof of work*.

> **Tip** Even though each attempt produces a random outcome, the probability of any possible outcome can be calculated in advance. Therefore, an outcome of specified difficulty constitutes proof of a specific amount of work.

In Simple proof-of-work implementation, the winning "nonce" is 32, and this result can be confirmed by anyone independently. Anyone can add the number 32 as a suffix to the phrase "Hello, world!" and compute the hash, verifying that it is less than the target:

```
$ echo "Hello, world! 32" | sha256sum
09cb91f8250df04a3db8bd98f47c7cecb712c99835f4123e8ea51460ccbec314  -
```

Although it only takes one hash computation to verify, it took us 32 hash computations to find a nonce that worked. If we had a lower target (higher difficulty), it would take many more hash computations to find a suitable nonce, but only one hash computation for anyone to verify. And by knowing the target, anyone can estimate the difficulty using statistics and therefore know roughly how much work was needed to find such a nonce.

> **Tip** The PoW must produce a hash that is *less than* the target. A higher target means it is less difficult to find a hash that is below the target. A lower target means it is more difficult to find a hash below the target. The target and difficulty are inversely related.

Bitcoin's PoW is very similar to the challenge shown in #sha256_example_generator_output2. The miner constructs a candidate block filled with transactions. Next, the miner calculates the hash of this block's header and sees if it is smaller than the current *target*. If the hash is not less than the target, the miner will modify the nonce (usually just incrementing it by one) and try again. At the current difficulty in the Bitcoin network, miners have to try a huge number of times before finding a nonce that results in a low enough block header hash.

12.6.2. Target Representation

Block headers contain the target in a notation called "target bits" or just "bits," which in block 277,316 has the value of 0x1903a30c. This notation expresses the proof-of-work target as a coefficient/exponent format, with the first two hexadecimal digits for the

exponent and the next six hex digits as the coefficient. In this block, therefore, the exponent is 0x19 and the coefficient is 0x03a30c.

The formula to calculate the difficulty target from this representation is:

- target = coefficient × $2^{(8 \times (\text{exponent} - 3))}$

Using that formula, and the difficulty bits value 0x1903a30c, we get:

- target = 0x03a30c × $2^{0x08 \times (0x19 - 0x03)}$

which is:

- 22,829,202,948,393,929,850,749,706,076,701,368,331,072,452,018,388,575,715,328

Or, in hexadecimal:

- 0x0000000000000003A30C0000000000000000000000000000000000000000000000

This means that a valid block for height 277,316 is one that has a block header hash less than the target. In binary that number must have more than 60 leading bits set to zero. With this level of difficulty, a single miner processing 1 trillion hashes per second (1 terahash per second or 1 TH/sec) would only find a solution once every 8,496 blocks or once every 59 days, on average.

12.6.3. Retargeting to Adjust Difficulty

As we saw, the target determines the difficulty and therefore affects how long it takes to find a solution to the proof-of-work algorithm. This leads to the obvious questions: Why is the difficulty adjustable, who adjusts it, and how?

Bitcoin's blocks are generated every 10 minutes, on average. This is Bitcoin's heartbeat and underpins the frequency of currency issuance and the speed of transaction settlement. It has to remain constant not just over the short term, but over a period of many decades. Over this time, it is expected that computer power will continue to increase at a rapid pace. Furthermore, the number of participants in mining and the computers they use will also constantly change. To keep the block generation time at 10 minutes, the difficulty of mining must be adjusted to account for these changes. In fact, the proof-of-work target is a dynamic parameter that is periodically adjusted to meet a 10-minute block interval goal. In simple terms, the target is set so that the current mining power will result in a 10-minute block interval.

How, then, is such an adjustment made in a completely decentralized network? Retargeting occurs automatically and on every node independently. Every 2,016 blocks, all nodes retarget the PoW. The ratio between the actual time span and desired time span of 10 minutes per block is calculated and a proportionate adjustment (up or down) is made to

the target. In simple terms: If the network is finding blocks faster than every 10 minutes, the difficulty increases (target decreases). If block discovery is slower than expected, the difficulty decreases (target increases).

The equation can be summarized as:

```
New Target = Old Target * (20,160 minutes / Actual Time of Last 2015 Blocks)
```

> **Note** While the target calibration happens every 2,016 blocks, because of an off-by-one error in the original Bitcoin software, it is based on the total time of the previous 2,015 blocks (not 2,016 as it should be), resulting in a retargeting bias toward higher difficulty by 0.05%.

Retargeting the proof of work: CalculateNextWorkRequired() in pow.cpp shows the code used in the Bitcoin Core client.

Example 18. Retargeting the proof of work: CalculateNextWorkRequired() in pow.cpp

```
// Limit adjustment step
 int64_t nActualTimespan = pindexLast->GetBlockTime() - nFirstBlockTime;
 LogPrintf("  nActualTimespan = %d  before bounds\n", nActualTimespan);
 if (nActualTimespan < params.nPowTargetTimespan/4)
     nActualTimespan = params.nPowTargetTimespan/4;
 if (nActualTimespan > params.nPowTargetTimespan*4)
     nActualTimespan = params.nPowTargetTimespan*4;

 // Retarget
 const arith_uint256 bnPowLimit = UintToArith256(params.powLimit);
 arith_uint256 bnNew;
 arith_uint256 bnOld;
 bnNew.SetCompact(pindexLast->nBits);
 bnOld = bnNew;
 bnNew *= nActualTimespan;
 bnNew /= params.nPowTargetTimespan;

 if (bnNew > bnPowLimit)
     bnNew = bnPowLimit;
```

The parameters Interval (2,016 blocks) and TargetTimespan (two weeks as 1,209,600 seconds) are defined in *chainparams.cpp*.

To avoid extreme volatility in the difficulty, the retargeting adjustment must be less than a factor of four (4) per cycle. If the required target adjustment is greater than a factor of four, it will be adjusted by a factor of 4 and not more. Any further adjustment will be accomplished in the next retargeting period because the imbalance will persist through the next 2,016 blocks. Therefore, large discrepancies between hashing power and difficulty might take several 2,016-block cycles to balance out.

Note that the target is independent of the number of transactions or the value of transactions. This means that the amount of hashing power and therefore electricity expended to secure bitcoin is also entirely independent of the number of transactions. Bitcoin can scale up and remain secure without any increase in hashing power from today's level. The increase in hashing power represents market forces as new miners enter the market. As long as enough hashing power is under the control of miners acting honestly in pursuit of the reward, it is enough to prevent "takeover" attacks and, therefore, it is enough to secure bitcoin.

The difficulty of mining is closely related to the cost of electricity and the exchange rate of bitcoin vis-a-vis the currency used to pay for electricity. High-performance mining systems are about as efficient as possible with the current generation of silicon fabrication, converting electricity into hashing computation at the highest rate possible. The primary influence on the mining market is the price of one kilowatt-hour of electricity in bitcoin because that determines the profitability of mining and therefore the incentives to enter or exit the mining market.

12.7. Median Time Past (MTP)

In Bitcoin there is a subtle, but very significant, difference between wall time and consensus time. Bitcoin is a decentralized network, which means that each participant has his or her own perspective of time. Events on the network do not occur instantaneously everywhere. Network latency must be factored into the perspective of each node. Eventually everything is synchronized to create a common blockchain. Bitcoin reaches consensus every 10 minutes about the state of the blockchain as it existed in the *past*.

The timestamps set in block headers are set by the miners. There is a certain degree of latitude allowed by the consensus rules to account for differences in clock accuracy between decentralized nodes. However, this creates an unfortunate incentive for miners to lie about the time in a block. For example, if a miner sets their time in the future, they can lower difficulty, allowing them to mine more blocks and claim some of the block subsidy reserved for future miners. If they can set their times in the past for some blocks, they can use the current time for some other blocks, and so again make it look like there's a long time between blocks for the purpose of manipulating difficulty.

To prevent manipulation, Bitcoin has two consensus rules. The first is that no node will accept any block with a time further in the future than two hours. The second is that no node will accept a block with a time less than or equal to the median time of the last 11 blocks, called *median time past* (MTP).

As part of the activation of BIP68 relative timelocks, there was also a change in the way "time" is calculated for timelocks (both absolute and relative) in transactions. Previously, a miner could include any transaction in a block with a timelock equal to or below the time of the block. That incentivized miners to use the latest time they thought was possible (close to two hours in the future) so that more transactions would be eligible for their block.

To remove the incentive to lie and strengthen the security of timelocks, BIP113 was proposed and activated at the same time as the BIPs for relative timelocks. The MTP became the consensus time used for all timelock calculations. By taking the midpoint from approximately two hours in the past, the influence of any one block's timestamp is reduced. By incorporating 11 blocks, no single miner can influence the timestamps in order to gain fees from transactions with a timelock that hasn't yet matured.

MTP changes the implementation of time calculations for lock time, CLTV, sequence, and CSV. The consensus time calculated by MTP is usually about one hour behind wall clock time. If you create timelock transactions, you should account for it when estimating the desired value to encode in lock time, sequence, CLTV, and CSV.

12.8. Successfully Mining the Block

As we saw earlier, Jing's node has constructed a candidate block and prepared it for mining. Jing has several hardware mining rigs with application-specific integrated circuits, where hundreds of thousands of integrated circuits run Bitcoin's double SHA256 algorithm in parallel at incredible speeds. Many of these specialized machines are connected to his mining node over USB or a local area network. Next, the mining node running on Jing's desktop transmits the block header to his mining hardware, which starts testing trillions of variations of the header per second. Because the nonce is only 32 bits, after exhausting all the nonce possibilities (about 4 billion), the mining hardware changes the block header (adjusting the coinbase extra nonce space, versionbits, or timestamp) and resets the nonce counter, testing new combinations.

Almost 11 minutes after starting to mine a particular block, one of the hardware mining machines finds a solution and sends it back to the mining node.

Immediately, Jing's mining node transmits the block to all its peers. They receive, validate, and then propagate the new block. As the block ripples out across the network, each node adds it to its own copy of the blockchain, extending it to a new height. As mining nodes receive and validate the block, they abandon their efforts to find a block at the same height and immediately start computing the next block in the chain, using Jing's block as the "parent." By building on top of Jing's newly discovered block, the other miners are essentially using their mining power to endorse Jing's block and the chain it extends.

In the next section, we'll look at the process each node uses to validate a block and select the most-work chain, creating the consensus that forms the decentralized blockchain.

12.9. Validating a New Block

The third step in Bitcoin's consensus mechanism is independent validation of each new block by every node on the network. As the newly solved block moves across the network, each node performs a series of tests to validate it. The independent validation also ensures that only blocks that follow the consensus rules are incorporated in the blockchain, thus

earning their miners the reward. Blocks that violate the rules are rejected and not only lose their miners the reward, but also waste the effort expended to find a proof-of-work solution, thus incurring upon those miners all of the costs of creating a block but giving them none of the rewards.

When a node receives a new block, it will validate the block by checking it against a long list of criteria that must all be met; otherwise, the block is rejected. These criteria can be seen in the Bitcoin Core client in the functions CheckBlock and CheckBlockHeader and include:

- The block data structure is syntactically valid.
- The block header hash is less than the target (enforces the proof of work).
- The block timestamp is between the MTP and two hours in the future (allowing for time errors).
- The block weight is within acceptable limits.
- The first transaction (and only the first) is a coinbase transaction.
- All transactions within the block are valid using the transaction checklist discussed in Independent Verification of Transactions.

The independent validation of each new block by every node on the network ensures that the miners cannot cheat. In previous sections we saw how miners get to write a transaction that awards them the new bitcoin created within the block and claim the transaction fees. Why don't miners write themselves a transaction for a thousand bitcoins instead of the correct reward? Because every node validates blocks according to the same rules. An invalid coinbase transaction would make the entire block invalid, which would result in the block being rejected and, therefore, that transaction would never become part of the blockchain. The miners have to construct a block, based on the shared rules that all nodes follow, and mine it with a correct solution to the PoW. To do so, they expend a lot of electricity in mining, and if they cheat, all the electricity and effort is wasted. This is why independent validation is a key component of decentralized consensus.

12.10. Assembling and Selecting Chains of Blocks

The final part in Bitcoin's decentralized consensus mechanism is the assembly of blocks into chains and the selection of the chain with the most proof of work.

A *best blockchain* is whichever valid chain of blocks has the most cumulative PoW associated with it. The best chain may also have branches with blocks that are "siblings" to the blocks on the best chain. These blocks are valid but not part of the best chain. They are kept for future reference in case one of those secondary chains later becomes primary. When sibling blocks occur, they're usually the result of an almost simultaneous mining of different blocks at the same height.

When a new block is received, a node will try to add it onto the existing blockchain. The node will look at the block's "previous block hash" field, which is the reference to the block's parent. Then, the node will attempt to find that parent in the existing blockchain. Most of the time, the parent will be the "tip" of the best chain, meaning this new block extends the best chain.

Sometimes the new block does not extend the best chain. In that case, the node will attach the new block's header to a secondary chain and then compare the work of the secondary chain to the previous best chain. If the secondary chain is now the best chain, the node will accordingly *reorganize* its view of confirmed transactions and available UTXOs. If the node is a miner, it will now construct a candidate block extending this new, more-proof-of-work, chain.

By selecting the greatest-cumulative-work valid chain, all nodes eventually achieve network-wide consensus. Temporary discrepancies between chains are resolved eventually as more work is added, extending one of the possible chains.

> **Tip** The blockchain forks described in this section occur naturally as a result of transmission delays in the global network. We will also look at deliberately induced forks later in this chapter.

Forks are almost always resolved within one block. It is possible for an accidental fork to extend to two blocks if both blocks are found almost simultaneously by miners on opposite "sides" of a previous fork. However, the chance of that happening is low.

Bitcoin's block interval of 10 minutes is a design compromise between fast confirmation times and the probability of a fork. A faster block time would make transactions seem to clear faster but lead to more frequent blockchain forks, whereas a slower block time would decrease the number of forks but make settlement seem slower.

Note Which is more secure: a transaction included in one block where the average time between blocks is 10 minutes, or a transaction included in a block with nine blocks built on top of it where the average time between blocks is one minute? The answer is that they're equally secure. A malicious miner wanting to double spend that transaction would need to do an amount of work equal to 10 minutes of the total network hash rate in order to create a chain with equal proof of work.

Shorter times between blocks doesn't result in earlier settlement. Its only advantage is providing weaker guarantees to people who are willing to accept those guarantees. For example, if you're willing to accept three minutes of miners agreeing on the best blockchain as sufficient security, you'd prefer a system with 1-minute blocks, where you could wait for three blocks, over a system with 10-minute blocks. The shorter the time between blocks, the more miner work is wasted on accidental forks (in addition to other problems), so many people prefer Bitcoin's 10-minute blocks over shorter block intervals.

12.11. Mining and the Hash Lottery

Bitcoin mining is an extremely competitive industry. The hashing power has increased exponentially every year of Bitcoin's existence. Some years the growth has reflected a complete change of technology, such as in 2010 and 2011 when many miners switched from using CPU mining to GPU mining and field programmable gate array (FPGA) mining. In 2013 the introduction of ASIC mining led to another giant leap in mining power, by placing the double-SHA256 function directly on silicon chips specialized for the purpose of mining. The first such chips could deliver more mining power in a single box than the entire Bitcoin network in 2010.

At the time of writing, it is believed that there are no more giant leaps left in Bitcoin mining equipment because the industry has reached the forefront of Moore's Law, which stipulates that computing density will double approximately every 18 months. Still, the mining power of the network continues to advance at a rapid pace.

12.11.1. The Extra Nonce Solution

Since 2012, mining has evolved to resolve a fundamental limitation in the structure of the block header. In the early days of Bitcoin, a miner could find a block by iterating through the nonce until the resulting hash was below the target. As difficulty increased, miners often cycled through all 4 billion values of the nonce without finding a block. However, this was easily resolved by updating the block timestamp to account for the elapsed time. Because the timestamp is part of the header, the change would allow miners to iterate through the values of the nonce again with different results. Once mining hardware exceeded 4 GH/sec, however, this approach became increasingly difficult because the

nonce values were exhausted in less than a second. As ASIC mining equipment started exceeding the TH/sec hash rate, the mining software needed more space for nonce values in order to find valid blocks. The timestamp could be stretched a bit, but moving it too far into the future would cause the block to become invalid. A new source of variation was needed in the block header.

One solution that was widely implemented was to use the coinbase transaction as a source of extra nonce values. Because the coinbase script can store between 2 and 100 bytes of data, miners started using that space as extra nonce space, allowing them to explore a much larger range of block header values to find valid blocks. The coinbase transaction is included in the merkle tree, which means that any change in the coinbase script causes the merkle root to change. Eight bytes of extra nonce, plus the 4 bytes of "standard" nonce, allow miners to explore a total 2^{96} (8 followed by 28 zeros) possibilities *per second* without having to modify the timestamp.

Another solution widely used today is to use up to 16 bits of the block header versionbits field for mining, as described in BIP320. If each piece of mining equipment has its own coinbase transaction, this allows an individual piece of mining equipment to perform up to 281 TH/s by only making changes to the block header. This keeps mining equipment and protocols simpler than incrementing the extra nonce in the coinbase transaction every 4 billion hashes, which requires recalculating the entire left flank of the merkle tree up to the root.

12.11.2. Mining Pools

In this highly competitive environment, individual miners working alone (also known as solo miners) don’t stand a chance. The likelihood of them finding a block to offset their electricity and hardware costs is so low that it represents a gamble, like playing the lottery. Even the fastest consumer ASIC mining system cannot keep up with commercial operations that stack tens of thousands of these systems in giant warehouses near powerstations. Many miners now collaborate to form mining pools, pooling their hashing power and sharing the reward among thousands of participants. By participating in a pool, miners get a smaller share of the overall reward, but typically get rewarded every day, reducing uncertainty.

Let’s look at a specific example. Assume a miner has purchased mining hardware with a combined hashing rate of 0.0001% of current the total network hash rate. If the protocol difficulty never changes, that miner will find a new block approximately once every 20 years. That’s a potentially long time to wait to get paid. However, if that miner works together in a mining pool with other miners whose aggregate hash rate is 1% of the total network hash rate, they’ll average more than one block per day. That miner will only receive their portion of the rewards (minus any fees charged by the pool), so they’ll only receive a small amount per day. If they mined every day for 20 years, they’d earn the same amount (not counting pool fees) as if they found an average block on their own. The only fundamental difference is the frequency of the payments they receive.

Mining pools coordinate many hundreds or thousands of miners over specialized pool-mining protocols. The individual miners configure their mining equipment to connect to a pool server, after creating an account with the pool. Their mining hardware remains connected to the pool server while mining, synchronizing their efforts with the other miners. Thus, the pool miners share the effort to mine a block and then share in the rewards.

Successful blocks pay the reward to a pool Bitcoin address rather than to individual miners. The pool server will periodically make payments to the miners' Bitcoin addresses once their share of the rewards has reached a certain threshold. Typically, the pool server charges a percentage fee of the rewards for providing the pool-mining service.

Miners participating in a pool split the work of searching for a solution to a candidate block, earning "shares" for their mining contribution. The mining pool sets a higher target (lower difficulty) for earning a share, typically more than 1,000 times easier than the Bitcoin network's target. When someone in the pool successfully mines a block, the reward is earned by the pool and then shared with all miners in proportion to the number of shares they contributed to the effort.

Many pools are open to any miner, big or small, professional or amateur. A pool will therefore have some participants with a single small mining machine, and others with a garage full of high-end mining hardware. Some will be mining with a few tens of a kilowatt of electricity, others will be running a data center consuming megawatts of power. How does a mining pool measure the individual contributions, so as to fairly distribute the rewards, without the possibility of cheating? The answer is to use Bitcoin's proof-of-work algorithm to measure each pool miner's contribution, but set at a lower difficulty so that even the smallest pool miners win a share frequently enough to make it worthwhile to contribute to the pool. By setting a lower difficulty for earning shares, the pool measures the amount of work done by each miner. Each time a pool miner finds a block header hash that is less than the pool target, they prove they have done the hashing work to find that result. That header ultimately commits to the coinbase transaction and can be used to prove the miner used a coinbase transaction that would have paid the block reward to the pool. Each pool miner is given a slightly different coinbase transaction template so each of them hashes different candidate block headers, preventing duplication of effort.

The work to find shares contributes, in a statistically measurable way, to the overall effort to find a hash lower than the Bitcoin network's target. Thousands of miners trying to find low-value hashes will eventually find one low enough to satisfy the Bitcoin network target.

Let's return to the analogy of a dice game. If the dice players are throwing dice with a goal of throwing less than four (the overall network difficulty), a pool would set an easier target, counting how many times the pool players managed to throw less than eight. When pool players throw less than eight (the pool share target), they earn shares, but they don't win the game because they don't achieve the game target (less than four). The pool players will achieve the easier pool target much more often, earning them shares very regularly, even when they don't achieve the harder target of winning the game. Every now and then, one of the pool players will throw a combined dice throw of less than four and the pool

wins. Then, the earnings can be distributed to the pool players based on the shares they earned. Even though the target of eight-or-less wasn't winning, it was a fair way to measure dice throws for the players, and it occasionally produces a less-than-four throw.

Similarly, a mining pool will set a (higher and easier) pool target that will ensure that an individual pool miner frequently earns shares by finding block header hashes that are less than the pool target. Every now and then, one of these attempts will produce a block header hash that is less than the Bitcoin network target, making it a valid block and the whole pool wins.

Managed pools

Most mining pools are "managed," meaning that there is a company or individual running a pool server. The owner of the pool server is called the *pool operator*, and they charge pool miners a percentage fee of the earnings.

The pool server runs specialized software and a pool-mining protocol that coordinate the activities of the pool miners. The pool server is also connected to one or more full Bitcoin nodes. This allows the pool server to validate blocks and transactions on behalf of the pool miners, relieving them of the burden of running a full node. For some miners, the ability to mine without running a full node is another benefit of joining a managed pool.

Pool miners connect to the pool server using a mining protocol such as Stratum (either version 1 or version 2). Stratum v1 creates block *templates* that contain a template of a candidate block header. The pool server constructs a candidate block by aggregating transactions, adding a coinbase transaction (with extra nonce space), calculating the merkle root, and linking to the previous block hash. The header of the candidate block is then sent to each of the pool miners as a template. Each pool miner then mines using the block template, at a higher (easier) target than the Bitcoin network target, and sends any successful results back to the pool server to earn shares.

Stratum v2 optionally allows individual miners in the pool to choose which transactions appear in their own blocks, which they can select using their own full node.

Peer-to-peer mining pool (P2Pool)

Managed pools using Stratum v1 create the possibility of cheating by the pool operator, who might direct the pool effort to double-spend transactions or invalidate blocks (see Hashrate Attacks). Furthermore, centralized pool servers represent a single point of failure. If the pool server is down or is slowed by a denial-of-service attack, the pool miners cannot mine. In 2011, to resolve these issues of centralization, a new pool mining method was proposed and implemented: P2Pool, a peer-to-peer mining pool without a central operator.

P2Pool works by decentralizing the functions of the pool server, implementing a parallel blockchain-like system called a *share chain*. A share chain is a blockchain running at a lower difficulty than the Bitcoin blockchain. The share chain allows pool miners to

collaborate in a decentralized pool by mining shares on the share chain at a rate of one share block every 30 seconds. Each of the blocks on the share chain records a proportionate share reward for the pool miners who contribute work, carrying the shares forward from the previous share block. When one of the share blocks also achieves the Bitcoin network target, it is propagated and included on the Bitcoin blockchain, rewarding all the pool miners who contributed to all the shares that preceded the winning share block. Essentially, instead of a pool server keeping track of pool miner shares and rewards, the share chain allows all pool miners to keep track of all shares using a decentralized consensus mechanism like Bitcoin's blockchain consensus mechanism.

P2Pool mining is more complex than pool mining because it requires that the pool miners run a dedicated computer with enough disk space, memory, and internet bandwidth to support a Bitcoin full node and the P2Pool node software. P2Pool miners connect their mining hardware to their local P2Pool node, which simulates the functions of a pool server by sending block templates to the mining hardware. On P2Pool, individual pool miners construct their own candidate blocks, aggregating transactions much like solo miners, but then mine collaboratively on the share chain. P2Pool is a hybrid approach that has the advantage of much more granular payouts than solo mining, but without giving too much control to a pool operator like managed pools.

Even though P2Pool reduces the concentration of power by mining pool operators, it is conceivably vulnerable to 51% attacks against the share chain itself. A much broader adoption of P2Pool does not solve the 51% attack problem for Bitcoin itself. Rather, P2Pool makes Bitcoin more robust overall, as part of a diversified mining ecosystem. As of this writing, P2Pool has fallen into disuse, but new protocols such as Stratum v2 can allow individual miners to choose the transactions they include in their blocks.

12.12. Hashrate Attacks

Bitcoin's consensus mechanism is, at least theoretically, vulnerable to attack by miners (or pools) that attempt to use their hashing power to dishonest or destructive ends. As we saw, the consensus mechanism depends on having a majority of the miners acting honestly out of self-interest. However, if a miner or group of miners can achieve a significant share of the mining power, they can attack the consensus mechanism so as to disrupt the security and availability of the Bitcoin network.

It is important to note that hashrate attacks have the greatest effect on future consensus. Confirmed transactions on the best blockchain become more and more immutable as time passes. While in theory, a fork can be achieved at any depth, in practice, the computing power needed to force a very deep fork is immense, making old blocks very hard to change. Hashrate attacks also do not affect the security of the private keys and signing algorithms.

One attack scenario against the consensus mechanism is called the *majority attack* or *51% attack*. In this scenario a group of miners, controlling a majority of the total network's hashing power (such as 51%), collude to attack Bitcoin. With the ability to mine the

majority of the blocks, the attacking miners can cause deliberate "forks" in the blockchain and double-spend transactions or execute denial-of-service attacks against specific transactions or addresses. A fork/double-spend attack is where the attacker causes previously confirmed blocks to be invalidated by forking below them and reconverging on an alternate chain. With sufficient power, an attacker can invalidate six or more blocks in a row, causing transactions that were considered immutable (six confirmations) to be invalidated. Note that a double-spend can only be done on the attacker's own transactions, for which the attacker can produce a valid signature. Double-spending one's own transactions can be profitable if invalidating a transaction allows the attacker to get an irreversible exchange payment or product without paying for it.

Let's examine a practical example of a 51% attack. In the first chapter, we looked at a transaction between Alice and Bob. Bob, the seller, is willing to accept payment without waiting for confirmation (mining in a block) because the risk of a double-spend on a small item is low in comparison to the convenience of rapid customer service. This is similar to the practice of coffee shops that accept credit card payments without a signature for amounts below $25, because the risk of a credit-card chargeback is low while the cost of delaying the transaction to obtain a signature is comparatively larger. In contrast, selling a more expensive item for bitcoins runs the risk of a double-spend attack, where the buyer broadcasts a competing transaction that spends one of the same inputs (UTXOs) and cancels the payment to the merchant. A 51% attack allows attackers to double-spend their own transactions in the new chain, thus undoing the corresponding transaction in the old chain.

In our example, malicious attacker Mallory goes to Carol's gallery and purchases a set of beautiful paintings depicting Satoshi Nakamoto as Prometheus. Carol sells the paintings for $250,000 in bitcoins to Mallory. Instead of waiting for six or more confirmations on the transaction, Carol wraps and hands the paintings to Mallory after only one confirmation. Mallory works with an accomplice, Paul, who operates a large mining pool, and the accomplice launches an attack as soon as Mallory's transaction is included in a block. Paul directs the mining pool to remine the same block height as the block containing Mallory's transaction, replacing Mallory's payment to Carol with a transaction that double-spends the same input as Mallory's payment. The double-spend transaction consumes the same UTXO and pays it back to Mallory's wallet instead of paying it to Carol, essentially allowing Mallory to keep the bitcoins. Paul then directs the mining pool to mine an additional block, so as to make the chain containing the double-spend transaction longer than the original chain (causing a fork below the block containing Mallory's transaction). When the blockchain fork resolves in favor of the new (longer) chain, the double-spent transaction replaces the original payment to Carol. Carol is now missing the three paintings and also has no payment. Throughout all this activity, Paul's mining pool participants might remain blissfully unaware of the double-spend attempt because they mine with automated miners and cannot monitor every transaction or block.

To protect against this kind of attack, a merchant selling large-value items must wait at least six confirmations before giving the product to the buyer. Waiting for more than six confirmations may sometimes be warranted. Alternatively, the merchant should use an

escrow multisignature account, again waiting for several confirmations after the escrow account is funded. The more confirmations elapse, the harder it becomes to invalidate a transaction by reorganizing the blockchain. For high-value items, payment by bitcoin will still be convenient and efficient even if the buyer has to wait 24 hours for delivery, which would correspond to approximately 144 confirmations.

In addition to a double-spend attack, the other scenario for a consensus attack is to deny service to specific participants (specific Bitcoin addresses). An attacker with a majority of the mining power can censor transactions. If they are included in a block mined by another miner, the attacker can deliberately fork and remine that block, again excluding the specific transactions. This type of attack can result in a sustained denial-of-service against a specific address or set of addresses for as long as the attacker controls the majority of the mining power.

Despite its name, the 51% attack scenario doesn’t actually require 51% of the hashing power. In fact, such an attack can be attempted with a smaller percentage of the hashing power. The 51% threshold is simply the level at which such an attack is almost guaranteed to succeed. A hashrate attack is essentially a tug-of-war for the next block, and the "stronger" group is more likely to win. With less hashing power, the probability of success is reduced because other miners control the generation of some blocks with their "honest" mining power. One way to look at it is that the more hashing power an attacker has, the longer the fork he can deliberately create, the more blocks in the recent past he can invalidate, or the more blocks in the future he can control. Security research groups have used statistical modeling to claim that various types of hashrate attacks are possible with as little as 30% of the hashing power.

The centralization of control caused by mining pools has introduced the risk of for-profit attacks by a mining pool operator. The pool operator in a managed pool controls the construction of candidate blocks and also controls which transactions are included. This gives the pool operator the power to exclude transactions or introduce double-spend transactions. If such abuse of power is done in a limited and subtle way, a pool operator could conceivably profit from a hashrate attack without being noticed.

Not all attackers will be motivated by profit, however. One potential attack scenario is where an attacker intends to disrupt the Bitcoin network without the possibility of profiting from such disruption. A malicious attack aimed at crippling Bitcoin would require enormous investment and covert planning but could conceivably be launched by a well-funded, most likely state-sponsored, attacker. Alternatively, a well-funded attacker could attack Bitcoin by simultaneously amassing mining hardware, compromising pool operators, and attacking other pools with denial-of-service. All of these scenarios are theoretically possible.

Undoubtedly, a serious hashrate attack would erode confidence in Bitcoin in the short term, possibly causing a significant price decline. However, the Bitcoin network and software are constantly evolving, so attacks would be met with countermeasures by the Bitcoin community.

12.13. Changing the Consensus Rules

The rules of consensus determine the validity of transactions and blocks. These rules are the basis for collaboration between all Bitcoin nodes and are responsible for the convergence of all local perspectives into a single consistent blockchain across the entire network.

While the consensus rules are invariable in the short term and must be consistent across all nodes, they are not invariable in the long term. In order to evolve and develop the Bitcoin system, the rules can change from time to time to accommodate new features, improvements, or bug fixes. Unlike traditional software development, however, upgrades to a consensus system are much more difficult and require coordination between all participants.

12.13.1. Hard Forks

In Assembling and Selecting Chains of Blocks we looked at how the Bitcoin network may briefly diverge, with two parts of the network following two different branches of the blockchain for a short time. We saw how this process occurs naturally, as part of the normal operation of the network and how the network converges on a common blockchain after one or more blocks are mined.

There is another scenario in which the network may diverge into following two chains: a change in the consensus rules. This type of fork is called a *hard fork*, because after the fork, the network may not converge onto a single chain. Instead, the two chains can evolve independently. Hard forks occur when part of the network is operating under a different set of consensus rules than the rest of the network. This may occur because of a bug or because of a deliberate change in the implementation of the consensus rules.

Hard forks can be used to change the rules of consensus, but they require coordination between all participants in the system. Any nodes that do not upgrade to the new consensus rules are unable to participate in the consensus mechanism and are forced onto a separate chain at the moment of the hard fork. Thus, a change introduced by a hard fork can be thought of as not "forward compatible," in that nonupgraded systems can no longer process blocks because of the new consensus rules.

Let’s examine the mechanics of a hard fork with a specific example.

A blockchain with forks. shows a blockchain with two forks. At block height 4, a one-block fork occurs. This is the type of spontaneous fork we saw in Assembling and Selecting Chains of Blocks. With the mining of block 5, the network converges on one chain and the fork is resolved.

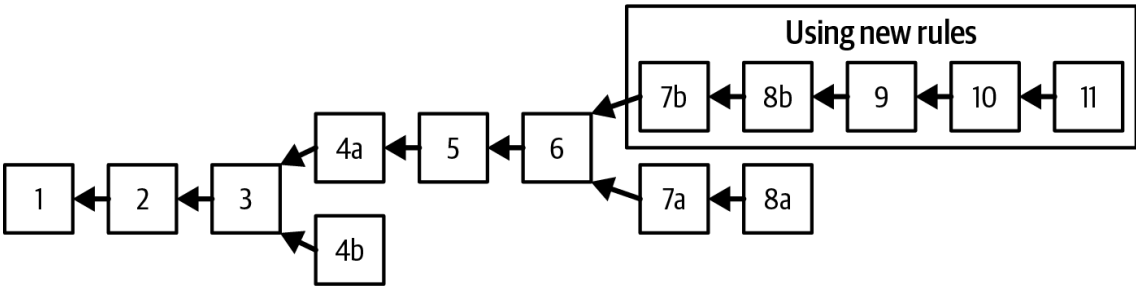


Figure 64. A blockchain with forks.

Later, however, at block height 6, a new implementation of the client is released with a change in the consensus rules. Starting on block height 7, miners running this new implementation will accept a new type of bitcoin; let’s call it a "foocoin." Immediately after, a node running the new implementation creates a transaction that contains a foocoin and a miner with the updated software mines block 7b containing this transaction.

Any node or miner that has not upgraded the software to validate foocoin is now unable to process block 7b. From their perspective, both the transaction that contained a foocoin and block 7b that contained that transaction are invalid because they are evaluating them based upon the old consensus rules. These nodes will reject the transaction and the block and will not propagate them. Any miners that are using the old rules will not accept block 7b and will continue to mine a candidate block whose parent is block 6. In fact, miners using the old rules may not even receive block 7b if all the nodes they are connected to are also obeying the old rules and therefore not propagating the block. Eventually, they will be able to mine block 7a, which is valid under the old rules and does not contain any transactions with foocoins.

The two chains continue to diverge from this point. Miners on the "b" chain will continue to accept and mine transactions containing foocoins, while miners on the "a" chain will continue to ignore these transactions. Even if block 8b does not contain any foocoin transactions, the miners on the "a" chain cannot process it. To them it appears to be an invalid block, as its parent "7b" is not recognized as a valid block.

Hard forks: Software, network, mining, and chain

For software developers, the term "fork" has another meaning, adding confusion to the term "hard fork." In open source software, a fork occurs when a group of developers choose to follow a different software roadmap and start a competing implementation of an open source project. We’ve already discussed two circumstances that will lead to a hard fork: a bug in the consensus rules and a deliberate modification of the consensus rules. In the case of a deliberate change to the consensus rules, a software fork precedes the hard fork. However, for this type of hard fork to occur, a new software implementation of the consensus rules must be developed, adopted, and launched.

Examples of software forks that have attempted to change consensus rules include Bitcoin XT and Bitcoin Classic. However, neither of those programs resulted in a hard fork. While a software fork is a necessary precondition, it is not in itself sufficient for a hard fork to occur. For a hard fork to occur, the competing implementation must be adopted and the new rules activated, by miners, wallets, and intermediary nodes. Conversely, there are numerous alternative implementations of Bitcoin Core, and even software forks, that do not change the consensus rules and barring a bug, can coexist on the network and interoperate without causing a hard fork.

Consensus rules may differ in obvious and explicit ways, in the validation of transactions or blocks. The rules may also differ in more subtle ways, in the implementation of the consensus rules as they apply to Bitcoin scripts or cryptographic primitives such as digital signatures. Finally, the consensus rules may differ in unanticipated ways because of implicit consensus constraints imposed by system limitations or implementation details. An example of the latter was seen in the unanticipated hard fork during the upgrade of Bitcoin Core 0.7 to 0.8, which was caused by a limitation in the Berkeley DB implementation used to store blocks.

Conceptually, we can think of a hard fork as developing in four stages: a software fork, a network fork, a mining fork, and a chain fork. The process begins when an alternative implementation of the client, with modified consensus rules, is created by developers.

When this forked implementation is deployed in the network, a certain percentage of miners, wallet users, and intermediate nodes may adopt and run this implementation. First, the network will fork. Nodes based on the original implementation of the consensus rules will reject any transactions and blocks that are created under the new rules. Furthermore, the nodes following the original consensus rules may disconnect from any nodes that are sending them these invalid transactions and blocks. As a result, the network may partition into two: old nodes will only remain connected to old nodes and new nodes will only be connected to new nodes. A single block based on the new rules will ripple through the network and result in a partition into two networks.

New miners may mine on top of the new block, while old miners will mine a separate chain based on the old rules. The partitioned network will make it so that the miners operating on separate consensus rules won’t likely receive each other’s blocks, as they are connected to two separate networks.

Diverging miners and difficulty

As miners diverge into mining two different chains, the hashing power is split between the chains. The mining power can be split in any proportion between the two chains. The new rules may only be followed by a minority, or by the vast majority of the mining power.

Let’s assume, for example, an 80%–20% split, with the majority of the mining power using the new consensus rules. Let’s also assume that the fork occurs immediately after a retargeting period.

The two chains would each inherit the difficulty from the retargeting period. The new consensus rules would have 80% of the previously available mining power committed to them. From the perspective of this chain, the mining power has suddenly declined by 20% vis-a-vis the previous period. Blocks will be found on average every 12.5 minutes, representing the 20% decline in mining power available to extend this chain. This rate of block issuance will continue (barring any changes in hashing power) until 2,016 blocks are mined, which will take approximately 25,200 minutes (at 12.5 minutes per block), or 17.5 days. After 17.5 days, a retarget will occur and the difficulty will adjust (reduced by 20%) to produce 10-minute blocks again, based on the reduced amount of hashing power in this chain.

The minority chain, mining under the old rules with only 20% of the hashing power, will face a much more difficult task. On this chain, blocks will now be mined every 50 minutes on average. The difficulty will not be adjusted for 2,016 blocks, which will take 100,800 minutes, or approximately 10 weeks to mine. Assuming a fixed capacity per block, this will also result in a reduction of transaction capacity by a factor of 5, as there are fewer blocks per hour available to record transactions.

Contentious hard forks

This is the dawn of the development of software for decentralized consensus. Just as other innovations in development changed both the methods and products of software and created new methodologies, new tools, and new communities in its wake, consensus software development also represents a new frontier in computer science. Out of the debates, experiments, and tribulations of Bitcoin development, we will see new development tools, practices, methodologies, and communities emerge.

Hard forks are seen as risky because they force a minority to either upgrade or remain on a minority chain. The risk of splitting the entire system into two competing systems is seen by many as an unacceptable risk. As a result, many developers are reluctant to use the hard fork mechanism to implement upgrades to the consensus rules, unless there is near-unanimous support from the entire network. Any hard fork proposals that do not have near-unanimous support are considered too contentious to attempt without risking a partition of the system.

Already we have seen the emergence of new methodologies to address the risks of hard forks. In the next section, we will look at soft forks and the methods for signaling and activation of consensus modifications.

12.13.2. Soft Forks

Not all consensus rule changes cause a hard fork. Only consensus changes that are forward-incompatible cause a fork. If the change is implemented in such a way that an unmodified client still sees the transaction or block as valid under the previous rules, the change can happen without a fork.

The term *soft fork* was introduced to distinguish this upgrade method from a "hard fork." In practice, a soft fork is not a fork at all. A soft fork is a forward-compatible change to the consensus rules that allows unupgraded clients to continue to operate in consensus with the new rules.

One aspect of soft forks that is not immediately obvious is that soft fork upgrades can only be used to constrain the consensus rules, not to expand them. In order to be forward compatible, transactions and blocks created under the new rules must be valid under the old rules too, but not vice versa. The new rules can only limit what is valid; otherwise, they will trigger a hard fork when rejected under the old rules.

Soft forks can be implemented in a number of ways—the term does not specify a particular method, but rather a set of methods that all have one thing in common: they don’t require all nodes to upgrade or force nonupgraded nodes out of consensus.

Two soft forks have been implemented in Bitcoin, based on the re-interpretation of NOP opcodes. Bitcoin Script had 10 opcodes reserved for future use, NOP1 through NOP10. Under the consensus rules, the presence of these opcodes in a script is interpreted as a null-potent operator, meaning they have no effect. Execution continues after the NOP opcode as if it wasn’t there.

A soft fork therefore can modify the semantics of a NOP code to give it new meaning. For example, BIP65 (CHECKLOCKTIMEVERIFY) reinterpreted the NOP2 opcode. Clients implementing BIP65 interpret NOP2 as OP_CHECKLOCKTIMEVERIFY and impose an absolute lock time consensus rule on UTXOs that contain this opcode in their locking scripts. This change is a soft fork because a transaction that is valid under BIP65 is also valid on any client that is not implementing (ignorant of) BIP65. To the old clients, the script contains an NOP code, which is ignored.

Criticisms of soft forks

Soft forks based on the NOP opcodes are relatively uncontroversial. The NOP opcodes were placed in Bitcoin Script with the explicit goal of allowing non-disruptive upgrades.

However, many developers are concerned that other methods of soft fork upgrades make unacceptable trade-offs. Common criticisms of soft fork changes include:

Technical debt
: Because soft forks are more technically complex than a hard fork upgrade, they introduce *technical debt*, a term that refers to increasing the future cost of code maintenance because of design trade-offs made in the past. Code complexity in turn increases the likelihood of bugs and security vulnerabilities.

Validation relaxation
: Unmodified clients see transactions as valid without evaluating the modified consensus rules. In effect, the unmodified clients are not validating using the full range of consensus rules, as they are blind to the new rules. This applies to NOP-based upgrades, as well as other soft fork upgrades.

Irreversible upgrades
: Because soft forks create transactions with additional consensus constraints, they become irreversible upgrades in practice. If a soft fork upgrade were to be reversed after being activated, any transactions created under the new rules could result in a loss of funds under the old rules. For example, if a CLTV transaction is evaluated under the old rules, there is no timelock constraint and it can be spent at any time. Therefore, critics contend that a failed soft fork that had to be reversed because of a bug would almost certainly lead to loss of funds.

Soft fork signaling with block version

Since soft forks allow unmodified clients to continue to operate within consensus, one mechanism for "activating" a soft fork is through miners signaling that they are ready and willing to enforce the new consensus rules. If all miners enforce the new rules, there's no risk of unmodified nodes accepting a block that upgraded nodes would reject. This mechanism was introduced by BIP34.

BIP34: Signaling and activation

BIP34 used the block version field to allow miners to signal readiness for a specific consensus rule change. Prior to BIP34, the block version was set to "1" by *convention* not enforced by *consensus*.

BIP34 defined a consensus rule change that required the coinbase field (input) of the coinbase transaction to contain the block height. Prior to BIP34, the coinbase could contain any arbitrary data the miners chose to include. After activation of BIP34, valid blocks had to contain a specific block height at the beginning of the coinbase and be identified with a block version number greater than or equal to "2."

To signal their readiness to enforce the rules of BIP34, miners set the block version to "2," instead of "1." This did not immediately make version "1" blocks invalid. Once activated, version "1" blocks would become invalid and all version "2" blocks would be required to contain the block height in the coinbase to be valid.

BIP34 defined a two-step activation mechanism based on a rolling window of 1,000 blocks. A miner would signal their individual readiness for BIP34 by constructing blocks with "2" as the version number. Strictly speaking, these blocks did not yet have to comply with the new consensus rule of including the block height in the coinbase transaction because the consensus rule had not yet been activated. The consensus rules activated in two steps:

- If 75% (750 of the most recent 1,000 blocks) are marked with version "2," then version "2" blocks must contain block height in the coinbase transaction or they are rejected as invalid. Version "1" blocks are still accepted by the network and do not need to contain block height. The old and new consensus rules coexist during this period.

- When 95% (950 of the most recent 1,000 blocks) are version "2," version "1" blocks are no longer considered valid. Version "2" blocks are valid only if they contain the block height in the coinbase (as per the previous threshold). Thereafter, all blocks must comply with the new consensus rules, and all valid blocks must contain block height in the coinbase transaction.

After successful signaling and activation under the BIP34 rules, this mechanism was used twice more to activate soft forks:

- BIP66 Strict DER Encoding of Signatures was activated by BIP34 style signaling with a block version "3."
- BIP65 CHECKLOCKTIMEVERIFY was activated by BIP34 style signaling with a block version "4."

After the activation of BIP65, the signaling and activation mechanism of BIP34 was retired and replaced with the BIP9 signaling mechanism described next.

BIP9: Signaling and activation

The mechanism used by BIP34, BIP66, and BIP65 was successful in activating three soft forks. However, it was replaced because it had several limitations:

- By using the integer value of the block version, only one soft fork could be activated at a time, so it required coordination between soft fork proposals and agreement on their prioritization and sequencing.
- Furthermore, because the block version was incremented, the mechanism didn’t offer a straightforward way to reject a change and then propose a different one. If old clients were still running, they could mistake signaling for a new change as signaling for the previously rejected change.
- Each new change irrevocably reduced the available block versions for future changes.

BIP9 was proposed to overcome these challenges and improve the rate and ease of implementing future changes.

BIP9 interprets the block version as a bit field instead of an integer. Because the block version was originally used as an integer for versions 1 through 4, only 29 bits remain available to be used as a bit field. This leaves 29 bits that can be used to independently and simultaneously signal readiness on 29 different proposals.

BIP9 also sets a maximum time for signaling and activation. This way miners don’t need to signal forever. If a proposal is not activated within the TIMEOUT period (defined in the proposal), the proposal is considered rejected. The proposal may be resubmitted for signaling with a different bit, renewing the activation period.

Furthermore, after the TIMEOUT has passed and a feature has been activated or rejected, the signaling bit can be reused for another feature without confusion. Therefore, up to 29 changes can be signaled in parallel. After TIMEOUT, the bits can be "recycled" to propose new changes.

> **Note** While signaling bits can be reused or recycled, as long as the voting period does not overlap, the authors of BIP9 recommend that bits are reused only when necessary; unexpected behavior could occur due to bugs in older software. In short, we should not expect to see reuse until all 29 bits have been used once.

Proposed changes are identified by a data structure that contains the following fields:

name
: A short description used to distinguish between proposals. Most often the BIP describing the proposal, as "bipN," where N is the BIP number.

bit
: 0 through 28, the bit in the block version that miners use to signal approval for this proposal.

starttime
: The time (based on MTP) that signaling starts after which the bit’s value is interpreted as signaling readiness for the proposal.

endtime
: The time (based on MTP) after which the change is considered rejected if it has not reached the activation threshold.

Unlike BIP34, BIP9 counts activation signaling in whole intervals based on the difficulty retarget period of 2,016 blocks. For every retarget period, if the sum of blocks signaling for a proposal exceeds 95% (1,916 of 2,016), the proposal will be activated one retarget period later.

BIP9 offers a proposal state diagram to illustrate the various stages and transitions for a proposal, as shown in BIP9 state transition diagram..

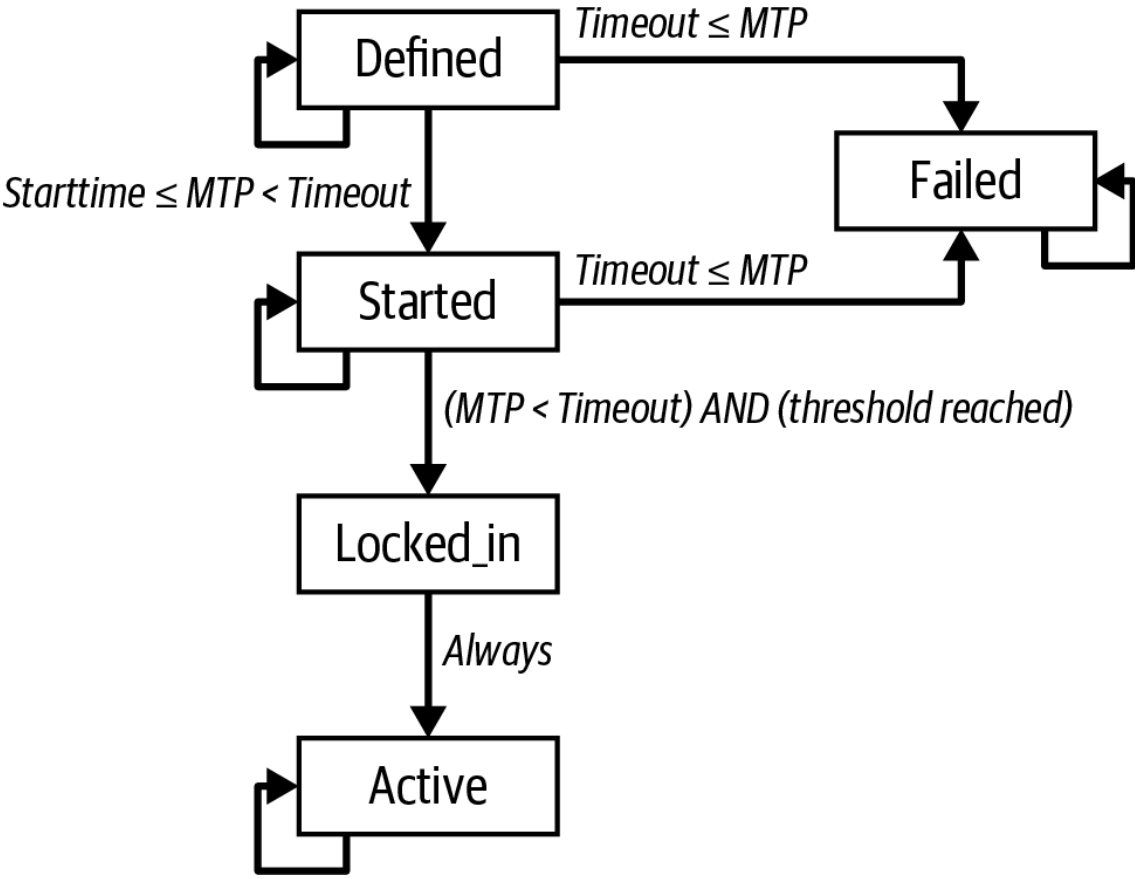


Figure 65. BIP9 state transition diagram.

Proposals start in the DEFINED state once their parameters are known (defined) in the Bitcoin software. For blocks with MTP after the start time, the proposal state transitions to STARTED. If the voting threshold is exceeded within a retarget period and the timeout has not been exceeded, the proposal state transitions to LOCKED_IN. One retarget period later, the proposal becomes ACTIVE. Proposals remain in the ACTIVE state perpetually once they reach that state. If the timeout elapses before the voting threshold has been reached, the proposal state changes to FAILED, indicating a rejected proposal. FAILED proposals remain in that state perpetually.

BIP9 was first implemented for the activation of CHECKSEQUENCEVERIFY and associated BIPs (68, 112, 113). The proposal named "csv" was activated successfully in July of 2016.

The standard is defined in BIP9 (Version bits with timeout and delay).

BIP8: Mandatory lock-in with early activation

After BIP9 was successfully used for the CSV-related soft fork, the next implementation of a soft fork consensus change also attempted to use it for miner-enforced activation. However, some people opposed that soft fork proposal, called *segwit*, and very few miners signaled readiness to enforce segwit for several months.

It was later discovered that some miners, especially miners associated with the dissenters, may have been using hardware that gave them a hidden advantage over other miners using a feature called *covert ASICBoost*. Unintentionally, segwit interfered with the ability to use covert ASICBoost—if segwit was activated, the miners using it would lose their hidden advantage.

After the community discovered this conflict of interest, some users decided they wanted to exercise their power not to accept blocks from miners unless those blocks followed certain rules. The rules the users ultimately wanted were the new rules added by segwit, but the users wanted to multiply their efforts by taking advantage of the large numbers of nodes that planned to enforce the rules of segwit if enough miners signaled readiness for it. A pseudonymous developer proposed BIP148, which required any node implementing it to reject all blocks that didn't signal for segwit starting on a certain date and continuing until segwit activated.

Although only a limited number of users actually ran BIP148 code, many other users seemed to agree with the sentiment and may have been prepared to commit to BIP148. A few days before BIP148 was due to go into effect, almost all miners began signaling their readiness to enforce segwit's rules. Segwit reached its lock-in threshold about two weeks later and activated about two weeks after that.

Many users came to believe that it was a flaw in BIP9 that miners could prevent an activation attempt from being successful by not signaling for a year. They wanted a mechanism that would ensure a soft fork was activated by a particular block height but which also allowed miners to signal they were ready to lock it in earlier.

The method developed for that was BIP8, which is similar to BIP9 except that it defines a MUST_SIGNAL period where miners must signal that they are ready to enforce the soft fork proposal.

Software was published that used BIP8 to attempt to activate the taproot proposal in 2021, and there was evidence that at least a small number of users ran that software. Some of those users also claim that their willingness to use BIP8 to force miners to activate taproot was the reason it did eventually activate. They claim that if taproot had not been activated quickly, other users would have also begun running BIP8. Unfortunately, there's no way to prove what would have happened, and so we can't say for sure how much BIP8 contributed to the activation of taproot.

Speedy trial: Fail fast or succeed eventually

Although BIP9 by itself did not seem to result in the activation of segwit despite widespread support for the proposal, it was unclear to many protocol developers that BIP9 was itself a failure. As mentioned, the failure of miners to initially signal support for segwit may have been largely the result of a one-time conflict of interest that wouldn't apply in the future. To some, it seemed worth trying BIP9 again. Others disagreed and wanted to use BIP8.

After months of discussions between those who were the most interested in specific activation ideas, a compromise was suggested in order to activate taproot. A modified version of BIP9 was suggested that would only give miners a very short amount of time to signal their intention to enforce taproot rules. If signaling was unsuccessful, a different activation mechanism could be used (or, potentially, the idea could be abandoned). If signaling was successful, enforcement would begin about six months later at a specified block height. This mechanism was named *speedy trial* by one of the people who helped promote it.

Speedy trial activation was tried, miners quickly signaled their willingness to enforce the rules of taproot, and taproot was successfully activated about six months later. To proponents of speedy trial, it was a clear success. Others were still disappointed that BIP8 wasn't used.

It's not clear whether or not speedy trial will be used again for a future attempt to activate a soft fork.

12.13.3. Consensus Software Development

Consensus software continues to evolve, and there is much discussion on the various mechanisms for changing the consensus rules. By its very nature, Bitcoin sets a very high bar on coordination and consensus for changes. As a decentralized system, it has no "authority" that can impose its will on the participants of the network. Power is diffused between multiple constituencies such as miners, protocol developers, wallet developers, exchanges, merchants, and end users. Decisions cannot be made unilaterally by any of these constituencies. For example, while miners can censor transactions by simple majority (51%), they are constrained by the consent of the other constituencies. If they act unilaterally, the rest of the participants may refuse to accept their blocks, keeping the economic activity on a minority chain. Without economic activity (transactions, merchants, wallets, exchanges), the miners will be mining a worthless currency with empty blocks. This diffusion of power means that all the participants must coordinate, or no changes can be made. Status quo is the stable state of this system with only a few changes possible if there is strong consensus by a very large majority. The 95% threshold for soft forks is reflective of this reality.

It is important to recognize that there is no perfect solution for consensus development. Both hard forks and soft forks involve trade-offs. For some types of changes, soft forks may be a better choice; for others, hard forks may be a better choice. There is no perfect choice; both carry risks. The one constant characteristic of consensus software development is that change is difficult and consensus forces compromise.

Some see this as a weakness of consensus systems. In time, you may come to see it as the system's greatest strength.

At this point in the book, we've finished talking about the Bitcoin system itself. What's left are software, tools, and other protocols built on top of Bitcoin.

13. Bitcoin Security

Securing your bitcoins is challenging because bitcoins are are not like a balance in a bank account. Your bitcoins are very much like digital cash or gold. You've probably heard the expression, "Possession is nine-tenths of the law." Well, in Bitcoin, possession is ten-tenths of the law. Possession of the keys to spend certain bitcoins is equivalent to possession of cash or a chunk of precious metal. You can lose it, misplace it, have it stolen, or accidentally give the wrong amount to someone. In every one of these cases, users have no recourse within the protocol, just as if they dropped cash on a public sidewalk.

However, the Bitcoin system has capabilities that cash, gold, and bank accounts do not. A Bitcoin wallet, containing your keys, can be backed up like any file. It can be stored in multiple copies, even printed on paper for hard-copy backup. You can't "back up" cash, gold, or bank accounts. Bitcoin is different enough from anything that has come before that we need to think about securing our bitcoins in a novel way too.

13.1. Security Principles

The core principle in Bitcoin is decentralization and it has important implications for security. A centralized model, such as a traditional bank or payment network, depends on access control and vetting to keep bad actors out of the system. By comparison, a decentralized system like Bitcoin pushes the responsibility and control to the users. Because the security of the network is based on independent verification, the network can be open and no encryption is required for Bitcoin traffic (although encryption can still be useful).

On a traditional payment network, such as a credit card system, the payment is open-ended because it contains the user's private identifier (the credit card number). After the initial charge, anyone with access to the identifier can "pull" funds and charge the owner again and again. Thus, the payment network has to be secured end-to-end with encryption and must ensure that no eavesdroppers or intermediaries can compromise the payment traffic in transit or when it is stored (at rest). If a bad actor gains access to the system, he can compromise current transactions *and* payment tokens that can be used to create new transactions. Worse, when customer data is compromised, the customers are exposed to identity theft and must take action to prevent fraudulent use of the compromised accounts.

Bitcoin is dramatically different. A Bitcoin transaction authorizes only a specific value to a specific recipient and cannot be forged. It does not reveal any private information, such as the identities of the parties, and cannot be used to authorize additional payments. Therefore, a Bitcoin payment network does not need to be encrypted or protected from eavesdropping. In fact, you can broadcast Bitcoin transactions over an open public channel, such as unsecured WiFi or Bluetooth, with no loss of security.

Bitcoin’s decentralized security model puts a lot of power in the hands of the users. With that power comes responsibility for maintaining the secrecy of their keys. For most users that is not easy to do, especially on general-purpose computing devices such as internet-connected smartphones or laptops. Although Bitcoin’s decentralized model prevents the type of mass compromise seen with credit cards, many users are not able to adequately secure their keys and get hacked, one by one.

13.1.1. Developing Bitcoin Systems Securely

A critical principle for Bitcoin developers is decentralization. Most developers will be familiar with centralized security models and might be tempted to apply these models to their Bitcoin applications, with disastrous results.

Bitcoin’s security relies on decentralized control over keys and on independent transaction validation by users. If you want to leverage Bitcoin’s security, you need to ensure that you remain within the Bitcoin security model. In simple terms: don’t take control of keys away from users and don’t outsource validation.

For example, many early Bitcoin exchanges concentrated all user funds in a single "hot" wallet with keys stored on a single server. Such a design removes control from users and centralizes control over keys in a single system. Many such systems have been hacked, with disastrous consequences for their customers.

Unless you are prepared to invest heavily in operational security, multiple layers of access control, and audits (as the traditional banks do), you should think very carefully before taking funds outside of Bitcoin’s decentralized security context. Even if you have the funds and discipline to implement a robust security model, such a design merely replicates the fragile model of traditional financial networks, plagued by identity theft, corruption, and embezzlement. To take advantage of Bitcoin’s unique decentralized security model, you have to avoid the temptation of centralized architectures that might feel familiar but ultimately subvert Bitcoin’s security.

13.1.2. The Root of Trust

Traditional security architecture is based upon a concept called the *root of trust*, which is a trusted core used as the foundation for the security of the overall system or application. Security architecture is developed around the root of trust as a series of concentric circles, like layers in an onion, extending trust outward from the center. Each layer builds upon the more-trusted inner layer using access controls, digital signatures, encryption, and other security primitives. As software systems become more complex, they are more likely to contain bugs, which make them vulnerable to security compromise. As a result, the more complex a software system becomes, the harder it is to secure. The root of trust concept ensures that most of the trust is placed within the least complex part of the system, and therefore the least vulnerable parts of the system, while more complex software is layered around it. This security architecture is repeated at different scales, first establishing a root

of trust within the hardware of a single system, then extending that root of trust through the operating system to higher-level system services, and finally across many servers layered in concentric circles of diminishing trust.

Bitcoin security architecture is different. In Bitcoin, the consensus system creates a trusted blockchain that is completely decentralized. A correctly validated blockchain uses the genesis block as the root of trust, building a chain of trust up to the current block. Bitcoin systems can and should use the blockchain as their root of trust. When designing a complex Bitcoin application that consists of services on many different systems, you should carefully examine the security architecture in order to ascertain where trust is being placed. Ultimately, the only thing that should be explicitly trusted is a fully validated blockchain. If your application explicitly or implicitly vests trust in anything but the blockchain, that should be a source of concern because it introduces vulnerability. A good method to evaluate the security architecture of your application is to consider each individual component and evaluate a hypothetical scenario where that component is completely compromised and under the control of a malicious actor. Take each component of your application, in turn, and assess the impacts on the overall security if that component is compromised. If your application is no longer secure when components are compromised, that shows you have misplaced trust in those components. A Bitcoin application without vulnerabilities should be vulnerable only to a compromise of the Bitcoin consensus mechanism, meaning that its root of trust is based on the strongest part of the Bitcoin security architecture.

The numerous examples of hacked Bitcoin exchanges serve to underscore this point because their security architecture and design fails even under the most casual scrutiny. These centralized implementations had invested trust explicitly in numerous components outside the Bitcoin blockchain, such as hot wallets, centralized databases, vulnerable encryption keys, and similar schemes.

13.2. User Security Best Practices

Humans have used physical security controls for thousands of years. By comparison, our experience with digital security is less than 50 years old. Modern general-purpose operating systems are not very secure and not particularly suited to storing digital money. Our computers are constantly exposed to external threats via always-on internet connections. They run thousands of software components from hundreds of authors, often with unconstrained access to the user’s files. A single piece of rogue software, among the many thousands installed on your computer, can compromise your keyboard and files, stealing any bitcoins stored in wallet applications. The level of computer maintenance required to keep a computer virus-free and trojan-free is beyond the skill level of all but a tiny minority of computer users.

Despite decades of research and advancements in information security, digital assets are still woefully vulnerable to a determined adversary. Even the most highly protected and restricted systems, in financial services companies, intelligence agencies, and defense

contractors, are frequently breached. Bitcoin creates digital assets that have intrinsic value and can be stolen and diverted to new owners instantly and irrevocably. This creates a massive incentive for hackers. Until now, hackers had to convert identity information or account tokens—such as credit cards and bank accounts—into value after compromising them. Despite the difficulty of fencing and laundering financial information, we have seen ever-escalating thefts. Bitcoin escalates this problem because it doesn't need to be fenced or laundered; bitcoins are valuable by themselves.

Bitcoin also creates the incentives to improve computer security. Whereas previously the risk of computer compromise was vague and indirect, Bitcoin makes these risks clear and obvious. Holding bitcoins on a computer serves to focus the user's mind on the need for improved computer security. As a direct result of the proliferation and increased adoption of Bitcoin and other digital currencies, we have seen an escalation in both hacking techniques and security solutions. In simple terms, hackers now have a very juicy target and users have a clear incentive to defend themselves.

Over the past three years, as a direct result of Bitcoin adoption, we have seen tremendous innovation in the realm of information security in the form of hardware encryption, key storage and hardware signing devices, multisignature technology, and digital escrow. In the following sections we will examine various best practices for practical user security.

13.2.1. Physical Bitcoin Storage

Because most users are far more comfortable with physical security than information security, a very effective method for protecting bitcoins is to convert them into physical form. Bitcoin keys, and the seeds used to create them, are nothing more than long numbers. This means that they can be stored in a physical form, such as printed on paper or etched on a metal plate. Securing the keys then becomes as simple as physically securing a printed copy of the key seed. A seed that is printed on paper is called a "paper backup," and many wallets can create them. Keeping bitcoins offline is called *cold storage* and it is one of the most effective security techniques. A cold storage system is one where the keys are generated on an offline system (one never connected to the internet) and stored offline either on paper or on digital media, such as a USB memory stick.

13.2.2. Hardware Signing Devices

In the long term, Bitcoin security may increasingly take the form of tamper-proof hardware signing devices. Unlike a smartphone or desktop computer, a Bitcoin hardware signing device only needs to hold keys and use them to generate signatures. Without general-purpose software to compromise and with limited interfaces, hardware signing devices can deliver strong security to nonexpert users. Hardware signing devices may become the predominant method of storing bitcoins.

13.2.3. Ensuring Your Access

Although most users are rightly concerned about theft of their bitcoins, there is an even bigger risk. Data files get lost all the time. If they contain Bitcoin keys, the loss is much more painful. In the effort to secure their Bitcoin wallets, users must be very careful not to go too far and end up losing their bitcoins. In July 2011, a well-known Bitcoin awareness and education project lost almost 7,000 bitcoin. In their effort to prevent theft, the owners had implemented a complex series of encrypted backups. In the end they accidentally lost the encryption keys, making the backups worthless and losing a fortune. Like hiding money by burying it in the desert, if you secure your bitcoins too well you might not be able to find them again.

> **Warning** To spend bitcoins, you may need to back up more than just your private keys or the BIP32 seed used to derive them. This is especially the case when multisignatures or complex scripts are being used. Most output scripts commit to the actual conditions that must be fulfilled to spend the bitcoins in that output, and it's not possible to fulfill that commitment unless your wallet software can reveal those conditions to the network. Wallet recovery codes must include this information. For more details, see Wallet Recovery.

13.2.4. Diversifying Risk

Would you carry your entire net worth in cash in your wallet? Most people would consider that reckless, yet Bitcoin users often keep all their bitcoins using a single wallet application. Instead, users should spread the risk among multiple and diverse Bitcoin applications. Prudent users will keep only a small fraction, perhaps less than 5%, of their bitcoins in an online or mobile wallet as "pocket change." The rest should be split between a few different storage mechanisms, such as a desktop wallet and offline (cold storage).

13.2.5. Multisig and Governance

Whenever a company or individual stores large amounts of bitcoins, they should consider using a multisignature Bitcoin address. Multisignature addresses secure funds by requiring more than one signature to make a payment. The signing keys should be stored in a number of different locations and under the control of different people. In a corporate environment, for example, the keys should be generated independently and held by several company executives to ensure that no single person can compromise the funds. Multisignature addresses can also offer redundancy, where a single person holds several keys that are stored in different locations.

13.2.6. Survivability

One important security consideration that is often overlooked is availability, especially in the context of incapacity or death of the key holder. Bitcoin users are told to use complex passwords and keep their keys secure and private, not sharing them with anyone. Unfortunately, that practice makes it almost impossible for the user’s family to recover any funds if the user is not available to unlock them. In most cases, in fact, the families of Bitcoin users might be completely unaware of the existence of the bitcoin funds.

If you have a lot of bitcoins, you should consider sharing access details with a trusted relative or lawyer. A more complex survivability scheme can be set up with multisignature access and estate planning through a lawyer specialized as a "digital asset executor."

Bitcoin is a complex new technology that is still being explored by developers. Over time we will develop better security tools and practices that are easier to use by nonexperts. For now, Bitcoin users can use many of the tips discussed here to enjoy a secure and trouble-free Bitcoin experience.

14. Second-Layer Applications

Let’s now build on our understanding of the primary Bitcoin system (the *first layer*) by looking at it as a platform for other applications, or *second layers*. In this chapter we will look at the features offered by Bitcoin as an application platform. We will consider the application building *primitives*, which form the building blocks of any blockchain application. We will look at several important applications that use these primitives, such as client-side validation, payment channels, and routed payment channels (Lightning Network).

14.1. Building Blocks (Primitives)

When operating correctly and over the long term, the Bitcoin system offers certain guarantees, which can be used as building blocks to create applications. These include:

No double-spend
: The most fundamental guarantee of Bitcoin’s decentralized consensus algorithm ensures that no UTXO can be spent twice in the same valid chain of blocks.

Immutability
: Once a transaction is recorded in the blockchain and sufficient work has been added with subsequent blocks, the transaction’s data becomes practically immutable. Immutability is underwritten by energy, as rewriting the blockchain requires the expenditure of energy to produce PoW. The energy required and therefore the degree of immutability increases with the amount of work committed on top of the block containing a transaction.

Neutrality
: The decentralized Bitcoin network propagates valid transactions regardless of the origin of those transactions. This means that anyone can create a valid transaction with sufficient fees and trust they will be able to transmit that transaction and have it included in the blockchain at any time.

Secure timestamping
: The consensus rules reject any block whose timestamp is too far in the future and attempt to prevent blocks with timestamps too far in the past. This ensures that timestamps on blocks can be trusted to a certain degree. The timestamp on a block implies an unspent-before reference for the inputs of all included transactions.

Authorization
: Digital signatures, validated in a decentralized network, offer authorization guarantees. Scripts that contain a requirement for a digital signature cannot be executed without authorization by the holder of the private key implied in the script.

Auditability
: All transactions are public and can be audited. All transactions and blocks can be linked back in an unbroken chain to the genesis block.

Accounting
: In any transaction (except the coinbase transaction) the value of inputs is equal to the value of outputs plus fees. It is not possible to create or destroy bitcoin value in a transaction. The outputs cannot exceed the inputs.

Nonexpiration
: A valid transaction does not expire. If it is valid today, it will be valid in the near future, as long as the inputs remain unspent and the consensus rules do not change.

Integrity
: The outputs of a Bitcoin transaction signed with SIGHASH_ALL or parts of a transaction signed by another SIGHASH type cannot be modified without invalidating the signature, thus invalidating the transaction itself.

Transaction atomicity
: Bitcoin transactions are atomic. They are either valid and confirmed (mined) or not. Partial transactions cannot be mined, and there is no interim state for a transaction. At any point in time a transaction is either mined or not.

Discrete (indivisible) units of value
: Transaction outputs are discrete and indivisible units of value. They can either be spent or unspent, in full. They cannot be divided or partially spent.

Quorum of control
: Multisignature constraints in scripts impose a quorum of authorization, predefined in the multisignature scheme. The requirement is enforced by the consensus rules.

Timelock/aging
: Any script clause containing a relative or absolute timelock can only be executed after its age exceeds the time specified.

Replication
: The decentralized storage of the blockchain ensures that when a transaction is mined, after sufficient confirmations, it is replicated across the network and becomes durable and resilient to power loss, data loss, etc.

Forgery protection
: A transaction can only spend existing, validated outputs. It is not possible to create or counterfeit value.

Consistency
: In the absence of miner partitions, blocks that are recorded in the blockchain are subject to reorganization or disagreement with exponentially decreasing likelihood, based on the depth at which they are recorded. Once deeply recorded, the computation and energy required to change makes change practically infeasible.

Recording external state
: A transaction can commit to a data value, via OP_RETURN or pay to contract, representing a state transition in an external state machine.

Predictable issuance
: Less than 21 million bitcoin will be issued at a predictable rate.

The list of building blocks is not complete, and more are added with each new feature introduced into Bitcoin.

14.2. Applications from Building Blocks

The building blocks offered by Bitcoin are elements of a trust platform that can be used to compose applications. Here are some examples of applications that exist today and the building blocks they use:

Proof-of-Existence (Digital Notary)
: Immutability + Timestamp + Durability. A transaction on the blockchain can commit to a value, proving that a piece of data existed at the time it was recorded (Timestamp). The commitment cannot be modified ex-post-facto (Immutability), and the proof will be stored permanently (Durability).

Kickstarter (Lighthouse)
: Consistency + Atomicity + Integrity. If you sign one input and the output (Integrity) of a fundraiser transaction, others can contribute to the fundraiser but it cannot be spent (Atomicity) until the goal (output amount) is funded (Consistency).

Payment Channels
: Quorum of Control + Timelock + No Double Spend + Nonexpiration + Censorship Resistance + Authorization. A multisig 2-of-2 (Quorum) with a timelock (Timelock) used as the "settlement" transaction of a payment channel can be held (Nonexpiration) and spent at any time (Censorship Resistance) by either party (Authorization). The two parties can then create commitment transactions that supersede (No Double-Spend) the settlement on a shorter timelock (Timelock).

14.3. Colored Coins

The first blockchain application we will discuss is *colored coins*.

Colored coins refers to a set of similar technologies that use Bitcoin transactions to record the creation, ownership, and transfer of extrinsic assets other than bitcoin. By "extrinsic" we mean assets that are not stored directly on the Bitcoin blockchain, as opposed to bitcoin itself, which is an asset intrinsic to the blockchain.

Colored coins are used to track digital assets as well as physical assets held by third parties and traded through certificates of ownership associated with colored coins. Digital asset colored coins can represent intangible assets such as a stock certificate, license, virtual property (game items), or most any form of licensed intellectual property (trademarks, copyrights, etc.). Tangible asset colored coins can represent certificates of ownership of commodities (gold, silver, oil), land titles, automobiles, boats, aircraft, etc.

The term derives from the idea of "coloring" or marking a nominal amount of bitcoin, for example, a single satoshi, to represent something other than the bitcoin amount itself. As an analogy, consider stamping a $1 note with a message saying, "this is a stock certificate of ACME" or "this note can be redeemed for 1 oz of silver" and then trading the $1 note as a certificate of ownership of this other asset. The first implementation of colored coins, named *Enhanced Padded-Order-Based Coloring* or *EPOBC*, assigned extrinsic assets to a 1-satoshi output. In this way, it was a true "colored coin," as each asset was added as an attribute (color) of a single satoshi.

More recent implementations of colored coins use other mechanisms to attach metadata with a transaction, in conjunction with external data stores that associate the metadata to specific assets. The three main mechanisms used as of this writing are single-use seals, pay to contract, and client-side validation.

14.3.1. Single-Use Seals

Single-use seals originate in physical security. Someone shipping an item through a third party needs a way to detect tampering, so they secure their package with a special mechanism that will become clearly damaged if the package is opened. If the package arrives with the seal intact, the sender and receiver can be confident that the package wasn’t opened in transit.

In the context of colored coins, single-use seals refer to a data structure than can only be associated with another data structure once. In Bitcoin, this definition is fulfilled by unspent transaction outputs (UTXOs). A UTXO can only be spent once within a valid blockchain, and the process of spending them associates them with the data in the spending transaction.

This provides part of the basis for the modern transfer for colored coins. One or more colored coins are received to a UTXO. When that UTXO is spent, the spending transaction must describe how the colored coins are to be spent. That brings us to pay to contract (P2C).

14.3.2. Pay to Contract (P2C)

We previously learned about P2C in Pay to Contract (P2C), where it became part of the basis for the taproot upgrade to Bitcoin’s consensus rules. As a short reminder, P2C allows a spender (Bob) and receiver (Alice) to agree on some data, such as a contract, and then tweak Alice’s public key so that it commits to the contract. At any time, Bob can reveal

Alice's underlying key and the tweak used to commit to the contract, proving that she received the funds. If Alice spends the funds, that fully proves that she knew about the contract, since the only way she could spend the funds received to a P2C tweaked key is by knowing the tweak (the contract).

A powerful attribute of P2C tweaked keys is that they look like any other public keys to everyone besides Alice and Bob, unless they choose to reveal the contract used to tweak the keys. Nothing is publicly revealed about the contract—not even that a contract between them exists.

A P2C contract can be arbitrarily long and detailed, the terms can be written in any language, and it can reference anything the participants want because the contract is not validated by full nodes and only the public key with the commitment is published to the blockchain.

In the context of colored coins, Bob can open the single-use seal containing his colored coins by spending the associated UTXO. In the transaction spending that UTXO, he can commit to a contract indicating the terms that the next owner (or owners) of the colored coins must fulfill in order to further spend the coins. The new owner doesn't need to be Alice, even though Alice is the one receiving the UTXO that Bob spends and Alice has tweaked her public key by the contract terms.

Because full nodes don't (and can't) validate that the contract is followed correctly, we need to figure out who is responsible for validation. That brings us to *client-side validation.*

14.3.3. Client-Side Validation

Bob had some colored coins associated with a UTXO. He spent that UTXO in a way that committed to a contract that indicated how the next receiver (or receivers) of the colored coins will prove their ownership over the coins in order to further spend them.

In practice, Bob's P2C contract likely simply committed to one or more unique identifiers for the UTXOs that will be used as single-use seals for deciding when the colored coins are next spent. For example, Bob's contract may have indicated that the UTXO Alice received to her P2C tweaked public key now controls half of his colored coins, with the other half of his colored coins now being assigned to a different UTXO that may have nothing to do with the transaction between Alice and Bob. This provides significant privacy against blockchain surveillance.

When Alice later wants to spend her colored coins to Dan, she first needs to prove to Dan that she controls the colored coins. Alice can do this by revealing to Dan her underlying P2C public key and the P2C contract terms chosen by Bob. Alice also reveals to Dan the UTXO that Bob used as the single-use seal and any information that Bob gave her about the previous owners of the colored coins. In short, Alice gives Dan a complete set of history about every previous transfer of the colored coins, with each step anchored in the Bitcoin

blockchain (but not storing any special data in the chain—just regular public keys). That history is a lot like the history of regular Bitcoin transactions that we call the blockchain, but the colored history is completely invisible to other users of the blockchain.

Dan validates this history using his software, called *client-side validation*. Notably, Dan only needs to receive and validate the parts of history that pertain to the colored coins he wants to receive. He doesn't need information about what happened to other people's colored coins—for example, he'll never need to know what happened to the other half of Bob's coins, the ones that Bob didn't transfer to Alice. This helps enhance the privacy of the colored coin protocol.

Now that we've learned about single-use seals, pay to contract, and client-side validation, we can look at the two main protocols that use them as of this writing, RGB and Taproot Assets.

14.3.4. RGB

Developers of the RGB protocol pioneered many of the ideas used in modern Bitcoin-based colored coin protocols. A primary requirement of the design for RGB was making the protocol compatible with offchain payment channels (see Payment Channels and State Channels), such as those used in Lightning Network (LN). That's accomplished at each layer of the RGB protocol:

Single-use seals
: To create a payment channel, Bob assigns his colored coins to a UTXO that requires signatures from both him and Alice to spend. Their mutual control over that UTXO serves as the single-use seal for future transfers.

Pay to contract (P2C)
: Alice and Bob can now sign multiple versions of a P2C contract. The enforcement mechanism of the underlying payment channel ensures that both parties are incentivized to only publish the latest version of the contract onchain.

Client-side validation
: To ensure that neither Alice nor Bob needs to trust each other, they each check all previous transfers of the colored coins back to their creation to ensure all contract rules were followed correctly.

The developers of RGB have described other uses for their protocol, such as creating identity tokens that can be periodically updated to protect against private key compromise.

For more information, see RGB's documentation.

14.3.5. Taproot Assets

Formerly called Taro, Taproot Assets are a colored coin protocol that is heavily influenced by RGB. Compared to RGB, Taproot Assets use a form of P2C contracts that is very similar to the version used by taproot for enabling MAST functionality (see Merklized Alternative Script Trees (MAST)). The claimed advantage of Taproot Assets over RGB is that its similarity to the widely used taproot protocol makes it simpler for wallets and other software to implement. One downside is that it may not be as flexible as the RGB protocol, especially when it comes to implementing nonasset features such as identity tokens.

> **Note** *Taproot* is part of the Bitcoin protocol. *Taproot Assets* is not, despite the similar name. Both RGB and Taproot Assets are protocols built on top of the Bitcoin protocol. The only asset natively supported by Bitcoin is bitcoin.

Even more than RGB, Taproot Assets has been designed to be compatible with LN. One challenge with forwarding nonbitcoin assets over LN is that there are two ways to accomplish the sending, each with a different set of trade-offs:

Native forwarding
: Every hop in the path between the spender and the receiver must know about the particular asset (type of colored coin) and have a sufficient balance of it to support forwarding a payment.

Translated forwarding
: The hop next to the spender and the hop next to the receiver must know about the particular asset and have a sufficient balance of it to support forwarding a payment, but every other hop only needs to support forwarding bitcoin payments.

Native forwarding is conceptually simpler but essentially requires a separate Lightning-type network for every asset. Translated forwarding allows building on the economies of scale of the Bitcoin LN, but it may be vulnerable to a problem called the *free American call option*, where a receiver may selectively accept or reject certain payments depending on recent changes to the exchange rate in order to siphon money from the hop next to them. Although there's no known perfect solution to the free American call option, there may be practical solutions that limit its harm.

Both Taproot Assets and RGB can technically support both native and translated forwarding. Taproot Assets is specifically designed around translated forwarding, whereas RGB has seen proposals to implement both.

For more information, see Taproot Asset's documentation. Additionally, the Taproot Asset developers are working on BIPs that may be available after this book goes into print.

14.4. Payment Channels and State Channels

Payment channels are a trustless mechanism for exchanging Bitcoin transactions between two parties outside of the Bitcoin blockchain. These transactions, which would be valid if settled on the Bitcoin blockchain, are held offchain instead, waiting for eventual batch settlement. Because the transactions are not settled, they can be exchanged without the usual settlement latency, allowing extremely high transaction throughput, low latency, and fine granularity.

Actually, the term *channel* is a metaphor. State channels are virtual constructs represented by the exchange of state between two parties outside of the blockchain. There are no "channels" per se, and the underlying data transport mechanism is not the channel. We use the term *channel* to represent the relationship and shared state between two parties outside of the blockchain.

To further explain this concept, think of a TCP stream. From the perspective of higher-level protocols, it is a "socket" connecting two applications across the internet. But if you look at the network traffic, a TCP stream is just a virtual channel over IP packets. Each endpoint of the TCP stream sequences and assembles IP packets to create the illusion of a stream of bytes. Underneath, it’s all disconnected packets. Similarly, a payment channel is just a series of transactions. If properly sequenced and connected, they create redeemable obligations that you can trust even though you don’t trust the other side of the channel.

In this section we will look at various forms of payment channels. First, we will examine the mechanisms used to construct a one-way (unidirectional) payment channel for a metered micropayment service, such as streaming video. Then, we will expand on this mechanism and introduce bidirectional payment channels. Finally, we will look at how bidirectional channels can be connected end-to-end to form multihop channels in a routed network, first proposed under the name *Lightning Network*.

Payment channels are part of the broader concept of a *state channel*, which represents an offchain alteration of state, secured by eventual settlement in a blockchain. A payment channel is a state channel where the state being altered is the balance of a virtual currency.

14.4.1. State Channels—Basic Concepts and Terminology

A state channel is established between two parties through a transaction that locks a shared state on the blockchain. This is called the *funding transaction*. This single transaction must be transmitted to the network and mined to establish the channel. In the example of a payment channel, the locked state is the initial balance (in currency) of the channel.

The two parties then exchange signed transactions, called *commitment transactions*, that alter the initial state. These transactions are valid transactions in that they *could* be submitted for settlement by either party, but instead are held offchain by each party

pending the channel closure. State updates can be created as fast as each party can create, sign, and transmit a transaction to the other party. In practice this means that dozens of transactions per second can be exchanged.

When exchanging commitment transactions the two parties also discourage use of the previous states, so that the most up-to-date commitment transaction is always the best one to be redeemed. This discourages either party from cheating by unilaterally closing the channel with a prior state that is more favorable to them than the current state. We will examine the various mechanisms that can be used to discourage publication of prior states in the rest of this chapter.

Finally, the channel can be closed either cooperatively, by submitting a final *settlement transaction* to the blockchain, or unilaterally, by either party submitting the last commitment transaction to the blockchain. A unilateral close option is needed in case one of the parties unexpectedly disconnects. The settlement transaction represents the final state of the channel and is settled on the blockchain.

In the entire lifetime of the channel, only two transactions need to be submitted for mining on the blockchain: the funding and settlement transactions. In between these two states, the two parties can exchange any number of commitment transactions that are never seen by anyone else or submitted to the blockchain.

A payment channel between Bob and Alice, showing the funding, commitment, and settlement transactions. illustrates a payment channel between Bob and Alice, showing the funding, commitment, and settlement transactions.

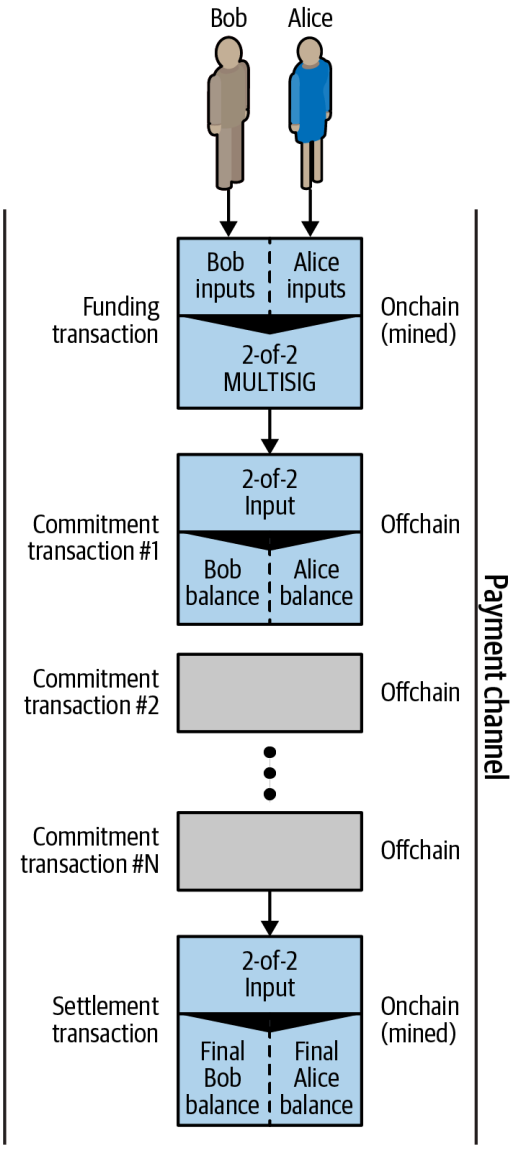


Figure 66. A payment channel between Bob and Alice, showing the funding, commitment, and settlement transactions.

14.4.2. Simple Payment Channel Example

To explain state channels, we start with a very simple example. We demonstrate a one-way channel, meaning that value is flowing in one direction only. We will also start with the naive assumption that no one is trying to cheat to keep things simple. Once we have the basic channel idea explained, we will then look at what it takes to make it trustless so that neither party *can* cheat, even if they are trying to.

For this example we will assume two participants: Emma and Fabian. Fabian offers a video streaming service that is billed by the second using a micropayment channel. Fabian charges 0.01 millibit (0.00001 BTC) per second of video, equivalent to 36 millibits (0.036 BTC) per hour of video. Emma is a user who purchases this streaming video service from Fabian. Emma purchases streaming video from Fabian with a payment channel, paying for each second of video. shows Emma buying the video streaming service from Fabian using a payment channel.

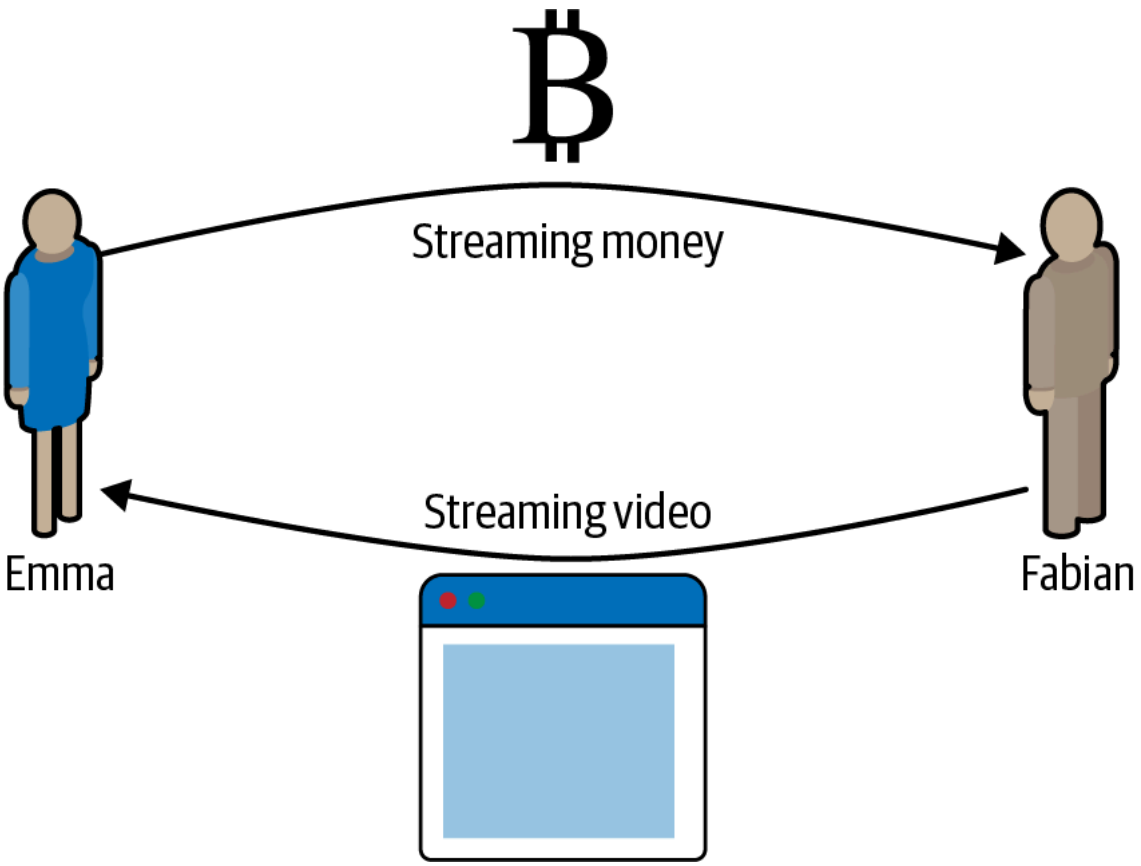


Figure 67. Emma purchases streaming video from Fabian with a payment channel, paying for each second of video.

In this example, Fabian and Emma are using special software that handles both the payment channel and the video streaming. Emma is running the software in her browser; Fabian is running it on a server. The software includes basic Bitcoin wallet functionality and can create and sign Bitcoin transactions. Both the concept and the term "payment channel" are completely hidden from the users. What they see is video that is paid for by the second.

To set up the payment channel, Emma and Fabian establish a 2-of-2 multisignature address, with each of them holding one of the keys. From Emma's perspective, the software in her browser presents a QR code with the address, and asks her to submit a "deposit" for up to 1 hour of video. The address is then funded by Emma. Emma's transaction, paying to the multisignature address, is the funding or anchor transaction for the payment channel.

For this example, let's say that Emma funds the channel with 36 millibits (0.036 BTC). This will allow Emma to consume *up to* 1 hour of streaming video. The funding transaction in this case sets the maximum amount that can be transmitted in this channel, setting the *channel capacity*.

The funding transaction consumes one or more inputs from Emma's wallet, sourcing the funds. It creates one output with an amount of 36 millibits paid to the multisignature 2-of-2 address controlled jointly between Emma and Fabian. It may have additional outputs for change back to Emma's wallet.

After the funding transaction is confirmed to a sufficient depth, Emma can start streaming video. Emma's software creates and signs a commitment transaction that changes the channel balance to credit 0.01 millibit to Fabian's address and refund 35.99 millibits back to Emma. The transaction signed by Emma consumes the 36 millibits output created by the funding transaction and creates two outputs: one for her refund, the other for Fabian's payment. The transaction is only partially signed—it requires two signatures (2-of-2), but only has Emma's signature. When Fabian's server receives this transaction, it adds the second signature (for the 2-of-2 input) and returns it to Emma together with 1 second worth of video. Now both parties have a fully signed commitment transaction that either can redeem, representing the correct up-to-date balance of the channel. Neither party broadcasts this transaction to the network.

In the next round, Emma's software creates and signs another commitment transaction (commitment #2) that consumes the *same* 2-of-2 output from the funding transaction. The second commitment transaction allocates one output of 0.02 millibits to Fabian's address and one output of 35.98 millibits back to Emma's address. This new transaction is payment for two cumulative seconds of video. Fabian's software signs and returns the second commitment transaction, together with another second of video.

In this way, Emma's software continues to send commitment transactions to Fabian's server in exchange for streaming video. The balance of the channel gradually accumulates in favor of Fabian as Emma consumes more seconds of video. Let's say Emma watches 600 seconds (10 minutes) of video, creating and signing 600 commitment transactions. The last commitment transaction (#600) will have two outputs, splitting the balance of the channel, 6 millibits to Fabian and 30 millibits to Emma.

Finally, Emma clicks "Stop" to stop streaming video. Either Fabian or Emma can now transmit the final state transaction for settlement. This last transaction is the *settlement transaction* and pays Fabian for all the video Emma consumed, refunding the remainder of the funding transaction to Emma.

Emma's payment channel with Fabian, showing the commitment transactions that update the balance of the channel. shows the channel between Emma and Fabian and the commitment transactions that update the balance of the channel.

In the end, only two transactions are recorded on the blockchain: the funding transaction that established the channel and a settlement transaction that allocated the final balance correctly between the two participants.

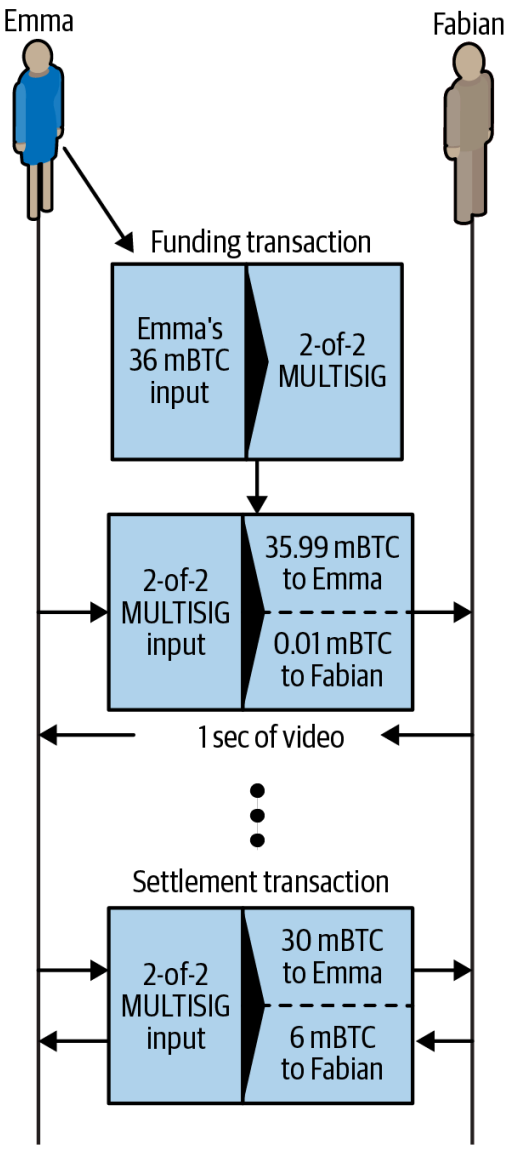


Figure 68. Emma's payment channel with Fabian, showing the commitment transactions that update the balance of the channel.

14.4.3. Making Trustless Channels

The channel we just described works, but only if both parties cooperate, without any failures or attempts to cheat. Let's look at some of the scenarios that break this channel and see what is needed to fix those:

- Once the funding transaction happens, Emma needs Fabian's signature to get any money back. If Fabian disappears, Emma's funds are locked in a 2-of-2 and effectively lost. This channel, as constructed, leads to a loss of funds if one of the parties becomes unavailable before there is at least one commitment transaction signed by both parties.
- While the channel is running, Emma can take any of the commitment transactions Fabian has countersigned and transmit one to the blockchain. Why pay for 600 seconds of video if she can transmit commitment transaction #1 and only pay for 1 second of video? The channel fails because Emma can cheat by broadcasting a prior commitment that is in her favor.

Both of these problems can be solved with timelocks—let's look at how we could use transaction-level timelocks.

Emma cannot risk funding a 2-of-2 multisig unless she has a guaranteed refund. To solve this problem, Emma constructs the funding and refund transaction at the same time. She signs the funding transaction but doesn't transmit it to anyone. Emma transmits only the refund transaction to Fabian and obtains his signature.

The refund transaction acts as the first commitment transaction, and its timelock establishes the upper bound for the channel's life. In this case, Emma could set the lock time to 30 days or 4,320 blocks into the future. All subsequent commitment transactions must have a shorter timelock so they can be redeemed before the refund transaction.

Now that Emma has a fully signed refund transaction, she can confidently transmit the signed funding transaction knowing that she can eventually, after the timelock expires, redeem the refund transaction even if Fabian disappears.

Every commitment transaction the parties exchange during the life of the channel will be timelocked into the future. But the delay will be slightly shorter for each commitment, so the most recent commitment can be redeemed before the prior commitment it invalidates. Because of the lock time, neither party can successfully propagate any of the commitment transactions until their timelock expires. If all goes well, they will cooperate and close the channel gracefully with a settlement transaction, making it unnecessary to transmit an intermediate commitment transaction. If not, the most recent commitment transaction can be propagated to settle the account and invalidate all prior commitment transactions.

For example, if commitment transaction #1 is timelocked to 4,320 blocks in the future, then commitment transaction #2 is timelocked to 4,319 blocks in the future. Commitment transaction #600 can be spent 600 blocks before commitment transaction #1 becomes valid.

Each commitment sets a shorter timelock, allowing it to be spent before the previous commitments become valid. shows each commitment transaction setting a shorter timelock, allowing it to be spent before the previous commitments become valid.

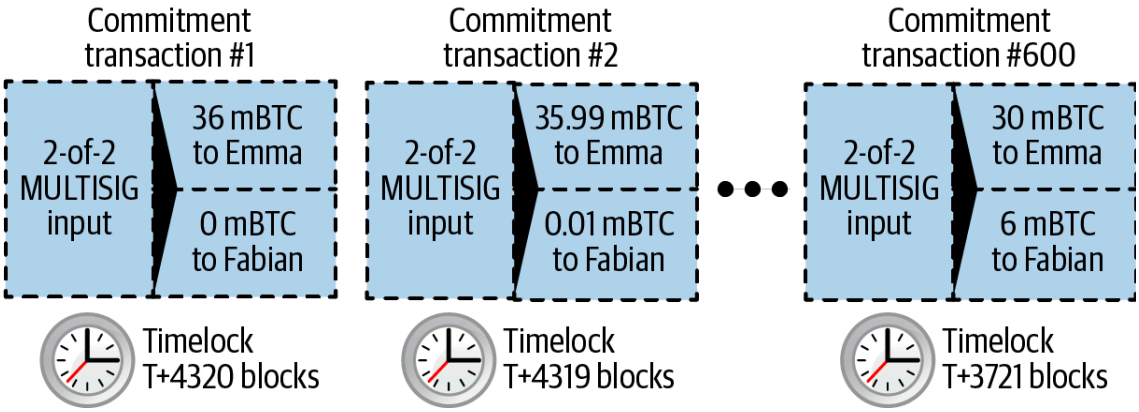


Figure 69. Each commitment sets a shorter timelock, allowing it to be spent before the previous commitments become valid.

Each subsequent commitment transaction must have a shorter timelock so that it may be broadcast before its predecessors and before the refund transaction. The ability to broadcast a commitment earlier ensures it will be able to spend the funding output and preclude any other commitment transaction from being redeemed by spending the output. The guarantees offered by the Bitcoin blockchain, preventing double-spends and enforcing timelocks, effectively allow each commitment transaction to invalidate its predecessors.

State channels use timelocks to enforce smart contracts across a time dimension. In this example we saw how the time dimension guarantees that the most recent commitment transaction becomes valid before any earlier commitments. Thus, the most recent commitment transaction can be transmitted, spending the inputs and invalidating prior commitment transactions. The enforcement of smart contracts with absolute timelocks protects against cheating by one of the parties. This implementation needs nothing more than absolute transaction-level lock time. Next, we will see how script-level timelocks, CHECKLOCKTIMEVERIFY and CHECKSEQUENCEVERIFY, can be used to construct more flexible, useful, and sophisticated state channels.

Timelocks are not the only way to invalidate prior commitment transactions. In the next sections we will see how a revocation key can be used to achieve the same result. Timelocks are effective, but they have two distinct disadvantages. By establishing a maximum timelock when the channel is first opened, they limit the lifetime of the channel. Worse, they force channel implementations to strike a balance between allowing long-lived channels and forcing one of the participants to wait a very long time for a refund in case of premature closure. For example, if you allow the channel to remain open for 30 days by

setting the refund timelock to 30 days, if one of the parties disappears immediately, the other party must wait 30 days for a refund. The more distant the endpoint, the more distant the refund.

The second problem is that since each subsequent commitment transaction must decrement the timelock, there is an explicit limit on the number of commitment transactions that can be exchanged between the parties. For example, a 30-day channel, setting a timelock of 4,320 blocks into the future, can only accommodate 4,320 intermediate commitment transactions before it must be closed. There is a danger in setting the timelock commitment transaction interval at 1 block. By setting the timelock interval between commitment transactions to 1 block, a developer is creating a very high burden for the channel participants who have to be vigilant, remain online and watching, and be ready to transmit the right commitment transaction at any time.

In the preceding example of a single-direction channel, it's easy to eliminate the per-commitment timelock. After Emma receives the signature on the timelocked refund transaction from Fabian, no timelocks are placed on the commitment transactions. Instead, Emma sends her signature on each commitment transaction to Fabian but Fabian doesn't send her any of his signatures on the commitment transactions. That means only Fabian has both signatures for a commitment transaction, so only he can broadcast one of those commitments. When Emma finishes streaming video, Fabian will always prefer to broadcast the transaction that pays him the most—which will be the latest state. This construction in called a Spillman-style payment channel, which was first described and implemented in 2013, although they are only safe to use with witness (segwit) transactions, which didn't become available until 2017.

Now that we understand how timelocks can be used to invalidate prior commitments, we can see the difference between closing the channel cooperatively and closing it unilaterally by broadcasting a commitment transaction. All commitment transactions in our prior example were timelocked, therefore broadcasting a commitment transaction will always involve waiting until the timelock has expired. But if the two parties agree on what the final balance is and know they both hold commitment transactions that will eventually make that balance a reality, they can construct a settlement transaction without a timelock representing that same balance. In a cooperative close, either party takes the most recent commitment transaction and builds a settlement transaction that is identical in every way except that it omits the timelock. Both parties can sign this settlement transaction knowing there is no way to cheat and get a more favorable balance. By cooperatively signing and transmitting the settlement transaction, they can close the channel and redeem their balance immediately. Worst case, one of the parties can be petty, refuse to cooperate, and force the other party to do a unilateral close with the most recent commitment transaction. If they do that, they have to wait for their funds too.

14.4.4. Asymmetric Revocable Commitments

Another way to handle the prior commitment states is to explicitly revoke them. However, this is not easy to achieve. A key characteristic of Bitcoin is that once a transaction is valid, it remains valid and does not expire. The only way to cancel a transaction is to get a conflicting transaction confirmed. That's why we used timelocks in the simple payment channel example to ensure that more recent commitments could be spent before older commitments were valid. However, sequencing commitments in time creates a number of constraints that make payment channels difficult to use.

Even though a transaction cannot be canceled, it can be constructed in such a way as to make it undesirable to use. The way we do that is by giving each party a *revocation key* that can be used to punish the other party if they try to cheat. This mechanism for revoking prior commitment transactions was first proposed as part of the LN.

To explain revocation keys, we will construct a more complex payment channel between two exchanges run by Hitesh and Irene. Hitesh and Irene run Bitcoin exchanges in India and the USA, respectively. Customers of Hitesh's Indian exchange often send payments to customers of Irene's USA exchange and vice versa. Currently, these transactions occur on the Bitcoin blockchain, but this means paying fees and waiting several blocks for confirmations. Setting up a payment channel between the exchanges will significantly reduce the cost and accelerate the transaction flow.

Hitesh and Irene start the channel by collaboratively constructing a funding transaction, each funding the channel with 5 bitcoin. Before they sign the funding transaction, they must sign the first set of commitments (called the *refund*) that assigns the initial balance of 5 bitcoin for Hitesh and 5 bitcoin for Irene. The funding transaction locks the channel state in a 2-of-2 multisig, just like in the example of a simple channel.

The funding transaction may have one or more inputs from Hitesh (adding up to 5 bitcoins or more), and one or more inputs from Irene (adding up to 5 bitcoins or more). The inputs have to slightly exceed the channel capacity in order to cover the transaction fees. The transaction has one output that locks the 10 total bitcoins to a 2-of-2 multisig address controlled by both Hitesh and Irene. The funding transaction may also have one or more outputs returning change to Hitesh and Irene if their inputs exceeded their intended channel contribution. This is a single transaction with inputs offered and signed by two parties. It has to be constructed in collaboration and signed by each party before it is transmitted.

Now, instead of creating a single commitment transaction that both parties sign, Hitesh and Irene create two different commitment transactions that are *asymmetric*.

Hitesh has a commitment transaction with two outputs. The first output pays Irene the 5 bitcoins she is owed *immediately*. The second output pays Hitesh the 5 bitcoins he is owed, but only after a timelock of 1,000 blocks. The transaction outputs look like this:

```
Input: 2-of-2 funding output, signed by Irene
```

```
Output 0 <5 bitcoins>:
    <Irene's Public Key> CHECKSIG

Output 1 <5 bitcoins>:
    <1000 blocks>
    CHECKSEQUENCEVERIFY
    DROP
    <Hitesh's Public Key> CHECKSIG
```

Irene has a different commitment transaction with two outputs. The first output pays Hitesh the 5 bitcoins he is owed immediately. The second output pays Irene the 5 bitcoins she is owed but only after a timelock of 1,000 blocks. The commitment transaction Irene holds (signed by Hitesh) looks like this:

```
Input: 2-of-2 funding output, signed by Hitesh

Output 0 <5 bitcoins>:
    <Hitesh's Public Key> CHECKSIG

Output 1 <5 bitcoins>:
    <1000 blocks>
    CHECKSEQUENCEVERIFY
    DROP
    <Irene's Public Key> CHECKSIG
```

This way, each party has a commitment transaction, spending the 2-of-2 funding output. This input is signed by the *other* party. At any time the party holding the transaction can also sign (completing the 2-of-2) and broadcast. However, if they broadcast the commitment transaction, it pays the other party immediately, whereas they have to wait for a timelock to expire. By imposing a delay on the redemption of one of the outputs, we put each party at a slight disadvantage when they choose to unilaterally broadcast a commitment transaction. But a time delay alone isn't enough to encourage fair conduct.

Two asymmetric commitment transactions with delayed payment for the party holding the transaction. shows two asymmetric commitment transactions, where the output paying the holder of the commitment is delayed.

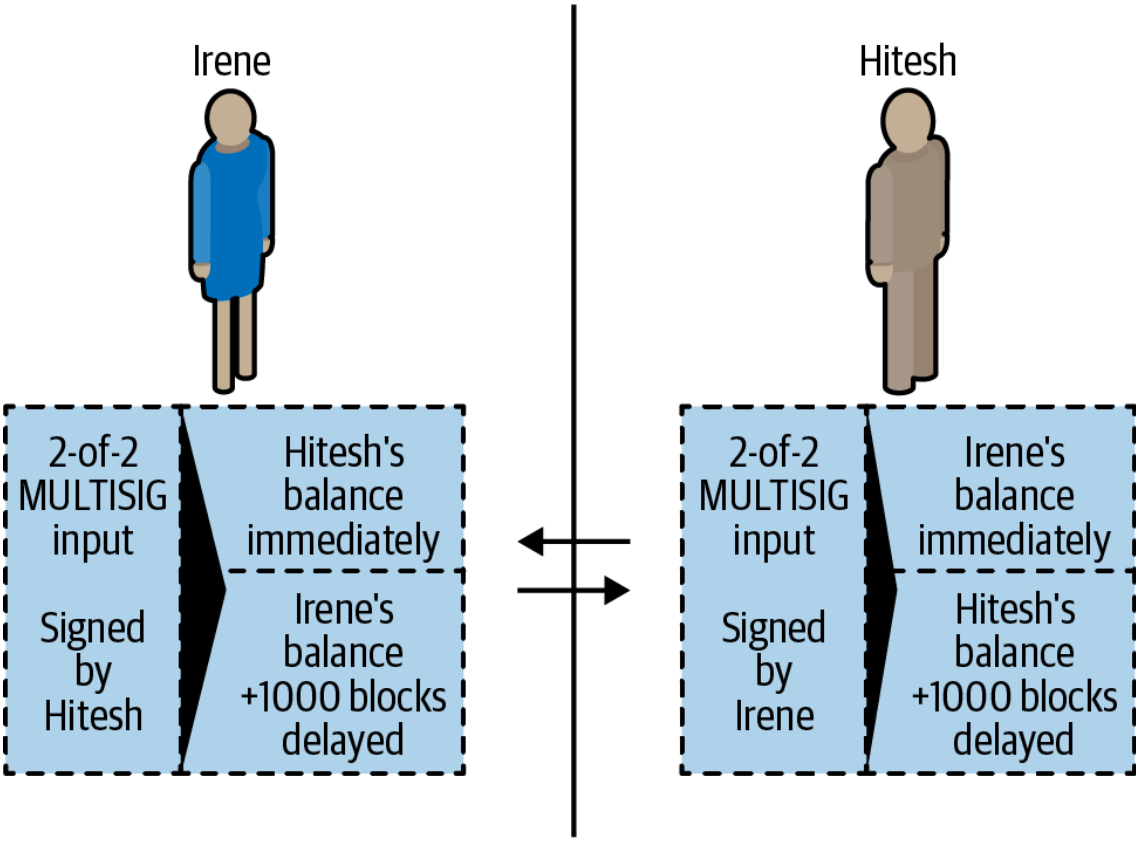


Figure 70. Two asymmetric commitment transactions with delayed payment for the party holding the transaction.

Now we introduce the final element of this scheme: a revocation key that prevents a cheater from broadcasting an expired commitment. The revocation key allows the wronged party to punish the cheater by taking the entire balance of the channel.

The revocation key is composed of two secrets, each half generated independently by each channel participant. It is similar to a 2-of-2 multisig, but constructed using elliptic curve arithmetic, so that both parties know the revocation public key but each party knows only half the revocation secret key.

In each round, both parties reveal their half of the revocation secret to the other party, thereby giving the other party (who now has both halves) the means to claim the penalty output if this revoked transaction is ever broadcast.

Each of the commitment transactions has a "delayed" output. The redemption script for that output allows one party to redeem it after 1,000 blocks, *or* the other party to redeem it if they have a revocation key, penalizing transmission of a revoked commitment.

So when Hitesh creates a commitment transaction for Irene to sign, he makes the second output payable to himself after 1,000 blocks or to the revocation public key (of which he only knows half the secret). Hitesh constructs this transaction. He will only reveal his half of the revocation secret to Irene when he is ready to move to a new channel state and wants to revoke this commitment.

The second output's script looks like this:

```
Output 0 <5 bitcoins>:
    <Irene's Public Key> CHECKSIG

Output 1 <5 bitcoins>:
IF
    # Revocation penalty output
    <Revocation Public Key>
ELSE
    <1000 blocks>
    CHECKSEQUENCEVERIFY
    DROP
    <Hitesh's Public Key>
ENDIF
CHECKSIG
```

Irene can confidently sign this transaction since if transmitted, it will immediately pay her what she is owed. Hitesh holds the transaction but knows that if he transmits it in a unilateral channel closing, he will have to wait 1,000 blocks to get paid.

After the channel is advanced to the next state, Hitesh has to *revoke* this commitment transaction before Irene will agree to sign any further commitment transactions. To do that, all he has to do is send his half of the *revocation key* to Irene. Once Irene has both halves of the revocation secret key for this commitment, she can sign a future commitment with confidence. She knows that if Hitesh tries to cheat by publishing the prior commitment, she can use the revocation key to redeem Hitesh's delayed output. *If Hitesh cheats, Irene gets BOTH outputs*. Meanwhile, Hitesh only has half the revocation secret for that revocation public key and can't redeem the output until 1,000 blocks. Irene will be able to redeem the output and punish Hitesh before the 1,000 blocks have elapsed.

The revocation protocol is bilateral, meaning that in each round, as the channel state is advanced, the two parties exchange new commitments, exchange revocation secrets for the previous commitments, and sign each other's new commitment transactions. After they accept a new state, they make the prior state impossible to use by giving each other the necessary revocation secrets to punish any cheating.

Let's look at an example of how it works. One of Irene's customers wants to send 2 bitcoins to one of Hitesh's customers. To transmit 2 bitcoins across the channel, Hitesh and Irene must advance the channel state to reflect the new balance. They will commit to a new state (state number 2) where the channel's 10 bitcoins are split, 7 bitcoins to Hitesh and 3 bitcoins to Irene. To advance the state of the channel, they will each create new commitment transactions reflecting the new channel balance.

As before, these commitment transactions are asymmetric so the commitment transaction each party holds forces them to wait if they redeem it. Crucially, before signing new commitment transactions, they must first exchange revocation keys to invalidate any outdated commitments. In this particular case, Hitesh's interests are aligned with the real state of the channel and therefore he has no reason to broadcast a prior state. However, for Irene, state number 1 leaves her with a higher balance than state 2. When Irene gives Hitesh the revocation key for her prior commitment transaction (state number 1), she is effectively revoking her ability to profit from regressing the channel to a prior state because with the revocation key, Hitesh can redeem both outputs of the prior commitment transaction without delay. Meaning if Irene broadcasts the prior state, Hitesh can exercise his right to take all of the outputs.

Importantly, the revocation doesn't happen automatically. While Hitesh has the ability to punish Irene for cheating, he has to watch the blockchain diligently for signs of cheating. If he sees a prior commitment transaction broadcast, he has 1,000 blocks to take action and use the revocation key to thwart Irene's cheating and punish her by taking the entire balance, all 10 bitcoins.

Asymmetric revocable commitments with relative time locks (CSV) are a much better way to implement payment channels and a very significant innovation in this technology. With this construct, the channel can remain open indefinitely and can have billions of intermediate commitment transactions. In implementations of LN, the commitment state is identified by a 48-bit index, allowing more than 281 trillion (2.8×10^{14}) state transitions in any single channel.

14.4.5. Hash Time Lock Contracts (HTLC)

Payment channels can be further extended with a special type of smart contract that allows the participants to commit funds to a redeemable secret, with an expiration time. This feature is called a *hash time lock contract*, or *HTLC*, and is used in both bidirectional and routed payment channels.

Let's first explain the "hash" part of the HTLC. To create an HTLC, the intended recipient of the payment will first create a secret *R*. They then calculate the hash of this secret *H*:

$$H = Hash(R)$$

This produces a hash *H* that can be included in an output's script. Whoever knows the secret can use it to redeem the output. The secret *R* is also referred to as a *preimage* to the hash function. The preimage is just the data that is used as input to a hash function.

The second part of an HTLC is the "time lock" component. If the secret is not revealed, the payer of the HTLC can get a "refund" after some time. This is achieved with an absolute timelock using CHECKLOCKTIMEVERIFY.

The script implementing an HTLC might look like this:

```
IF
    # Payment if you have the secret R
    HASH160 <H> EQUALVERIFY
    <Receiver Public Key> CHECKSIG
ELSE
    # Refund after timeout.
    <lock time> CHECKLOCKTIMEVERIFY DROP
    <Payer Public Key> CHECKSIG
ENDIF
```

Anyone who knows the secret *R*, which when hashed equals to *H*, can redeem this output by exercising the first clause of the IF flow.

If the secret is not revealed and the HTLC claimed after a certain number of blocks, the payer can claim a refund using the second clause in the IF flow.

This is a basic implementation of an HTLC. This type of HTLC can be redeemed by *anyone* who has the secret *R*. An HTLC can take many different forms with slight variations to the script. For example, adding a CHECKSIG operator and a public key in the first clause restricts redemption of the hash to a particular recipient, who must also know the secret *R*.

14.5. Routed Payment Channels (Lightning Network)

The Lightning Network (LN) is a proposed routed network of bidirectional payment channels connected end-to-end. A network like this can allow any participant to route a payment from channel to channel without trusting any of the intermediaries. The LN was first described by Joseph Poon and Thadeus Dryja in February 2015, building on the concept of payment channels as proposed and elaborated upon by many others.

"Lightning Network" refers to a specific design for a routed payment channel network, which has now been implemented by at least five different open source teams. The independent implementations are coordinated by a set of interoperability standards described in the *Basics of Lightning Technology (BOLT)* repository.

14.5.1. Basic Lightning Network Example

Let’s see how this works.

In this example, we have five participants: Alice, Bob, Carol, Diana, and Eric. These five participants have opened payment channels with each other, in pairs. Alice has a payment channel with Bob. Bob is connected to Carol, Carol to Diana, and Diana to Eric. For simplicity let’s assume each channel is funded with 2 bitcoins by each participant, for a total capacity of 4 bitcoins in each channel.

A series of bidirectional payment channels linked to form an LN that can route a payment from Alice to Eric. shows five participants in an LN, connected by bidirectional payment channels that can be linked to make a payment from Alice to Eric (see Routed Payment Channels (Lightning Network)).

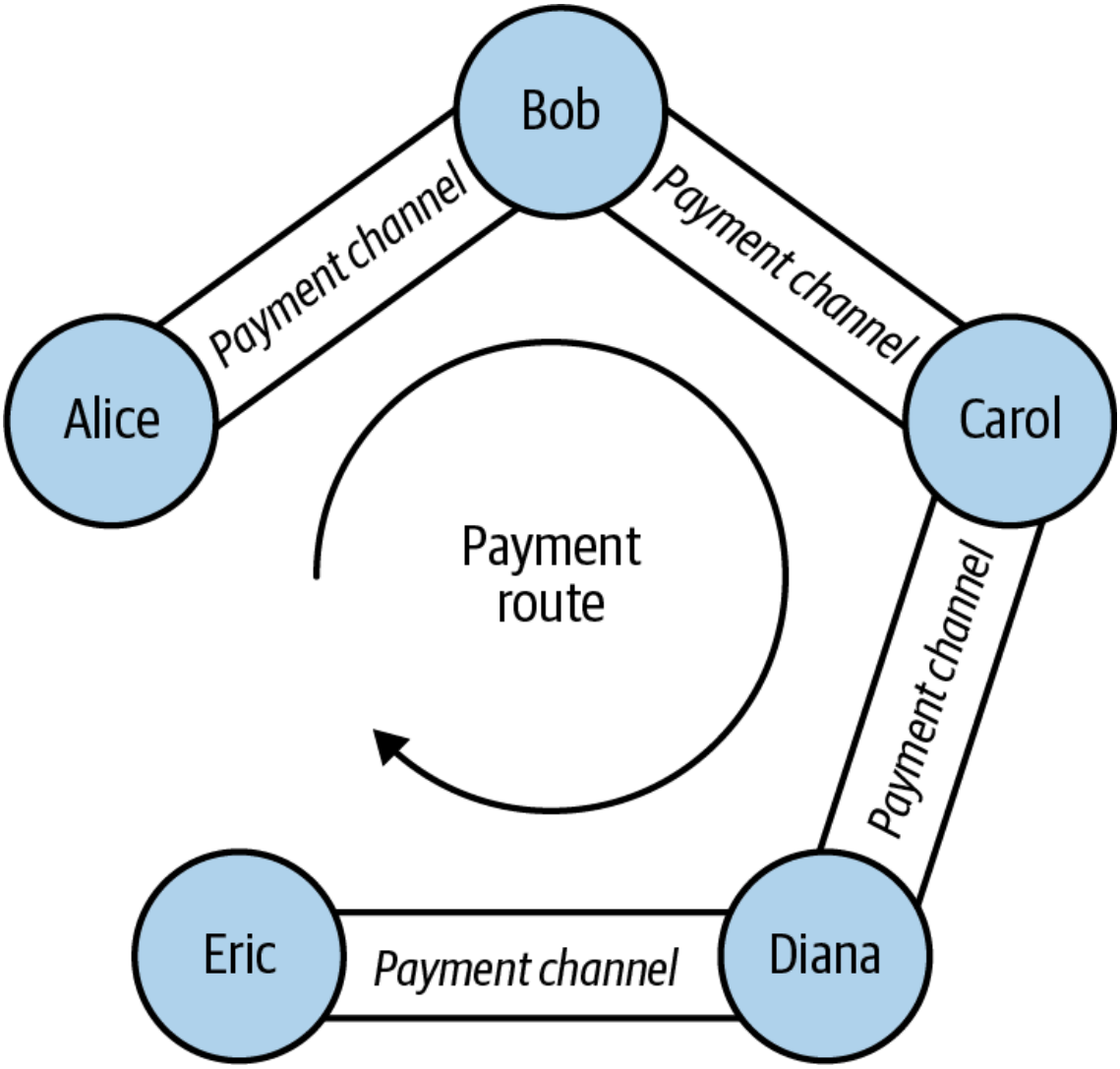


Figure 71. A series of bidirectional payment channels linked to form an LN that can route a payment from Alice to Eric.

Alice wants to pay Eric 1 bitcoin. However, Alice is not connected to Eric by a payment channel. Creating a payment channel requires a funding transaction, which must be committed to the Bitcoin blockchain. Alice does not want to open a new payment channel and commit more of her funds. Is there a way to pay Eric indirectly?

Step-by-step payment routing through an LN. shows the step-by-step process of routing a payment from Alice to Eric, through a series of HTLC commitments on the payment channels connecting the participants.

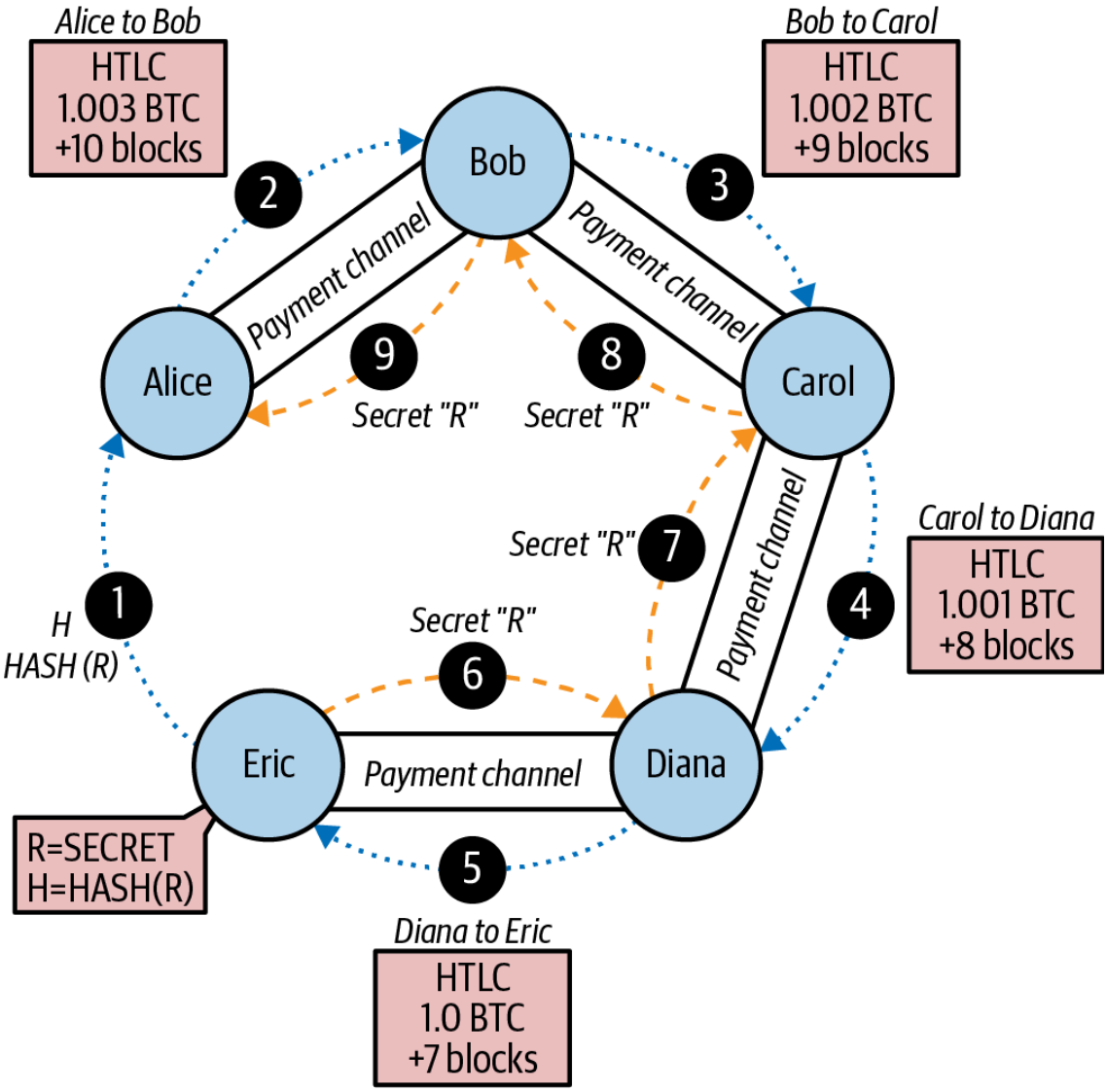


Figure 72. Step-by-step payment routing through an LN.

Alice is running an LN node that is keeping track of her payment channel to Bob and has the ability to discover routes between payment channels. Alice's LN node also has the ability to connect over the internet to Eric's LN node. Eric's LN node creates a secret R using a random number generator. Eric's node does not reveal this secret to anyone. Instead, Eric's node calculates a hash H of the secret R and transmits this hash to Alice's node in the form of an invoice (see Step-by-step payment routing through an LN., step 1).

Now Alice's LN node constructs a route between Alice's LN node and Eric's LN node. The pathfinding algorithm used will be examined in more detail later, but for now let's assume that Alice's node can find an efficient route.

Alice’s node then constructs an HTLC, payable to the hash H, with a 10-block refund timeout (current block + 10), for an amount of 1.003 bitcoins (see Step-by-step payment routing through an LN., step 2). The extra 0.003 will be used to compensate the intermediate nodes for their participation in this payment route. Alice offers this HTLC to Bob, deducting 1.003 bitcoins from her channel balance with Bob and committing it to the HTLC. The HTLC has the following meaning: "*Alice is committing 1.003 bitcoins of her channel balance to be paid to Bob if Bob knows the secret, or refunded back to Alice’s balance if 10 blocks elapse*." The channel balance between Alice and Bob is now expressed by commitment transactions with three outputs: 2 bitcoins balance to Bob, 0.997 bitcoins balance to Alice, 1.003 bitcoins committed in Alice’s HTLC. Alice’s balance is reduced by the amount committed to the HTLC.

Bob now has a commitment that if he is able to get the secret R within the next 10 blocks, he can claim the 1.003 bitcoins locked by Alice. With this commitment in hand, Bob’s node constructs an HTLC on his payment channel with Carol. Bob’s HTLC commits 1.002 bitcoins to hash H for 9 blocks, which Carol can redeem if she has secret R (see Step-by-step payment routing through an LN. step 3). Bob knows that if Carol can claim his HTLC, she has to produce R. If Bob has R in nine blocks, he can use it to claim Alice’s HTLC to him. He also makes 0.001 bitcoins for committing his channel balance for nine blocks. If Carol is unable to claim his HTLC and he is unable to claim Alice’s HTLC, everything reverts back to the prior channel balances and no one is at a loss. The channel balance between Bob and Carol is now: 2 to Carol, 0.998 to Bob, 1.002 committed by Bob to the HTLC.

Carol now has a commitment that if she gets R within the next nine blocks, she can claim 1.002 bitcoins locked by Bob. Now she can make an HTLC commitment on her channel with Diana. She commits an HTLC of 1.001 bitcoins to hash H, for eight blocks, which Diana can redeem if she has secret R (see Step-by-step payment routing through an LN., step 4). From Carol’s perspective, if this works she is 0.001 bitcoins better off and if it doesn’t she loses nothing. Her HTLC to Diana is only viable if R is revealed, at which point she can claim the HTLC from Bob. The channel balance between Carol and Diana is now: 2 to Diana, 0.999 to Carol, 1.001 committed by Carol to the HTLC.

Finally, Diana can offer an HTLC to Eric, committing 1 bitcoin for seven blocks to hash H (see Step-by-step payment routing through an LN., step 5). The channel balance between Diana and Eric is now: 2 to Eric, 1 to Diana, 1 committed by Diana to the HTLC.

However, at this hop in the route, Eric *has* secret R. He can therefore claim the HTLC offered by Diana. He sends R to Diana and claims the 1 bitcoin, adding it to his channel balance (see Step-by-step payment routing through an LN., step 6). The channel balance is now: 1 to Diana, 3 to Eric.

Now, Diana has secret R. Therefore, she can now claim the HTLC from Carol. Diana transmits R to Carol and adds the 1.001 bitcoins to her channel balance (see Step-by-step payment routing through an LN., step 7). Now the channel balance between Carol and Diana is: 0.999 to Carol, 3.001 to Diana. Diana has "earned" 0.001 for participating in this payment route.

Flowing back through the route, the secret R allows each participant to claim the outstanding HTLCs. Carol claims 1.002 from Bob, setting the balance on their channel to: 0.998 to Bob, 3.002 to Carol (see Step-by-step payment routing through an LN., step 8). Finally, Bob claims the HTLC from Alice (see Step-by-step payment routing through an LN., step 9). Their channel balance is updated as: 0.997 to Alice, 3.003 to Bob.

Alice has paid Eric 1 bitcoin without opening a channel to Eric. None of the intermediate parties in the payment route had to trust each other. For the short-term commitment of their funds in the channel they are able to earn a small fee, with the only risk being a small delay in refund if the channel was closed or the routed payment failed.

14.5.2. Lightning Network Transport and Pathfinding

All communications between LN nodes are encrypted point-to-point. In addition, nodes have a long-term public key that they use as an identifier and to authenticate each other.

Whenever a node wishes to send a payment to another node, it must first construct a *path* through the network by connecting payment channels with sufficient capacity. Nodes advertise routing information, including what channels they have open, how much capacity each channel has, and what fees they charge to route payments. The routing information can be shared in a variety of ways, and different pathfinding protocols have emerged as LN technology has advanced. Current implementations of route discovery use a P2P model where nodes propagate channel announcements to their peers in a "flooding" model, similar to how Bitcoin propagates transactions.

In our previous example, Alice's node uses one of these route discovery mechanisms to find one or more paths connecting her node to Eric's node. Once Alice's node has constructed a path, she will initialize that path through the network by propagating a series of encrypted and nested instructions to connect each of the adjacent payment channels.

Importantly, this path is only known to Alice's node. All other participants in the payment route see only the adjacent nodes. From Carol's perspective, this looks like a payment from Bob to Diana. Carol does not know that Bob is actually relaying a payment from Alice. She also doesn't know that Diana will be relaying a payment to Eric.

This is a critical feature of the LN because it ensures privacy of payments and makes it difficult to apply surveillance, censorship, or blacklists. But how does Alice establish this payment path without revealing anything to the intermediary nodes?

The LN implements an onion-routed protocol based on a scheme called Sphinx. This routing protocol ensures that a payment sender can construct and communicate a path through the LN such that:

- Intermediate nodes can verify and decrypt their portion of route information and find the next hop.
- Other than the previous and next hops, they cannot learn about any other nodes that are part of the path.

- They cannot identify the length of the payment path or their own position in that path.
- Each part of the path is encrypted in such a way that a network-level attacker cannot associate the packets from different parts of the path to each other.
- Unlike Tor (an onion-routed anonymization protocol on the internet), there are no "exit nodes" that can be placed under surveillance. The payments do not need to be transmitted to the Bitcoin blockchain; the nodes just update channel balances.

Using this onion-routed protocol, Alice wraps each element of the path in a layer of encryption, starting with the end and working backward. She encrypts a message to Eric with Eric's public key. This message is wrapped in a message encrypted to Diana, identifying Eric as the next recipient. The message to Diana is wrapped in a message encrypted to Carol's public key and identifying Diana as the next recipient. The message to Carol is encrypted to Bob's key. Thus, Alice has constructed this encrypted multilayer "onion" of messages. She sends this to Bob, who can only decrypt and unwrap the outer layer. Inside, Bob finds a message addressed to Carol that he can forward to Carol but cannot decipher himself. Following the path, the messages get forwarded, decrypted, forwarded, etc., all the way to Eric. Each participant knows only the previous and next node in each hop.

Each element of the path contains information on the HTLC that must be extended to the next hop, the amount that is being sent, the fee to include, and the CLTV lock time (in blocks) expiration of the HTLC. As the route information propagates, the nodes make HTLC commitments forward to the next hop.

At this point, you might be wondering how it is possible that the nodes do not know the length of the path and their position in that path. After all, they receive a message and forward it to the next hop. Doesn't it get shorter, allowing them to deduce the path size and their position? To prevent this, the packet size is fixed and padded with random data. Each node sees the next hop and a fixed-length encrypted message to forward. Only the final recipient sees that there is no next hop. To everyone else it seems as if there are always more hops to go.

14.5.3. Lightning Network Benefits

An LN is a second-layer routing technology. It can be applied to any blockchain that supports some basic capabilities, such as multisignature transactions, timelocks, and basic smart contracts.

LN is layered on top of the Bitcoin network, giving Bitcoin a significant increase in capacity, privacy, granularity, and speed, without sacrificing the principles of trustless operation without intermediaries:

Privacy

LN payments are much more private than payments on the Bitcoin blockchain, as they are not public. While participants in a route can see payments propagated across their channels, they do not know the sender or recipient.

Fungibility
: An LN makes it much more difficult to apply surveillance and blacklists on Bitcoin, increasing the fungibility of the currency.

Speed
: Bitcoin transactions using LN are settled in milliseconds, rather than minutes or hours, as HTLCs are cleared without committing transactions to a block.

Granularity
: An LN can enable payments at least as small as the Bitcoin "dust" limit, perhaps even smaller.

Capacity
: An LN increases the capacity of the Bitcoin system by several orders of magnitude. The upper bound to the number of payments per second that can be routed over a Lightning Network depends only on the capacity and speed of each node.

Trustless Operation
: An LN uses Bitcoin transactions between nodes that operate as peers without trusting each other. Thus, an LN preserves the principles of the Bitcoin system, while expanding its operating parameters significantly.

We have examined just a few of the emerging applications that can be built using the Bitcoin blockchain as a trust platform. These applications expand the scope of Bitcoin beyond payments.

Now that you have reached the end of this book, what will you do with the knowledge you have gained? Millions of people, perhaps billions, know the name "Bitcoin," but only a small percentage of them know as much about how Bitcoin works as you now do. That knowledge is precious. Even more precious are the people, such as yourself, who are so interested in Bitcoin that you are willing to read several hundred pages about it.

If you haven’t already begun doing so, please consider contributing to Bitcoin in some way. You can run a full node to validate the Bitcoin payments you receive, build applications that make it easier for other people to use Bitcoin, or help educate other people about Bitcoin and its potential. You can even take the rare step of contributing to open source Bitcoin infrastructure software, such as Bitcoin Core, carefully working with a small number of incredibly smart people to build tools that no one will ever pay for but that billions may one day depend upon.

Whatever your Bitcoin journey, we thank you for making *Mastering Bitcoin* a part of it.

Appendix A: The Bitcoin Whitepaper by Satoshi Nakamoto

> **Note** This is the original whitepaper, reproduced in its entirety exactly as it was published by Satoshi Nakamoto in October 2008.

A.1. Bitcoin - A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto

satoshin@gmx.com

www.bitcoin.org

Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

A.1.1. Introduction

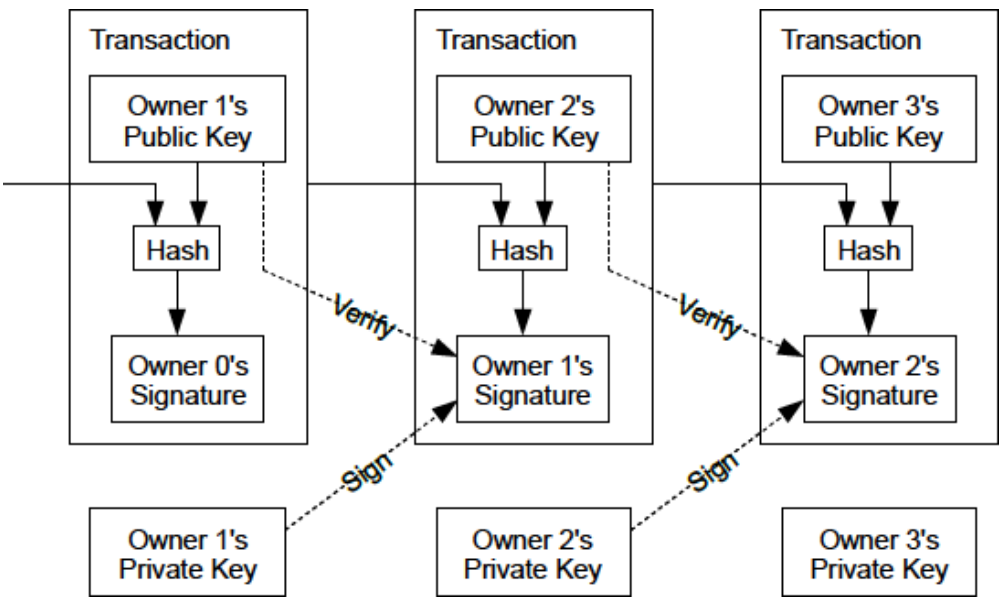
Commerce on the Internet has come to rely almost exclusively on financial institutions serving as trusted third parties to process electronic payments. While the system works well enough for most transactions, it still suffers from the inherent weaknesses of the trust based model. Completely non-reversible transactions are not really possible, since financial institutions cannot avoid mediating disputes. The cost of mediation increases transaction costs, limiting the minimum practical transaction size and cutting off the possibility for small casual transactions, and there is a broader cost in the loss of ability to make non-reversible payments for nonreversible services. With the possibility of reversal, the need for trust spreads. Merchants must be wary of their customers, hassling them for more information than they would otherwise need. A certain percentage of fraud is accepted as

unavoidable. These costs and payment uncertainties can be avoided in person by using physical currency, but no mechanism exists to make payments over a communications channel without a trusted party.

What is needed is an electronic payment system based on cryptographic proof instead of trust, allowing any two willing parties to transact directly with each other without the need for a trusted third party. Transactions that are computationally impractical to reverse would protect sellers from fraud, and routine escrow mechanisms could easily be implemented to protect buyers. In this paper, we propose a solution to the double-spending problem using a peer-to-peer distributed timestamp server to generate computational proof of the chronological order of transactions. The system is secure as long as honest nodes collectively control more CPU power than any cooperating group of attacker nodes.

A.1.2. Transactions

We define an electronic coin as a chain of digital signatures. Each owner transfers the coin to the next by digitally signing a hash of the previous transaction and the public key of the next owner and adding these to the end of the coin. A payee can verify the signatures to verify the chain of ownership.



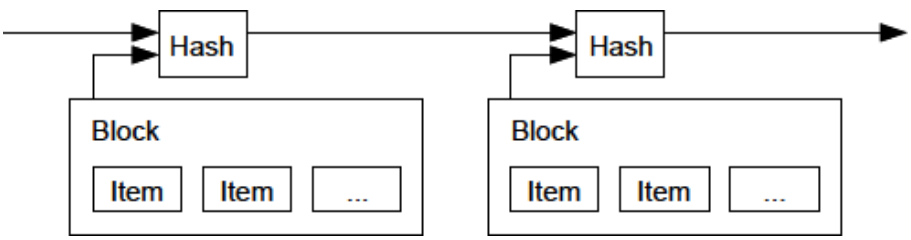
The problem of course is the payee can’t verify that one of the owners did not double-spend the coin. A common solution is to introduce a trusted central authority, or mint, that checks every transaction for double spending. After each transaction, the coin must be returned to the mint to issue a new coin, and only coins issued directly from the mint are

trusted not to be double-spent. The problem with this solution is that the fate of the entire money system depends on the company running the mint, with every transaction having to go through them, just like a bank.

We need a way for the payee to know that the previous owners did not sign any earlier transactions. For our purposes, the earliest transaction is the one that counts, so we don't care about later attempts to double-spend. The only way to confirm the absence of a transaction is to be aware of all transactions. In the mint based model, the mint was aware of all transactions and decided which arrived first. To accomplish this without a trusted party, transactions must be publicly announced [1], and we need a system for participants to agree on a single history of the order in which they were received. The payee needs proof that at the time of each transaction, the majority of nodes agreed it was the first received.

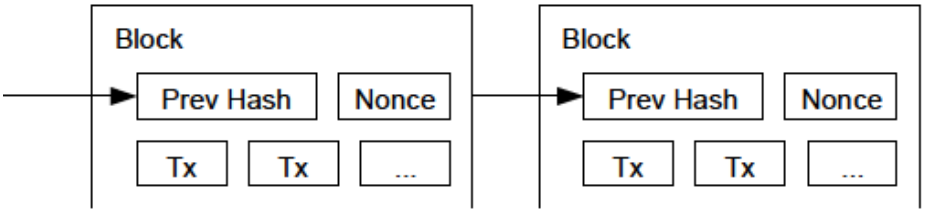
A.1.3. Timestamp Server

The solution we propose begins with a timestamp server. A timestamp server works by taking a hash of a block of items to be timestamped and widely publishing the hash, such as in a newspaper or Usenet post [2-5]. The timestamp proves that the data must have existed at the time, obviously, in order to get into the hash. Each timestamp includes the previous timestamp in its hash, forming a chain, with each additional timestamp reinforcing the ones before it.



A.1.4. Proof-of-Work

To implement a distributed timestamp server on a peer-to-peer basis, we will need to use a proof-of-work system similar to Adam Back's Hashcash [6], rather than newspaper or Usenet posts. The proof-of-work involves scanning for a value that when hashed, such as with SHA-256, the hash begins with a number of zero bits. The average work required is exponential in the number of zero bits required and can be verified by executing a single hash. For our timestamp network, we implement the proof-of-work by incrementing a nonce in the block until a value is found that gives the block's hash the required zero bits. Once the CPU effort has been expended to make it satisfy the proof-of-work, the block cannot be changed without redoing the work. As later blocks are chained after it, the work to change the block would include redoing all the blocks after it.



The proof-of-work also solves the problem of determining representation in majority decision making. If the majority were based on one-IP-address-one-vote, it could be subverted by anyone able to allocate many IPs. Proof-of-work is essentially one-CPU-one-vote. The majority decision is represented by the longest chain, which has the greatest proof-of-work effort invested in it. If a majority of CPU power is controlled by honest nodes, the honest chain will grow the fastest and outpace any competing chains. To modify a past block, an attacker would have to redo the proof-of-work of the block and all blocks after it and then catch up with and surpass the work of the honest nodes. We will show later that the probability of a slower attacker catching up diminishes exponentially as subsequent blocks are added.

To compensate for increasing hardware speed and varying interest in running nodes over time, the proof-of-work difficulty is determined by a moving average targeting an average number of blocks per hour. If they’re generated too fast, the difficulty increases.

A.1.5. Network

The steps to run the network are as follows:

1. New transactions are broadcast to all nodes.
2. Each node collects new transactions into a block.
3. Each node works on finding a difficult proof-of-work for its block.
4. When a node finds a proof-of-work, it broadcasts the block to all nodes.
5. Nodes accept the block only if all transactions in it are valid and not already spent.
6. Nodes express their acceptance of the block by working on creating the next block in the chain, using the hash of the accepted block as the previous hash.

Nodes always consider the longest chain to be the correct one and will keep working on extending it. If two nodes broadcast different versions of the next block simultaneously, some nodes may receive one or the other first. In that case, they work on the first one they received, but save the other branch in case it becomes longer. The tie will be broken when the next proof-of-work is found and one branch becomes longer; the nodes that were working on the other branch will then switch to the longer one.

New transaction broadcasts do not necessarily need to reach all nodes. As long as they reach many nodes, they will get into a block before long. Block broadcasts are also tolerant of dropped messages. If a node does not receive a block, it will request it when it receives the next block and realizes it missed one.

A.1.6. Incentive

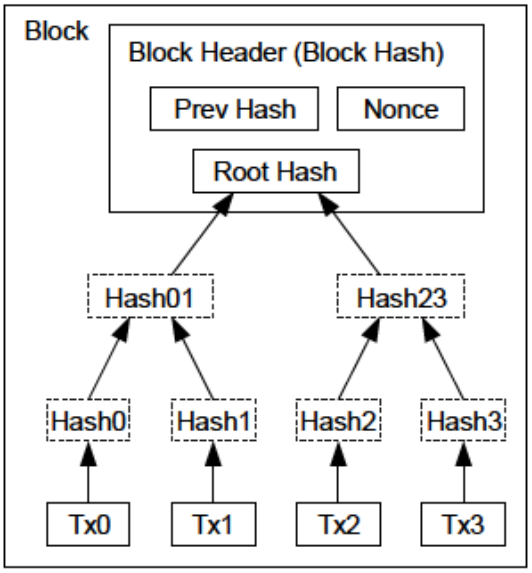
By convention, the first transaction in a block is a special transaction that starts a new coin owned by the creator of the block. This adds an incentive for nodes to support the network, and provides a way to initially distribute coins into circulation, since there is no central authority to issue them. The steady addition of a constant of amount of new coins is analogous to gold miners expending resources to add gold to circulation. In our case, it is CPU time and electricity that is expended.

The incentive can also be funded with transaction fees. If the output value of a transaction is less than its input value, the difference is a transaction fee that is added to the incentive value of the block containing the transaction. Once a predetermined number of coins have entered circulation, the incentive can transition entirely to transaction fees and be completely inflation free.

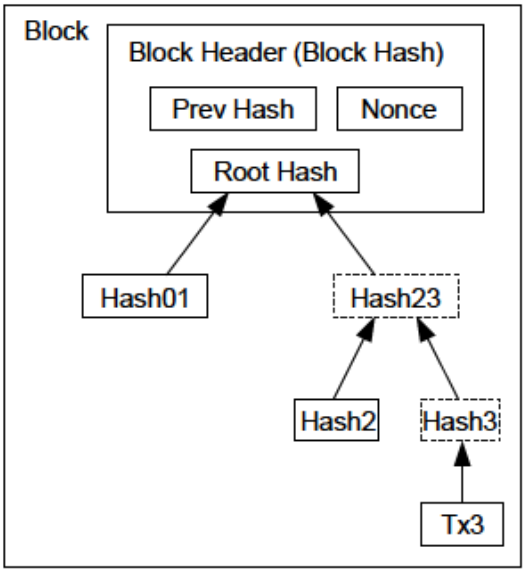
The incentive may help encourage nodes to stay honest. If a greedy attacker is able to assemble more CPU power than all the honest nodes, he would have to choose between using it to defraud people by stealing back his payments, or using it to generate new coins. He ought to find it more profitable to play by the rules, such rules that favour him with more new coins than everyone else combined, than to undermine the system and the validity of his own wealth.

A.1.7. Reclaiming Disk Space

Once the latest transaction in a coin is buried under enough blocks, the spent transactions before it can be discarded to save disk space. To facilitate this without breaking the block's hash, transactions are hashed in a Merkle Tree [7] [2] [5], with only the root included in the block's hash. Old blocks can then be compacted by stubbing off branches of the tree. The interior hashes do not need to be stored.



Transactions Hashed in a Merkle Tree

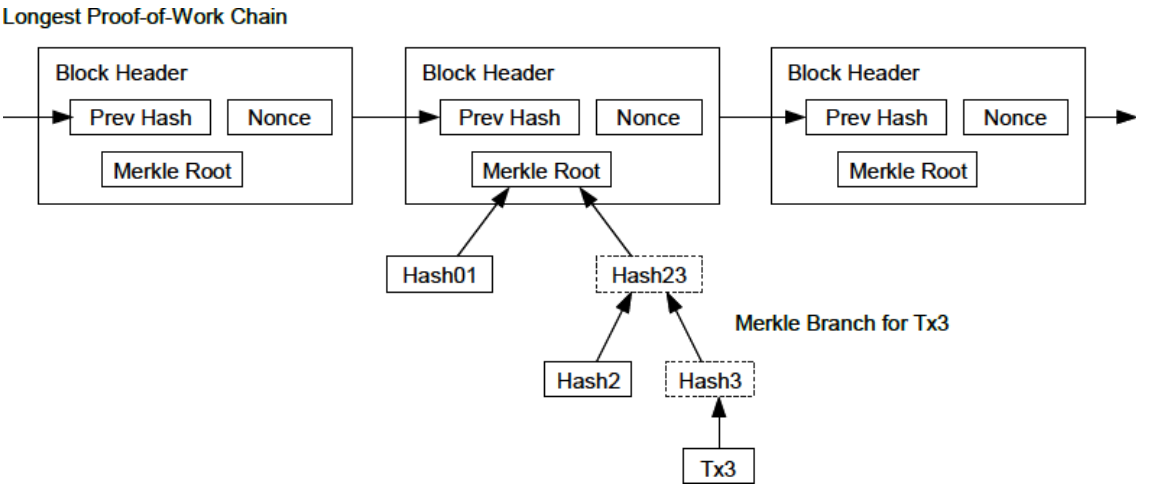


After Pruning Tx0-2 from the Block

A block header with no transactions would be about 80 bytes. If we suppose blocks are generated every 10 minutes, 80 bytes * 6 * 24 * 365 = 4.2MB per year. With computer systems typically selling with 2GB of RAM as of 2008, and Moore's Law predicting current growth of 1.2GB per year, storage should not be a problem even if the block headers must be kept in memory.

A.1.8. Simplified Payment Verification

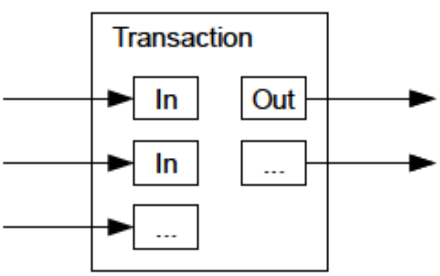
It is possible to verify payments without running a full network node. A user only needs to keep a copy of the block headers of the longest proof-of-work chain, which he can get by querying network nodes until he's convinced he has the longest chain, and obtain the Merkle branch linking the transaction to the block it's timestamped in. He can't check the transaction for himself, but by linking it to a place in the chain, he can see that a network node has accepted it, and blocks added after it further confirm the network has accepted it.



As such, the verification is reliable as long as honest nodes control the network, but is more vulnerable if the network is overpowered by an attacker. While network nodes can verify transactions for themselves, the simplified method can be fooled by an attacker's fabricated transactions for as long as the attacker can continue to overpower the network. One strategy to protect against this would be to accept alerts from network nodes when they detect an invalid block, prompting the user's software to download the full block and alerted transactions to confirm the inconsistency. Businesses that receive frequent payments will probably still want to run their own nodes for more independent security and quicker verification.

A.1.9. Combining and Splitting Value

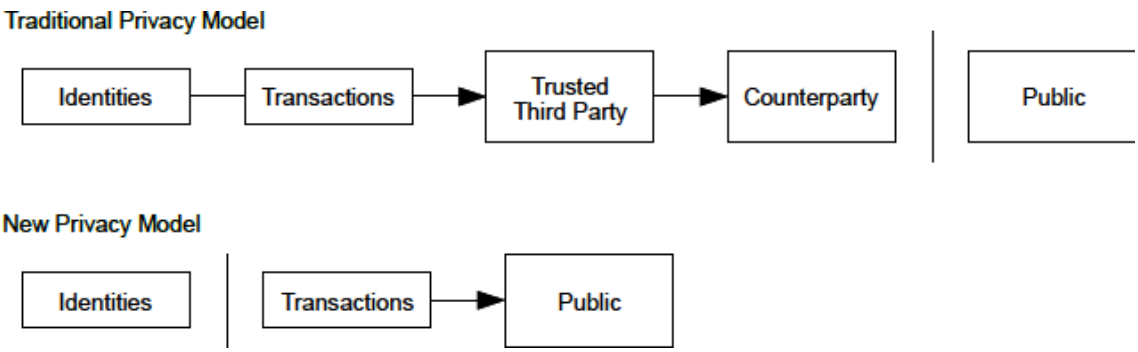
Although it would be possible to handle coins individually, it would be unwieldy to make a separate transaction for every cent in a transfer. To allow value to be split and combined, transactions contain multiple inputs and outputs. Normally there will be either a single input from a larger previous transaction or multiple inputs combining smaller amounts, and at most two outputs: one for the payment, and one returning the change, if any, back to the sender.



It should be noted that fan-out, where a transaction depends on several transactions, and those transactions depend on many more, is not a problem here. There is never the need to extract a complete standalone copy of a transaction’s history.

A.1.10. Privacy

The traditional banking model achieves a level of privacy by limiting access to information to the parties involved and the trusted third party. The necessity to announce all transactions publicly precludes this method, but privacy can still be maintained by breaking the flow of information in another place: by keeping public keys anonymous. The public can see that someone is sending an amount to someone else, but without information linking the transaction to anyone. This is similar to the level of information released by stock exchanges, where the time and size of individual trades, the "tape", is made public, but without telling who the parties were.



As an additional firewall, a new key pair should be used for each transaction to keep them from being linked to a common owner. Some linking is still unavoidable with multi-input transactions, which necessarily reveal that their inputs were owned by the same owner. The risk is that if the owner of a key is revealed, linking could reveal other transactions that belonged to the same owner.

A.1.11. Calculations

We consider the scenario of an attacker trying to generate an alternate chain faster than the honest chain. Even if this is accomplished, it does not throw the system open to arbitrary changes, such as creating value out of thin air or taking money that never belonged to the attacker. Nodes are not going to accept an invalid transaction as payment, and honest nodes will never accept a block containing them. An attacker can only try to change one of his own transactions to take back money he recently spent.

The race between the honest chain and an attacker chain can be characterized as a Binomial Random Walk. The success event is the honest chain being extended by one block, increasing its lead by +1, and the failure event is the attacker’s chain being

extended by one block, reducing the gap by -1.

The probability of an attacker catching up from a given deficit is analogous to a Gambler's Ruin problem. Suppose a gambler with unlimited credit starts at a deficit and plays potentially an infinite number of trials to try to reach breakeven. We can calculate the probability he ever reaches breakeven, or that an attacker ever catches up with the honest chain, as follows [8]:

p = probability an honest node finds the next block

q = probability the attacker finds the next block

q_z = probability the attacker will ever catch up from z blocks behind

$$q_z = \begin{cases} 1 & \text{if } p \leq q \\ (q/p)^z & \text{if } p > q \end{cases}$$

Given our assumption that p > q, the probability drops exponentially as the number of blocks the attacker has to catch up with increases. With the odds against him, if he doesn't make a lucky lunge forward early on, his chances become vanishingly small as he falls further behind.

We now consider how long the recipient of a new transaction needs to wait before being sufficiently certain the sender can't change the transaction. We assume the sender is an attacker who wants to make the recipient believe he paid him for a while, then switch it to pay back to himself after some time has passed. The receiver will be alerted when that happens, but the sender hopes it will be too late.

The receiver generates a new key pair and gives the public key to the sender shortly before signing. This prevents the sender from preparing a chain of blocks ahead of time by working on it continuously until he is lucky enough to get far enough ahead, then executing the transaction at that moment. Once the transaction is sent, the dishonest sender starts working in secret on a parallel chain containing an alternate version of his transaction.

The recipient waits until the transaction has been added to a block and z blocks have been linked after it. He doesn't know the exact amount of progress the attacker has made, but assuming the honest blocks took the average expected time per block, the attacker's potential progress will be a Poisson distribution with expected value:

$$\lambda = z\frac{q}{p}$$

To get the probability the attacker could still catch up now, we multiply the Poisson density for each amount of progress he could have made by the probability he could catch up from that point:

$$\sum_{k=0}^{\infty} \frac{\lambda^k e^{-\lambda}}{k!} \cdot \begin{cases} (q/p)^{(z-k)} & \text{if } k \leq z \\ 1 & \text{if } k > z \end{cases}$$

Rearranging to avoid summing the infinite tail of the distribution...

$$1 - \sum_{k=0}^{z} \frac{\lambda^k e^{-\lambda}}{k!}\left(1 - (q/p)^{(z-k)}\right)$$

Converting to C code...

```
#include <math.h>
double AttackerSuccessProbability(double q, int z)
{
    double p = 1.0 - q;
    double lambda = z * (q / p);
    double sum = 1.0;
    int i, k;
    for (k = 0; k <= z; k++)
    {
        double poisson = exp(-lambda);
        for (i = 1; i <= k; i++)
            poisson *= lambda / i;
        sum -= poisson * (1 - pow(q / p, z - k));
    }
    return sum;
}
```

Running some results, we can see the probability drop off exponentially with z.

```
q=0.1
z=0 P=1.0000000
z=1 P=0.2045873
z=2 P=0.0509779
z=3 P=0.0131722
z=4 P=0.0034552
z=5 P=0.0009137
z=6 P=0.0002428
z=7 P=0.0000647
z=8 P=0.0000173
z=9 P=0.0000046
z=10 P=0.0000012
```

```
q=0.3
z=0 P=1.0000000
z=5 P=0.1773523
z=10 P=0.0416605
z=15 P=0.0101008
z=20 P=0.0024804
z=25 P=0.0006132
z=30 P=0.0001522
z=35 P=0.0000379
z=40 P=0.0000095
z=45 P=0.0000024
z=50 P=0.0000006
```

Solving for P less than 0.1%...

```
P < 0.001
q=0.10 z=5
q=0.15 z=8
```

```
q=0.20 z=11
q=0.25 z=15
q=0.30 z=24
q=0.35 z=41
q=0.40 z=89
q=0.45 z=340
```

A.1.12. Conclusion

We have proposed a system for electronic transactions without relying on trust. We started with the usual framework of coins made from digital signatures, which provides strong control of ownership, but is incomplete without a way to prevent double-spending. To solve this, we proposed a peer-to-peer network using proof-of-work to record a public history of transactions that quickly becomes computationally impractical for an attacker to change if honest nodes control a majority of CPU power. The network is robust in its unstructured simplicity. Nodes work all at once with little coordination. They do not need to be identified, since messages are not routed to any particular place and only need to be delivered on a best effort basis. Nodes can leave and rejoin the network at will, accepting the proof-of-work chain as proof of what happened while they were gone. They vote with their CPU power, expressing their acceptance of valid blocks by working on extending them and rejecting invalid blocks by refusing to work on them. Any needed rules and incentives can be enforced with this consensus mechanism.

A.1.13. References

[1] W. Dai, "b-money," *http://www.weidai.com/bmoney.txt*, 1998.

[2] H. Massias, X.S. Avila, and J.-J. Quisquater, "Design of a secure timestamping service with minimal trust requirements," In 20th Symposium on Information Theory in the Benelux, May 1999.

[3] S. Haber, W.S. Stornetta, "How to time-stamp a digital document," In Journal of Cryptology, vol 3, no 2, pages 99-111, 1991.

[4] D. Bayer, S. Haber, W.S. Stornetta, "Improving the efficiency and reliability of digital time-stamping," In Sequences II: Methods in Communication, Security and Computer Science, pages 329-334, 1993.

[5] S. Haber, W.S. Stornetta, "Secure names for bit-strings," In Proceedings of the 4th ACM Conference on Computer and Communications Security, pages 28-35, April 1997.

[6] A. Back, "Hashcash - a denial of service counter-measure," *http://www.hashcash.org/papers/hashcash.pdf*, 2002.

[7] R.C. Merkle, "Protocols for public key cryptosystems," In Proc. 1980 Symposium on Security and Privacy, IEEE Computer Society, pages 122-133, April 1980.

[8] W. Feller, "An introduction to probability theory and its applications," 1957.

A.2. License

This whitepaper was published in October 2008 by Satoshi Nakamoto. It was later (2009) added as supporting documentation to the bitcoin software and carries the same MIT license. It has been reproduced in this book, without modification other than formatting, under the terms of the MIT license:

The MIT License (MIT) Copyright (c) 2008 Satoshi Nakamoto

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS," WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Appendix B: Errata to the Bitcoin Whitepaper

This appendix contains a description of known problems in Satoshi Nakamoto's paper, "Bitcoin: A Peer-to-Peer Electronic Cash System," as well as notes on terminology changes and how Bitcoin's implementation differs from that described in the paper.

This document was originally published by a coauthor of this book in 2016; it is reproduced here with updates. The names of sections in this errata correspond to the names of the sections in Nakamoto's original paper.

B.1. Abstract

> "The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power."

- **Implementation detail:** If each link in the chain (called "blocks" in Bitcoin) was built using the same amount of *proof of work* (PoW), the longest chain would be the one backed by the largest pool of computational power. However, Bitcoin was implemented in such a way that the amount of PoW can vary between blocks, so it became important not to check for the "the longest chain" but rather "the chain demonstrating the most PoW"; this is often shortened to "most-work chain."

  The change from checking for the longest chain to checking for the most-work chain occurred in July 2010, long after Bitcoin's initial release:

```
-    if (pindexNew->nHeight > nBestHeight)
+    if (pindexNew->bnChainWork > bnBestChainWork)
```

- **Terminology change:** General CPUs were used to generate the PoW for the earliest Bitcoin blocks, but PoW generation today is mostly performed by specialist Application Specific Integrated Circuits (ASICs), so instead of saying "CPU power" it is perhaps more correct to say "computational power" or, simply, "hash rate" for the hashing used in generating the PoW.

  > "As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers."

- **Terminology change:** The term "nodes" today is used to refer to full validation nodes, which are programs that enforce all the rules of the system. Programs (and hardware) that extend the chain today are called "miners" based on Nakamoto's analogy to gold miners in section 6 of the paper. Nakamoto expected all miners to be

nodes but the software he released did not require all nodes to be miners. In the original software, a simple menu item in the node GUI allowed toggling the mining function on or off.

Today it is the case that the overwhelming number of nodes are not miners and that many individuals who own mining hardware do not use it with their own nodes (and even those that do mine with their own nodes often mine for short periods of time on top of newly discovered blocks without ensuring their node considers the new block valid). The early parts of the paper where "nodes" is mostly used without modification refer to mining using a full validation node; the later parts of the paper which refer to "network nodes" is mainly about what nodes can do even if they aren't mining.

- **Post-publication discovery:** When a new block is produced, the miner who produces that block can begin working on its sequel immediately but all other miners are unaware of the new block and cannot begin working on it until it has propagated across the network to them. This gives miners who produce many blocks an edge over miners who produce fewer blocks, and this can be exploited in what's known as the *selfish mining attack* to allow an attacker with around 30% of total network hash rate to make other miners less profitable, perhaps driving them into following the attacking miner's policy. So instead of saying "a majority of CPU power is controlled by nodes that are not cooperating to attack the network," it is perhaps more correct to say "as long as nodes cooperating to attack the network control less than about 30% of the network."

B.2. Transactions

"We define an electronic coin as a chain of digital signatures. Each owner transfers the coin to the next by digitally signing a hash of the previous transaction and the public key of the next owner and adding these to the end of the coin."

- **Implementation detail:** Bitcoin implements a more general version of this system where digital signatures are not used directly but rather a "deterministic expression" is used instead. Just as a signature that matches a known public key can be used to enable a payment, the data that satisfies a known expression can also enable a payment. Generically, the expression that must be satisfied in Bitcoin in order to spend a coin is known as an "encumbrance." Almost all encumbrances in Bitcoin to date require providing at least one signature. So instead of saying "a chain of digital signatures," it is more correct to say "a chain of encumbrances." Given that transactions often have more than one input and more than one output, the structure is not very chain-like; it's more accurately described as a directed acyclic graph (DAG).

B.3. Proof of Work

"…we implement the proof-of-work by incrementing a nonce in the block until a value is found that gives the block's hash the required zero bits."

- **Implementation detail:** Adam Back's Hashcash implementation requires finding a hash with the required number of leading zero bits. Bitcoin treats the hash as an integer and requires that it be less than a specified integer, which effectively allows a fractional number of bits to be specified.

  "Proof-of-work is essentially one-CPU-one-vote."

- **Important note:** The vote here is not on the rules of the system but merely on the ordering of the transactions in order to provide assurances that an "electronic coin" cannot be easily double spent. This is described in more detail in section 11 of the paper where it says, "We consider the scenario of an attacker trying to generate an alternate chain faster than the honest chain. Even if this is accomplished, it does not throw the system open to arbitrary changes, such as creating value out of thin air or taking money that never belonged to the attacker. Nodes are not going to accept an invalid transaction as payment, and honest nodes will never accept a block containing them."

  "…proof-of-work difficulty is determined by a moving average targeting an average number of blocks per hour."

- **Implementation detail:** A moving average is not used. Instead, every 2,016th block has its reported generation time compared to the generation time for an earlier block, and the difference between them is used to calculate the average used for adjustment.

  Further, the average implemented in Bitcoin targets an average number of blocks per two weeks (not per hour as might be implied by the text). Other implemented rules may further slow adjustments, such as a rule that the adjustment cannot increase block production speed by more than 300% per period, nor slow it by more than 75%.

B.4. Reclaiming Disk Space

"Once the latest transaction in a coin is buried under enough blocks, the spent transactions before it can be discarded to save disk space."

- **Possible post-publication discovery:** Although the merkle tree structure described in this section can prove a transaction was included in a particular block, there is currently no way in Bitcoin to prove that a transaction has not been spent except to

process all subsequent data in the blockchain. This means the method described here cannot be universally used for reclaiming disk space among all nodes, as all new nodes will need to process all transactions.

B.5. Simplified Payment Verification

"One strategy to protect against this would be to accept alerts from network nodes when they detect an invalid block, prompting the user's software to download the full block and alerted transactions to confirm the inconsistency."

- **Important Note:** Although software has been produced that implements some parts of this section and calls that Simplified Payment Verification (SPV), none of these programs currently accepts alerts from network nodes (full validation nodes) when invalid blocks have been detected. This has placed bitcoins in so-called SPV wallets at risk in the past.

B.6. Privacy

"Some linking is still unavoidable with multi-input transactions, which necessarily reveal that their inputs were owned by the same owner."

- **Post-publication invention:** It isn't clear that different inputs in the same transaction have the same owner if owners often mix their inputs with inputs belonging to other owners. For example, there's no public difference between Alice and Bob each contributing one of their inputs toward paying Charlie and Dan than there is between just Alice contributing two of her inputs toward paying Charlie and Dan.

  This technique is known today as CoinJoin, and software implementing it has been in use since 2015.

B.7. Calculations

"The receiver generates a new key pair and gives the public key to the sender shortly before signing. This prevents the sender from preparing a chain of blocks ahead of time by working on it continuously until he is lucky enough to get far enough ahead, then executing the transaction at that moment."

- **Post-publication discovery:** Nothing about the receiver generating a public key shortly before the spender signs a transaction prevents the spender from preparing a chain of blocks ahead of time. Early Bitcoin user Hal Finney discovered this attack and described it: "Suppose the attacker is generating blocks occasionally. In each block he generates, he includes a transfer from address A to address B, both of which he controls.

"To cheat you, when he generates a block, he doesn’t broadcast it. Instead, he runs down to your store and makes a payment to your address C with his address A. You wait a few seconds, don’t hear anything, and transfer the goods. He broadcasts his block now, and his transaction will take precedence over yours."

The attack works for any number of confirmations, and is sometimes named the Finney Attack.

Disclaimer: The author of this document was not the first person to identify any of the problems described here—he has merely collected them into a single document.

License: This errata document is released under the CC0 1.0 Universal Public Domain Dedication

For updates made after the publication of this book, please see the Original document.

Appendix C: Bitcoin Improvement Proposals

Bitcoin Improvement Proposals are design documents providing information to the Bitcoin community or describing a new feature for Bitcoin or its processes or environment.

As per BIP1 *BIP Purpose and Guidelines*, there are three kinds of BIPs:

Standard BIP
: Describes any change that affects most or all Bitcoin implementations, such as a change to the network protocol, a change in block or transaction validity rules, or any change or addition that affects the interoperability of applications using Bitcoin.

Informational BIP
: Describes a Bitcoin design issue or provides general guidelines or information to the Bitcoin community, but does not propose a new feature. Informational BIPs do not necessarily represent a Bitcoin community consensus or recommendation, so users and implementors may ignore informational BIPs or follow their advice.

Process BIP
: Describes a Bitcoin process or proposes a change to (or an event in) a process. Process BIPs are like standard BIPs but apply to areas other than the Bitcoin protocol itself. They might propose an implementation but not to Bitcoin's codebase; they often require community consensus. Unlike informational BIPs, they are more than recommendations, and users are typically not free to ignore them. Examples include procedures, guidelines, changes to the decision-making process, and changes to the tools or environment used in Bitcoin development. Any meta-BIP is also considered a process BIP.

BIPs are recorded in a versioned repository on GitHub. An MIT-licensed document from the open source Bitcoin Core project, reproduced here in edited form, describes which BIPs it implements, including listing the Pull Request (PR) and version of Bitcoin Core where support for each BIP was added or significantly changed.

BIPs that are implemented by Bitcoin Core:

- BIP9: The changes allowing multiple soft forks to be deployed in parallel have been implemented since v0.12.1 (PR #7575).
- BIP11: Multisig outputs are standard since v0.6.0 (PR #669).
- BIP13: The address format for P2SH addresses has been implemented since v0.6.0 (PR #669).
- BIP14: The subversion string is being used as User Agent since v0.6.0 (PR #669).

- BIP16: The pay-to-script-hash evaluation rules have been implemented since v0.6.0, and took effect on April 1st 2012 (PR #748).
- BIP21: The URI format for Bitcoin payments has been implemented since v0.6.0 (PR #176).
- BIP22: The 'getblocktemplate' (GBT) RPC protocol for mining has been implemented since v0.7.0 (PR #936).
- BIP23: Some extensions to GBT have been implemented since v0.10.0rc1, including longpolling and block proposals (PR #1816).
- BIP30: The evaluation rules to forbid creating new transactions with the same txid as previous not-fully-spent transactions were implemented since v0.6.0, and the rule took effect on March 15th 2012 (PR #915).
- BIP31: The 'pong' protocol message (and the protocol version bump to 60001) has been implemented since v0.6.1 (PR #1081).
- BIP32: Hierarchical Deterministic Wallets has been implemented since v0.13.0 (PR #8035).
- BIP34: The rule that requires blocks to contain their height (number) in the coinbase input, and the introduction of version 2 blocks has been implemented since v0.7.0. The rule took effect for version 2 blocks as of block 224413 (March 5th 2013), and version 1 blocks are no longer allowed since block 227931 (March 25th 2013) (PR #1526).
- BIP35: The 'mempool' protocol message (and the protocol version bump to 60002) has been implemented since v0.7.0 (PR #1641). As of v0.13.0, this is only available for NODE_BLOOM (BIP111) peers.
- BIP37: The bloom filtering for transaction relaying, partial Merkle trees for blocks, and the protocol version bump to 70001 (enabling low-bandwidth lightweight clients) has been implemented since v0.8.0 (PR #1795). Disabled by default since v0.19.0, can be enabled by the -peerbloomfilters option.
- BIP42: The bug that would have caused the subsidy schedule to resume after block 13440000 was fixed in v0.9.2 (PR #3842).
- BIP43: The experimental descriptor wallets introduced in v0.21.0 by default use the Hierarchical Deterministic Wallet derivation proposed by BIP43 (PR #16528).
- BIP44: The experimental descriptor wallets introduced in v0.21.0 by default use the Hierarchical Deterministic Wallet derivation proposed by BIP44 (PR #16528).
- BIP49: The experimental descriptor wallets introduced in v0.21.0 by default use the Hierarchical Deterministic Wallet derivation proposed by BIP49 (PR #16528).

- BIP61: The 'reject' protocol message (and the protocol version bump to 70002) was added in v0.9.0 (PR #3185). Starting v0.17.0, whether to send reject messages can be configured with the -enablebip61 option, and support is deprecated (disabled by default) as of v0.18.0. Support was removed in v0.20.0 (PR #15437).
- BIP65: The CHECKLOCKTIMEVERIFY soft fork was merged in v0.12.0 (PR #6351), and backported to v0.11.2 and v0.10.4. Mempool-only CLTV was added in PR #6124.
- BIP66: The strict DER rules and associated version 3 blocks have been implemented since v0.10.0 (PR #5713).
- BIP68: Sequence locks have been implemented as of v0.12.1 (PR #7184), and have been buried since v0.19.0 (PR #16060).
- BIP70 71 72: Payment Protocol support has been available in Bitcoin Core GUI since v0.9.0 (PR #5216). Support can be optionally disabled at build time since v0.18.0 (PR 14451), and it is disabled by default at build time since v0.19.0 (PR #15584). It has been removed as of v0.20.0 (PR 17165).
- BIP84: The experimental descriptor wallets introduced in v0.21.0 by default use the Hierarchical Deterministic Wallet derivation proposed by BIP84. (PR #16528)
- BIP86: Descriptor wallets by default use the Hierarchical Deterministic Wallet derivation proposed by BIP86 since v23.0 (PR #22364).
- BIP90: Trigger mechanism for activation of BIPs 34, 65, and 66 has been simplified to block height checks since v0.14.0 (PR #8391).
- BIP111: NODE_BLOOM service bit added and enforced for all peer versions as of v0.13.0 (PR #6579 and PR #6641).
- BIP112: The CHECKSEQUENCEVERIFY opcode has been implemented since v0.12.1 (PR #7524), and has been buried since v0.19.0 (PR #16060).
- BIP113: Median time past lock-time calculations have been implemented since v0.12.1 (PR #6566), and has been buried since v0.19.0 (PR #16060).
- BIP125: Opt-in full replace-by-fee signaling partially implemented.
- BIP130: direct headers announcement is negotiated with peer versions ≥70012 as of v0.12.0 (PR 6494).
- BIP133: feefilter messages are respected and sent for peer versions ≥70013 as of v0.13.0 (PR 7542).
- BIP141: Segregated Witness (Consensus Layer) as of v0.13.0 (PR 8149), defined for mainnet as of v0.13.1 (PR 8937), and buried since v0.19.0 (PR #16060).
- BIP143: Transaction Signature Verification for Version 0 Witness Program as of v0.13.0 (PR 8149), defined for mainnet as of v0.13.1 (PR 8937), and buried since v0.19.0 (PR #16060).
- BIP144: Segregated Witness as of 0.13.0 (PR 8149).

- BIP145: getblocktemplate updates for Segregated Witness as of v0.13.0 (PR 8149).
- BIP147: NULLDUMMY soft fork as of v0.13.1 (PR 8636 and PR 8937), buried since v0.19.0 (PR #16060).
- BIP152: Compact block transfer and related optimizations are used as of v0.13.0 (PR 8068).
- BIP155: The 'addrv2' and 'sendaddrv2' messages which enable relay of Tor V3 addresses (and other networks) are supported as of v0.21.0 (PR 19954).
- BIP157 158: Compact Block Filters for Light Clients can be indexed as of v0.19.0 (PR #14121) and served to peers on the P2P network as of v0.21.0 (PR #16442).
- BIP159: The NODE_NETWORK_LIMITED service bit is signalled as of v0.16.0 (PR 11740), and such nodes are connected to as of v0.17.0 (PR 10387).
- BIP173: Bech32 addresses for native Segregated Witness outputs are supported as of v0.16.0 (PR 11167). Bech32 addresses are generated by default as of v0.20.0 (PR 16884).
- BIP174: RPCs to operate on Partially Signed Bitcoin Transactions (PSBT) are present as of v0.17.0 (PR 13557).
- BIP176: Bits Denomination [QT only] is supported as of v0.16.0 (PR 12035).
- BIP325: Signet test network is supported as of v0.21.0 (PR 18267).
- BIP339: Relay of transactions by wtxid is supported as of v0.21.0 (PR 18044).
- BIP340 341 342: Validation rules for Taproot (including Schnorr signatures and Tapscript leaves) are implemented as of v0.21.0 (PR 19953), with mainnet activation as of v0.21.1 (PR 21377, PR 21686).
- BIP350: Addresses for native v1+ segregated Witness outputs use bech32m instead of bech32 as of v22.0 (PR 20861).
- BIP371: Taproot fields for PSBT as of v24.0 (PR 22558).
- BIP380 381 382 383 384 385: Output Script Descriptors, and most of Script Expressions are implemented as of v0.17.0 (PR 13697).
- BIP386: tr() Output Script Descriptors are implemented as of v22.0 (PR 22051).

Index

- 51% attacks > id=attack-51 1
- 51% attacks > startref=attack-51 1
- absolute timelocks 1
- abstract syntax trees (AST) 1
- absurd fees 1
- acquiring bitcoins > id=acquire-bitcoin 1
- acquiring bitcoins > startref=acquire-bitcoin 1
- activation (soft forks) > BIP34 > secondary-sortas=BIP034 > id=activation-bip34 1
- activation (soft forks) > BIP34 > secondary-sortas=BIP034 > startref=activation-bip34 1
- activation (soft forks) > BIP8 > secondary-sortas=BIP008 > id=activate-soft-fork-bip8 1
- activation (soft forks) > BIP8 > secondary-sortas=BIP008 > startref=activate-soft-fork-bip8 1
- activation (soft forks) > BIP9 > secondary-sortas=BIP009 > id=activation-bip9 1
- activation (soft forks) > BIP9 > secondary-sortas=BIP009 > startref=activation-bip9 1
- activation (soft forks) > speedy trial > id=activation-speed 1
- additivity 1
- addresses > bech32 > advantages of > id=address-bech32-adv 1
- addresses > bech32 > advantages of > startref=address-bech32-adv 1
- addresses > bech32 > problems with > id=address-bech32-prob 1
- addresses > bech32 > problems with > startref=address-bech32-prob 1
- addresses > bech32m > id=address-bech32m 1
- addresses > bech32m > startref=address-bech32m 1
- addresses > explained 1
- addresses > multisignature 1
- addresses > P2PK (pay to public key) > id=address-p2pk-ch4 1
- addresses > P2PK (pay to public key) > startref=address-p2pk-ch4 1
- addresses > P2PKH (pay to public key hash) 1
- addresses > P2PKH (pay to public key hash) > id=address-p2pkh-legacy 1
- addresses > P2PKH (pay to public key hash) > startref=address-p2pkh-legacy 1
- addresses > P2SH (pay to script hash) > id=address-p2sh-ch4 1
- addresses > P2SH (pay to script hash) > id=address-p2sh-ch7 1
- addresses > P2SH (pay to script hash) > startref=address-p2sh-ch4 1
- addresses > P2SH (pay to script hash) > startref=address-p2sh-ch7 1
- addresses > P2WPKH (pay to witness public key hash) > id=address-p2wpkh 1
- addresses > P2WPKH (pay to witness public key hash) > nesting > id=address-p2wpkh-nest 1
- addresses > P2WPKH (pay to witness public key hash) > nesting > startref=address-p2wpkh-nest 1
- addresses > P2WPKH (pay to witness public key hash) > startref=address-p2wpkh 1
- addresses > P2WSH (pay to witness script hash) > id=address-p2wsh 1
- addresses > P2WSH (pay to witness script hash) > nesting > id=address-p2wsh-nest 1
- addresses > P2WSH (pay to witness script hash) > nesting > startref=address-p2wsh-nest 1
- addresses > P2WSH (pay to witness script hash) > startref=address-p2wsh 1
- addresses > vanity > id=address-vanity 1
- addresses > vanity > startref=address-vanity 1
- Aezeed recovery codes 1
- aggregated public keys 1
- alertnotify option (bitcoind option) 1
- amount field (transaction outputs) > id=amount-field 1
- amount field (transaction outputs) > startref=amount-field 1
- ancestor fee rate mining 1
- anchor outputs (CPFP) 1
- API access > id=api 1
- API access > startref=api 1
- application platform, Bitcoin as > colored coins application > id=app-platform-color 1
- application platform, Bitcoin as > colored coins application > startref=app-platform-color 1
- application platform, Bitcoin as > example applications > id=app-platform-example 1

- application platform, Bitcoin as > example applications > startref=app-platform-example 1
- application platform, Bitcoin as > payment channels > id=app-platform-payment 1
- application platform, Bitcoin as > payment channels > startref=app-platform-payment 1
- application platform, Bitcoin as > primitives, list of > id=app-platform-primitive 1
- application platform, Bitcoin as > primitives, list of > startref=app-platform-primitive 1
- application platform, Bitcoin as > routed payment channels (Lightning Network) > id=app-platform-ln 1
- application platform, Bitcoin as > routed payment channels (Lightning Network) > startref=app-platform-ln 1
- archival full nodes 1
- AST (abstract syntax trees) 1
- asymmetric cryptography > see=public key cryptography 1
- asymmetric revocable commitments > id=asymmetric-revoke-commit 1
- asymmetric revocable commitments > startref=asymmetric-revoke-commit 1
- authentication 1 2
- authentication path 1
- authorization 1
- backing up > importance of 1
- backing up > key derivation paths > id=backup-key-derive 1
- backing up > key derivation paths > startref=backup-key-derive 1
- backing up > nonkey data > id=backup-nonkey 1
- backing up > nonkey data > startref=backup-nonkey 1
- backing up > recovery codes > see=recovery codes 1
- balanced merkle trees 1
- base58check encoding 1
- base58check encoding > id=base58-ch4 1
- base58check encoding > startref=base58-ch4 1
- base64 encoding 1
- batch verification of digital signatures 1
- bcoin 1
- bech32 addresses > advantages of > id=bech32-adv 1
- bech32 addresses > advantages of > startref=bech32-adv 1
- bech32 addresses > bech32m > id=bech32-bech32m 1
- bech32 addresses > bech32m > startref=bech32-bech32m 1
- bech32 addresses > problems with > id=bech32-prob 1
- bech32 addresses > problems with > startref=bech32-prob 1
- bech32m addresses > id=bech32m 1
- bech32m addresses > startref=bech32m 1
- best practices, security > id=best-practice-security 1
- best practices, security > startref=best-practice-security 1
- binary hash trees 1
- Binomial Random Walk 1
- BIP118 SIGHASH flags 1
- BIP144 extended serialization format 1
- BIP148 activation of segwit 1
- BIP32 HD (hierarchical deterministic) key generation > primary-sortas=BIP032 1
- BIP32 HD (hierarchical deterministic) key generation > primary-sortas=BIP032 > id=bip32 1
- BIP32 HD (hierarchical deterministic) key generation > primary-sortas=BIP032 > startref=bip32 1
- BIP34 signaling/activation > primary-sortas=BIP034 > id=bip34 1
- BIP34 signaling/activation > primary-sortas=BIP034 > startref=bip34 1
- BIP39 recovery codes > generating > primary-sortas=BIP039 > id=bip39-recovery-generate 1
- BIP39 recovery codes > generating > primary-sortas=BIP039 > startref=bip39-recovery-generate 1
- BIP39 recovery codes > passphrases > primary-sortas=BIP039 > id=bip39-recovery-passphrase 1
- BIP39 recovery codes > passphrases > primary-sortas=BIP039 > startref=bip39-recovery-passphrase 1
- BIP39 recovery codes > primary-sortas=BIP039 1
- BIP39 recovery codes > primary-sortas=BIP039 > id=bip39-recovery 1
- BIP39 recovery codes > primary-sortas=BIP039 > startref=bip39-recovery 1
- BIP39 recovery codes > seed generation > primary-sortas=BIP039 > id=bip39-recovery-seed 1
- BIP39 recovery codes > seed generation > primary-sortas=BIP039 > startref=bip39-recovery-seed 1
- BIP43 HD wallet tree structure > primary-sortas=BIP043 1
- BIP44 HD wallet tree structure > primary-sortas=BIP044 > id=bip44 1

- BIP44 HD wallet tree structure > primary-sortas=BIP044 > startref=bip44 1
- BIP8 mandatory lock-in > primary-sortas=BIP008 > id=bip8 1
- BIP8 mandatory lock-in > primary-sortas=BIP008 > startref=bip8 1
- BIP9 signaling/activation > primary-sortas=BIP009 > id=bip9-ch12 1
- BIP9 signaling/activation > primary-sortas=BIP009 > startref=bip9-ch12 1
- BIPs (Bitcoin Improvement Proposals) 1
- BIPs (Bitcoin Improvement Proposals) > implemented by Bitcoin Core > id=bips-implement 1
- BIPs (Bitcoin Improvement Proposals) > implemented by Bitcoin Core > startref=bips-implement 1
- BIPs (Bitcoin Improvement Proposals) > types of 1
- Bitcoin > as application platform > colored coins application > secondary-sortas=application platform > id=bitcoin-app-platform-color 1
- Bitcoin > as application platform > colored coins application > secondary-sortas=application platform > startref=bitcoin-app-platform-color 1
- Bitcoin > as application platform > example applications > secondary-sortas=application platform > id=bitcoin-app-platform-example 1
- Bitcoin > as application platform > example applications > secondary-sortas=application platform > startref=bitcoin-app-platform-example 1
- Bitcoin > as application platform > payment channels > secondary-sortas=application platform > id=bitcoin-app-platform-payment 1
- Bitcoin > as application platform > payment channels > secondary-sortas=application platform > startref=bitcoin-app-platform-payment 1
- Bitcoin > as application platform > primitives, list of > secondary-sortas=application platform > id=bitcoin-app-platform-primitive 1
- Bitcoin > as application platform > primitives, list of > secondary-sortas=application platform > startref=bitcoin-app-platform-primitive 1
- Bitcoin > as application platform > routed payment channels (Lightning Network) > secondary-sortas=application platform > id=bitcoin-app-platform-ln 1
- Bitcoin > as application platform > routed payment channels (Lightning Network) > secondary-sortas=application platform > startref=bitcoin-app-platform-ln 1
- Bitcoin > as peer-to-peer network > secondary-sortas=peer-to-peer network 1 2
- Bitcoin > as peer-to-peer network > secondary-sortas=peer-to-peer network > seealso=Bitcoin network 1
- Bitcoin > economics of > id=bitcoin-economics 1
- Bitcoin > economics of > startref=bitcoin-economics 1
- Bitcoin > history of 1
- Bitcoin > operational overview > id=bitcoin-operational-overview 1
- Bitcoin > operational overview > id=bitcoin-operational-overview-ch2 1
- Bitcoin > operational overview > startref=bitcoin-operational-overview 1
- Bitcoin > operational overview > startref=bitcoin-operational-overview-ch2 1
- Bitcoin > security > best practices > id=bitcoin-security-best-practice 1
- Bitcoin > security > best practices > startref=bitcoin-security-best-practice 1
- Bitcoin > security > principles of > id=bitcoin-security-principle 1
- Bitcoin > security > principles of > startref=bitcoin-security-principle 1
- Bitcoin > wallets > see=wallets 1
- Bitcoin Core 1
- Bitcoin Core > authentication 1
- Bitcoin Core > BIPs implemented by > id=bitcoin-core-bips 1
- Bitcoin Core > BIPs implemented by > startref=bitcoin-core-bips 1
- Bitcoin Core > command-line interface > API access > id=bitcoin-core-command-api 1
- Bitcoin Core > command-line interface > API access > startref=bitcoin-core-command-api 1
- Bitcoin Core > command-line interface > exploring blocks > id=bitcoin-core-command-blocks 1
- Bitcoin Core > command-line interface > exploring blocks > id=bitcoin-core-command-blocks2 1
- Bitcoin Core > command-line interface > exploring blocks > startref=bitcoin-core-command-blocks 1
- Bitcoin Core > command-line interface > exploring blocks > startref=bitcoin-core-command-blocks2 1
- Bitcoin Core > command-line interface > exploring/decoding transactions > id=bitcoin-core-command-transaction 1
- Bitcoin Core > command-line interface > exploring/decoding transactions > id=bitcoin-core-command-transaction2 1

- Bitcoin Core > command-line interface > exploring/decoding transactions > startref=bitcoin-core-command-transaction 1
- Bitcoin Core > command-line interface > exploring/decoding transactions > startref=bitcoin-core-command-transaction2 1
- Bitcoin Core > command-line interface > help command > id=bitcoin-core-command-help 1
- Bitcoin Core > command-line interface > help command > startref=bitcoin-core-command-help 1
- Bitcoin Core > command-line interface > status information > id=bitcoin-core-command-status 1
- Bitcoin Core > command-line interface > status information > startref=bitcoin-core-command-status 1
- Bitcoin Core > compiling from source code > building executables 1
- Bitcoin Core > compiling from source code > configuring build > id=bitcoin-core-compile-configure 1
- Bitcoin Core > compiling from source code > configuring build > startref=bitcoin-core-compile-configure 1
- Bitcoin Core > compiling from source code > id=bitcoin-core-compile 1
- Bitcoin Core > compiling from source code > selecting release version > id=bitcoin-core-compile-release 1
- Bitcoin Core > compiling from source code > selecting release version > startref=bitcoin-core-compile-release 1
- Bitcoin Core > compiling from source code > startref=bitcoin-core-compile 1
- Bitcoin Core > explained > id=bitcoin-core-explain 1
- Bitcoin Core > explained > startref=bitcoin-core-explain 1
- Bitcoin Core > genesis block > id=bitcoin-core-genesis 1
- Bitcoin Core > genesis block > startref=bitcoin-core-genesis 1
- Bitcoin Core > nodes > configuring > id=bitcoin-core-nodes-configure 1
- Bitcoin Core > nodes > configuring > startref=bitcoin-core-nodes-configure 1
- Bitcoin Core > nodes > running > id=bitcoin-core-nodes-running 1
- Bitcoin Core > nodes > running > startref=bitcoin-core-nodes-running 1
- Bitcoin Core > RBF variants > id=bitcoin-core-rbf 1
- Bitcoin Core > RBF variants > startref=bitcoin-core-rbf 1
- Bitcoin Core > regtest 1
- Bitcoin Core > serialized transactions > id=bitcoin-core-serial-transaction 1
- Bitcoin Core > serialized transactions > startref=bitcoin-core-serial-transaction 1
- Bitcoin Core > signet 1
- Bitcoin Core > testnet 1
- Bitcoin Core > Tor transport 1
- Bitcoin Core > wrapper libraries 1
- Bitcoin Improvement Proposals > see=BIPs 1
- Bitcoin network 1
- Bitcoin network > bloom filters > lightweight clients and > id=bitcoin-network-bloom-lightweight 1
- Bitcoin network > bloom filters > lightweight clients and > startref=bitcoin-network-bloom-lightweight 1
- Bitcoin network > bloom filters > operational overview > id=bitcoin-network-bloom-overview 1
- Bitcoin network > bloom filters > operational overview > startref=bitcoin-network-bloom-overview 1
- Bitcoin network > compact block filters > downloading multiple > id=bitcoin-network-compact-filter-download 1
- Bitcoin network > compact block filters > downloading multiple > startref=bitcoin-network-compact-filter-download 1
- Bitcoin network > compact block filters > GCS (Golomb-Rice coded sets) > id=bitcoin-network-compact-filter-gcs 1
- Bitcoin network > compact block filters > GCS (Golomb-Rice coded sets) > startref=bitcoin-network-compact-filter-gcs 1
- Bitcoin network > compact block filters > id=bitcoin-network-compact-filter 1
- Bitcoin network > compact block filters > lossy encoding 1
- Bitcoin network > compact block filters > startref=bitcoin-network-compact-filter 1
- Bitcoin network > compact block filters > what to include > id=bitcoin-network-compact-filter-include 1
- Bitcoin network > compact block filters > what to include > startref=bitcoin-network-compact-filter-include 1
- Bitcoin network > encryption 1
- Bitcoin network > full nodes, purpose of 1

- Bitcoin network > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper 1
- Bitcoin network > lightweight clients > id=bitcoin-network-lightweight 1
- Bitcoin network > lightweight clients > merkle trees and 1
- Bitcoin network > lightweight clients > privacy 1
- Bitcoin network > lightweight clients > startref=bitcoin-network-lightweight 1
- Bitcoin network > mempools 1
- Bitcoin network > nodes > compact block relay > id=bitcoin-network-node-compact-relay 1
- Bitcoin network > nodes > compact block relay > startref=bitcoin-network-node-compact-relay 1
- Bitcoin network > nodes > network discovery > id=bitcoin-network-node-discovery 1
- Bitcoin network > nodes > network discovery > startref=bitcoin-network-node-discovery 1
- Bitcoin network > nodes > number of 1
- Bitcoin network > nodes > private block relay > id=bitcoin-network-node-private-relay 1
- Bitcoin network > nodes > private block relay > startref=bitcoin-network-node-private-relay 1
- Bitcoin network > nodes > syncing blockchain > id=bitcoin-network-node-sync 1
- Bitcoin network > nodes > syncing blockchain > startref=bitcoin-network-node-sync 1
- Bitcoin network > nodes > types of 1
- Bitcoin network > orphan pools 1
- Bitcoin Relay Network 1
- Bitcoin whitepaper > errata > id=bitcoin-whitepaper-errata 1
- Bitcoin whitepaper > errata > startref=bitcoin-whitepaper-errata 1
- Bitcoin whitepaper > original version > id=bitcoin-whitepaper-original 1
- Bitcoin whitepaper > original version > startref=bitcoin-whitepaper-original 1
- bitcoin-cli command > see=command-line interface (Bitcoin Core) 1
- bitcoin-s 1
- bitcoinj 1
- bitcoins > acquiring > id=bitcoin-acquire 1
- bitcoins > acquiring > startref=bitcoin-acquire 1
- bitcoins > clearing transactions 1
- bitcoins > currency exchanges 1
- bitcoins > defined 1
- bitcoins > exchange rate > id=bitcoin-exchange-rate 1
- bitcoins > fractional values 1
- bitcoins > key control 1
- bitcoins > mining 1
- bitcoins > mining > coinbase transactions > id=bitcoin-mining-coinbase 1
- bitcoins > mining > coinbase transactions > startref=bitcoin-mining-coinbase 1
- bitcoins > mining > currency creation > id=bitcoin-mining-create 1
- bitcoins > mining > currency creation > startref=bitcoin-mining-create 1
- bitcoins > mining > incentives 1
- bitcoins > physical storage 1
- bitcoins > receiving 1
- bitcoins > receiving > id=bitcoin-receive 1
- bitcoins > receiving > startref=bitcoin-receive 1
- bitcoins > spending > id=bitcoin-send 1
- bitcoins > spending > id=bitcoin-spend 1
- bitcoins > spending > id=bitcoin-spend2 1
- bitcoins > spending > startref=bitcoin-send 1
- bitcoins > spending > startref=bitcoin-spend 1
- bitcoins > transactions > see=transactions 1
- Bitcore 1
- block header 1
- block header > constructing > id=block-header-construct 1
- block header > constructing > startref=block-header-construct 1
- block header hash > id=block-header-hash 1
- block header hash > startref=block-header-hash 1
- block height > id=block-height 1
- block height > startref=block-height 1
- block reward 1
- block subsidy 1
- block-finding races 1
- blockchain > adding transactions to > id=blockchain-add-transaction 1
- blockchain > adding transactions to > startref=blockchain-add-transaction 1
- blockchain > assembling > id=blockchain-assemble 1
- blockchain > assembling > startref=blockchain-assemble 1
- blockchain > explained > id=blockchain-explain 1

- blockchain > explained > startref=blockchain-explain 1
- blockchain > forks 1
- blockchain > genesis block > id=blockchain-genesis 1
- blockchain > genesis block > startref=blockchain-genesis 1
- blockchain > linking blocks > id=blockchain-link 1
- blockchain > linking blocks > startref=blockchain-link 1
- blockchain > merkle trees > id=blockchain-merkle 1
- blockchain > merkle trees > startref=blockchain-merkle 1
- blockchain > nonpayment data in > id=blockchain-nonpayment 1
- blockchain > nonpayment data in > startref=blockchain-nonpayment 1
- blockchain > syncing > id=blockchain-sync 1
- blockchain > syncing > startref=blockchain-sync 1
- blockchain > test blockchains > development usage 1
- blockchain > test blockchains > regtest > id=blockchain-test-regtest 1
- blockchain > test blockchains > regtest > startref=blockchain-test-regtest 1
- blockchain > test blockchains > signet > id=blockchain-test-signet 1
- blockchain > test blockchains > signet > startref=blockchain-test-signet 1
- blockchain > test blockchains > testnet > id=blockchain-test-testnet 1
- blockchain > test blockchains > testnet > startref=blockchain-test-testnet 1
- blockchain explorers > id=blockchain-explorers 1
- blockchain explorers > startref=blockchain-explorers 1
- blocks 1
- blocks > block header 1
- blocks > candidate blocks 1
- blocks > candidate blocks > mining > id=block-candidate-mine 1
- blocks > candidate blocks > mining > startref=block-candidate-mine 1
- blocks > compact block filters > downloading multiple > id=block-compact-filter-download 1
- blocks > compact block filters > downloading multiple > startref=block-compact-filter-download 1
- blocks > compact block filters > GCS (Golomb-Rice coded sets) > id=block-compact-filter-gcs 1
- blocks > compact block filters > GCS (Golomb-Rice coded sets) > startref=block-compact-filter-gcs 1
- blocks > compact block filters > id=block-compact-filter 1
- blocks > compact block filters > lossy encoding 1
- blocks > compact block filters > startref=block-compact-filter 1
- blocks > compact block filters > what to include > id=block-compact-filter-include 1
- blocks > compact block filters > what to include > startref=block-compact-filter-include 1
- blocks > compact block relay > id=block-compact-relay 1
- blocks > compact block relay > startref=block-compact-relay 1
- blocks > exploring > id=blocks-explore 1
- blocks > exploring > id=blocks-explore2 1
- blocks > exploring > startref=blocks-explore 1
- blocks > exploring > startref=blocks-explore2 1
- blocks > identifiers > id=block-identify 1
- blocks > identifiers > startref=block-identify 1
- blocks > linking in blockchain > id=block-link 1
- blocks > linking in blockchain > startref=block-link 1
- blocks > private block relay > id=block-private-relay 1
- blocks > private block relay > startref=block-private-relay 1
- blocks > reclaiming disk space 1 2
- blocks > structure of 1
- blocks > transactions in 1
- blocks > validating > id=block-validate 1
- blocks > validating > startref=block-validate 1
- blocksonly option (bitcoind option) 1
- bloom filters > lightweight clients and > id=bloom-lightweight 1
- bloom filters > lightweight clients and > startref=bloom-lightweight 1
- bloom filters > operational overview > id=bloom-overview 1
- bloom filters > operational overview > startref=bloom-overview 1
- brainwallets 1
- btcd 1
- building blocks > id=build-block 1
- building blocks > startref=build-block 1
- Byzantine Generals' Problem 1

- C/C++ toolkits 1
- C# toolkits 1
- calculations > errata in Bitcoin whitepaper 1
- calculations > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper > id=calculate-whitepaper 1
- calculations > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper > startref=calculate-whitepaper 1
- candidate blocks 1 2
- candidate blocks > mining > id=candidate-mine 1
- candidate blocks > mining > startref=candidate-mine 1
- carve outs (CPFP) > id=carveout 1
- carve outs (CPFP) > startref=carveout 1
- chain forks > id=chain-fork 1
- chain forks > startref=chain-fork 1
- challenge script 1
- change output 1
- change output > id=change-output 1
- change output > transaction fees and 1
- changeless transactions 1
- checksums 1
- child blocks 1
- child key pair derivation > hardened derivation > id=child-key-pair-harden 1
- child key pair derivation > hardened derivation > startref=child-key-pair-harden 1
- child key pair derivation > id=child-key-pair 1
- child key pair derivation > private keys > id=child-key-pair-private 1
- child key pair derivation > private keys > startref=child-key-pair-private 1
- child key pair derivation > public keys > id=child-key-pair-public 1
- child key pair derivation > public keys > startref=child-key-pair-public 1
- child key pair derivation > startref=child-key-pair 1
- choosing > see=selecting 1
- circular dependencies > id=circular 1
- clearing transactions 1 2
- client-side validation 1
- clients 1
- Codex32 recovery codes 1
- coin selection in transactions 1
- coinbase data 1
- coinbase transactions > id=coinbase 1
- coinbase transactions > id=coinbase-transaction 1
- coinbase transactions > startref=coinbase 1
- coinbase transactions > startref=coinbase-transaction 1
- cold storage 1
- collision attacks > id=collision 1
- colored coins application > client-side validation 1
- colored coins application > id=color-coin 1
- colored coins application > P2C (pay to contract) > id=color-coin-p2c 1
- colored coins application > P2C (pay to contract) > startref=color-coin-p2c 1
- colored coins application > RGB protocol > id=color-coin-rgb 1
- colored coins application > RGB protocol > startref=color-coin-rgb 1
- colored coins application > single-use seals 1
- colored coins application > startref=color-coin 1
- colored coins application > Taproot Assets > id=color-coin-taproot 1
- colored coins application > Taproot Assets > startref=color-coin-taproot 1
- command-line interface (Bitcoin Core) > API access > id=command-api 1
- command-line interface (Bitcoin Core) > API access > startref=command-api 1
- command-line interface (Bitcoin Core) > exploring blocks > id=command-blocks 1
- command-line interface (Bitcoin Core) > exploring blocks > id=command-blocks2 1
- command-line interface (Bitcoin Core) > exploring blocks > startref=command-blocks 1
- command-line interface (Bitcoin Core) > exploring blocks > startref=command-blocks2 1
- command-line interface (Bitcoin Core) > exploring/decoding transactions > id=command-transaction 1
- command-line interface (Bitcoin Core) > exploring/decoding transactions > id=command-transaction2 1
- command-line interface (Bitcoin Core) > exploring/decoding transactions > startref=command-transaction 1
- command-line interface (Bitcoin Core) > exploring/decoding transactions > startref=command-transaction2 1

- command-line interface (Bitcoin Core) > help command > id=command-help 1
- command-line interface (Bitcoin Core) > help command > startref=command-help 1
- command-line interface (Bitcoin Core) > status information > id=command-status 1
- command-line interface (Bitcoin Core) > status information > startref=command-status 1
- commitment hash 1 2
- commitment transactions 1
- commitment transactions > asymmetric revocable commitments > id=commit-revoke 1
- commitment transactions > asymmetric revocable commitments > startref=commit-revoke 1
- commitment transactions > trustless channels > id=commit-trustless 1
- commitment transactions > trustless channels > startref=commit-trustless 1
- commitments > id=commitment 1
- commitments > startref=commitment 1
- compact block filters > downloading multiple > id=compact-block-filter-download 1
- compact block filters > downloading multiple > startref=compact-block-filter-download 1
- compact block filters > GCS (Golomb-Rice coded sets) > id=compact-block-filter-gcs 1
- compact block filters > GCS (Golomb-Rice coded sets) > startref=compact-block-filter-gcs 1
- compact block filters > id=compact-block-filter 1
- compact block filters > lossy encoding 1
- compact block filters > startref=compact-block-filter 1
- compact block filters > what to include > id=compact-block-filter-include 1
- compact block filters > what to include > startref=compact-block-filter-include 1
- compact block relay > id=compact-block-relay 1
- compact block relay > startref=compact-block-relay 1
- compactSize unsigned integers > id=compactsize 1
- compiling Bitcoin Core from source code > building executables 1
- compiling Bitcoin Core from source code > configuring build > id=compile-bitcoin-core-configure 1
- compiling Bitcoin Core from source code > configuring build > startref=compile-bitcoin-core-configure 1
- compiling Bitcoin Core from source code > id=compile-bitcoin-core 1
- compiling Bitcoin Core from source code > selecting release version > id=compile-bitcoin-core-release 1
- compiling Bitcoin Core from source code > selecting release version > startref=compile-bitcoin-core-release 1
- compiling Bitcoin Core from source code > startref=compile-bitcoin-core 1
- compressed private keys > id=compress-private-key 1
- compressed private keys > startref=compress-private-key 1
- compressed public keys > id=compress-pub-key 1
- compressed public keys > startref=compress-pub-key 1
- conditional clauses in scripts > id=conditional-clause-script 1
- conditional clauses in scripts > startref=conditional-clause-script 1
- conf option (bitcoind option) 1
- configuring > Bitcoin Core build > id=configure-build 1
- configuring > Bitcoin Core build > startref=configure-build 1
- configuring > nodes > id=configure-nodes 1
- configuring > nodes > startref=configure-nodes 1
- confirmations 1 2
- conflicting transactions 1 2 3
- consensus > see=decentralized consensus 1
- consensus rules 1 2
- consensus rules > hard forks > contentious forks 1
- consensus rules > hard forks > difficulty and > id=consensus-hard-fork-difficult 1
- consensus rules > hard forks > explained > id=consensus-hard-fork 1
- consensus rules > hard forks > explained > startref=consensus-hard-fork 1
- consensus rules > hard forks > types of > id=consensus-hard-fork-type 1
- consensus rules > hard forks > types of > startref=consensus-hard-fork-type 1
- consensus rules > soft forks > BIP34 signaling/activation > tertiary-sortas=BIP034 > id=consensus-soft-bip34 1
- consensus rules > soft forks > BIP34 signaling/activation > tertiary-sortas=BIP034 > startref=consensus-soft-bip34 1

- consensus rules > soft forks > BIP8 mandatory lock-in > tertiary-sortas=BIP008 > id=consensus-soft-bip8 1
- consensus rules > soft forks > BIP8 mandatory lock-in > tertiary-sortas=BIP008 > startref=consensus-soft-bip8 1
- consensus rules > soft forks > BIP9 signaling/activation > tertiary-sortas=BIP009 > id=consensus-soft-bip9 1
- consensus rules > soft forks > BIP9 signaling/activation > tertiary-sortas=BIP009 > startref=consensus-soft-bip9 1
- consensus rules > soft forks > criticisms of > id=consensus-soft-critic 1
- consensus rules > soft forks > explained > id=consensus-soft-explain 1
- consensus rules > soft forks > explained > startref=consensus-soft-explain 1
- consensus rules > soft forks > speedy trial activation > id=consensus-soft-speed 1
- consensus rules > software development 1
- consensus rules > timestamps and > id=consensus-timestamp 1
- consolidation transactions 1
- contentious hard forks 1
- CPFP (child pays for parent) fee bumping > ancestor fee rate mining 1
- CPFP (child pays for parent) fee bumping > carve outs > id=cpfp-carveout 1
- CPFP (child pays for parent) fee bumping > carve outs > startref=cpfp-carveout 1
- CPFP (child pays for parent) fee bumping > id=cpfp-ch9 1
- CPFP (child pays for parent) fee bumping > startref=cpfp-ch9 1
- CPFP (child pays for parent) fee bumping > transaction pinning > id=cpfp-pin 1
- CPFP (child pays for parent) fee bumping > transaction pinning > startref=cpfp-pin 1
- crowdfunding 1
- cryptography 1
- currency creation > id=currency-create 1
- currency creation > startref=currency-create 1
- currency exchanges 1
- current price of bitcoins > id=current-price 1
- custom signets 1
- datadir option (bitcoind option) 1
- dbcache option (bitcoind option) 1
- decentralized consensus > as security principle > secondary-sortas=security principle > id=decentral-consensus-principle 1
- decentralized consensus > as security principle > secondary-sortas=security principle > startref=decentral-consensus-principle 1
- decentralized consensus > assembling blockchain > id=decentral-consensus-assemble 1
- decentralized consensus > assembling blockchain > startref=decentral-consensus-assemble 1
- decentralized consensus > hashrate attacks > id=decentral-consensus-hashrate 1
- decentralized consensus > hashrate attacks > startref=decentral-consensus-hashrate 1
- decentralized consensus > id=decentral-consensus 1
- decentralized consensus > startref=decentral-consensus 1
- decentralized consensus > timestamps and > id=decentral-consensus-timestamp 1
- decentralized consensus > validating blocks > id=decentral-consensus-validate 1
- decentralized consensus > validating blocks > startref=decentral-consensus-validate 1
- decoding > addresses > see=addresses 1
- decoding > transactions > id=decode-transaction 1
- decoding > transactions > id=decode-transaction2 1
- decoding > transactions > startref=decode-transaction 1
- decoding > transactions > startref=decode-transaction2 1
- default signet 1
- deflation > id=deflation 1
- deflation > startref=deflation 1
- desktop wallets 1
- deterministic key generation > id=determine-keygen 1
- deterministic key generation > startref=determine-keygen 1
- difficulty > adjusting > id=difficulty-adjust 1
- difficulty > adjusting > startref=difficulty-adjust 1
- difficulty > hard forks and > id=difficulty-hardfork 1
- digests 1
- digital currencies, history of 1
- digital signatures 1 2
- digital signatures > creating 1
- digital signatures > ECDSA 1

- digital signatures > ECDSA > id=digital-signature-ecdsa 1
- digital signatures > ECDSA > startref=digital-signature-ecdsa 1
- digital signatures > purpose of 1
- digital signatures > randomness, importance of > id=digital-signature-random 1
- digital signatures > randomness, importance of > startref=digital-signature-random 1
- digital signatures > schnorr signature algorithm 1
- digital signatures > schnorr signature algorithm > examples of usage 1
- digital signatures > schnorr signature algorithm > id=digital-sigs-schnorr 1
- digital signatures > schnorr signature algorithm > properties of 1
- digital signatures > schnorr signature algorithm > scriptless multisignatures > id=digital-sigs-schnorr-multisig 1
- digital signatures > schnorr signature algorithm > scriptless multisignatures > startref=digital-sigs-schnorr-multisig 1
- digital signatures > schnorr signature algorithm > scriptless threshold signatures > id=digital-sigs-schnorr-threshold 1
- digital signatures > schnorr signature algorithm > scriptless threshold signatures > startref=digital-sigs-schnorr-threshold 1
- digital signatures > schnorr signature algorithm > security features 1
- digital signatures > schnorr signature algorithm > serialization 1
- digital signatures > schnorr signature algorithm > startref=digital-sigs-schnorr 1
- digital signatures > segregated witness and 1
- digital signatures > SIGHASH flags > id=digital-signature-sighash 1
- digital signatures > SIGHASH flags > startref=digital-signature-sighash 1
- digital signatures > verifying 1
- disk space > reclaiming 1
- disk space, reclaiming 1
- display byte order 1
- distributed computing problem 1
- DNS seeds 1
- double spending 1
- double-spend attacks > id=double-spend 1
- double-spend attacks > startref=double-spend 1
- downloading multiple block filters > id=download-multiple-block 1
- downloading multiple block filters > startref=download-multiple-block 1
- dummy stack element 1
- dust policies > id=dust 1
- dust policies > startref=dust 1
- ECDSA (Elliptic Curve Digital Signature Algorithm) 1
- ECDSA (Elliptic Curve Digital Signature Algorithm) > id=ecdsa 1
- ECDSA (Elliptic Curve Digital Signature Algorithm) > startref=ecdsa 1
- economics of Bitcoin > id=economics 1
- economics of Bitcoin > startref=economics 1
- Electrum v2 recovery codes 1
- elliptic curve cryptography (ECC) > id=ecc 1
- elliptic curve cryptography (ECC) > startref=ecc 1
- elliptic curve multiplication > id=elliptic-multiply 1
- elliptic curve multiplication > startref=elliptic-multiply 1
- embedded segregated witness 1
- emergent consensus > id=emergent-consensus 1
- emergent consensus > startref=emergent-consensus 1
- encoding > base58check 1
- encoding > base58check > id=encode-base58 1
- encoding > base58check > startref=encode-base58 1
- encoding > bech32m addresses > id=encode-bech32m 1
- encoding > bech32m addresses > startref=encode-bech32m 1
- encryption 1
- entropy 1
- entropy > recovery code generation > id=entropy-recovery-generate 1
- entropy > recovery code generation > startref=entropy-recovery-generate 1
- entropy > seed generation > id=entropy-seed-generate 1
- entropy > seed generation > startref=entropy-seed-generate 1
- estate planning 1
- estimating fee rates > id=estimate-fee-rate 1
- excessive fees 1
- exchange rate > id=exchange-rate 1
- executables (Bitcoin Core), building 1

- explicit paths > id=explicit-path 1
- explicit paths > startref=explicit-path 1
- exploring > blocks > id=explore-blocks 1
- exploring > blocks > id=explore-blocks2 1
- exploring > blocks > startref=explore-blocks 1
- exploring > blocks > startref=explore-blocks2 1
- exploring > transactions > id=explore-transaction 1
- exploring > transactions > id=explore-transaction2 1
- exploring > transactions > startref=explore-transaction 1
- exploring > transactions > startref=explore-transaction2 1
- extended keys > explained > id=extend-key 1
- extended keys > web store example > id=extend-key-webstore 1
- extended keys > web store example > startref=extend-key-webstore 1
- extended serialization format 1
- extra nonce solution > id=extra-nonce 1
- extra nonce solution > startref=extra-nonce 1
- false positives 1
- Fast Internet Bitcoin Relay Engine (FIBRE) 1
- FEC (Forward Error Correction) 1
- fee bumping > CPFP (child pays for parent) > id=fee-bump-cpfp 1
- fee bumping > CPFP (child pays for parent) > startref=fee-bump-cpfp 1
- fee bumping > CPFP carve outs > id=fee-bump-carveout 1
- fee bumping > CPFP carve outs > startref=fee-bump-carveout 1
- fee bumping > RBF (replace by fee) > id=fee-bump-rbf 1
- fee bumping > RBF (replace by fee) > startref=fee-bump-rbf 1
- fee bumping > transaction pinning > id=fee-bump-pin 1
- fee bumping > transaction pinning > startref=fee-bump-pin 1
- fee rates > id=fee-rate 1
- fee rates > startref=fee-rate 1
- fee sniping > id=fee-snipe 1
- fees > see=transaction fees 1
- FIBRE (Fast Internet Bitcoin Relay Engine) 1
- flow control in scripts > id=flow-control-script 1
- flow control in scripts > startref=flow-control-script 1
- forks 1
- forks > consensus rule software development 1
- forks > hard forks 1
- forks > hard forks > contentious forks 1
- forks > hard forks > difficulty and > id=forks-hard-difficult 1
- forks > hard forks > explained > id=forks-hard 1
- forks > hard forks > explained > startref=forks-hard 1
- forks > hard forks > types of > id=forks-hard-type 1
- forks > hard forks > types of > startref=forks-hard-type 1
- forks > hashrate attacks > id=fork-hashrate 1
- forks > hashrate attacks > startref=fork-hashrate 1
- forks > soft forks 1
- forks > soft forks > BIP34 signaling/activation > tertiary-sortas=BIP034 > id=fork-soft-bip34 1
- forks > soft forks > BIP34 signaling/activation > tertiary-sortas=BIP034 > startref=fork-soft-bip34 1
- forks > soft forks > BIP8 mandatory lock-in > tertiary-sortas=BIP008 > id=fork-soft-bip8 1
- forks > soft forks > BIP8 mandatory lock-in > tertiary-sortas=BIP008 > startref=fork-soft-bip8 1
- forks > soft forks > BIP9 signaling/activation > tertiary-sortas=BIP009 > id=fork-soft-bip9 1
- forks > soft forks > BIP9 signaling/activation > tertiary-sortas=BIP009 > startref=fork-soft-bip9 1
- forks > soft forks > criticisms of > id=fork-soft-critic 1
- forks > soft forks > explained > id=fork-soft-explain 1
- forks > soft forks > explained > startref=fork-soft-explain 1
- forks > soft forks > speedy trial activation > id=fork-soft-speed 1
- Forward Error Correction (FEC) 1
- fractional values of bitcoins 1
- free American call option 1
- full nodes 1 2 3
- full nodes > purpose of 1 2
- full nodes > syncing blockchain > id=full-node-sync 1
- full nodes > syncing blockchain > startref=full-node-sync 1
- funding transactions 1
- Gambler's Ruin problem 1
- gap limit > id=gap-limit 1
- gap limit > startref=gap-limit 1

- GCS (Golomb-Rice coded sets) > id=gcs-filter 1
- GCS (Golomb-Rice coded sets) > startref=gcs-filter 1
- generation transactions 1
- genesis block 1 2
- genesis block > id=genesis-block 1
- genesis block > startref=genesis-block 1
- Go toolkits 1
- gossiping 1
- guard clauses in scripts 1
- hard forks 1
- hard forks > contentious forks 1
- hard forks > difficulty and > id=hard-forks-difficult 1
- hard forks > explained > id=hard-forks-ch12 1
- hard forks > explained > startref=hard-forks-ch12 1
- hard forks > types of > id=hard-forks-type 1
- hard forks > types of > startref=hard-forks-type 1
- hardened child key derivation > id=harden-child-key 1
- hardened child key derivation > startref=harden-child-key 1
- hardware signing devices 1 2 3
- hash functions 1
- hash functions > Bitcoin payments and > id=hash-payment 1
- hash functions > Bitcoin payments and > id=hash-payment2 1
- hash functions > Bitcoin payments and > startref=hash-payment 1
- hash functions > Bitcoin payments and > startref=hash-payment2 1
- hash functions > deterministic key generation > id=hash-determine 1
- hash functions > deterministic key generation > startref=hash-determine 1
- hash functions > digests 1
- hash functions > proof-of-work algorithm > id=hash-proof 1
- hash functions > proof-of-work algorithm > startref=hash-proof 1
- Hash Time Lock Contract (HTLC) > id=hash-time-lock-contract 1
- Hash Time Lock Contract (HTLC) > startref=hash-time-lock-contract 1
- HASH160 1
- hashrate attacks > id=hashrate 1
- hashrate attacks > startref=hashrate 1
- HD (hierarchical deterministic) key generation 1
- HD (hierarchical deterministic) key generation > extended keys > explained > id=hd-keygen-extend 1
- HD (hierarchical deterministic) key generation > extended keys > web store example > id=hd-keygen-extend-webstore 1
- HD (hierarchical deterministic) key generation > extended keys > web store example > startref=hd-keygen-extend-webstore 1
- HD (hierarchical deterministic) key generation > id=hd-keygen 1
- HD (hierarchical deterministic) key generation > path references 1
- HD (hierarchical deterministic) key generation > private child key derivation > id=hd-keygen-private-child 1
- HD (hierarchical deterministic) key generation > private child key derivation > startref=hd-keygen-private-child 1
- HD (hierarchical deterministic) key generation > public child key derivation > id=hd-keygen-public-child 1
- HD (hierarchical deterministic) key generation > public child key derivation > startref=hd-keygen-public-child 1
- HD (hierarchical deterministic) key generation > startref=hd-keygen 1
- HD (hierarchical deterministic) key generation > tree structure > id=hd-keygen-tree 1
- HD (hierarchical deterministic) key generation > tree structure > startref=hd-keygen-tree 1
- help command (Bitcoin Core) > id=help 1
- help command (Bitcoin Core) > startref=help 1
- high-bandwidth mode (compact block relay) 1
- high-frequency transactions 1
- history > of Bitcoin > secondary-sortas=Bitcoin 1
- history > of digital currencies > secondary-sortas=digital currencies 1
- homogeneity of degree 1 1
- HTLC (Hash Time Lock Contract) > id=htlc 1
- HTLC (Hash Time Lock Contract) > startref=htlc 1
- implicit paths > id=implicit-path 1
- implicit paths > startref=implicit-path 1
- incentives 1
- incentives > id=incentive-whitepaper 1
- incentives > startref=incentive-whitepaper 1
- independent key generation > id=independent-keygen 1

- independent key generation > startref=independent-keygen 1
- independent transaction verification > id=independent-transaction-verify 1
- independent transaction verification > startref=independent-transaction-verify 1
- index numbers for hardened derivation 1
- inflation > id=inflation 1
- inflation > startref=inflation 1
- input scripts 1
- input scripts > constructing > id=input-script-construct 1
- input scripts > examples of > id=input-script-example 1
- input scripts > examples of > startref=input-script-example 1
- input scripts > id=input-script 1
- input scripts > separate execution from output scripts 1
- input scripts > startref=input-script 1
- inputs > coinbase versus regular transactions 1
- inputs > constructing transactions 1
- inputs > id=input 1
- inputs > id=input-transaction 1
- inputs > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper 1
- inputs > input script 1
- inputs > length of list > id=input-transaction-length 1
- inputs > length of list > startref=input-transaction-length 1
- inputs > outpoint field > id=input-transaction-outpoint 1
- inputs > outpoint field > startref=input-transaction-outpoint 1
- inputs > sequence field > id=input-transaction-sequence 1
- inputs > sequence field > startref=input-transaction-sequence 1
- inputs > startref=input-transaction 1
- inputs > transaction fees and 1
- internal byte order 1
- invoices 1 2
- IP addresses for Bitcoin payments > id=ipaddress-payment 1
- IP addresses for Bitcoin payments > startref=ipaddress-payment 1
- Java toolkits 1
- JavaScript toolkits 1
- key cancellation attacks 1
- key generation > backing up derivation paths > id=keygen-backups 1
- key generation > backing up derivation paths > startref=keygen-backups 1
- key generation > deterministic > id=keygen-determine 1
- key generation > deterministic > startref=keygen-determine 1
- key generation > HD (hierarchical deterministic) 1
- key generation > HD (hierarchical deterministic) > extended keys > id=keygen-hd-extend 1
- key generation > HD (hierarchical deterministic) > extended keys > id=keygen-hd-extend-webstore 1
- key generation > HD (hierarchical deterministic) > extended keys > startref=keygen-hd-extend-webstore 1
- key generation > HD (hierarchical deterministic) > id=keygen-hd 1
- key generation > HD (hierarchical deterministic) > path references 1
- key generation > HD (hierarchical deterministic) > private child key derivation > id=keygen-hd-private-child 1
- key generation > HD (hierarchical deterministic) > private child key derivation > startref=keygen-hd-private-child 1
- key generation > HD (hierarchical deterministic) > public child key derivation > id=keygen-hd-public-child 1
- key generation > HD (hierarchical deterministic) > public child key derivation > startref=keygen-hd-public-child 1
- key generation > HD (hierarchical deterministic) > startref=keygen-hd 1
- key generation > HD (hierarchical deterministic) > tree structure > id=keygen-hd-tree 1
- key generation > HD (hierarchical deterministic) > tree structure > startref=keygen-hd-tree 1
- key generation > independent > id=keygen-independent 1
- key generation > independent > startref=keygen-independent 1
- key generation > public child key derivation > id=keygen-public-child 1
- key generation > public child key derivation > startref=keygen-public-child 1
- key pairs > id=key-pair 1

- key tweaks 1 2
- key tweaks > id=key-tweak 1
- key tweaks > startref=key-tweak 1
- key-stretching functions > id=key-stretch 1
- key-stretching functions > startref=key-stretch 1
- keypath spending 1
- keys, control of 1
- labels, backing up > id=label-backup 1
- legacy serialization 1 2
- Lightning Network (LN) > benefits of > id=lightning-benefits 1
- Lightning Network (LN) > benefits of > startref=lightning-benefits 1
- Lightning Network (LN) > example of > id=lightning-example 1
- Lightning Network (LN) > example of > startref=lightning-example 1
- Lightning Network (LN) > id=lightning 1
- Lightning Network (LN) > pathfinding > id=lightning-path 1
- Lightning Network (LN) > pathfinding > startref=lightning-path 1
- Lightning Network (LN) > startref=lightning 1
- lightweight clients 1 2
- lightweight clients > bloom filters and > id=lightweight-bloom 1
- lightweight clients > bloom filters and > startref=lightweight-bloom 1
- lightweight clients > id=lightweight 1
- lightweight clients > merkle trees and 1
- lightweight clients > privacy 1
- lightweight clients > startref=lightweight 1
- linearity 1
- linking blocks in blockchain > id=link-block 1
- linking blocks in blockchain > startref=link-block 1
- LN > see=Lightning Network 1
- lock time 1
- lock time > conflicts 1
- lock time > fee sniping and > id=lock-time-fee-snipe 1
- lock time > limitations of 1
- lock time > relative > id=locktime-relative 1
- lock time > relative > startref=locktime-relative 1
- lock time > verifying > id=lock-time-op-cltv 1
- lock time > verifying > startref=lock-time-op-cltv 1
- lock-in, mandatory > id=lockin-mandatory 1
- lock-in, mandatory > startref=lockin-mandatory 1
- lossless encoding 1
- lossy encoding 1
- low-bandwidth mode (compact block relay) 1
- mainnet 1
- majority attacks > id=majority-attack 1
- majority attacks > startref=majority-attack 1
- managed pools > id=manage-pool 1
- mandatory lock-in > id=mandatory-lockin 1
- mandatory lock-in > startref=mandatory-lockin 1
- MAST (merklized alternative script trees) > id=mast-ch7 1
- MAST (merklized alternative script trees) > startref=mast-ch7 1
- MAST (merklized alternative script trees) > taproot > id=mast-taproot 1
- MAST (merklized alternative script trees) > taproot > startref=mast-taproot 1
- maturity rule 1
- maxmempool option (bitcoind option) 1
- median time past (MTP) 1
- median time past (MTP) > id=median-time-past 1
- memorizing recovery codes 1
- memory pool 1 2
- merkle path 1
- merkle trees > id=merkle-tree-explain 1
- merkle trees > MAST > id=merkle-tree-mast 1
- merkle trees > MAST > startref=merkle-tree-mast 1
- merkle trees > MAST > taproot > id=merkle-tree-mast-taproot 1
- merkle trees > MAST > taproot > startref=merkle-tree-mast-taproot 1
- merkle trees > startref=merkle-tree-explain 1
- millibitcoins 1
- mining 1
- mining > adjusting difficulty > id=mining-difficulty 1
- mining > adjusting difficulty > startref=mining-difficulty 1
- mining > assembling blockchain > id=mining-assemble 1
- mining > assembling blockchain > startref=mining-assemble 1
- mining > blocks > compact block relay > id=mining-block-compact-relay 1
- mining > blocks > compact block relay > startref=mining-block-compact-relay 1

- mining > blocks > private block relay > id=mining-block-private-relay 1
- mining > blocks > private block relay > startref=mining-block-private-relay 1
- mining > candidate blocks > id=mining-candidate 1
- mining > candidate blocks > startref=mining-candidate 1
- mining > coinbase transactions > id=mining-coinbase 1
- mining > coinbase transactions > startref=mining-coinbase 1
- mining > competitiveness of > id=mining-competitive 1
- mining > competitiveness of > startref=mining-competitive 1
- mining > constructing block header > id=mining-blockheader 1
- mining > constructing block header > startref=mining-blockheader 1
- mining > currency creation > id=mining-create 1
- mining > currency creation > startref=mining-create 1
- mining > decentralized consensus > id=mining-consensus 1
- mining > decentralized consensus > startref=mining-consensus 1
- mining > extra nonce solution > id=mining-nonce 1
- mining > extra nonce solution > startref=mining-nonce 1
- mining > hashrate attacks > id=mining-hashrate 1
- mining > hashrate attacks > startref=mining-hashrate 1
- mining > incentives 1
- mining > independent transaction verification > id=mining-verify 1
- mining > independent transaction verification > startref=mining-verify 1
- mining > miner nodes, purpose of > id=mining-nodes-purpose 1
- mining > miner nodes, purpose of > startref=mining-nodes-purpose 1
- mining > mining pools > id=mining-mining-pools 1
- mining > mining pools > startref=mining-mining-pools 1
- mining > operational overview > id=mining-operational-overview 1
- mining > operational overview > id=mining-overview 1
- mining > operational overview > startref=mining-operational-overview 1
- mining > operational overview > startref=mining-overview 1
- mining > proof-of-work algorithm > id=mining-proof 1
- mining > proof-of-work algorithm > startref=mining-proof 1
- mining > purpose of 1
- mining > target representation > id=mining-target 1
- mining > target representation > startref=mining-target 1
- mining > timestamps > id=mining-timestamps 1
- mining > validating blocks > id=mining-validate 1
- mining > validating blocks > startref=mining-validate 1
- mining forks > id=mining-fork 1
- mining forks > startref=mining-fork 1
- mining pools 1
- mining pools > id=mining-pools 1
- mining pools > startref=mining-pools 1
- mnemonic phrases > see=recovery codes 1
- mobile wallets 1
- Moore's Law 1
- MTP (median time past) 1
- MTP (median time past) > id=mtp-median 1
- multiple block filters, downloading > id=multiple-block-download 1
- multiple block filters, downloading > startref=multiple-block-download 1
- multisignature addresses 1
- multisignature scripts 1
- multisignature scripts > id=multi-script 1
- multisignature scripts > in schnorr signature algorithm > secondary-sortas=schnorr > id=multi-script-schnorr 1
- multisignature scripts > in schnorr signature algorithm > secondary-sortas=schnorr > startref=multi-script-schnorr 1
- multisignature scripts > scriptless > id=multisignature-scriptless 1
- multisignature scripts > scriptless > startref=multisignature-scriptless 1
- multisignature scripts > startref=multi-script 1
- MuSig protocol 1
- MuSig-DN protocol 1
- MuSig2 protocol 1

- mutual satisfaction contracts > taproot > id=mutual-satisfaction-taproot 1
- mutual satisfaction contracts > taproot > startref=mutual-satisfaction-taproot 1
- mutual satisfaction contracts > tapscript > id=mutual-satisfaction-tapscript 1
- mutual satisfaction contracts > tapscript > startref=mutual-satisfaction-tapscript 1
- Muun recovery codes 1
- Nakamoto, Satoshi 1 2 3 4
- native forwarding 1
- NBitcoin 1
- nesting > P2WPKH (pay to witness public key hash) > id=nest-p2wpkh-ch7 1
- nesting > P2WPKH (pay to witness public key hash) > startref=nest-p2wpkh-ch7 1
- nesting > P2WSH (pay to witness script hash) > id=nest-p2wsh-ch7 1
- nesting > P2WSH (pay to witness script hash) > startref=nest-p2wsh-ch7 1
- network discovery > id=network-discovery 1
- network discovery > startref=network-discovery 1
- network forks > id=network-fork 1
- network forks > startref=network-fork 1
- networks (Bitcoin) > see=Bitcoin network 1
- nodes > compact block relay > id=node-compact-relay 1
- nodes > compact block relay > startref=node-compact-relay 1
- nodes > configuring > id=nodes-configure 1
- nodes > configuring > startref=nodes-configure 1
- nodes > in Bitcoin > secondary-sortas=Bitcoin whitepaper 1
- nodes > miner nodes > coinbase transactions > id=nodes-miner-coinbase 1
- nodes > miner nodes > coinbase transactions > startref=nodes-miner-coinbase 1
- nodes > miner nodes > constructing block header > id=nodes-miner-blockheader 1
- nodes > miner nodes > constructing block header > startref=nodes-miner-blockheader 1
- nodes > miner nodes > purpose of > id=nodes-miner-purpose 1
- nodes > miner nodes > purpose of > startref=nodes-miner-purpose 1
- nodes > network discovery > id=node-discovery 1
- nodes > network discovery > startref=node-discovery 1
- nodes > number of 1
- nodes > private block relay > id=node-private-relay 1
- nodes > private block relay > startref=node-private-relay 1
- nodes > running > id=nodes-running 1
- nodes > running > startref=nodes-running 1
- nodes > syncing blockchain > id=node-sync 1
- nodes > syncing blockchain > startref=node-sync 1
- nodes > transaction verification > id=node-verify 1
- nodes > transaction verification > startref=node-verify 1
- nodes > types of 1
- nodes > validating blocks > id=nodes-validate 1
- nodes > validating blocks > startref=nodes-validate 1
- nonce attacks 1
- noncustodial wallets 1
- nonkey data, backing up > id=nonkey-backups 1
- nonkey data, backing up > startref=nonkey-backups 1
- nonpayment data > id=nonpayment-data 1
- nonpayment data > startref=nonpayment-data 1
- NOP opcodes 1
- offchain technology 1
- OP_CHECKMULTISIG execution > id=op-checkmultisig 1
- OP_CHECKMULTISIG execution > startref=op-checkmultisig 1
- OP_CLTV script operator > id=op-cltv 1
- OP_CLTV script operator > startref=op-cltv 1
- OP_CSV script opcode > id=op-csv 1
- OP_CSV script opcode > startref=op-csv 1
- OP_NOP opcodes 1
- OP_RETURN scripts > id=op-return 1
- OP_RETURN scripts > startref=op-return 1
- opt-in transaction replacement 1
- orphan pools 1
- outpoint field (transaction inputs) > id=outpoint 1
- outpoint field (transaction inputs) > startref=outpoint 1
- output indexes 1
- output scripts > constructing > id=output-script-construct 1
- output scripts > examples of > id=output-script-example 1

- output scripts > examples of > startref=output-script-example 1
- output scripts > id=output-script 1
- output scripts > id=output-script2 1
- output scripts > OP_RETURN > id=output-script-op-return 1
- output scripts > OP_RETURN > startref=output-script-op-return 1
- output scripts > P2WPKH (pay to witness public key hash) > id=output-script-p2wpkh 1
- output scripts > P2WPKH (pay to witness public key hash) > startref=output-script-p2wpkh 1
- output scripts > P2WSH (pay to witness script hash) > id=output-script-p2wsh 1
- output scripts > P2WSH (pay to witness script hash) > startref=output-script-p2wsh 1
- output scripts > separate execution from input scripts 1
- output scripts > startref=output-script 1
- output scripts > startref=output-script2 1
- outputs > amount field > id=output-transaction-amount 1
- outputs > amount field > startref=output-transaction-amount 1
- outputs > change output > id=output-change 1
- outputs > constructing transactions 1
- outputs > count 1
- outputs > id=output 1
- outputs > id=output-transaction 1
- outputs > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper 1
- outputs > output scripts > id=output-transaction-script 1
- outputs > output scripts > startref=output-transaction-script 1
- outputs > startref=output-transaction 1
- outputs > transaction fees and 1
- overpaying transaction fees 1
- P2C (pay to contract) > id=p2c 1
- P2C (pay to contract) > id=p2c-color-coin 1
- P2C (pay to contract) > startref=p2c 1
- P2C (pay to contract) > startref=p2c-color-coin 1
- P2PK (pay to public key) > id=p2pk-ch4 1
- P2PK (pay to public key) > startref=p2pk-ch4 1
- P2PKH (pay to public key hash) 1
- P2PKH (pay to public key hash) > id=p2pkh-legacy 1
- P2PKH (pay to public key hash) > startref=p2pkh-legacy 1
- P2Pool (peer-to-peer mining pool) > id=p2pool 1
- P2Pool (peer-to-peer mining pool) > startref=p2pool 1
- P2SH (pay to script hash) > embedded segregated witness 1
- P2SH (pay to script hash) > id=p2sh-ch4 1
- P2SH (pay to script hash) > id=p2sh-ch7 1
- P2SH (pay to script hash) > startref=p2sh-ch4 1
- P2SH (pay to script hash) > startref=p2sh-ch7 1
- P2WPKH (pay to witness public key hash) > id=p2wpkh-ch7 1
- P2WPKH (pay to witness public key hash) > nesting > id=p2wpkh-nest 1
- P2WPKH (pay to witness public key hash) > nesting > startref=p2wpkh-nest 1
- P2WPKH (pay to witness public key hash) > startref=p2wpkh-ch7 1
- P2WSH (pay to witness script hash) > id=p2wsh-ch7 1
- P2WSH (pay to witness script hash) > nesting > id=p2wsh-nest 1
- P2WSH (pay to witness script hash) > nesting > startref=p2wsh-nest 1
- P2WSH (pay to witness script hash) > startref=p2wsh-ch7 1
- package fee rate 1
- package relay > id=package-relay 1
- paper wallets > id=paper-wallet 1
- paper wallets > startref=paper-wallet 1
- parent blocks 1
- partial private keys 1
- partially signed bitcoin transaction (PSBT) format 1
- passphrases (for recovery codes) > id=passphrase 1
- passphrases (for recovery codes) > id=passphrase-optional 1
- passphrases (for recovery codes) > startref=passphrase 1
- passphrases (for recovery codes) > startref=passphrase-optional 1
- path references in HD wallets 1
- pathfinding in Lightning Network > id=path-lightning 1
- pathfinding in Lightning Network > startref=path-lightning 1
- payment batching 1
- payment channels 1

- payment channels > asymmetric revocable commitments > id=payment-channel-revoke 1
- payment channels > asymmetric revocable commitments > startref=payment-channel-revoke 1
- payment channels > example of > id=payment-channel-example 1
- payment channels > example of > startref=payment-channel-example 1
- payment channels > HTLC (Hash Time Lock Contract) > id=payment-channel-htlc 1
- payment channels > HTLC (Hash Time Lock Contract) > startref=payment-channel-htlc 1
- payment channels > id=payment-channel 1
- payment channels > Lightning Network > benefits of > id=payment-channel-ln-benefits 1
- payment channels > Lightning Network > benefits of > startref=payment-channel-ln-benefits 1
- payment channels > Lightning Network > example of > id=payment-channel-ln-example 1
- payment channels > Lightning Network > example of > startref=payment-channel-ln-example 1
- payment channels > Lightning Network > id=payment-channel-ln 1
- payment channels > Lightning Network > pathfinding > id=payment-channel-ln-path 1
- payment channels > Lightning Network > pathfinding > startref=payment-channel-ln-path 1
- payment channels > Lightning Network > startref=payment-channel-ln 1
- payment channels > startref=payment-channel 1
- payment channels > state channels > id=payment-channel-state 1
- payment channels > state channels > startref=payment-channel-state 1
- payment channels > trustless channels > id=payment-channel-trustless 1
- payment channels > trustless channels > startref=payment-channel-trustless 1
- payment verification > errata in Bitcoin whitepaper 1
- payment verification > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper > id=payment-verify 1
- payment verification > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper > startref=payment-verify 1
- payments > transaction fees > see=transaction fees 1
- payments > via IP addresses > id=payment-ipaddress 1
- payments > via IP addresses > startref=payment-ipaddress 1
- payments > with hash functions > secondary-sortas=hash functions > id=payment-hash 1
- payments > with hash functions > secondary-sortas=hash functions > id=payment-hash2 1
- payments > with hash functions > secondary-sortas=hash functions > startref=payment-hash 1
- payments > with hash functions > secondary-sortas=hash functions > startref=payment-hash2 1
- peer-to-peer networks, Bitcoin as 1 2
- peer-to-peer networks, Bitcoin as > seealso=Bitcoin network 1
- peers 1 2
- physical bitcoin storage 1
- preimage attacks 1
- presigned transactions > id=presign-transaction 1
- primitives > id=primitive-list 1
- primitives > startref=primitive-list 1
- privacy > blockchain explorers 1
- privacy > errata in Bitcoin whitepaper 1
- privacy > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper 1
- privacy > lightweight clients 1
- privacy > vanity addresses > id=privacy-vanity 1
- private block relay > id=private-block-relay 1
- private block relay > startref=private-block-relay 1
- private child key derivation > hardened derivation > id=private-child-harden 1
- private child key derivation > hardened derivation > startref=private-child-harden 1
- private child key derivation > id=private-child 1
- private child key derivation > startref=private-child 1
- private keys > compressed > id=private-key-compress 1
- private keys > compressed > startref=private-key-compress 1
- private keys > formats > id=private-key-format 1
- private keys > formats > startref=private-key-format 1
- private keys > generating > id=private-key-generate 1
- private keys > generating > startref=private-key-generate 1
- private keys > partial 1
- private keys > purpose of 1
- proof-of-work algorithm 1

- proof-of-work algorithm > adjusting difficulty > id=proof-difficulty 1
- proof-of-work algorithm > adjusting difficulty > startref=proof-difficulty 1
- proof-of-work algorithm > errata in Bitcoin whitepaper > id=proof-errata 1
- proof-of-work algorithm > errata in Bitcoin whitepaper > startref=proof-errata 1
- proof-of-work algorithm > id=proof-mining 1
- proof-of-work algorithm > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper 1
- proof-of-work algorithm > seealso=mining 1
- proof-of-work algorithm > startref=proof-mining 1
- proof-of-work algorithm > target representation > id=proof-target 1
- proof-of-work algorithm > target representation > startref=proof-target 1
- prune option (bitcoind option) 1
- PSBT (partially signed bitcoin transaction) format 1
- public child key derivation > id=public-child 1
- public child key derivation > id=public-child-derive 1
- public child key derivation > startref=public-child 1
- public child key derivation > startref=public-child-derive 1
- public key cryptography > base58check encoding 1
- public key cryptography > base58check encoding > id=pub-key-base58 1
- public key cryptography > base58check encoding > startref=pub-key-base58 1
- public key cryptography > bech32 addresses > advantages of > id=pub-key-bech32-adv 1
- public key cryptography > bech32 addresses > advantages of > startref=pub-key-bech32-adv 1
- public key cryptography > bech32 addresses > bech32m > id=pub-key-bech32-bech32m 1
- public key cryptography > bech32 addresses > bech32m > startref=pub-key-bech32-bech32m 1
- public key cryptography > bech32 addresses > problems with > id=pub-key-bech32-prob 1
- public key cryptography > bech32 addresses > problems with > startref=pub-key-bech32-prob 1
- public key cryptography > compressed public keys > id=pub-key-compress 1
- public key cryptography > compressed public keys > startref=pub-key-compress 1
- public key cryptography > elliptic curve cryptography as > id=pub-key-ecc 1
- public key cryptography > elliptic curve cryptography as > startref=pub-key-ecc 1
- public key cryptography > hash functions and > id=pub-key-hash 1
- public key cryptography > hash functions and > id=pub-key-hash2 1
- public key cryptography > hash functions and > startref=pub-key-hash 1
- public key cryptography > hash functions and > startref=pub-key-hash2 1
- public key cryptography > id=pub-key 1
- public key cryptography > input/output scripts > id=pub-key-input-output 1
- public key cryptography > input/output scripts > startref=pub-key-input-output 1
- public key cryptography > IP address payments and > id=pub-key-ipaddress 1
- public key cryptography > IP address payments and > startref=pub-key-ipaddress 1
- public key cryptography > key tweaks > id=pub-key-tweak 1
- public key cryptography > key tweaks > startref=pub-key-tweak 1
- public key cryptography > paper wallets > id=pub-key-paper 1
- public key cryptography > paper wallets > startref=pub-key-paper 1
- public key cryptography > purpose in Bitcoin 1
- public key cryptography > scriptless multisignatures > id=pub-key-scriptless 1
- public key cryptography > scriptless multisignatures > startref=pub-key-scriptless 1
- public key cryptography > vanity addresses > id=pub-key-vanity 1
- public key cryptography > vanity addresses > startref=pub-key-vanity 1
- public key cryptography > wallet recovery key generation > see=key generation 1
- public keys 1
- public keys > aggregated 1
- public keys > generating > id=public-key-generate 1
- public keys > generating > startref=public-key-generate 1
- public keys > purpose of 1
- pycoin 1
- Python toolkits 1
- python-bitcoinlib 1
- QR codes 1

- randomness, importance in digital signatures > id=random-digital-signature 1
- randomness, importance in digital signatures > startref=random-digital-signature 1
- RBF (replace by fee) fee bumping > id=rbf-ch9 1
- RBF (replace by fee) fee bumping > startref=rbf-ch9 1
- RBF (replace by fee) fee bumping > transaction pinning > id=rbf-pin 1
- RBF (replace by fee) fee bumping > transaction pinning > startref=rbf-pin 1
- receiving bitcoins 1
- receiving bitcoins > id=receive-bitcoin 1
- receiving bitcoins > startref=receive-bitcoin 1
- reclaiming > disk space 1
- reclaiming disk space 1
- recovery codes 1
- recovery codes > generating > id=recovery-code-bip39-generate 1
- recovery codes > generating > startref=recovery-code-bip39-generate 1
- recovery codes > id=recovery-code 1
- recovery codes > id=recovery-code-bip39 1
- recovery codes > id=recovery-code2 1
- recovery codes > passphrases > id=recovery-code-bip39-passphrase 1
- recovery codes > passphrases > id=recovery-code-passphrase 1
- recovery codes > passphrases > startref=recovery-code-bip39-passphrase 1
- recovery codes > passphrases > startref=recovery-code-passphrase 1
- recovery codes > seed generation > id=recovery-code-bip39-seed 1
- recovery codes > seed generation > startref=recovery-code-bip39-seed 1
- recovery codes > startref=recovery-code 1
- recovery codes > startref=recovery-code-bip39 1
- recovery codes > startref=recovery-code2 1
- recovery codes > types of > id=recovery-code-type 1
- recovery codes > types of > startref=recovery-code-type 1
- redeem scripts 1
- redeem scripts > id=redeem-script 1
- redeem scripts > startref=redeem-script 1
- redeem scripts > validating 1
- regtest > id=regtest 1
- regtest > startref=regtest 1
- relative timelocks > id=relative-timelock 1
- relative timelocks > id=relative-timelock2 1
- relative timelocks > startref=relative-timelock2 1
- release candidates 1
- release version (Bitcoin Core), selecting > id=release-select 1
- release version (Bitcoin Core), selecting > startref=release-select 1
- repeated session attacks 1
- replace by fee (RBF) 1
- revocable commitments > id=revoke-commit 1
- revocable commitments > startref=revoke-commit 1
- rewards 1
- rewards > id=reward-coinbase 1
- rewards > startref=reward-coinbase 1
- RGB protocol > id=rgb 1
- RGB protocol > startref=rgb 1
- RIPEMD-160 hash function 1
- risk diversification 1
- root of trust > id=root-trust 1
- root of trust > startref=root-trust 1
- root seeds 1
- routed payment channels > see=Lightning Network 1
- RPC commands > see=command-line interface (Bitcoin Core) 1
- running nodes > id=running-nodes 1
- running nodes > startref=running-nodes 1
- Rust toolkits 1
- rust-bitcoin 1
- salt 1
- satoshis 1 2
- Scala toolkits 1
- schnorr signature algorithm 1
- schnorr signature algorithm > examples of usage 1
- schnorr signature algorithm > id=schnorr 1
- schnorr signature algorithm > properties of 1
- schnorr signature algorithm > scriptless multisignatures > id=schnorr-multisig 1
- schnorr signature algorithm > scriptless multisignatures > startref=schnorr-multisig 1
- schnorr signature algorithm > scriptless threshold signatures > id=schnorr-threshold 1
- schnorr signature algorithm > scriptless threshold signatures > startref=schnorr-threshold 1

- schnorr signature algorithm > security features 1
- schnorr signature algorithm > serialization 1
- schnorr signature algorithm > startref=schnorr 1
- Script programming language 1
- scripted multisignatures > id=script-multisignature 1
- scripted multisignatures > startref=script-multisignature 1
- scriptless multisignatures > id=scriptless-multi 1
- scriptless multisignatures > in schnorr signature algorithm > secondary-sortas=schnorr > id=scriptless-multi-schnorr 1
- scriptless multisignatures > in schnorr signature algorithm > secondary-sortas=schnorr > startref=scriptless-multi-schnorr 1
- scriptless multisignatures > startref=scriptless-multi 1
- scriptless threshold signatures > id=scriptless-threshold-schnorr 1
- scriptless threshold signatures > startref=scriptless-threshold-schnorr 1
- scriptpath spending 1
- scripts > examples of > id=script-example-complex 1
- scripts > examples of > startref=script-example-complex 1
- scripts > flow control > id=script-flow 1
- scripts > flow control > startref=script-flow 1
- scripts > input/output > constructing > id=script-input-output-construct 1
- scripts > input/output > examples of > id=script-input-output-example 1
- scripts > input/output > examples of > startref=script-input-output-example 1
- scripts > input/output > id=script-input-output 1
- scripts > input/output > separate execution 1
- scripts > input/output > startref=script-input-output 1
- scripts > MAST > id=script-mast 1
- scripts > MAST > startref=script-mast 1
- scripts > MAST > taproot > id=script-mast-taproot 1
- scripts > MAST > taproot > startref=script-mast-taproot 1
- scripts > OP_RETURN > id=script-op-return 1
- scripts > OP_RETURN > startref=script-op-return 1
- scripts > P2PKH (pay to public key hash) 1
- scripts > P2SH (pay to script hash) > id=script-p2sh 1
- scripts > P2SH (pay to script hash) > startref=script-p2sh 1
- scripts > segregated witness > id=script-segwit 1
- scripts > segregated witness > P2WPKH > id=script-segwit-p2wpkh 1
- scripts > segregated witness > P2WPKH > startref=script-segwit-p2wpkh 1
- scripts > segregated witness > P2WSH > id=script-segwit-p2wsh 1
- scripts > segregated witness > P2WSH > startref=script-segwit-p2wsh 1
- scripts > segregated witness > startref=script-segwit 1
- scripts > segregated witness > upgrading to > id=script-segwit-upgrade 1
- scripts > segregated witness > upgrading to > startref=script-segwit-upgrade 1
- scripts > stack > id=script-stack 1
- scripts > stack > startref=script-stack 1
- scripts > stateless verification 1
- scripts > timelocks > conflicts 1
- scripts > timelocks > limitations of 1
- scripts > timelocks > relative > id=script-timelock-relative 1
- scripts > timelocks > relative > startref=script-timelock-relative 1
- scripts > timelocks > verifying > id=script-timelock-verify 1
- scripts > timelocks > verifying > startref=script-timelock-verify 1
- scripts > Turing incompleteness 1
- second preimage attacks 1
- second-party transaction malleability > id=second-party 1
- second-party transaction malleability > startref=second-party 1
- security > best practices > id=security-best-practice 1
- security > best practices > startref=security-best-practice 1
- security > principles of > id=security-principle 1
- security > principles of > startref=security-principle 1
- seed phrases > see=recovery codes 1
- seeds 1 2
- seeds > generating > id=seed-generate 1
- seeds > generating > startref=seed-generate 1
- seeds > HD wallet creation > id=seed-hdwallet 1

- seeds > HD wallet creation > startref=seed-hdwallet 1
- segregated witness (segwit) > digital signatures and 1
- segregated witness (segwit) > id=segregated-witness-segwit 1
- segregated witness (segwit) > id=segwit-bech32 1
- segregated witness (segwit) > id=segwit-bip8 1
- segregated witness (segwit) > scripts and > id=segwit-script 1
- segregated witness (segwit) > scripts and > P2WPKH > id=segwit-script-p2wpkh 1
- segregated witness (segwit) > scripts and > P2WPKH > startref=segwit-script-p2wpkh 1
- segregated witness (segwit) > scripts and > P2WSH > id=segwit-script-p2wsh 1
- segregated witness (segwit) > scripts and > P2WSH > startref=segwit-script-p2wsh 1
- segregated witness (segwit) > scripts and > startref=segwit-script 1
- segregated witness (segwit) > scripts and > upgrading to > id=segwit-script-upgrade 1
- segregated witness (segwit) > scripts and > upgrading to > startref=segwit-script-upgrade 1
- segregated witness (segwit) > startref=segregated-witness-segwit 1
- segregated witness (segwit) > startref=segwit-bech32 1
- segregated witness (segwit) > startref=segwit-bip8 1
- selecting > coins in transactions 1
- selecting > release version (Bitcoin Core) > id=select-release 1
- selecting > release version (Bitcoin Core) > startref=select-release 1
- selecting > wallets 1
- sequence field (transaction inputs) > id=sequence-field 1
- sequence field (transaction inputs) > startref=sequence-field 1
- sequence-based transaction replacement > id=sequence-replace 1
- sequence-based transaction replacement > startref=sequence-replace 1
- serialization > ECDSA signatures 1
- serialization > of schnorr signature algorithm > secondary-sortas=schnorr 1
- serialized transactions > id=serial-transactions 1
- serialized transactions > startref=serial-transactions 1
- settlement transactions 1
- setup transactions 1
- SHA256 hash function 1 2
- share chains 1
- SIGHASH flags > id=sighash 1
- SIGHASH flags > startref=sighash 1
- signaling > BIP34 > secondary-sortas=BIP034 > id=signal-bip34 1
- signaling > BIP34 > secondary-sortas=BIP034 > startref=signal-bip34 1
- signaling > BIP9 > secondary-sortas=BIP009 > id=signal-bip9 1
- signaling > BIP9 > secondary-sortas=BIP009 > startref=signal-bip9 1
- signatures > see=digital signatures 1
- signet > id=signet 1
- signet > startref=signet 1
- simplified-payment-verification (SPV) clients 1
- single-use seals 1
- SLIP39 recovery codes 1
- soft forks 1
- soft forks > BIP34 signaling/activation > secondary-sortas=BIP034 > id=soft-fork-bip34 1
- soft forks > BIP34 signaling/activation > secondary-sortas=BIP034 > startref=soft-fork-bip34 1
- soft forks > BIP8 mandatory lock-in > secondary-sortas=BIP008 > id=soft-fork-bip8 1
- soft forks > BIP8 mandatory lock-in > secondary-sortas=BIP008 > startref=soft-fork-bip8 1
- soft forks > BIP9 signaling/activation > secondary-sortas=BIP009 > id=soft-fork-bip9 1
- soft forks > BIP9 signaling/activation > secondary-sortas=BIP009 > startref=soft-fork-bip9 1
- soft forks > criticisms of > id=soft-fork-critic 1
- soft forks > explained > id=soft-fork-explain 1
- soft forks > explained > startref=soft-fork-explain 1
- soft forks > speedy trial activation > id=soft-fork-speed 1
- software development for consensus rules 1
- software forks > id=software-fork 1
- software forks > startref=software-fork 1
- source code, compiling Bitcoin Core > building executables 1
- source code, compiling Bitcoin Core > configuring build > id=source-code-compile-configure 1
- source code, compiling Bitcoin Core > configuring build > startref=source-code-compile-configure 1

- source code, compiling Bitcoin Core > id=source-code-compile 1
- source code, compiling Bitcoin Core > selecting release version > id=source-code-compile-release 1
- source code, compiling Bitcoin Core > selecting release version > startref=source-code-compile-release 1
- source code, compiling Bitcoin Core > startref=source-code-compile 1
- speedy trial activation > id=speed-trial 1
- spending bitcoins > id=send-bitcoin 1
- spending bitcoins > id=spend-bitcoin 1
- spending bitcoins > id=spend-bitcoin2 1
- spending bitcoins > startref=send-bitcoin 1
- spending bitcoins > startref=spend-bitcoin 1
- SPV (simplified-payment-verification) clients 1 2
- SPV (simplified-payment-verification) clients > id=spv-lightweight 1
- SPV (simplified-payment-verification) clients > startref=spv-lightweight 1
- stack > id=stack 1
- stack > startref=stack 1
- standard transaction outputs 1
- state channels > id=state-channel-terminology 1
- state channels > startref=state-channel-terminology 1
- stateless script verification 1
- status information (Bitcoin Core) > id=status-bitcoin-core 1
- status information (Bitcoin Core) > startref=status-bitcoin-core 1
- storing bitcoins > id=storing-bitcoin 1
- storing bitcoins > startref=storing-bitcoin 1
- Stratum 1
- survivability (of bitcoin access) 1
- syncing blockchain > id=sync-blockchain 1
- syncing blockchain > startref=sync-blockchain 1
- taproot > id=taproot-ch7 1
- taproot > startref=taproot-ch7 1
- Taproot Assets > id=taproot-assets 1
- Taproot Assets > startref=taproot-assets 1
- tapscript > id=tapscript 1
- tapscript > startref=tapscript 1
- targets > adjusting difficulty > id=target-difficulty 1
- targets > adjusting difficulty > startref=target-difficulty 1
- targets > representation of > id=target-represent 1
- targets > representation of > startref=target-represent 1
- technical debt 1
- test blockchains > development usage 1
- test blockchains > regtest > id=test-block-regtest 1
- test blockchains > regtest > startref=test-block-regtest 1
- test blockchains > signet > id=test-block-signet 1
- test blockchains > signet > startref=test-block-signet 1
- test blockchains > testnet > id=test-block-testnet 1
- test blockchains > testnet > startref=test-block-testnet 1
- testnet > id=testnet 1
- testnet > startref=testnet 1
- third-party API clients 1
- third-party transaction malleability > id=third-party 1
- third-party transaction malleability > startref=third-party 1
- threshold signatures > id=threshold-signature 1
- threshold signatures > in schnorr signature algorithm > secondary-sortas=schnorr > id=threshold-schnorr 1
- threshold signatures > in schnorr signature algorithm > secondary-sortas=schnorr > startref=threshold-schnorr 1
- threshold signatures > startref=threshold-signature 1
- timelocks > conflicts 1
- timelocks > fee sniping and > id=timelock-fee-snipe 1
- timelocks > limitations of 1
- timelocks > relative > id=timelock-relative 1
- timelocks > relative > startref=timelock-relative 1
- timelocks > trustless channels > id=timelock-trustless 1
- timelocks > trustless channels > startref=timelock-trustless 1
- timelocks > verifying > id=timelock-op-cltv 1
- timelocks > verifying > startref=timelock-op-cltv 1
- timestamp servers 1
- timestamps > id=timestamp 1
- Tor transport 1
- transaction chains > id=transaction-chains 1
- transaction fees 1 2 3
- transaction fees > change outputs and 1
- transaction fees > fee bumping > CPFP (child pays for parent) > id=transaction-fees-bump-cpfp 1

- transaction fees > fee bumping > CPFP (child pays for parent) > startref=transaction-fees-bump-cpfp 1
- transaction fees > fee bumping > CPFP carve outs > id=transaction-fee-bump-carveout 1
- transaction fees > fee bumping > CPFP carve outs > startref=transaction-fee-bump-carveout 1
- transaction fees > fee bumping > RBF (replace by fee) > id=transaction-fees-bump-rbf 1
- transaction fees > fee bumping > RBF (replace by fee) > startref=transaction-fees-bump-rbf 1
- transaction fees > fee bumping > transaction pinning > id=transaction-fee-bump-pin 1
- transaction fees > fee bumping > transaction pinning > startref=transaction-fee-bump-pin 1
- transaction fees > fee rates > id=fees-rates 1
- transaction fees > fee rates > startref=fees-rates 1
- transaction fees > fee sniping > id=transaction-fee-sniping 1
- transaction fees > in coinbase transactions > secondary-sortas=coinbase transactions > id=transaction-fee-coinbase 1
- transaction fees > in coinbase transactions > secondary-sortas=coinbase transactions > startref=transaction-fee-coinbase 1
- transaction fees > opt-in transaction replacement 1
- transaction fees > overpaying 1
- transaction fees > package relay > id=transaction-fee-package-relay 1
- transaction fees > responsibility for > id=fees-responsibility 1
- transaction fees > responsibility for > startref=fees-responsibility 1
- transaction IDs (txid) 1
- transaction pinning > id=transaction-pin 1
- transaction pinning > startref=transaction-pin 1
- transaction scripts > see=scripts 1
- transactions > adding to blockchain > id=transaction-add-blockchain 1
- transactions > adding to blockchain > startref=transaction-add-blockchain 1
- transactions > building complete index 1
- transactions > change output > id=transaction-change-output 1
- transactions > changeless 1
- transactions > clearing 1 2
- transactions > coin selection 1
- transactions > coinbase > id=transaction-coinbase 1
- transactions > coinbase > id=transaction-coinbase2 1
- transactions > coinbase > startref=transaction-coinbase 1
- transactions > coinbase > startref=transaction-coinbase2 1
- transactions > common types > id=transaction-common-ch2 1
- transactions > conflicts in 1 2
- transactions > constructing > id=transaction-construct 1
- transactions > constructing > startref=transaction-construct 1
- transactions > defined 1
- transactions > errata in Bitcoin whitepaper > id=transaction-errata 1
- transactions > errata in Bitcoin whitepaper > startref=transaction-errata 1
- transactions > exploring/decoding > id=transactions-explore-decode 1
- transactions > exploring/decoding > id=transactions-explore-decode2 1
- transactions > exploring/decoding > startref=transactions-explore-decode 1
- transactions > exploring/decoding > startref=transactions-explore-decode2 1
- transactions > extended serialization format 1
- transactions > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper > id=transaction-whitepaper 1
- transactions > in Bitcoin whitepaper > secondary-sortas=Bitcoin whitepaper > startref=transaction-whitepaper 1
- transactions > in blocks > secondary-sortas=blocks 1 2
- transactions > independent verification > id=transaction-verify 1
- transactions > independent verification > startref=transaction-verify 1
- transactions > inputs > id=transaction-input 1
- transactions > inputs > id=transaction-input-ch2 1
- transactions > inputs > in Bitcoin whitepaper > tertiary-sortas=Bitcoin whitepaper 1
- transactions > inputs > input script 1
- transactions > inputs > length of list > id=transaction-input-length 1
- transactions > inputs > length of list > startref=transaction-input-length 1

- transactions > inputs > outpoint field > id=transaction-input-outpoint 1
- transactions > inputs > outpoint field > startref=transaction-input-outpoint 1
- transactions > inputs > sequence field > id=transaction-input-sequence 1
- transactions > inputs > sequence field > startref=transaction-input-sequence 1
- transactions > inputs > startref=transaction-input 1
- transactions > legacy serialization 1 2
- transactions > lock time 1
- transactions > outputs > amount field > id=transaction-output-amount 1
- transactions > outputs > amount field > startref=transaction-output-amount 1
- transactions > outputs > count 1
- transactions > outputs > id=transaction-output 1
- transactions > outputs > id=transaction-output-ch2 1
- transactions > outputs > in Bitcoin whitepaper > tertiary-sortas=Bitcoin whitepaper 1
- transactions > outputs > output scripts > id=transaction-output-script 1
- transactions > outputs > output scripts > startref=transaction-output-script 1
- transactions > outputs > startref=transaction-output 1
- transactions > presigned > id=transaction-presign 1
- transactions > serialized > id=transaction-serialize 1
- transactions > serialized > startref=transaction-serialize 1
- transactions > signatures > see=digital signatures 1
- transactions > spending bitcoins > id=transaction-spend 1
- transactions > spending bitcoins > id=transaction-spend2 1
- transactions > spending bitcoins > startref=transaction-spend 1
- transactions > state channels > id=transaction-state 1
- transactions > state channels > startref=transaction-state 1
- transactions > timelocks > conflicts 1
- transactions > timelocks > limitations of 1
- transactions > timelocks > relative > id=transaction-timelock-relative 1
- transactions > timelocks > relative > startref=transaction-timelock-relative 1
- transactions > timelocks > verifying > id=transaction-timelock-op-cltv 1
- transactions > timelocks > verifying > startref=transaction-timelock-op-cltv 1
- transactions > unconfirmed 1
- transactions > validating 1 2
- transactions > version of > id=transactions-version 1
- transactions > version of > startref=transactions-version 1
- transactions > weights > id=transactions-weight 1
- transactions > weights > startref=transactions-weight 1
- transactions > witnesses > circular dependencies > id=transaction-witness-circular 1
- transactions > witnesses > count > id=transaction-witness-count 1
- transactions > witnesses > count > startref=transaction-witness-count 1
- transactions > witnesses > id=transaction-witness 1
- transactions > witnesses > second-party transaction malleability > id=transaction-witness-second-party 1
- transactions > witnesses > second-party transaction malleability > startref=transaction-witness-second-party 1
- transactions > witnesses > segregated witness > id=transaction-witness-segwit 1
- transactions > witnesses > segregated witness > startref=transaction-witness-segwit 1
- transactions > witnesses > startref=transaction-witness 1
- transactions > witnesses > third-party transaction malleability > id=transaction-witness-third-party 1
- transactions > witnesses > third-party transaction malleability > startref=transaction-witness-third-party 1
- translated forwarding 1
- tree structure in HD wallets > id=tree-hd-wallet 1
- tree structure in HD wallets > startref=tree-hd-wallet 1
- trustless channels > id=trustless-channel 1
- trustless channels > startref=trustless-channel 1
- trustless protocols 1
- Turing Complete 1
- txindex option (bitcoind option) 1 2
- uncompressed public keys > id=uncompress-pub-key 1
- uncompressed public keys > startref=uncompress-pub-key 1

- unconfirmed transactions 1
- uneconomical outputs > id=uneconomical 1
- uneconomical outputs > startref=uneconomical 1
- unsigned integers > id=unsigned 1
- upgrading to segregated witness > id=upgrade-segwit 1
- upgrading to segregated witness > startref=upgrade-segwit 1
- Utreexo 1
- UTXOs (unspent transaction outputs) 1 2 3
- validating > blocks > id=validate-block 1
- validating > blocks > startref=validate-block 1
- validating > redeem scripts 1
- validating > transactions 1 2
- validating > with client-side validation > secondary-sortas=client-side validation 1
- vanity addresses > id=vanity-addr 1
- vanity addresses > startref=vanity-addr 1
- vanity pools 1
- vbytes > id=vbytes 1
- vbytes > startref=vbytes 1
- VERIFY opcodes 1
- verifying > digital signatures 1
- verifying > lock time > id=verify-lock-time 1
- verifying > lock time > startref=verify-lock-time 1
- verifying > payment > errata in Bitcoin whitepaper 1
- verifying > payment > in Bitcoin whitepaper > tertiary-sortas=Bitcoin whitepaper > id=verify-payment 1
- verifying > payment > in Bitcoin whitepaper > tertiary-sortas=Bitcoin whitepaper > startref=verify-payment 1
- verifying > scripts 1
- verifying > transactions > id=verify-transaction 1
- verifying > transactions > startref=verify-transaction 1
- version (of transactions) > id=version-transactions 1
- version (of transactions) > startref=version-transactions 1
- version prefixes > id=version-prefix 1
- version prefixes > startref=version-prefix 1
- wallets > choosing 1
- wallets > explained 1
- wallets > key control 1
- wallets > key generation > backing up derivation paths > id=wallet-keygen-backups 1
- wallets > key generation > backing up derivation paths > startref=wallet-keygen-backups 1
- wallets > key generation > deterministic > id=wallet-keygen-determine 1
- wallets > key generation > deterministic > startref=wallet-keygen-determine 1
- wallets > key generation > HD (hierarchical deterministic) 1
- wallets > key generation > HD (hierarchical deterministic) > id=wallet-keygen-hd 1
- wallets > key generation > HD (hierarchical deterministic) > startref=wallet-keygen-hd 1
- wallets > key generation > independent > id=wallet-keygen-independent 1
- wallets > key generation > independent > startref=wallet-keygen-independent 1
- wallets > key generation > public child key derivation > id=wallet-keygen-public-child 1
- wallets > key generation > public child key derivation > startref=wallet-keygen-public-child 1
- wallets > noncustodial 1
- wallets > nonkey data, backing up > id=wallet-nonkey-backups 1
- wallets > nonkey data, backing up > startref=wallet-nonkey-backups 1
- wallets > P2WPKH (pay to witness public key hash) 1
- wallets > paper > id=wallet-paper 1
- wallets > paper > startref=wallet-paper 1
- wallets > private key formats 1
- wallets > recovery codes 1
- wallets > recovery codes > generating > id=wallet-recovery-bip39-generate 1
- wallets > recovery codes > generating > startref=wallet-recovery-bip39-generate 1
- wallets > recovery codes > id=wallet-recovery 1
- wallets > recovery codes > id=wallet-recovery-bip39 1
- wallets > recovery codes > id=wallet-recovery2 1
- wallets > recovery codes > passphrases > id=wallet-recovery-bip39-passphrase 1
- wallets > recovery codes > passphrases > id=wallet-recovery-passphrase 1
- wallets > recovery codes > passphrases > startref=wallet-recovery-bip39-passphrase 1
- wallets > recovery codes > passphrases > startref=wallet-recovery-passphrase 1
- wallets > recovery codes > seed generation > id=wallet-recovery-bip39-seed 1

- wallets > recovery codes > seed generation > startref=wallet-recovery-bip39-seed 1
- wallets > recovery codes > startref=wallet-recovery 1
- wallets > recovery codes > startref=wallet-recovery-bip39 1
- wallets > recovery codes > startref=wallet-recovery2 1
- wallets > recovery codes > types of > id=wallet-recovery-type 1
- wallets > recovery codes > types of > startref=wallet-recovery-type 1
- wallets > types of > id=wallet-type 1
- wallets > types of > startref=wallet-type 1
- web store example (extended keys) > id=webstore-extend-key 1
- web store example (extended keys) > startref=webstore-extend-key 1
- web wallets 1
- weights (of transactions) > id=weights 1
- weights (of transactions) > startref=weights 1
- whitepaper (Bitcoin) > errata > id=whitepaper-errata 1
- whitepaper (Bitcoin) > errata > startref=whitepaper-errata 1
- whitepaper (Bitcoin) > original version > id=whitepaper-original 1
- whitepaper (Bitcoin) > original version > startref=whitepaper-original 1
- witness items 1
- witnesses > circular dependencies > id=witness-circular 1
- witnesses > count > id=witness-count 1
- witnesses > count > startref=witness-count 1
- witnesses > id=witness 1
- witnesses > second-party transaction malleability > id=witness-circular-second-party 1
- witnesses > second-party transaction malleability > startref=witness-circular-second-party 1
- witnesses > segregated witness > id=witness-segwit 1
- witnesses > segregated witness > startref=witness-segwit 1
- witnesses > startref=witness 1
- witnesses > third-party transaction malleability > id=witness-circular-third-party 1
- witnesses > third-party transaction malleability > startref=witness-circular-third-party 1
- wrapper libraries 1
- zero-knowledge proof 1

Font redistribution notices

```
Fonts are (c) Bitstream (see below). DejaVu changes are in public domain.
Glyphs imported from Arev fonts are (c) Tavmjong Bah (see below)


Bitstream Vera Fonts Copyright
------------------------------

Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera is
a trademark of Bitstream, Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy
of the fonts accompanying this license ("Fonts") and associated
documentation files (the "Font Software"), to reproduce and distribute the
Font Software, including without limitation the rights to use, copy, merge,
publish, distribute, and/or sell copies of the Font Software, and to permit
persons to whom the Font Software is furnished to do so, subject to the
following conditions:

The above copyright and trademark notices and this permission notice shall
be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular
the designs of glyphs or characters in the Fonts may be modified and
additional glyphs or characters may be added to the Fonts, only if the fonts
are renamed to names not containing either the words "Bitstream" or the word
"Vera".

This License becomes null and void to the extent applicable to Fonts or Font
Software that has been modified and is distributed under the "Bitstream
Vera" names.

The Font Software may be sold as part of a larger software package but no
copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT,
TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME
FOUNDATION BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING
ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES,
WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF
THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE
FONT SOFTWARE.

Except as contained in this notice, the names of Gnome, the Gnome
Foundation, and Bitstream Inc., shall not be used in advertising or
otherwise to promote the sale, use or other dealings in this Font Software
without prior written authorization from the Gnome Foundation or Bitstream
Inc., respectively. For further information, contact: fonts at gnome dot
org.

Arev Fonts Copyright
------------------------------

Copyright (c) 2006 by Tavmjong Bah. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining
a copy of the fonts accompanying this license ("Fonts") and
associated documentation files (the "Font Software"), to reproduce
and distribute the modifications to the Bitstream Vera Font Software,
including without limitation the rights to use, copy, merge, publish,
```

distribute, and/or sell copies of the Font Software, and to permit
persons to whom the Font Software is furnished to do so, subject to
the following conditions:

The above copyright and trademark notices and this permission notice
shall be included in all copies of one or more of the Font Software
typefaces.

The Font Software may be modified, altered, or added to, and in
particular the designs of glyphs or characters in the Fonts may be
modified and additional glyphs or characters may be added to the
Fonts, only if the fonts are renamed to names not containing either
the words "Tavmjong Bah" or the word "Arev".

This License becomes null and void to the extent applicable to Fonts
or Font Software that has been modified and is distributed under the
"Tavmjong Bah Arev" names.

The Font Software may be sold as part of a larger software package but
no copy of one or more of the Font Software typefaces may be sold by
itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT
OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL
TAVMJONG BAH BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL
DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM
OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the name of Tavmjong Bah shall not
be used in advertising or otherwise to promote the sale, use or other
dealings in this Font Software without prior written authorization
from Tavmjong Bah. For further information, contact: tavmjong @ free
. fr.

TeX Gyre DJV Math

Fonts are (c) Bitstream (see below). DejaVu changes are in public domain.

Math extensions done by B. Jackowski, P. Strzelczyk and P. Pianowski
(on behalf of TeX users groups) are in public domain.

Letters imported from Euler Fraktur from AMSfonts are (c) American
Mathematical Society (see below).
Bitstream Vera Fonts Copyright
Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera
is a trademark of Bitstream, Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy
of the fonts accompanying this license (“Fonts”) and associated
documentation
files (the “Font Software”), to reproduce and distribute the Font Software,
including without limitation the rights to use, copy, merge, publish,
distribute,
and/or sell copies of the Font Software, and to permit persons to whom
the Font Software is furnished to do so, subject to the following
conditions:

The above copyright and trademark notices and this permission notice
shall be
included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular
the designs of glyphs or characters in the Fonts may be modified and
additional
glyphs or characters may be added to the Fonts, only if the fonts are

renamed
to names not containing either the words “Bitstream” or the word “Vera”.

This License becomes null and void to the extent applicable to Fonts or
Font Software
that has been modified and is distributed under the “Bitstream Vera”
names.

The Font Software may be sold as part of a larger software package but
no copy
of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT,
TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME
FOUNDATION
BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL,
SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN
ACTION
OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR
INABILITY TO USE
THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.
Except as contained in this notice, the names of GNOME, the GNOME
Foundation,
and Bitstream Inc., shall not be used in advertising or otherwise to promote
the sale, use or other dealings in this Font Software without prior written
authorization from the GNOME Foundation or Bitstream Inc., respectively.
For further information, contact: fonts at gnome dot org.

AMSFonts (v. 2.2) copyright

The PostScript Type 1 implementation of the AMSFonts produced by and
previously distributed by Blue Sky Research and Y&Y, Inc. are now freely
available for general use. This has been accomplished through the
cooperation
of a consortium of scientific publishers with Blue Sky Research and Y&Y.
Members of this consortium include:

Elsevier Science IBM Corporation Society for Industrial and Applied
Mathematics (SIAM) Springer-Verlag American Mathematical Society (AMS)

In order to assure the authenticity of these fonts, copyright will be
held by
the American Mathematical Society. This is not meant to restrict in any way
the legitimate use of the fonts, such as (but not limited to) electronic
distribution of documents containing these fonts, inclusion of these fonts
into other public domain or commercial font collections or computer
applications, use of the outline data to create derivative fonts and/or
faces, etc. However, the AMS does require that the AMS copyright notice be
removed from any derivative versions of the fonts which have been altered in
any way. In addition, to ensure the fidelity of TeX documents using Computer
Modern fonts, Professor Donald Knuth, creator of the Computer Modern faces,
has requested that any alterations which yield different font metrics be
given a different name.

Id

MathJax 3.2.2 equation glyphs: Copyright 2009–2022 The MathJax Consortium.

                                 Apache License
                           Version 2.0, January 2004
                        http://www.apache.org/licenses/

   TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

   1. Definitions.

"License" shall mean the terms and conditions for use, reproduction,
and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by
the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all
other entities that control, are controlled by, or are under common
control with that entity. For the purposes of this definition,
"control" means (i) the power, direct or indirect, to cause the
direction or management of such entity, whether by contract or
otherwise, or (ii) ownership of fifty percent (50%) or more of the
outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity
exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications,
including but not limited to software source code, documentation
source, and configuration files.

"Object" form shall mean any form resulting from mechanical
transformation or translation of a Source form, including but
not limited to compiled object code, generated documentation,
and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or
Object form, made available under the License, as indicated by a
copyright notice that is included in or attached to the work
(an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object
form, that is based on (or derived from) the Work and for which the
editorial revisions, annotations, elaborations, or other modifications
represent, as a whole, an original work of authorship. For the purposes
of this License, Derivative Works shall not include works that remain
separable from, or merely link (or bind by name) to the interfaces of,
the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including
the original version of the Work and any modifications or additions
to that Work or Derivative Works thereof, that is intentionally
submitted to Licensor for inclusion in the Work by the copyright owner
or by an individual or Legal Entity authorized to submit on behalf of
the copyright owner. For the purposes of this definition, "submitted"
means any form of electronic, verbal, or written communication sent
to the Licensor or its representatives, including but not limited to
communication on electronic mailing lists, source code control systems,
and issue tracking systems that are managed by, or on behalf of, the
Licensor for the purpose of discussing and improving the Work, but
excluding communication that is conspicuously marked or otherwise
designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity
on behalf of whom a Contribution has been received by Licensor and
subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of
   this License, each Contributor hereby grants to You a perpetual,
   worldwide, non-exclusive, no-charge, royalty-free, irrevocable
   copyright license to reproduce, prepare Derivative Works of,
   publicly display, publicly perform, sublicense, and distribute the
   Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of
   this License, each Contributor hereby grants to You a perpetual,
   worldwide, non-exclusive, no-charge, royalty-free, irrevocable
   (except as stated in this section) patent license to make, have made,
   use, offer to sell, sell, import, and otherwise transfer the Work,

where such license applies only to those patent claims licensable
by such Contributor that are necessarily infringed by their
Contribution(s) alone or by combination of their Contribution(s)
with the Work to which such Contribution(s) was submitted. If You
institute patent litigation against any entity (including a
cross-claim or counterclaim in a lawsuit) alleging that the Work
or a Contribution incorporated within the Work constitutes direct
or contributory patent infringement, then any patent licenses
granted to You under this License for that Work shall terminate
as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the
Work or Derivative Works thereof in any medium, with or without
modifications, and in Source or Object form, provided that You
meet the following conditions:

(a) You must give any other recipients of the Work or
Derivative Works a copy of this License; and

(b) You must cause any modified files to carry prominent notices
stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works
that You distribute, all copyright, patent, trademark, and
attribution notices from the Source form of the Work,
excluding those notices that do not pertain to any part of
the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its
distribution, then any Derivative Works that You distribute must
include a readable copy of the attribution notices contained
within such NOTICE file, excluding those notices that do not
pertain to any part of the Derivative Works, in at least one
of the following places: within a NOTICE text file distributed
as part of the Derivative Works; within the Source form or
documentation, if provided along with the Derivative Works; or,
within a display generated by the Derivative Works, if and
wherever such third-party notices normally appear. The contents
of the NOTICE file are for informational purposes only and
do not modify the License. You may add Your own attribution
notices within Derivative Works that You distribute, alongside
or as an addendum to the NOTICE text from the Work, provided
that such additional attribution notices cannot be construed
as modifying the License.

You may add Your own copyright statement to Your modifications and
may provide additional or different license terms and conditions
for use, reproduction, or distribution of Your modifications, or
for any such Derivative Works as a whole, provided Your use,
reproduction, and distribution of the Work otherwise complies with
the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise,
any Contribution intentionally submitted for inclusion in the Work
by You to the Licensor shall be under the terms and conditions of
this License, without any additional terms or conditions.
Notwithstanding the above, nothing herein shall supersede or modify
the terms of any separate license agreement you may have executed
with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade
names, trademarks, service marks, or product names of the Licensor,
except as required for reasonable and customary use in describing the
origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or
agreed to in writing, Licensor provides the Work (and each
Contributor provides its Contributions) on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or

```
   implied, including, without limitation, any warranties or conditions
   of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A
   PARTICULAR PURPOSE. You are solely responsible for determining the
   appropriateness of using or redistributing the Work and assume any
   risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory,
   whether in tort (including negligence), contract, or otherwise,
   unless required by applicable law (such as deliberate and grossly
   negligent acts) or agreed to in writing, shall any Contributor be
   liable to You for damages, including any direct, indirect, special,
   incidental, or consequential damages of any character arising as a
   result of this License or out of the use or inability to use the
   Work (including but not limited to damages for loss of goodwill,
   work stoppage, computer failure or malfunction, or any and all
   other commercial damages or losses), even if such Contributor
   has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing
   the Work or Derivative Works thereof, You may choose to offer,
   and charge a fee for, acceptance of support, warranty, indemnity,
   or other liability obligations and/or rights consistent with this
   License. However, in accepting such obligations, You may act only
   on Your own behalf and on Your sole responsibility, not on behalf
   of any other Contributor, and only if You agree to indemnify,
   defend, and hold each Contributor harmless for any liability
   incurred by, or claims asserted against, such Contributor by reason
   of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

   To apply the Apache License to your work, attach the following
   boilerplate notice, with the fields enclosed by brackets "[]"
   replaced with your own identifying information. (Don't include
   the brackets!)  The text should be enclosed in the appropriate
   comment syntax for the file format. We also recommend that a
   file or class name and description of purpose be included on the
   same "printed page" as the copyright notice for easier
   identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
```

About the Authors

Andreas M. Antonopoulos is a noted technologist and serial entrepreneur who has become one of the most well-known and well-respected figures in Bitcoin. As an engaging public speaker, teacher, and writer, Andreas makes complex subjects accessible and easy to understand. As an advisor, he helps startups recognize, evaluate, and navigate security and business risks.

Andreas grew up with the internet, starting his first company, an early BBS and proto-ISP, as a teenager in his home in Greece. He earned degrees in computer science, data communications, and distributed systems from University College London (UCL)—recently ranked among the world's top 10 universities. After moving to the United States, Andreas cofounded and managed a successful technology research company, and in that role advised dozens of Fortune 500 company executives on networking, security, data centers, and cloud computing. More than two hundred of his articles on security, cloud computing, and data centers have been published in print and syndicated worldwide. He holds two patents in networking and security.

In 1990, Andreas started teaching various IT topics in private, professional, and academic environments. He honed his speaking skills in front of audiences ranging in size from five executives in a boardroom to thousands of people in large conferences. With more than four hundred speaking engagements under his belt, he is considered a world-class and charismatic public speaker and teacher. In 2014, he was appointed as a teaching fellow with the University of Nicosia, the first university in the world to offer a master's degree in digital currency. In this role, he helped develop the curriculum and cotaught the "Introduction to Digital Currencies" course, offered as a massive open online course (MOOC) through the university.

As a Bitcoin entrepreneur, Andreas has founded a number of Bitcoin businesses and launched several community open source projects. He serves as an advisor to several Bitcoin and cryptocurrency companies. He is a widely published author of articles and blog posts on Bitcoin, a permanent host on the popular *Let's Talk Bitcoin* podcast, and a frequent speaker at technology and security conferences worldwide.

David A. Harding is a technical writer focused on creating documentation for open source software. He is the coauthor of the *Bitcoin Optech* weekly newsletter (2018–2023), *21.co Bitcoin Computer* tutorials (2015–2017), and Bitcoin.org developer documentation (2014–2015). He is also a Brink.dev grant committee member (2022–2023) and former board member (2020–2022).

Colophon

The animal on the cover of *Mastering Bitcoin* is a leafcutter ant (*Atta colombica*). The leafcutter ant (a nongeneric name) is a tropical, fungus-growing ant endemic to South and Central America, Mexico, and southern United States. Aside from humans, leafcutter ants form the largest and most complex animal societies on the planet. They are named for the way they chew leaves, which serve as nutrition for their fungal garden.

Winged ants, both male and female, take part in a mass exit of their nest known as the *revoada*, or a nuptial flight. Females mate with multiple males to collect the 300 million sperm necessary to set up a colony. Females also store bits of the parental fungus garden mycelium in the infrabuccal pocket located in their oral cavity; they will use this to start their own fungal gardens. Once grounded, the female loses her wings and sets up an underground lair for her colony. The success rate for new queens is low: 2.5% establish a long-lived colony.

Once a colony has matured, ants are divided into castes based on size, with each caste performing various functions. There are usually four castes: minims, the smallest workers that tend to the young and fungus gardens; minors, slightly larger than minima, are the first line of defense for the colony and patrol the surrounding terrain and attack enemies; mediae, the general foragers that cut leaves and bring back leaf fragments to the nest; and majors, the largest worker ants that act as soldiers, defending the nest from intruders. Recent research has shown that majors also clear main foraging trails and carry bulky items back to the nest.

Many of the animals on O'Reilly covers are endangered; all of them are important to the world. To learn more about how you can help, go to *animals.oreilly.com*.

The cover illustration is by Karen Montgomery, based on an image from *Insects Abroad*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

---

1. "Bitcoin: A Peer-to-Peer Electronic Cash System", Satoshi Nakamoto.